

O'REILLY®

Second
Edition

Learning Go

An Idiomatic Approach to
Real-World Go Programming



Jon Bodner

Praise for *Learning Go*, Second Edition

对《学习围棋（第二版）》的赞誉

“The first edition of Learning Go was an excellent starting point for any developer interested in learning Go, and the second edition is even better. This book is thorough without being monotonous, which is perfect for introducing newcomers to the Go ecosystem.”

—Jonathan Hall, Go Developer and Content Creator

“《*Learning Go*》第一版对于任何对学习 Go 感兴趣的开发者来说都是一个绝佳的起点，而第二版则更胜一筹。本书内容详尽而不枯燥，非常适合引导新手进入 Go 生态系统。”——Jonathan Hall, Go 开发者和内容创作者

"Learning Go does more than teach Go, it teaches good idiomatic Go. It is the perfect book for programmers familiar with other languages who want to learn Go."

—Chris Hines, Senior Principal Software Engineer,
Comcast

“《学习 Go》这本书不仅教授 Go 语言，更教授地道的 Go 语言表达方式。对于熟悉其他语言并想学习 Go 的程序员来说，它是一本完美的教材。”——克里斯·海因斯，康卡斯特高级首席软件工程师

Praise for *Learning Go*, First Edition

《学习围棋》（第一版）好评如潮

“Go is unique and even experienced programmers have to unlearn a few things and think differently about software. Learning Go does a good job of working through the big features of the language while pointing out idiomatic code, pitfalls, and design patterns along the way.”

—Aaron Schlesinger, Senior Engineer, Microsoft

“Go 语言独树一帜，即使是经验丰富的程序员也需要摒弃一些旧观念，用不同的方式思考软件开发。Go 语言学习指南能够很好地引导读者掌握这门语言的主要特性，同时还会指出惯用的代码规范、常见的陷阱和设计模式。”——微软高级工程师 Aaron Schlesinger

“Jon has been a critical voice in the Go community for many years and we have been strongly benefitted from his talks and articles. With Learning Go, Jon has written the programmers’ guide to learning Go. It strikes just the right balance of giving a good overview of what you need to know without rehashing well understood concepts from other languages.”

—Steve Francia, Go language product lead, Google,
and author of Hugo, Cobra, and Viper

“多年来，Jon一直是Go社区的重要人物，他的演讲和文章让我们受益匪浅。Jon撰写的《Learning Go》堪称程序员学习Go的指南。它恰到好处地平衡了对你需要了解的内容的全面概述，避免了重复其他语言中早已为人熟知的概念。”——Steve Francia，谷歌Go语言产品负责人，《Hugo》、《Cobra》和《Viper》的作者

“Bodner gets Go. In clear, lively prose, he teaches the language from its basics to advanced topics like reflection and C interop. He demonstrates through numerous examples how to write idiomatic Go, with its emphasis on clarity and simplicity. He also takes the time to explain the underlying concepts that can affect your program’s behavior, like the effects of pointers on memory layout and garbage collection. Beginners who read this book will come up to speed quickly, and even experienced Go programmers are likely to learn something.”

—Jonathan Amsterdam, Software Engineer on the Go
team at Google

“Bodner 对 Go 语言的理解非常透彻。他以清晰生动的语言，从基础讲解到反射和 C 语言互操作等高级主题，全面深入地介绍了这门语言。他通过大量示例演示了如何编写地道的 Go 代码，强调清晰简洁。他还花时间解释了可能影响程序行为的底层概念，例如指针对内存布局和垃圾回收的影响。初学者阅读本书后可以快速入门，即使是经验丰富的 Go 程序员也能从中获益匪浅。”——谷歌 Go 团队软件工程师 Jonathan Amsterdam

“Learning Go is the essential introduction to what makes the Go programming language unique as well as the design patterns and idioms that make it so powerful. Jon Bodner manages to connect the fundamentals of the language to Go’s philosophy, guiding readers to write Go the way it was meant to be written.”

—Robert Liebowitz, Software Engineer at Morning
Consult

“《学习 Go》是了解 Go 编程语言独特之处以及使其如此强大的设计模式和惯用法的必备入门读物。Jon Bodner 将语言基础知识与 Go 的哲学理念巧妙地结合起来，引导读者以 Go 应有的方式编写代码。”——Robert Liebowitz, Morning Consult 软件工程师

“Jon wrote a book that does more than just reference Go; it provides an idiomatic and practical understanding of the language. Jon’s industry experience is what drives this book, and it will help those looking to be immediately productive in the language.”

—William Kennedy, Managing Partner at Ardan Labs

“Jon 的这本书不仅仅是对 Go 语言的简单介绍，它还提供了对这门语言的地道且实用的理解。Jon 的行业经验是本书的驱动力，它将帮助那些希望立即掌握 Go 语言并高效工作的人们。”——
Ardan Labs 管理合伙人 William Kennedy

Learning Go

学习

SECOND EDITION

第二版

An Idiomatic Approach to Real-World Go Programming

面向实际 Go 编程的惯用方法

Jon Bodner

乔恩·博德纳

O'REILLY®

Beijing • Boston • Farnham • Sebastopol • Tokyo

Learning Go

学习

by Jon Bodner

作者：乔恩·博德纳

Copyright © 2024 Jon Bodner. All rights reserved.

版权所有 © 2024 Jon Bodner。保留所有权利。

Printed in the United States of America.

美国印刷。

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North,
Sebastopol, CA 95472.

由 O'Reilly Media, Inc. 出版，地址：1005 Gravenstein Highway North,
Sebastopol, CA 95472。

O'Reilly books may be purchased for educational, business, or sales
promotional use. Online editions are also available for most titles
(<http://oreilly.com>). For more information, contact our

corporate/institutional sales department: 800-998-9938 or
corporate@oreilly.com.

O'Reilly 出版的图书可用于教育、商业或销售推广用途。大多数图书也提供在线版本 (<http://oreilly.com>)。欲了解更多信息, 请联系我们
的企业/机构销售部门: 800-998-9938 或 *corporate@oreilly.com*。

- Acquisitions Editor: Suzanne McQuade

收购编辑: 苏珊娜·麦奎德

- Developmental Editor: Rita Fernando

发展编辑: 丽塔·费尔南多

- Production Editor: Beth Kelly

制作编辑: 贝丝·凯利

- Copyeditor: Kim Cofer

文字编辑: 金·科弗

- Proofreader: Sharon Wilkey

校对员: 莎伦·威尔基

- Indexer: Sue Klefsaad

索引员: 苏·克莱夫斯塔德

- Interior Designer: David Futato

室内设计师: 大卫·富塔托

- Cover Designer: Karen Montgomery

封面设计: 凯伦·蒙哥马利

- Illustrator: Kate Dullea
插画师：凯特·杜利亚
- March 2021: First Edition
2021年3月：第一版
- January 2024: Second Edition
2024年1月：第二版

Revision History for the Second Edition

第二版修订历史

- 2024-01-11: First Release
2024年1月11日：首次发布

See <http://oreilly.com/catalog/errata.csp?isbn=0636920787686> for release details.

有关发行详情，请访问 <http://oreilly.com/catalog/errata.csp?isbn=0636920787686>。

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc.

Learning Go, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

O'Reilly 标志是 O'Reilly Media, Inc. 的注册商标。Learning Go、封面图像和相关商业外观是 O'Reilly Media, Inc. 的商标。

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

本文所表达的观点仅代表作者个人观点，并不代表出版商的观点。尽管出版商和作者已尽力确保本文所含信息和说明的准确性，但出版商和作者对任何错误或遗漏概不负责，包括但不限于因使用或依赖本文而造成的任何损失。使用本文所含信息和说明的风险由您自行承担。如果本文包含或描述的任何代码示例或其他技术受开源许可或其他知识产权的约束，您有责任确保您的使用符合此类许可和/或权利。

978-1-098-13929-2

[LSI]

[LSI]

Preface

前言

In the preface of the first edition, I wrote:

我在第一版序言中写道：

My first choice for a book title was Boring Go because, properly written, Go is boring....

Boring does not mean trivial. Using Go correctly requires an understanding of how its features are intended to fit together. While you can write Go code that looks like Java or Python, you're going to be unhappy with the result and wonder what all the fuss is about. That's where this book comes in. It walks through the features of Go, explaining how to best use them to write idiomatic code that can grow.

我最初想把书名定为《枯燥的Go语言》，因为如果写得好，Go语言确实很枯燥.....枯燥并不意味着简单。正确使用Go语言需要理解其特性是如何协同工作的。虽然你可以写出看起来像Java或Python的Go代码，但你最终会不满意，并且疑惑大家为什么对Go语言如此着迷。而这本书正是为了解决这个问题。它会一步步讲解Go语言的特性，并解释如何才能最好地利用它们来编写符合语言习惯且可扩展的代码。

Go remains a small language with a small feature set. It still lacks inheritance, aspect-oriented programming, function overloading, operator overloading, pattern matching, named parameters, exceptions, and many additional features that complicate other languages. So why does a book on a boring language need an update?

Go 仍然是一门功能较少的语言。它仍然缺少继承、面向切面编程、函数重载、运算符重载、模式匹配、命名参数、异常处理以及许多其他语言中常见的复杂特性。那么，一本关于这种枯燥语言的书为什么需要更新呢？

There are a few reasons for this edition. First, just as boring doesn't mean *trivial*, it also does not mean *unchanging*. Over the past three years, new features, tools, and libraries have arrived. Improvements like structured logging, fuzzing, workspaces, and vulnerability checking help Go developers create reliable, lasting, maintainable code. Now that Go developers have several years of experience with generics, the standard

library has started to include type constraints and generic functions to reduce repetitive code. Even the unsafe package has been updated to make it a little safer. Go developers need a resource that explains how to best use these new features.

推出本版的原因有几个。首先，正如“枯燥”并不意味着“琐碎”，它也并不意味着“一成不变”。过去三年里，Go 语言涌现出了许多新特性、工具和库。结构化日志、模糊测试、工作区和漏洞检查等改进，帮助 Go 开发者编写出可靠、持久且易于维护的代码。如今，Go 开发者已经积累了多年的泛型使用经验，标准库也开始引入类型约束和泛型函数，以减少重复代码。就连 unsafe 包也进行了更新，使其更加安全。Go 开发者需要一份资源，来解释如何才能最好地使用这些新特性。

Secondly, some aspects of Go weren't done justice by the first edition. The introductory chapter didn't flow as well as I'd like. The rich Go tool ecosystem wasn't explored. And first-edition readers asked for exercises and additional sample code. This edition attempts to address those limitations.

其次，第一版对 Go 语言的某些方面介绍得不够全面。引言部分不够流畅，也没有深入探讨 Go 语言丰富的工具生态系统。此外，第一版的读者还要求增加练习题和示例代码。本版旨在弥补这些不足。

Finally, the Go team has introduced something new, and, dare I say, *exciting*. There's now a strategy that allows Go to keep the backward

compatibility required for long-term software engineering projects while providing the ability to introduce backward-breaking changes to address long-standing design flaws. The new `for` loop variable scoping rules introduced in Go 1.22 are the first feature to take advantage of this approach.

最后，Go 团队推出了一些新东西，而且，恕我直言，令人兴奋。现在，Go 采用了一种策略，既能保持长期软件工程项目所需的向后兼容性，又能引入破坏性变更来解决长期存在的缺陷。Go 1.22 中引入的新的 `for` 循环变量作用域规则是第一个利用这种方法的特性。

Go is still boring, it's still fantastic, and it's better than ever. I hope you enjoy this second edition.

围棋依然枯燥乏味，依然精彩绝伦，而且比以往任何时候都更加出色。希望您喜欢这第二版。

Who Should Read This Book

谁应该读这本书？

This book is targeted at developers who are looking to pick up a second (or fifth) language. The focus is on people who are new to Go. This ranges from those who don't know anything about Go other than it has a cute

mascot, to those who have already worked through a Go tutorial or even written some Go code. The focus for *Learning Go* isn't just how to write programs in Go; it's how to write Go *idiomatically*. More experienced Go developers can find advice on how to best use the newer features of the language. The most important thing is that the reader wants to learn how to write Go code that looks like Go.

本书面向希望学习第二门（或第五门）编程语言的开发者。本书重点面向 Go 语言新手，涵盖了从对 Go 一无所知（只知道它有个可爱的吉祥物）到已经完成 Go 教程甚至编写过一些 Go 代码的初学者。本书的重点不仅在于如何用 Go 编写程序，更在于如何编写符合 Go 语言习惯的代码。经验丰富的 Go 开发者也能从中找到如何更好地利用 Go 语言新特性的建议。最重要的是，本书旨在帮助读者学习如何编写符合 Go 语言规范的代码。

Experience is assumed with the tools of the developer trade, such as version control (preferably Git) and IDEs. Readers should be familiar with basic computer science concepts like concurrency and abstraction, as the book explains how they work in Go. Some of the code examples are downloadable from GitHub, and dozens more can be tried out online on The Go Playground. While an internet connection isn't required, it is helpful when reviewing executable examples. Since Go is often used to build and call HTTP servers, some examples assume familiarity with basic HTTP concepts.

本书假定读者已熟练使用开发工具，例如版本控制系统（最好是 Git）和集成开发环境 (IDE)。读者应熟悉并发和抽象等基本计算机科学概念，因为本书会讲解这些概念在 Go 语言中的应用。部分代码示例可从 GitHub 下载，另有数十个示例可在 Go Playground 在线试运行。虽然并非必须连接互联网，但连接互联网有助于查看可执行示例。由于 Go 语言常用于构建和调用 HTTP 服务器，因此部分示例假定读者熟悉基本的 HTTP 概念。

While most of Go's features are found in other languages, Go makes different tradeoffs, so programs written in it have a different structure. *Learning Go* starts by looking at how to set up a Go development environment, and then covers variables, types, control structures, and functions. If you are tempted to skip over this material, resist the urge and take a look. It is often the details that make your Go code idiomatic. Some of what seems obvious at first glance might actually be subtly surprising when you think about it in depth.

虽然 Go 语言的大部分特性在其他语言中也能找到，但 Go 语言做出了不同的权衡，因此用它编写的程序结构也有所不同。学习 Go 语言首先要了解如何搭建 Go 开发环境，然后讲解变量、类型、控制结构和函数。如果你想跳过这部分内容，请克制住这种冲动，仔细阅读一下。细节往往决定了你的 Go 代码是否符合语言习惯。有些乍看之下显而易见的东西，深入思考后可能会让你感到意想不到的惊喜。

Conventions Used in This Book

本书中使用的约定

The following typographical conventions are used in this book:

本书采用以下排版规范：

Italic 斜体

Indicates new terms, URLs, email addresses, filenames, and file extensions.

指示新的术语、网址、电子邮件地址、文件名和文件扩展名。

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

用于程序清单，以及在段落中引用程序元素，例如变量或函数名称、数据库、数据类型、环境变量、语句和关键字。

Constant width bold

Shows commands or other text that should be typed literally by the user.

显示用户需要逐字输入的命令或其他文本。

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

显示应替换为用户提供的值或根据上下文确定的值的文本。

TIP

提示

This element signifies a tip or suggestion.

此元素表示提示或建议。

NOTE

笔记

This element signifies a general note.

此元素表示一般性说明。

WARNING

警告

This element indicates a warning or caution.

此元素表示警告或注意事项。

Using Code Examples

使用代码示例

Supplemental material (code examples, exercises, etc.) is available for download at <https://github.com/learning-go-book-2e>.

补充材料（代码示例、练习等）可在 <https://github.com/learning-go-book-2e> 下载。

If you have a technical question or a problem using the code examples, please send email to bookquestions@oreilly.com.

如果您在使用代码示例时遇到技术问题或问题，请发送电子邮件至 bookquestions@oreilly.com。

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're

reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

本书旨在帮助您完成工作。一般来说，如果本书提供了示例代码，您可以在自己的程序和文档中使用这些代码。除非您复制了大量代码，否则无需联系我们获得许可。例如，编写一个使用本书中几段代码的程序无需获得许可。但出售或分发 O'Reilly 出版的书籍中的示例需要获得许可。回答问题时引用本书并引用示例代码无需获得许可。但将本书中的大量示例代码整合到您的产品文档中则需要获得许可。

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Learning Go* by Jon Bodner (O'Reilly). Copyright 2024 Jon Bodner, 978-1-098-13929-2.”

我们感谢您注明出处，但并非强制要求。注明出处通常包括书名、作者、出版社和ISBN。例如：“《学习围棋》，作者：Jon Bodner（O'Reilly出版社）。版权所有© 2024 Jon Bodner，ISBN：978-1-098-13929-2。”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

如果您认为您对代码示例的使用超出了合理使用范围或上述许可范围，请随时通过 permissions@oreilly.com 与我们联系。

O'Reilly Online Learning

奥莱利在线学习

NOTE

笔记

For more than 40 years, [O'Reilly Media](#) has provided technology and business training, knowledge, and insight to help companies succeed.

40多年来，O'Reilly Media一直致力于提供技术和商业培训、知识和见解，以帮助企业取得成功。

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform.

O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <http://oreilly.com>.

我们独特的专家和创新者网络通过书籍、文章和我们的在线学习平台分享他们的知识和专业技能。O'Reilly 的在线学习平台让您可以根据需要访问直播培训课程、深入学习路径、交互式编程环境，以及来自 O'Reilly 和 200 多家其他出版商的大量文本和视频资源。欲了解更多信息，请访问 <http://oreilly.com>。

How to Contact Us

如何联系我们

Please address comments and questions concerning this book to the publisher:

请将有关本书的意见和问题直接发送给出版社：

O'Reilly Media, Inc.

奥莱利传媒公司

1005 Gravenstein Highway North

格雷文斯坦公路北段1005号

Sebastopol, CA 95472

加利福尼亚州塞巴斯托波尔，邮编 95472

800-889-8969 (in the United States or Canada)

800-889-8969（美国或加拿大境内）

707-827-7019 (international or local)

707-827-7019 (国际或本地)

707-829-0104 (fax)

707-829-0104 (传真)

support@oreilly.com support@oreilly.com

<https://www.oreilly.com/about/contact.html>

<https://www.oreilly.com/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/learning-go-2ed>.

我们为本书设立了一个网页，其中列出了勘误、示例和任何其他信息。您可以通过以下网址访问该网页：<https://oreil.ly/learning-go-2ed>。

For news and information about our books and courses, visit <https://oreilly.com>.

有关我们书籍和课程的新闻和信息，请访问 <https://oreilly.com>。

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>

在领英上找到我们：<https://linkedin.com/company/oreilly-media>

Follow us on Twitter: <https://twitter.com/oreillymedia>

请在Twitter上关注我们: <https://twitter.com/oreillymedia>

Watch us on YouTube: <https://youtube.com/oreillymedia>

请在 YouTube 上观看我们: <https://youtube.com/oreillymedia>

Acknowledgments for the 2nd Edition

第二版致谢

I've been humbled and flattered by the response to the first edition of *Learning Go*. It was started in late 2019, completed in late 2020, and released in early 2021. It was, unintentionally, my own pandemic project. Some people baked bread; I explained pointers.

《学习围棋》第一版的反响令我既感到荣幸又倍感谦卑。这本书始于2019年末，完成于2020年末，并于2021年初正式出版。无意间，它成了我疫情期间的个人项目。有人烤面包，我则讲解了棋步。

After I finished, people asked me what my next book would be. I told them that I was going to take a well-deserved break and then get to work on a romance novel. But before I could start writing about pirates and princesses, something amazing happened: *Learning Go* was a huge success. It spent the better part of a year as one of the five most popular books on O'Reilly Learning. As sales continued, I saw things in the first edition that I wanted

to fix and received letters from readers pointing out errors and omissions. Some were fixed as errata in later printings, but I contacted O'Reilly to see if they would like a new edition, and they were interested.

写完之后，人们问我下一本书会写什么。我告诉他们，我要好好休息一下，然后再开始写一本爱情小说。但还没等我开始写海盗和公主的故事，就发生了一件意想不到的事：《学习围棋》大获成功。它在奥莱利学习出版社（O'Reilly Learning）畅销书排行榜上稳居前五，持续了将近一年之久。随着销量持续增长，我发现第一版中有些地方需要修改，也收到了读者来信指出错误和遗漏。有些错误在后来的印刷版本中以勘误表的形式进行了修正，但我还是联系了奥莱利出版社，询问他们是否愿意出版新版，他们对此很感兴趣。

Returning to *Learning Go* and O'Reilly for the second edition was a pleasant experience. Rita Fernando provided guidance, feedback, and editing. This edition has been made immeasurably better by feedback from Jonathan Amsterdam, Leam Hall, Katie Hockman, Thomas Hunter, Max Horstmann, and Natalie Pistunovich. Chris Hines did an incredibly thorough job finding my mistakes and suggesting better examples. The comments and feedback from Abby Deng and the members of the Go Book Club at Datadog allowed me to make changes based on feedback from developers new to Go. The (hopefully few) remaining errors are all mine.

重返《Learning Go》和 O'Reilly 出版社进行第二版的编写是一次愉快的经历。Rita Fernando 提供了指导、反馈和编辑。Jonathan

Amsterdam、Leam Hall、Katie Hockman、Thomas Hunter、Max Horstmann 和 Natalie Pistunovich 的反馈使这一版有了巨大的改进。Chris Hines 非常细致地找到了我的错误并提出了更好的示例。Abby Deng 和 Datadog Go 书友会的成员们的评论和反馈使我能够根据 Go 新手开发者的反馈进行修改。剩下的（希望不多的）错误都是我的责任。

My wife and children were kind enough to not mind me skipping out on family movie nights while I figured out the best way to describe new Go features.

我的妻子和孩子们很体贴，并不介意我为了思考如何最好地描述 Go 的新功能而缺席家庭电影之夜。

I also want to thank the readers of the first edition who reached out to me with their kind words. I thank you all for your support and encouragement.

我还要感谢第一版读者们，感谢你们的鼓励和支持。感谢大家的支持和鼓励。

Acknowledgments for the 1st Edition

第一版致谢

Writing a book seems like a solitary task, but it doesn't happen without the help of a great number of people. I mentioned to Carmen Andoh that I wanted to write a book on Go and at GopherCon 2019, and she introduced me to Zan McQuade at O'Reilly. Zan guided me through the acquisition process and continued to provide me advice while I was writing *Learning Go*. Michele Cronin edited the text, gave feedback, and listened during the inevitable rough patches. Tonya Trybula's copyediting and Beth Kelly's production editing made my draft production-quality.

写书看似是一项孤独的任务，但实际上离不开众多人的帮助。我曾向卡门·安多（Carmen Andoh）提及我想写一本关于围棋的书，在2019年的GopherCon大会上，她向我介绍了O'Reilly出版社的赞·麦奎德（Zan McQuade）。赞指导我完成了收购流程，并在我撰写《学习围棋》（*Learning Go*）的过程中持续提供建议。米歇尔·克罗宁（Michele Cronin）负责编辑文本、提供反馈，并在我遇到写作瓶颈时给予耐心倾听。托尼亚·特里布拉（Tonya Trybula）的文稿编辑和贝丝·凯利（Beth Kelly）的制作编辑使我的初稿达到了出版水准。

While writing, I received critical feedback (and encouragement) from many people including Jonathan Altman, Jonathan Amsterdam, Johnny Ray

Austin, Chris Fauerbach, Chris Hines, Bill Kennedy, Tony Nelson, Phil Pearl, Liz Rice, Aaron Schlesinger, Chris Stout, Kapil Thangavelu, Claire Trivisonno, Volker Uhrig, Jeff Wendling, and Kris Zaragoza. I'd especially like to recognize Rob Liebowitz, whose detailed notes and rapid responses made this book far better than it would have been without his efforts.

在写作过程中，我收到了许多人的宝贵意见（以及鼓励），其中包括 Jonathan Altman、Jonathan Amsterdam、Johnny Ray Austin、Chris Fauerbach、Chris Hines、Bill Kennedy、Tony Nelson、Phil Pearl、Liz Rice、Aaron Schlesinger、Chris Stout、Kapil Thangavelu、Claire Trivisonno、Volker Uhrig、Jeff Wendling和Kris Zaragoza。我尤其要感谢Rob Liebowitz，他详尽的笔记和及时的回复使本书的质量大大提升，没有他的帮助，本书的质量将远不及现在。

My family put up with me spending nights and weekends at the computer instead of with them. In particular, my wife, Laura, graciously pretended that I didn't wake her up when I'd come to bed at 1 A.M. or later.

我的家人忍受着我晚上和周末都泡在电脑前，而不是陪伴他们。尤其是我的妻子劳拉，她非常体贴地假装我凌晨一点甚至更晚上床睡觉时没有吵醒她。

Finally, I want to remember the two people who started me on this path four decades ago. The first is Paul Goldstein, the father of a childhood friend. In 1982, Paul showed us a Commodore PET, typed PRINT 2 + 2, and hit the

Enter key. I was amazed when the screen said 4 and was instantly hooked. He later taught me how to program and even let me borrow the PET for a few weeks. Second, I'd like to thank my mother for encouraging my interest in programming and computers, despite having no idea what any of it was for. She bought me the BASIC programming cartridge for the Atari 2600, a VIC-20, and then a Commodore 64, along with the programming books that inspired me to want to write my own someday.

最后，我想感谢四十年前引领我走上这条道路的两个人。第一个是保罗·戈德斯坦，我儿时好友的父亲。1982年，保罗给我们展示了一台Commodore PET电脑，他输入了“PRINT 2 + 2”，然后按下了回车键。屏幕上显示“4”时，我惊叹不已，并立刻被电脑深深吸引。后来，他教我编程，甚至还借给我那台PET电脑玩了几个星期。第二个，我要感谢我的母亲，尽管她当时对编程和电脑一无所知，但她仍然鼓励我对它们的兴趣。她给我买了Atari 2600的BASIC编程卡带、VIC-20电脑，后来又买了Commodore 64电脑，还买了一些编程书籍，这些书籍激发了我日后编写自己程序的梦想。

Thank you all for helping make this dream of mine come true.

感谢大家帮助我实现这个梦想。

Chapter 1. Setting Up Your Go Environment

第一章 设置 Go 环境

Every programming language needs a development environment, and Go is no exception. If you've already built a Go program or two, then you have a working environment, but you might have missed out on some of the newer techniques and tools. If this is your first time setting up Go on your computer, don't worry; it's easy to install Go and its supporting tools. After setting up the environment and verifying it, you will build a simple program, learn about the different ways to build and run Go code, and then explore some tools and techniques that make Go development easier.

每种编程语言都需要开发环境，Go 也不例外。如果您已经编写过一两个 Go 程序，那么您就拥有了一个可用的开发环境，但您可能错过了一些较新的技术和工具。如果您是第一次在计算机上安装 Go，请不要担心；安装 Go 及其配套工具非常简单。在设置并验证开发环境之后，您将编写一个简单的程序，学习构建和运行 Go 代码的不同方法，并探索一些能够简化 Go 开发的工具和技巧。

Installing the Go Tools

安装 Go 工具

To build Go code, you need to download and install the Go development tools. You can find the latest version of the tools at the downloads page on the [Go website](#). Choose the download for your platform and install it. The *.pkg* installer for Mac and the *.msi* installer for Windows automatically install Go in the correct location, remove any old installations, and put the Go binary in the default executable path.

要编写 Go 代码，您需要下载并安装 Go 开发工具。您可以在 Go 官网的下载页面找到最新版本的工具。选择适合您平台的版本并进行安装。Mac 平台的 *.pkg* 安装程序和 Windows 平台的 *.msi* 安装程序会自动将 Go 安装到正确的位置，删除任何旧的安装，并将 Go 二进制文件放在默认的可执行文件路径中。

TIP

提示

If you are a Mac developer, you can install Go using [Homebrew](#) with the command `brew install go`. Windows developers who use [Chocolatey](#) can install Go with the command `choco install golang`.

如果您是 Mac 开发者，可以使用 Homebrew 通过命令 `brew install go` 安装 Go。使用 Chocolatey 的 Windows 开发者可以使用命令 `choco install golang` 安装 Go。

The various Linux and BSD installers are gzipped TAR files and expand to a directory named `go`. Copy this directory to `/usr/local` and add `/usr/local/go/bin` to your `$PATH` so that the `go` command is accessible:

各种 Linux 和 BSD 安装程序都是经过 gzip 压缩的 TAR 文件，解压后会生成一个名为 `go` 的目录。将此目录复制到 `/usr/local`，并将 `/usr/local/go/bin` 添加到您的 `$PATH` 中，以便可以访问 `go` 命令：

```
$ tar -C /usr/local -xzf go1.20.5.linux-  
amd64.tar.gz  
$ echo 'export PATH=$PATH:/usr/local/go/bin'  
>> $HOME/.bash_profile  
$ source $HOME/.bash_profile
```

You might need root permissions to write to `/usr/local`. If the `tar` command fails, rerun it with `sudo tar -C /usr/local -xzf`

```
go1.20.5.linux-amd64.tar.gz .
```

您可能需要 root 权限才能写入 /usr/local 目录。如果 tar 命令失败，请使用 `sudo tar -C /usr/local -xzf go1.20.5.linux-amd64.tar.gz` 重新运行该命令。

NOTE

笔记

Go programs compile to a single native binary and do not require any additional software to be installed in order to run them. This is in contrast to languages like Java, Python, and JavaScript, which require you to install a virtual machine to run your program. Using a single native binary makes it a lot easier to distribute programs written in Go. This book doesn't cover containers, but developers who use Docker or Kubernetes can often package a Go app inside a scratch or distroless image. You can find details in Geert Baeke's blog post [“Distroless or Scratch for Go Apps”](#).

Go 程序编译后生成单个本地二进制文件，无需安装任何额外的软件即可运行。这与 Java、Python 和 JavaScript 等语言截然不同，后者需要安装虚拟机才能运行程序。使用单个本地二进制文件大大简化了 Go 程序的分发。本书不涉及容器技术，但使用 Docker 或 Kubernetes 的开发者通常可以将 Go 应用打包到 Scratch 或 Distroless 镜像中。您可以在 Geert Baeke 的博客文章“Go 应用的 Distroless 或 Scratch”中找到更多详细信息。

You can validate that your environment is set up correctly by opening up a terminal or command prompt and typing:

您可以通过打开终端或命令提示符并输入以下命令来验证您的环境是否已正确设置：

```
$ go version
```

If everything is set up correctly, you should see something like this printed:

如果一切设置正确，您应该会看到类似这样的打印信息：

```
go version go1.20.5 darwin/arm64
```

This tells you that this is Go version 1.20.5 on macOS. (Darwin is the operating system at the heart of macOS, and arm64 is the name for the 64-bit chips based on ARM's designs.) On x64 Linux, you would see:

这表示这是 macOS 上的 Go 版本 1.20.5。（Darwin 是 macOS 的核心操作系统，arm64 是基于 ARM 架构的 64 位芯片的名称。）在 x64 Linux 系统上，你会看到：

```
go version go1.20.5 linux/amd64
```

Troubleshooting Your Go Installation

Go 安装故障排除

If you get an error instead of the version message, it's likely that you don't have `go` in your executable path, or you have another program named `go`

in your path. On macOS and other Unix-like systems, use `which go` to see the `go` command being executed, if any. If nothing is returned, you need to fix your executable path.

如果您看到的是错误信息而不是版本信息，则可能是您的可执行文件路径中缺少 `go`，或者路径中存在名为 `go` 的其他程序。在 macOS 和其他类 Unix 系统中，可以使用 `which go` 查看是否正在执行 `go` 命令。如果没有返回任何内容，则需要修复可执行文件路径。

If you're on Linux or BSD, it's possible you installed the 64-bit Go development tools on a 32-bit system or the development tools for the wrong chip architecture.

如果您使用的是 Linux 或 BSD 系统，则可能是您在 32 位系统上安装了 64 位 Go 开发工具，或者安装了适用于错误芯片架构的开发工具。

Go Tooling

Go Tooling

All of the Go development tools are accessed via the `go` command. In addition to `go version`, there's a compiler (`go build`), code formatter (`go fmt`), dependency manager (`go mod`), test runner (`go test`), a tool that scans for common coding mistakes (`go vet`), and more. They are covered in detail in Chapters [10](#), [11](#), and [15](#). For now, let's take a quick

look at the most commonly used tools by writing everyone's favorite first application: "Hello, World!"

所有 Go 开发工具都可通过 `go` 命令访问。除了 `go version` 之外，还有编译器 (`go build`)、代码格式化工具 (`go fmt`)、依赖管理器 (`go mod`)、测试运行器 (`go test`)、常见代码错误扫描工具 (`go vet`) 等等。它们在第 10、11 和 15 章中有详细介绍。现在，让我们通过编写每个人最喜欢的第一个应用程序“Hello, World!”来快速了解最常用的工具。

NOTE

笔记

Since the introduction of Go in 2009, several changes have occurred in the way Go developers organize their code and their dependencies. Because of this churn, there's lots of conflicting advice, and most of it is obsolete (for example, you can safely ignore discussions about `GOROOT` and `GOPATH`).

自 2009 年 Go 语言问世以来，Go 开发者组织代码及其依赖项的方式发生了多次变化。由于这种变化，出现了许多相互矛盾的建议，而且其中大部分已经过时（例如，您可以完全忽略关于 `GOROOT` 和 `GOPATH` 的讨论）。

For modern Go development, the rule is simple: you are free to organize your projects as you see fit and store them anywhere you want.

对于现代 Go 开发而言，规则很简单：您可以自由地按照自己的意愿组织项目，并将其存储在任何您想要的地方。

Your First Go Program

您的首次尝试计划

Let's go over the basics of writing a Go program. Along the way, you will see the parts that make up a simple Go program. You might not understand everything just yet, and that's OK; that's what the rest of the book is for!

让我们一起来学习编写 Go 程序的基础知识。在此过程中，你会看到构成一个简单的 Go 程序的各个部分。你可能现在还不能完全理解所有内容，这没关系；本书的其余部分就是为了解决这个问题而准备的！

Making a Go Module

制作 Go 模块

The first thing you need to do is create a directory to hold your program. Call it *ch1*. On the command line, enter the new directory. If your computer's terminal uses bash or zsh, this looks like the following:

首先，你需要创建一个目录来存放你的程序。将其命名为 `ch1`。在命令行中，输入这个新目录。如果你的计算机终端使用 `bash` 或 `zsh`，则命令如下所示：

```
$ mkdir ch1
$ cd ch1
```

Inside the directory, run the `go mod init` command to mark this directory as a Go module:

在该目录下，运行 `go mod init` 命令，将该目录标记为 Go 模块：

```
$ go mod init hello_world
go: creating new go.mod: module hello_world
```

You will learn more about what a module is in [Chapter 10](#), but for now, all you need to know is that a Go project is called a *module*. A module is not just source code. It is also an exact specification of the dependencies of the code within the module. Every module has a *go.mod* file in its root directory. Running `go mod init` creates this file for you. The contents of a basic *go.mod* file look like this:

你将在第 10 章详细了解模块的概念，但现在，你只需要知道 Go 项目被称为模块。模块不仅仅是源代码，它还精确地定义了模块内部代码的依赖关系。每个模块的根目录下都有一个 `go.mod` 文件。运行 `go mod init` 会自动创建这个文件。一个基本的 `go.mod` 文件的内容如下所示：


```
module hello_world

go 1.20
```

The *go.mod* file declares the name of the module, the minimum supported version of Go for the module, and any other modules that your module depends on. You can think of it as being similar to the *requirements.txt* file used by Python or the *Gemfile* used by Ruby.

go.mod 文件声明模块名称、模块支持的最低 Go 版本以及模块依赖的所有其他模块。您可以将其视为类似于 Python 的 requirements.txt 文件或 Ruby 的 Gemfile 文件。

You shouldn't edit the *go.mod* file directly. Instead, use the `go get` and `go mod tidy` commands to manage changes to the file. Again, all things related to modules are covered in [Chapter 10](#).

你不应该直接编辑 go.mod 文件。而应该使用 `go get` 和 `go mod tidy` 命令来管理对文件的更改。同样，所有与模块相关的内容都将在第 10 章中介绍。

go build

去建造

Now let's write some code! Open up a text editor, enter the following text, and save it inside *ch1* with the filename *hello.go*:

现在让我们来编写一些代码！打开文本编辑器，输入以下文本，并将其保存在 *ch1* 文件夹中，文件名为 *hello.go*：

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, world!")
}
```

(Yes, the indentation in this code example looks sloppy: I meant to do that! You will see why in just a bit.)

（是的，这段代码示例中的缩进看起来很凌乱：我故意这么做的！稍后您就会明白原因。）

Let's quickly go over the parts of the Go file you created. The first line is a package declaration. Within a Go module, code is organized into one or

more packages. The `main` package in a Go module contains the code that starts a Go program.

我们快速浏览一下你创建的 Go 文件的各个部分。第一行是包声明。在 Go 模块中，代码被组织成一个或多个包。Go 模块中的 `main` 包包含了启动 Go 程序的代码。

Next there is an import declaration. The `import` statement lists the packages referenced in this file. You're using a function in the `fmt` (usually pronounced “fumpt”) package from the standard library, so you list the package here. Unlike other languages, Go imports only whole packages. You can't limit the import to specific types, functions, constants, or variables within a package.

接下来是导入声明。`import` 语句列出了此文件中引用的包。您正在使用标准库中的 `fmt`（通常读作“fumpt”）包中的一个函数，因此需要在此处列出该包。与其他语言不同，Go 语言只能导入整个包。您无法将导入限制为包内的特定类型、函数、常量或变量。

All Go programs start from the `main` function in the `main` package. You declare this function with `func main()` and a left brace. Like Java, JavaScript, and C, Go uses braces to mark the start and end of code blocks.

所有 Go 程序都从 `main` 包中的 `main` 函数开始。您可以使用 `func main()` 和一个左大括号来声明此函数。与 Java、JavaScript 和 C 一样，Go 使用大括号来标记代码块的开始和结束。

The body of the function consists of a single line. It says that you are calling the `Println` function in the `fmt` package with the argument `"Hello, world!"`. As an experienced developer, you can likely guess what this function call does.

该函数体只有一行代码。它表明你正在调用 `fmt` 包中的 `Println` 函数，并传递参数 `"Hello, world!"`。作为一名经验丰富的开发者，你很可能已经猜到这个函数调用的作用了。

After the file is saved, go back to your terminal or command prompt and type:

文件保存后，返回终端或命令提示符并输入：

```
$ go build
```

This creates an executable called `hello_world` (or `hello_world.exe` on Windows) in the current directory. Run it and you will unsurprisingly see `Hello, world!` printed on the screen:

这会在当前目录下创建一个名为 `hello_world`（或 Windows 系统下的 `hello_world.exe`）的可执行文件。运行它，不出所料，屏幕上会显示 `Hello, world!`：

```
$ ./hello_world  
Hello, world!
```

The name of the binary matches the name in the module declaration. If you want a different name for your application, or if you want to store it in a different location, use the `-o` flag. For example, if you wanted to compile the code to a binary called “hello,” you would use the following:

二进制文件的名称与模块声明中的名称一致。如果您希望应用程序使用不同的名称，或者希望将其存储在不同的位置，请使用 `-o` 标志。例如，如果您希望将代码编译成名为“hello”的二进制文件，则可以使用以下代码：

```
$ go build -o hello
```

In [“Using go run to Try Out Small Programs”](#), I will cover another way to execute a Go program.

在“使用 go run 尝试小程序”一文中，我将介绍执行 Go 程序的另一种方法。

go fmt

去 fmt

One of the chief design goals for Go was to create a language that allowed you to write code efficiently. This meant having simple syntax and a fast compiler. It also led Go's authors to reconsider code formatting. Most languages allow a great deal of flexibility in the way code is formatted. Go does not. Enforcing a standard format makes it a great deal easier to write tools that manipulate source code. This simplifies the compiler and allows the creation of some clever tools for generating code.

Go 语言的主要设计目标之一是创造一种能够高效编写代码的语言。这意味着要拥有简洁的语法和快速的编译器。这也促使 Go 的开发者优先思考代码格式。大多数语言在代码格式方面都提供了很大的灵活性，而 Go 则不然。强制使用标准格式可以大大简化编写源代码操作工具的过程。这不仅简化了编译器，还使得一些巧妙的代码生成工具的开发成为可能。

There is a secondary benefit as well. Developers have historically wasted extraordinary amounts of time on format wars. Since Go defines a standard way of formatting code, Go developers avoid arguments over [brace style](#) and [tabs versus spaces](#). For example, Go programs use tabs to indent, and it is a syntax error if the opening brace is not on the same line as the declaration or command that begins the block.

此外，还有一个次要的好处。开发者们过去常常在代码格式之争上浪费大量时间。由于 Go 定义了一种标准的格式化方式，Go 开发者避免了关于大括号样式以及制表符与空格的争论。例如，Go 程序使用制表符进行缩进，如果左大括号与代码块的声明或命令不在同一行，则会被视为语法错误。

NOTE

笔记

Many Go developers think the Go team defined a standard format as a way to avoid developer arguments and discovered the tooling advantages later. However, [Russ Cox, the development lead for Go, has publicly stated](#) that better tooling was his original motivation.

许多 Go 开发者认为 Go 团队定义标准格式是为了避免开发者之间的争论，后来才发现了其在工具方面的优势。然而，Go 的开发负责人 Russ Cox 曾公开表示，更好的工具才是他最初的动机。

The Go development tools include a command, `go fmt`, which automatically fixes the whitespace in your code to match the standard format. However, it can't fix braces on the wrong line. Run it with the following:

Go 开发工具包含一个命令 `go fmt`，它可以自动将代码中的空格修正为标准格式。但是，它无法修正错行的大括号。运行以下命令即可：

```
$ go fmt ./...
```

```
hello.go
```

Using `./...` tells a Go tool to apply the command to all the files in the current directory and all subdirectories. You will see it again as you learn about more Go tools.

使用 `./...` 会指示 Go 工具将命令应用于当前目录及其所有子目录中的所有文件。在学习更多 Go 工具时，您还会再次遇到它。

If you open up *hello.go*, you'll see that the line with `fmt.Println` is now properly indented with a single tab.

打开 `hello.go` 文件，你会看到包含 `fmt.Println` 的那一行现在已经正确地缩进了单个制表符。

TIP

提示

Remember to run `go fmt` before you compile your code, and, at the very least, before you commit source code changes to your repository! If you forget, make a separate commit that does *only* `go fmt ./...` so you don't hide logic changes in an avalanche of formatting changes.

请记住在编译代码之前运行 `go fmt`，至少在将源代码更改提交到代码仓库之前也要运行！如果您忘记了，请单独提交一次，只执行 `go fmt ./...`，这样就不会让逻辑更改淹没在大量的格式更改中。

THE SEMICOLON INSERTION RULE

分号插入规则

The `go fmt` command won't fix braces on the wrong line because of the *semicolon insertion rule*. Like C or Java, Go requires a semicolon at the end of every statement. However, Go developers should never put the semicolons in themselves. The Go compiler adds them automatically, following a simple rule described in [Effective Go](#). If the last token before a newline is any of the following, the lexer inserts a semicolon after the token:

由于分号插入规则，`go fmt` 命令无法修复错误行上的大括号。与 C 或 Java 一样，Go 要求每个语句末尾都必须有分号。但是，Go 开发者不应该手动添加分号。Go 编译器会根据 Effective Go 中描述的简单规则自动添加分号。如果换行符前的最后一个标记是以下任何一种，词法分析器都会在该标记后插入一个分号：

- An identifier (which includes words like `int` and `float64`)
标识符（包含类似 `int` 和 `float64` 的词语）
- A basic literal such as a number or string constant
基本字面量，例如数字或字符串常量
- One of the tokens: `break`, `continue`, `fallthrough`, `return`, `++`, `--`, `)`, or `}`
以下令牌之一：`break`、`continue`、`fallthrough`、`return`、`++`、`--`、`)` 或 `}`

With this simple rule in place, you can see why putting a brace in the wrong place breaks. If you write your code like this:

有了这条简单的规则，你就能明白为什么把大括号放在错误的位置会导致错误。如果你像这样编写代码：

```
func main()  
{  
    fmt.Println("Hello, world!")  
}
```

the semicolon insertion rule sees the `)` at the end of the `func main()` line and turns that into:

分号插入规则会将 `)` 插入到 `{{id_00001}}` 行的末尾，并将其转换为：

```
func main();  
{  
    fmt.Println("Hello, world!");  
};
```

And that's not valid Go.

那不是有效的 Go 代码。

The semicolon insertion rule and the resulting restriction on brace placement is one of the things that makes the Go compiler simpler and faster, while at the same time enforcing a coding style. That's clever.

分号插入规则以及由此产生的对大括号位置的限制，是使 Go 编译器更简洁、更快速的原因之一，同时还能强制执行一种编码风格。这很巧妙。

go vet

去兽医

In one class of bugs, the code is syntactically valid but quite likely incorrect. The `go` tool includes a command called `go vet` to detect these kinds of errors. Add one to the program and watch it get detected. Change the `fmt.Println` line in *hello.go* to the following:

有一类错误，代码语法正确，但很可能存在错误。`go` 工具包含一个名为 `go vet` 的命令，用于检测这类错误。将一个这样的错误添加到程序中，看看它是否会被检测到。将 *hello.go* 文件中的 `fmt.Println` 行更改为以下内容：

```
fmt.Printf("Hello, %s!\n")
```

NOTE

笔记

`fmt.Printf` is similar to `printf` in C, Java, Ruby, and many other languages. If you haven't seen `fmt.Printf` before, it is a function with a template for its first parameter, and values for the placeholders in the template in the remaining parameters.

`fmt.Printf` 与 C、Java、Ruby 和许多其他语言中的 `printf` 类似。如果您之前没有见过 `fmt.Printf`，它是一个函数，其第一个参数是模板，其余参数是模板中占位符的值。

In this example, you have a template (`"Hello, %s!\n"`) with a `%s` placeholder, but no value was specified for the placeholder. This code will compile and run, but it's not correct. One of the things that `go vet` detects is whether a value exists for every placeholder in a formatting template. Run `go vet` on the modified code, and it finds an error:

在这个例子中，你有一个模板 (`"Hello, %s!\n"`)，其中包含一个 `%s` 占位符，但该占位符没有指定值。这段代码可以编译运行，但结果不正确。`go vet` 的作用之一是检测格式模板中每个占位符是否存在值。对修改后的代码运行 `go vet`，它会发现一个错误：

```
$ go vet ./...  
# hello_world  
./hello.go:6:2: fmt.Printf format %s reads arg #1,  
but call has 0 args
```

Now that `go vet` found the bug, you can easily fix it. Change line 6 in *hello.go* to:

既然 `go vet` 发现了这个 bug，你可以轻松修复它。将 `hello.go` 文件中的第 6 行修改为：

```
fmt.Printf("Hello, %s!\n", "world")
```

While `go vet` catches several common programming errors, there are things that it cannot detect. Luckily, third-party Go code-quality tools can close the gap. Some of the most popular code-quality tools are covered in [“Using Code-Quality Scanners”](#).

虽然 `go vet` 可以检测到许多常见的编程错误，但它也无法检测到所有错误。幸运的是，第三方 Go 代码质量工具可以弥补这一不足。“使用代码质量扫描器”一文介绍了一些最流行的代码质量工具。

TIP

提示

Just as you should run `go fmt` to make sure your code is formatted properly, run `go vet` to scan for possible bugs in valid code. These commands are just the first step in ensuring that your code is of high quality. In addition to the advice in this book, all Go developers should read through [Effective Go](#) and the [Code Review Comments page on Go's wiki](#) to understand what idiomatic Go code looks like.

就像您应该运行 `go fmt` 来确保代码格式正确一样，也应该运行 `go vet` 来扫描有效代码中可能存在的错误。这些命令只是确保代码高质量的第一步。除了本书中的建议之外，所有 Go 开发者都应该阅读《Effective Go》以及 Go wiki 上的“代码审查注释”页面，以了解惯用的 Go 代码是什么样的。

Choose Your Tools

选择您的工具

While you wrote a small Go program using nothing more than a text editor and the `go` command, you'll probably want more advanced tools when working on larger projects. Go IDEs provide many advantages over text editors, including automatic formatting on save, code completion, type checking, error reporting, and integrated debugging. Excellent [Go development tools](#) are available for most text editors and IDEs. If you don't already have a favorite tool, two of the most popular Go development environments are Visual Studio Code and GoLand.

虽然你可能只用文本编辑器和 `go` 命令就编写了一个小型 Go 程序，但在开发大型项目时，您可能需要更高级的工具。Go IDE 相比文本编辑器有很多优势，包括保存时自动格式化、代码补全、类型检查、错误报告和集成调试。大多数文本编辑器和 IDE 都提供了优秀的 Go 开发工具。如果你还没有自己喜欢的工具，Visual Studio Code 和 GoLand 是两个最流行的 Go 开发环境。

Visual Studio Code

Visual Studio Code

If you are looking for a free development environment, [Visual Studio Code](#) from Microsoft is your best option. Since it was released in 2015, VS Code has become the most popular source code editor for developers. It does not ship with Go support, but you can make it a Go development environment by downloading the Go extension from the extensions gallery.

如果您正在寻找免费的开发环境，微软的 Visual Studio Code 是您的最佳选择。自 2015 年发布以来，VS Code 已成为开发者最常用的源代码编辑器。它本身并不支持 Go 语言，但您可以从扩展库下载 Go 扩展，将其转换为 Go 开发环境。

VS Code's Go support relies on third-party extensions that are accessed via its built-in Marketplace. This includes the Go Development tools, the [Delve debugger](#), and [gopls](#), a Go language server developed by the Go team.

While you need to install the Go compiler yourself, the Go extension will install Delve and gopls for you.

VS Code 对 Go 语言的支持依赖于第三方扩展，这些扩展可通过其内置的 Marketplace 获取。其中包括 Go 开发工具、Delve 调试器以及 gopls（由 Go 团队开发的 Go 语言服务器）。虽然您需要自行安装 Go 编译器，但 Go 扩展会自动为您安装 Delve 和 gopls。

NOTE

笔记

What is a language server? It's a standard specification for an API that enables editors to implement intelligent editing behavior, like code completion, quality checks, or finding all the places a variable or function is used in your code. You can learn more about language servers and their capabilities by checking out the [Language Server Protocol website](#).

什么是语言服务器？它是一种标准规范，用于定义一个 API，使编辑器能够实现智能编辑行为，例如代码补全、质量检查，或者查找代码中所有使用变量或函数的地方。您可以访问语言服务器协议（LSP）网站，了解更多关于语言服务器及其功能的信息。

Once your tools are set up, you can open your project and work with it.

[Figure 1-1](#) shows you what your project window should look like. [“Getting Started with VS Code Go”](#) is a walkthrough that demonstrates the VS Code Go extension.

工具设置完成后，即可打开项目并开始工作。图 1-1 显示了项目窗口的默认外观。“VS Code Go 入门”是一个演示 VS Code Go 扩展的详细

教程。

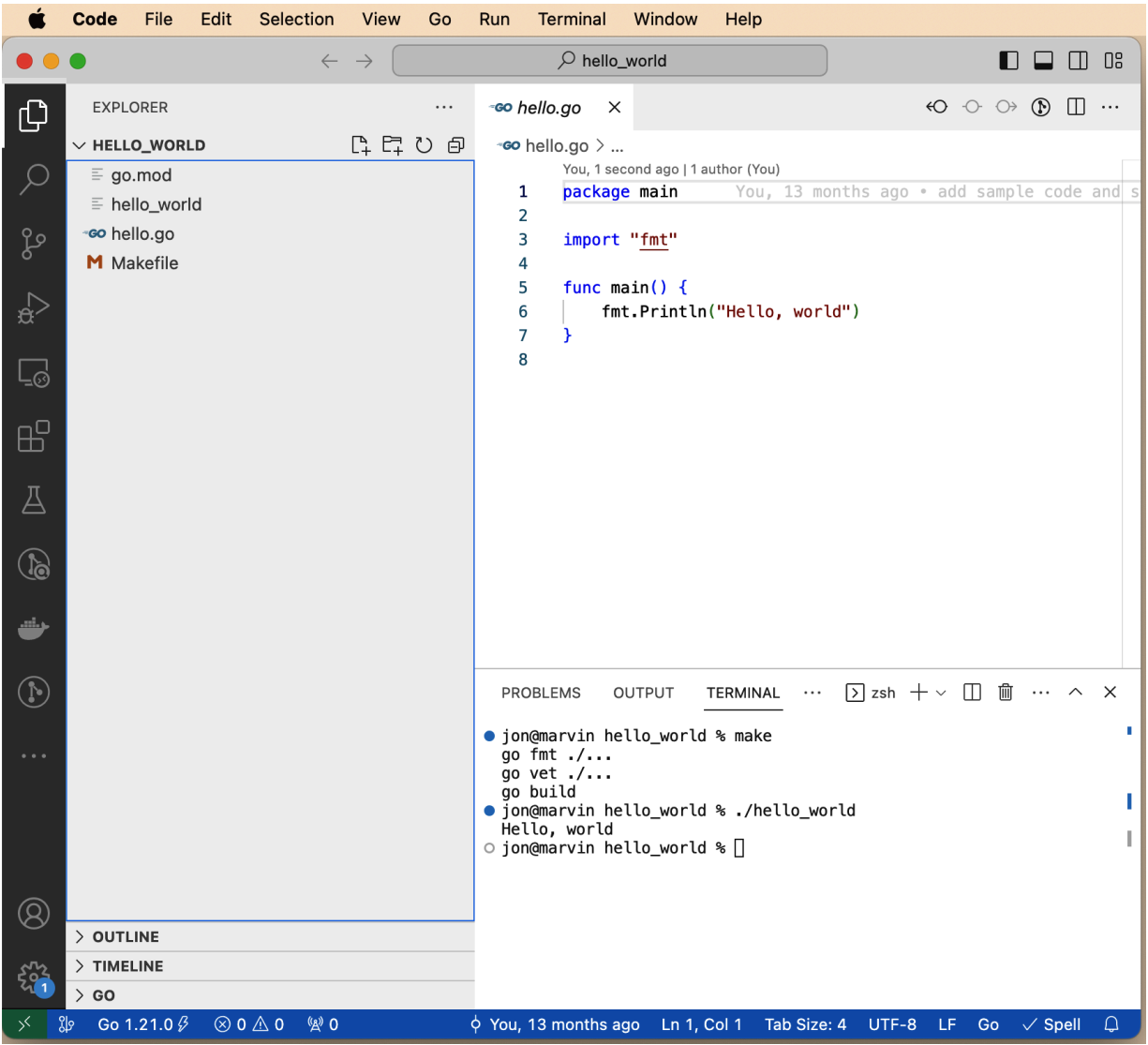


Figure 1-1. Visual Studio Code

图 1-1. Visual Studio Code

GoLand

戈兰

GoLand is the Go-specific IDE from JetBrains. While JetBrains is best known for Java-centric tools, GoLand is an excellent Go development environment. As you can see in [Figure 1-2](#), GoLand's user interface looks similar to IntelliJ, PyCharm, RubyMine, WebStorm, Android Studio, or any of the other JetBrains IDEs. Its Go support includes refactoring, syntax highlighting, code completion and navigation, documentation pop-ups, a debugger, code coverage, and more. In addition to Go support, GoLand includes JavaScript/HTML/CSS and SQL database tools. Unlike VS Code, GoLand doesn't require you to install a plug-in to get it to work.

GoLand 是 JetBrains 出品的 Go 语言专用 IDE。虽然 JetBrains 最为人们熟知的是其 Java 开发工具，但 GoLand 也是一款优秀的 Go 开发环境。如图 1-2 所示，GoLand 的用户界面与 IntelliJ、PyCharm、RubyMine、WebStorm、Android Studio 或其他 JetBrains IDE 类似。它对 Go 语言的支持包括重构、语法高亮、代码补全和导航、文档弹出窗口、调试器、代码覆盖率等等。除了 Go 支持之外，GoLand 还包含 JavaScript/HTML/CSS 和 SQL 数据库工具。与 VS Code 不同，GoLand 无需安装任何插件即可使用。

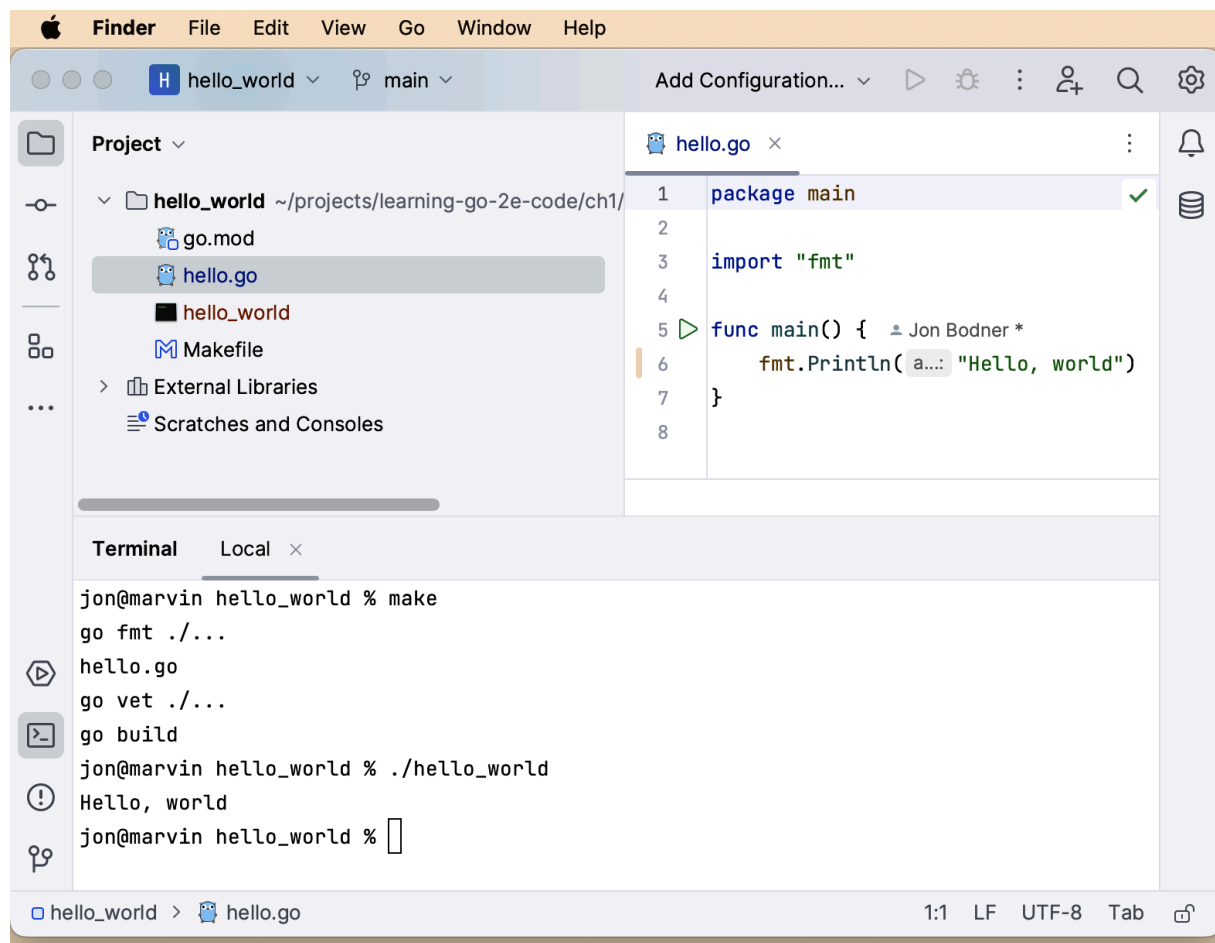


Figure 1-2. GoLand

图 1-2. GoLand

If you have already subscribed to IntelliJ Ultimate, you can add Go support via a plug-in. While GoLand is commercial software, JetBrains has a [Free License Program](#) for students and core open source contributors. If you don't qualify for a free license, a 30-day free trial is available. After that, you have to pay for GoLand.

如果您已订阅 IntelliJ Ultimate，可以通过插件添加 Go 支持。虽然 GoLand 是商业软件，但 JetBrains 为学生和核心开源贡献者提供免费

许可计划。如果您不符合免费许可条件，可以享受 30 天免费试用。试用期结束后，您需要付费使用 GoLand。

The Go Playground

Go Playground

There's one more important tool for Go development, but this is one that you don't install. Visit [The Go Playground](#) and you'll see a window that resembles [Figure 1-3](#). If you have used a command-line environment like `irb`, `node`, or `python`, you'll find The Go Playground has a similar feel. It gives you a place to try out and share small programs. Enter your program into the window and click the Run button to execute the code. The Format button runs `go fmt` on your program and updates your imports. The Share button creates a unique URL that you can send to someone else to take a look at your program or to come back to your code at a future date (the URLs have proven to be persistent for a long time, but I wouldn't rely on the playground as your source code repository).

还有一个对 Go 开发至关重要的工具，但它无需安装。访问 Go Playground，你会看到一个类似于图 1-3 的窗口。如果你使用过类似 `irb`、`node` 或 `python` 的命令行环境，你会发现 Go Playground 与之类似。它提供了一个可以尝试和分享小型程序的地方。在窗口中输入你的程序，然后点击“运行”按钮即可执行代码。“格式化”按钮会对你的程序运行 `go fmt` 并更新你的导入语句。“分享”按钮会生成一个

唯一的 URL，你可以将其发送给其他人，让他们查看你的程序或在未来某个时候重新访问你的代码（这些 URL 已被证明可以长期有效，但我建议不要将 Playground 作为你的源代码仓库）。



Figure 1-3. The Go Playground

图 1-3. 游乐场

As you can see in [Figure 1-4](#), you can simulate multiple files by separating each file with a line that looks like `-- filename.go --`. You can even create simulated subdirectories by including a `/` in the filename, such as `- subdir/my_code.go --`.

如图 1-4 所示，您可以通过用类似 `-- filename.go --` 的行分隔每个文件来模拟多个文件。您甚至可以通过在文件名中包含 `/` 来创建模拟子目录，例如 `-- subdir/my_code.go --`。

Be aware that The Go Playground is on someone else's computer (in particular, Google's computer), so you don't have completely free rein. It gives you a choice of a few versions of Go (usually the current release, the previous release, and the latest development version). You can make network connections only to `localhost`, and processes that run for too long or use too much memory are stopped. If your program depends on time, you need to take into account that the clock is set to November 10, 2009, 23:00:00 UTC (the date of the initial announcement of Go). Even with these limitations, The Go Playground is a useful way to try out new ideas without creating a new project locally. Throughout this book, you'll find links to The Go Playground so you can run code examples without copying them onto your computer.

请注意，Go Playground 运行在别人的电脑上（具体来说是 Google 的电脑），因此您并非拥有完全的自由。它提供了几个 Go 版本供您选择（通常是当前版本、上一个版本和最新的开发版本）。您只能连接到 `localhost`，并且运行时间过长或占用过多内存的进程会被终止。如果您的程序依赖于时间，则需要注意其时钟设置为 2009 年 11 月 10 日 23:00:00 UTC（Go 语言首次发布的日期）。即使存在这些限制，Go Playground 仍然是尝试新想法而无需在本地创建新项目的有效途径。本书中会提供 Go Playground 的链接，以便您可以运行代码示例而无需将其复制到自己的电脑上。

WARNING

警告

Do not put sensitive information (such as personally identifiable information, passwords, or private keys) into your playground! If you click the Share button, the information is saved on Google's servers and is accessible to anyone who has the associated Share URL. If you do this by accident, contact Google at security@golang.org with the URL and the reason the content needs to be removed.

请勿在您的 Playground 中放置敏感信息（例如个人身份信息、密码或私钥）！如果您点击“分享”按钮，信息将保存在 Google 的服务器上，任何拥有相应分享链接的人都可以访问这些信息。如果您不小心执行了此操作，请发送电子邮件至 security@golang.org 联系 Google，并提供链接以及需要删除内容的原因。

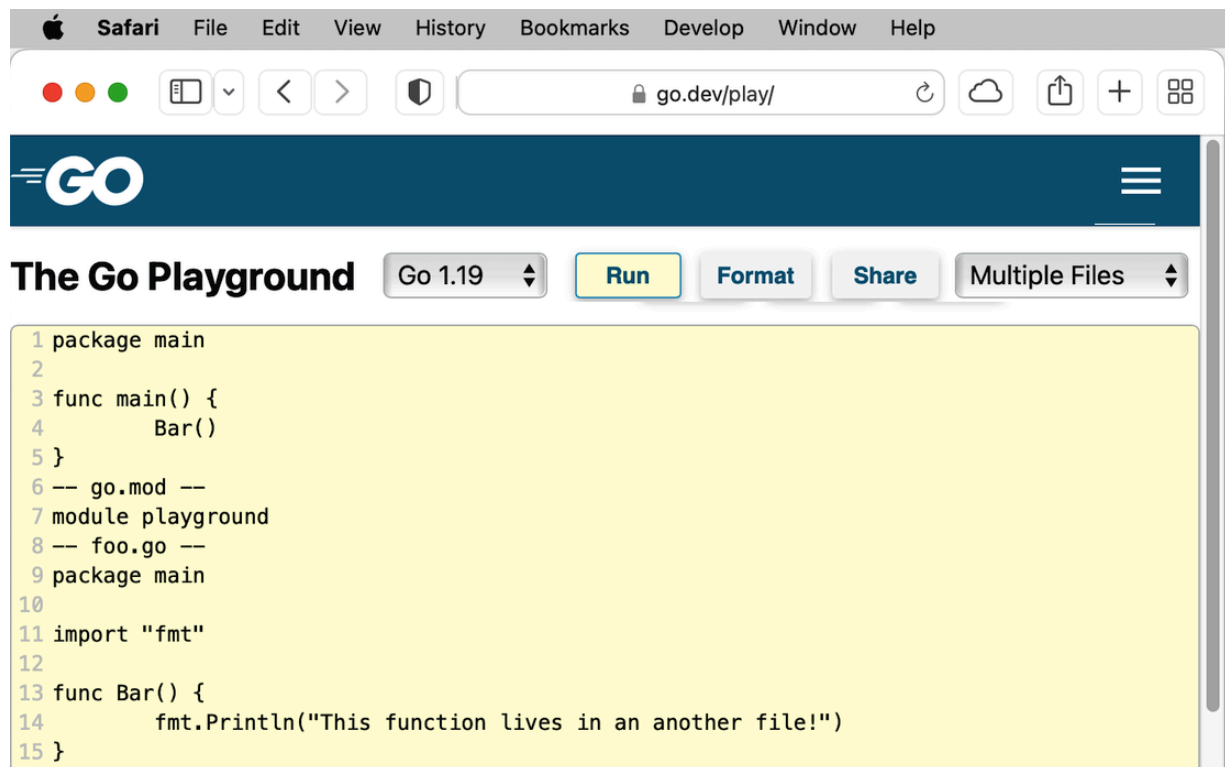


Figure 1-4. The Go Playground supports multiple files

图 1-4. Go Playground 支持多个文件

Makefiles

Makefiles

An IDE is nice to use, but it's hard to automate. Modern software development relies on repeatable, automatable builds that can be run by anyone, anywhere, at any time. Requiring this kind of tooling is good software engineering practice. It avoids the age-old situation where a developer absolves themselves of any build problems with a shrug and the statement, "It works on my machine!" The way to do this is to use some kind of script to specify your build steps. Go developers have adopted `make` as their solution. It lets developers specify a set of operations that are necessary to build a program and the order in which the steps must be performed. You may not be familiar with `make`, but it's been used to build programs on Unix systems since 1976.

集成开发环境 (IDE) 用起来很方便，但很难实现自动化。现代软件开发依赖于可重复、可自动化的构建流程，任何人都可以随时随地运行这些流程。要求使用这类工具是良好的软件工程实践。它避免了开发者耸耸肩，说一句“在我机器上没问题！”就推卸所有构建问题的老套路。实现这一目标的方法是使用某种脚本来指定构建步骤。Go 开发者采用了 `make` 作为他们的解决方案。它允许开发者指定构建程序所需的一系列操作以及这些步骤的执行顺序。你可能不熟悉 `make`，但它自 1976 年以来就一直用于在 Unix 系统上构建程序。

Create a file called *Makefile* in the *ch1* directory with the following contents:

在 ch1 目录下创建一个名为 Makefile 的文件，内容如下：

```
.DEFAULT_GOAL := build

.PHONY:fmt vet build
fmt:
    go fmt ./...

vet: fmt
    go vet ./...

build: vet
    go build
```

Even if you haven't seen a Makefile before, figuring out what's going on is not too difficult. Each possible operation is called a *target*. The

`.DEFAULT_GOAL` defines which target is run when no target is specified. In this case, the default is the `build` target. Next you have the target definitions. The word before the colon (`:`) is the name of the target. Any words after the target (like `vet` in the line `build: vet`) are the other targets that must be run before the specified target runs. The tasks that are performed by the target are on the indented lines after the target. The

`.PHONY` line keeps `make` from getting confused if a directory or file in your project has the same name as one of the listed targets.

即使你以前没见过 Makefile，理解它的工作原理也并不难。每个可能的操作都称为一个目标（target）。`.DEFAULT_GOAL` 定义了在没有指定目标时要运行的目标。在本例中，默认目标为 `build`。接下来是目标定义。冒号前的单词（`:`）是目标的名称。目标名称后的任何单词（例如 `build: vet` 行中的 `vet`）都是在指定目标运行之前必须运行的其他目标。目标执行的任务位于目标名称后的缩进行中。`.PHONY` 行可以防止 `make` 在项目中的目录或文件与列出的某个目标同名时出错。

Run `make` and you should see the following output:

运行 `make`，您应该会看到以下输出：

```
$ make
go fmt ./...
go vet ./...
go build
```

Entering a single command formats the code correctly, checks it for nonobvious errors, and compiles it. You can also vet the code with `make vet`, or just run the formatter with `make fmt`. This might not seem like a big improvement, but ensuring that formatting and vetting always happen

before a developer (or a script running on a continuous integration build server) triggers a build means you won't miss any steps.

只需输入一条命令，即可正确格式化代码、检查隐蔽错误并进行编译。您还可以使用 `make vet` 来验证代码，或者直接使用 `make fmt` 运行格式化程序。这看似改进不大，但确保在开发人员（或持续集成构建服务器上运行的脚本）触发构建之前始终完成格式化和验证，意味着您不会遗漏任何步骤。

One drawback to Makefiles is that they are exceedingly picky. You *must* indent the steps in a target with a tab. They are also not supported out-of-the-box on Windows. If you are doing your Go development on a Windows computer, you need to install `make` first. The easiest way to do so is to first install a package manager like [Chocolatey](#) and then use it to install `make` (for Chocolatey, the command is `choco install make`).

Makefile 的一个缺点是它对语法要求非常严格。你必须使用制表符缩进目标中的步骤。此外，Windows 系统本身并不支持 Makefile。如果你在 Windows 电脑上进行 Go 开发，你需要先安装 `make`。最简单的方法是先安装一个包管理器，例如 Chocolatey，然后使用它来安装 `make`（对于 Chocolatey，命令是 `choco install make`）。

If you want to learn more about writing Makefiles, there's a good [tutorial by Chase Lambert](#), but it does use a tiny bit of C to explain the concepts.

如果你想了解更多关于编写 Makefile 的知识，Chase Lambert 写了一篇很好的教程，不过它确实用了一点 C 语言来解释概念。

You can find the code from this chapter in the [Chapter 1 repository](#) for this book.

您可以在本书第一章的代码库中找到本章的代码。

The Go Compatibility Promise

Go兼容性承诺

As with all programming languages, the Go development tools are periodically updated. Since Go 1.2, a new release has occurred roughly every six months. Patch releases with bug and security fixes are also released as needed. Given the rapid development cycles and the Go team's commitment to backward compatibility, Go releases tend to be incremental rather than expansive. The [Go Compatibility Promise](#) is a detailed description of how the Go team plans to avoid breaking Go code. It says that there won't be backward-breaking changes to the language or the standard library for any Go version that starts with 1, unless the change is required for a bug or security fix. In his GopherCon 2022 keynote talk [“Compatibility: How Go Programs Keep Working”](#), Russ Cox discusses all the ways that the Go Team works to keep Go code from breaking. He says,

“I believe that prioritizing compatibility was the most important design decision that we made in Go 1.”

与所有编程语言一样，Go 开发工具也会定期更新。自 Go 1.2 以来，大约每六个月发布一个新版本。此外，还会根据需要发布包含错误修复和安全补丁的补丁版本。鉴于 Go 开发周期短，且团队致力于向后兼容，因此 Go 的版本更新往往是增量式的，而非大规模的。Go 兼容性承诺详细描述了 Go 团队如何避免破坏 Go 代码。该承诺规定，对于任何以 1 开头的 Go 版本，语言本身或标准库都不会出现破坏向后兼容性的更改，除非是为了修复错误或安全问题。在 2022 年 GopherCon 大会上，Russ Cox 发表了题为“兼容性：Go 程序如何保持运行”的主题演讲，详细探讨了 Go 团队为防止 Go 代码破坏而采取的各种措施。他表示：“我认为，优先考虑兼容性是我们在 Go 1 中做出的最重要的设计决策。”

This guarantee doesn't apply to the `go` commands. There have been backward-incompatible changes to the flags and functionality of the `go` commands, and it's entirely possible that it will happen again.

此保证不适用于 `go` 命令。`go` 命令的标志和功能已发生过向后不兼容的更改，而且这种情况完全有可能再次发生。

Staying Up-to-Date

保持最新状态

Go programs compile to a standalone native binary, so you don't need to worry that updating your development environment could cause your currently deployed programs to fail. You can have programs compiled with different versions of Go running simultaneously on the same computer or virtual machine.

Go 程序编译成独立的本地二进制文件，因此您无需担心更新开发环境会导致当前部署的程序运行失败。您可以让使用不同 Go 版本编译的程序同时运行在同一台计算机或虚拟机上。

When you are ready to update the Go development tools installed on your computer, Mac and Windows users have the easiest path. Those who installed with `brew` or `chocolatey` can use those tools to update. Those who used the installers on <https://golang.org/dl> can download the latest installer, which removes the old version when it installs the new one.

当您准备更新计算机上安装的 Go 开发工具时，Mac 和 Windows 用户可以使用最简便的方法。使用 `brew` 或 `chocolatey` 安装的用户可以使用这些工具进行更新。使用 <https://golang.org/dl> 上的安装程序的

用户可以下载最新的安装程序，该程序会在安装新版本时自动卸载旧版本。

Linux and BSD users need to download the latest version, move the old version to a backup directory, unpack the new version, and then delete the old version:

Linux 和 BSD 用户需要下载最新版本，将旧版本移动到备份目录，解压新版本，然后删除旧版本：

```
$ mv /usr/local/go /usr/local/old-go
$ tar -C /usr/local -xzf go1.20.6.linux-
amd64.tar.gz
$ rm -rf /usr/local/old-go
```

NOTE

笔记

Technically, you don't need to move the existing installation to a new location; you could just delete it and install the new version. However, this falls in the “better safe than sorry” category. If something goes wrong while installing the new version, it's good to have the previous one around.

理论上，您无需将现有安装迁移到新位置；您可以直接删除旧版本并安装新版本。但是，这属于“防患于未然”的范畴。如果在安装新版本时出现问题，保留旧版本会很有帮助。

Exercises

练习

Each chapter has exercises at the end to let you try out the ideas that I cover. You can find answers to these exercises in the [Chapter 1 repository](#).

每章末尾都附有练习题，帮助你实践我所讲解的内容。你可以在第一章的题库中找到这些练习题的答案。

1. Take the “Hello, world!” program and run it on The Go Playground.

Share a link to the code in the playground with a coworker who would love to learn about Go.

把“Hello, world!”程序放到 Go Playground 上运行。然后把代码链接分享给一位对学习 Go 语言感兴趣的同事。

2. Add a target to the Makefile called `clean` that removes the `hello_world` binary and any other temporary files created by `go build`. Take a look at the [Go command documentation](#) to find a `go` command to help implement this.

在 Makefile 中添加一个名为 `clean` 的目标，该目标会移除 `hello_world` 二进制文件以及 `go build` 创建的任何其他临时文件。请查阅 Go 命令文档，找到一个 `go` 命令来帮助实现此功能。

3. Experiment with modifying the formatting in the “Hello, world!”

program. Add blank lines, spaces, change indentation, insert newlines.

After making a modification, run `go fmt` to see if the formatting change is undone. Also, run `go build` to see if the code still compiles. You can also add additional `fmt.Println` calls so you can see what happens if you put blank lines in the middle of a function. 尝试修改“Hello, world!”程序中的格式。添加空行、空格、更改缩进、插入换行符。修改后，运行 `go fmt` 查看格式更改是否被撤销。同时，运行 `go build` 查看代码是否仍然可以编译。您还可以添加额外的 `fmt.Println` 调用，以便查看在函数中间插入空行会发生什么情况。

Wrapping Up

总结

In this chapter, you learned how to install and configure your Go development environment. You also learned about tools for building Go programs and ensuring code quality. Now that your environment is ready, you’re on to the next chapter, where you’ll explore the built-in types in Go and how to declare variables.

在本章中，您学习了如何安装和配置 Go 开发环境。您还了解了用于构建 Go 程序和确保代码质量的工具。现在您的环境已准备就绪，接

下来您将进入下一章，探索 Go 的内置类型以及如何声明变量。

Chapter 2. Predeclared Types and Declarations

第二章 预声明类型和声明

Now that you have your development environment set up, it's time to start looking at Go's language features and how to best use them. When trying to figure out what “best” means, there is one overriding principle: write your programs in a way that makes your intentions clear. As I go through features and discuss the options, I'll explain why I find that a particular approach produces clearer code.

现在你的开发环境已经搭建完毕，是时候开始了解 Go 语言的特性以及如何才能最好地使用它们了。在思考“最好”的含义时，有一个最重要的原则：编写程序时要确保你的意图清晰明了。在介绍特性和讨论各种选项的过程中，我会解释为什么我认为某种特定的方法能够生成更清晰的代码。

I'll start by looking at the types that are built into Go and how to declare variables of those types. While every programmer has experience with these concepts, Go does some things differently, and subtle differences exist between Go and other languages.

我首先会介绍 Go 语言内置的类型以及如何声明这些类型的变量。虽然每个程序员都接触过这些概念，但 Go 语言在某些方面有所不同，与其他语言之间存在一些细微差别。

The Predeclared Types

预先声明的类型

Go has many types built into the language. These are called *predeclared* types. They are similar to types that are found in other languages: booleans, integers, floats, and strings. Using these types idiomatically is sometimes a challenge for developers who are transitioning from another language. You'll look at these types and see how they work best in Go. Before I review the types, let's cover some of the concepts that apply to all types.

Go 语言内置了许多类型，这些类型被称为预声明类型。它们类似于其他语言中的类型：布尔值、整数、浮点数和字符串。对于从其他语言过渡到 Go 的开发者来说，如何自然地使用这些类型有时会是一个挑战。本文将介绍这些类型，并探讨它们在 Go 中的最佳应用方式。在详细介绍这些类型之前，我们先来了解一些适用于所有类型的基本概念。

The Zero Value

零值

Go, like most modern languages, assigns a default *zero value* to any variable that is declared but not assigned a value. Having an explicit zero value makes code clearer and removes a source of bugs found in C and C++ programs. As I talk about each type, I will also cover the zero value for the type. You can find details on the zero value in [The Go Programming Language Specification](#).

与大多数现代语言一样，Go 语言会将未赋值的变量默认值设为零。显式地使用零值可以使代码更清晰，并消除 C 和 C++ 程序中常见的 bug 来源。在讲解每种类型时，我也会介绍该类型的零值。您可以在《Go 编程语言规范》中找到关于零值的详细信息。

Literals

字面意义

A Go *literal* is an explicitly specified number, character, or string. Go programs have four common kinds of literals. (I'll cover a rare fifth kind of literal when discussing complex numbers.)

Go 字面量是指明确指定的数字、字符或字符串。Go 程序有四种常见的字面量。（在讨论复数时，我会介绍一种不常见的第五种字面量。）

An *integer literal* is a sequence of numbers. Integer literals are base 10 by default, but different prefixes are used to indicate other bases: 0b for binary (base 2), 0o for octal (base 8), or 0x for hexadecimal (base 16). You can use either upper- or lowercase letters for the prefix. A leading 0 with no letter after it is another way to represent an octal literal. Do not use it, as it is very confusing.

整数字面量是一串数字。整数字面量默认以十进制为基数，但使用不同的前缀来表示其他进制：0b 表示二进制（以 2 为基数），0o 表示八进制（以 8 为基数），0x 表示十六进制（以 16 为基数）。前缀可以使用大写字母或小写字母。另一种表示八进制字面量的方式是使用前导零，但这种方式非常容易混淆，因此不建议使用。

To make it easier to read longer integer literals, Go allows you to put underscores in the middle of your literal. This allows you to, for example, group by thousands in base 10 (1_234). These underscores have no effect on the value of the number. The only limitations on underscores are that they can't be at the beginning or end of numbers, and you can't have them next to each other. You could put an underscore between every digit in your literal (1_2_3_4), but don't. Use them to improve readability by breaking

up base 10 numbers at the thousands place or to break up binary, octal, or hexadecimal numbers at 1-, 2-, or 4-byte boundaries.

为了便于阅读较长的整数字面量，Go 允许你在字面量中间添加下划线。例如，你可以用下划线将十进制数按千位分组（1_234）。这些下划线不会影响数字的值。下划线的唯一限制是不能出现在数字的开头或结尾，也不能相邻出现。你可以在字面量的每个数字之间都添加下划线（1_2_3_4），但不要这样做。使用下划线可以提高可读性，例如在千位处分割十进制数，或者在 1 字节、2 字节或 4 字节边界处分割二进制、八进制或十六进制数。

A *floating-point literal* has a decimal point to indicate the fractional portion of the value. They can also have an exponent specified with the letter `e` and a positive or negative number (such as 6.03e23). You also have the option to write them in hexadecimal by using the 0x prefix and the letter `p` for indicating any exponent (0x12.34p5, which is equal to 582.5 in base 10). As integer literals, you can use underscores to format your floating-point literals.

浮点字面量使用小数点来表示数值的小数部分。它们还可以使用字母 `e` 和一个正数或负数来指定指数（例如 6.03e23）。您也可以选择使用十六进制来表示浮点数，方法是使用 0x 前缀和字母 `p` 来表示任意指数（例如 0x12.34p5，等于十进制的 582.5）。与整数字面量一样，您可以使用下划线来格式化浮点字面量。

A *rune literal* represents a character and is surrounded by single quotes.

Unlike many other languages, in Go single quotes and double quotes are *not* interchangeable. Rune literals can be written as single Unicode characters (`'a'`), 8-bit octal numbers (`'\141'`), 8-bit hexadecimal numbers (`'\x61'`), 16-bit hexadecimal numbers (`'\u0061'`), or 32-bit Unicode numbers (`'\U00000061'`). There are also several backslash-escaped rune literals, with the most useful ones being newline (`'\n'`), tab (`'\t'`), single quote (`'\''`), and backslash (`'\\'`).

符文字面量代表一个字符，并用单引号括起来。与许多其他语言不同，在 Go 语言中，单引号和双引号不能互换使用。符文字面量可以写成单个 Unicode 字符（`'a'`）、8 位八进制数（`'\141'`）、8 位十六进制数（`'\x61'`）、16 位十六进制数（`'\u0061'`）或 32 位 Unicode 数（`'\U00000061'`）。此外，还有一些反斜杠转义的符文字面量，其中最常用的是换行符（`'\n'`）、制表符（`'\t'`）、单引号（`'\''`）和反斜杠（`'\\'`）。

Practically speaking, use base 10 to represent your integer and floating-point literals. Octal representations are rare, mostly used to represent POSIX permission flag values (such as `0o777` for `rw-rw-rwx`).

Hexadecimal and binary are sometimes used for bit filters or networking and infrastructure applications. Avoid using any of the numeric escapes for rune literals, unless the context makes your code clearer.

实际上，整数和浮点数字面量最好使用十进制表示。八进制表示法很少见，主要用于表示 POSIX 权限标志值（例如 0o777 表示 rwxrwxrwx）。十六进制和二进制有时用于位过滤器或网络和基础设施应用。除非上下文能使代码更清晰，否则请避免对符文字面量使用任何数字转义符。

There are two ways to indicate *string literals*. Most of the time, you should use double quotes to create an *interpreted string literal* (e.g., type **"Greetings and Salutations"**). These contain zero or more rune literals. They are called “interpreted” because they interpret rune literals (both numeric and backslash escaped) into single characters.

有两种方法可以表示字符串字面量。大多数情况下，应该使用双引号来创建解释型字符串字面量（例如，输入 **"Greetings and Salutations"**）。这些字面量包含零个或多个符文字面量。之所以称为“解释型”，是因为它们会将符文字面量（包括数字和反斜杠转义的符文）解释为单个字符。

NOTE

笔记

One rune literal backslash escape is not legal in a string literal: the single quote escape. It is replaced by a backslash escape for double quotes.

字符串字面量中有一个转义字符是不允许的：单引号转义。它被替换为双引号转义字符。

The only characters that cannot appear in an interpreted string literal are unescaped backslashes, unescaped newlines, and unescaped double quotes. If you use an interpreted string literal and want your greetings on a different line from your salutations and want “Salutations” to appear in quotes, you need to type `"Greetings and\n\"Salutations\""`.

唯一不能出现在解释型字符串字面量中的字符是未转义的反斜杠、未转义的换行符和未转义的双引号。如果您使用解释型字符串字面量，并且希望问候语和致意语分行显示，并且希望“致意语”用引号括起来，则需要输入 `"Greetings and\n\"Salutations\""`。

If you need to include backslashes, double quotes, or newlines in your string, using a *raw string literal* is easier. These are delimited with backquotes (```) and can contain any character except a backquote. There's no escape character in a raw string literal; all characters are included as is. When using a raw string literal, you write a multiline greeting like so:

如果需要在字符串中包含反斜杠、双引号或换行符，使用原始字符串字面量会更方便。原始字符串字面量用反引号分隔（例如 ``` ），可以包含除反引号之外的任何字符。原始字符串字面量中没有转义字符；所有字符都按原样包含。使用原始字符串字面量时，您可以像这样编写多行问候语：

```
`Greetings and
```

"Salutations"

Literals are considered *untyped*. I'll explore this concept more in [“Literals Are Untyped”](#). As you will see in [“var Versus :=”](#), there are situations in Go where the type isn't explicitly declared. In those cases, Go uses the *default type* for a literal; if there's nothing in the expression that makes clear what the type of the literal is, the literal defaults to a type. I will mention the default type for literals when discussing the different predeclared types.

字面量被认为是无类型的。我将在“字面量是无类型的”一节中更详细地探讨这个概念。正如你将在“var 与 := 的区别”一节中看到的那样，在 Go 语言中，有些情况下类型没有被显式声明。在这些情况下，Go 会使用字面量的默认类型；如果表达式中没有任何内容明确地表明字面量的类型，则字面量会使用默认类型。我将在讨论不同的预声明类型时提及字面量的默认类型。

Booleans

布尔值

The `bool` type represents Boolean variables. Variables of `bool` type can have one of two values: `true` or `false`. The zero value for a `bool` is `false`:

`bool` 类型表示布尔变量。 `bool` 类型的变量可以有两个值： `true` 或 `false`。 `bool` 的零值为 `false`。

```
var flag bool // no value assigned, set to false
var isAwesome = true
```

It's hard to talk about variable types without showing a variable declaration, and vice versa. I'll use variable declarations first and describe them in “`var` Versus `:=`”.

如果不展示变量声明，就很难讨论变量类型，反之亦然。我将首先使用变量声明，并在“`var` 与 `:=`”部分进行描述。

Numeric Types

数字类型

Go has a large number of numeric types: 12 types (and a few special names) that are grouped into three categories. If you are coming from a language like JavaScript that gets along with only a single numeric type, this might seem like a lot. And in fact, some types are used frequently while others are more esoteric. I'll start by looking at integer types before moving on to floating-point types and the very unusual complex type.

Go 语言拥有大量的数值类型：12 种类型（以及一些特殊名称），它们被分为三类。如果你之前使用的是像 JavaScript 这样只有一种数值类型的语言，那么 Go 的数值类型数量可能会让你觉得很多。事实上，有些类型使用频繁，而另一些则比较少见。我将首先介绍整数类型，然后再讲解浮点类型和非常特殊的复数类型。

Integer types

整数类型

Go provides both signed and unsigned integers in a variety of sizes, from one to eight bytes. They are shown in [Table 2-1](#).

Go 语言提供了有符号整数和无符号整数，大小从 1 字节到 8 字节不等。它们如表 2-1 所示。

Table 2-1. The integer types in Go
表 2-1. Go 语言中的整数类型

Type name 类型名称	Value range 值范围
int8 int8	-128 to 127 -128 至 127
int16 int16	-32768 to 32767 -32768 至 32767
int32 int32	-2147483648 to 2147483647 -2147483648 至 2147483647
int64 int64	-9223372036854775808 to 9223372036854775807 -9223372036854775808 至 9223372036854775807
uint8 uint8	0 to 255 0 到 255
uint16 uint16	0 to 65535 0 到 65535
uint32 uint32	0 to 4294967295 0 到 4294967295

Type name	Value range
类型名称	值范围
uint64	0 to 18446744073709551615
uint64	0 至 18446744073709551615

It might be obvious from the name, but the zero value for all of the integer types is `0`.

从名称上可能很明显，所有整数类型的零值都是 `0`。

The special integer types

特殊整数类型

Go does have some special names for integer types. A `byte` is an alias for `uint8`; it is legal to assign, compare, or perform mathematical operations between a `byte` and a `uint8`. However, you rarely see `uint8` used in Go code; just call it a `byte`.

Go 语言确实有一些特殊的整数类型名称。例如，`byte` 是 `uint8` 的别名；对 `byte` 和 `uint8` 进行赋值、比较或数学运算都是合法的。但是，在 Go 代码中很少见到 `uint8` 的使用；通常直接称之为 `byte` 即可。

The second special name is `int`. On a 32-bit CPU, `int` is a 32-bit signed integer like an `int32`. On most 64-bit CPUs, `int` is a 64-bit signed integer, just like an `int64`. Because `int` isn't consistent from platform to platform, it is a compile-time error to assign, compare, or perform mathematical operations between an `int` and an `int32` or `int64` without a type conversion (see [“Explicit Type Conversion”](#) for more details). Integer literals default to being of `int` type.

第二个特殊名称是 `int`。在 32 位 CPU 上，`int` 是一个 32 位有符号整数，类似于 `int32`。在大多数 64 位 CPU 上，`int` 是一个 64 位有符号整数，就像 `int64` 一样。由于 `int` 在不同平台上不一致，因此在未进行类型转换的情况下，对 `int` 与 `int32` 或 `int64` 进行赋值、比较或执行数学运算会导致编译时错误（更多详情请参阅“显式类型转换”）。整数字面量默认为 `int` 类型。

NOTE

笔记

Some uncommon 64-bit CPU architectures use a 32-bit signed integer for the `int` type. Go supports three of them: `amd64p32`, `mips64p32`, and `mips64p32le`.

一些不常见的 64 位 CPU 架构使用 32 位有符号整数作为 `int` 类型。Go 支持其中三种：`amd64p32`、`mips64p32` 和 `mips64p32le`。

The third special name is `uint`. It follows the same rules as `int`, only it is unsigned (the values are always 0 or positive).

第三个特殊名称是 `uint`。它遵循与 `int` 相同的规则，只是它是无符号的（值始终为 0 或正数）。

There are two other special names for integer types, `rune` and `uintptr`. You looked at rune literals earlier and I'll discuss the `rune` type in [“A Taste of Strings and Runes”](#) and `uintptr` in [Chapter 16](#).

整数类型还有两个特殊名称，分别是 `rune` 和 `uintptr`。你之前已经看过符文字面量，我将在“字符串和符文入门”中讨论 `rune` 类型，在第 16 章讨论 `uintptr` 类型。

Choosing which integer to use

选择要使用的整数

Go provides more integer types than some other languages. Given all these choices, you might wonder when you should use each of them. You should follow three simple rules:

Go 语言提供的整数类型比其他一些语言更多。面对如此多的选择，你可能会想知道何时应该使用哪一种。你应该遵循以下三个简单的规则：

- If you are working with a binary file format or network protocol that has an integer of a specific size or sign, use the corresponding integer type.

如果您正在使用具有特定大小或符号的整数的二进制文件格式或网络协议，请使用相应的整数类型。

- If you are writing a library function that should work with any integer type, take advantage of Go's generics support and use a generic type parameter to represent any integer type (I talk more about functions and their parameters in [Chapter 5](#) and more about generics in [Chapter 8](#).)

如果您正在编写一个应该能处理任何整数类型的库函数，请利用 Go 的泛型支持，并使用泛型类型参数来表示任何整数类型（我将在第 5 章中更详细地讨论函数及其参数，在第 8 章中更详细地讨论泛型）。

- In all other cases, just use `int` .
其他情况下，只需使用 `int` 。

NOTE

笔记

You'll likely find legacy code where there's a pair of functions that do the same thing, but one has `int64` for the parameters and variables and the other has `uint64`. The reason is that the API was created before generics were added to Go. Without generics, you needed to write functions with slightly different names to implement the same algorithm with different types. Using `int64` and `uint64` meant that you could write the code once and let callers use type conversions to pass values in and convert data that's returned.

你很可能发现一些遗留代码，其中有两个函数功能相同，但一个函数的参数和变量使用 `int64`，另一个使用 `uint64`。这是因为该 API 是在 Go 引入泛型之前创建的。如果没有泛型，你需要编写名称略有不同的函数来实现相同算法的不同类型。使用 `int64` 和 `uint64` 意味着你只需编写一次代码，调用者就可以使用类型转换来传递值并转换返回的数据。

You can see this pattern in the Go standard library within the functions `FormatInt` and `FormatUint` in the `strconv` package.

您可以在 Go 标准库的 `FormatInt` 和 `FormatUint` 函数中看到这种模式，这些函数位于 `strconv` 包中。

Integer operators

整数运算符

Go integers support the usual arithmetic operators: `+`, `-`, `*`, `/`, with `%` for modulus. The result of an integer division is an integer; if you want to get a floating-point result, you need to use a type conversion to make your integers into floating-point numbers. Also, be careful not to divide an

integer by 0; this causes a panic (I talk more about panics in [“panic and recover”](#)).

Go 语言的整数支持常见的算术运算符：+、-、*、/，以及用于取模的 %。整数除法的结果仍然是整数；如果需要浮点数结果，则需要使用类型转换将整数转换为浮点数。此外，请注意不要将整数除以 0；这会导致程序崩溃（关于崩溃的更多内容，请参阅[“panic and recover”](#)）。

NOTE

笔记

Integer division in Go follows truncation toward zero; see the Go spec’s section on [arithmetic operators](#) for the full details.

Go 语言中的整数除法遵循向零截断的原则；有关完整详细信息，请参阅 Go 规范中关于算术运算符的部分。

You can combine any of the arithmetic operators with = to modify a variable: +=, -=, *=, /=, and %= . For example, the following code results in x having the value 20:

您可以将任意算术运算符与 = 结合使用来修改变量：+=、-=、*=、/= 和 % = 。例如，以下代码会将 x 的值为 20：

```
var x int = 10
x *= 2
```

You compare integers with `==`, `!=`, `>`, `>=`, `<`, and `<=`.

您可以将整数与 `==`、`!=`、`>`、`>=`、`<` 和 `<=` 进行比较。

Go also has bit-manipulation operators for integers. You can bit shift left and right with `<<` and `>>`, or do bit masks with `&` (bitwise AND), `|` (bitwise OR), `^` (bitwise XOR), and `&^` (bitwise AND NOT). As with the arithmetic operators, you can also combine all the bitwise operators with `=` to modify a variable: `&=`, `|=`, `^=`, `&^=`, `<<=`, and `>>=`.

Go 语言也提供了整数的位操作运算符。你可以使用 `<<` 和 `>>` 进行位移操作，或者使用 `&`（按位与）、`|`（按位或）、`^`（按位异或）和 `&^`（按位非）进行位掩码运算。与算术运算符类似，你也可以使用 `=` 组合所有位运算符来修改变量：`&=`、`|=`、`^=`、`&^=`、`<<=` 和 `>>=`。

Floating-point types

浮点类型

Go has two floating-point types, as shown in [Table 2-2](#).

Go 有两种浮点类型，如表 2-2 所示。

Table 2-2. The floating-point types in Go
表 2-2. Go 语言中的浮点类型

Type name	Largest absolute value
类型名称	最大绝对值
float32	3.40282346638528859811704183484516925440e+38
float32	3.40282346638528859811704183484516925440e+38
float64	1.797693134862315708145274237317043567981e+308
float64	1.797693134862315708145274237317043567981e+308

Like the integer types, the zero value for the floating-point types is 0.

与整数类型一样，浮点类型的零值也是 0。

Floating point in Go is similar to floating-point math in other languages. Go uses the IEEE 754 specification, giving a large range and limited precision. Picking which floating-point type to use is straightforward: unless you have to be compatible with an existing format, use `float64`. Floating-point literals have a default type of `float64`, so always using `float64` is the

simplest option. It also helps mitigate floating-point accuracy issues since a `float32` has only six- or seven-decimal digits of precision. Don't worry about the difference in memory size unless you have used the profiler to determine that it is a significant source of problems. (Testing and profiling are covered in [Chapter 15](#).)

Go 语言中的浮点运算与其他语言中的浮点运算类似。Go 使用 IEEE 754 规范，该规范提供了较大的取值范围和有限的精度。选择使用哪种浮点类型非常简单：除非需要兼容现有格式，否则请使用 `float64`。浮点字面量的默认类型为 `float64`，因此始终使用 `float64` 是最简单的选择。这也有助于缓解浮点精度问题，因为 `float32` 只有六位或七位小数精度。除非您使用性能分析器确定内存大小的差异是造成问题的重要原因，否则无需担心内存大小的差异。（测试和性能分析将在第 15 章中介绍。）

The bigger question is whether you should be using a floating-point number at all. In many cases, the answer is no. Just like other languages, Go floating-point numbers have a huge range, but they cannot store every value in that range; they store the nearest approximation. Because floats aren't exact, they can be used only in situations where inexact values are acceptable or the rules of floating point are well understood. That limits them to things like graphics, statistics, and scientific operations.

更重要的问题是，你是否应该使用浮点数。在很多情况下，答案是否定的。与其他语言一样，Go 语言的浮点数范围非常大，但它们无法存

储该范围内的所有值；它们只能存储最接近的近似值。由于浮点数并不精确，因此它们只能用于可以接受不精确值或对浮点数规则非常了解的场景。这使得它们主要应用于图形、统计和科学运算等领域。

WARNING

警告

A floating-point number cannot represent a decimal value exactly. Do not use them to represent money or any other value that must have an exact decimal representation! You'll look at a third-party module for handling exact decimal values in [“Importing Third-Party Code”](#).

浮点数无法精确表示十进制值。请勿使用浮点数表示货币或其他任何必须精确表示为十进制值的数值！您将在“导入第三方代码”部分了解如何使用第三方模块来处理精确的十进制值。

IEEE 754

IEEE 754

As mentioned earlier, Go (and most other programming languages) stores floating-point numbers using a specification called IEEE 754.

如前所述，Go（以及大多数其他编程语言）使用名为 IEEE 754 的规范来存储浮点数。

The actual rules are outside the scope of this book, and they aren't straightforward. You can learn more about IEEE 754 from [The Floating Point Guide](#).

具体规则超出了本书的讨论范围，而且也并不简单。您可以从《浮点运算指南》中了解更多关于 IEEE 754 的信息。

You can use all the standard mathematical and comparison operators with floats, except `%`. Floating-point division has a couple of interesting properties. Dividing a nonzero floating-point variable by 0 returns `+Inf` or `-Inf` (positive or negative infinity), depending on the sign of the number. Dividing a floating-point variable set to 0 by 0 returns `NaN` (Not a Number).

除了 `%` 之外，您可以使用所有标准的数学运算符和比较运算符来处理浮点数。浮点数除法有一些有趣的特性。将一个非零浮点变量除以

0 会返回 `+Inf` 或 `-Inf`（正无穷或负无穷），具体取决于数字的符号。将一个值为 0 的浮点变量除以 0 会返回 `NaN`（非数字）。

While Go lets you use `==` and `!=` to compare floats, don't do it. Because of the inexact nature of floats, two floating-point values might not be equal when you think they should be. Instead, define a maximum allowed variance and see if the difference between two floats is less than that. This value (sometimes called *epsilon*) depends on your accuracy needs; I can't give you a simple rule. If you aren't sure, consult your friendly local mathematician for advice. If you can't find one, [The Floating Point Guide has a “Comparison” page](#) that can help you out (or possibly convince you to avoid floating-point numbers unless absolutely necessary).

虽然 Go 语言允许你使用 `==` 和 `!=` 来比较浮点数，但最好不要这样做。由于浮点数本身的不精确性，两个浮点值可能并不相等，即使你认为它们应该相等。正确的做法是，定义一个最大允许偏差值，然后比较两个浮点数之间的差异是否小于这个值。这个值（有时称为 *epsilon*）取决于你的精度需求；我无法提供一个简单的规则。如果你不确定，可以咨询你身边的数学专家。如果找不到合适的专家，《浮点数指南》的“比较”页面或许能帮到你（或者让你明白，除非绝对必要，否则最好避免使用浮点数）。

Complex types (you're probably not going to use

these)

复杂类型（您可能用不到这些）

There is one more numeric type and it is pretty unusual. Go has first-class support for complex numbers. If you don't know what complex numbers are, you are not the target audience for this feature; feel free to skip ahead.

还有一种数值类型，它相当不常见。Go 语言对复数提供了完善的支持。如果您不了解复数，那么本文的目标读者可能并不了解复数，您可以直接跳过这部分。

There isn't a lot to the complex number support in Go. Go defines two complex number types. `complex64` uses `float32` values to represent the real and imaginary part, and `complex128` uses `float64` values. Both are declared with the `complex` built-in function:

Go 语言对复数的支持并不多。它定义了两种复数类型：`complex64` 使用 `float32` 个值来表示实部和虚部，而 `complex128` 使用 `float64` 个值。两者都使用 `complex` 内置函数声明。

```
var complexNum = complex(20.3, 10.2)
```

Go uses a few rules to determine the type of the value returned by `complex`:

Go 使用一些规则来确定 `complex` 返回的值的类型：

- If you use untyped constants or literals for both function parameters, you'll create an untyped complex literal, which has a default type of `complex128`.

如果对函数的两个参数都使用无类型常量或字面量，则会创建一个无类型复杂字面量，其默认类型为 `complex128`。

- If both values passed into `complex` are of `float32` type, you'll create a `complex64`.

如果传递给 `complex` 的两个值都是 `float32` 类型，则会创建一个 `complex64`。

- If one value is a `float32` and the other value is an untyped constant or literal that can fit within a `float32`, you'll create a `complex64`.

如果一个值是 `float32`，另一个值是可以放入 `float32` 中的未类型化常量或字面量，则会创建一个 `complex64`。

- Otherwise, you'll create a `complex128`.

否则，您将创建一个 `complex128`。

All the standard floating-point arithmetic operators work on complex numbers. Just as with floats, you can use `==` or `!=` to compare them, but they have the same precision limitations, so it's best to use the epsilon technique. You can extract the real and imaginary portions of a complex number with the `real` and `imag` built-in functions, respectively. The `math/cmplx` package has additional functions for manipulating `complex128` values.

所有标准的浮点运算运算符都适用于复数。与浮点数一样，您可以使用 `==` 或 `!=` 来比较复数，但它们的精度限制相同，因此最好使用 epsilon 技术。您可以使用 `real` 和 `imag` 内置函数分别提取复数的实部和虚部。`math/cmplx` 包提供了用于操作 `complex128` 值的其他函数。

The zero value for both types of complex numbers has 0 assigned to both the real and imaginary portions of the number.

两种类型的复数的零值都是实部和虚部都为 0。

[Example 2-1](#) shows a simple program that demonstrates how complex numbers work. You can run it for yourself on [The Go Playground](#) or in the `sample_code/complex_numbers` directory in the [Chapter 2 repository](#).

例 2-1 展示了一个简单的程序，演示了复数的运算方式。您可以在 Go Playground 或第 2 章代码库的 `sample_code/complex_numbers` 目录中运行该程序。

Example 2-1. Complex numbers

例 2-1. 复数

```
func main() {  
    x := complex(2.5, 3.1)  
    y := complex(10.2, 2)  
    fmt.Println(x + y)  
}
```

```
    fmt.Println(x - y)
    fmt.Println(x * y)
    fmt.Println(x / y)
    fmt.Println(real(x))
    fmt.Println(imag(x))
    fmt.Println(cmplx.Abs(x))
}
```

Running this code gives you the following:

运行这段代码会得到以下结果：

```
(12.7+5.1i)
(-7.699999999999999+1.1i)
(19.3+36.62i)
(0.2934098482043688+0.24639022584228065i)
2.5
3.1
3.982461550347975
```

You can see floating-point imprecision on display here too.

这里也能看到浮点运算的不精确性。

In case you were wondering what the fifth kind of primitive literal was, Go supports imaginary literals to represent the imaginary portion of a complex

number. They look just like floating-point literals, but they have an `i` for a suffix.

如果你想知道第五种原始字面量是什么，Go 支持虚数字面量来表示复数的虚部。它们看起来就像浮点字面量，但它们的后缀是 `i`。

Despite having complex numbers as a predeclared type, Go is not a popular language for numerical computing. Adoption has been limited because other features (like matrix support) are not part of the language and libraries have to use inefficient replacements, like slices of slices. (You'll look at slices in [Chapter 3](#) and how they are implemented in [Chapter 6](#).) But if you need to calculate a Mandelbrot set as part of a larger program, or implement a quadratic equation solver, complex number support is there for you.

尽管 Go 预定义了复数类型，但它并非数值计算领域的热门语言。其普及程度受限的原因在于，其他一些特性（例如矩阵支持）并非语言本身固有的，库不得不使用效率低下的替代方案，例如切片的切片。

（您将在第 3 章了解切片，并在第 6 章学习其实现方式。）但是，如果您需要在大型程序中计算曼德勃罗集，或者实现二次方程求解器，那么 Go 的复数支持功能就能满足您的需求。

You might be wondering why Go includes complex numbers. The answer is simple: Ken Thompson, one of the creators of Go (and Unix), thought they would be [interesting](#). There has been discussion about [removing complex](#)

[numbers](#) from a future version of Go, but it's easier to just ignore the feature.

你可能想知道为什么 Go 语言会包含复数。答案很简单：Go（以及 Unix）的创建者之一 Ken Thompson 认为复数会很有趣。虽然有人讨论过在未来的 Go 版本中移除复数，但最简单的办法就是忽略它。

NOTE

笔记

If you do want to write numerical computing applications in Go, you can use the third-party [Gonum](#) package. It takes advantage of complex numbers and provides useful libraries for things like linear algebra, matrices, integration, and statistics. But you should consider other languages first.

如果你确实想用 Go 编写数值计算应用程序，可以使用第三方包 Gonum。它利用了复数，并提供了用于线性代数、矩阵、积分和统计等方面的实用库。但你应该首先考虑其他语言。

A Taste of Strings and Runes

弦乐与符文的滋味

This brings us to strings. Like most modern languages, Go includes strings as a built-in type. The zero value for a string is the empty string. Go supports Unicode; as I showed [“Literals”](#), you can put any Unicode character into a string. Like integers and floats, strings are compared for equality using `==`, difference with `!=`, or ordering with `>`, `>=`, `<`, or `<=`. They are concatenated by using the `+` operator.

这就引出了字符串。和大多数现代语言一样，Go 语言内置了字符串类型。字符串的零值是空字符串。Go 支持 Unicode；正如我在“字面量”部分展示的，你可以将任何 Unicode 字符放入字符串中。与整数和浮点数类似，字符串之间也使用 `==` 来比较相等性，使用 `!=` 来比较差异性，或者使用 `>`、`>=`、`<` 或 `<=` 来比较排序。字符串之间使用 `+` 运算符进行连接。

Strings in Go are immutable; you can reassign the value of a string variable, but you cannot change the value of the string that is assigned to it.

Go 语言中的字符串是不可变的；你可以重新赋值给一个字符串变量，但你不能改变赋值给它的字符串的值。

Go also has a type that represents a single code point. The *rune* type is an alias for the `int32` type, just as `byte` is an alias for `uint8`. As you could probably guess, a rune literal's default type is a rune, and a string literal's default type is a string.

Go 语言也有一种类型来表示单个代码点。rune 类型是 `int32` 类型的别名，就像 `byte` 是 `uint8` 的别名一样。正如你可能已经猜到的，rune 字面量的默认类型是 rune，而 string 字面量的默认类型是 string。

If you are referring to a character, use the rune type, not the `int32` type. They might be the same to the compiler, but you want to use the type that clarifies the intent of your code:

如果你指的是一个角色，请使用符文类型，而不是 `int32` 类型。虽然编译器可能认为它们相同，但你应该使用能够更清晰地表达代码意图的类型：

```
var myFirstInitial rune = 'J' // good - the type
name matches the usage
var myLastInitial int32 = 'B' // bad - legal but
confusing
```

I am going to talk a lot more about strings in the next chapter, covering some implementation details, relationships with bytes and runes, as well as advanced features and pitfalls.

下一章我将更详细地讨论字符串，包括一些实现细节、与字节和符文的关系，以及高级特性和陷阱。

Explicit Type Conversion

显式类型转换

Most languages that have multiple numeric types automatically convert from one to another when needed. This is called *automatic type promotion*, and while it seems very convenient, it turns out that the rules to properly convert one type to another can get complicated and produce unexpected results. As a language that values clarity of intent and readability, Go

doesn't allow automatic type promotion between variables. You must use a *type conversion* when variable types do not match. Even different-sized integers and floats must be converted to the same type to interact. This makes it clear exactly what type you want without having to memorize any type conversion rules (see [Example 2-2](#)).

大多数支持多种数值类型的语言会在需要时自动进行类型转换。这被称为自动类型提升，虽然看起来很方便，但实际上，类型转换的规则可能非常复杂，并产生意想不到的结果。Go 语言重视意图的清晰性和可读性，因此不允许变量之间进行自动类型提升。当变量类型不匹配时，必须使用类型转换。即使是不同大小的整数和浮点数，也必须转换为相同的类型才能进行交互。这样可以清晰地表达你想要的类型，而无需记住任何类型转换规则（参见示例 2-2）。

Example 2-2. Type conversions

例 2-2. 类型转换

```
var x int = 10
var y float64 = 30.2
var sum1 float64 = float64(x) + y
var sum2 int = x + int(y)
fmt.Println(sum1, sum2)
```

In this sample code, you define four variables. `x` is an `int` with the value 10, and `y` is a `float64` with the value 30.2. Since these are not identical

types, you need to convert them to add them together. For `sum1`, you convert `x` to a `float64` using a `float64` type conversion, and for `sum2`, you convert `y` to an `int` using an `int` type conversion. When you run this code, it prints out 40.2 40.

在这段示例代码中，您定义了四个变量。`x` 是一个值为 10 的 `int`，`y` 是一个值为 30.2 的 `float64`。由于它们的类型不同，您需要进行类型转换才能将它们相加。对于 `sum1`，您需要使用 `float64` 类型转换将 `x` 转换为 `float64`；对于 `sum2`，您需要使用 `int` 类型转换将 `y` 转换为 `int`。运行这段代码后，它会输出 40.2 40。

The same behavior applies with different-sized integer types (see [Example 2-3](#)).

同样的行为也适用于不同大小的整数类型（参见示例 2-3）。

Example 2-3. Integer type conversions

例 2-3. 整数类型转换

```
var x int = 10
var b byte = 100
var sum3 int = x + int(b)
var sum4 byte = byte(x) + b
fmt.Println(sum3, sum4)
```

You can run these examples on [The Go Playground](#) or in the `sample_code/type_conversion` directory in the [Chapter 2 repository](#).

您可以在 Go Playground 或第 2 章存储库中的 `sample_code/type_conversion` 目录中运行这些示例。

This strictness around types has other implications. Since all type conversions in Go are explicit, you cannot treat another Go type as a boolean. In many languages, a nonzero number or a nonempty string can be interpreted as a boolean `true`. Just like automatic type promotion, the rules for “truthy” values vary from language to language and can be confusing. Unsurprisingly, Go doesn’t allow truthiness. In fact, *no other type can be converted to a bool, implicitly or explicitly*. If you want to convert from another data type to boolean, you must use one of the comparison operators (`==`, `!=`, `>`, `<`, `<=`, or `>=`). For example, to check if variable `x` is equal to 0, the code would be `x == 0`. If you want to check if string `s` is empty, use `s == ""`.

这种对类型的严格要求还有其他影响。由于 Go 中的所有类型转换都是显式的，因此你不能将其他 Go 类型视为布尔值。在许多语言中，非零数字或非空字符串可以被解释为布尔值 `true`。就像自动类型提升一样，“真值”的规则因语言而异，可能会令人困惑。不出所料，Go 不允许真值。事实上，任何其他类型都不能隐式或显式地转换为布尔值。如果要将其他数据类型转换为布尔值，必须使用比较运算符（`==`、`!=`、`>`、`<`、`<=` 或 `>=`）。例如，要检查变量 `x` 是否等

于 0，代码应该是 `x == 0`。如果要检查字符串 `s` 是否为空，请使用 `s == ""`。

NOTE

笔记

Type conversions are one of the places where Go chooses to add a little verbosity in exchange for a great deal of simplicity and clarity. You'll see this trade-off multiple times. Idiomatic Go values comprehensibility over conciseness.

类型转换是 Go 语言为了追求简洁性和清晰度而牺牲一些代码长度的地方之一。你会发现这种权衡在很多情况下都会出现。Go 语言的惯用写法更注重可理解性而非简洁性。

Literals Are Untyped

字面量未指定类型

While you can't add two integer variables together if they are declared to be of different types of integers, Go lets you use an integer literal in floating-point expressions or even assign an integer literal to a floating-point variable:

虽然不能将两个声明为不同类型的整数变量相加，但 Go 允许你在浮点表达式中使用整数字面量，甚至可以将整数字面量赋值给浮点变量：

```
var x float64 = 10
var y float64 = 200.3 * 5
```

This is because, as I mentioned earlier, literals in Go are untyped. Go is a practical language, and it makes sense to avoid forcing a type until the developer specifies one. This means they can be used with any variable whose type is compatible with the literal. When you look at user-defined types in [Chapter 7](#), you'll see that you can even use literals with user-defined types based on predefined types. Being untyped goes only so far; you can't assign a literal string to a variable with a numeric type or a literal number to a string variable, nor can you assign a float literal to an `int`. These are all flagged by the compiler as errors. Size limitations also exist; while you can write numeric literals that are larger than any integer can hold, it is a compile-time error to try to assign a literal whose value overflows the specified variable, such as trying to assign the literal 1000 to a variable of type `byte`.

这是因为，正如我之前提到的，Go 语言中的字面量是无类型的。Go 是一种实用语言，在开发者指定类型之前避免强制指定类型是合理的。这意味着字面量可以与任何类型兼容的变量一起使用。当你查看第 7 章的用户自定义类型时，你会发现你甚至可以将字面量与基于预定义类型的用户自定义类型一起使用。无类型也有一定的限制；你不能将字符串字面量赋值给数值类型的变量，也不能将数字字面量赋值给字符串变量，更不能将浮点数字字面量赋值给 `int` 类型的变量。这

些都会被编译器标记为错误。此外，还存在大小限制；虽然你可以编写大于任何整数容量的数值字面量，但尝试将值超出指定变量的字面量赋值（例如，尝试将字面量 1000 赋值给 `byte` 类型的变量）会导致编译时错误。

var Versus :=

var Versus :=

For a small language, Go has a lot of ways to declare variables. There's a reason for this: each declaration style communicates something about how the variable is used. Let's go through the ways you can declare a variable in Go and see when each is appropriate.

对于一门小语言来说，Go 提供了多种变量声明方式。这是有原因的：每种声明方式都传达了变量的用途。让我们一起来看看 Go 中声明变量的各种方式，以及每种方式的适用场景。

The most verbose way to declare a variable in Go uses the `var` keyword, an explicit type, and an assignment. It looks like this:

在 Go 语言中，声明变量最繁琐的方式是使用 `var` 关键字、显式类型和赋值语句。它看起来像这样：


```
var x int = 10
```

If the type on the righthand side of the `=` is the expected type of your variable, you can leave off the type from the left side of the `=`. Since the default type of an integer literal is `int`, the following declares `x` to be a variable of type `int`:

如果 `=` 右侧的类型是预期的变量类型，则可以省略 `=` 左侧的类型。由于整数字面量的默认类型是 `int`，因此以下代码将 `x` 声明为类型为 `int` 的变量：

```
var x = 10
```

Conversely, if you want to declare a variable and assign it the zero value, you can keep the type and drop the `=` on the righthand side:

反之，如果您想声明一个变量并将其赋值为零，您可以保留类型并删除右侧的 `=`：

```
var x int
```

You can declare multiple variables at once with `var`, and they can be of the same type:

您可以使用 `var` 一次声明多个变量，并且它们可以是同一类型：

```
var x, y int = 10, 20
```

You can declare all zero values of the same type:

您可以声明所有相同类型的值为零：

```
var x, y int
```

or of different types:

或者不同类型的：

```
var x, y = 10, "hello"
```

There's one more way to use `var`. If you are declaring multiple variables at once, you can wrap them in a *declaration list*:

还有一种使用 `var` 的方法。如果您一次声明多个变量，可以将它们放在声明列表中：

```
var (  
    x    int  
    y          = 20  
    z    int = 30  
    d, e      = 40, "hello"
```

```
f, g string
)
```

Go also supports a short declaration and assignment format. When you are within a function, you can use the `:=` operator to replace a `var` declaration that uses type inference. The following two statements do exactly the same thing—they declare `x` to be an `int` with the value of 10:

Go 也支持简短的声明和赋值格式。在函数内部，可以使用 `:=` 运算符来替换使用类型推断的 `var` 声明。以下两条语句的作用完全相同——它们将 `x` 声明为一个值为 10 的 `int`：

```
var x = 10
x := 10
```

As with `var`, you can declare multiple variables at once using `:=`. These two lines both assign 10 to `x` and “hello” to `y`:

与 `var` 一样，您可以使用 `:=` 一次声明多个变量。以下两行代码分别将 10 赋值给 `x`，并将“hello”赋值给 `y`：

```
var x, y = 10, "hello"
x, y := 10, "hello"
```

The `:=` operator can do one trick that you cannot do with `var`: it allows you to assign values to existing variables too. As long as at least one new variable is on the lefthand side of the `:=`, any of the other variables can already exist:

`:=` 运算符可以做到 `var` 做不到的一点：它允许你给现有变量赋值。只要 `:=` 左侧至少有一个新变量，其他任何变量都可以已经存在：

```
x := 10
x, y := 30, "hello"
```

Using `:=` has one limitation. If you are declaring a variable at the package level, you must use `var` because `:=` is not legal outside of functions.

使用 `:=` 有一个限制。如果您在包级别声明变量，则必须使用 `var`，因为 `:=` 在函数之外是不合法的。

How do you know which style to use? As always, choose what makes your intent clearest. The most common declaration style within functions is `:=`. Outside of a function, use declaration lists on the rare occasions when you are declaring multiple package-level variables.

如何确定使用哪种声明风格？和往常一样，选择最能清晰表达你意图的风格。函数内部最常见的声明风格是 `:=`。在函数外部，只有在极少数情况下需要声明多个包级变量时，才使用声明列表。

In some situations within functions, you should avoid `:=`:

在某些情况下，函数中应避免使用 `:=`：

- When initializing a variable to its zero value, use `var x int`. This makes it clear that the zero value is intended.

将变量初始化为零值时，请使用 `var x int`。这样可以明确表明该值为零。

- When assigning an untyped constant or a literal to a variable and the default type for the constant or literal isn't the type you want for the variable, use the long `var` form with the type specified. While it is legal to use a type conversion to specify the type of the value and use `:=` to write `x := byte(20)`, it is idiomatic to write `var x byte = 20`.

当将一个无类型常量或字面量赋值给一个变量，且该常量或字面量的默认类型与你想要的变量类型不符时，请使用长格式 `var`，并指定类型。虽然可以使用类型转换来指定值的类型，例如使用 `:=` 来写 `x := byte(20)`，但更规范的写法是 `var x byte = 20`。

- Because `:=` allows you to assign to both new and existing variables, it sometimes creates new variables when you think you are reusing existing ones (see [“Shadowing Variables”](#) for details). In those situations, explicitly declare all your new variables with `var` to make it clear which variables are new, and then use the assignment operator `(=)` to assign values to both new and old variables.

由于 `:=` 允许您同时为新变量和现有变量赋值，因此有时当您认为正在重用现有变量时，它会创建新变量（详情请参阅“变量阴影”）。在这种情况下，请使用 `var` 显式声明所有新变量，以明确哪些变量是新变量，然后使用赋值运算符 (`=`) 为新旧变量赋值。

While `var` and `:=` allow you to declare multiple variables on the same line, use this style only when assigning multiple values returned from a function or the comma ok idiom (see [Chapter 5](#) and [“The comma ok Idiom”](#)).

虽然 `var` 和 `:=` 允许你在同一行声明多个变量，但只有在赋值从函数返回的多个值或使用逗号 ok 惯用法时才使用此样式（参见第 5 章和“逗号 ok 惯用法”）。

You should rarely declare variables outside of functions, in what’s called the *package block* (see [“Blocks”](#)). Package-level variables whose values change are a bad idea. When you have a variable outside of a function, it can be difficult to track the changes made to it, which makes it hard to understand how data is flowing through your program. This can lead to subtle bugs. As a general rule, you should only declare variables in the package block that are effectively immutable.

你应该尽量避免在函数之外，也就是所谓的包代码块（参见“代码块”）中声明变量。包级别的变量如果其值会发生变化，那就非常不明

智。当变量位于函数之外时，很难追踪其变化，这会导致难以理解数据在程序中的流动方式，进而可能引发一些不易察觉的错误。一般来说，你只应该在包代码块中声明那些实际上不可变的变量。

TIP

提示

Avoid declaring variables outside of functions because they complicate data flow analysis.

避免在函数外部声明变量，因为这会使数据流分析变得复杂。

You might be wondering: does Go provide a way to *ensure* that a value is immutable? It does, but it is a bit different from what you may have seen in other programming languages. It's time to learn about `const`.

你可能想知道：Go 是否提供了确保值不可变的方法？答案是肯定的，但它与其他编程语言中的实现方式略有不同。现在是时候了解一下 `const` 了。

Using `const`

使用 `const`

Many languages have a way to declare a value as immutable. In Go, this is done with the `const` keyword. At first glance, it seems to work exactly as

it would in other languages. Try out the code in [Example 2-4](#) on [The Go Playground](#) or in the `sample_code/const_declaration` directory in the [Chapter 2 repository](#).

许多语言都有声明值不可变的方法。在 Go 语言中，可以使用 `const` 关键字来实现。乍一看，它的工作方式似乎与其他语言完全相同。您可以尝试 Go Playground 上的示例 2-4 代码，或者查看第二章代码库中的 `sample_code/const_declaration` 目录。

Example 2-4. `const` declarations

示例 2-4. `const` 声明

```
package main

import "fmt"

const x int64 = 10

const (
    idKey    = "id"
    nameKey  = "name"
)

const z = 20 * 10

func main() {
```



```
const y = "hello"

fmt.Println(x)
fmt.Println(y)

x = x + 1 // this will not compile!
y = "bye" // this will not compile!
fmt.Println(x)
fmt.Println(y)
}
```

When you run this code, compilation fails with the following error messages:

运行此代码时，编译失败并显示以下错误消息：

```
./prog.go:20:2: cannot assign to x (constant 10 of
type int64)
./prog.go:21:2: cannot assign to y (untyped string
constant "hello")
```

As you see, you declare a constant at the package level or within a function. Just as with `var`, you can (and should) declare a group of related constants within a set of parentheses.

如您所见，您可以在包级别或函数内部声明常量。就像 `var` 一样，您可以（也应该）在一组括号内声明一组相关的常量。

Be aware that `const` in Go is very limited. Constants in Go are a way to give names to literals. They can only hold values that the compiler can figure out at compile time. This means that they can be assigned:

请注意，Go 语言中的 `const` 非常有限。Go 语言中的常量是一种为字面量命名的方式。它们只能保存编译器在编译时可以确定的值。这意味着它们可以被赋值：

- Numeric literals

数字文字

- `true` and `false`
`true` 和 `false`

- Strings

字符串

- Runes

符文

- The values returned by the built-in functions `complex`, `real`, `imag`, `len`, and `cap`

内置函数 `complex`、`real`、`imag`、`len` 和 `cap` 返回的值

- Expressions that consist of operators and the preceding values

由运算符和前面的值组成的表达式

NOTE

笔记

I'll cover the `len` and `cap` functions in the next chapter. Another value that can be used with `const` is called `iota`. I'll talk about `iota` when I discuss creating your own types in [Chapter 7](#).

我将在下一章介绍 `len` 和 `cap` 函数。`const` 还可以使用 `iota` 值。我将在第 7 章讨论创建自定义类型时介绍 `iota`。

Go doesn't provide a way to specify that a value calculated at runtime is immutable. For example, the following code will fail to compile with the error `x + y (value of type int) is not constant`:

Go 语言没有提供指定运行时计算的值是不可变的方法。例如，以下代码将无法编译，并出现错误 `x + y (value of type int) is not constant`：

```
x := 5
y := 10
const z = x + y // this won't compile!
```

As you'll see in the next chapter, there are no immutable arrays, slices, maps, or structs, and there's no way to declare that a field in a struct is immutable. This is less limiting than it sounds. Within a function, it is clear if a variable is being modified, so immutability is less important. In [“Go Is](#)

Call by Value”, you’ll see how Go prevents modifications to variables that are passed as parameters to functions.

正如你将在下一章中看到的，Go 中没有不可变的数组、切片、映射或结构体，也无法声明结构体中的字段是不可变的。但这并没有听起来那么限制多大。在函数内部，变量是否被修改是显而易见的，因此不可变性的重要性相对较低。在“Go 是按值调用”一节中，你将看到 Go 如何防止修改作为参数传递给函数的变量。

TIP

提示

Constants in Go are a way to give names to literals. There is *no* way in Go to declare that a variable is immutable.

Go 语言中的常量是给字面量命名的一种方式。Go 语言中没有办法声明一个变量是不可变的。

Typed and Untyped Constants

类型化常量和非类型化常量

Constants can be typed or untyped. An untyped constant works exactly like a literal; it has no type of its own but does have a default type that is used

when no other type can be inferred. A typed constant can be directly assigned only to a variable of that type.

常量可以是类型化的，也可以是无类型的。无类型常量的工作方式与字面量完全相同；它本身没有类型，但有一个默认类型，当无法推断出其他类型时使用该默认类型。类型化常量只能直接赋值给该类型的变量。

Whether to make a constant typed depends on why the constant was declared. If you are giving a name to a mathematical constant that could be used with multiple numeric types, keep the constant untyped. In general, leaving a constant untyped gives you more flexibility. In certain situations, you'll want a constant to enforce a type. You'll see a use for typed constants when I cover enumerations with `iota` in ["iota Is for Enumerations—Sometimes"](#).

是否要为常量指定类型取决于声明该常量的目的。如果您要为一个可能与多种数值类型一起使用的数学常量命名，则应保持常量未指定类型。通常，保持常量未指定类型可以提供更大的灵活性。在某些情况下，您可能需要使用常量来强制指定类型。在“iota 用于枚举——有时”一文中，我将介绍使用 `iota` 的枚举，届时您将看到指定类型常量的用途。

Here's what an untyped constant declaration looks like:

以下是一个无类型常量声明的示例：

```
const x = 10
```

All of the following assignments are legal:

以下所有转让均合法：

```
var y int = x  
var z float64 = x  
var d byte = x
```

Here's what a typed constant declaration looks like:

以下是类型化常量声明的示例：

```
const typedX int = 10
```

This constant can be assigned directly only to an `int`. Assigning it to any other type produces a compile-time error like this:

此常量只能直接赋值给 `int` 类型。将其赋值给任何其他类型都会导致编译时错误，如下所示：

```
cannot use typedX (type int) as type float64 in  
assignment
```

Unused Variables

未使用的变量

One of the goals for Go is to make it easier for large teams to collaborate on programs. To do so, Go has some rules that are unique among programming languages. In [Chapter 1](#), you saw that Go programs need to be formatted in a specific way with `go fmt` to make it easier to write code-manipulation tools and to provide coding standards. Another Go requirement is that *every declared local variable must be read*. It is a *compile-time error* to declare a local variable and to not read its value.

Go 的目标之一是让大型团队更容易协作开发程序。为此，Go 制定了一些与其他编程语言不同的规则。在第一章中，您已经了解到，Go 程序需要使用 `go fmt` 进行特定格式设置，以便于编写代码修改工具并提供编码规范。Go 的另一个要求是，每个声明的局部变量都必须被读取。如果声明了局部变量却不读取其值，则会在编译时报错。

The compiler's unused variable check is not exhaustive. As long as a variable is read once, the compiler won't complain, even if there are writes to the variable that are never read. The following is a valid Go program that you can run on [The Go Playground](#) or in the `sample_code/assignments_not_read` directory in the [Chapter 2 repository](#):

编译器的未使用变量检查并非穷尽所有。只要变量被读取过一次，即使存在从未被读取的写入操作，编译器也不会报错。以下是一个有效的 Go 程序，您可以在 Go Playground 或 Chapter 2 代码库的 `sample_code/assignments_not_read` 目录中运行它：

```
func main() {  
    x := 10 // this assignment isn't read!  
    x = 20  
    fmt.Println(x)  
    x = 30 // this assignment isn't read!  
}
```

While the compiler and `go vet` do not catch the unused assignments of 10 and 30 to `x`, third-party tools can detect them. I'll talk about these tools in [“Using Code-Quality Scanners”](#).

虽然编译器和 `go vet` 无法检测到将 10 和 30 赋值给 `x` 的未使用赋值，但第三方工具可以检测到它们。我将在“使用代码质量扫描器”一节中讨论这些工具。

NOTE

笔记

The Go compiler won't stop you from creating unread package-level variables. This is one more reason you should avoid creating package-level variables.

Go 编译器不会阻止你创建未读取的包级变量。这也是你应该避免创建包级变量的另一个原因。

UNUSED CONSTANTS

未使用的常量

Perhaps surprisingly, the Go compiler allows you to create unread constants with `const`. This is because constants in Go are calculated at compile time and cannot have any side effects. This makes them easy to eliminate: if a constant isn't used, it is simply not included in the compiled binary.

或许令人惊讶的是，Go 编译器允许你使用 `const` 创建未读常量。这是因为 Go 中的常量是在编译时计算的，不会产生任何副作用。因此，它们很容易被移除：如果一个常量没有被使用，它就不会被包含在编译后的二进制文件中。

Naming Variables and Constants

变量和常量的命名

There is a difference between Go’s rules for naming variables and the patterns that Go developers follow when naming their variables and constants. Like most languages, Go requires identifier names to start with a letter or underscore, and the name can contain numbers, underscores, and letters. Go’s definition of “letter” and “number” is a bit broader than many languages. Any Unicode character considered a letter or digit is allowed. This makes all the variable definitions in [Example 2-5](#) perfectly valid Go.

Go 语言的变量命名规则与 Go 开发者在命名变量和常量时遵循的模式有所不同。与大多数语言一样，Go 要求标识符名称以字母或下划线开头，并且名称可以包含数字、下划线和字母。Go 对“字母”和“数字”的定义比许多语言更宽泛。任何被视为字母或数字的 Unicode 字符都是允许的。这使得示例 2-5 中的所有变量定义在 Go 语言中都是完全有效的。

Example 2-5. Variable names you should never use

示例 2-5. 永远不应该使用的变量名

```
_0 := 0_0
```

```
_1 := 20
π := 3
a := "hello" // Unicode U+FF41
__ := "double underscore" // two underscores
fmt.Println(_0)
fmt.Println(_1)
fmt.Println(π)
fmt.Println(a)
fmt.Println(__)
```

You can test out this awful code on [The Go Playground](#). While it works, *do not* use variable names like this. These names are considered nonidiomatic because they break the fundamental rule of making sure that your code communicates what it is doing. These names are confusing or difficult to type on many keyboards. Look-alike Unicode code points are the most insidious, because even if they appear to be the same character, they represent entirely different variables. You can run the code shown in [Example 2-6](#) on [The Go Playground](#) or in the `sample_code/look_alike_code_points` directory in the [Chapter 2 repository](#).

你可以在 Go Playground 上测试这段糟糕的代码。虽然它能运行，但请不要使用这样的变量名。这些名称被认为是不规范的，因为它们违反了确保代码能够清晰表达其功能的基本原则。这些名称容易造成混淆，或者在许多键盘上难以输入。Unicode 代码点的相似性尤其危险，因为它们看起来像同一个字符，它们代表的也是完全不同的

变量。你可以在 Go Playground 或第二章代码库的 `sample_code/look_alike_code_points` 目录中运行示例 2-6 中的代码。

Example 2-6. Using look-alike code points for variable names

示例 2-6：使用相似代码点作为变量名

```
func main() {  
    a := "hello"    // Unicode U+FF41  
    a := "goodbye" // standard lowercase a  
                  (Unicode U+0061)  
    fmt.Println(a)  
    fmt.Println(a)  
}
```

When you run this program, you get:

运行此程序后，您将得到：

```
hello  
goodbye
```

Even though the underscore is a valid character in a variable name, it is rarely used, because idiomatic Go doesn't use snake case (names like `index_counter` or `number_tries`). Instead, idiomatic Go uses camel

case (names like `indexCounter` or `numberTries`) when an identifier name consists of multiple words.

尽管下划线是变量名中的有效字符，但它很少使用，因为惯用的 Go 语言并不使用蛇形命名法（例如 `index_counter` 或 `number_tries`）。相反，当标识符名称由多个单词组成时，惯用的 Go 语言会使用驼峰命名法（例如 `indexCounter` 或 `numberTries`）。

NOTE

笔记

An underscore by itself (`_`) is a special identifier name in Go; I'll talk more about it when I cover functions in [Chapter 5](#).

在 Go 语言中，单独一个下划线（`_`）是一个特殊的标识符名称；我将在第 5 章介绍函数时详细介绍它。

In many languages, constants are always written in all uppercase letters, with words separated by underscores (names like `INDEX_COUNTER` or `NUMBER_TRIES`). Go does not follow this pattern. This is because Go uses the case of the first letter in the name of a package-level declaration to determine if the item is accessible outside the package. I will revisit this when I talk about packages in [Chapter 10](#).

在许多语言中，常量总是用全大写字母书写，单词之间用下划线分隔（例如 `INDEX_COUNTER` 或 `NUMBER_TRIES`）。Go 语言不遵循这种模式。这是因为 Go 语言使用包级声明名称首字母的大小写来判断该项是否可以在包外访问。我将在第 10 章讨论包时再次提及这一点。

Within a function, favor short variable names. *The smaller the scope for a variable, the shorter the name that's used for it.* It is common in Go to see single-letter variable names used with `for` loops. For example, the names `k` and `v` (short for *key* and *value*) are used as the variable names in a `for-range` loop. If you are using a standard `for` loop, `i` and `j` are common names for the index variable. There are other idiomatic ways to name variables of common types; I will mention them as I cover more parts of the standard library.

在函数内部，尽量使用简短的变量名。变量的作用域越小，其名称就应该越短。在 Go 语言中，我们经常会在 `for` 循环中使用单字母变量名。例如，在 `for-range` 循环中，变量名可以是 `k` 和 `v`（分别代表 `key` 和 `value`）。如果使用标准的 `for` 循环，则 `i` 和 `j` 是索引变量的常用名称。对于常见的变量类型，还有其他一些惯用的命名方式；我会在讲解标准库的更多内容时提及它们。

Some languages with weaker type systems encourage developers to include the expected type of the variable in the variable's name. Since Go is strongly typed, you don't need to do this to keep track of the underlying type. However, you may see Go code where the first letter of a type is used

as the variable name (for example, `i` for integers or `f` for floats). When you define your own types, similar patterns apply, especially when naming receiver variables (which are covered in [“Methods”](#)).

一些类型系统较弱的语言鼓励开发者在变量名中包含变量的预期类型。由于 Go 是强类型语言，因此无需这样做即可追踪底层类型。但是，您可能会看到 Go 代码中使用类型首字母作为变量名（例如，`i` 表示整数，`f` 表示浮点数）。当您定义自己的类型时，也适用类似的模式，尤其是在命名接收变量时（这将在“方法”部分介绍）。

These short names serve two purposes. The first is that they eliminate repetitive typing, keeping your code shorter. Second, they serve as a check on how complicated your code is. If you find it hard to keep track of your short-named variables, your block of code is likely doing too much.

这些简短的变量名有两个作用。首先，它们可以避免重复输入，使代码更简洁。其次，它们可以帮助你检查代码的复杂程度。如果你发现难以记住这些简短的变量名，那么你的代码块可能执行了过多的操作。

When naming variables and constants in the package block, use more descriptive names. The type should still be excluded from the name, but since the scope is wider, you need a more complete name to clarify what the value represents.

在包块中命名变量和常量时，请使用更具描述性的名称。类型仍然应该从名称中省略，但由于作用域更广，因此需要更完整的名称来阐明值的含义。

For more discussion of Go naming recommendations, read the [Naming section of Google's Go Style Decisions](#).

有关 Go 命名建议的更多讨论，请阅读 Google Go 风格决策中的“命名”部分。

Exercises

练习

These exercises demonstrate the concepts discussed in the chapter.

Solutions to these exercises, along with the programs in this chapter, are in the [Chapter 2 repository](#).

这些练习演示了本章讨论的概念。这些练习的答案以及本章中的程序都可以在第二章的资料库中找到。

1. Write a program that declares an integer variable called `i` with the value 20. Assign `i` to a floating-point variable named `f`. Print out `i` and `f`.

编写一个程序，声明一个名为 `i` 的整型变量，其值为 20。将 `i` 赋值给一个名为 `f` 的浮点型变量。打印出 `i` 和 `f`。

2. Write a program that declares a constant called `value` that can be assigned to both an integer and a floating-point variable. Assign it to an integer called `i` and a floating-point variable called `f`. Print out `i` and `f`.

编写一个程序，声明一个名为 `value` 的常量，该常量可以赋值给一个整型变量和一个浮点型变量。将其分别赋值给一个名为 `i` 的整型变量和一个名为 `f` 的浮点型变量。打印出 `i` 和 `f`。

3. Write a program with three variables, one named `b` of type `byte`, one named `smallI` of type `int32`, and one named `bigI` of type `uint64`. Assign each variable the maximum legal value for its type; then add 1 to each variable. Print out their values.

编写一个程序，其中包含三个变量：一个名为 `b`、类型为 `byte` 的变量；一个名为 `smallI`、类型为 `int32` 的变量；以及一个名为 `bigI`、类型为 `uint64` 的变量。首先，将每个变量赋值为其类型允许的最大值；然后，将 1 加到每个变量上。最后，打印出这些变量的值。

Wrapping Up

总结

You've covered a lot of ground here, understanding how to use the predeclared types, declare variables, and work with assignments and operators. In the next chapter, we will look at the composite types in Go: arrays, slices, maps, and structs. We will also take another look at strings and runes and how they interact with character encodings.

你已经学习了很多内容，包括如何使用预声明类型、声明变量以及使用赋值和运算符。在下一章中，我们将学习 Go 中的复合类型：数组、切片、映射和结构体。我们还将再次探讨字符串和符文，以及它们如何与字符编码交互。

Chapter 3. Composite Types

第三章 复合类型

In the previous chapter, you looked at literals and predeclared variable types: numbers, booleans, and strings. In this chapter, you'll learn about the composite types in Go, the built-in functions that support them, and the best practices for working with them.

上一章中，你学习了字面量和预声明的变量类型：数字、布尔值和字符串。本章，你将学习 Go 语言中的复合类型、支持它们的内置函数以及使用它们的最佳实践。

Arrays—Too Rigid to Use Directly

数组——过于僵化，无法直接使用

Like most programming languages, Go has arrays. However, arrays are rarely used directly in Go. You'll learn why in a bit, but first let's quickly cover array declaration syntax and use.

和大多数编程语言一样，Go 语言也支持数组。然而，在 Go 语言中很少直接使用数组。稍后你会明白原因，但首先让我们快速了解一下数

组的声明语法和用法。

All elements in the array must be of the type that's specified. There are a few declaration styles. In the first, you specify the size of the array and the type of the elements in the array:

数组中的所有元素必须是指定的类型。声明数组的方式有几种。第一种方式是，你需要指定数组的大小和数组中元素的类型：

```
var x [3]int
```

This creates an array of three `int` s. Since no values were specified, all of the elements (`x[0]` , `x[1]` , and `x[2]`) are initialized to the zero value for an `int` , which is (of course) 0. If you have initial values for the array, you specify them with an *array literal*:

这将创建一个包含三个 `int` 的数组。由于没有指定值，所有元素（`x[0]`、`x[1]` 和 `x[2]`）都被初始化为 `int` 的初始值 0，也就是（当然）0。如果数组有初始值，则可以使用数组字面量来指定：

```
var x = [3]int{10, 20, 30}
```

If you have a *sparse array* (an array where most elements are set to their zero value), you can specify only the indices with nonzero values in the array literal:

如果数组是稀疏数组（大部分元素的值都为零），则在数组字面量中只能指定非零值的索引：

```
var x = [12]int{1, 5: 4, 6, 10: 100, 15}
```

This creates an array of 12 `int`s with the following values: [1, 0, 0, 0, 0, 4, 6, 0, 0, 0, 100, 15].

这将创建一个包含 12 个 `int` 的数组，其值为：[1, 0, 0, 0, 0, 4, 6, 0, 0, 0, 100, 15]。

When using an array literal to initialize an array, you can replace the number that specifies the number of elements in the array with `...` :

使用数组字面量初始化数组时，可以将指定数组元素数量的数字替换为 `...` :

```
var x = [...]int{10, 20, 30}
```

You can use `==` and `!=` to compare two arrays. Arrays are equal if they are the same length and contain equal values:

您可以使用 `==` 和 `!=` 来比较两个数组。如果两个数组长度相同且包含相同的值，则它们相等：

```
var x = [...]int{1, 2, 3}
var y = [3]int{1, 2, 3}
fmt.Println(x == y) // prints true
```

Go has only one-dimensional arrays, but you can simulate multidimensional arrays:

Go语言只有一维数组，但你可以模拟多维数组：

```
var x [2][3]int
```

This declares `x` to be an array of length 2 whose type is an array of `int`s of length 3. This sounds pedantic, but some languages have true matrix support, like Fortran or Julia; Go isn't one of them.

这声明 `x` 是一个长度为 2 的数组，其类型是一个长度为 3 的 `int` 数组。这听起来有点吹毛求疵，但有些语言确实支持矩阵，例如 Fortran 或 Julia；Go 语言并不支持。

Like most languages, arrays in Go are read and written using bracket syntax:

与大多数语言一样，Go 语言中的数组也是使用方括号语法进行读写的：

```
x[0] = 10  
fmt.Println(x[2])
```

You cannot read or write past the end of an array or use a negative index. If you do this with a constant or literal index, it is a compile-time error. An out-of-bounds read or write with a variable index compiles but fails at runtime with a *panic* (you'll learn more about panics in [“panic and recover”](#)).

你不能读取或写入数组末尾之外的数据，也不能使用负索引。如果使用常量或字面索引执行此操作，则会产生编译时错误。使用可变索引进行越界读取或写入操作虽然可以编译通过，但在运行时会引发 panic（你可以在“panic and recover”中了解更多关于 panic 的信息）。

Finally, the built-in function `len` takes in an array and returns its length:

最后，内置函数 `len` 接收一个数组并返回其长度：

```
fmt.Println(len(x))
```

Earlier I said that arrays in Go are rarely used explicitly. This is because they come with an unusual limitation: Go considers the *size* of the array to be part of the *type* of the array. This makes an array that's declared to be `[3]int` a different type from an array that's declared to be `[4]int`. This

also means that you cannot use a variable to specify the size of an array, because types must be resolved at compile time, not at runtime.

我之前说过，Go 语言中的数组很少被显式使用。这是因为它们有一个特殊的限制：Go 将数组的大小视为数组类型的一部分。这使得声明为 `[3]int` 的数组与声明为 `[4]int` 的数组类型不同。这也意味着你不能使用变量来指定数组的大小，因为类型必须在编译时解析，而不是在运行时解析。

What's more, *you can't use a type conversion to directly convert arrays of different sizes to identical types*. Because you can't convert arrays of different sizes into each other, you can't write a function that works with arrays of any size and you can't assign arrays of different sizes to the same variable.

此外，你不能使用类型转换直接将不同大小的数组转换为相同类型。由于不同大小的数组无法相互转换，因此你无法编写一个可以处理任意大小数组的函数，也无法将不同大小的数组赋值给同一个变量。

NOTE

笔记

You'll learn how arrays work behind the scenes when I discuss memory layout in [Chapter 6](#).

在第 6 章讨论内存布局时，你会了解到数组在幕后的工作原理。

Because of these restrictions, don't use arrays unless you know the exact length you need ahead of time. For example, some of the cryptographic functions in the standard library return arrays because the sizes of checksums are defined as part of the algorithm. This is the exception, not the rule.

由于这些限制，除非您事先知道所需的确切长度，否则请勿使用数组。例如，标准库中的一些加密函数会返回数组，因为校验和的大小是在算法中定义的。但这只是例外情况，并非普遍规则。

This raises the question: why is such a limited feature in the language? The main reason arrays exist in Go is to provide the backing store for *slices*, which are one of the most useful features of Go.

这就引出了一个问题：为什么 Go 语言中数组的功能如此受限？数组存在的主要原因是为切片提供底层存储，而切片是 Go 语言最有用的功能之一。

Slices

切片

Most of the time, when you want a data structure that holds a sequence of values, a slice is what you should use. What makes slices so useful is that

you can grow slices as needed. This is because the length of a slice is *not* part of its type. This removes the biggest limitations of arrays and allows you to write a single function that processes slices of any size (I'll cover function writing in [Chapter 5](#)). After going over the basics of using slices in Go, I'll cover the best ways to use them.

大多数情况下，当你需要一个能够存储一系列值的数据结构时，切片（slice）是最佳选择。切片的优势在于可以根据需要扩展其大小。这是因为切片的长度并非其类型的一部分。这克服了数组最大的局限性，使你能够编写一个处理任意大小切片的函数（我将在第五章讲解函数编写）。在介绍完 Go 语言中切片的基础知识后，我将介绍使用切片的最佳实践。

Working with slices looks a lot like working with arrays, but subtle differences exist. The first thing to notice is that you don't specify the size of the slice when you declare it:

使用切片与使用数组非常相似，但两者之间存在一些细微差别。首先需要注意的是，声明切片时不需要指定其大小：

```
var x = []int{10, 20, 30}
```

TIP

提示

Using `[...]` makes an array. Using `[]` makes a slice.

使用 `[...]` 创建一个数组。使用 `[]` 创建一个切片。

This creates a slice of three `int`s using a *slice literal*. Just as with arrays, you can also specify only the indices with nonzero values in the slice literal:

这将使用切片字面量创建一个包含三个 `int` 的切片。与数组一样，您也可以在切片字面量中仅指定索引值非零的部分：

```
var x = []int{1, 5: 4, 6, 10: 100, 15}
```

This creates a slice of 12 `int`s with the following values: `[1, 0, 0, 0, 0, 4, 6, 0, 0, 0, 100, 15]`.

这将创建一个包含 12 个 `int` 的切片，其值为：`[1, 0, 0, 0, 0, 4, 6, 0, 0, 0, 100, 15]`。

You can simulate multidimensional slices and make a slice of slices:

您可以模拟多维切片，并创建切片的切片：

```
var x [][]int
```

You read and write slices using bracket syntax, and, just as with arrays, you can't read or write past the end or use a negative index:

您可以使用方括号语法读取和写入切片，并且与数组一样，您不能读取或写入超出末尾的内容，也不能使用负索引：

```
x[0] = 10  
fmt.Println(x[2])
```

So far, slices have seemed identical to arrays. You start to see the differences between arrays and slices when you look at declaring slices without using a literal:

到目前为止，切片看起来和数组完全一样。但当你不用字面量来声明切片时，你就会开始发现数组和切片之间的区别：

```
var x []int
```

This creates a slice of `int` s. Since no value is assigned, `x` is assigned the zero value for a slice, which is something you haven't seen before: `nil`. I'll talk more about `nil` in [Chapter 6](#), but it is slightly different from the `null` that's found in other languages. In Go, `nil` is an identifier that represents the lack of a value for some types. Like the untyped numeric constants you saw in the previous chapter, `nil` has no type, so it can be

assigned or compared against values of different types. A `nil` slice contains nothing.

这将创建一个 `int` 的切片。由于没有赋值，`x` 被赋值为切片的零值，这是你之前没有见过的：`nil`。我将在第 6 章详细讨论 `nil`，但它与其他语言中的 `null` 略有不同。在 Go 语言中，`nil` 是一个标识符，表示某些类型没有值。就像你在上一章看到的无类型数值常量一样，`nil` 没有类型，因此它可以被赋值或与不同类型的值进行比较。`nil` 切片不包含任何内容。

A slice is the first type you've seen that isn't *comparable*. It is a compile-time error to use `==` to see if two slices are identical or `!=` to see if they are different. The only thing you can compare a slice with using `==` is `nil`:

切片是你遇到的第一个不可比较的类型。使用 `==` 来判断两个切片是否相同，或者使用 `!=` 来判断它们是否不同，都会导致编译时错误。唯一能用 `==` 来比较切片的是 `nil`。

```
fmt.Println(x == nil) // prints true
```

Since Go 1.21, the `slices` package in the standard library includes two functions to compare slices. The `slices.Equal` function takes in two slices and returns `true` if the slices are the same length, and all of the elements are equal. It requires the elements of the slice to be comparable.

The other function, `slices.EqualFunc`, lets you pass in a function to determine equality and does not require the slice elements to be comparable. You'll learn about passing functions into functions in [“Passing Functions as Parameters”](#). The other functions in the `slices` package are covered in [“Adding Generics to the Standard Library”](#).

自 Go 1.21 起，标准库中的 `slices` 包包含了两个用于比较切片的函数。`slices.Equal` 函数接收两个切片作为参数，如果两个切片长度相同且所有元素都相等，则返回 `true`。它要求切片中的元素必须是可比较的。另一个函数 `slices.EqualFunc` 允许你传入一个函数来判断相等性，并且它不要求切片中的元素必须是可比较的。你将在“将函数作为参数传递”一节中学习如何将函数传递给其他函数。`slices` 包中的其他函数将在“向标准库添加泛型”一节中介绍。

```
x := []int{1, 2, 3, 4, 5}
y := []int{1, 2, 3, 4, 5}
z := []int{1, 2, 3, 4, 5, 6}
s := []string{"a", "b", "c"}
fmt.Println(slices.Equal(x, y)) // prints true
fmt.Println(slices.Equal(x, z)) // prints false
fmt.Println(slices.Equal(x, s)) // does not
compile
```

WARNING

警告

The `reflect` package contains a function called `DeepEqual` that can compare almost anything, including slices. It's a legacy function, primarily intended for testing. Before the inclusion of `slices.Equal` and `slices.EqualFunc`, `reflect.DeepEqual` was often used to compare slices. Don't use it in new code, as it is slower and less safe than using the functions in the `slices` package.

`reflect` 包包含一个名为 `DeepEqual` 的函数，它可以比较几乎任何对象，包括切片。这是一个遗留函数，主要用于测试。在 `slices.Equal` 和 `slices.EqualFunc` 包加入之前，`reflect.DeepEqual` 经常用于比较切片。请勿在新代码中使用它，因为它比使用 `slices` 包中的函数速度更慢且安全性更低。

len

镜头

Go provides several built-in functions to work with slices. You've already seen the built-in `len` function when looking at arrays. It works for slices too. Passing a `nil` slice to `len` returns 0.

Go 提供了几个用于处理切片的内置函数。在学习数组时，你已经见过内置的 `len` 函数。它也适用于切片。将 `nil` 切片传递给 `len` 会返回 0。

NOTE

笔记

Functions like `len` are built into Go because they can do things that can't be done by the functions that you can write. You've already seen that `len`'s parameter can be any type of array or any type of slice. You'll soon see that it also works for strings and maps. In [“Channels”](#), you'll see it working with channels. Trying to pass a variable of any other type to `len` is a compile-time error. As you'll see in [Chapter 5](#), Go doesn't let developers write a function that accepts any string, array, slice, channel, or map, but rejects other types.

像 `len` 这样的函数是 Go 语言内置的，因为它们可以完成一些你编写的函数无法完成的操作。你已经看到 `len` 的参数可以是任何类型的数组或任何类型的切片。很快你就会发现它也适用于字符串和映射。在“通道”部分，你会看到它如何与通道一起使用。尝试将任何其他类型的变量传递给 `len` 都会导致编译时错误。正如你将在第 5 章中看到的那样，Go 不允许开发者编写接受任何字符串、数组、切片、通道或映射，但拒绝其他类型的函数。

append

附加

The built-in `append` function is used to grow slices:

内置的 `append` 函数用于扩展切片：

```
var x []int
x = append(x, 10) // assign result to the variable
that's passed in
```


The `append` function takes at least two parameters, a slice of any type and a value of that type. It returns a slice of the same type, which is assigned to the variable that was passed to `append`. In this example, you are appending to a `nil` slice, but you can append to a slice that already has elements:

`append` 函数至少接受两个参数：一个任意类型的切片和一个该类型的值。它返回一个相同类型的切片，并将其赋值给传递给 `append` 的变量。在本例中，您正在向 `nil` 切片追加元素，但您也可以向已包含元素的切片追加元素：

```
var x = []int{1, 2, 3}
x = append(x, 4)
```

You can append more than one value at a time:

您可以一次添加多个值：

```
x = append(x, 5, 6, 7)
```

One slice is appended onto another by using the `...` operator to expand the source slice into individual values (you'll learn more about the `...` operator in [“Variadic Input Parameters and Slices”](#)):

使用 `...` 运算符将一个切片附加到另一个切片上，该运算符可以将源切片展开为单个值（您将在“可变输入参数和切片”中了解有关 `...` 运算符的更多信息）：

```
y := []int{20, 30, 40}
x = append(x, y...)
```

It is a compile-time error if you forget to assign the value returned from `append`. You might be wondering why as it seems a bit repetitive. I will talk about this in greater detail in [Chapter 5](#), but Go is a *call-by-value* language. Every time you pass a parameter to a function, Go makes a copy of the value that's passed in. Passing a slice to the `append` function actually passes a copy of the slice to the function. The function adds the values to the copy of the slice and returns the copy. You then assign the returned slice back to the variable in the calling function.

如果你忘记给 `append` 返回的值赋值，就会出现编译时错误。你可能会疑惑为什么会这样，因为这看起来有点重复。我会在第五章详细解释，但 Go 是一种按值调用的语言。每次你向函数传递参数时，Go 都会复制传入的值。将切片传递给 `append` 函数，实际上是将切片的副本传递给了该函数。函数会将值添加到切片的副本中，然后返回该副本。之后，你需要将返回的切片赋值回调用函数中的变量。

Capacity

容量

As you've seen, a slice is a sequence of values. Each element in a slice is assigned to consecutive memory locations, which makes it quick to read or write these values. The length of a slice is the number of consecutive memory locations that have been assigned a value. Every slice also has a *capacity*, which is the number of consecutive memory locations reserved. This can be larger than the length. Each time you append to a slice, one or more values is added to the end of the slice. Each value added increases the length by one. When the length reaches the capacity, there's no more room to put values. If you try to add additional values when the length equals the capacity, the `append` function uses the Go runtime to allocate a new backing array for the slice with a larger capacity. The values in the original backing array are copied to the new one, the new values are added to the end of the new backing array, and the slice is updated to refer to the new backing array. Finally, the updated slice is returned.

如您所见，切片是一系列值的序列。切片中的每个元素都被分配到连续的内存位置，这使得读取或写入这些值变得非常快捷。切片的长度是指被分配了值的连续内存位置的数量。每个切片还有一个容量，即预留的连续内存位置的数量。容量可以大于长度。每次向切片追加元素时，都会在切片的末尾添加一个或多个值。每个添加的值都会使长度增加 1。当长度达到容量时，就没有空间再添加值了。如果您尝试

在长度等于容量时添加其他值，`append` 函数会使用 Go 运行时为切片分配一个容量更大的新后备数组。原始后备数组中的值会被复制到新数组中，新值会被添加到新后备数组的末尾，并且切片会被更新为指向新的后备数组。最后，更新后的切片会被返回。

THE GO RUNTIME

GO运行时

Every high-level language relies on a set of libraries to enable programs written in that language to run, and Go is no exception. The Go runtime provides services like memory allocation and garbage collection, concurrency support, networking, and implementations of built-in types and functions.

每种高级语言都依赖于一系列库来运行用该语言编写的程序，Go 语言也不例外。Go 运行时提供诸如内存分配和垃圾回收、并发支持、网络通信以及内置类型和函数的实现等服务。

The Go runtime is compiled into every Go binary. This is different from languages that use a virtual machine, which must be installed separately to allow programs written in those languages to function. Including the runtime in the binary makes it easier to distribute Go programs and avoids worries about compatibility issues between the runtime and the program. The drawback of including the runtime in the binary is that even the simplest Go program produces a binary that's about 2 MB.

Go 运行时环境会被编译到每个 Go 二进制文件中。这与其他使用虚拟机的语言不同，后者需要单独安装虚拟机才能运行。将运行时环境包含在二进制文件中，可以更轻松地分发 Go 程序，并避免运行时环境与程序之间的兼容性问题。但缺点是，即使是最简单的 Go 程序，生成的二进制文件大小也约为 2 MB。

When a slice grows via `append`, it takes time for the Go runtime to allocate new memory and copy the existing data from the old memory to the new. The old memory also needs to be garbage collected. For this reason, the Go runtime usually increases a slice by more than one each time it runs out of capacity. The rule as of Go 1.18 is to double the capacity of a slice when the current capacity is less than 256. A bigger slice increases by $(\text{current_capacity} + 768)/4$. This slowly converges at 25% growth (a slice with capacity of 512 will grow by 63%, but a slice with capacity 4,096 will grow by only 30%).

当切片通过 `append` 增长时，Go 运行时需要时间来分配新内存并将现有数据从旧内存复制到新内存。旧内存也需要进行垃圾回收。因此，Go 运行时通常会在每次切片容量不足时将其容量增加不止一个单位。从 Go 1.18 开始，规则是当切片的当前容量小于 256 时，将其容量翻倍。更大的切片则以 $(\text{current_capacity} + 768)/4$ 的单位递增。这种增长速度会逐渐收敛到 25%（例如，容量为 512 的切片将增长 63%，而容量为 4096 的切片将仅增长 30%）。

Just as the built-in `len` function returns the current length of a slice, the built-in `cap` function returns the current capacity of a slice. It is used far less frequently than `len`. Most of the time, `cap` is used to check if a slice is large enough to hold new data, or if a call to `make` is needed to create a new slice.

就像内置函数 `len` 返回切片的当前长度一样，内置函数 `cap` 返回切片的当前容量。它的使用频率远低于 `len`。大多数情况下，`cap` 用于检查切片是否足够大以容纳新数据，或者是否需要调用 `make` 来创建新切片。

You can also pass an array to the `cap` function, but `cap` always returns the same value as `len` for arrays. Don't put it in your code, but save this trick for Go trivia night.

你也可以将数组传递给 `cap` 函数，但对于数组，`cap` 总是返回与 `len` 相同的值。不要把它写进你的代码里，而是把它留到 Go 知识问答之夜吧。

Let's take a look at how adding elements to a slice changes the length and capacity. Run the code in [Example 3-1](#) on [The Go Playground](#) or in the `sample_code/len_cap` directory in the [Chapter 3 repository](#).

我们来看看向切片中添加元素如何改变其长度和容量。请在 Go Playground 或第 3 章代码库的 `sample_code/len_cap` 目录中运行示例 3-1 中的代码。

Example 3-1. Understanding capacity

例 3-1. 理解容量

```
var x []int
fmt.Println(x, len(x), cap(x))
```

```
x = append(x, 10)
fmt.Println(x, len(x), cap(x))
x = append(x, 20)
fmt.Println(x, len(x), cap(x))
x = append(x, 30)
fmt.Println(x, len(x), cap(x))
x = append(x, 40)
fmt.Println(x, len(x), cap(x))
x = append(x, 50)
fmt.Println(x, len(x), cap(x))
```

When you build and run the code, you'll see the following output. Notice how and when the capacity increases:

编译并运行代码后，您将看到以下输出。请注意容量增加的方式和时间：

```
[ ] 0 0
[10] 1 1
[10 20] 2 2
[10 20 30] 3 4
[10 20 30 40] 4 4
[10 20 30 40 50] 5 8
```

While it's nice that slices grow automatically, it's far more efficient to size them once. If you know how many things you plan to put into a slice, create

it with the correct initial capacity. You do that with the `make` function.

虽然切片会自动增长很方便，但一次性设置切片大小效率更高。如果您知道计划在切片中放入多少内容，请使用正确的初始容量创建切片。您可以使用 `make` 函数来实现这一点。

make

制作

You've already seen two ways to declare a slice, using a slice literal or the `nil` zero value. While useful, neither way allows you to create an empty slice that already has a length or capacity specified. That's the job of the built-in `make` function. It allows you to specify the type, length, and, optionally, the capacity. Let's take a look:

您已经了解了声明切片的两种方法：使用切片字面量或 `nil` 零值。虽然这两种方法都很有用，但它们都无法创建已指定长度或容量的空切片。内置的 `make` 函数可以实现这一点。它允许您指定类型、长度以及（可选）容量。让我们来看一下：

```
x := make([]int, 5)
```

This creates an `int` slice with a length of 5 and a capacity of 5. Since it has a length of 5, `x[0]` through `x[4]` are valid elements, and they are all

initialized to 0.

这将创建一个长度为 5、容量为 5 的 `int` 切片。由于其长度为 5，因此 `x[0]` 到 `x[4]` 都是有效元素，并且它们都被初始化为 0。

One common beginner mistake is to try to populate those initial elements using `append`:

初学者常犯的一个错误是尝试使用 `append` 来填充这些初始元素：

```
x := make([]int, 5)
x = append(x, 10)
```

The 10 is placed at the end of the slice, *after* the zero values in elements 0–4 because `append` always increases the length of a slice. The value of `x` is now `[0 0 0 0 0 10]`, with a length of 6 and a capacity of 10 (the capacity was doubled as soon as the sixth element was appended).

10 被放置在切片的末尾，位于元素 0 到 4 的零值之后，因为 `append` 总是会增加切片的长度。`x` 的值现在为 `[0 0 0 0 0 10]`，长度为 6，容量为 10（添加第六个元素后，容量立即翻倍）。

You can also specify an initial capacity with `make`:

您还可以使用 `make` 指定初始容量：

```
x := make([]int, 5, 10)
```

This creates an `int` slice with a length of 5 and a capacity of 10.

这将创建一个长度为 5、容量为 10 的 `int` 切片。

You can also create a slice with zero length but a capacity that's greater than zero:

您还可以创建长度为零但容量大于零的切片：

```
x := make([]int, 0, 10)
```

In this case, you have a non-`nil` slice with a length of 0 but a capacity of 10. Since the length is 0, you can't directly index into it, but you can append values to it:

在这种情况下，你有一个长度为 0 但容量为 10 的非 `nil` 切片。由于长度为 0，你不能直接对其进行索引，但你可以向其中追加值：

```
x := make([]int, 0, 10)
x = append(x, 5, 6, 7, 8)
```

The value of `x` is now `[5 6 7 8]`, with a length of 4 and a capacity of 10.

x 的值现在为 [5 6 7 8]，长度为 4，容量为 10。

WARNING

警告

Never specify a capacity that's less than the length! It is a compile-time error to do so with a constant or numeric literal. If you use a variable to specify a capacity that's smaller than the length, your program will panic at runtime.

切勿指定小于长度的容量！使用常量或数值字面量这样做会导致编译时错误。如果使用变量指定小于长度的容量，程序将在运行时崩溃。

Emptying a Slice

清空一片

Go 1.21 added a `clear` function that takes in a slice and sets all of the slice's elements to their zero value. The length of the slice remains unchanged. The following code:

Go 1.21 新增了一个 `clear` 函数，该函数接收一个切片，并将切片中的所有元素的值设置为零。切片的长度保持不变。以下是相关代码：

```
s := []string{"first", "second", "third"}
fmt.Println(s, len(s))
```

```
clear(s)
fmt.Println(s, len(s))
```

prints out:

打印输出:

```
[first second third] 3
[   ] 3
```

(Remember, the zero value for a string is an empty string `""` !)

(请记住，字符串的零值是空字符串 `""` !)

Declaring Your Slice

宣布你的份额

Now that you've seen all these ways to create slices, how do you choose which slice declaration style to use? The primary goal is to minimize the number of times the slice needs to grow. If it's possible that the slice won't need to grow at all, use a `var` declaration with no assigned value to create a `nil` slice, as shown in [Example 3-2](#).

既然您已经了解了创建切片的所有方法，那么如何选择使用哪种切片声明样式呢？主要目标是尽量减少切片需要增长的次数。如果切片可能完全不需要增长，则可以使用未赋值的 `var` 声明来创建 `nil` 切片，如示例 3-2 所示。

Example 3-2. Declaring a slice that might stay `nil`

示例 3-2. 声明一个可能保持不变的切片 `nil`

```
var data []int
```

NOTE

笔记

You can create a slice using an empty slice literal:

您可以使用空切片字面量创建切片：

```
var x = []int{}
```

This creates a slice with zero length and zero capacity. It is confusingly different from a `nil` slice. Because of implementation reasons, comparing a zero-length slice to `nil` returns `false`, while comparing a `nil` slice to `nil` returns `true`. For simplicity, favor `nil` slices. A zero-length slice is useful only when converting a slice to JSON. You'll look at this more in [“encoding/json”](#).

这将创建一个长度和容量均为零的切片。它与 `nil` 切片有明显的区别。由于实现原因，将零长度切片与 `nil` 进行比较会返回 `false`，而将 `nil` 切片与 `nil` 进行比较则会返回 `true`。为了简单起见，建议使用 `nil` 切片。零长度切片仅在将切片转换为 JSON 时才有用。您将在“encoding/json”部分了解更多相关信息。

If you have some starting values, or if a slice's values aren't going to change, then a slice literal is a good choice (see [Example 3-3](#)).

如果你有一些初始值，或者切片的值不会改变，那么切片字面量是一个不错的选择（参见示例 3-3）。

Example 3-3. Declaring a slice with default values

示例 3-3. 声明带有默认值的切片

```
data := []int{2, 4, 6, 8} // numbers we appreciate
```

If you have a good idea of how large your slice needs to be, but don't know what those values will be when you are writing the program, use `make`.

The question then becomes whether you should specify a nonzero length in the call to `make` or specify a zero length and a nonzero capacity. There are three possibilities:

如果您大致知道切片需要多大，但在编写程序时不知道这些值具体是多少，请使用 `make`。接下来的问题是，您应该在调用 `make` 时指定非零长度，还是指定零长度和非零容量。有三种可能性：

- If you are using a slice as a buffer (you'll see this in [“io and Friends”](#)), then specify a nonzero length.

如果您将切片用作缓冲区（您会在“io 和朋友”中看到这一点），则指定一个非零长度。

- If you are *sure* you know the exact size you want, you can specify the length and index into the slice to set the values. This is often done when transforming values in one slice and storing them in a second. The downside to this approach is that if you have the size wrong,

you'll end up with either zero values at the end of the slice or a panic from trying to access elements that don't exist.

如果您确定知道所需的确切大小，可以指定切片的长度和索引来设置值。这通常用于转换一个切片中的值并将其存储在另一个切片中。这种方法的缺点是，如果大小错误，切片末尾的值可能为零，或者由于尝试访问不存在的元素而导致 panic。

- In other situations, use `make` with a zero length and a specified capacity. This allows you to use `append` to add items to the slice. If the number of items turns out to be smaller, you won't have an extraneous zero value at the end. If the number of items is larger, your code will not panic.

在其他情况下，可以使用长度为零且容量指定的 `make`。这样，您就可以使用 `append` 向切片中添加项目。如果项目数量较少，则末尾不会出现多余的零值。如果项目数量较多，您的代码也不会发生 panic。

The Go community is split between the second and third approaches. I personally prefer using `append` with a slice initialized to a zero length. It might be slower in some situations, but it is less likely to introduce a bug.

Go 社区对于第二种和第三种方法存在分歧。我个人更倾向于使用 `append` 并将切片初始化为零长度。虽然在某些情况下速度可能会慢一些，但它引入 bug 的可能性更小。

WARNING

警告

`append` always increases the length of a slice! If you have specified a slice's length using `make`, be sure that you mean to append to it before you do so, or you might end up with a bunch of surprise zero values at the beginning of your slice.

`append` 总是会增加切片的长度！如果您使用 `make` 指定了切片的长度，请确保您确实想要追加长度，否则切片的开头可能会出现一堆意想不到的零值。

Slicing Slices

切片

A *slice expression* creates a slice from a slice. It's written inside brackets and consists of a starting offset and an ending offset, separated by a colon (:). The starting offset is the first position in the slice that is included in the new slice, and the ending offset is one past the last position to include. If you leave off the starting offset, 0 is assumed. Likewise, if you leave off the ending offset, the end of the slice is substituted. You can see how this works by running the code in [Example 3-4](#) on [The Go Playground](#) or in the `sample_code/slicing_slices` directory in the [Chapter 3 repository](#).

切片表达式用于从现有切片创建新切片。它写在方括号内，由起始偏移量和结束偏移量组成，两者之间用冒号 (:) 分隔。起始偏移量是新切片中包含的原始切片的第一个位置，结束偏移量是要包含的最后一个位置之后的位置。如果省略起始偏移量，则默认为 0。同样，如果省

略结束偏移量，则默认使用原始切片的末尾位置。您可以通过运行 Go Playground 上的示例 3-4 或第三章代码库中的 `sample_code/slicing_slices` 目录来查看其工作原理。

Example 3-4. Slicing slices

例 3-4. 切片

```
x := []string{"a", "b", "c", "d"}
y := x[:2]
z := x[1:]
d := x[1:3]
e := x[:]
fmt.Println("x:", x)
fmt.Println("y:", y)
fmt.Println("z:", z)
fmt.Println("d:", d)
fmt.Println("e:", e)
```

It gives the following output:

它输出以下结果：

```
x: [a b c d]
y: [a b]
z: [b c d]
```

```
d: [b c]
e: [a b c d]
```

When you take a slice from a slice, you are *not* making a copy of the data. Instead, you now have two variables that are sharing memory. This means that changes to an element in a slice affect all slices that share that element. Let's see what happens when you change values. You can run the code in [Example 3-5](#) on [The Go Playground](#) or in the `sample_code/slice_share_storage` directory in the [Chapter 3 repository](#).

当你从一个切片中取出另一个切片时，你并没有复制数据。相反，现在有两个变量共享内存。这意味着对切片中某个元素的更改会影响所有共享该元素的切片。让我们看看更改值时会发生什么。你可以在 Go Playground 或第三章代码库的 `sample_code/slice_share_storage` 目录中运行示例 3-5 中的代码。

Example 3-5. Slices with overlapping storage

示例 3-5. 具有重叠存储的切片

```
x := []string{"a", "b", "c", "d"}
y := x[:2]
z := x[1:]
x[1] = "y"
y[0] = "x"
z[1] = "z"
```

```
fmt.Println("x:", x)
fmt.Println("y:", y)
fmt.Println("z:", z)
```

You get the following output:

您将得到以下输出：

```
x: [x y z d]
y: [x y]
z: [y z d]
```

Changing `x` modified both `y` and `z`, while changes to `y` and `z` modified `x`.

更改 `x` 会同时修改 `y` 和 `z`，而更改 `y` 和 `z` 会同时修改 `x`。

Slicing slices gets extra confusing when combined with `append`. Try out the code in [Example 3-6](#) on [The Go Playground](#) or in the `sample_code/slice_append_storage` directory in the [Chapter 3 repository](#).

当与 `append` 结合使用时，切片操作会变得更加复杂。请尝试 Go Playground 上的示例 3-6 中的代码，或者第三章代码库中的 `sample_code/slice_append_storage` 目录中的代码。

Example 3-6. `append` makes overlapping slices more confusing

示例 3-6。 `append` 使重叠切片更加混乱。

```
x := []string{"a", "b", "c", "d"}
y := x[:2]
fmt.Println(cap(x), cap(y))
y = append(y, "z")
fmt.Println("x:", x)
fmt.Println("y:", y)
```

Running this code gives the following output:

运行此代码会得到以下输出：

```
4 4
x: [a b z d]
y: [a b z]
```

What's going on? Whenever you take a slice from another slice, the subslice's capacity is set to the capacity of the original slice, minus the starting offset of the subslice within the original slice. This means elements of the original slice beyond the end of the subslice, including unused capacity, are shared by both slices.

这是怎么回事？每当您从一个切片中取出一个切片时，子切片的容量会被设置为原始切片的容量减去子切片在原始切片中的起始偏移量。这意味着原始切片中超出子切片末尾的部分（包括未使用的容量）将由两个切片共享。

When you make the `y` slice from `x`, the length is set to 2, but the capacity is set to 4, the same as `x`. Since the capacity is 4, appending onto the end of `y` puts the value in the third position of `x`.

当您从 `x` 创建 `y` 切片时，长度设置为 2，但容量设置为 4，与 `x` 相同。由于容量为 4，因此将其附加到 `y` 的末尾会将值放入 `x` 的第三个位置。

This behavior creates some odd scenarios, with multiple slices appending and overwriting each other's data. See if you can guess what the code in [Example 3-7](#) prints out, then run it on [The Go Playground](#) or in the `sample_code/confusing_slices` directory in the [Chapter 3 repository](#) to see if you guessed correctly.

这种行为会产生一些奇怪的情况，例如多个切片相互追加和覆盖数据。试着猜猜示例 3-7 中的代码会输出什么，然后在 Go Playground 或第三章代码库中的 `sample_code/confusing_slices` 目录下运行它，看看你的猜测是否正确。

Example 3-7. Even more confusing slices

例 3-7. 更令人困惑的切片

```
x := make([]string, 0, 5)
x = append(x, "a", "b", "c", "d")
y := x[:2]
z := x[2:]
fmt.Println(cap(x), cap(y), cap(z))
y = append(y, "i", "j", "k")
x = append(x, "x")
z = append(z, "y")
fmt.Println("x:", x)
fmt.Println("y:", y)
fmt.Println("z:", z)
```

To avoid complicated slice situations, you should either never use `append` with a subslice or make sure that `append` doesn't cause an overwrite by using a *full slice expression*. This is a little weird, but it makes clear how much memory is shared between the parent slice and the subslice. The full slice expression includes a third part, which indicates the last position in the parent slice's capacity that's available for the subslice. Subtract the starting offset from this number to get the subslice's capacity. [Example 3-8](#) shows the first four lines from the previous example, modified to use full slice expressions.

为了避免复杂的切片情况，您应该始终避免在子切片中使用 `append`，或者确保 `append` 不会导致覆盖，方法是使用完整的切片表达式。这有点奇怪，但它清楚地显示了父切片和子切片之间共享的内存量。完整的切片表达式包含第三部分，它指示父切片容量中可供子切片使用的最后一个位置。从该数字中减去起始偏移量即可得到子切片的容量。示例 3-8 显示了上一个示例中的前四行，并对其进行了修改以使用完整的切片表达式。

Example 3-8. The full slice expression protects against `append`

示例 3-8。完整的切片表达式可防止 `append`

```
x := make([]string, 0, 5)
x = append(x, "a", "b", "c", "d")
y := x[:2:2]
z := x[2:4:4]
```

Try out this code on [The Go Playground](#) or in the *sample_code/full_slice_expression* directory in the [Chapter 3 repository](#). Both `y` and `z` have a capacity of `2`. Because you limited the capacity of the subslices to their lengths, appending additional elements onto `y` and `z` created new slices that didn't interact with the other slices. After this code runs, `x` is set to `[a b c d x]`, `y` is set to `[a b i j k]`, and `z` is set to `[c d y]`.

请在 Go Playground 或第三章代码库的

`sample_code/full_slice_expression` 目录中尝试这段代码。 `y` 和 `z` 的容量均为 2。由于您将子切片的容量限制为其长度，因此向 `y` 和 `z` 追加元素会创建新的切片，这些新切片不会与其他切片交互。代码运行后， `x` 将被设置为 `[a b c d x]`， `y` 将被设置为 `[a b i j k]`， `z` 将被设置为 `[c d y]`。

WARNING

警告

Be careful when taking a slice of a slice! Both slices share the same memory, and changes to one are reflected in the other. Avoid modifying slices after they have been sliced or if they were produced by slicing. Use a three-part slice expression to prevent `append` from sharing capacity between slices.

切取切片中的切片时务必小心！两个切片共享同一内存，对其中一个切片的更改会反映到另一个切片中。避免在切片完成后或切片是由切片生成的切片上进行修改。使用三段式切片表达式可以防止 `append` 在切片之间共享内存。

copy

复制

If you need to create a slice that's independent of the original, use the built-in `copy` function. Let's take a look at a simple example, which you can run on [The Go Playground](#) or in the `sample_code/copy_slice` directory in the [Chapter 3 repository](#):

如果需要创建一个独立于原始切片的切片，请使用内置的 `copy` 函数。我们来看一个简单的例子，您可以在 Go Playground 或第 3 章代码库的 `sample_code/copy_slice` 目录中运行该示例：

```
x := []int{1, 2, 3, 4}
y := make([]int, 4)
num := copy(y, x)
fmt.Println(y, num)
```

You get this output:

您将得到以下输出：

```
[1 2 3 4] 4
```

The `copy` function takes two parameters. The first is the destination slice, and the second is the source slice. The function copies as many values as it can from source to destination, limited by whichever slice is smaller, and returns the number of elements copied. The *capacity* of `x` and `y` doesn't matter; it's the length that's important.

`copy` 函数接受两个参数。第一个参数是目标切片，第二个参数是源切片。该函数会尽可能多地将源切片中的值复制到目标切片，复制数量受限于较小的切片，并返回复制的元素数量。`x` 和 `y` 的容量无关紧要；重要的是它们的长度。

You can also copy a subset of a slice. The following code copies the first two elements of a four-element slice into a two-element slice:

您还可以复制切片的子集。以下代码将包含四个元素的切片的前两个元素复制到一个包含两个元素的切片中：

```
x := []int{1, 2, 3, 4}
y := make([]int, 2)
num := copy(y, x)
```

The variable `y` is set to `[1 2]`, and `num` is set to 2.

变量 `y` 设置为 `[1 2]`, `num` 设置为 2。

You could also copy from the middle of the source slice:

您也可以从源切片的中间部分复制：

```
x := []int{1, 2, 3, 4}
y := make([]int, 2)
copy(y, x[2:])
```

You are copying the third and fourth elements in `x` by taking a slice of the slice. Also note that *you don't assign the output of `copy` to a variable*. If you don't need the number of elements copied, you don't need to assign it.

您通过对 `x` 进行切片操作，复制了其中的第三个和第四个元素。另请注意，您没有将 `copy` 的输出赋值给变量。如果您不需要知道复制的元素数量，则无需赋值。

The `copy` function allows you to copy between two slices that cover overlapping sections of an underlying slice:

`copy` 函数允许您在覆盖底层切片重叠部分的两个切片之间进行复制：

```
x := []int{1, 2, 3, 4}
num := copy(x[:3], x[1:])
fmt.Println(x, num)
```

In this case, you are copying the last three values in `x` on top of the first three values of `x`. This prints out `[2 3 4 4] 3`.

在这种情况下，您将 `x` 中的最后三个值复制到 `x` 的前三个值之上。这将输出 `[2 3 4 4] 3`。

You can use `copy` with arrays by taking a slice of the array. You can make the array either the source or the destination of the copy. You can try out the following code on [The Go Playground](#) or in the `sample_code/copy_array` directory in the [Chapter 3 repository](#):

你可以通过对数组进行切片操作来使用 `copy`。你可以将数组设置为复制操作的源数组或目标数组。你可以在 Go Playground 或第三章代码库的 `sample_code/copy_array` 目录中尝试以下代码：

```
x := []int{1, 2, 3, 4}
d := [4]int{5, 6, 7, 8}
y := make([]int, 2)
copy(y, d[:])
fmt.Println(y)
copy(d[:], x)
fmt.Println(d)
```

The first call to `copy` copies the first two values in array `d` into slice `y`. The second copies all of the values in slice `x` into array `d`. This produces the following output:

第一次调用 `copy` 会将数组 `d` 中的前两个值复制到切片 `y` 中。第二次调用会将切片 `x` 中的所有值复制到数组 `d` 中。这将产生以下输出：

```
[5 6]
[1 2 3 4]
```

Converting Arrays to Slices

将数组转换为切片

Slices aren't the only thing you can slice. If you have an array, you can take a slice from it using a slice expression. This is a useful way to bridge an array to a function that takes only slices. To convert an entire array into a slice, use the `[:]` syntax:

切片并非唯一可以进行切片的对象。如果您有一个数组，可以使用切片表达式从中截取一个切片。这是一种将数组连接到仅接受切片的函数的有效方法。要将整个数组转换为切片，请使用 `[:]` 语法：

```
xArray := [4]int{5, 6, 7, 8}
xSlice := xArray[:]
```

You can also convert a subset of an array into a slice:

你还可以将数组的子集转换为切片：

```
x := [4]int{5, 6, 7, 8}
y := x[:2]
z := x[2:]
```

Be aware that taking a slice from an array has the same memory-sharing properties as taking a slice from a slice. If you run the following code on [The Go Playground](#) or in the `sample_code/slice_array_memory` directory in the [Chapter 3 repository](#):

请注意，从数组中切取切片与从切片中切取切片具有相同的内存共享特性。如果您在 Go Playground 或第 3 章代码库中的 `sample_code/slice_array_memory` 目录下运行以下代码：

```
x := [4]int{5, 6, 7, 8}
y := x[:2]
z := x[2:]
x[0] = 10
fmt.Println("x:", x)
fmt.Println("y:", y)
fmt.Println("z:", z)
```

you get this output:

你会得到这样的输出：

```
x: [10 6 7 8]
y: [10 6]
z: [7 8]
```


Converting Slices to Arrays

将切片转换为数组

Use a type conversion to make an array variable from a slice. You can convert an entire slice to an array of the same type, or you can create an array from a subset of the slice.

使用类型转换可以将切片转换为数组变量。您可以将整个切片转换为相同类型的数组，也可以从切片的子集创建数组。

When you convert a slice to an array, the data in the slice is copied to new memory. That means that changes to the slice won't affect the array, and vice versa.

将切片转换为数组时，切片中的数据会被复制到新的内存空间。这意味着对切片的更改不会影响数组，反之亦然。

The following code:

以下代码：

```
xSlice := []int{1, 2, 3, 4}
xArray := [4]int(xSlice)
smallArray := [2]int(xSlice)
xSlice[0] = 10
fmt.Println(xSlice)
```

```
fmt.Println(xArray)
fmt.Println(smallArray)
```

prints out:

打印输出:

```
[10 2 3 4]
[1 2 3 4]
[1 2]
```

The size of the array must be specified at compile time. It's a compile-time error to use `[...]` in a slice to array type conversion.

数组的大小必须在编译时指定。在切片到数组类型转换中使用 `[...]` 会导致编译时错误。

While the size of the array can be smaller than the size of the slice, it cannot be bigger. Unfortunately, the compiler cannot detect this, and your code will panic at runtime if you specify an array size that's bigger than the length (not the capacity) of the slice. The following code:

虽然数组的大小可以小于切片的大小，但不能大于切片的大小。遗憾的是，编译器无法检测到这一点，如果您指定的数组大小大于切片的长度（而非容量），则代码会在运行时崩溃。以下代码：

```
panicArray := [5]int(xSlice)
fmt.Println(panicArray)
```

panics at runtime with the message:

运行时出现 panic, 并显示以下消息:

```
panic: runtime error: cannot convert slice with
length 4 to array
    or pointer to array with length 5
```

NOTE

笔记

I haven't talked about pointers yet, but you can also use a type conversion to convert a slice into a pointer to an array:

我还没谈到指针，但你也可以使用类型转换将切片转换为指向数组的指针：

```
xSlice := []int{1,2,3,4}
xArrayPointer := (*[4]int)(xSlice)
```

After converting a slice to an array pointer, the storage between the two is shared. A change to one will change the other:

将切片转换为数组指针后，两者共享存储空间。对其中一个的更改会影响另一个：

```
xSlice[0] = 10
xArrayPointer[1] = 20
fmt.Println(xSlice) // prints [10 20 3 4]
fmt.Println(xArrayPointer) // prints &[10 20 3 4]
```

Pointers are covered in [Chapter 6](#).

指针相关内容在第6章中介绍。

You can try all of the array type conversions on [The Go Playground](#) or in the *sample_code/array_conversion* directory in the [Chapter 3 repository](#).

您可以在 Go Playground 或第 3 章存储库的 *sample_code/array_conversion* 目录中尝试所有数组类型转换。

In [“Arrays—Too Rigid to Use Directly”](#), I mentioned that you can’t use arrays as function parameters when the size of the array being passed in might vary. Technically, you can work around this limitation by converting an array to a slice, converting the slice to an array of a different size, and then passing the second array in to a function. The second array must be shorter than the first array, or your program will panic. While this might be helpful in a pinch, if you find yourself doing this frequently, strongly consider changing your function’s API to take a slice instead of an array.

在《数组——过于死板，无法直接使用》一文中，我提到过，当传入的数组大小可能变化时，不能直接将数组作为函数参数。理论上，你可以通过将数组转换为切片，再将切片转换为不同大小的数组，最后将第二个数组传递给函数来规避这个限制。第二个数组必须比第一个数组短，否则程序会崩溃。虽然这种方法在紧急情况下可能有用，但如果你发现自己经常需要这样做，强烈建议你修改函数的 API，使其接受切片而不是数组作为参数。

Strings and Runes and Bytes

字符串、符文和字节

Now that I’ve talked about slices, we can look at strings again. You might think that a string in Go is made out of runes, but that’s not the case. Under the covers, Go uses a sequence of bytes to represent a string. These bytes

don't have to be in any particular character encoding, but several Go library functions (and the `for-range` loop that I discuss in the next chapter) assume that a string is composed of a sequence of UTF-8-encoded code points.

既然我已经讲完了切片，我们可以再来看看字符串。你可能认为 Go 语言中的字符串是由符文组成的，但事实并非如此。Go 语言底层使用字节序列来表示字符串。这些字节序列不需要采用任何特定的字符编码，但一些 Go 库函数（以及我将在下一章讨论的 `for-range` 循环）假定字符串是由 UTF-8 编码的码位序列组成的。

NOTE

笔记

According to the language specification, Go source code is always written in UTF-8. Unless you use hexadecimal escapes in a string literal, your string literals are written in UTF-8.

根据语言规范，Go 源代码始终以 UTF-8 编码编写。除非你在字符串字面量中使用十六进制转义符，否则你的字符串字面量都将以 UTF-8 编码编写。

Just as you can extract a single value from an array or a slice, you can extract a single value from a string by using an *index expression*:

就像可以从数组或切片中提取单个值一样，也可以使用索引表达式从字符串中提取单个值：

```
var s string = "Hello there"
var b byte = s[6]
```

Like arrays and slices, string indexes are zero-based; in this example, `b` is assigned the numeric value of the seventh position in `s`, which is `116` (the UTF-8 value of a lowercase `t`).

与数组和切片一样，字符串索引也是从零开始的；在本例中，`b` 被赋予 `s` 中第七个位置的数值，即 `116`（小写字母 `t` 的 UTF-8 值）。

The slice expression notation that you used with arrays and slices also works with strings:

你之前用于数组和切片的切片表达式表示法也适用于字符串：

```
var s string = "Hello there"
var s2 string = s[4:7]
var s3 string = s[:5]
var s4 string = s[6:]
```

This assigns “`ot`” to `s2`, “`Hello`” to `s3`, and “`there`” to `s4`. You can try out this code [on The Go Playground](#) or in the `sample_code/string_slicing` directory in the [Chapter 3 repository](#).

这段代码将“`ot`”赋值给 `s2`，将“`Hello`”赋值给 `s3`，将“`there`”赋值给 `s4`。您可以在 Go Playground 或第 3 章代码库的

sample_code/string_slicing 目录中尝试这段代码。

While it's handy that Go allows you to use slicing notation to make substrings and use index notation to extract individual entries from a string, you should be careful when doing so. Since strings are immutable, they don't have the modification problems that slices of slices do. There is a different problem, though. A string is composed of a sequence of bytes, while a code point in UTF-8 can be anywhere from one to four bytes long. The previous example was entirely composed of code points that are one byte long in UTF-8, so everything worked out as expected. But when dealing with languages other than English or with emojis, you run into code points that are multiple bytes long in UTF-8:

虽然 Go 语言允许你使用切片表示法来创建子字符串，并使用索引表示法从字符串中提取单个条目，这很方便，但使用时需要格外小心。由于字符串是不可变的，它们不存在切片的切片所存在的修改问题。然而，这里存在另一个问题。字符串由一系列字节组成，而 UTF-8 编码中的一个代码点长度可以是 1 到 4 个字节。前面的例子完全由 UTF-8 编码中长度为 1 个字节的代码点组成，因此一切都按预期运行。但是，当处理英语以外的语言或表情符号时，你会遇到 UTF-8 编码中长度为多个字节的代码点：

```
var s string = "Hello 🌞"  
var s2 string = s[4:7]
```



```
var s3 string = s[:5]
var s4 string = s[6:]
```

In this example, `s3` will still be equal to “Hello.” The variable `s4` is set to the sun emoji. But `s2` is not set to “o 🌞.” Instead, you get “o ❓.” That’s because you copied only the first byte of the sun emoji’s code point, which is not a valid code point on its own.

在这个例子中，`s3` 仍然等于“Hello”。变量 `s4` 被设置为太阳表情符号。但是 `s2` 并没有被设置为“o 🌞”，而是被设置为“o ❓”。这是因为你只复制了太阳表情符号代码点的第一个字节，而它本身并不是一个有效的代码点。

Go allows you to pass a string to the built-in `len` function to find the length of the string. Given that string index and slice expressions count positions in bytes, it’s not surprising that the length returned is the length in bytes, not in code points:

Go 语言允许你将一个字符串传递给内置的 `len` 函数来获取字符串的长度。鉴于字符串索引和切片表达式是以字节为单位计算位置的，因此返回的长度是字节长度而不是代码点长度也就不足为奇了：

```
var s string = "Hello 🌞"
```

```
fmt.Println(len(s))
```

This code prints out 10, not 7, because it takes four bytes to represent the sun with smiling face emoji in UTF-8. You can run these sun emoji examples on [The Go Playground](#) or in the *sample_code/sun_slicing* directory in the [Chapter 3 repository](#).

这段代码输出的是 10 而不是 7，因为用 UTF-8 编码表示一个带笑脸的太阳表情符号需要四个字节。你可以在 Go Playground 或第三章代码库的 *sample_code/sun_slicing* 目录中运行这些太阳表情符号示例。

WARNING

警告

Even though Go allows you to use slicing and indexing syntax with strings, you should use it only when you know that your string contains only characters that take up one byte.

尽管 Go 语言允许你对字符串使用切片和索引语法，但只有当你确定你的字符串只包含占用一个字节的字符时，才应该使用它。

Because of this complicated relationship among runes, strings, and bytes, Go has some interesting type conversions between these types. A single rune or byte can be converted to a string:

由于符文、字符串和字节之间存在这种复杂的关系，Go 语言在这些类型之间提供了一些有趣的类型转换。单个符文或字节可以转换为字符

串:

```
var a rune    = 'x'
var s string  = string(a)
var b byte    = 'y'
var s2 string = string(b)
```

WARNING

警告

A common bug for new Go developers is to try to make an `int` into a `string` by using a type conversion:

对于 Go 新手开发者来说，一个常见的错误是尝试使用类型转换将 `int` 转换为 `string`：

```
var x int = 65
var y = string(x)
fmt.Println(y)
```

This results in `y` having the value “A,” not “65.” As of Go 1.15, `go vet` blocks a type conversion to string from any integer type other than `rune` or `byte`.

这导致 `y` 的值为“A”，而不是“65”。从 Go 1.15 开始，`go vet` 会阻止从除 `rune` 或 `byte` 之外的任何整数类型到字符串的类型转换。

A string can be converted back and forth to a slice of bytes or a slice of runes. Try [Example 3-9](#) on [The Go Playground](#) or in the `sample_code/string_to_slice` directory in the [Chapter 3 repository](#).

字符串可以与字节切片或符文切片相互转换。请尝试 Go Playground 上的示例 3-9，或第三章代码库中的 `sample_code/string_to_slice` 目录下的示例。

Example 3-9. Converting strings to slices

例 3-9. 将字符串转换为切片

```
var s string = "Hello, 🌞"  
var bs []byte = []byte(s)  
var rs []rune = []rune(s)  
fmt.Println(bs)  
fmt.Println(rs)
```

When you run this code, you see the following:

运行这段代码后，你会看到以下内容：

```
[72 101 108 108 111 44 32 240 159 140 158]  
[72 101 108 108 111 44 32 127774]
```

The first output line has the string converted to UTF-8 bytes. The second has the string converted to runes.

第一行输出的是转换成 UTF-8 字节的字符串。第二行输出的是转换成 rune 字符的字符串。

Most data in Go is read and written as a sequence of bytes, so the most common string type conversions are back and forth with a slice of bytes. Slices of runes are uncommon.

在 Go 语言中，大多数数据都是以字节序列的形式读写的，因此最常见的字符串类型转换是与字节切片之间的转换。符文切片并不常见。

UTF-8

UTF-8

UTF-8 is the most commonly used encoding for Unicode. Unicode uses four bytes (32 bits) to represent each *code point*, the technical name for each character and modifier. Given this, the simplest way to represent Unicode code points is to store four bytes for each code point. This is called UTF-32. It is mostly unused because it wastes so much space. Due to Unicode implementation details, 11 of the 32 bits are always zero. Another common encoding is UTF-16, which uses one or two 16-bit (2-byte) sequences to represent each code point. This is also wasteful; much of the content in the world is written using code points that fit into a single byte. And that's where UTF-8 comes in.

UTF-8 是 Unicode 最常用的编码。Unicode 使用四个字节（32 位）来表示每个代码点，代码点是每个字符和修饰符的技术名称。因此，表示 Unicode 代码点的最简单方法是为每个代码点存储四个字节。这被称为 UTF-32。由于它浪费大量空间，所以现在很少使用。由于 Unicode 的实现细节，32 位中有 11 位始终为零。另一种常见的编码是 UTF-16，它使用一个或两个 16 位（2 字节）序列来表示每个代码点。这也很浪费空间；世界上大部分内容都是使用单个字节即可容纳的代码点编写的。而这正是 UTF-8 的用武之地。

UTF-8 is clever. It lets you use a single byte to represent the Unicode characters whose values are below 128 (which includes all of the letters, numbers, and punctuation commonly used in English), but expands to a

maximum of four bytes to represent Unicode code points with larger values. The result is that the *worst* case for UTF-8 is the same as using UTF-32. UTF-8 has some other nice properties. Unlike UTF-32 and UTF-16, you don't have to worry about little-endian versus big-endian. It also allows you to look at any byte in a sequence and tell if you are at the start of a UTF-8 sequence or somewhere in the middle. That means you can't accidentally read a character incorrectly.

UTF-8 编码非常巧妙。它允许你用单个字节来表示 Unicode 值小于 128 的字符（包括所有常用的英文字母、数字和标点符号），但对于 Unicode 值更大的字符，它会扩展到最多四个字节。因此，UTF-8 的最坏情况与使用 UTF-32 相同。UTF-8 还有一些其他的优点。与 UTF-32 和 UTF-16 不同，你无需担心小端序和大端序的问题。它还允许你查看序列中的任何字节，并判断当前位置是 UTF-8 序列的开头还是中间。这意味着你不会意外地读错字符。

The only downside is that you cannot randomly access a string encoded with UTF-8. While you can detect if you are in the middle of a character, you can't tell how many characters in you are. You need to start at the beginning of the string and count. Go doesn't require a string to be written in UTF-8 but strongly encourages it. You'll see how to work with UTF-8 strings in upcoming chapters.

唯一的缺点是无法随机访问 UTF-8 编码的字符串。虽然可以检测到是否位于某个字符的中间，但无法确定当前访问了多少个字符。你需要

从字符串开头开始计数。Go 语言并不强制要求字符串必须使用 UTF-8 编码，但强烈建议这样做。你将在后续章节中学习如何处理 UTF-8 字符串。

Fun fact: UTF-8 was invented in 1992 by Ken Thompson and Rob Pike, two of the creators of Go.

有趣的事实：UTF-8 是 Go 语言的两位创始人 Ken Thompson 和 Rob Pike 于 1992 年发明的。

Rather than use the slice and index expressions with strings, you should extract substrings and code points from strings using the functions in the `strings` and `unicode/utf8` packages in the standard library. In the next chapter, you'll see how to use a `for-range` loop to iterate over the code points in a string.

与其使用字符串的切片和索引表达式，不如使用标准库中 `strings` 和 `unicode/utf8` 包中的函数从字符串中提取子字符串和代码点。下一章将介绍如何使用 `for-range` 循环遍历字符串中的代码点。

Maps

地图

Slices are useful when you have sequential data. Like most languages, Go provides a built-in data type for situations where you want to associate one value to another. The map type is written as `map[keyType]valueType`. Let's take a look at a few ways to declare maps. First, you can use a `var` declaration to create a map variable that's set to its zero value:

切片在处理顺序数据时非常有用。与大多数语言一样，Go 提供了一种内置数据类型，用于将一个值与另一个值关联起来。映射类型写法为 `map[keyType]valueType`。我们来看几种声明映射的方法。首先，可以使用 `var` 声明创建一个初始值为零的映射变量：

```
var nilMap map[string]int
```

In this case, `nilMap` is declared to be a map with `string` keys and `int` values. The zero value for a map is `nil`. A `nil` map has a length of 0. Attempting to read a `nil` map always returns the zero value for the map's value type. However, *attempting to write to a `nil` map variable causes a panic.*

在这种情况下，`nilMap` 被声明为一个映射，其键为 `string`，值为 `int`。映射的零值为 `nil`。`nil` 映射的长度为 0。尝试读取 `nil` 映射总是返回该映射值类型的零值。但是，尝试写入 `nil` 映射变量会导致 panic。

You can use a `:=` declaration to create a map variable by assigning it a *map literal*:

您可以使用 `:=` 声明，通过为其赋值映射字面量来创建映射变量：

```
totalWins := map[string]int{}
```

In this case, you are using an empty map literal. This is not the same as a `nil` map. It has a length of 0, but you can read and write to a map assigned an empty map literal. Here's what a nonempty map literal looks like:

在这种情况下，您使用的是空映射字面量。这与 `nil` 映射不同。它的长度为 0，但您可以对分配了空映射字面量的映射进行读写操作。以下是非空映射字面量的示例：

```
teams := map[string][]string {  
    "Orcas": []string{"Fred", "Ralph", "Bijou"},  
    "Lions": []string{"Sarah", "Peter", "Billie"},  
}
```

```
"Kittens": []string{"Waldo", "Raul", "Ze"},  
}
```

A map literal's body is written as the key, followed by a colon (:), then the value. A comma separates each key-value pair in the map, even on the last line. In this example, the value is a slice of strings. The type of the value in a map can be anything. There are some restrictions on the types of the keys that I'll discuss in a bit.

映射字面量的主体写法为：键，后跟冒号 (:), 再后跟值。映射中每个键值对之间用逗号分隔，即使是最后一行也不例外。在本例中，值是一个字符串切片。映射中值的类型可以是任何类型。键的类型有一些限制，我稍后会讨论。

If you know how many key-value pairs you intend to put in the map but don't know the exact values, you can use `make` to create a map with a default size:

如果您知道要将多少个键值对放入映射中，但不知道确切的值，则可以使用 `make` 创建一个具有默认大小的映射：

```
ages := make(map[int][]string, 10)
```

Maps created with `make` still have a length of 0, and they can grow past the initially specified size.

使用 `make` 创建的地图长度仍然为 0，并且它们可以增长到超过最初指定的大小。

Maps are like slices in several ways:

地图在很多方面都像切片：

- Maps automatically grow as you add key-value pairs to them.
当您向映射中添加键值对时，映射会自动增长。
- If you know how many key-value pairs you plan to insert into a map, you can use `make` to create a map with a specific initial size.
如果您知道计划在映射中插入多少个键值对，则可以使用 `make` 创建一个具有特定初始大小的映射。
- Passing a map to the `len` function tells you the number of key-value pairs in a map.
将映射传递给 `len` 函数可以告诉你映射中键值对的数量。
- The zero value for a map is `nil`.
地图的零值为 `nil`。
- Maps are not comparable. You can check if they are equal to `nil`, but you cannot check if two maps have identical keys and values using `==` or differ using `!=`.
映射之间不具有可比性。您可以使用 `nil` 检查它们是否相等，但您无法使用 `==` 检查两个映射是否具有相同的键和值，也无法使用 `!=` 检查它们是否不同。

The key for a map can be any comparable type. This means *you cannot use a slice or a map as the key for a map*.

地图的键可以是任何可比较的类型。这意味着你不能使用切片或地图本身作为地图的键。

TIP

提示

When should you use a map, and when should you use a slice? You should use slices for lists of data when the data should be processed sequentially or the order of the elements is important.

何时应该使用映射（map），何时应该使用切片（slice）？当数据需要按顺序处理或元素顺序很重要时，应该对数据列表使用切片。

Maps are useful when you need to organize values using something other than an increasing integer value, such as a name.

当您需要使用除递增整数值之外的其他方式（例如名称）来组织值时，映射非常有用。

WHAT IS A HASH MAP?

什么是哈希映射？

In computer science, a *map* is a data structure that associates (or maps) one value to another. Maps can be implemented in several ways, each with its own trade-offs. The map that's built into Go is a *hash map*, or *hash table*. If you aren't familiar with the concept, Chapter 5 in [*Grokking Algorithms*](#) by Aditya Bhargava (Manning) describes what a hash table is and why they are so useful.

在计算机科学中，映射（map）是一种将一个值关联（或映射）到另一个值的数据结构。映射可以用多种方式实现，每种方式都有其自身的优缺点。Go 语言内置的映射是哈希映射，也称为哈希表。如果您不熟悉这个概念，Aditya Bhargava 所著的《算法入门》（Grokking Algorithms, Manning 出版社）第五章介绍了哈希表的定义及其用途。

It's great that Go includes a hash map implementation as part of the runtime, because building your own is hard to get right. If you'd like to learn more about how Go does it, watch [“Inside the Map Implementation”](#), a talk from GopherCon 2016 by Keith Randall.

Go 语言运行时内置了哈希映射的实现，这非常棒，因为自己构建哈希映射很难做好。如果您想了解更多关于 Go 如何实现哈希映射的信息，可以观看 Keith Randall 在 2016 年 GopherCon 大会上的演讲“Inside the Map Implementation”。

Go doesn't require (or even allow) you to define your own hash algorithm or equality definition. Instead, the Go runtime that's compiled into every Go program has code that implements hash algorithms for all types that are allowed to be keys.

Go 语言不要求（甚至不允许）你自定义哈希算法或相等性定义。相反，编译到每个 Go 程序中的 Go 运行时环境已经包含了为所有允许作为键的类型实现的哈希算法代码。

Reading and Writing a Map

阅读和绘制地图

Let's look at a short program that declares, writes to, and reads from a map. You can run the program in [Example 3-10](#) on [The Go Playground](#) or in the *sample_code/map_read_write* directory in the [Chapter 3 repository](#).

我们来看一个简短的程序，它声明、写入和读取一个映射表。你可以在 Go Playground 上运行示例 3-10 中的程序，或者在第三章代码库的 *sample_code/map_read_write* 目录中运行该程序。

Example 3-10. Using a map

例 3-10. 使用地图

```
totalWins := map[string]int{}
totalWins["Orcas"] = 1
totalWins["Lions"] = 2
fmt.Println(totalWins["Orcas"])
fmt.Println(totalWins["Kittens"])
totalWins["Kittens"]++
fmt.Println(totalWins["Kittens"])
totalWins["Lions"] = 3
fmt.Println(totalWins["Lions"])
```

When you run this program, you'll see the following output:

运行此程序后，您将看到以下输出：

```
1
0
1
3
```

You assign a value to a map key by putting the key within brackets and using `=` to specify the value, and you read the value assigned to a map key by putting the key within brackets. Note that you cannot use `:=` to assign a value to a map key.

您可以通过将键放在方括号内，并使用 `=` 指定值来为映射键赋值；同样，您可以通过将键放在方括号内来读取映射键的值。请注意，您不能使用 `:=` 为映射键赋值。

When you try to read the value assigned to a map key that was never set, the map returns the zero value for the map's value type. In this case, the value type is an `int`, so you get back a 0. You can use the `++` operator to increment the numeric value for a map key. Because a map returns its zero value by default, this works even when there's no existing value associated with the key.

当您尝试读取从未设置的映射键的值时，映射会返回该映射值类型的零值。在本例中，值类型为 `int`，因此返回值为 0。您可以使用 `++` 运算符递增映射键的数值。由于映射默认返回零值，因此即使键没有关联的值，此方法也有效。

The comma ok Idiom

逗号ok习语

As you've seen, a map returns the zero value if you ask for the value associated with a key that's not in the map. This is handy when implementing things like the `totalWins` counter you saw earlier. However, you sometimes do need to find out if a key is in a map. Go

provides the *comma ok idiom* to tell the difference between a key that's associated with a zero value and a key that's not in the map:

正如你所见，如果你请求的键不在映射表中，映射表会返回零值。这在实现类似你之前看到的 `totalWins` 计数器之类的功能时非常有用。然而，有时你确实需要判断某个键是否在映射表中。Go 提供了逗号 `ok` 惯用法来区分与零值关联的键和不在映射表中的键：

```
m := map[string]int{
    "hello": 5,
    "world": 0,
}
v, ok := m["hello"]
fmt.Println(v, ok)

v, ok = m["world"]
fmt.Println(v, ok)

v, ok = m["goodbye"]
fmt.Println(v, ok)
```

Rather than assign the result of a map read to a single variable, with the comma ok idiom you assign the results of a map read to two variables. The first gets the value associated with the key. The second value returned is a bool. It is usually named `ok`. If `ok` is `true`, the key is present in the

map. If `ok` is `false`, the key is not present. In this example, the code prints out `5 true`, `0 true`, and `0 false`.

使用逗号 `ok` 惯用法，您可以将映射读取的结果赋值给两个变量，而不是像通常那样只赋值给一个变量。第一个变量存储与键关联的值。第二个变量返回的值是一个布尔值，通常命名为 `ok`。如果 `ok` 等于 `true`，则表示该键存在于映射中；如果 `ok` 等于 `false`，则表示该键不存在。在本例中，代码会输出 `5 true`、`0 true` 和 `0 false`。

NOTE

笔记

The comma `ok` idiom is used in Go when you want to differentiate between reading a value and getting back the zero value. You'll see it again when you read from channels in [Chapter 12](#) and when you use type assertions in [Chapter 7](#).

在 Go 语言中，逗号 `ok` 表示法用于区分读取值和返回零值。你会在第 12 章从通道读取数据以及第 7 章使用类型断言时再次看到它。

Deleting from Maps

从地图中删除

Key-value pairs are removed from a map via the built-in `delete` function:

通过内置的 `delete` 函数从映射中删除键值对：

```
m := map[string]int{
    "hello": 5,
    "world": 10,
}
delete(m, "hello")
```

The `delete` function takes a map and a key and then removes the key-value pair with the specified key. If the key isn't present in the map or if the map is `nil`, nothing happens. The `delete` function doesn't return a value.

`delete` 函数接受一个映射和一个键，然后删除具有指定键的键值对。如果映射中不存在该键，或者映射为 `nil`，则不执行任何操作。`delete` 函数不返回任何值。

Emptying a Map

清空地图

The `clear` function that you saw in [“Emptying a Slice”](#) works on maps also. A cleared map has its length set to zero, unlike a cleared slice. The following code:

您在“清空切片”中看到的 `clear` 函数也适用于映射。清空映射的长度会被设置为零，这与清空切片不同。以下代码：

```
m := map[string]int{
    "hello": 5,
    "world": 10,
}
fmt.Println(m, len(m))
clear(m)
fmt.Println(m, len(m))
```

prints out:

打印输出:

```
map[hello:5 world:10] 2
map[] 0
```

Comparing Maps

地图对比

Go 1.21 added a package to the standard library called `maps` that contains helper functions for working with maps. You'll learn more about this package in [“Adding Generics to the Standard Library”](#). Two functions in the package are useful for comparing if two maps are equal, `maps.Equal` and `maps.EqualFunc`. They are analogous to the `slices.Equal` and `slices.EqualFunc` functions:

Go 1.21 在标准库中添加了一个名为 `maps` 的包，其中包含用于处理映射的辅助函数。您将在“向标准库添加泛型”一节中了解更多关于此包的信息。该包中的两个函数 `maps.Equal` 和 `maps.EqualFunc` 可用于比较两个映射是否相等。它们类似于 `slices.Equal` 和 `slices.EqualFunc` 函数：

```
m := map[string]int{
    "hello": 5,
    "world": 10,
}
n := map[string]int{
    "world": 10,
    "hello": 5,
}
fmt.Println(maps.Equal(m, n)) // prints true
```

Using Maps as Sets

将地图用作集合

Many languages include a set in their standard library. A *set* is a data type that ensures there is at most one of a value, but doesn't guarantee that the values are in any particular order. Checking to see if an element is in a set is fast, no matter how many elements are in the set. (Checking to see if an element is in a slice takes longer, as you add more elements to the slice.)

许多编程语言的标准库中都包含集合（Set）类型。集合是一种数据类型，它保证每个值最多只有一个，但不保证这些值遵循任何特定的顺序。检查一个元素是否在集合中非常快，无论集合中有多少个元素。

（检查一个元素是否在切片中则需要更长时间，因为切片中添加的元素越多，所需时间就越长。）

Go doesn't include a set, but you can use a map to simulate some of its features. Use the key of the map for the type that you want to put into the set and use a `bool` for the value. The code in [Example 3-11](#) demonstrates the concept. You can run it on [The Go Playground](#) or in the `sample_code/map_set` directory in the [Chapter 3 repository](#).

Go 语言本身没有集合（Set），但你可以使用映射（Map）来模拟它的一些功能。使用映射的键（key）来指定要放入集合中的类型，并使用 `bool` 作为值。示例 3-11 中的代码演示了这个概念。你可以在 Go Playground 或第三章代码库的 `sample_code/map_set` 目录中运行它。

Example 3-11. Using a map as a set

例 3-11. 将地图用作集合

```
intSet := map[int]bool{}
vals := []int{5, 10, 2, 5, 8, 7, 3, 9, 1, 2, 10}
for _, v := range vals {
    intSet[v] = true
}
```

```
fmt.Println(len(vals), len(intSet))
fmt.Println(intSet[5])
fmt.Println(intSet[500])
if intSet[100] {
    fmt.Println("100 is in the set")
}
```

You want a set of `int`s, so you create a map where the keys are of `int` type and the values are of `bool` type. You iterate over the values in `vals` using a `for-range` loop (which I discuss in [“The for-range Statement”](#)) to place them into `intSet`, associating each `int` with the boolean value `true`.

你需要一组 `int`，所以你创建一个映射，其中键的类型为 `int`，值的类型为 `bool`。你使用 `for-range` 循环（我在“for-range 语句”中讨论过）遍历 `vals` 中的值，并将它们放入 `intSet` 中，同时将每个 `int` 与布尔值 `true` 关联起来。

We wrote 11 values into `intSet`, but the length of `intSet` is 8, because you cannot have duplicate keys in a map. If you look for 5 in `intSet`, it returns `true`, because there is a key with the value 5. However, if you look for 500 or 100 in `intSet`, it returns `false`. This is because you haven't put either value into `intSet`, which causes the map to return the zero value for the map value, and the zero value for a `bool` is `false`.

我们向 `intSet` 写入了 11 个值，但 `intSet` 的长度是 8，因为映射中不能有重复的键。如果在 `intSet` 中查找值 5，它会返回 `true`，因为存在一个值为 5 的键。但是，如果在 `intSet` 中查找值 500 或 100，它会返回 `false`。这是因为您没有将这两个值放入 `intSet` 中，导致映射返回该映射值的零值，而 `bool` 的零值是 `false`。

If you need sets that provide operations like union, intersection, and subtraction, you can either write one yourself or use one of the many third-party libraries that provide the functionality. (You'll learn more about using third-party libraries in [Chapter 10](#).)

如果你需要提供并集、交集和减法等运算的集合，你可以自己编写，也可以使用众多提供这些功能的第三方库。（你将在第 10 章中了解更多关于使用第三方库的信息。）

NOTE

笔记

Some people prefer to use `struct{}` for the value when a map is being used to implement a set. (I'll discuss structs in the next section.) The advantage is that an empty struct uses zero bytes, while a boolean uses one byte.

当使用映射（map）来实现集合（set）时，有些人喜欢使用 `struct{}` 来表示值。（我将在下一节讨论结构体。）这样做的好处是，空结构体占用零字节，而布尔值占用一个字节。

The disadvantage is that using a `struct{}` makes your code clumsier. You have a less obvious assignment, and you need to use the comma ok idiom to check if a value is in the set:

缺点是使用 `struct{}` 会使你的代码显得笨拙。赋值语句不够直观，你需要使用逗号 ok 的惯用法来检查某个值是否在集合中：

```
intSet := map[int]struct{}{}
vals := []int{5, 10, 2, 5, 8, 7, 3, 9, 1, 2, 10}
for _, v := range vals {
    intSet[v] = struct{}{}
}
if _, ok := intSet[5]; ok {
    fmt.Println("5 is in the set")
}
```

Unless you have very large sets, the difference in memory usage will not likely be significant enough to outweigh the disadvantages.

除非你的数据集非常大，否则内存使用量的差异不太可能大到足以抵消其缺点。

Structs

结构

Maps are a convenient way to store some kinds of data, but they have limitations. They don't define an API since there's no way to constrain a map to allow only certain keys. Also, all values in a map must be of the same type. For these reasons, maps are not an ideal way to pass data from function to function. When you have related data that you want to group together, you should define a *struct*.

映射（Map）是一种便捷的数据存储方式，但它也存在局限性。由于无法限制映射只允许特定的键，因此它没有定义 API。此外，映射中的所有值必须是同一类型。鉴于这些原因，映射并非在函数之间传递数据的理想方式。当您需要将相关数据分组在一起时，应该定义一个结构体（struct）。

NOTE

笔记

If you already know an object-oriented language, you might be wondering about the difference between classes and structs. The difference is simple: Go doesn't have classes, because it doesn't have inheritance. This doesn't mean that Go doesn't have some of the features of object-oriented languages it just does things a little differently. You'll learn more about the object-oriented features of Go in [Chapter 7](#).

如果你已经了解一种面向对象语言，你可能会好奇类和结构体之间的区别。区别很简单：Go 语言没有类，因为它不支持继承。但这并不意味着 Go 语言没有面向对象语言的一些特性，它只是实现方式略有不同。你将在第 7 章中了解更多关于 Go 语言面向对象特性的信息。

Most languages have a concept that's similar to a struct, and the syntax that Go uses to read and write structs should look familiar:

大多数语言都有类似于结构体的概念，Go 语言用于读写结构体的语法应该看起来很熟悉：

```
type person struct {  
    name string  
    age  int  
    pet  string  
}
```

A struct type is defined with the keyword `type`, the name of the struct type, the keyword `struct`, and a pair of braces (`{}`). Within the braces, you list the fields in the struct. Just as you put the variable name first and

the variable type second in a `var` declaration, you put the struct field name first and the struct field type second. Also note that unlike in map literals, no commas separate the fields in a struct declaration. You can define a struct type inside or outside of a function. A struct type that's defined within a function can be used only within that function. (You'll learn more about functions in [Chapter 5](#).)

结构体类型由关键字 `type`、结构体类型名称、关键字 `struct` 和一对大括号 (`{}`) 定义。在大括号内，列出结构体中的字段。就像在 `var` 声明中先写变量名后写变量类型一样，结构体字段名称也应该放在前面，字段类型放在后面。另请注意，与映射字面量不同，结构体声明中字段之间不需要用逗号分隔。结构体类型可以在函数内部或外部定义。在函数内部定义的结构体类型只能在该函数内部使用。（您将在第 5 章中学习更多关于函数的内容。）

NOTE

笔记

Technically, you can scope a struct definition to any block level. You'll learn more about blocks in [Chapter 4](#).

理论上，你可以将结构体定义的作用域限定在任何代码块级别。你将在第 4 章中了解更多关于代码块的内容。

Once a struct type is declared, you can define variables of that type:

声明结构体类型后，就可以定义该类型的变量：

```
var fred person
```

Here we are using a `var` declaration. Since no value is assigned to `fred`, it gets the zero value for the `person` struct type. A zero value struct has every field set to the field's zero value.

这里我们使用了 `var` 声明。由于 `fred` 没有赋值，因此它对于 `person` 结构体类型而言，其值为零。零值结构体的每个字段都设置为该字段的零值。

A *struct literal* can be assigned to a variable as well:

结构体字面量也可以赋值给一个变量：

```
bob := person{}
```

Unlike maps, there is no difference between assigning an empty struct literal and not assigning a value at all. Both initialize all fields in the struct to their zero values. There are two styles for a nonempty struct literal. First, a struct literal can be specified as a comma-separated list of values for the fields inside of braces:

与映射不同，给结构体赋值一个空结构体字面量和完全不赋值之间没有区别。两者都会将结构体中的所有字段初始化为零值。非空结构体字面量有两种写法。第一种写法是将结构体字面量指定为花括号内以逗号分隔的字段值列表：

```
julia := person{  
    "Julia",  
    40,  
    "cat",  
}
```

When using this struct literal format, a value for every field in the struct must be specified, and the values are assigned to the fields in the order they were declared in the struct definition.

使用此结构体字面量格式时，必须为结构体中的每个字段指定一个值，并且这些值将按照它们在结构体定义中声明的顺序分配给字段。

The second struct literal style looks like the map literal style:

第二种结构体字面量风格看起来像映射字面量风格：

```
beth := person{  
    age: 30,
```

```
    name: "Beth",  
}
```

You use the names of the fields in the struct to specify the values. This style has some advantages. It allows you to specify the fields in any order, and you don't need to provide a value for all fields. Any field not specified is set to its zero value.

您可以使用结构体中的字段名称来指定值。这种方式有一些优点。它允许您以任意顺序指定字段，并且无需为所有字段都提供值。任何未指定的字段都将被设置为其零值。

You cannot mix the two struct literal styles: either all fields are specified with names, or none of them are. For small structs where all fields are always specified, the simpler struct literal style is fine. In other cases, use the names. It's more verbose, but it makes clear what value is being assigned to what field without having to reference the struct definition. It's also more maintainable. If you initialize a struct without using the field names and a future version of the struct adds additional fields, your code will no longer compile.

结构体字面量和字段名两种风格不能混用：要么所有字段都使用名称，要么一个都不使用。对于所有字段都始终指定的小型结构体，使用更简单的结构体字面量风格即可。在其他情况下，请使用字段名。虽然这种方式更冗长，但它无需引用结构体定义就能清楚地表明每个

字段的值。此外，它也更易于维护。如果您在初始化结构体时未使用字段名，而后续版本又添加了其他字段，则您的代码将无法编译。

A field in a struct is accessed with dot notation:

结构体中的字段可以用点号表示法访问：

```
bob.name = "Bob"  
fmt.Println(bob.name)
```

Just as you use brackets for both reading and writing to a map, you use dotted notation for reading and writing to struct fields.

就像在读取和写入映射时使用方括号一样，在读取和写入结构体字段时使用点号表示法。

Anonymous Structs

匿名结构

You can also declare that a variable implements a struct type without first giving the struct type a name. This is called an *anonymous struct*:

你还可以声明一个变量实现了一个结构体类型，而无需事先给该结构体类型命名。这被称为匿名结构体：

```
var person struct {  
    name string  
    age  int  
    pet  string  
}  
  
person.name = "bob"  
person.age  = 50  
person.pet  = "dog"  
  
pet := struct {  
    name string  
    kind string  
} {  
    name: "Fido",  
    kind: "dog",  
}
```

In this example, the types of the variables `person` and `pet` are anonymous structs. You assign (and read) fields in an anonymous struct just as you do for a named struct type. Just as you can initialize an instance of a named struct with a struct literal, you can do the same for an anonymous struct as well.

在这个例子中，变量 `person` 和 `pet` 的类型是匿名结构体。您可以像操作命名结构体一样，对匿名结构体中的字段进行赋值（和读

取)。就像您可以使用结构体字面量初始化命名结构体的实例一样，您也可以对匿名结构体执行相同的操作。

You might wonder when it's useful to have a data type that's associated only with a single instance. Anonymous structs are handy in two common situations. The first is when you translate external data into a struct or a struct into external data (like JSON or Protocol Buffers). This is called *unmarshaling* and *marshaling* data, respectively. You'll learn how to do this in [“encoding/json”](#).

你可能会好奇，什么时候使用只与单个实例关联的数据类型才有用。匿名结构体在两种常见情况下非常方便。第一种情况是将外部数据转换为结构体，或将结构体转换为外部数据（例如 JSON 或 Protocol Buffers）。这分别称为数据反序列化和数据序列化。你将在“encoding/json”部分学习如何操作。

Writing tests is another place where anonymous structs pop up. You'll use a slice of anonymous structs when writing table-driven tests in [Chapter 15](#).

编写测试时也会用到匿名结构体。在第 15 章编写基于表的测试时，你会用到匿名结构体的切片。

Comparing and Converting Structs

比较和转换结构体

Whether a struct is comparable depends on the struct's fields. Structs that are entirely composed of comparable types are comparable; those with slice or map fields are not (as you will see in later chapters, function and channel fields also prevent a struct from being comparable).

结构体是否可比较取决于其字段。完全由可比较类型组成的结构体是可比较的；而包含切片或映射字段的结构体则不可比较（正如你将在后面的章节中看到的，函数字段和通道字段也会阻止结构体可比较）。

Unlike in Python or Ruby, in Go there's no magic method that can be overridden to redefine equality and make `==` and `!=` work for incomparable structs. You can, of course, write your own function that you use to compare structs.

与 Python 或 Ruby 不同，Go 中没有可以重写的魔法方法来重新定义相等性，从而使 `==` 和 `!=` 能够用于不可比较的结构体。当然，您可以编写自己的函数来比较结构体。

Just as Go doesn't allow comparisons between variables of different primitive types, Go doesn't allow comparisons between variables that represent structs of different types. Go does allow you to perform a type

conversion from one struct type to another *if the fields of both structs have the same names, order, and types*. Let's see what this means. Given this struct:

就像 Go 不允许比较不同基本类型的变量一样，Go 也不允许比较表示不同类型结构体的变量。但是，如果两个结构体的字段名称、顺序和类型都相同，Go 允许你执行类型转换。让我们看看这意味着什么。假设有这样一个结构体：

```
type firstPerson struct {  
    name string  
    age  int  
}
```

you can use a type conversion to convert an instance of `firstPerson` to `secondPerson`, but you can't use `==` to compare an instance of `firstPerson` and an instance of `secondPerson`, because they are different types:

您可以使用类型转换将 `firstPerson` 的实例转换为 `secondPerson`，但您不能使用 `==` 来比较 `firstPerson` 的实例和 `secondPerson` 的实例，因为它们是不同的类型：

```
type secondPerson struct {  
    name string
```

```
    age int
}
```

You can't convert an instance of `firstPerson` to `thirdPerson`, because the fields are in a different order:

您无法将 `firstPerson` 的实例转换为 `thirdPerson`，因为字段顺序不同：

```
type thirdPerson struct {
    age int
    name string
}
```

You can't convert an instance of `firstPerson` to `fourthPerson` because the field names don't match:

您无法将 `firstPerson` 的实例转换为 `fourthPerson`，因为字段名称不匹配：

```
type fourthPerson struct {
    firstName string
    age        int
}
```

Finally, you can't convert an instance of `firstPerson` to `fifthPerson` because there's an additional field:

最后，您无法将 `firstPerson` 的实例转换为 `fifthPerson`，因为还有一个额外的字段：

```
type fifthPerson struct {  
    name      string  
    age       int  
    favoriteColor string  
}
```

Anonymous structs add a small twist: if two struct variables are being compared and at least one has a type that's an anonymous struct, you can compare them without a type conversion if the fields of both structs have the same names, order, and types. You can also assign between named and anonymous struct types if the fields of both structs have the same names, order, and types:

匿名结构体带来了一些变化：如果要比较两个结构体变量，并且至少其中一个变量的类型是匿名结构体，那么只要这两个结构体的字段名称、顺序和类型都相同，就可以直接比较它们而无需进行类型转换。此外，如果两个结构体的字段名称、顺序和类型都相同，也可以在命名结构体和匿名结构体之间进行赋值操作。

```
type firstPerson struct {
    name string
    age  int
}
f := firstPerson{
    name: "Bob",
    age:  50,
}
var g struct {
    name string
    age  int
}

// compiles -- can use = and == between identical
// named and anonymous structs
g = f
fmt.Println(f == g)
```

Exercises

练习

The following exercises will test what you've learned about Go's composite types. You can find solutions in the *exercise_solutions* directory in the

[Chapter 3 Repository](#).

以下练习将检验你对 Go 语言复合类型的理解。你可以在第三章代码库的 `exercise_solutions` 目录中找到答案。

1. Write a program that defines a variable named `greetings` of type slice of strings with the following values: `"Hello"`, `"Hola"`, `"नमस्कार"`, `"こんにちは"`, and `"Привіт"`. Create a subslice containing the first two values; a second subslice with the second, third, and fourth values; and a third subslice with the fourth and fifth values. Print out all four slices.

编写一个程序，定义一个名为 `greetings` 的字符串切片变量，其值分别为：`"Hello"`、`"Hola"`、`"नमस्कार"`、`"こんにちは"` 和 `"Привіт"`。创建四个子切片，分别包含前两个值；第二个子切片包含第二个、第三个和第四个值；第三个子切片包含第四个和第五个值。打印出所有四个切片。

2. Write a program that defines a string variable called `message` with the value `"Hi 🙋 and 🙋"` and prints the fourth rune in it as a character, not a number.

编写一个程序，定义一个名为 `message` 的字符串变量，其值为 `"Hi 🙋 and 🙋"`，并将其中的第四个符文作为字符而不是数字打印出来。

3. Write a program that defines a struct called `Employee` with three fields: `firstName`, `lastName`, and `id`. The first two fields are of type `string`, and the last field (`id`) is of type `int`. Create three instances of this struct using whatever values you'd like. Initialize the first one using the struct literal style without names, the second using

the struct literal style with names, and the third with a `var` declaration. Use dot notation to populate the fields in the third struct. Print out all three structs.

编写一个程序，定义一个名为 `Employee` 的结构体，该结构体包含三个字段：`firstName`、`lastName` 和 `id`。前两个字段的类型为 `string`，最后一个字段 (`id`) 的类型为 `int`。创建三个该结构体的实例，并为其赋值。第一个实例使用不带名称的结构体字面量初始化，第二个实例使用带名称的结构体字面量初始化，第三个实例使用 `var` 声明。使用点号表示法填充第三个结构体的字段。打印出所有三个结构体。

Wrapping Up

总结

You've learned a lot about composite types in Go. In addition to learning more about strings, you now know how to use the built-in generic container types: slices and maps. You can also construct your own composite types via structs. In the next chapter, you're going to take a look at Go's control structures: `for`, `if/else`, and `switch`. You will also learn how Go organizes code into blocks and how the different block levels can lead to surprising behavior.

你已经学习了很多关于 Go 语言复合类型的知识。除了进一步了解字符串之外，你现在还知道如何使用内置的泛型容器类型：切片和映射。你还可以通过结构体来构建自己的复合类型。在下一章中，你将了解 Go 的控制结构：`for`、`if/else` 和 `switch`。你还将学习 Go 如何将代码组织成代码块，以及不同的代码块级别如何导致一些意想不到的行为。

Chapter 4. Blocks, Shadows, and Control Structures

第四章 块、阴影和控制结构

Now that I have covered variables, constants, and built-in types, you are ready to look at programming logic and organization. I'll start by explaining blocks and how they control when an identifier is available. Then I'll present Go's control structures: `if`, `for`, and `switch`. Finally, I will talk about `goto` and the one situation when you should use it.

现在我已经讲解了变量、常量和内置类型，接下来就可以学习编程逻辑和组织结构了。首先，我会解释代码块以及它们如何控制标识符的可用性。然后，我会介绍 Go 的控制结构：`if`、`for` 和 `switch`。最后，我会讲解 `goto` 以及它的唯一适用场景。

Blocks

积木

Go lets you declare variables in lots of places. You can declare them outside of functions, as the parameters to functions, and as local variables within

functions.

Go 语言允许你在很多地方声明变量。你可以在函数外部声明变量，也可以将它们作为函数的参数，还可以将它们作为函数内部的局部变量。

NOTE

笔记

So far, you've written only the `main` function, but you'll write functions with parameters in the next chapter.

到目前为止，你只编写了 `main` 函数，但在下一章中你将编写带参数的函数。

Each place where a declaration occurs is called a *block*. Variables, constants, types, and functions declared outside of any functions are placed in the *package* block. You've used `import` statements in your programs to gain access to printing and math functions (and I will talk about them in detail in [Chapter 10](#)). They define names for other packages that are valid for the file that contains the `import` statement. These names are in the *file* block. All the variables defined at the top level of a function (including the parameters to a function) are in a block. Within a function, every set of braces (`{ }`) defines another block, and in a bit you will see that the control structures in Go define blocks of their own.

每个声明发生的地方都称为一个代码块。在任何函数之外声明的变量、常量、类型和函数都放在包代码块中。你在程序中使用了 `import` 语句来访问打印和数学函数（我将在第 10 章详细讨论它们）。它们定义了其他包的名称，这些包对于包含 `import` 语句的文件是有效的。这些名称位于文件代码块中。在函数顶层定义的所有变量（包括函数的参数）都位于一个代码块中。在函数内部，每组大括号（`{}`）定义另一个代码块，稍后你会看到 Go 中的控制结构也定义了它们自己的代码块。

You can access an identifier defined in any outer block from within any inner block. This raises the question: what happens when you have a declaration with the same name as an identifier in a containing block? If you do that, you *shadow* the identifier created in the outer block.

你可以从任何内部代码块访问任何外部代码块中定义的标识符。这就引出了一个问题：当一个声明与包含代码块中的标识符同名时会发生什么？如果这样做，就会覆盖外部代码块中创建的标识符。

Shadowing Variables

阴影变量

Before I explain what shadowing is, let's take a look at some code (see [Example 4-1](#)). You can run it on [The Go Playground](#) or in the

`sample_code/shadow_variables` directory in the [Chapter 4 repository](#).

在解释什么是变量遮蔽之前，我们先来看一些代码（参见示例 4-1）。您可以在 Go Playground 上运行它，也可以在第四章代码库的 `sample_code/shadow_variables` 目录中运行它。

Example 4-1. Shadowing variables

例 4-1. 阴影变量

```
func main() {  
    x := 10  
    if x > 5 {  
        fmt.Println(x)  
        x := 5  
        fmt.Println(x)  
    }  
    fmt.Println(x)  
}
```

Before you run this code, try to guess what it's going to print out:

在运行这段代码之前，试着猜猜它会输出什么内容：

- Nothing prints; the code does not compile
没有任何输出；代码无法编译。
- 10 on line one, 5 on line two, 5 on line three

第一行有10个，第二行有5个，第三行有5个。

- 10 on line one, 5 on line two, 10 on line three

第一行10，第二行5，第三行10

Here's what happens:

事情经过是这样的：

```
10
5
10
```

A *shadowing variable* is a variable that has the same name as a variable in a containing block. For as long as the shadowing variable exists, you cannot access a shadowed variable.

遮蔽变量是指与包含代码块中的变量同名的变量。只要遮蔽变量存在，就无法访问被遮蔽的变量。

In this case, you almost certainly didn't want to create a brand-new `x` inside the `if` statement. Instead, you probably wanted to assign 5 to the `x` declared at the top level of the function block. At the first `fmt.Println` inside the `if` statement, you are able to access the `x` declared at the top level of the function. On the next line, though, you *shadow* `x` by *declaring a new variable with the same name* inside the block created by the `if` statement's body. At the second `fmt.Println`, when you access the

variable named `x`, you get the shadowing variable, which has the value of 5. The closing brace for the `if` statement's body ends the block where the shadowing `x` exists, and at the third `fmt.Println`, when you access the variable named `x`, you get the variable declared at the top level of the function, which has the value of 10. Notice that this `x` didn't disappear or get reassigned; there was just no way to access it once it was shadowed in the inner block.

在这种情况下，你几乎肯定不想在 `if` 语句内部创建一个全新的 `x`。相反，你可能想将 5 赋值给函数块顶层声明的 `x`。在 `if` 语句内部的第一个 `fmt.Println`，你可以访问函数顶层声明的 `x`。然而，在下一行，你通过在 `if` 语句体创建的代码块内声明一个同名的新变量，对 `x` 进行了覆盖。在第二个 `fmt.Println` 处，当你访问名为 `x` 的变量时，你会得到一个被遮蔽的变量，其值为 5。`if` 语句体的右大括号结束了包含遮蔽变量 `x` 的代码块。在第三个 `fmt.Println` 处，当你访问名为 `x` 的变量时，你会得到函数顶层声明的变量，其值为 10。请注意，这个 `x` 并没有消失或被重新赋值；只是在内部代码块中被遮蔽后，就无法再访问它了。

I mentioned in the previous chapter that in some situations I avoid using `:=` because it can make it unclear what variables are being used. That's because it is very easy to accidentally shadow a variable when using `:=`. Remember, you can use `:=` to create and assign to multiple variables at once. Also, not all variables on the lefthand side have to be new for `:=` to be legal. You can use `:=` as long as there is at least one new variable on the

lefthand side. Let's look at another program (see [Example 4-2](#)), which you can find on [The Go Playground](#) or in the `sample_code/shadow_multiple_assignment` directory in the [Chapter 4 repository](#).

我在上一章提到过，在某些情况下我会避免使用 `:=`，因为它可能会让人不清楚正在使用哪些变量。这是因为在使用 `:=` 时很容易意外地覆盖其他变量。记住，你可以使用 `:=` 一次性创建并赋值多个变量。此外，并非等式左侧的所有变量都必须是新变量，`:=` 才合法。只要左侧至少有一个新变量，你就可以使用 `:=`。让我们来看另一个程序（参见示例 4-2），你可以在 Go Playground 或第 4 章代码库的 `sample_code/shadow_multiple_assignment` 目录中找到它。

Example 4-2. Shadowing with multiple assignment

例 4-2. 多重任务的影子学习

```
func main() {  
    x := 10  
    if x > 5 {  
        x, y := 5, 20  
        fmt.Println(x, y)  
    }  
    fmt.Println(x)  
}
```

Running this code gives you the following:

运行这段代码会得到以下结果：

```
5 20
10
```

Although there was an existing definition of `x` in an outer block, `x` was still shadowed within the `if` statement. That's because `:=` reuses only variables that are declared in the current block. When using `:=`, make sure that you don't have any variables from an outer scope on the lefthand side unless you intend to shadow them.

尽管外部代码块中已经定义了 `x`，但 `x` 仍然在 `if` 语句中被覆盖。这是因为 `:=` 只重用当前代码块中声明的变量。使用 `:=` 时，请确保左侧没有来自外部作用域的变量，除非您打算覆盖它们。

You also need to be careful to ensure that you don't shadow a package import. I'll talk more about importing packages in [Chapter 10](#), but you've been importing the `fmt` package to print out results of our programs. Let's see what happens when you declare a variable called `fmt` within your `main` function, as shown in [Example 4-3](#). You can try to run it on [The Go Playground](#) or in the `sample_code/shadow_package_names` directory in the [Chapter 4 repository](#).

你还需要注意不要覆盖其他包的导入。我会在第 10 章详细讲解包的导入，但你之前已经导入了 `fmt` 包来打印程序的运行结果。让我们看看，如果你在 `main` 函数中声明一个名为 `fmt` 的变量会发生什么，如示例 4-3 所示。你可以在 Go Playground 或第 4 章代码库的 `sample_code/shadow_package_names` 目录中运行它。

Example 4-3. Shadowing package names

示例 4-3. 包名称重叠

```
func main() {  
    x := 10  
    fmt.Println(x)  
    fmt := "oops"  
    fmt.Println(fmt)  
}
```

When you try to run this code, you get an error:

当你尝试运行这段代码时，会收到错误提示：

```
fmt.Println undefined (type string has no field or  
method Println)
```

Notice that the problem isn't that you named your variable `fmt`; it's that you tried to access something that the local variable `fmt` didn't have. Once

the local variable `fmt` is declared, it shadows the package named `fmt` in the file block, making it impossible to use the `fmt` package for the rest of the `main` function.

请注意，问题不在于您将变量命名为 `fmt`；而在于您试图访问局部变量 `fmt` 不存在的内容。一旦声明了局部变量 `fmt`，它就会覆盖文件中名为 `fmt` 的包，导致在 `main` 函数的其余部分无法使用 `fmt` 包。

THE UNIVERSE BLOCK

宇宙方块

One more block is a little weird: the universe block. Remember, Go is a small language with only 25 keywords. What's interesting is that the built-in types (like `int` and `string`), constants (like `true` and `false`), and functions (like `make` or `close`) aren't included in that list. Neither is `nil`. So, where are they?

还有一个代码块有点奇怪：universe 代码块。要知道，Go 语言很小，只有 25 个关键字。有趣的是，内置类型（例如 `int` 和 `string`）、常量（例如 `true` 和 `false`）以及函数（例如 `make` 或 `close`）都不在这个列表中。`nil` 也不在。那么，它们去哪儿了呢？

Rather than make them keywords, Go considers these *predeclared identifiers* and defines them in the universe block, which is the block that contains all other blocks.

Go 并没有将这些标识符设为关键字，而是将其视为预先声明的标识符，并在 universe 代码块中定义它们，universe 代码块包含所有其他代码块。

Because these names are declared in the universe block, they can be shadowed in other scopes. You can see this happen by running the code in [Example 4-4](#) on [The Go Playground](#) or in the `sample_code/shadow_true` directory in the [Chapter 4 repository](#).

由于这些名称是在 `universe` 代码块中声明的，因此它们可以在其他作用域中被覆盖。您可以通过运行 Go Playground 上的示例 4-4 代码或第四章代码库中的 `sample_code/shadow_true` 目录来查看这种情况。

Example 4-4. Shadowing `true`

示例 4-4. 阴影 `true`

```
fmt.Println(true)
true := 10
fmt.Println(true)
```

When you run it, you'll see:

运行后，你会看到：

```
true
10
```

You must be very careful to never redefine any identifiers in the universe block. If you accidentally do so, you will get some strange behavior. If you are lucky, you'll get compilation failures. If you are not, you'll have a harder time tracking down the source of your problems.

务必小心，切勿在 `universe` 代码块中重新定义任何标识符。如果不小心这样做，就会出现一些奇怪的行为。运气好的话，只会编译失败。运气不好的话，就很难找到问题的根源。

Since shadowing is useful in a few instances (later chapters will point them out), `go vet` does not report it as a likely error. In [“Using Code-Quality Scanners”](#), you’ll learn about third-party tools that can detect accidental shadowing in your code.

由于代码遮蔽在某些情况下是有用的（后续章节会详细说明），因此 `go vet` 不会将其报告为可能的错误。在“使用代码质量扫描器”一节中，您将了解可以检测代码中意外代码遮蔽的第三方工具。

if

如果

The `if` statement in Go is much like the `if` statement in most programming languages. Because it is such a familiar construct, I’ve used it in previous sample code without worrying that it’d be confusing.

[Example 4-5](#) shows a more complete sample.

Go 语言中的 `if` 语句与大多数编程语言中的 `if` 语句非常相似。由于这是一个非常常见的结构，我在之前的示例代码中使用它时并没有担心会造成混淆。示例 4-5 展示了一个更完整的示例。

Example 4-5. if and else

示例 4-5. if 和 else

```
n := rand.Intn(10)
if n == 0 {
    fmt.Println("That's too low")
} else if n > 5 {
    fmt.Println("That's too big:", n)
} else {
    fmt.Println("That's a good number:", n)
}
```

The most visible difference between `if` statements in Go and other languages is that you don't put parentheses around the condition. But Go adds another feature to `if` statements that helps you better manage your variables.

Go 语言中 `if` 语句与其他语言最明显的区别在于，条件语句不需要加括号。但 Go 语言还为 `if` 语句添加了另一个特性，可以帮助你更好地管理变量。

As I discussed in [“Shadowing Variables”](#), any variable declared within the braces of an `if` or `else` statement exists only within that block. This isn't that uncommon; it is true in most languages. What Go adds is the ability to declare variables that are scoped to the condition and to both the

`if` and `else` blocks. Take a look at [Example 4-6](#), which rewrites our previous example to use this scope.

正如我在“变量阴影”一节中讨论的那样，在 `if` 或 `else` 语句的大括号内声明的任何变量都仅存在于该语句块内。这并不罕见；大多数语言都是如此。Go 语言的特别之处在于，它允许声明作用域同时限定于条件语句和 `if` 以及 `else` 语句块的变量。请看示例 4-6，它重写了我们之前的示例以使用这种作用域。

Example 4-6. Scoping a variable to an `if` statement

示例 4-6. 将变量的作用域限定为 `if` 语句

```
if n := rand.Intn(10); n == 0 {  
    fmt.Println("That's too low")  
} else if n > 5 {  
    fmt.Println("That's too big:", n)  
} else {  
    fmt.Println("That's a good number:", n)  
}
```

Having this special scope is handy. It lets you create variables that are available only where they are needed. Once the series of `if/else` statements ends, `n` is undefined. You can test this by trying to run the code in [Example 4-7](#) on [The Go Playground](#) or in the `sample_code/if_bad_scope` directory in the [Chapter 4 repository](#).

这种特殊作用域非常实用。它允许你创建仅在需要时才可用的变量。当 `if/else` 语句序列结束后, `n` 将变为未定义。你可以在 Go Playground 或第 4 章代码库的 `sample_code/if_bad_scope` 目录中运行示例 4-7 中的代码来验证这一点。

Example 4-7. Out of scope...

例 4-7. 超出范围.....

```
if n := rand.Intn(10); n == 0 {  
    fmt.Println("That's too low")  
} else if n > 5 {  
    fmt.Println("That's too big:", n)  
} else {  
    fmt.Println("That's a good number:", n)  
}  
fmt.Println(n)
```

Attempting to run this code produces a compilation error:

尝试运行此代码会产生编译错误:

```
undefined: n
```

NOTE

笔记

Technically, you can put any *simple statement* before the comparison in an `if` statement—including a function call that doesn't return a value or an assignment of a new value to an existing variable. But don't do this. Use this feature only to define new variables that are scoped to the `if/else` statements; anything else would be confusing.

理论上，你可以在 `if` 语句的比较语句之前放置任何简单的语句——包括不返回值的函数调用或将新值赋给现有变量。但请不要这样做。此功能仅用于定义作用域限定于 `if/else` 语句的新变量；否则会造成混淆。

Also be aware that just like any other block, a variable declared as part of an `if` statement will shadow variables with the same name that are declared in containing blocks.

还要注意，就像任何其他代码块一样，作为 `if` 语句的一部分声明的变量将覆盖包含代码块中声明的同名变量。

for, Four Ways

四种方法

As in other languages in the C family, Go uses a `for` statement to loop. What makes Go different from other languages is that `for` is the *only* looping keyword in the language. Go accomplishes this by using the `for` keyword in four formats:

与 C 语言家族的其他语言一样，Go 使用 `for` 语句进行循环。Go 与其他语言的不同之处在于，`for` 是该语言中唯一的循环关键字。Go 通过使用 `for` 关键字的四种格式来实现这一点：

- A complete, C-style `for`
一个完整的、C 风格的 `for`
- A condition-only `for`
仅限条件 `for`
- An infinite `for`
无限的 `for`
- `for-range`
`for-range`

The Complete for Statement

完整声明

The first `for` loop style is the complete `for` declaration you might be familiar with from C, Java, or JavaScript, as shown in [Example 4-8](#).

第一个 `for` 循环样式是完整的 `for` 声明，您可能在 C、Java 或 JavaScript 中熟悉它，如示例 4-8 所示。

Example 4-8. A complete `for` statement

示例 4-8. 完整的 `for` 声明

```
for i := 0; i < 10; i++ {  
    fmt.Println(i)  
}
```

You might be unsurprised to find that this program prints out the numbers from 0 to 9, inclusive.

你可能不会感到惊讶，因为这个程序会打印出从 0 到 9 的数字，包括 0 和 9。

Just like the `if` statement, the `for` statement does not use parentheses around its parts. Otherwise, it should look familiar. The `if` statement has three parts, separated by semicolons. The first part is an initialization that sets one or more variables before the loop begins. You should remember two important details about the initialization section. First, you *must* use `:=` to initialize the variables; `var` is *not* legal here. Second, just as variable declarations in `if` statements, you can shadow a variable here.

与 `if` 语句一样，`for` 语句的各个部分也没有使用括号。除此之外，它看起来应该很熟悉。`if` 语句由三个部分组成，用分号分隔。第一部分是初始化，在循环开始之前设置一个或多个变量。关于初始化部分，你应该记住两个重要的细节。首先，你必须使用 `:=` 来初始化变

量；这里不能使用 `var`。其次，就像 `if` 语句中的变量声明一样，你可以在这里使用变量遮蔽。

The second part is the comparison. This must be an expression that evaluates to a `bool`. It is checked immediately *before* each iteration of the loop. If the expression evaluates to `true`, the loop body is executed.

第二部分是比较。这必须是一个计算结果为 `bool` 的表达式。它会在每次循环迭代之前立即进行检查。如果表达式的计算结果为 `true`，则执行循环体。

The last part of a standard `for` statement is the increment. You usually see something like `i{plus}{plus}` here, but any assignment is valid. It runs immediately after each iteration of the loop, before the condition is evaluated.

标准 `for` 语句的最后部分是递增操作。通常这里会看到类似 `i{plus}{plus}` 这样的赋值，但任何赋值都是有效的。它会在循环每次迭代之后、条件判断之前立即执行。

Go allows you to leave out one or more of the three parts of the `for` statement. Most commonly, you'll either leave off the initialization if it is based on a value calculated before the loop:

Go 语言允许你省略 `for` 语句中的一个或多个部分。最常见的情况是，如果初始化值基于循环之前计算的值，则可以省略初始化部分：

```
i := 0
for ; i < 10; i++ {
    fmt.Println(i)
}
```

or you'll leave off the increment because you have a more complicated increment rule inside the loop:

或者，由于循环内部的递增规则更复杂，你会省略递增步骤：

```
for i := 0; i < 10; {
    fmt.Println(i)
    if i % 2 == 0 {
        i++
    } else {
        i+=2
    }
}
```

The Condition-Only for Statement

仅声明的条件

When you leave off both the initialization and the increment in a `for` statement, do not include the semicolons. (If you do, `go fmt` will remove

them.) That leaves a `for` statement that functions like the `while` statement found in C, Java, JavaScript, Python, Ruby, and many other languages. It looks like [Example 4-9](#).

如果在 `for` 语句中省略初始化和递增操作，则不要包含分号。（如果包含分号，`go fmt` 会将其删除。）这样就得到了一个 `for` 语句，其功能类似于 C、Java、JavaScript、Python、Ruby 和许多其他语言中的 `while` 语句。它看起来像示例 4-9。

Example 4-9. A condition-only `for` statement

示例 4-9. 仅包含条件的 `for` 语句

```
i := 1
for i < 100 {
    fmt.Println(i)
    i = i * 2
}
```

The Infinite for Statement

无限语句

The third `for` statement format does away with the condition too. Go has a version of a `for` loop that loops forever. If you learned to program in the

1980s, your first program was probably an infinite loop in BASIC that printed HELLO to the screen forever:

第三种 for 语句格式也去掉了条件语句。Go 语言有一个 for 循环，可以无限循环。如果你在 20 世纪 80 年代学习编程，你的第一个程序很可能就是一个 BASIC 无限循环，它会一直向屏幕上打印 HELLO：

```
10 PRINT "HELLO"  
20 GOTO 10
```

[Example 4-10](#) shows the Go version of this program. You can run it locally or try it out on [The Go Playground](#) or in the *sample_code/infinite_for* directory in the [Chapter 4 repository](#).

例 4-10 展示了该程序的 Go 版本。您可以在本地运行它，也可以在 Go Playground 或第 4 章代码库的 *sample_code/infinite_for* 目录中尝试运行它。

Example 4-10. Infinite looping nostalgia

例 4-10. 无限循环的怀旧

```
package main  
  
import "fmt"
```

```
func main() {  
    for {  
        fmt.Println("Hello")  
    }  
}
```

Running this program gives you the same output that filled the screens of millions of Commodore 64s and Apple][s:

运行此程序后，您将看到与数百万台 Commodore 64 和 Apple][电脑屏幕上相同的输出结果：

```
Hello  
Hello  
Hello  
Hello  
Hello  
Hello  
Hello  
...
```

Press Ctrl-C when you are tired of walking down memory lane.

当你厌倦了回忆往事时，按下 Ctrl-C 即可。

NOTE

笔记

If you run [Example 4-10](#) on The Go Playground, you'll find that it will stop execution after a few seconds. As a shared resource, the playground doesn't allow any one program to run for too long.

如果你在 Go Playground 上运行示例 4-10，你会发现它会在几秒钟后停止执行。作为共享资源，Playground 不允许任何一个程序运行时间过长。

break and continue

休息并继续

How do you get out of an infinite `for` loop without using the keyboard or turning off your computer? That's the job of the `break` statement. It exits the loop immediately, just like the `break` statement in other languages. Of course, you can use `break` with any `for` statement, not just the infinite `for` statement.

如何在不使用键盘或关闭计算机的情况下退出无限循环？这就是 `for` 的作用。它能立即退出循环，就像其他语言中的 `break` 语句一样。当然，你可以将 `break` 与任何 `for` 语句一起使用，而不仅仅是无限循环的 `for` 语句。

NOTE

笔记

Go has no equivalent of the `do` keyword in Java, C, and JavaScript. If you want to iterate at least once, the cleanest way is to use an infinite `for` loop that ends with an `if` statement. If you have some Java code, for example, that uses a `do/while` loop:

Go 语言中没有与 Java、C 和 JavaScript 中的 `do` 关键字等效的关键字。如果想要至少迭代一次，最简洁的方法是使用无限循环 `for`，并以 `if` 语句结束。例如，如果您有一些 Java 代码使用了 `do/while` 循环：

```
do {  
    // things to do in the loop  
} while (CONDITION);
```

the Go version looks like this:

Go 版本的代码如下所示：

```
for {  
    // things to do in the loop  
    if !CONDITION {  
        break  
    }  
}
```

Note that the condition has a leading `!` to *negate* the condition from the Java code. The Go code is specifying how to *exit* the loop, while the Java code specifies how to stay in it.

请注意，条件语句开头有 `!`，用于否定 Java 代码中的条件。Go 代码指定如何退出循环，而 Java 代码指定如何继续执行循环。

Go also includes the `continue` keyword, which skips over the rest of the `for` loop's body and proceeds directly to the next iteration. Technically, you don't need a `continue` statement. You could write code like

[Example 4-11](#).

Go 语言还包含 `continue` 关键字，它可以跳过 `for` 循环体的剩余部分，直接进入下一次迭代。严格来说，你不需要 `continue` 语句。你可以编写类似示例 4-11 的代码。

Example 4-11. Confusing code

示例 4-11. 令人困惑的代码

```
for i := 1; i <= 100; i++ {
    if i%3 == 0 {
        if i%5 == 0 {
            fmt.Println("FizzBuzz")
        } else {
            fmt.Println("Fizz")
        }
    } else if i%5 == 0 {
        fmt.Println("Buzz")
    } else {
        fmt.Println(i)
    }
}
```

But this is not idiomatic. Go encourages short `if` statement bodies, as left-aligned as possible. Nested code is more difficult to follow. Using a `continue` statement makes it easier to understand what's going on.

[Example 4-12](#) shows the code from the previous example, rewritten to use `continue` instead.

但这并非惯用写法。Go 鼓励使用简短的 `if` 语句体，并尽可能左对齐。嵌套代码更难理解。使用 `continue` 语句可以更清晰地展现代码的运行机制。示例 4-12 展示了上一个示例中的代码，并改写为使用 `continue`。

Example 4-12. Using `continue` to make code clearer

示例 4-12. 使用 `continue` 使代码更清晰

```
for i := 1; i <= 100; i++ {
    if i%3 == 0 && i%5 == 0 {
        fmt.Println("FizzBuzz")
        continue
    }
    if i%3 == 0 {
        fmt.Println("Fizz")
        continue
    }
    if i%5 == 0 {
        fmt.Println("Buzz")
        continue
    }
}
```

```
    }  
    fmt.Println(i)  
}
```

As you can see, replacing chains of `if/else` statements with a series of `if` statements that use `continue` makes the conditions line up. This improves the layout of your conditions, which means your code is easier to read and understand.

如您所见，将一系列 `if/else` 语句替换为使用 `continue` 的一系列 `if` 语句，可以使条件排列更加清晰。这优化了条件的布局，使您的代码更易于阅读和理解。

The for-range Statement

for-range 语句

The fourth `for` statement format is for iterating over elements in some of Go's built-in types. It is called a `for-range` loop and resembles the iterators found in other languages. This section shows how to use a `for-range` loop with strings, arrays, slices, and maps. When I cover channels in [Chapter 12](#), I will talk about how to use them with `for-range` loops.

第四种 `for` 语句格式用于遍历 Go 语言内置类型的一些元素。它被称为 `for-range` 循环，类似于其他语言中的迭代器。本节将展示如何

将 `for-range` 循环与字符串、数组、切片和映射一起使用。在第 12 章讲解通道时，我将讨论如何将它们与 `for-range` 循环一起使用。

NOTE

笔记

You can use a `for-range` loop only to iterate over the built-in compound types and user-defined types that are based on them.

您只能使用 `for-range` 循环来遍历内置复合类型和基于这些类型的用户定义类型。

First, let's take a look at using a `for-range` loop with a slice. You can try out the code in [Example 4-13](#) on [The Go Playground](#) or in the `forRangeKeyValue` function in `main.go` in the `sample_code/for_range` directory in the [Chapter 4 repository](#).

首先，我们来看看如何使用 `for-range` 循环和切片。您可以在 Go Playground 的示例 4-13 中尝试代码，或者在第四章代码库的 `sample_code/for_range` 目录下的 `main.go` 文件中的 `forRangeKeyValue` 函数中尝试。

Example 4-13. The `for-range` loop

示例 4-13. `for-range` 循环

```
evenVals := []int{2, 4, 6, 8, 10, 12}
for i, v := range evenVals {
```

```
    fmt.Println(i, v)
}
```

Running this code produces the following output:

运行此代码会产生以下输出：

```
0 2
1 4
2 6
3 8
4 10
5 12
```

What makes a `for-range` loop interesting is that you get two loop variables. The first variable is the position in the data structure being iterated, while the second is the value at that position. The idiomatic names for the two loop variables depend on what is being looped over. When looping over an array, slice, or string, an `i` for *index* is commonly used. When iterating through a map, `k` (for *key*) is used instead.

`for-range` 循环的有趣之处在于它使用了两个循环变量。第一个变量表示数据结构中被迭代的位置，第二个变量表示该位置的值。这两个循环变量的命名取决于循环的对象。当遍历数组、切片或字符串时，通常使用 `i` 表示索引。而当遍历映射时，则使用 `k` 表示键。

The second variable is frequently called `v` for *value*, but is sometimes given a name based on the type of the values being iterated. Of course, you can give the variables any names that you like. If the body of the loop contains only a few statements, single-letter variable names work well. For longer (or nested) loops, you'll want to use more descriptive names.

第二个变量通常被命名为 `v` 来表示其值，但有时也会根据迭代值的类型来命名。当然，您可以随意给变量命名。如果循环体只包含少量语句，使用单字母变量名即可。对于较长的（或嵌套的）循环，则需要使用更具描述性的名称。

What if you don't need to use the first variable within your `for-range` loop? Remember, Go requires you to access all declared variables, and this rule applies to the ones declared as part of a `for` loop too. If you don't need to access the key, use an underscore (`_`) as the variable's name. This tells Go to ignore the value.

如果你不需要在 `for-range` 循环中使用第一个变量该怎么办？记住，Go 语言要求你访问所有已声明的变量，这条规则也适用于在 `for` 循环中声明的变量。如果你不需要访问该键，请使用下划线 (`_`) 作为变量名。这样 Go 就会忽略该值。

Let's rewrite the slice ranging code to not print out the position. You can run the code in [Example 4-14](#) on [The Go Playground](#) or in the

`forRangeIgnoreKey` function in *main.go* in the *sample_code/for_range* directory in the [Chapter 4 repository](#).

让我们重写切片取值范围的代码，使其不再打印位置信息。您可以在 Go Playground 的示例 4-14 中运行代码，或者在第四章代码库的 *sample_code/for_range* 目录下的 *main.go* 文件中的 `forRangeIgnoreKey` 函数中运行。

Example 4-14. Ignoring the slice index in a `for-range` loop

示例 4-14. 在 `for-range` 循环中忽略切片索引

```
evenVals := []int{2, 4, 6, 8, 10, 12}
for _, v := range evenVals {
    fmt.Println(v)
}
```

Running this code produces the following output:

运行此代码会产生以下输出：

```
2
4
6
8
```

10

12

TIP

提示

Anytime you are in a situation where a value is returned, but you want to ignore it, use an underscore to hide the value. You'll see the underscore pattern again when I talk about functions in [Chapter 5](#) and packages in [Chapter 10](#).

当你遇到需要返回值但又不想返回的情况时，可以使用下划线来隐藏该值。我会在第 5 章讲解函数以及第 10 章讲解包时再次提到下划线模式。

What if you want the key but don't want the value? In this situation, Go allows you to just leave off the second variable. This is valid Go code:

如果你只需要键而不需要值怎么办？在这种情况下，Go 允许你直接省略第二个变量。以下是有效的 Go 代码：

```
uniqueNames := map[string]bool{"Fred": true,
    "Raul": true, "Wilma": true}
for k := range uniqueNames {
    fmt.Println(k)
}
```

The most common reason for iterating over the key is when a map is being used as a set. In those situations, the value is unimportant. However, you can also leave off the value when iterating over arrays or slices. This is rare, as the usual reason for iterating over a linear data structure is to access the data. If you find yourself using this format for an array or slice, there's an excellent chance that you have chosen the wrong data structure and should consider refactoring.

遍历键最常见的原因是将映射用作集合。在这种情况下，值并不重要。但是，在遍历数组或切片时，也可以省略值。这种情况比较少见，因为遍历线性数据结构的通常目的是为了访问数据。如果您发现自己对数组或切片使用了这种格式，那么很可能您选择了错误的数据结构，应该考虑重构。

NOTE

笔记

When you look at channels in [Chapter 12](#), you'll see a situation where a `for-range` loop returns only a single value each time the loop iterates.

当你查看第 12 章中的通道时，你会看到这样一种情况：`for-range` 循环每次迭代都只返回一个值。

Iterating over maps

遍历地图

There's something interesting about how a `for-range` loop iterates over a map. You can run the code in [Example 4-15](#) on [The Go Playground](#) or in the `sample_code/iterate_map` directory in the [Chapter 4 repository](#).

`for-range` 循环遍历 map 的方式很有趣。你可以在 Go Playground 上运行示例 4-15 中的代码，或者在第四章代码库的 `sample_code/iterate_map` 目录中运行。

Example 4-15. Map iteration order varies

例 4-15. 映射迭代顺序可变

```
m := map[string]int{
    "a": 1,
    "c": 3,
    "b": 2,
}

for i := 0; i < 3; i++ {
    fmt.Println("Loop", i)
    for k, v := range m {
        fmt.Println(k, v)
    }
}
```

```
    }  
}
```

When you build and run this program, the output varies. Here is one possibility:

当你编译并运行这个程序时，输出结果会有所不同。以下是其中一种可能的结果：

```
Loop 0  
c 3  
b 2  
a 1  
Loop 1  
a 1  
c 3  
b 2  
Loop 2  
b 2  
a 1  
c 3
```

The order of the keys and values varies; some runs may be identical. This is actually a security feature. In earlier Go versions, the iteration order for keys in a map was usually (but not always) the same if you inserted the same items into a map. This caused two problems:

键和值的顺序是可变的；有些运行结果可能完全相同。这实际上是一个安全特性。在早期的 Go 版本中，如果向映射中插入相同的项，则映射中键的迭代顺序通常（但不总是）相同。这会导致两个问题：

- People would write code that assumed that the order was fixed, and this code would break at weird times.

人们编写的代码假定顺序是固定的，而这种代码会在奇怪的时间出错。

- If maps always hash items to the exact same values, and you know that a server is storing user data in a map, you can slow down a server with an attack called *Hash DoS* by sending it specially crafted data with keys that all hash to the same bucket.

如果映射始终将项目哈希到完全相同的值，并且你知道服务器正在将用户数据存储在映射中，那么你可以通过向服务器发送特制的数据（所有键的哈希值都指向同一个桶）来使用称为哈希拒绝服务 (Hash DoS) 的攻击来减慢服务器的速度。

To prevent both of these problems, the Go team made two changes to the map implementation. First, they modified the hash algorithm for maps to include a random number that's generated every time a map variable is created. Next, they made the order of a `for-range` iteration over a map vary a bit each time the map is looped over. These two changes make it far harder to implement a Hash DoS attack.

为了防止这两个问题，Go 团队对 map 的实现做了两项改动。首先，他们修改了 map 的哈希算法，使其包含一个每次创建 map 变量时都会生成的随机数。其次，他们让每次遍历 map 时 `for-range` 次迭代的顺序略有不同。这两项改动大大增加了哈希拒绝服务攻击的难度。

NOTE

笔记

This rule has one exception. To make it easier to debug and log maps, the formatting functions (like `fmt.Println`) always output maps with their keys in ascending sorted order.

这条规则有一个例外。为了便于调试和记录映射，格式化函数（例如 `fmt.Println`）总是按键升序输出映射。

Iterating over strings

遍历字符串

As I mentioned earlier, you can also use a string with a `for-range` loop. Let's take a look. You can run the code in [Example 4-16](#) on your computer or on [The Go Playground](#) or in the `sample_code/iterate_string` directory in the [Chapter 4 repository](#).

正如我之前提到的，你也可以使用带有 `for-range` 循环的字符串。我们来看一下。你可以在你的电脑上运行示例 4-16 中的代码，也可以

在 Go Playground 或第 4 章代码库的 sample_code/iterate_string 目录中运行。

Example 4-16. Iterating over strings

例 4-16. 遍历字符串

```
samples := []string{"hello", "apple_π!"}
for _, sample := range samples {
    for i, r := range sample {
        fmt.Println(i, r, string(r))
    }
    fmt.Println()
}
```

The output when the code iterates over the word “hello” has no surprises:

当代码遍历单词“hello”时，输出结果不出所料：

```
0 104 h
1 101 e
2 108 l
3 108 l
4 111 o
```

In the first column is the index; in the second, the numeric value of the letter; and in the third is the numeric value of the letter type converted to a

string.

第一列是索引；第二列是字母的数值；第三列是字母类型转换为字符串后的数值。

Looking at the result for “apple_π!” is more interesting:

查看“apple_π!”的结果更有意思：

```
0 97 a
1 112 p
2 112 p
3 108 l
4 101 e
5 95 _
6 960 π
8 33 !
```

Notice two things about this output. First, notice that the first column skips the number 7. Second, the value at position 6 is 960. That’s far larger than what can fit in a byte. But in [Chapter 3](#), you saw that strings were made out of bytes. What’s going on?

注意这个输出结果的两个问题。首先，第一列跳过了数字 7。其次，位置 6 的值为 960。这远远大于一个字节所能容纳的值。但在第三章中，你学过字符串是由字节组成的。这是怎么回事？

What you are seeing is special behavior from iterating over a string with a `for-range` loop. It iterates over the *runes*, not the *bytes*. Whenever a `for-range` loop encounters a multibyte rune in a string, it converts the UTF-8 representation into a single 32-bit number and assigns it to the value. The offset is incremented by the number of bytes in the rune. If the `for-range` loop encounters a byte that doesn't represent a valid UTF-8 value, the Unicode replacement character (hex value 0xfffd) is returned instead.

您看到的是使用 `for-range` 循环遍历字符串时的特殊行为。它遍历的是符文，而不是字节。每当 `for-range` 循环在字符串中遇到多字节符文时，它会将 UTF-8 表示形式转换为单个 32 位数字并将其赋值给该符文。偏移量会递增符文中的字节数。如果 `for-range` 循环遇到的字节不代表有效的 UTF-8 值，则会返回 Unicode 替换字符（十六进制值 0xfffd）。

TIP

提示

Use a `for-range` loop to access the runes in a string in order. The first variable holds the number of bytes from the beginning of the string, but the type of the second variable is rune.

使用 `for-range` 循环按顺序访问字符串中的符文。第一个变量保存从字符串开头算起的字节数，但第二个变量的类型是符文。

The for-range value is a copy

for 循环的值是副本

You should be aware that each time the `for-range` loop iterates over your compound type, it *copies* the value from the compound type to the value variable. *Modifying the value variable will not modify the value in the compound type.* [Example 4-17](#) shows a quick program to demonstrate this. You can try it out on [The Go Playground](#) or in the `forRangeIsACopy` function in `main.go` in the `sample_code/for_range` directory in the [Chapter 4 repository](#).

需要注意的是，每次 `for-range` 循环遍历复合类型时，它都会将复合类型的值复制到 `value` 变量中。修改 `value` 变量不会修改复合类型的值。示例 4-17 提供了一个简单的程序来演示这一点。您可以在 Go Playground 上尝试，也可以在第四章代码库的 `sample_code/for_range` 目录下的 `main.go` 文件中的 `forRangeIsACopy` 函数中尝试。

Example 4-17. Modifying the value doesn't modify the source

例 4-17. 修改值不会修改源数据

```
evenVals := []int{2, 4, 6, 8, 10, 12}
for _, v := range evenVals {
    v *= 2
}
```

```
}  
fmt.Println(evenVals)
```

Running this code gives the following output:

运行此代码会得到以下输出：

```
[2 4 6 8 10 12]
```

In versions of Go before 1.22, the value variable is created once and is reused on each iteration through the `for` loop. Since Go 1.22, the default behavior is to create a new index and value variable on each iteration through the `for` loop. This change may not seem important, but it prevents a common bug. When I talk about goroutines and `for-range` loops in [“Goroutines, for Loops, and Varying Variables”](#), you’ll see that prior to Go 1.22, if you launched goroutines in a `for-range` loop, you needed to be careful in how you passed the index and value to the goroutines, or you’d get surprisingly wrong results.

在 Go 1.22 之前的版本中，`value` 变量只创建一次，并在 `for` 循环的每次迭代中重复使用。从 Go 1.22 开始，默认行为是在 `for` 循环的每次迭代中创建一个新的索引和 `value` 变量。这个改变看似微不足道，但它避免了一个常见的 bug。在“Goroutines、for 循环和可变变量”一文中，当我讨论 goroutine 和 `for-range` 循环时，你会发现，在 Go 1.22 之前的版本中，如果在 `for-range` 循环中启动 goroutine，

则需要小心地将索引和值传递给 goroutine，否则会得到意想不到的错误结果。

Because this is a backward-breaking change (even if it is a change that eliminates a common bug), you can control whether to enable this behavior by specifying the Go version in the `go` directive in the `go.mod` file for your module. I talk about this in greater detail in [“Using go.mod”](#).

由于这是一个会破坏旧版本兼容性的更改（即使它修复了一个常见错误），您可以通过在模块的 `go.mod` 文件中的 `go` 指令中指定 Go 版本来控制是否启用此行为。我在“使用 `go.mod`”一节中对此进行了更详细的讨论。

Just as with the other three forms of the `for` statement, you can use `break` and `continue` with a `for-range` loop.

就像 `for` 语句的其他三种形式一样，你可以将 `break` 和 `continue` 与 `for-range` 循环一起使用。

Labeling Your for Statements

为您的声明贴标签

By default, the `break` and `continue` keywords apply to the `for` loop that directly contains them. What if you have nested `for` loops and want to exit or skip over an iterator of an outer loop? Let's look at an example.

You're going to modify the earlier string iterating program to stop iterating through a string as soon as it hits a letter "l." You can run the code in [Example 4-18](#) on [The Go Playground](#) or in the `sample_code/for_label` directory in the [Chapter 4 repository](#).

默认情况下, `break` 和 `continue` 关键字应用于直接包含它们的 `for` 循环。如果您有嵌套的 `for` 循环, 并且想要退出或跳过外层循环的某个迭代器, 该怎么办? 我们来看一个例子。您将修改之前的字符串迭代程序, 使其在遇到字母“l”时停止迭代。您可以在 Go Playground 上运行示例 4-18 中的代码, 或者在第四章代码库的 `sample_code/for_label` 目录中运行该代码。

Example 4-18. Labels

例 4-18. 标签

```
func main() {
    samples := []string{"hello", "apple_π!"}
outer:
    for _, sample := range samples {
        for i, r := range sample {
            fmt.Println(i, r, string(r))
            if r == 'l' {
                continue outer
            }
        }
    }
    fmt.Println()
```

```
    }  
}
```

Notice that the label `outer` is indented by `go fmt` to the same level as the surrounding function. Labels are always indented to the same level as the braces for the block. This makes them easier to notice. Running the program gives the following output:

请注意，标签 `outer` 比 `go fmt` 缩进到与周围函数相同的级别。标签始终与代码块的大括号缩进到同一级别。这样更容易注意到它们。运行程序会得到以下输出：

```
0 104 h  
1 101 e  
2 108 l  
0 97 a  
1 112 p  
2 112 p  
3 108 l
```

Nested `for` loops with labels are rare. They are most commonly used to implement algorithms similar to the following pseudocode:

带有标签的嵌套 `for` 循环很少见。它们最常用于实现类似于以下伪代码的算法：

```
outer:
    for _, outerVal := range outerValues {
        for _, innerVal := range outerVal {
            // process innerVal
            if invalidSituation(innerVal) {
                continue outer
            }
        }
        // here we have code that runs only when
all of the
        // innerVal values were successfully
processed
    }
```

Choosing the Right for Statement

选择合适的声明

Now that I've covered all forms of the `for` statement, you might be wondering when to use which format. Most of the time, you're going to use the `for-range` format. A `for-range` loop is the best way to walk through a string, since it properly gives you back runes instead of bytes. You have also seen that a `for-range` loop works well for iterating through slices and maps, and you'll see in [Chapter 12](#) that channels work naturally with `for-range` as well.

既然我已经讲解了所有形式的 `for` 语句，你可能想知道何时使用哪种格式。大多数情况下，你会使用 `for-range` 格式。`for-range` 循环是遍历字符串的最佳方式，因为它能正确地返回符文而不是字节。你也已经看到 `for-range` 循环非常适合遍历切片和映射，并且你会在第 12 章看到通道也能自然地与 `for-range` 配合使用。

TIP

提示

Favor a `for-range` loop when iterating over all the contents of an instance of one of the built-in compound types. It avoids a great deal of boilerplate code that's required when you use an array, slice, or map with one of the other `for` loop styles.

当遍历内置复合类型实例的所有内容时，建议使用 `for-range` 循环。这样可以避免在使用数组、切片或映射以及其他 `for` 循环样式时所需的大量样板代码。

When should you use the complete `for` loop? The best place for it is when you aren't iterating from the first element to the last element in a compound type. While you could use some combination of `if`, `continue`, and `break` within a `for-range` loop, a standard `for` loop is a clearer way to indicate the start and end of your iteration. Compare these two code snippets, both of which iterate over the second through the second-to-last elements in an array. First the `for-range` loop:

何时应该使用完整的 `for` 循环？最佳使用场景是当您不需要遍历复合类型数组中的第一个元素到最后一个元素时。虽然您可以在 `for-`

range 循环中使用 if、continue 和 break 的组合，但标准的 for 循环能更清晰地指示迭代的开始和结束。比较以下两个代码片段，它们都遍历数组中的第二个元素到倒数第二个元素。首先是 for-range 循环：

```
evenVals := []int{2, 4, 6, 8, 10}
for i, v := range evenVals {
    if i == 0 {
        continue
    }
    if i == len(evenVals)-1 {
        break
    }
    fmt.Println(i, v)
}
```

And here's the same code, with a standard for loop:

以下是相同的代码，但使用了标准的 for 循环：

```
evenVals := []int{2, 4, 6, 8, 10}
for i := 1; i < len(evenVals)-1; i++ {
    fmt.Println(i, evenVals[i])
}
```

The standard for loop code is both shorter and easier to understand.

标准的 `for` 循环代码更短，也更容易理解。

WARNING

警告

This pattern does not work for skipping over the beginning of a string. Remember, a standard `for` loop doesn't properly handle multibyte characters. If you want to skip over some of the runes in a string, you need to use a `for-range` loop so that it will properly process runes for you.

这种模式不适用于跳过字符串的开头。请记住，标准的 `for` 循环无法正确处理多字节字符。如果您想跳过字符串中的某些符文，则需要使用 `for-range` 循环，以便它能够正确处理符文。

The remaining two `for` statement formats are used less frequently. The condition-only `for` loop is, like the `while` loop it replaces, useful when you are looping based on a calculated value.

其余两种 `for` 语句格式使用频率较低。仅包含条件的 `for` 循环（与它所替代的 `while` 循环类似）在基于计算值进行循环时非常有用。

The infinite `for` loop is useful in some situations. The body of the `for` loop should always contain a `break` or `return` because you'll rarely want to loop forever. Real-world programs should bound iteration and fail gracefully when operations cannot be completed. As shown previously, an infinite `for` loop can be combined with an `if` statement to simulate the `do` statement that's present in other languages. An infinite `for` loop is

also used to implement some versions of the *iterator* pattern, which you will look at when I review the standard library in [“io and Friends”](#).

在某些情况下，无限循环非常有用。循环体应该始终包含一个 `for` 或 `break`，因为你很少会希望无限循环下去。实际程序应该限制迭代次数，并在操作无法完成时优雅地失败。如前所述，无限循环可以与 `for` 语句结合使用，以模拟其他语言中的 `do` 语句。无限循环也用于实现某些迭代器模式，我将在“io and Friends”一节中回顾标准库时详细介绍。

switch

转变

Like many C-derived languages, Go has a `switch` statement. Most developers in those languages avoid `switch` statements because of their limitations on values that can be switched on and the default fall-through behavior. But Go is different. It makes `switch` statements useful.

与许多源自 C 语言的语言一样，Go 也拥有 `switch` 语句。大多数使用这些语言的开发者都会避免使用 `switch` 语句，因为它们对可启用的值有限制，并且默认行为是向下取整。但 Go 语言有所不同，它使得 `switch` 语句非常有用。

NOTE

笔记

For those readers who are more familiar with Go, I'm going to cover *expression switch* statements in this chapter. I'll discuss *type switch* statements when I talk about interfaces in [Chapter 7](#).

对于比较熟悉 Go 语言的读者，本章将主要讲解表达式 switch 语句。类型 switch 语句将在第七章讨论接口时进行讲解。

At first glance, `switch` statements in Go don't look all that different from how they appear in C/C++, Java, or JavaScript, but there are a few surprises. Let's take a look at a sample `switch` statement. You can run the code in [Example 4-19](#) on [The Go Playground](#) or in the `basicSwitch` function in `main.go` in the `sample_code/switch` directory in the [Chapter 4 repository](#).

乍一看，Go 语言中的 `switch` 语句与 C/C++、Java 或 JavaScript 中的语句并没有太大区别，但实际上有一些不同之处。我们来看一个 `switch` 语句的示例。您可以在 Go Playground 上运行示例 4-19 中的代码，或者在第四章代码库的 `sample_code/switch` 目录下的 `main.go` 函数中的 `basicSwitch` 函数中运行该代码。

Example 4-19. The `switch` statement

示例 4-19. `switch` 语句

```
words := []string{"a", "cow", "smile", "gopher",  
    "octopus", "anthropologist"}  
for _, word := range words {  
    switch size := len(word); size {  
    case 1, 2, 3, 4:  
        fmt.Println(word, "is a short word!")  
    case 5:  
        wordLen := len(word)  
        fmt.Println(word, "is exactly the right  
length:", wordLen)  
    case 6, 7, 8, 9:  
    default:  
        fmt.Println(word, "is a long word!")  
    }  
}
```

When you run this code, you get the following output:

运行这段代码后，你会得到以下输出：

```
a is a short word!  
cow is a short word!
```

```
smile is exactly the right length: 5  
anthropologist is a long word!
```

I'll go over the features of the `switch` statement to explain the output. As is the case with `if` statements, you don't put parentheses around the value being compared in a `switch`. Also as if an `if` statement, you can declare a variable that's scoped to all branches of the `switch` statement. In this case, you are scoping the variable `size` to all cases in the `switch` statement.

我将讲解 `switch` 语句的特性，以便解释其输出结果。与 `if` 语句一样，在 `switch` 语句中，比较的值不需要用括号括起来。同样，与 `if` 语句一样，您可以声明一个作用域涵盖 `switch` 语句所有分支的变量。在本例中，您将变量 `size` 的作用域限定为 `switch` 语句中的所有分支。

All `case` clauses (and the optional `default` clause) are contained inside a set of braces. But you should note that you don't put braces around the contents of the `case` clauses. You can have multiple lines inside a `case` (or `default`) clause, and they are all considered to be part of the same block.

所有 `case` 子句（以及可选的 `default` 子句）都包含在花括号内。但请注意，`case` 子句的内容不需要用花括号括起来。一个 `case`

(或 `default`) 子句中可以包含多行代码，它们都被视为同一个代码块的一部分。

Inside `case 5:`, you declare `wordLen`, a new variable. Since this is a new block, you can declare new variables within it. Just like any other block, any variables declared within a `case` clause's block are visible only within that block.

在 `case 5:` 内部，您声明了 `wordLen`，一个新变量。由于这是一个新代码块，您可以在其中声明新变量。与其他代码块一样，在 `case` 子句代码块中声明的任何变量仅在该代码块内可见。

If you are used to putting a `break` statement at the end of every `case` in your `switch` statements, you'll be happy to notice that they are gone. By default, cases in `switch` statements in Go don't fall through. This is more in line with the behavior in Ruby or (if you are an old-school programmer) Pascal.

如果您习惯在 `switch` 语句中每个 `case` 的末尾添加 `break` 语句，您会很高兴地发现它们已经消失了。默认情况下，Go 语言中 `switch` 语句中的 `case` 不会执行。这与 Ruby 或（如果您是老程序员）Pascal 的行为更为一致。

This prompts the question: if cases don't fall through, what do you do if there are multiple values that should trigger the exact same logic? In Go, you separate multiple matches with commas, as you do when matching 1, 2,

3, and 4 or 6, 7, 8, and 9. That's why you get the same output for both `a` and `cow`.

这就引出了一个问题：如果所有情况都符合逻辑，那么当有多个值应该触发完全相同的逻辑时该怎么办？在 Go 语言中，多个匹配项之间用逗号分隔，就像匹配 1、2、3 和 4 或 6、7、8 和 9 时一样。这就是为什么 `a` 和 `cow` 会得到相同的输出。

Which leads to the next question: if you don't have fall-through, and you have an empty case (as you do in the sample program when the length of your argument is 6, 7, 8, or 9 characters), what happens? In Go, *an empty case means nothing happens*. That's why you don't see any output from your program when you use *octopus* or *gopher* as the parameter.

这就引出了下一个问题：如果没有 fall-through 机制，并且有一个空 case（就像示例程序中参数长度为 6、7、8 或 9 个字符时那样），会发生什么？在 Go 语言中，空 case 意味着什么都不会发生。这就是为什么当你使用 `octopus` 或 `gopher` 作为参数时，程序不会输出任何内容的原因。

TIP

提示

For the sake of completeness, Go does include a `fallthrough` keyword, which lets one case continue on to the next one. Please think twice before implementing an algorithm that uses it. If you find yourself needing to use `fallthrough`, try to restructure your logic to remove the dependencies between cases.

为了完整起见，Go 语言确实包含一个 `fallthrough` 关键字，它允许一个 case 延续到下一个 case。请在实现使用它的算法之前仔细考虑。如果您发现自己需要使用 `fallthrough`，请尝试重构您的逻辑以消除 case 之间的依赖关系。

In the sample program, you are switching on the value of an integer, but that's not all you can do. You can switch on any type that can be compared with `==`, which includes all built-in types except slices, maps, channels, functions, and structs that contain fields of these types.

在示例程序中，您是在对一个整数的值进行切换，但这并非全部功能。您可以对任何可以与 `==` 进行比较的类型进行切换，这包括所有内置类型，但不包括切片、映射、通道、函数以及包含这些类型字段的结构体。

Even though you don't need to put a `break` statement at the end of each `case` clause, you can use them when you want to exit early from a `case`. However, the need for a `break` statement might indicate that you are doing something too complicated. Consider refactoring your code to remove it.

即使你不需要在每个 `case` 子句末尾添加 `break` 语句，但当你想要提前退出 `case` 子句时，可以使用它们。然而，需要 `break` 语句可能表明你的代码过于复杂。考虑重构代码以移除它。

There is one more place where you might find yourself using a `break` statement in a `case` in a `switch` statement. If you have a `switch` statement inside a `for` loop, and you want to break out of the `for` loop, put a label on the `for` statement and put the name of the label on the `break`. If you don't use a label, Go assumes that you want to break out of the `case`. Let's look at a quick example where you want to break out of the `for` loop once it reaches 7. You can run the code in [Example 4-20](#) on [The Go Playground](#) or in the `missingLabel` function in `main.go` in the `sample_code/switch` directory in the [Chapter 4 repository](#).

还有一种情况是，你可能会在 `break` 语句中，在 `case` 语句中，再在 `switch` 语句中，使用 `break` 语句。例如，如果你在 `for` 循环中有一个 `switch` 语句，并且想要跳出 `for` 循环，那么请在 `for` 语句上添加一个标签，并将标签名称添加到 `break` 语句中。如果不使用标签，Go 会默认你想要跳出 `case` 循环。我们来看一个简单的例子，假设你想要在 `for` 循环执行到第 7 步时跳出该循环。你可以在 Go Playground 上运行示例 4-20 中的代码，或者在第四章代码库的 `sample_code/switch` 目录下的 `main.go` 文件中的 `missingLabel` 函数中运行该代码。

Example 4-20. The case of the missing label

例 4-20. 标签缺失的情况

```
func main() {  
    for i := 0; i < 10; i++ {  
        switch i {  
        case 0, 2, 4, 6:  
            fmt.Println(i, "is even")  
        case 3:  
            fmt.Println(i, "is divisible by 3 but  
not 2")  
        case 7:  
            fmt.Println("exit the loop!")  
            break  
        default:  
            fmt.Println(i, "is boring")  
        }  
    }  
}
```

Running this code produces the following output:

运行此代码会产生以下输出：

```
0 is even  
1 is boring
```

```
2 is even
3 is divisible by 3 but not 2
4 is even
5 is boring
6 is even
exit the loop!
8 is boring
9 is boring
```

That's not what is intended. The goal was to break out of the `for` loop when it got a 7, but the `break` was interpreted as exiting the `case`. To resolve this, you need to introduce a label, just as you did when breaking out of a nested `for` loop. First, you label the `for` statement:

这并非预期结果。目标是在 `for` 循环返回 7 时跳出该循环，但 `break` 被错误地解释为退出 `case` 循环。要解决这个问题，你需要添加一个标签，就像你之前跳出嵌套的 `for` 循环时所做的那样。首先，你需要给 `for` 语句添加标签：

```
loop:
    for i := 0; i < 10; i++ {
```

Then you use the label on your break:

然后，你可以在休息时间使用标签：

break loop

You can see these changes on [The Go Playground](#) or in the `labeledBreak` function in `main.go` in the `sample_code/switch` directory in the [Chapter 4 repository](#). When you run it again, you get the expected output:

您可以在 Go Playground 或第四章代码库 `sample_code/switch` 目录下的 `main.go` 文件中的 `labeledBreak` 函数中看到这些更改。再次运行后，您将得到预期的输出：

```
0 is even
1 is boring
2 is even
3 is divisible by 3 but not 2
4 is even
5 is boring
6 is even
exit the loop!
```

Blank Switches

空白开关

You can use `switch` statements in another, more powerful way. Just as Go allows you to leave out parts from a `for` statement's declaration, you can write a `switch` statement that doesn't specify the value that you're comparing against. This is called a *blank switch*. A regular `switch` only allows you to check a value for equality. A blank `switch` allows you to use any boolean comparison for each `case`. You can try out the code in [Example 4-21](#) on [The Go Playground](#) or in the `basicBlankSwitch` function in `main.go` in the `sample_code/blank_switch` directory in the [Chapter 4 repository](#).

您可以使用 `switch` 语句以更强大的方式进行操作。就像 Go 允许您省略 `for` 语句声明中的部分内容一样，您可以编写一个 `switch` 语句，其中不指定要比较的值。这称为空 `switch`。普通的 `switch` 语句仅允许您检查值是否相等。空的 `switch` 语句允许您对每个 `case` 使用任何布尔比较。您可以在 Go Playground 上尝试示例 4-21 中的代码，或者在第四章代码库的 `sample_code/blank_switch` 目录中 `main.go` 文件中的 `basicBlankSwitch` 函数中尝试。

Example 4-21. The blank switch

示例 4-21. 空白 switch

```
words := []string{"hi", "salutations", "hello"}
for _, word := range words {
    switch wordLen := len(word); {
    case wordLen < 5:
        fmt.Println(word, "is a short word!")
    case wordLen > 10:
        fmt.Println(word, "is a long word!")
    default:
        fmt.Println(word, "is exactly the right
length.")
    }
}
```

When you run this program, you get the following output:

运行此程序后，您将得到以下输出：

```
hi is a short word!
salutations is a long word!
hello is exactly the right length.
```

Just as with a regular switch statement, you can optionally include a short variable declaration as part of your blank switch. But unlike a

regular `switch` , you can write logical tests for your cases. Blank switches are pretty cool, but don't overdo them. If you find that you have written a blank `switch` where all your cases are equality comparisons against the same variable:

就像普通的 `switch` 语句一样，你可以在空白的 `switch` 语句中选择性地包含一个简短的变量声明。但与普通的 `switch` 语句不同的是，你可以为你的 case 编写逻辑测试。空白 `switch` 语句非常实用，但不要过度使用。如果你发现自己编写了一个空白的 `switch` 语句，其中所有的 case 都是对同一个变量进行相等性比较：

```
switch {  
case a == 2:  
    fmt.Println("a is 2")  
case a == 3:  
    fmt.Println("a is 3")  
case a == 4:  
    fmt.Println("a is 4")  
default:  
    fmt.Println("a is ", a)  
}
```

you should replace it with an expression `switch` statement:

你应该将其替换为表达式 `switch` 语句：

```
switch a {  
case 2:  
    fmt.Println("a is 2")  
case 3:  
    fmt.Println("a is 3")  
case 4:  
    fmt.Println("a is 4")  
default:  
    fmt.Println("a is ", a)  
}
```

Choosing Between if and switch

在 if 和 switch 之间进行选择

As a matter of functionality, there isn't much difference between a series of `if/else` statements and a blank `switch` statement. Both allow a series of comparisons. So, when should you use `switch`, and when should you use a set of `if` or `if/else` statements? A `switch` statement, even a blank `switch`, indicates that a relationship exists between the values or comparisons in each case. To demonstrate the difference in clarity, rewrite the FizzBuzz program from [Example 4-11](#) using a blank `switch`, as shown in [Example 4-22](#). You can also find this code in the `sample_code/simplest_fizzbuzz` directory in the [Chapter 4 repository](#).

从功能上看，一系列 `if/else` 语句和一个空白的 `switch` 语句之间并没有太大区别。两者都允许进行一系列比较。那么，何时应该使用 `switch`，何时应该使用一组 `if` 或 `if/else` 语句呢？一个 `switch` 语句，甚至一个空白的 `switch`，都表明每个例子中的值或比较之间存在某种关系。为了更清晰地展示这种区别，请使用空白的 `switch` 重写示例 4-11 中的 FizzBuzz 程序，如示例 4-22 所示。您也可以在第 4 章代码库的 `sample_code/simplest_fizzbuzz` 目录中找到这段代码。

Example 4-22. Rewriting a series of `if` statements with a blank `switch`

例 4-22. 将一系列 `if` 语句替换为空的 `switch`

```
for i := 1; i <= 100; i++ {
    switch {
    case i%3 == 0 && i%5 == 0:
        fmt.Println("FizzBuzz")
    case i%3 == 0:
        fmt.Println("Fizz")
    case i%5 == 0:
        fmt.Println("Buzz")
    default:
        fmt.Println(i)
    }
}
```

Most people would agree that this is the most readable version. There's no more need for `continue` statements, and the default behavior is made explicit with the `default` case.

大多数人都会认同这是最易读的版本。不再需要 `continue` 语句，并且使用 `default` 情况明确定义了默认行为。

Of course, nothing in Go prevents you from doing all sorts of unrelated comparisons on each `case` in a blank `switch`. However, this is not idiomatic. If you find yourself in a situation where you want to do this, use a series of `if/else` statements (or perhaps consider refactoring your code).

当然，Go 语言并没有禁止你对空白 `switch` 中的每个 `case` 进行各种不相关的比较。然而，这并非惯用做法。如果你确实需要这样做，请使用一系列 `if/else` 语句（或者考虑重构你的代码）。

TIP

提示

Favor blank `switch` statements over `if/else` chains when you have multiple related cases. Using a `switch` makes the comparisons more visible and reinforces that they are a related set of concerns.

当有多个相关案例时，建议使用空白的 `switch` 语句，而不是 `if/else` 链。使用 `switch` 可以使比较更加清晰，并强化它们是一组相关问题的特征。

goto—Yes, goto

goto—是的, goto

Go has a fourth control statement, but chances are, you will never use it. Ever since Edsger Dijkstra wrote [“Go To Statement Considered Harmful”](#) in 1968, the `goto` statement has been the black sheep of the coding family. There are good reasons for this. Traditionally, `goto` was dangerous because it could jump to nearly anywhere in a program; you could jump into or out of a loop, skip over variable definitions, or into the middle of a set of statements in an `if` statement. This made it difficult to understand what a `goto`-using program did.

Go 语言有第四个控制语句，但你很可能永远不会用到它。自从 Edsger Dijkstra 在 1968 年发表了《Go To 语句有害论》以来，`goto` 语句就成了编程语言中的异类。这其中是有原因的。传统上，`goto` 语句很危险，因为它几乎可以跳转到程序中的任何位置；你可以跳转到循环内部或外部，跳过变量定义，或者跳转到 `if` 语句中的一组语句的中间。这使得理解使用 `goto` 语句的程序究竟在做什么变得困难。

Most modern languages don't include `goto`. Yet Go has a `goto` statement. You should still do what you can to avoid using it, but it has some uses, and the limitations that Go places on it make it a better fit with structured programming.

大多数现代语言都不包含 `goto` 语句。然而，Go 语言却有 `goto` 语句。虽然你仍然应该尽量避免使用它，但它确实有一些用途，而且 Go 语言对它的限制也使其更适合结构化编程。

In Go, a `goto` statement specifies a labeled line of code, and execution jumps to it. However, you can't jump anywhere. Go forbids jumps that skip over variable declarations and jumps that go into an inner or parallel block.

在 Go 语言中，`goto` 语句指定一个带标签的代码行，程序会跳转到该行执行。但是，你不能跳转到任何位置。Go 禁止跳过变量声明以及跳转到内部代码块或并行代码块。

The program in [Example 4-23](#) shows two illegal `goto` statements. You can attempt to run it on [The Go Playground](#) or in the `sample_code/broken_goto` directory in the [Chapter 4 repository](#).

示例 4-23 中的程序显示了两条非法的 `goto` 语句。您可以在 Go Playground 或第 4 章代码库中的 `sample_code/broken_goto` 目录中尝试运行它。

Example 4-23. Go's `goto` has rules

示例 4-23。Go 的 `goto` 有规则

```
func main() {  
    a := 10  
    goto skip
```

```

    b := 20
skip:
    c := 30
    fmt.Println(a, b, c)
    if c > a {
        goto inner
    }
    if a < b {
inner:
        fmt.Println("a is less than b")
    }
}

```

Trying to run this program produces the following errors:

运行此程序时出现以下错误：

```

goto skip jumps over declaration of b at
./main.go:8:4
goto inner jumps into block starting at
./main.go:15:11

```

So what should you use `goto` for? Mostly, you shouldn't. Labeled `break` and `continue` statements allow you to jump out of deeply nested loops or skip iteration. The program in [Example 4-24](#) has a legal `goto` and

demonstrates one of the few valid use cases. You can also find this code in the *sample_code/good_goto* directory in the [Chapter 4 repository](#).

那么，`goto` 应该用来做什么呢？大多数情况下，你不应该使用它。带有标签的 `break` 和 `continue` 语句允许你跳出深度嵌套的循环或跳过迭代。示例 4-24 中的程序使用了合法的 `goto`，并展示了少数几个有效的用例之一。你也可以在第四章代码库的 *sample_code/good_goto* 目录中找到这段代码。

Example 4-24. A reason to use `goto`

示例 4-24. 使用 `goto` 的理由

```
func main() {
    a := rand.Intn(10)
    for a < 100 {
        if a%5 == 0 {
            goto done
        }
        a = a*2 + 1
    }
    fmt.Println("do something when the loop
completes normally")
done:
    fmt.Println("do complicated stuff no matter
why we left the loop")
}
```

```
    fmt.Println(a)
}
```

This example is contrived, but it shows how `goto` can make a program clearer. In this simple case, there is some logic that you don't want to run in the middle of the function, but you do want to run at the end of the function. There are ways to do this without `goto`. You could set up a boolean flag or duplicate the complicated code after the `for` loop instead of having a `goto`, but both approaches have drawbacks. Littering your code with boolean flags to control the logic flow is arguably the same functionality as the `goto` statement, just more verbose. Duplicating complicated code is problematic because it makes your code harder to maintain. These situations are rare, but if you cannot find a way to restructure your logic, using a `goto` like this actually improves your code.

这个例子虽然是人为设计的，但它展示了 `goto` 如何让程序更清晰。在这个简单的例子中，有一些逻辑你不想在函数中间运行，但希望在函数末尾运行。不用 `goto` 也能实现这一点。你可以设置一个布尔标志，或者在 `for` 循环之后复制这段复杂的代码，而不是使用 `goto`，但这两种方法都有缺点。用布尔标志来控制逻辑流程，其功能与 `goto` 语句基本相同，只是更加冗长。复制复杂的代码会带来问题，因为它会增加代码的维护难度。这种情况虽然很少见，但如果你找不到重构逻辑的方法，使用像这样的 `goto` 实际上可以改进你的代码。

If you want to see a real-world example, you can take a look at the `floatBits` method in the file `atof.go` in the `strconv` package in the standard library. It's too long to include in its entirety, but the method ends with this code:

如果你想看一个实际的例子，可以看看标准库中 `strconv` 包的 `atof.go` 文件中的 `floatBits` 方法。它太长了，无法完整地列出，但该方法以以下代码结尾：

```
overflow:
    // ±Inf
    mant = 0
    exp = 1<<flt.expbits - 1 + flt.bias
    overflow = true

out:
    // Assemble bits.
    bits := mant & (uint64(1)<<flt.mantbits - 1)
    bits |= uint64((exp-flt.bias)&(1<<flt.expbits-
1)) << flt.mantbits
    if d.neg {
        bits |= 1 << flt.mantbits << flt.expbits
    }
    return bits, overflow
```

Before these lines, there are several condition checks. Some require the code after the `overflow` label to run, while other conditions require skipping that code and going directly to `out`. Depending on the condition, there are `goto` statements that jump to `overflow` or `out`. You could probably come up with a way to avoid the `goto` statements, but they all make the code harder to understand.

在这些代码行之前，有几个条件检查。有些条件要求执行 `overflow` 标签之后的代码，而另一些条件则要求跳过该代码并直接跳转到 `out`。根据条件的不同，还有 `goto` 语句会跳转到 `overflow` 或 `out`。你或许可以找到方法来避免执行 `goto` 语句，但这样做都会使代码更难理解。

TIP

提示

You should try very hard to avoid using `goto`. But in the rare situations where it makes your code more readable, it is an option.

你应该尽量避免使用 `goto`。但在极少数情况下，如果它能使你的代码更易读，那么使用 `goto` 也是一种选择。

Exercises

练习

Now it's time to apply everything you've learned about control structures and blocks in Go. You can find answers to these exercises in the [Chapter 4 repository](#).

现在是时候运用你所学的关于 Go 语言控制结构和代码块的知识了。你可以在第四章的练习题库中找到这些练习的答案。

1. Write a `for` loop that puts 100 random numbers between 0 and 100 into an `int` slice.

编写一个 `for` 循环，将 0 到 100 之间的 100 个随机数放入 `int` 切片中。

2. Loop over the slice you created in exercise 1. For each value in the slice, apply the following rules:

遍历你在练习 1 中创建的切片。对于切片中的每个值，应用以下规则：

- a. If the value is divisible by 2, print "Two!"

如果该值能被 2 整除，则输出“Two! ”

- b. If the value is divisible by 3, print "Three!"

如果该值能被 3 整除，则打印“三! ”

c. If the value is divisible by 2 and 3, print “Six!”. Don’t print anything else.

如果该值能被 2 和 3 整除，则输出“六！”。其他任何内容都不输出。

d. Otherwise, print “Never mind”.

否则，打印“没关系”。

3. Start a new program. In `main`, declare an `int` variable called `total`. Write a `for` loop that uses a variable named `i` to iterate from 0 (inclusive) to 10 (exclusive). The body of the `for` loop should be as follows:

创建一个新程序。在 `main` 中，声明一个名为 `int` 的变量，并命名为 `total`。编写一个 `for` 循环，使用名为 `i` 的变量从 0（包含）迭代到 10（不包含）。`for` 循环体应如下所示：

```
total := total + i
fmt.Println(total)
```

After the `for` loop, print out the value of `total`. What is printed out? What is the likely bug in this code?

在 `for` 循环结束后，打印出 `total` 的值。打印出来的是什么？这段代码可能存在什么错误？

Wrapping Up

总结

This chapter covered a lot of important topics for writing idiomatic Go. You've learned about blocks, shadowing, and control structures, and how to use them correctly. At this point, you're able to write simple Go programs that fit within the `main` function. It's time to move on to larger programs, using functions to organize your code.

本章涵盖了许多编写地道 Go 代码的重要主题。你已经学习了代码块、代码遮蔽和控制结构，以及如何正确使用它们。现在，你已经能够编写简单的 Go 程序，这些程序可以放入 `main` 函数中。是时候开始编写更大型的程序，并使用函数来组织你的代码了。

Chapter 5. Functions

第五章 函数

So far, your programs have been limited to a few lines in the `main` function. It's time to get bigger. In this chapter, you're going to learn how to write functions in Go and see all the interesting things you can do with them.

到目前为止，你的程序仅限于 `main` 函数中的几行代码。是时候扩展你的程序了。在本章中，你将学习如何在 Go 语言中编写函数，并了解你可以用它们做的所有有趣的事情。

Declaring and Calling Functions

声明和调用函数

The basics of Go functions are familiar to anyone who has programmed in other languages with first-class functions, like C, Python, Ruby, or JavaScript. (Go also has methods, which I'll cover in [Chapter 7](#).) Just as with control structures, Go adds its own twist on function features. Some are improvements, and others are experiments that should be avoided. I'll cover both in this chapter.

对于使用过其他支持一等函数的语言（例如 C、Python、Ruby 或 JavaScript）进行编程的人来说，Go 函数的基础知识并不陌生。（Go 也支持方法，我将在第 7 章介绍。）与控制结构一样，Go 在函数特性方面也做出了自己的一些创新。有些是改进，有些则是应该避免的实验性尝试。本章将对这两方面进行讲解。

You've already seen functions being declared and used. Every Go program starts from a `main` function, and you've been calling the `fmt.Println` function to print to the screen. Since a `main` function doesn't take in parameters or return values, let's see what it looks like when a function does:

你已经见过函数的声明和使用。每个 Go 程序都以 `main` 函数开始，而你一直在调用 `fmt.Println` 函数来向屏幕打印内容。由于 `main` 函数不接受参数或返回值，让我们来看看当函数接受参数或返回值时是什么样子：

```
func div(num int, denom int) int {  
    if denom == 0 {  
        return 0  
    }  
    return num / denom  
}
```

Let's look at all the new things in this code sample. A function declaration has four parts: the keyword `func`, the name of the function, the input parameters, and the return type. The input parameters are listed in parentheses, separated by commas, with the parameter name first and the type second. Go is a typed language, so you must specify the types of the parameters. The return type is written between the input parameter's closing parenthesis and the opening brace for the function body.

让我们来看看这段代码示例中的所有新内容。函数声明包含四个部分：关键字 `func`、函数名、输入参数和返回类型。输入参数列在括号内，用逗号分隔，参数名在前，类型在后。Go 是一种类型化语言，因此必须指定参数的类型。返回类型写在输入参数的右括号和函数体的左大括号之间。

Just like other languages, Go has a `return` keyword for returning values from a function. If a function returns a value, you *must* supply a `return`. If a function returns nothing, a `return` statement is not needed at the end of the function. The `return` keyword is needed in a function that returns nothing only if you are exiting from the function before the last line.

与其他语言一样，Go 语言也使用 `return` 关键字来表示函数的返回值。如果函数返回值，则必须提供 `return`。如果函数不返回任何值，则函数末尾不需要 `return` 语句。对于不返回任何值的函数，只有在最后一行代码执行完毕之前就退出函数时，才需要 `return` 关键字。

The `main` function has no input parameters or return values. When a function has no input parameters, use empty parentheses `( )`. When a function returns nothing, you write nothing between the input parameter's closing parenthesis and the opening brace for the function body:

`main` 函数没有输入参数或返回值。当函数没有输入参数时，请使用空括号 `( )`。当函数不返回任何值时，输入参数的右括号和函数体的左大括号之间不需要写入任何内容。

```
func main() {  
    result := div(5, 2)  
    fmt.Println(result)  
}
```

The code for this simple program is in the *sample_code/simple_div* directory in the [Chapter 5 repository](#).

这个简单程序的代码位于第 5 章代码库的 `sample_code/simple_div` 目录中。

Calling a function should be familiar to experienced developers. On the right side of the `:=`, you call the `div` function with the values 5 and 2. On the left side, you assign the returned value to the variable `result`.

调用函数对于经验丰富的开发人员来说应该很熟悉。在 `:=` 的右侧，您调用 `div` 函数，并传入参数 5 和 2。在左侧，您将返回值赋给变量

result。

TIP

提示

When you have two or more consecutive input parameters of the same type, you can specify the type once for all of them like this:

当您有两个或多个连续的、类型相同的输入参数时，您可以像这样为它们指定一次类型：

```
func div(num, denom int) int {
```

Simulating Named and Optional Parameters

模拟命名参数和可选参数

Before getting to the unique function features that Go has, I'll mention two that Go *doesn't* have: named and optional input parameters. With one exception that I will cover in the next section, you must supply all the parameters for a function. If you want to emulate named and optional parameters, define a struct that has fields that match the desired parameters, and pass the struct to your function. [Example 5-1](#) shows a snippet of code that demonstrates this pattern.

在介绍 Go 语言独特的函数特性之前，我想先提一下 Go 语言缺少的两个特性：命名参数和可选参数。除了下一节会提到的一个例外情况

外，你必须为函数提供所有参数。如果你想模拟命名参数和可选参数，可以定义一个结构体，其字段与所需的参数相匹配，然后将该结构体传递给你的函数。示例 5-1 展示了一段演示这种模式的代码片段。

Example 5-1. Using a struct to simulate named parameters

示例 5-1. 使用结构体模拟命名参数

```
type MyFuncOpts struct {
    FirstName string
    LastName  string
    Age       int
}

func MyFunc(opts MyFuncOpts) error {
    // do something here
}

func main() {
    MyFunc(MyFuncOpts{
        LastName: "Patel",
        Age:      50,
    })
    MyFunc(MyFuncOpts{
        FirstName: "Joe",
        LastName:  "Smith",
    })
}
```

```
    })  
}
```

The code for this program is in the *sample_code/named_optional_parameters* directory in the [Chapter 5 repository](#).

该程序的代码位于第 5 章存储库的 *sample_code/named_optional_parameters* 目录中。

In practice, not having named and optional parameters isn't a limitation. A function shouldn't have more than a few parameters, and named and optional parameters are mostly useful when a function has many inputs. If you find yourself in that situation, your function is quite possibly too complicated.

实际上，没有命名参数和可选参数并不是限制。一个函数不应该有太多参数，命名参数和可选参数主要在函数有很多输入时才有用。如果你发现自己的函数属于这种情况，那么它很可能过于复杂了。

Variadic Input Parameters and Slices

可变输入参数和切片

You've been using `fmt.Println` to print results to the screen and you've probably noticed that it allows any number of input parameters. How does it do that? Like many languages, Go supports *variadic parameters*. The variadic parameter *must* be the last (or only) parameter in the input parameter list. You indicate it with three dots (`...`) *before* the type. The variable that's created within the function is a slice of the specified type. You use it just like any other slice. Let's see how they work by writing a program that adds a base number to a variable number of parameters and returns the result as a slice of `int`. You can run this program on [The Go Playground](#) or in the `sample_code/variadic` directory in the [Chapter 5 repository](#). First, write the variadic function:

您一直使用 `fmt.Println` 将结果打印到屏幕上，并且可能已经注意到它允许任意数量的输入参数。它是如何实现的呢？与许多语言一样，Go 支持可变参数。可变参数必须是输入参数列表中的最后一个（或唯一）参数。您需要在类型前加上三个点（`...`）来表示它。函数内部创建的变量是指定类型的切片。您可以像使用任何其他切片一样使用它。让我们通过编写一个程序来了解它们是如何工作的：该程序将一个基数加到可变数量的参数上，并将结果作为 `int` 的切片返回。您可以在 Go Playground 或第 5 章代码库中的 `sample_code/variadic` 目录中运行此程序。首先，编写可变参数函数：

```
func addTo(base int, vals ...int) []int {
    out := make([]int, 0, len(vals))
    for _, v := range vals {
        out = append(out, base+v)
    }
    return out
}
```

And now call it in a few ways:

现在我们可以用几种不同的方式来称呼它：

```
func main() {
    fmt.Println(addTo(3))
    fmt.Println(addTo(3, 2))
    fmt.Println(addTo(3, 2, 4, 6, 8))
    a := []int{4, 3}
    fmt.Println(addTo(3, a...))
    fmt.Println(addTo(3, []int{1, 2, 3, 4, 5}...))
}
```

As you can see, you can supply however many values you want for the variadic parameter or no values at all. Since the variadic parameter is converted to a slice, you can supply a slice as the input. However, you must put three dots (`...`) *after* the variable or slice literal. If you do not, it is a compile-time error.

如您所见，您可以为可变参数提供任意数量的值，也可以不提供任何值。由于可变参数会被转换为切片，因此您可以将切片作为输入。但是，您必须在变量或切片字面量后添加三个点（`...`）。否则，将会出现编译时错误。

When you build and run this program, you get this output:

编译并运行此程序后，您将得到以下输出：

```
[ ]  
[5]  
[5 7 9 11]  
[7 6]  
[4 5 6 7 8]
```

Multiple Return Values

多个返回值

The first difference that you'll see between Go and other languages is that Go allows for multiple return values. Let's add a small feature to the previous division program. You're going to return both the dividend and the remainder from the function. Here's the updated function:

Go 与其他语言的第一个区别在于，Go 允许返回多个值。让我们在之前的除法程序中添加一个小功能。函数将同时返回被除数和余数。以

下是更新后的函数：

```
func divAndRemainder(num, denom int) (int, int,
error) {
    if denom == 0 {
        return 0, 0, errors.New("cannot divide by
zero")
    }
    return num / denom, num % denom, nil
}
```

There are a few changes to support multiple return values. When a Go function returns multiple values, the types of the return values are listed in parentheses, separated by commas. Also, if a function returns multiple values, you must return all of them, separated by commas. Don't put parentheses around the returned values; that's a compile-time error.

为了支持多个返回值，Go 做了一些改动。当 Go 函数返回多个值时，返回值的类型需要用逗号分隔，并用括号括起来。此外，如果函数返回多个值，则必须返回所有返回值，并用逗号分隔。不要在返回值周围添加括号；这样做会导致编译时错误。

There's something else that you haven't seen yet: creating and returning an `error`. If you want to learn more about errors, skip ahead to [Chapter 9](#). For now, you need to know only that you use Go's multiple return value support to return an `error` if something goes wrong in a function. If the

function completes successfully, you return `nil` for the error's value. By convention, the `error` is always the last (or only) value returned from a function.

还有一些你还没见过的内容：创建并返回一个 `error`。如果你想了解更多关于错误的信息，请跳到第 9 章。现在，你只需要知道，如果函数出错，可以使用 Go 的多返回值支持来返回 `error`。如果函数成功执行，则返回 `nil` 作为错误值。按照惯例，`error` 始终是函数返回的最后一个（或唯一）值。

Calling the updated function looks like this:

调用更新后的函数如下所示：

```
func main() {
    result, remainder, err := divAndRemainder(5,
2)
    if err != nil {
        fmt.Println(err)
        os.Exit(1)
    }
    fmt.Println(result, remainder)
}
```

The code for this program is in the *sample_code/updated_div* directory in the [Chapter 5 repository](#).

该程序的代码位于第 5 章存储库的 `sample_code/updated_div` 目录中。

I talked about assigning multiple values at once in [“var Versus :=”](#). Here you are using that feature to assign the results of a function call to three variables. On the right side of the `:=`, you call the `divAndRemainder` function with the values 5 and 2. On the left side, you assign the returned values to the variables `result`, `remainder`, and `err`. You check to see if there was an error by comparing `err` to `nil`.

我在“var 与 :=”部分讲解了如何一次性赋值多个值。这里你使用了这个功能，将函数调用的结果赋值给三个变量。在 `:=` 的右侧，你调用了 `divAndRemainder` 函数，并传入了值 5 和 2。在左侧，你将返回值分别赋值给变量 `result`、`remainder` 和 `err`。你通过比较 `err` 和 `nil` 来检查是否存在错误。

Multiple Return Values Are Multiple Values

多个返回值就是多个值

If you are familiar with Python, you might think that multiple return values are like Python functions returning a tuple that’s optionally destructured if the tuple’s values are assigned to multiple variables. [Example 5-2](#) shows some sample code run in the Python interpreter.

如果您熟悉 Python，您可能会认为多个返回值类似于 Python 函数返回一个元组，如果该元组的值被赋给多个变量，则可以选择解构该元组。示例 5-2 展示了一些在 Python 解释器中运行的示例代码。

Example 5-2. Multiple return values in Python are destructured tuples

示例 5-2. Python 中的多个返回值是解构元组

```
>>> def div_and_remainder(n,d):
...     if d == 0:
...         raise Exception("cannot divide by zero")
...     return n / d, n % d
>>> v = div_and_remainder(5,2)
>>> v
(2.5, 1)
>>> result, remainder = div_and_remainder(5,2)
>>> result
2.5
>>> remainder
1
```

In Python, you can assign all the values returned to a single variable or to multiple variables. That's not how Go works. You must assign each value returned from a function. If you try to assign multiple return values to one variable, you get a compile-time error.

在 Python 中，你可以将所有返回值赋给一个变量，也可以赋给多个变量。但 Go 语言并非如此。你必须为每个函数返回的值单独赋值。如果你尝试将多个返回值赋给同一个变量，就会出现编译时错误。

Ignoring Returned Values

忽略返回值

What if you call a function and don't want to use all the returned values? As covered in [“Unused Variables”](#), Go does not allow unused variables. If a function returns multiple values, but you don't need to read one or more of the values, assign the unused values to the name `_`. For example, if you weren't going to read `remainder`, you would write the assignment as `result, _, err := divAndRemainder(5, 2)`.

如果你调用一个函数，但并不想使用所有返回值该怎么办？正如“未使用的变量”一节所述，Go 语言不允许存在未使用的变量。如果一个函数返回多个值，但你不需要读取其中一个或多个值，则将这些未使用的值赋给 `_`。例如，如果你不需要读取 `remainder`，则应将赋值语句写成 `result, _, err := divAndRemainder(5, 2)`。

Surprisingly, Go does let you implicitly ignore *all* of the return values for a function. You can write `divAndRemainder(5,2)`, and the returned values are dropped. You have actually been doing this since the earliest examples: `fmt.Println` returns two values, but it is idiomatic to ignore

them. In almost all other cases, you should make it explicit that you are ignoring return values by using underscores.

令人惊讶的是，Go 允许你隐式地忽略函数的所有返回值。你可以写成 `divAndRemainder(5,2)`，这样返回值就会被丢弃。实际上，从最早的示例开始你就一直在这样做：`fmt.Println` 返回两个值，但忽略它们是符合 Go 惯用的写法。在几乎所有其他情况下，你都应该使用下划线来显式地表示你忽略了返回值。

TIP

提示

Use `_` whenever you don't need to read a value that's returned by a function.

当您不需要读取函数返回的值时，请使用 `_`。

Named Return Values

命名返回值

In addition to letting you return more than one value from a function, Go allows you to specify *names* for your return values. Rewrite the `divAndRemainder` function one more time, this time using named return values:

除了允许函数返回多个值之外，Go 还允许你为返回值指定名称。请再次重写 `divAndRemainder` 函数，这次使用命名返回值：

```
func divAndRemainder(num, denom int) (result int,
remainder int, err error) {
    if denom == 0 {
        err = errors.New("cannot divide by zero")
        return result, remainder, err
    }
    result, remainder = num/denom, num%denom
    return result, remainder, err
}
```

When you supply names to your return values, what you are doing is predeclaring variables that you use within the function to hold the return values. They are written as a comma-separated list within parentheses. *You must surround named return values with parentheses, even if there is only a single return value.* Named return values are initialized to their zero values when created. This means that you can return them before any explicit use or assignment.

为返回值命名时，实际上是预先声明了函数内部用于存储返回值的变量。这些变量以逗号分隔的列表形式写在括号内。即使只有一个返回值，也必须用括号将命名的返回值括起来。命名的返回值在创建时会被初始化为零。这意味着你可以在任何显式使用或赋值之前直接返回它们。

TIP

提示

If you want to name only *some* of the return values, you can do so by using `_` as the name for any return values you want to remain nameless.

如果您只想为部分返回值命名，可以使用 `_` 作为您希望保持匿名的任何返回值的名称。

One important thing to note: the name that's used for a named returned value is local to the function; it doesn't enforce any name outside of the function. It is perfectly legal to assign the return values to variables of different names:

需要注意的一点是：用于命名返回值的名称是函数内部的；它不会强制要求在函数外部使用任何特定名称。将返回值赋给不同名称的变量是完全合法的：

```
func main() {  
    x, y, z := divAndRemainder(5, 2)  
    fmt.Println(x, y, z)  
}
```

While named return values can sometimes help clarify your code, they do have some potential corner cases. First is the problem of shadowing. Just as with any other variable, you can shadow a named return value. Be sure that you are assigning to the return value and not to a shadow of it.

虽然命名返回值有时可以帮助你更清晰地表达代码，但它们也存在一些潜在的特殊情况。首先是变量遮蔽的问题。就像其他任何变量一样，命名返回值也可能被遮蔽。务必确保你赋值的对象是返回值本身，而不是它的某个副本。

The other problem with named return values is that you don't have to return them. Let's take a look at another variation on `divAndRemainder`. You can run it on [The Go Playground](#) or in the `divAndRemainderConfusing` function in `main.go` in the `sample_code/named_div` directory in the [Chapter 5 repository](#):

命名返回值的另一个问题是，你并非必须返回它们。让我们来看 `divAndRemainder` 的另一种变体。你可以在 Go Playground 上运行它，也可以在 Chapter 5 代码库的 `sample_code/named_div` 目录下的 `main.go` 文件中的 `divAndRemainderConfusing` 函数中运行它：

```
func divAndRemainder(num, denom int) (result int,
remainder int, err error) {
    // assign some values
    result, remainder = 20, 30
    if denom == 0 {
        return 0, 0, errors.New("cannot divide by
zero")
    }
}
```

```
    return num / denom, num % denom, nil
}
```

Notice that you assigned values to `result` and `remainder` and then returned different values directly. Before running this code, try to guess what happens when 5 and 2 are passed to this function. The result might surprise you:

请注意，您已为 `result` 和 `remainder` 赋值，然后直接返回了不同的值。在运行此代码之前，请尝试猜测当向此函数传递 5 和 2 时会发生什么。结果可能会让您感到惊讶：

```
2 1
```

The values from the `return` statement were returned even though they were never assigned to the named return parameters. That's because the Go compiler inserts code that assigns whatever is returned to the return parameters. The named return parameters give a way to declare an *intent* to use variables to hold the return values, but don't *require* you to use them.

即使 `return` 语句返回的值从未被赋值给指定的返回参数，它们仍然被返回。这是因为 Go 编译器会插入代码，将返回值赋给返回参数。指定的返回参数允许你声明使用变量来保存返回值，但并非强制要求你使用这些变量。

Some developers like to use named return parameters because they provide additional documentation. However, I find them of limited value.

Shadowing makes them confusing, as does simply ignoring them. Named return parameters are essential in one situation. I will talk about that when I cover `defer` later in the chapter.

有些开发者喜欢使用命名返回参数，因为它们可以提供额外的文档。然而，我认为它们的价值有限。参数遮蔽会使它们变得令人困惑，直接忽略它们也会造成同样的问题。命名返回参数在一种情况下是必不可少的。我将在本章后面讲解 `defer` 时详细说明。

Blank Returns—Never Use These!

空白回车符——千万不要用！

If you use named return values, you need to be aware of one severe misfeature in Go: blank (sometimes called *naked*) returns. If you have named return values, you can just write `return` without specifying the values that are returned. This returns the last values assigned to the named return values. Rewrite the `divAndRemainder` function one last time, this time using blank returns:

如果您使用命名返回值，则需要注意 Go 语言的一个严重缺陷：空返回值（有时也称为裸返回值）。如果您使用了命名返回值，则可以直

接写 `return` 而不指定返回值。这将返回分配给命名返回值的最后一个值。请再次重写 `divAndRemainder` 函数，这次使用空返回值：

```
func divAndRemainder(num, denom int) (result int,
remainder int, err error) {
    if denom == 0 {
        err = errors.New("cannot divide by zero")
        return
    }
    result, remainder = num/denom, num%denom
    return
}
```

Using blank returns makes a few additional changes to the function. When there's invalid input, the function returns immediately. Since no values were assigned to `result` and `remainder`, their zero values are returned. If you are returning the zero values for your named return values, be sure they make sense. Also note that you still have to put a `return` at the end of the function. Even though the function is using blank returns, this function returns values. Leaving out `return` is a compile-time error.

使用空返回值会对函数进行一些额外的更改。当输入无效时，函数会立即返回。由于 `result` 和 `remainder` 没有被赋值，因此它们返回零值。如果您要为指定的返回值返回零值，请确保这些值有意义。另请注意，您仍然需要在函数末尾添加 `return`。即使函数使用了空返回值，它仍然会返回值。省略 `return` 会导致编译时错误。

At first, you might find blank returns handy since they allow you to avoid some typing. However, most experienced Go developers consider blank returns a bad idea because they make it harder to understand data flow. Good software is clear and readable; it's obvious what is happening. When you use a blank return, the reader of your code needs to scan back through the program to find the last value assigned to the return parameters to see what is actually being returned.

一开始，你可能会觉得空返回很方便，因为它可以让你省去一些输入。然而，大多数经验丰富的 Go 开发者都认为空返回是个糟糕的做法，因为它会使数据流难以理解。优秀的软件应该清晰易读，让读者一目了然。当你使用空返回时，代码的阅读者需要回溯程序，找到最后赋给返回参数的值，才能知道实际返回的是什么。

WARNING

警告

If your function returns values, *never* use a blank return. It can make it very confusing to figure out what value is actually returned.

如果你的函数会返回值，千万不要使用空的返回值。这会让人很难弄清楚实际返回的值是什么。

Functions Are Values

函数即值

Just as in many other languages, functions in Go are values. The type of a function is built out of the keyword `func` and the types of the parameters and return values. This combination is called the *signature* of the function. Any function that has the exact same number and types of parameters and return values meets the type signature.

与许多其他语言一样，Go 语言中的函数也是值。函数的类型由关键字 `func` 以及参数和返回值的类型构成。这种组合称为函数的签名。任何具有完全相同数量和类型的参数和返回值的函数都满足类型签名。

Since functions are values, you can declare a function variable:

由于函数也是值，因此可以声明一个函数变量：

```
var myFuncVariable func(string) int
```

`myFuncVariable` can be assigned any function that has a single parameter of type `string` and returns a single value of type `int`. Here's a longer example:

`myFuncVariable` 可以赋值给任何具有一个类型为 `string` 的参数并返回一个类型为 `int` 的值的函数。以下是一个更详细的示例：

```
func f1(a string) int {
    return len(a)
}

func f2(a string) int {
    total := 0
    for _, v := range a {
        total += int(v)
    }
    return total
}

func main() {
    var myFuncVariable func(string) int
    myFuncVariable = f1
    result := myFuncVariable("Hello")
    fmt.Println(result)

    myFuncVariable = f2
    result = myFuncVariable("Hello")
    fmt.Println(result)
}
```

You can run this program on [The Go Playground](#) or in the *sample_code/func_value* directory in the [Chapter 5 repository](#). The output is:

您可以在 Go Playground 或第 5 章代码库的 *sample_code/func_value* 目录中运行此程序。输出结果为：

```
5
500
```

The default zero value for a function variable is `nil`. Attempting to run a function variable with a `nil` value results in a panic (which is covered in [“panic and recover”](#)).

函数变量的默认值为 `nil`。尝试使用 `nil` 值运行函数变量会导致 panic（详见“panic and recover”）。

Having functions as values allows you to do some clever things, such as build a simple calculator using functions as values in a map. Let’s see how this works. The following code is available on [The Go Playground](#) or in the *sample_code/calculator* directory in the [Chapter 5 repository](#). First, create a set of functions that all have the same signature:

将函数作为值可以实现一些巧妙的功能，例如使用映射中的函数值构建一个简单的计算器。让我们看看它是如何工作的。以下代码可在 Go

Playground 或第 5 章代码库的 sample_code/calculator 目录中找到。首先，创建一组具有相同签名的函数：

```
func add(i int, j int) int { return i + j }

func sub(i int, j int) int { return i - j }

func mul(i int, j int) int { return i * j }

func div(i int, j int) int { return i / j }
```

Next, create a map to associate a math operator with each function:

接下来，创建一个映射，将数学运算符与每个函数关联起来：

```
var opMap = map[string]func(int, int) int{
    "+": add,
    "-": sub,
    "*": mul,
    "/": div,
}
```

Finally, try out the calculator with a few expressions:

最后，用几个表达式测试一下计算器：

```

func main() {
    expressions := [][]string{
        {"2", "+", "3"},
        {"2", "-", "3"},
        {"2", "*", "3"},
        {"2", "/", "3"},
        {"2", "%", "3"},
        {"two", "+", "three"},
        {"5"},
    }
    for _, expression := range expressions {
        if len(expression) != 3 {
            fmt.Println("invalid expression:",
expression)
            continue
        }
        p1, err := strconv.Atoi(expression[0])
        if err != nil {
            fmt.Println(err)
            continue
        }
        op := expression[1]
        opFunc, ok := opMap[op]
        if !ok {
            fmt.Println("unsupported operator:",
op)

            continue
        }
    }
}

```

```

        p2, err := strconv.Atoi(expression[2])
        if err != nil {
            fmt.Println(err)
            continue
        }
        result := opFunc(p1, p2)
        fmt.Println(result)
    }
}

```

You're using the `strconv.Atoi` function in the standard library to convert a `string` to an `int`. The second value returned by this function is an `error`. Just as before, you check for errors that are returned by functions and handle error conditions properly.

你正在使用标准库中的 `strconv.Atoi` 函数将 `string` 转换为 `int`。该函数返回的第二个值是 `error`。和之前一样，你需要检查函数返回的错误并正确处理错误情况。

You use `op` as the key to the `opMap` map, and assign the value associated with the key to the variable `opFunc`. The type of `opFunc` is `func(int, int) int`. If there wasn't a function in the map associated with the provided key, you print an error message and skip the rest of the loop. You then call the function assigned to the `opFunc` variable with the `p1` and `p2` variables that you decoded earlier. Calling a function in a variable looks just like calling a function directly.

您使用 `op` 作为 `opMap` 映射的键，并将与该键关联的值赋给变量 `opFunc`。`opFunc` 的类型为 `func(int, int) int`。如果映射中没有与提供的键关联的函数，则打印错误消息并跳过循环的剩余部分。然后，您使用之前解码的 `p1` 和 `p2` 变量调用赋给 `opFunc` 变量的函数。在变量中调用函数与直接调用函数的方式完全相同。

When you run this program, you can see the simple calculator at work:

运行此程序后，您可以看到简单的计算器是如何工作的：

```
5
-1
6
0
unsupported operator: %
strconv.Atoi: parsing "two": invalid syntax
invalid expression: [5]
```

NOTE

笔记

Don't write fragile programs. The core logic for this example is relatively short. Of the 22 lines inside the `for` loop, 6 of them implement the actual algorithm, and the other 16 are for error checking and data validation. You might be tempted to not validate incoming data or check errors, but doing so produces unstable, unmaintainable code. Error handling is what separates the professionals from the amateurs.

不要编写脆弱的程序。这个例子的核心逻辑相对较短。在 `for` 循环内的 22 行代码中，有 6 行实现了实际的算法，其余 16 行用于错误检查和数据验证。你可能不想验证传入的数据或检查错误，但这样做会导致代码不稳定且难以维护。错误处理是区分专业人士和业余人士的关键所在。

Function Type Declarations

函数类型声明

Just as you can use the `type` keyword to define a `struct`, you can use it to define a function type too (I'll go into more details on type declarations in [Chapter 7](#)):

就像您可以使用 `type` 关键字来定义 `struct` 一样，您也可以使用它来定义函数类型（我将在第 7 章中详细介绍类型声明）：

```
type opFuncType func(int,int) int
```

You can then rewrite the `opMap` declaration to look like this:

然后您可以将 `opMap` 声明重写如下所示：

```
var opMap = map[string]opFuncType {  
    // same as before  
}
```

You don't have to modify the functions at all. Any function that has two input parameters of type `int` and a single return value of type `int` automatically meets the type and can be assigned as a value in the map.

您完全不需要修改函数。任何具有两个类型为 `int` 的输入参数和一个类型为 `int` 的返回值的函数都自动符合类型要求，并且可以作为映射中的值赋给它。

What's the advantage of declaring a function type? One use is documentation. It's useful to give something a name if you are going to refer to it multiple times. You will see another use in [“Function Types Are a Bridge to Interfaces”](#).

声明函数类型有什么好处？其中一个用途是编写文档。如果你要多次引用某个东西，给它命名就很有用。你将在“函数类型是连接接口的桥梁”一文中看到另一个用途。

Anonymous Functions

匿名函数

You can not only assign functions to variables, but also define new functions within a function and assign them to variables. Here's a program to demonstrate this, which you can run on [The Go Playground](#). The code is also available in the *sample_code/anon_func* directory in the [Chapter 5 repository](#):

你不仅可以将函数赋值给变量，还可以在函数内部定义新函数并将其赋值给变量。以下程序演示了这一点，你可以在 Go Playground 上运行它。该代码也可以在第五章代码库的 *sample_code/anon_func* 目录中找到：

```
func main() {  
    f := func(j int) {  
        fmt.Println("printing", j, "from inside of  
an anonymous function")  
    }  
    for i := 0; i < 5; i++ {  
        f(i)  
    }  
}
```

Inner functions are *anonymous*; they don't have a name. You declare an anonymous function with the keyword `func` immediately followed by the input parameters, the return values, and the opening brace. It is a compile-time error to try to put a function name between `func` and the input parameters.

内部函数是匿名的，它们没有名称。要声明匿名函数，可以使用关键字 `func`，后面紧跟输入参数、返回值和左大括号。尝试在 `func` 和输入参数之间放置函数名会导致编译时错误。

Just like any other function, an anonymous function is called by using parentheses. In this example, you are passing the `i` variable from the `for` loop in here. It is assigned to the `j` input parameter of the anonymous function.

与其他函数一样，匿名函数也需要使用括号调用。在本例中，您将 `i` 变量从 `for` 循环传递到这里。它被赋值给匿名函数的 `j` 输入参数。

The program gives the following output:

程序输出如下：

```
printing 0 from inside of an anonymous function
printing 1 from inside of an anonymous function
printing 2 from inside of an anonymous function
```

```
printing 3 from inside of an anonymous function
printing 4 from inside of an anonymous function
```

You don't have to assign an anonymous function to a variable. You can write them inline and call them immediately. The previous program can be rewritten into this:

你不必将匿名函数赋值给变量。你可以直接在代码中编写匿名函数并立即调用它们。之前的程序可以重写成这样：

```
func main() {
    for i := 0; i < 5; i++ {
        func(j int) {
            fmt.Println("printing", j, "from
inside of an anonymous function")
        }(i)
    }
}
```

You can run this example on [The Go Playground](#) or in the *sample_code/anon_func* directory in the [Chapter 5 repository](#).

您可以在 Go Playground 上运行此示例，也可以在 Chapter 5 存储库的 *sample_code/anon_func* 目录中运行此示例。

Now, this is not something that you would normally do. If you are declaring and executing an anonymous function immediately, you might as well get rid of the anonymous function and just call the code. However, declaring anonymous functions without assigning them to variables is useful in two situations: `defer` statements and launching goroutines. I'll talk about `defer` statements in a bit. Goroutines are covered in [Chapter 12](#).

通常情况下，你不会这样做。如果你要立即声明并执行一个匿名函数，那么还不如直接去掉匿名函数，直接调用代码。然而，在两种情况下，声明匿名函数而不将其赋值给变量是有用的：`defer` 语句和启动 goroutine。稍后我会讨论 `defer` 语句。goroutine 将在第 12 章中介绍。

Since you can declare variables at the package scope, you can also declare package scope variables that are assigned anonymous functions:

由于可以在包作用域内声明变量，因此也可以声明被赋值为匿名函数的包作用域变量：

```
var (
    add = func(i, j int) int { return i + j }
    sub = func(i, j int) int { return i - j }
    mul = func(i, j int) int { return i * j }
    div = func(i, j int) int { return i / j }
)
```

```
func main() {  
    x := add(2, 3)  
    fmt.Println(x)  
}
```

Unlike a normal function definition, you can assign a new value to a package-level anonymous function:

与普通函数定义不同，您可以为包级匿名函数赋新值：

```
func main() {  
    x := add(2, 3)  
    fmt.Println(x)  
    changeAdd()  
    y := add(2, 3)  
    fmt.Println(y)  
}  
  
func changeAdd() {  
    add = func(i, j int) int { return i + j + j }  
}
```

Running this code gives the following output:

运行此代码会得到以下输出：

You can try this sample on [The Go Playground](#). The code is also in the *sample_code/package_level_anon* directory in the [Chapter 5 repository](#).

您可以在 Go Playground 上尝试这个示例。代码也位于第 5 章代码库的 *sample_code/package_level_anon* 目录中。

Before using a package-level anonymous function, be very sure you need this capability. Package-level state should be immutable to make data flow easier to understand. If a function's meaning changes while a program is running, it becomes difficult to understand not just how data flows, but how it is processed.

在使用包级匿名函数之前，请务必确认您确实需要此功能。包级状态应该是不可变的，这样才能使数据流更容易理解。如果函数的含义在程序运行时发生变化，不仅难以理解数据流向，也难以理解数据的处理方式。

Closures

关闭

Functions declared inside functions are special; they are *closures*. This is a computer science word that means that functions declared inside functions are able to access and modify variables declared in the outer function. Let's look at a quick example to see how this works. You can find the code on [The Go Playground](#). The code is also in the *sample_code/closure* directory in the [Chapter 5 repository](#):

在函数内部声明的函数很特殊，它们被称为闭包。这是一个计算机科学术语，指的是在函数内部声明的函数能够访问和修改外部函数中声明的变量。让我们来看一个简单的例子来了解它是如何工作的。你可以在 Go Playground 上找到这段代码。代码也位于第五章代码库的 *sample_code/closure* 目录中：

```
func main() {  
    a := 20  
    f := func() {  
        fmt.Println(a)  
        a = 30  
    }  
    f()  
}
```



```
    fmt.Println(a)
}
```

Running this program gives the following output:

运行此程序会得到以下输出：

```
20
30
```

The anonymous function assigned to `f` can read and write `a`, even though `a` is not passed in to the function.

分配给 `f` 的匿名函数可以读取和写入 `a`，即使 `a` 没有传递给该函数。

Just as with any inner scope, you can shadow a variable inside a closure. If you change the code to:

就像任何内部作用域一样，你可以在闭包内部对变量进行遮蔽。如果你将代码更改为：

```
func main() {
    a := 20
    f := func() {
        fmt.Println(a)
    }
}
```

```
        a := 30
        fmt.Println(a)
    }
    f()
    fmt.Println(a)
}
```

this will print out the following:

这将打印出以下内容：

```
20
30
20
```

Using `:=` instead of `=` inside the closure creates a new `a` that ceases to exist when the closure exits. When working with inner functions, be careful to use the correct assignment operator, especially when multiple variables are on the lefthand side. You can try this code on [The Go Playground](#). The code is also in the `sample_code/closure_shadow` directory in the [Chapter 5 repository](#).

在闭包中使用 `:=` 而不是 `=` 会创建一个新的 `a`，该变量会在闭包退出时消失。使用内部函数时，务必注意使用正确的赋值运算符，尤其是在左侧有多个变量的情况下。您可以在 Go Playground 上尝试这段

代码。该代码也位于第 5 章代码库的 `sample_code/closure_shadow` 目录中。

This inner function and closure stuff might not seem all that useful at first. What benefit do you get from making mini-functions within a larger function? Why does Go have this feature?

乍一看，内部函数和闭包之类的东西似乎没什么用。在一个大函数里创建小函数有什么好处呢？为什么 Go 语言要加入这个特性？

One thing that closures allow you to do is to limit a function's scope. If a function is going to be called from only one other function, but it's called multiple times, you can use an inner function to “hide” the called function. This reduces the number of declarations at the package level, which can make it easier to find an unused name.

闭包的一个作用是限制函数的作用域。如果一个函数只会被另一个函数调用，但却被多次调用，你可以使用内部函数来“隐藏”被调用的函数。这可以减少包级别的声明数量，从而更容易找到未使用的函数名。

If you have a piece of logic that is repeated multiple times within a function, a closure can be used to remove that repetition. I wrote a simple Lisp interpreter with a `Scan` function that processes an input string to find the parts of a Lisp program. It relies on two closures, `buildCurToken`

and `update` , to make the code shorter and easier to understand. You can see it on [GitHub](#).

如果函数中某段逻辑重复出现多次，可以使用闭包来消除这种重复。我编写了一个简单的 Lisp 解释器，其中包含一个名为 `Scan` 的函数，用于处理输入字符串以查找 Lisp 程序的各个部分。它依赖于两个闭包 `buildCurToken` 和 `update` ，使代码更简洁易懂。您可以在 GitHub 上查看。

Closures really become interesting when they are passed to other functions or returned from a function. They allow you to take the variables within your function and use those values *outside* of your function.

当闭包被传递给其他函数或从函数返回时，它们才真正变得有趣起来。闭包允许你获取函数内部的变量，并在函数外部使用这些值。

Passing Functions as Parameters

将函数作为参数传递

Since functions are values and you can specify the type of a function using its parameter and return types, you can pass functions as parameters into functions. If you aren't used to treating functions like data, you might need a moment to think about the implications of creating a closure that references local variables and then passing that closure to another function. It's a very useful pattern that appears several times in the standard library.

由于函数本质上是值，并且可以通过参数类型和返回类型来指定函数的类型，因此可以将函数作为参数传递给其他函数。如果您不习惯将函数视为数据，可能需要花点时间思考一下，创建一个引用局部变量的闭包，然后将该闭包传递给另一个函数会带来哪些影响。这是一个非常有用的模式，在标准库中多次出现。

One example is sorting slices. The `sort` package in the standard library has a function called `sort.Slice`. It takes in any slice and a function that is used to sort the slice that's passed in. Let's see how it works by sorting a slice of a struct using two different fields.

一个例子是对切片进行排序。标准库中的 `sort` 包有一个名为 `sort.Slice` 的函数。它接受一个切片和一个用于对传入的切片进行排序的函数作为参数。让我们通过对一个包含两个不同字段的结构体的切片进行排序来看看它是如何工作的。

NOTE

笔记

The `sort.Slice` function predates the addition of generics to Go, so it does some internal magic to make it work with any kind of slice. I'll talk about this magic more in [Chapter 16](#).

`sort.Slice` 函数早于 Go 语言引入泛型，因此它内部进行了一些特殊处理，使其能够处理任何类型的切片。我将在第 16 章详细讨论这些特殊处理。

Let's see how to use closures to sort the same data different ways. You can run this code on [The Go Playground](#). The code is also in the *sample_code/sort_sample* directory in the [Chapter 5 repository](#). First, define a simple type, a slice of values of that type, and print out the slice:

让我们看看如何使用闭包以不同的方式对相同的数据进行排序。您可以在 Go Playground 上运行这段代码。代码也位于第 5 章代码库的 *sample_code/sort_sample* 目录中。首先，定义一个简单的类型，一个包含该类型值的切片，并打印出该切片：

```
type Person struct {
    FirstName string
    LastName  string
    Age       int
}

people := []Person{
    {"Pat", "Patterson", 37},
    {"Tracy", "Bobdaughter", 23},
    {"Fred", "Fredson", 18},
}

fmt.Println(people)
```

Next, sort the slice by last name and print out the results:

接下来，按姓氏对切片进行排序，并打印出结果：

```
// sort by last name
sort.Slice(people, func(i, j int) bool {
    return people[i].LastName < people[j].LastName
})
fmt.Println(people)
```

The closure that's passed to `sort.Slice` has two parameters, `i` and `j`, but within the closure, `people` is used, so you can sort it by the `LastName` field. In computer science terms, `people` is *captured* by the closure. Next you do the same, sorting by the `Age` field:

传递给 `sort.Slice` 的闭包有两个参数，`i` 和 `j`，但在闭包内部，使用的是 `people`，因此您可以按 `LastName` 字段进行排序。用计算机科学术语来说，`people` 被闭包捕获。接下来，您执行相同的操作，按 `Age` 字段进行排序：

```
// sort by age
sort.Slice(people, func(i, j int) bool {
    return people[i].Age < people[j].Age
})
fmt.Println(people)
```

Running this code gives the following output:

运行此代码会得到以下输出：

```
[{Pat Patterson 37} {Tracy Bobdaughter 23} {Fred  
Fredson 18}]  
[{Tracy Bobdaughter 23} {Fred Fredson 18} {Pat  
Patterson 37}]  
[{Fred Fredson 18} {Tracy Bobdaughter 23} {Pat  
Patterson 37}]
```

The `people` slice is changed by the call to `sort.Slice`. I talk about this briefly in [“Go Is Call by Value”](#) and in more detail in the next chapter.

调用 `sort.Slice` 会改变 `people` 切片。我在“Go 语言是按值调用”一章中简要提及了这一点，并在下一章中进行了更详细的讲解。

TIP

提示

Passing functions as parameters to other functions is often useful for performing different operations on the same kind of data.

将函数作为参数传递给其他函数，通常有助于对同一种数据执行不同的操作。

Returning Functions from Functions

从函数返回函数

In addition to using a closure to pass some function state to another function, you can also return a closure from a function. Let's demonstrate this by writing a function that returns a multiplier function. You can run this program on [The Go Playground](#). The code is also in the *sample_code/makeMult* directory in the [Chapter 5 repository](#). Here is a function that returns a closure:

除了使用闭包将函数状态传递给另一个函数之外，你还可以从函数返回一个闭包。让我们通过编写一个返回乘法函数的函数来演示这一点。你可以在 Go Playground 上运行此程序。代码也位于第 5 章代码库的 *sample_code/makeMult* 目录中。以下是一个返回闭包的函数：

```
func makeMult(base int) func(int) int {  
    return func(factor int) int {  
        return base * factor  
    }  
}
```

And here is how the function is used:

以下是该函数的使用方法：

```
func main() {  
    twoBase := makeMult(2)  
    threeBase := makeMult(3)  
    for i := 0; i < 3; i++ {  
        fmt.Println(twoBase(i), threeBase(i))  
    }  
}
```

Running this program gives the following output:

运行此程序会得到以下输出：

```
0 0  
2 3  
4 6
```

Now that you’ve seen closures in action, you might wonder how often they are used by Go developers. It turns out that they are surprisingly useful. You saw how they are used to sort slices. A closure is also used to efficiently search a sorted slice with `sort.Search`. As for returning closures, you will see this pattern used when you build middleware for a web server in [“Middleware”](#). Go also uses closures to implement resource cleanup, via the `defer` keyword.

既然你已经了解了闭包的实际应用，你可能会好奇 Go 开发者使用闭包的频率有多高。事实证明，闭包的用途非常广泛。你已经看到了如何使用闭包对切片进行排序。闭包还可以用于高效地搜索已排序的切片，例如使用 `sort.Search`。至于返回闭包，你可以在“Middleware”中看到这种模式，它用于构建 Web 服务器的中间件。Go 还使用闭包来实现资源清理，通过 `defer` 关键字来实现。

NOTE

笔记

If you spend any time with programmers who use functional programming languages like Haskell, you might hear the term *higher-order functions*. That's a fancy way to say that a function has a function for an input parameter or a return value. As a Go developer, you are as cool as they are!

如果你和使用 Haskell 等函数式编程语言的程序员交流过，你可能会听到“高阶函数”这个术语。这是一种比较专业的说法，意思是说一个函数对于输入参数或返回值还有一个子函数。作为一名 Go 开发者，你和他们一样酷！

defer

推迟

Programs often create temporary resources, like files or network connections, that need to be cleaned up. This cleanup has to happen, no matter how many exit points a function has, or whether a function

completed successfully or not. In Go, the cleanup code is attached to the function with the `defer` keyword.

程序经常会创建临时资源，例如文件或网络连接，这些资源需要清理。无论函数有多少个退出点，也无论函数是否成功执行，都必须进行清理。在 Go 语言中，清理代码通过 `defer` 关键字附加到函数上。

Let's take a look at how to use `defer` to release resources. You'll do this by writing a simple version of `cat`, the Unix utility for printing the contents of a file. You can't pass in arguments on The Go Playground, but you can find the code for this example in the *sample_code/simple_cat* directory in the [Chapter 5 repository](#):

让我们来看看如何使用 `defer` 来释放资源。你需要编写一个简单的 `cat` 版本，它是 Unix 系统中用于打印文件内容的实用程序。在 Go Playground 中，你无法传递参数，但你可以在第 5 章代码库的 *sample_code/simple_cat* 目录中找到此示例的代码：

```
func main() {
    if len(os.Args) < 2 {
        log.Fatal("no file specified")
    }
    f, err := os.Open(os.Args[1])
    if err != nil {
        log.Fatal(err)
    }
}
```

```

    }
    defer f.Close()
    data := make([]byte, 2048)
    for {
        count, err := f.Read(data)
        os.Stdout.Write(data[:count])
        if err != nil {
            if err != io.EOF {
                log.Fatal(err)
            }
            break
        }
    }
}

```

This example introduces a few new features that I cover in more detail in later chapters. Feel free to read ahead to learn more.

本示例引入了一些新功能，我将在后续章节中详细介绍。欢迎继续阅读以了解更多信息。

First, you make sure that a filename was specified on the command line by checking the length of `os.Args`, a slice in the `os` package. The first value in `os.Args` is the name of the program. The remaining values are the arguments passed to the program. You check that the length of `os.Args` is at least 2 to determine whether the argument to the program

was provided. If it wasn't, use the `Fatal` function in the `log` package to print a message and exit the program. Next, you acquire a read-only file handle with the `Open` function in the `os` package. The second value that's returned by `Open` is an error. If there's a problem opening the file, you print the error message and exit the program. As mentioned earlier, I'll talk about errors in [Chapter 9](#).

首先，通过检查 `os` 包中的切片 `os.Args` 的长度，确保命令行中指定了文件名。`os.Args` 中的第一个值是程序名称，其余值是传递给程序的参数。检查 `os.Args` 的长度是否至少为 2，以确定是否提供了传递给程序的参数。如果没有，则使用 `log` 包中的 `Fatal` 函数打印一条消息并退出程序。接下来，使用 `os` 包中的 `Open` 函数获取一个只读文件句柄。`Open` 返回的第二个值是一个错误。如果打开文件时出现问题，则打印错误消息并退出程序。如前所述，我将在第 9 章讨论错误。

Once you know there is a valid file handle, you need to close it after you use it, no matter how you exit the function. To ensure that the cleanup code runs, you use the `defer` keyword, followed by a function or method call. In this case, you use the `Close` method on the file variable. (I cover `at` methods in Go in [Chapter 7](#).) Normally, a function call runs immediately, but `defer` delays the invocation until the surrounding function exits.

一旦确定存在有效的文件句柄，无论函数如何退出，使用后都必须将其关闭。为了确保清理代码运行，可以使用 `defer` 关键字，后跟函

数或方法调用。在本例中，对文件变量使用 `Close` 方法。（我将在 Go 的第 7 章介绍 `at` 方法。）通常，函数调用会立即执行，但 `defer` 会延迟调用，直到其所在的函数退出。

You read from a file handle by passing a slice of bytes into the `Read` method on a file variable. I'll cover how to use this method in detail in [“io and Friends”](#), but `Read` returns the number of bytes that were read into the slice and an error. If an error occurs, you check whether it's an end-of-file marker. If you are at the end of the file, you use `break` to exit the `for` loop. For all other errors, you report it and exit immediately, using `log.Fatal`. I'll talk a little more about slices and function parameters in [“Go Is Call by Value”](#) and go into details on this pattern when I discuss pointers in the next chapter.

您可以通过将字节切片传递给文件变量的 `Read` 方法来从文件句柄读取数据。我将在“io 和朋友”一节中详细介绍如何使用此方法，但 `Read` 返回已读取到切片中的字节数以及一个错误。如果发生错误，您需要检查它是否是文件结束标记。如果是文件末尾，则使用 `break` 退出 `for` 循环。对于所有其他错误，您需要报告错误并立即退出，使用 `log.Fatal`。我将在“Go 是按值调用”一节中更详细地讨论切片和函数参数，并在下一章讨论指针时详细介绍这种模式。

Building and running the program from within the *simple_cat* directory produces the following result:

从 `simple_cat` 目录下构建并运行程序会产生以下结果：

```
$ go build
$ ./simple_cat simple_cat.go
package main

import (
    "io"
    "log"
    "os"
)
...
```

You should know a few more things about `defer`. First, you can use a function, method, or closure with `defer`. (When I mention functions with `defer`, mentally expand that to “functions, methods, or closures.”)

关于 `defer`，你还需要了解一些其他信息。首先，你可以将函数、方法或闭包与 `defer` 一起使用。（当我提到使用 `defer` 调用函数时，请将其理解为“函数、方法或闭包”。）

You can `defer` multiple functions in a Go function. They run in last-in, first-out (LIFO) order; the last `defer` registered runs first.

你可以在 Go 函数中使用 `defer` 个函数。它们按后进先出 (LIFO) 的顺序运行；最后注册的 `defer` 个函数最先运行。

The code within `defer` functions runs *after* the return statement. As I mentioned, you can supply a function with input parameters to a `defer`. The input parameters are evaluated immediately and their values are stored until the function runs.

`defer` 函数中的代码在 `return` 语句之后执行。正如我之前提到的，你可以向 `defer` 提供一个带有输入参数的函数。这些输入参数会立即被求值，其值会被存储起来，直到函数执行完毕。

Here's a quick example that demonstrates both of these features of `defer`:

以下是一个简单的示例，演示了 `defer` 的这两个特性：

```
func deferExample() int {
    a := 10
    defer func(val int) {
        fmt.Println("first:", val)
    }(a)
    a = 20
    defer func(val int) {
        fmt.Println("second:", val)
    }(a)
    a = 30
    fmt.Println("exiting:", a)
```

```
    return a  
}
```

Running this code gives the following output:

运行此代码会得到以下输出：

```
exiting: 30  
second: 20  
first: 10
```

You can run this on [The Go Playground](#). It is also available in the *sample_code/defer_example* directory in the [Chapter 5 repository](#).

您可以在 Go Playground 上运行此代码。它也可以在第五章代码库的 *sample_code/defer_example* 目录中找到。

NOTE

笔记

You can supply a function that returns values to a `defer`, but there's no way to read those values:

您可以提供一个函数，该函数会将值返回给 `defer`，但是无法读取这些值：

```
func example() {  
    defer func() int {  
        return 2 // there's no way to read this value  
    }()  
}
```

You might be wondering whether there's a way for a deferred function to examine or modify the return values of its surrounding function. There is, and it's the best reason to use named return values. It allows your code to take actions based on an error. When I talk about errors in [Chapter 9](#), I will discuss a pattern that uses `defer` to add contextual information to an error returned from a function. Let's look at a way to handle database transaction cleanup using named return values and `defer`:

你可能想知道延迟函数是否可以检查或修改其外部函数的返回值。答案是肯定的，这也是使用命名返回值的最佳理由。它允许你的代码根据错误采取相应的操作。在第 9 章讨论错误时，我会介绍一种使用 `defer` 为函数返回的错误添加上下文信息的模式。接下来，我们来看看如何使用命名返回值和 `defer` 来处理数据库事务清理：

```

func DoSomeInserts(ctx context.Context, db
*sql.DB, value1, value2 string)
    (err error) {
    tx, err := db.BeginTx(ctx, nil)
    if err != nil {
        return err
    }
    defer func() {
        if err == nil {
            err = tx.Commit()
        }
        if err != nil {
            tx.Rollback()
        }
    }()
    _, err = tx.ExecContext(ctx, "INSERT INTO FOO
(val) values $1", value1)
    if err != nil {
        return err
    }
    // use tx to do more database inserts here
    return nil
}

```

I'm not going to cover Go's database support in this book, but the standard library includes extensive support for databases in the `database/sql` package. In the example function, you create a transaction to do a series of

database inserts. If any of them fails, you want to roll back (not modify the database). If all of them succeed, you want to commit (store the database changes). You use a closure with `defer` to check whether `err` has been assigned a value. If it hasn't, you run `tx.Commit`, which could also return an error. If it does, the value `err` is modified. If any database interaction returned an error, you call `tx.Rollback`.

本书不会详细介绍 Go 的数据库支持，但标准库中的 `database/sql` 包提供了丰富的数据库支持。在示例函数中，您创建了一个事务来执行一系列数据库插入操作。如果其中任何一次插入失败，您需要回滚（不修改数据库）。如果所有插入都成功，您需要提交（保存数据库更改）。您可以使用一个闭包和 `defer` 来检查 `err` 是否已被赋值。如果没有，则运行 `tx.Commit`，该函数也可能返回错误。如果返回错误，则修改值 `err`。如果任何数据库交互返回错误，则调用 `tx.Rollback`。

NOTE

笔记

New Go developers tend to forget the set of parentheses after the braces when specifying a function for `defer`. It is a compile-time error to leave them out, and eventually the habit sets in. It helps to remember that supplying parentheses allows you to specify values that will be passed into the function when it runs.

刚接触 Go 的开发者在为 `defer` 指定函数时，常常会忘记在大括号后面加上括号。省略括号会导致编译时错误，而且这种习惯很容易养成。记住这一点很有帮助：加上括号可以让你指定函数运行时要传递的值。

A common pattern in Go is for a function that allocates a resource to also return a closure that cleans up the resource. In the *sample_code/simple_cat_cancel* directory in the [Chapter 5 repository](#), there is a rewrite of the simple `cat` program that does this. First you write a helper function that opens a file and returns a closure:

在 Go 语言中，一个常见的模式是，分配资源的函数同时返回一个用于清理资源的闭包。在第五章代码库的 *sample_code/simple_cat_cancel* 目录中，有一个对简单的 `cat` 程序的重写版本，它实现了这一功能。首先，你需要编写一个辅助函数来打开文件并返回一个闭包：

```
func getFile(name string) (*os.File, func(),
error) {
    file, err := os.Open(name)
    if err != nil {
        return nil, nil, err
    }
    return file, func() {
        file.Close()
    }, nil
}
```

The helper function returns a file, a function, and an error. That `*` means that a file reference in Go is a pointer. I'll talk more about that in the next chapter.

辅助函数会返回一个文件、一个函数和一个错误。* 表示 Go 语言中的文件引用是一个指针。我将在下一章详细讨论这一点。

Now in `main`, you use the `getFile` function:

现在在 `main` 中，您使用 `getFile` 函数：

```
f, closer, err := getFile(os.Args[1])
if err != nil {
    log.Fatal(err)
}
defer closer()
```

Because Go doesn't allow unused variables, returning the `closer` from the function means that the program will not compile if the function is not called. That reminds the user to use `defer`. As covered earlier, you put parentheses after `closer` when you `defer` it.

由于 Go 语言不允许未使用的变量，因此从函数返回 `closer` 意味着如果该函数未被调用，程序将无法编译。这提醒用户使用 `defer`。如前所述，当您使用 `defer` 时，需要在 `closer` 后面加上括号。

NOTE

笔记

Using `defer` can feel strange if you are used to a language that uses a block within a function to control when a resource is cleaned up, like the `try/catch/finally` blocks in Java, JavaScript, and Python or the `begin/rescue/ensure` blocks in Ruby.

如果您习惯于使用函数中的代码块来控制何时清理资源的语言，例如 Java、JavaScript 和 Python 中的 `try/catch/finally` 代码块或 Ruby 中的 `begin/rescue/ensure` 代码块，那么使用 `defer` 可能会感觉很奇怪。

The downside to these resource-cleanup blocks is that they create another level of indentation in your function, and that makes the code harder to read. It's not just my opinion that nested code is harder to follow. In research described in a [2017 paper in *Empirical Software Engineering*](#), Vard Antinyan et al. discovered that “Of...eleven proposed code characteristics, only two markedly influence complexity growth: the nesting depth and the lack of structure.”

这些资源清理块的缺点在于它们会在函数中增加一层缩进，从而降低代码的可读性。嵌套代码难以理解并非我个人的观点。Vard Antinyan 等人在 2017 年发表于《实证软件工程》（*Empirical Software Engineering*）的一篇论文中指出，“在提出的 11 个代码特征中，只有两个会显著影响代码复杂性的增长：嵌套深度和缺乏结构。”

Research on what makes a program easier to read and understand isn't new. You can find papers that are many decades old, including a [paper from 1983](#) by Richard Miara et al. that tries to figure out the right amount of indentation to use (according to their results, two to four spaces).

关于如何使程序更易于阅读和理解的研究并非新鲜事。你可以找到几十年前的论文，包括 Richard Miara 等人 1983 年发表的一篇论文，该论文试图找出合适的缩进量（根据他们的研究结果，两到四个空格）。

Go Is Call by Value

Go 是按值调用

You might hear people say that Go is a *call-by-value* language and wonder what that means. It means that when you supply a variable for a parameter to a function, Go *always* makes a copy of the value of the variable. Let's take a look. You can run this code on [The Go Playground](#). It is also in the `sample_code/pass_value_type` directory in the [Chapter 5 repository](#). First, you define a simple struct:

你可能听说过 Go 是一种按值调用的语言，并想知道这是什么意思。这意味着当你给函数传递一个变量作为参数时，Go 总是会复制该变量的值。我们来看一看。你可以在 Go Playground 上运行这段代码。它也位于第五章代码库的 `sample_code/pass_value_type` 目录中。首先，定义一个简单的结构体：

```
type person struct {  
    age  int  
    name string  
}
```

Next, you write a function that takes in an `int`, a `string`, and a `person`, and modifies their values:

接下来，编写一个函数，该函数接收 `int`、`string` 和 `person` 三个参数，并修改它们的值：

```
func modifyFails(i int, s string, p person) {  
    i = i * 2  
    s = "Goodbye"  
    p.name = "Bob"  
}
```

You then call this function from `main` and see whether the modifications stick:

然后从 `main` 调用此函数，并查看修改是否生效：

```
func main() {  
    p := person{}  
    i := 2  
    s := "Hello"  
    modifyFails(i, s, p)  
    fmt.Println(i, s, p)  
}
```

As the name of the function indicates, running this code shows that a function won't change the values of the parameters passed into it:

正如函数名称所示，运行此代码表明，函数不会改变传递给它的参数值：

```
2 Hello {0 }
```

I included the `person` struct to show that this isn't true just for primitive types. If you have programming experience in Java, JavaScript, Python, or Ruby, you might find the struct behavior strange. After all, those languages let you modify the fields in an object when you pass an object as a parameter to a function. The reason for the difference is something I will cover when I talk about pointers.

我加入了 `person` 结构体，是为了说明这并非仅适用于基本类型。如果您有 Java、JavaScript、Python 或 Ruby 的编程经验，可能会觉得这种结构体的行为很奇怪。毕竟，这些语言允许您在将对象作为参数传递给函数时修改对象的字段。造成这种差异的原因，我将在讲解指针时进行解释。

The behavior is a little different for maps and slices. Let's see what happens when you try to modify them within a function. You can run this code on [The Go Playground](#). It is also in the `sample_code/pass_map_slice` directory in the [Chapter 5 repository](#). You're going to write a function to modify a map parameter and a function to modify a slice parameter:

映射和切片的行为略有不同。让我们看看在函数中修改它们会发生什么。你可以在 Go Playground 上运行这段代码。它也位于第 5 章代码库的 `sample_code/pass_map_slice` 目录中。你将编写一个函数来修改映射参数，以及一个函数来修改切片参数：

```
func modMap(m map[int]string) {  
    m[2] = "hello"  
    m[3] = "goodbye"  
    delete(m, 1)  
}  
  
func modSlice(s []int) {  
    for k, v := range s {  
        s[k] = v * 2  
    }  
    s = append(s, 10)  
}
```

You then call these functions from `main`:

然后，您从 `main` 调用这些函数：

```
func main() {  
    m := map[int]string{  
        1: "first",  
        2: "second",  
    }  
}
```

```
modMap(m)
fmt.Println(m)

s := []int{1, 2, 3}
modSlice(s)
fmt.Println(s)
}
```

When you run this code, you'll see something interesting:

运行这段代码后，你会看到一些有趣的事情：

```
map[2:hello 3:goodbye]
[2 4 6]
```

For the map, it's easy to explain what happens: any changes made to a map parameter are reflected in the variable passed into the function. For a slice, it's more complicated. You can modify any element in the slice, but you can't lengthen the slice. This is true for maps and slices that are passed directly into functions as well as map and slice fields in structs.

对于映射（map）来说，其工作原理很容易解释：对映射参数所做的任何更改都会反映在传递给函数的变量中。而对于切片（slice）来说，情况则更为复杂。您可以修改切片中的任何元素，但不能增加切

片的长度。这不仅适用于直接传递给函数的映射和切片，也适用于结构体中的映射和切片字段。

This program leads to the question: why do maps and slices behave differently than other types? It's because maps and slices are both implemented with pointers. I'll go into more detail in the next chapter.

这个程序引出了一个问题：为什么映射和切片的行为与其他类型不同？这是因为映射和切片都是用指针实现的。我将在下一章详细解释。

TIP

提示

Every type in Go is a value type. It's just that sometimes the value is a pointer.

Go 语言中的所有类型都是值类型。只不过有时值是指针。

Call by value is one reason that Go's limited support for constants is only a minor handicap. Since variables are passed by value, you can be sure that calling a function doesn't modify the variable whose value was passed in (unless the variable is a slice or map). In general, this is a good thing. It makes it easier to understand the flow of data through your program when functions don't modify their input parameters and instead return newly computed values.

按值传递是 Go 语言对常量支持有限这一缺陷微乎其微的原因之一。由于变量是按值传递的，因此可以确保调用函数不会修改传入的变量值（除非该变量是切片或映射）。通常来说，这是一件好事。当函数不修改其输入参数，而是返回新计算的值时，程序中的数据流更容易理解。

While this approach is easy to understand, at times you need to pass something mutable to a function. What do you do then? That's when you need a pointer.

虽然这种方法很容易理解，但有时你需要将可变对象传递给函数。这时该怎么办呢？这就需要用到指针了。

Exercises

练习

These exercises test your knowledge of functions in Go and how to use them. Solutions are available in the *exercise_solutions* directory in the [Chapter 5 repository](#).

这些练习旨在测试你对 Go 语言函数及其用法的掌握程度。答案可在第五章代码库的 `exercise_solutions` 目录中找到。

1. The simple calculator program doesn't handle one error case: division by zero. Change the function signature for the math operations to return both an `int` and an `error`. In the `div` function, if the divisor is `0`, return `errors.New("division by zero")` for the error. In all other cases, return `nil`. Adjust the `main` function to check for this error.

这个简单的计算器程序无法处理一种错误情况：除以零。修改数学运算的函数签名，使其同时返回 `int` 和 `error`。在 `div` 函数中，如果除数为 `0`，则返回 `errors.New("division by zero")` 表示错误。在所有其他情况下，返回 `nil`。调整 `main` 函数以检查此错误。

2. Write a function called `fileLen` that has an input parameter of type `string` and returns an `int` and an `error`. The function takes in a filename and returns the number of bytes in the file. If there is an error reading the file, return the error. Use `defer` to make sure the file is closed properly.

编写一个名为 `fileLen` 的函数，该函数接受一个类型为 `string` 的输入参数，并返回一个 `int` 和一个 `error`。该函数接收一个文件名作为参数，并返回该文件的字节数。如果读取文件时发生错误，则返回该错误。使用 `defer` 来确保文件已正确关闭。

3. Write a function called `prefixer` that has an input parameter of type `string` and returns a function that has an input parameter of type `string` and returns a `string`. The returned function should prefix

its input with the string passed into `prefixer`. Use the following

`main` function to test `prefixer`:

编写一个名为 `prefixer` 的函数，该函数有一个类型为 `string` 的输入参数，并返回一个函数，该函数有一个类型为 `string` 的输入参数，并返回一个 `string` 的函数。返回的函数应该在其输入参数前加上传递给 `prefixer` 的字符串作为前缀。使用以下 `main` 函数来测试 `prefixer`:

```
func main() {  
    helloPrefix := prefixer("Hello")  
    fmt.Println(helloPrefix("Bob")) // should  
    print Hello Bob  
    fmt.Println(helloPrefix("Maria")) //  
    should print Hello Maria  
}
```

Wrapping Up

总结

In this chapter, you've looked at functions in Go, how they are similar to functions in other languages, and their unique features. In the next chapter, you're going to look at pointers, find out that they aren't nearly as scary as

many new Go developers expect them to be, and learn how to take advantage of them to write efficient programs.

在本章中，你已经了解了 Go 语言中的函数，它们与其他语言中的函数有何相似之处，以及各自的独特特性。在下一章中，你将学习指针，发现它们远没有许多 Go 新手开发者想象的那么可怕，并学习如何利用指针编写高效的程序。

Chapter 6. Pointers

第六章 指针

Now that you've seen variables and functions, it's time to learn about pointer syntax. Then I'll clarify the behavior of pointers in Go by comparing them to the behavior of classes in other languages. You'll also learn how and when to use pointers, how memory is allocated in Go, and how using pointers and values properly makes Go programs faster and more efficient.

既然你已经了解了变量和函数，现在就该学习指针语法了。接下来，我会通过与其他语言中类的行为进行比较，来阐明 Go 语言中指针的行为。你还将学习如何以及何时使用指针，Go 语言中内存的分配方式，以及如何正确使用指针和值来提高 Go 程序的运行速度和效率。

A Quick Pointer Primer

快速入门指南

A *pointer* is a variable that holds the location in memory where a value is stored. If you've taken computer science courses, you might have seen a graphic to represent the way variables are stored in memory. The

representation of the following two variables would look something like [Figure 6-1](#):

指针是一种变量，它保存着内存中存储值的位置。如果你学过计算机科学课程，你可能见过用图形来表示变量在内存中存储方式的图示。以下两个变量的表示方法类似于图 6-1：

```
var x int32 = 10
var y bool = true
```

Value	0	0	0	10	1
Address	1	2	3	4	5
Variable	x				y

Figure 6-1. Storing two variables in memory

图 6-1. 在内存中存储两个变量

Every variable is stored in one or more contiguous memory locations, called *addresses*. Different types of variables can take up different amounts of memory. In this example, you have two variables, `x`, which is a 32-bit int, and `y`, which is a boolean. Storing a 32-bit int requires four bytes, so the value for `x` is stored in four bytes, starting at address 1 and ending at address 4. A boolean requires only a single byte (you need only a bit to

represent `true` or `false`, but the smallest amount of memory that can be independently addressed is a byte), so the value for `y` is stored in one byte at address 5, with `true` represented by the value 1.

每个变量都存储在一个或多个连续的内存位置，称为地址。不同类型的变量占用的内存量不同。在这个例子中，有两个变量：`x`，它是一个 32 位整数；以及 `y`，它是一个布尔值。存储一个 32 位整数需要四个字节，因此 `x` 的值存储在四个字节中，从地址 1 开始，到地址 4 结束。一个布尔值只需要一个字节（表示 `true` 或 `false` 只需要一位，但可独立寻址的最小内存单位是一个字节），因此 `y` 的值存储在地址 5 的一个字节中，而 `true` 的值则用 1 表示。

A pointer is a variable that contains the address where another variable is stored. [Figure 6-2](#) demonstrates how the following pointers are stored in memory:

指针是一种变量，它包含另一个变量的存储地址。图 6-2 展示了以下指针在内存中的存储方式：

```
var x int32 = 10
var y bool = true
pointerX := &x
pointerY := &y
var pointerZ *string
```

Value	0	0	0	10	1	0	0	0	1	0	0	0	5	0	0	0	0
Address	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Variable	x				y	pointerX				pointerY				pointerZ			

Figure 6-2. Storing pointers in memory

图 6-2. 将指针存储在内存中

While different types of variables can take up different numbers of memory locations, every pointer, no matter what type it is pointing to, is always the same number of memory locations. The examples in this chapter use 4-byte pointers, but many modern computers use 8 bytes for a pointer. A pointer holds a number that indicates the location in memory where the data being pointed to is stored. That number is called the *address*. Our pointer to `x`, `pointerX`, is stored at location 6 and has the value 1, the address of `x`. Similarly, our pointer to `y`, `pointerY`, is stored at location 10 and has the value 5, the address of `y`. The last pointer, `pointerZ`, is stored at location 14 and has the value 0, because it doesn't point to anything.

虽然不同类型的变量占用的内存空间数量可能不同，但每个指针，无论指向什么类型，始终占用相同数量的内存空间。本章示例使用 4 字节指针，但许多现代计算机使用 8 字节指针。指针存储一个数字，指示其指向的数据在内存中的存储位置。这个数字称为地址。指向 `x` 的指针 `pointerX` 存储在地址 6，值为 1，即 `x` 的地址。类似地，指向 `y` 的指针 `pointerY` 存储在地址 10，值为 5，即 `y` 的地址。最后一个指针 `pointerZ` 存储在地址 14，值为 0，因为它没有指向任何内容。

The zero value for a pointer is `nil`. You've seen `nil` a few times before, as the zero value for slices, maps, and functions. All these types are implemented with pointers. (Two more types, channels and interfaces, are also implemented with pointers. You'll look at them in detail in [“A Quick Lesson on Interfaces”](#) and [“Channels”](#).) As I covered in [Chapter 3](#), `nil` is an untyped identifier that represents the lack of a value for certain types. Unlike `NULL` in C, `nil` is not another name for 0; you can't convert it back and forth with a number.

指针的零值是 `nil`。你之前已经见过 `nil` 几次了，它代表切片、映射和函数的零值。所有这些类型都是用指针实现的。（通道和接口这两种类型也是用指针实现的。你将在“接口速成课”和“通道”中详细了解它们。）正如我在第 3 章中提到的，`nil` 是一个无类型标识符，表示某些类型没有值。与 C 语言中的 `NULL` 不同，`nil` 不是 0 的另一种表示方式；你不能将它与数字相互转换。

WARNING

警告

As alluded to in [Chapter 4](#), `nil` is defined in the universe block. Because `nil` is a value defined in the universe block, it can be shadowed. Never name a variable or function `nil`, unless you are trying to trick your coworker and are unconcerned about your annual review.

如第四章所述，`nil` 定义在 universe 代码块中。由于 `nil` 是 universe 代码块中定义的值，因此它可以被覆盖。除非你想戏弄同事并且不在乎年终考核，否则千万不要将变量或函数命名为 `nil`。

Go's pointer syntax is partially borrowed from C and C++. Since Go has a garbage collector, most memory management pain is removed.

Furthermore, some tricks that you can do with pointers in C and C++, including *pointer arithmetic*, are not allowed in Go.

Go 的指针语法部分借鉴自 C 和 C++。由于 Go 拥有垃圾回收机制，因此大部分内存管理方面的难题都迎刃而解。此外，一些在 C 和 C++ 中可以使用的指针技巧，例如指针运算，在 Go 中是不允许的。

NOTE

笔记

The Go standard library does have an `unsafe` package that lets you do some low-level operations on data structures. While pointer manipulation is used in C for common operations, it is exceedingly rare for Go developers to use `unsafe`. You'll take a quick look at it in [Chapter 16](#).

Go 标准库确实有一个 `unsafe` 包，允许你对数据结构进行一些底层操作。虽然指针操作在 C 语言中用于常见操作，但 Go 开发者极少使用 `unsafe`。你将在第 16 章中快速了解它。

The `&` is the *address* operator. It precedes a value type and returns the address where the value is stored:

`&` 是地址运算符。它位于值类型之前，并返回存储该值的地址：

```
x := "hello"
pointerToX := &x
```


The `*` is the *indirection* operator. It precedes a variable of pointer type and returns the pointed-to value. This is called *dereferencing*:

`*` 是间接引用运算符。它位于指针类型变量之前，并返回指向的值。这称为解引用：

```
x := 10
pointerToX := &x
fmt.Println(pointerToX) // prints a memory
                        address
fmt.Println(*pointerToX) // prints 10
z := 5 + *pointerToX
fmt.Println(z)           // prints 15
```

Before dereferencing a pointer, you must make sure that the pointer is non-`nil`. Your program will panic if you attempt to dereference a `nil` pointer:

在解引用指针之前，必须确保指针不为空。如果尝试解引用 `nil` 指针，程序将会崩溃：

```
var x *int
fmt.Println(x == nil) // prints true
fmt.Println(*x)       // panics
```

A *pointer type* is a type that represents a pointer. It is written with a `*` before a type name. A pointer type can be based on any type:

指针类型是表示指针的类型。它在类型名称前加上 `*` 即可。指针类型可以基于任何类型的类型：

```
x := 10
var pointerToX *int
pointerToX = &x
```

The built-in function `new` creates a pointer variable. It returns a pointer to a zero-value instance of the provided type:

内置函数 `new` 创建一个指针变量。它返回指向所提供类型的零值实例的指针：

```
var x = new(int)
fmt.Println(x == nil) // prints false
fmt.Println(*x)       // prints 0
```

The `new` function is rarely used. For structs, use an `&` before a struct literal to create a pointer instance. You can't use an `&` before a primitive literal (numbers, booleans, and strings) or a constant because they don't have memory addresses; they exist only at compile time. When you need a pointer to a primitive type, declare a variable and point to it:

`new` 函数很少使用。对于结构体，在结构体字面量前使用 `&` 可以创建指针实例。不能在基本类型字面量（数字、布尔值和字符串）或常

量前使用 `&`，因为它们没有内存地址；它们只存在于编译时。当需要指向基本类型的指针时，请声明一个变量并指向它：

```
x := &Foo{}  
var y string  
z := &y
```

Not being able to take the address of a constant is sometimes inconvenient. If you have a struct with a field of a pointer to a primitive type, you can't assign a literal directly to the field:

无法获取常量的地址有时会很不方便。例如，如果一个结构体中包含一个指向原始类型的指针字段，则不能直接将字面量赋值给该字段：

```
type person struct {  
    FirstName string  
    MiddleName *string  
    LastName  string  
}  
  
p := person{  
    FirstName: "Pat",  
    MiddleName: "Perry", // This line won't compile  
    LastName:  "Peterson",  
}
```

Compiling this code returns the error:

编译这段代码会返回错误:

```
cannot use "Perry" (type string) as type *string  
in field value
```

If you try to put an `&` before `"Perry"`, you'll get the error message:

如果你尝试在 `&` 前面加上 `"Perry"`, 你会收到以下错误信息:

```
cannot take the address of "Perry"
```

There are two ways around this problem. The first is to do what was shown previously, which is to introduce a variable to hold the constant value. The second way is to write a generic helper function that takes in a parameter of any type and returns a pointer to that type:

解决这个问题有两种方法。第一种方法是像之前演示的那样, 引入一个变量来保存常量值。第二种方法是编写一个通用的辅助函数, 该函数接受任意类型的参数, 并返回指向该类型的指针:

```
func makePointer[T any](t T) *T {  
    return &t  
}
```

With that function, you can now write:

有了这个函数，你现在可以这样写：

```
p := person{
  FirstName: "Pat",
  MiddleName: makePointer("Perry"), // This works
  LastName: "Peterson",
}
```

Why does this work? When you pass a constant to a function, the constant is copied to a parameter, which is a variable. Since it's a variable, it has an address in memory. The function then returns the variable's memory address. Writing generic functions is covered in [Chapter 8](#).

为什么这种方法有效？当你将一个常量传递给函数时，该常量会被复制到一个参数中，而这个参数实际上是一个变量。由于它是一个变量，因此它在内存中有一个地址。然后，函数会返回该变量的内存地址。编写泛型函数将在第 8 章中介绍。

TIP

提示

Use a helper function to turn a constant value into a pointer.

使用辅助函数将常量值转换为指针。

Don't Fear the Pointers

不要害怕指针

The first rule of pointers is to not be afraid of them. If you are used to Java, JavaScript, Python, or Ruby, you might find pointers intimidating.

However, pointers are actually the familiar behavior for classes. It's the nonpointer structs in Go that are unusual.

使用指针的第一条原则就是不要害怕它。如果你习惯了 Java、JavaScript、Python 或 Ruby，你可能会觉得指针令人生畏。然而，指针实际上是类的常见行为。真正不寻常的是 Go 中的非指针结构体。

In Java and JavaScript, there is a difference in the behavior between primitive types and classes (Python and Ruby don't have primitive values but use immutable instances to simulate them). When a primitive value is assigned to another variable or passed to a function or method, any changes made to the other variable aren't reflected in the original, as shown in

[Example 6-1](#).

在 Java 和 JavaScript 中，基本类型和类的行为有所不同（Python 和 Ruby 没有基本类型值，而是使用不可变实例来模拟它们）。当一个基本类型值被赋值给另一个变量或传递给函数或方法时，对另一个变量所做的任何更改都不会反映到原始变量中，如示例 6-1 所示。

Example 6-1. Assigning primitive variables doesn't share memory in Java

例 6-1. 在 Java 中，赋值基本类型变量不共享内存

```
int x = 10;
int y = x;
y = 20;
System.out.println(x); // prints 10
```

However, let's take a look at what happens when an instance of a class is assigned to another variable or passed to a function or method (the code in [Example 6-2](#) is written in Python, but you can find similar code for Java, JavaScript, and Ruby in the *sample_code/language_pointer_examples* directory in the [Chapter 6 repository](#)).

但是，让我们来看看当一个类的实例被赋值给另一个变量或传递给函数或方法时会发生什么（示例 6-2 中的代码是用 Python 编写的，但你可以第 6 章存储库的 *sample_code/language_pointer_examples* 目录中找到类似的 Java、JavaScript 和 Ruby 代码）。

Example 6-2. Passing a class instance into a function

例 6-2. 将类实例传递给函数

```
class Foo:
    def __init__(self, x):
```

```
        self.x = x

def outer():
    f = Foo(10)
    inner1(f)
    print(f.x)
    inner2(f)
    print(f.x)
    g = None
    inner2(g)
    print(g is None)

def inner1(f):
    f.x = 20

def inner2(f):
    f = Foo(30)

outer()
```

Running this code prints the following output:

运行此代码会输出以下信息：


```
20
20
True
```

That's because the following scenarios are true in Java, Python, JavaScript, and Ruby:

这是因为以下情况在 Java、Python、JavaScript 和 Ruby 中都成立：

- If you pass an instance of a class to a function and you change the value of a field, the change is reflected in the variable that was passed in.

如果将一个类的实例传递给一个函数，并且更改了某个字段的值，则该更改会反映在传入的变量中。

- If you reassign the parameter, the change is *not* reflected in the variable that was passed in.

如果重新赋值参数，则更改不会反映在传入的变量中。

- If you pass `nil/null/None` for a parameter value, setting the parameter itself to a new value doesn't modify the variable in the calling function.

如果将 `nil/null/None` 作为参数值传递，则将参数本身设置为新值不会修改调用函数中的变量。

Some people explain this behavior by saying that class instances are passed by reference in these languages. This is untrue. If they were being passed by

reference, scenarios two and three would change the variable in the calling function. These languages are always pass-by-value, just as in Go.

有些人解释这种行为时说，这些语言中的类实例是按引用传递的。这是不正确的。如果是按引用传递，那么第二种和第三种情况就会改变调用函数中的变量。这些语言始终是按值传递的，就像 Go 语言一样。

What you are seeing is that every instance of a class in these languages is implemented as a pointer. When a class instance is passed to a function or method, the value being copied is the pointer to the instance. Since `outer` and `inner1` are referring to the same memory, changes made to fields in `f` in `inner1` are reflected in the variable in `outer`. When `inner2` reassigns `f` to a new class instance, this creates a separate instance and does not affect the variable in `outer`.

您看到的是，在这些语言中，类的每个实例都以指针的形式实现。当一个类实例被传递给函数或方法时，被复制的值是指向该实例的指针。由于 `outer` 和 `inner1` 指向同一块内存，因此对 `inner1` 中 `f` 的字段所做的更改会反映在 `outer` 的变量中。当 `inner2` 将 `f` 重新赋值给一个新的类实例时，这会创建一个单独的实例，并且不会影响 `outer` 中的变量。

When you use a pointer variable or parameter in Go, you see the exact same behaviors. The difference between Go and these languages is that Go gives

you the *choice* to use pointers or values for both primitives and structs. Most of the time, you should use a value. Values make it easier to understand how and when your data is modified. A secondary benefit is that using values reduces the amount of work that the garbage collector has to do. I'll talk about that in [“Reducing the Garbage Collector’s Workload”](#).

在 Go 语言中使用指针变量或参数时，你会看到完全相同的行为。Go 与其他语言的区别在于，Go 允许你选择对基本类型和结构体使用指针或值。大多数情况下，你应该使用值。值可以让你更容易理解数据是如何以及何时被修改的。另一个好处是，使用值可以减少垃圾回收器的工作量。我将在“减少垃圾回收器的工作负载”一节中详细讨论这一点。

Pointers Indicate Mutable Parameters

指针指示可变参数

As you’ve already seen, Go constants provide names for literal expressions that can be calculated at compile time. Go has no mechanism to declare that other kinds of values are immutable. Modern software engineering embraces immutability. [MIT’s course on Software Construction](#) sums up the reasons: “[I]mmutable types are safer from bugs, easier to understand, and more ready for change. Mutability makes it harder to understand what your program is doing, and much harder to enforce contracts.”

正如你所见，Go 语言的常量为可在编译时计算的字面表达式提供名称。Go 语言没有机制来声明其他类型的值是不可变的。现代软件工程崇尚不可变性。麻省理工学院的软件构造课程总结了其中的原因：“不可变类型更不容易出现 bug，更容易理解，也更容易适应变化。可变性会使程序难以理解其运行机制，也更难强制执行契约。”

The lack of immutable declarations in Go might seem problematic, but the ability to choose between value and pointer parameter types addresses the issue. As the Software Construction course materials go on to explain: “[U]sing mutable objects is just fine if you are using them entirely locally within a method, and with only one reference to the object.” Rather than declare that some variables and parameters are immutable, Go developers use pointers to indicate that a parameter is mutable.

Go 语言中缺少不可变声明看似是个问题，但允许在值类型和指针类型之间进行选择可以解决这个问题。正如软件构造课程材料所解释的：“如果你在方法内部完全局部地使用可变对象，并且只使用一个引用，那么使用可变对象是完全没问题的。” Go 开发者并没有声明某些变量和参数是不可变的，而是使用指针来指示参数是否可变。

Since Go is a call-by-value language, the values passed to functions are copies. For nonpointer types like primitives, structs, and arrays, this means that the called function cannot modify the original. Since the called function has a copy of the original data, the original data’s immutability is guaranteed.

由于 Go 是一种按值调用的语言，传递给函数的值是副本。对于非指针类型（例如基本类型、结构体和数组），这意味着被调用的函数无法修改原始数据。由于被调用的函数拥有原始数据的副本，因此原始数据的不可变性得到了保证。

NOTE

笔记

I'll talk about passing maps and slices to functions in [“The Difference Between Maps and Slices”](#).

我将在“Maps 和 Slices 的区别”一节中讨论如何将 map 和 slices 传递给函数。

However, if a pointer is passed to a function, the function gets a copy of the pointer. This still points to the original data, which means that the original data can be modified by the called function.

但是，如果将指针传递给函数，函数得到的是指针的副本。这个副本仍然指向原始数据，这意味着被调用的函数可以修改原始数据。

This has a couple of related implications.

这会带来一些相关的后果。

The first implication is that when you pass a `nil` pointer to a function, you cannot make the value non-nil. You can reassign the value only if there was a value already assigned to the pointer. While confusing at first, it makes

sense. Since the memory location was passed to the function via call-by-value, you can't change the memory address, any more than you could change the value of an `int` parameter. You can demonstrate this with the following program:

首先，当你将 `nil` 指针传递给函数时，你不能将该指针的值设为非空值。只有当该指针已被赋值时，你才能重新赋值。这乍一看可能令人困惑，但仔细想想其实很合理。因为内存地址是通过按值传递的方式传递给函数的，所以你不能更改内存地址，就像你不能更改 `int` 参数的值一样。你可以用以下程序来演示这一点：

```
func failedUpdate(g *int) {  
    x := 10  
    g = &x  
}  
  
func main() {  
    var f *int // f is nil  
    failedUpdate(f)  
    fmt.Println(f) // prints nil  
}
```

The flow through this code is shown in [Figure 6-3](#).

该代码的流程如图 6-3 所示。

Value	0	0	0	0
Address	1	2	3	4
Variable	f			
Function	main			

Value	0	0	0	0	0	0	0	0
Address	1	2	3	4	5	6	7	8
Variable	f				g			
Function	main				failedUpdate			

Value	0	0	0	0	0	0	0	0	0	0	10	
Address	1	2	3	4	5	6	7	8	9	10	11	12
Variable	f				g				x			
Function	main				failedUpdate							

Value	0	0	0	0	0	0	0	9	0	0	0	10
Address	1	2	3	4	5	6	7	8	9	10	11	12
Variable	f				g				x			
Function	main				failedUpdate							

Value	0	0	0	0
Address	1	2	3	4
Variable	f			
Function	main			

Figure 6-3. Failing to update a `nil` pointer

图 6-3. 更新 `nil` 指针失败

You start with a `nil` variable `f` in `main`. When you call `failedUpdate`, you copy the value of `f`, which is `nil`, into the parameter named `g`. This means that `g` is also set to `nil`. You then declare a new variable `x` within `failedUpdate` with the value 10. Next,

you change `g` in `failedUpdate` to point to `x`. This does not change the `f` in `main`, and when you exit `failedUpdate` and return to `main`, `f` is still `nil`.

首先，在 `main` 中定义一个名为 `nil` 的变量 `f`。调用 `failedUpdate` 时，将 `f` 的值（即 `nil`）复制到名为 `g` 的参数中。这意味着 `g` 的值也被设置为 `nil`。然后，在 `failedUpdate` 中声明一个名为 `x` 的新变量，并将其值设为 10。接下来，将 `failedUpdate` 中的 `g` 指向 `x`。这不会改变 `main` 中的 `f`，当你退出 `failedUpdate` 并返回到 `main` 时，`f` 仍然是 `nil`。

The second implication of copying a pointer is that if you want the value assigned to a pointer parameter to still be there when you exit the function, you must dereference the pointer and set the value. If you change the pointer, you have changed the copy, not the original. Dereferencing puts the new value in the memory location pointed to by both the original and the copy. Here's a short program that shows how this works:

复制指针的第二个含义是，如果你希望指针参数的值在函数退出时仍然存在，你必须解引用该指针并设置其值。如果你修改了指针，你修改的是副本，而不是原始指针。解引用会将新值放入原始指针和副本指针都指向的内存位置。以下是一个演示其工作原理的小程序：

```
func failedUpdate(px *int) {  
    x2 := 20
```



```

    px = &x2
}

func update(px *int) {
    *px = 20
}

func main() {
    x := 10
    failedUpdate(&x)
    fmt.Println(x) // prints 10
    update(&x)
    fmt.Println(x) // prints 20
}

```

The flow through this code is shown in [Figure 6-4](#).

该代码的流程如图 6-4 所示。

In this example, you start with `x` in `main` set to 10. When you call `failedUpdate`, you copy the address of `x` into the parameter `px`. Next, you declare `x2` in `failedUpdate`, set to 20. You then point `px` in `failedUpdate` to the address of `x2`. When you return to `main`, the value of `x` is unchanged. When you call `update`, you copy the address of `x` into `px` again. However, this time you change the value of what `px` in

`update` points to, the variable `x` in `main`. When you return to `main`, `x` has been changed.

在这个例子中，首先将 `x` 的值赋给 `x`，并将其赋值为 10。调用 `failedUpdate` 时，将 `x` 的地址复制到参数 `px` 中。接下来，在 `failedUpdate` 中声明 `x2`，并将其赋值为 20。然后，将 `failedUpdate` 中的 `px` 指向 `x2` 的地址。返回到 `main` 时，`x` 的值保持不变。调用 `update` 时，再次将 `x` 的地址复制到 `px` 中。但是，这次您更改了 `update` 中 `px` 指向的值，即 `main` 中的变量 `x`。当您返回到 `main` 时，`x` 的值已经改变。

Value	0	0	0	10
Address	1	2	3	4
Variable	x			
Function	main			

Value	0	0	0	10	0	0	0	1
Address	1	2	3	4	5	6	7	8
Variable	x				px			
Function	main				failedUpdate			

Value	0	0	0	10	0	0	0	1	0	0	0	20
Address	1	2	3	4	5	6	7	8	9	10	11	12
Variable	x				px				x2			
Function	main				failedUpdate							

Value	0	0	0	10	0	0	0	9	0	0	0	20
Address	1	2	3	4	5	6	7	8	9	10	11	12
Variable	x				px				x2			
Function	main				failedUpdate							

Value	0	0	0	10
Address	1	2	3	4
Variable	x			
Function	main			

Value	0	0	0	10	0	0	0	1
Address	1	2	3	4	5	6	7	8
Variable	x				px			
Function	main				Update			

Value	0	0	0	20	0	0	0	1
Address	1	2	3	4	5	6	7	8
Variable	x				px			
Function	main				Update			

Value	0	0	0	20
Address	1	2	3	4
Variable	x			
Function	main			

Figure 6-4. The wrong way and the right way to update a pointer

图 6-4. 更新指针的错误方法和正确方法

Pointers Are a Last Resort

指针是最后的手段

That said, you should be careful when using pointers in Go. As discussed earlier, they make it harder to understand data flow and can create extra work for the garbage collector. Rather than populating a struct by passing a pointer to it into a function, have the function instantiate and return the struct (see Examples [6-3](#) and [6-4](#)).

也就是说，在 Go 中使用指针时要格外小心。如前所述，指针会使数据流难以理解，并可能给垃圾回收器增加额外的工作量。与其将指向结构体的指针传递给函数来填充结构体，不如让函数实例化并返回该结构体（参见示例 6-3 和 6-4）。

Example 6-3. Don't do this

例 6-3. 不要这样做

```
func MakeFoo(f *Foo) error {  
    f.Field1 = "val"  
    f.Field2 = 20  
}
```

```
    return nil
}
```

Example 6-4. Do this

例 6-4. 这样做

```
func MakeFoo() (Foo, error) {
    f := Foo{
        Field1: "val",
        Field2: 20,
    }
    return f, nil
}
```

The only time you should use pointer parameters to modify a variable is when the function expects an interface. You see this pattern when working with JSON (I'll talk more about the JSON support in Go's standard library in [“encoding/json”](#)):

只有当函数需要接口时，才应该使用指针参数来修改变量。处理 JSON 时就会用到这种模式（关于 Go 标准库中对 JSON 的支持，我会在“encoding/json”部分详细讨论）：

```
f := struct {
    Name string `json:"name"`
}
```

```
Age int `json:"age"`  
}{}  
err := json.Unmarshal([]byte(`{"name": "Bob",  
"age": 30}`), &f)
```

The `Unmarshal` function populates a variable from a slice of bytes containing JSON. It is declared to take a slice of bytes and an `any` parameter. The value passed in for the `any` parameter must be a pointer. If it is not, an error is returned.

`Unmarshal` 函数从包含 JSON 的字节切片中填充一个变量。它声明接受一个字节切片和一个 `any` 参数。传递给 `any` 参数的值必须是指针。如果不是，则返回错误。

Why do you pass a pointer into `Unmarshal` instead of having it return a value? There are two reasons. First, this function predates the addition of generics to Go, and without generics (which I'll talk about in detail in [Chapter 8](#)), there is no way to know what type of value to create and return.

为什么要把指针传递给 `Unmarshal` 而不是让它返回一个值呢？原因有二。首先，这个函数是在 Go 语言引入泛型之前编写的，如果没有泛型（我会在第 8 章详细讲解），就无法确定要创建和返回什么类型的值。

The second reason is that passing in a pointer gives you control over memory allocation. Iterating over data and converting it from JSON to a Go struct is a common design pattern, so `Unmarshal` is optimized for this case. If the `Unmarshal` function returned a value and `Unmarshal` was called in a loop, one struct instance would be created on each loop iteration. This creates a lot more work for the garbage collector, which slows down your program. You'll see another use of this pattern when you look at [“Slices as Buffers”](#), and I'll talk more about efficient memory usage in [“Reducing the Garbage Collector's Workload”](#).

第二个原因是，传入指针可以让你控制内存分配。遍历数据并将其从 JSON 转换为 Go 结构体是一种常见的设计模式，因此 `Unmarshal` 针对这种情况进行了优化。如果 `Unmarshal` 函数返回了一个值，并且在循环中调用了 `Unmarshal`，则每次循环迭代都会创建一个结构体实例。这会给垃圾回收器增加很多工作量，从而降低程序运行速度。在“切片作为缓冲区”一节中，你会看到这种模式的另一种用法，而我将在此“减少垃圾回收器的工作负载”一节中更详细地讨论高效的内存使用。

Because JSON integration is so common, this API is sometimes treated as a common case by new Go developers, instead of the exception that it should be.

由于 JSON 集成非常普遍，因此新 Go 开发人员有时会将此 API 视为常见情况，而不是它应该被视为的例外情况。

When returning values from a function, you should favor value types. Use a pointer type as a return type only if there is state within the data type that needs to be modified. When you look at I/O in [“io and Friends”](#), you’ll see that with buffers for reading or writing data. In addition, some data types used with concurrency must always be passed as pointers. You’ll see those in [Chapter 12](#).

从函数返回值时，应优先使用值类型。只有当数据类型内部存在需要修改的状态时，才应使用指针类型作为返回类型。在《I/O 及其相关知识》一书中，你会看到关于 I/O 的讨论，其中使用了缓冲区进行数据读写。此外，某些用于并发的数据类型必须始终以指针形式传递。你将在第 12 章中看到这些内容。

Pointer Passing Performance

指针传递性能

If a struct is large enough, using a pointer to the struct as either an input parameter or a return value improves performance. The time to pass a pointer into a function is constant for all data sizes, roughly one nanosecond. This makes sense, as the size of a pointer is the same for all data types. Passing a value into a function takes longer as the data gets larger. It takes about 0.7 milliseconds once the value gets to be around 10 megabytes of data.

如果结构体足够大，使用指向该结构体的指针作为输入参数或返回值可以提高性能。将指针传递给函数所需的时间对于所有数据大小都是恒定的，大约为 1 纳秒。这很合理，因为指针的大小对于所有数据类型都是相同的。随着数据量的增大，将值传递给函数所需的时间也会增加。当数据量达到 10 兆字节左右时，传递值大约需要 0.7 毫秒。

The behavior for returning a pointer versus returning a value is more interesting. For data structures that are smaller than 10 megabytes, it is actually *slower* to return a pointer type than a value type. For example, a 100-byte data structure takes around 10 nanoseconds to be returned, but a pointer to that data structure takes about 30 nanoseconds. As your data structures get larger, the performance advantage flips. It takes nearly 1.5 milliseconds to return 10 megabytes of data, but a little less than half a millisecond to return a pointer to it.

返回指针和返回值的行为差异更有意思。对于小于 10 兆字节的数据结构，返回指针类型的速度实际上比返回值类型的速度更慢。例如，返回一个 100 字节的数据结构大约需要 10 纳秒，而返回指向该数据结构的指针则需要大约 30 纳秒。随着数据结构越来越大，性能优势会发生逆转。返回 10 兆字节的数据需要近 1.5 毫秒，而返回指向该数据的指针则只需不到半毫秒。

You should be aware that these are very short times. For the vast majority of cases, the difference between using a pointer and a value won't affect your program's performance. But if you are passing megabytes of data between

functions, consider using a pointer even if the data is meant to be immutable.

您应该意识到，这些时间非常短暂。绝大多数情况下，使用指针和使用值之间的区别不会影响程序的性能。但是，如果您需要在函数之间传递兆字节的数据，即使数据本身是不可变的，也应该考虑使用指针。

All these numbers are from an i7-8700 computer with 32 GB of RAM. Different CPUs can produce different crossover points. For example, on an Apple M1 CPU with 16 GB of RAM, it is faster to return a pointer (5 microseconds) than a value (8 microseconds) at sizes of around 100 kilobytes. You can run your own performance tests with the code in the *sample_code/pointer_perf* directory in the [Chapter 6 repository](#). Run the command `go test ./... -bench=.` to find your own results. (Benchmarks are covered in [“Using Benchmarks”](#).)

所有这些数据均来自配备 32 GB 内存的 i7-8700 计算机。不同的 CPU 可能会产生不同的交叉点。例如，在配备 16 GB 内存的 Apple M1 CPU 上，对于大约 100 KB 的数据大小，返回指针（5 微秒）比返回值（8 微秒）更快。您可以使用第 6 章代码库中 *sample_code/pointer_perf* 目录下的代码运行自己的性能测试。运行命令 `go test ./... -bench=.` 即可查看您自己的结果。（基准测试的相关内容请参见“使用基准测试”部分。）

The Zero Value Versus No Value

零值与无值

Go pointers are also commonly used to indicate the difference between a variable or field that's been assigned the zero value and a variable or field that hasn't been assigned a value at all. If this distinction matters in your program, use a `nil` pointer to represent an unassigned variable or struct field.

Go 指针也常用于区分已赋值为零的变量或字段和尚未赋值的变量或字段。如果这种区别在你的程序中很重要，请使用 `nil` 指针来表示未赋值的变量或结构体字段。

Because pointers also indicate mutability, be careful when using this pattern. Rather than return a pointer set to `nil` from a function, use the comma ok idiom that you saw for maps, and return a value type and a boolean.

由于指针也表示可变性，因此在使用此模式时要格外小心。不要从函数返回一个设置为 `nil` 的指针，而是使用您在映射中看到的逗号分隔符，并返回一个值类型和一个布尔值。

Remember, if a `nil` pointer is passed into a function via a parameter or a field on a parameter, you cannot set the value within the function, as there's nowhere to store the value. If a non-nil value is passed in for the pointer, do not modify it unless you document the behavior.

请记住，如果通过参数或参数上的字段将 `nil` 指针传递给函数，则无法在函数内部设置其值，因为没有地方可以存储该值。如果为该指针传递了一个非空值，除非您已记录此行为，否则请勿修改它。

Again, JSON conversions are the exception that proves the rule. When converting data back and forth from JSON (yes, I'll talk more about the JSON support in Go's standard library in [“encoding/json”](#)), you often need a way to differentiate between the zero value and not having a value assigned at all. Use a pointer value for fields in the struct that are nullable.

再次强调，JSON 转换是例外情况，但也恰恰证明了这条规则。在 JSON 数据与其他格式之间进行转换时（是的，我会在“encoding/json”部分详细讨论 Go 标准库对 JSON 的支持），通常需要一种方法来区分零值和完全未赋值的情况。对于结构体中可为空的字段，可以使用指针值。

When not working with JSON (or other external protocols), resist the temptation to use a pointer field to indicate no value. While a pointer does provide a handy way to indicate no value, if you are not going to modify the value, you should use a value type instead, paired with a boolean.

当不使用 JSON（或其他外部协议）时，请避免使用指针字段来表示无值。虽然指针确实提供了一种便捷的方式来表示无值，但如果您不打算修改该值，则应该使用值类型并搭配布尔值。

The Difference Between Maps and Slices

地图和切片的区别

As you saw in the previous chapter, any modifications made to a map that's passed to a function are reflected in the original variable that was passed in. Now that you know about pointers, you can understand why: within the Go runtime, a map is implemented as a pointer to a struct. Passing a map to a function means that you are copying a pointer.

正如你在上一章中看到的，对传递给函数的映射所做的任何修改都会反映到最初传入的变量中。现在你了解了指针，就能明白其中的原因：在 Go 运行时中，映射被实现为指向结构体的指针。将映射传递给函数实际上就是复制一个指针。

Because of this, you should consider carefully before using maps for input parameters or return values, especially on public APIs. On an API-design level, maps are a bad choice because they say nothing about the values contained within; nothing explicitly defines any keys in the map, so the only way to know what they are is to trace through the code. From the

standpoint of immutability, maps are bad because the only way to know what ended up in the map is to trace through all the functions that interact with it. This prevents your API from being self-documenting. If you are used to dynamic languages, don't use a map as a replacement for another language's lack of structure. Go is a strongly typed language; rather than passing a map around, use a struct. (You'll learn another reason to prefer structs when I talk about memory layout in [“Reducing the Garbage Collector's Workload”](#).)

因此，在将映射（map）用作输入参数或返回值之前，尤其是在公共 API 中，您应该谨慎考虑。从 API 设计的角度来看，映射并非理想之选，因为它没有明确说明映射中包含的值；映射中的任何键都没有被显式定义，因此唯一了解键值的方法就是追踪代码。从不可变性的角度来看，映射也不好，因为唯一知道映射最终存储了什么的方法就是追踪所有与它交互的函数。这会妨碍 API 的自文档化。如果您习惯使用动态语言，请不要用映射来弥补其他语言缺乏结构。Go 是一种强类型语言；与其传递映射，不如使用结构体（struct）。（在“减少垃圾回收器的负载”一节中，我将讨论内存布局，届时您会了解到另一个更倾向于使用结构体的原因。）

NOTE

笔记

A map input parameter or return value is the correct choice in certain situations. A struct requires you to name its fields at compile time. If the keys for your data aren't known at compile time, a map is ideal.

在某些情况下，使用映射的输入参数或返回值是正确的选择。结构体要求在编译时为其字段命名。如果编译时无法确定数据的键，则映射是理想的选择。

Meanwhile, passing a slice to a function has more complicated behavior: any modification to the slice's contents is reflected in the original variable, but using `append` to change the length isn't reflected in the original variable, even if the slice has a capacity greater than its length. That's because a slice is implemented as a struct with three fields: an `int` field for length, an `int` field for capacity, and a pointer to a block of memory.

[Figure 6-5](#) demonstrates the relationship.

同时，将切片传递给函数时，其行为更为复杂：对切片内容的任何修改都会反映在原始变量中，但使用 `append` 更改长度并不会反映在原始变量中，即使切片的容量大于其长度。这是因为切片被实现为一个包含三个字段的结构体：用于存储长度的 `int` 字段、用于存储容量的 `int` 字段以及指向内存块的指针。图 6-5 展示了这种关系。

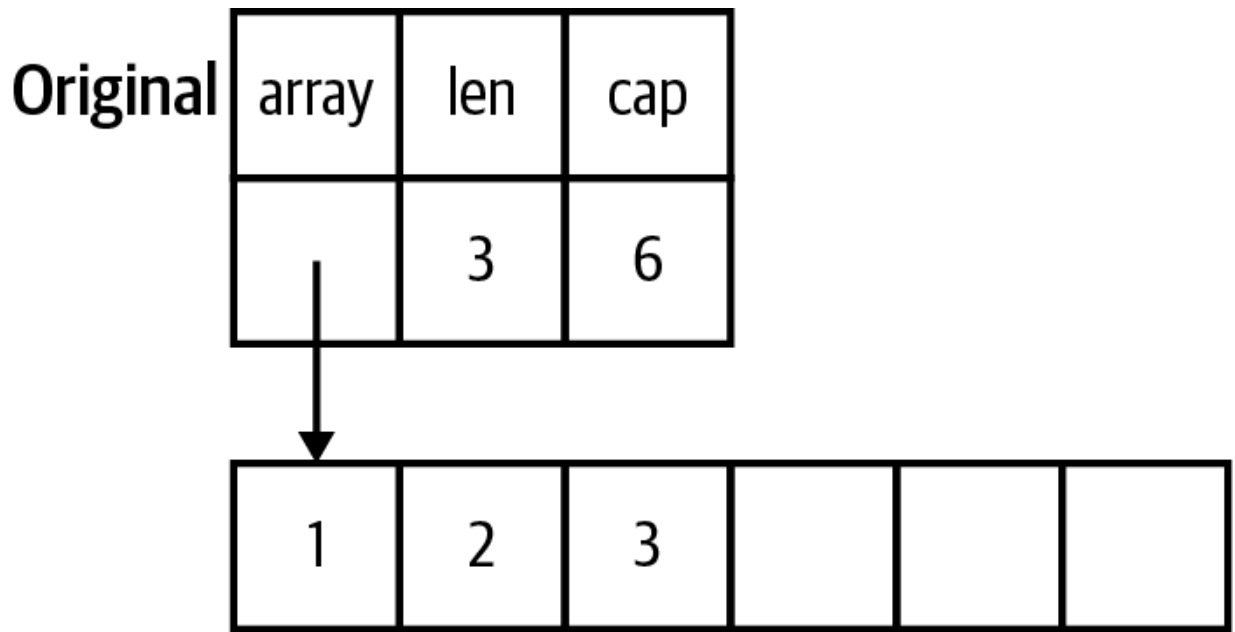


Figure 6-5. The memory layout of a slice

图 6-5. 切片内存布局

When a slice is copied to a different variable or passed to a function, a copy is made of the length, capacity, and the pointer. [Figure 6-6](#) shows how both slice variables point to the same memory.

当切片被复制到另一个变量或传递给函数时，其长度、容量和指针都会被复制。图 6-6 显示了两个切片变量如何指向同一块内存。

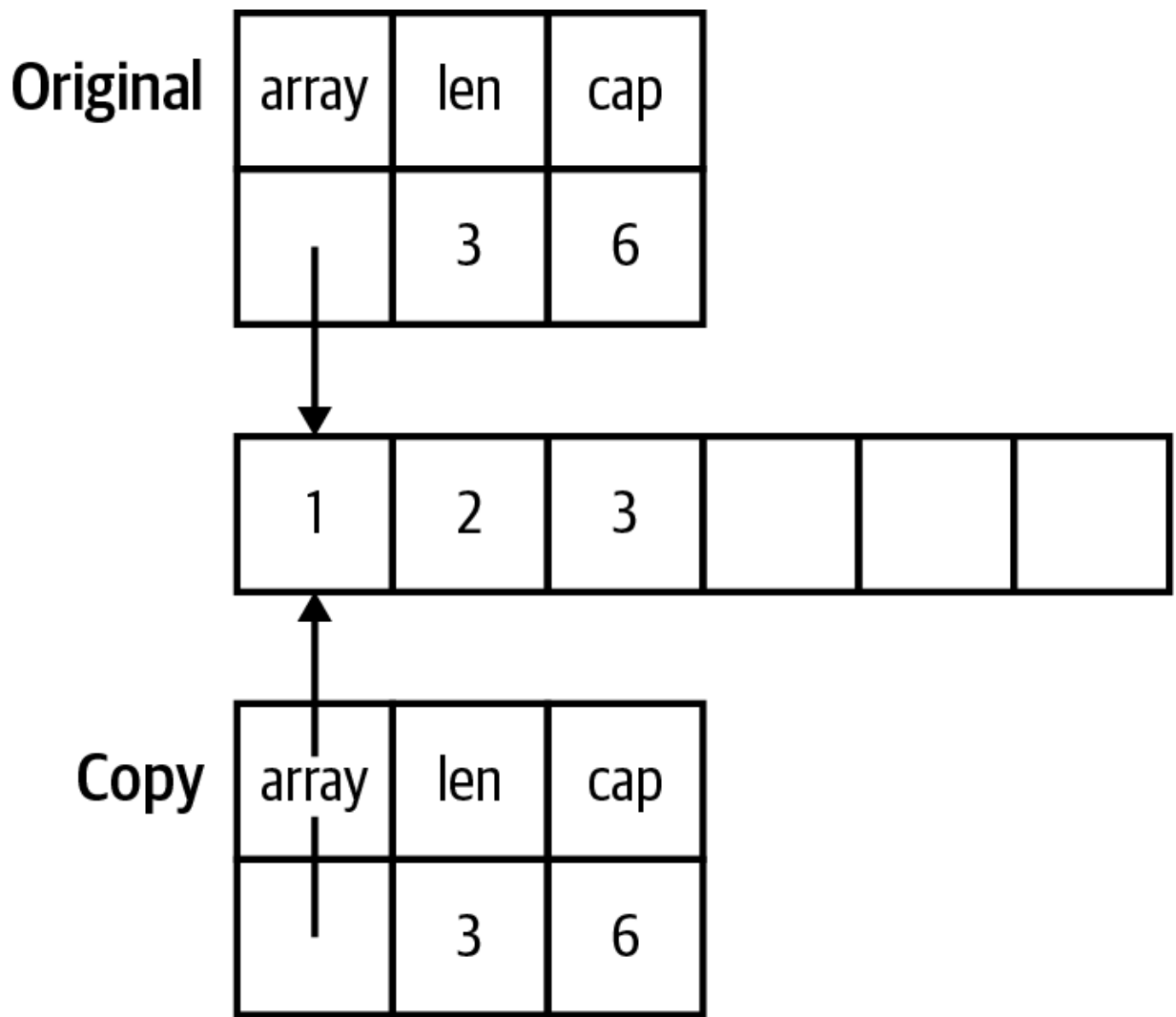


Figure 6-6. The memory layout of a slice and its copy

图 6-6. 切片及其副本的内存布局

Changing the values in the slice changes the memory that the pointer points to, so the changes are seen in both the copy and the original. You see in [Figure 6-7](#) how this looks in memory.

更改切片中的值会改变指针指向的内存，因此副本和原始数据都会受到影响。图 6-7 显示了这在内存中的表现。

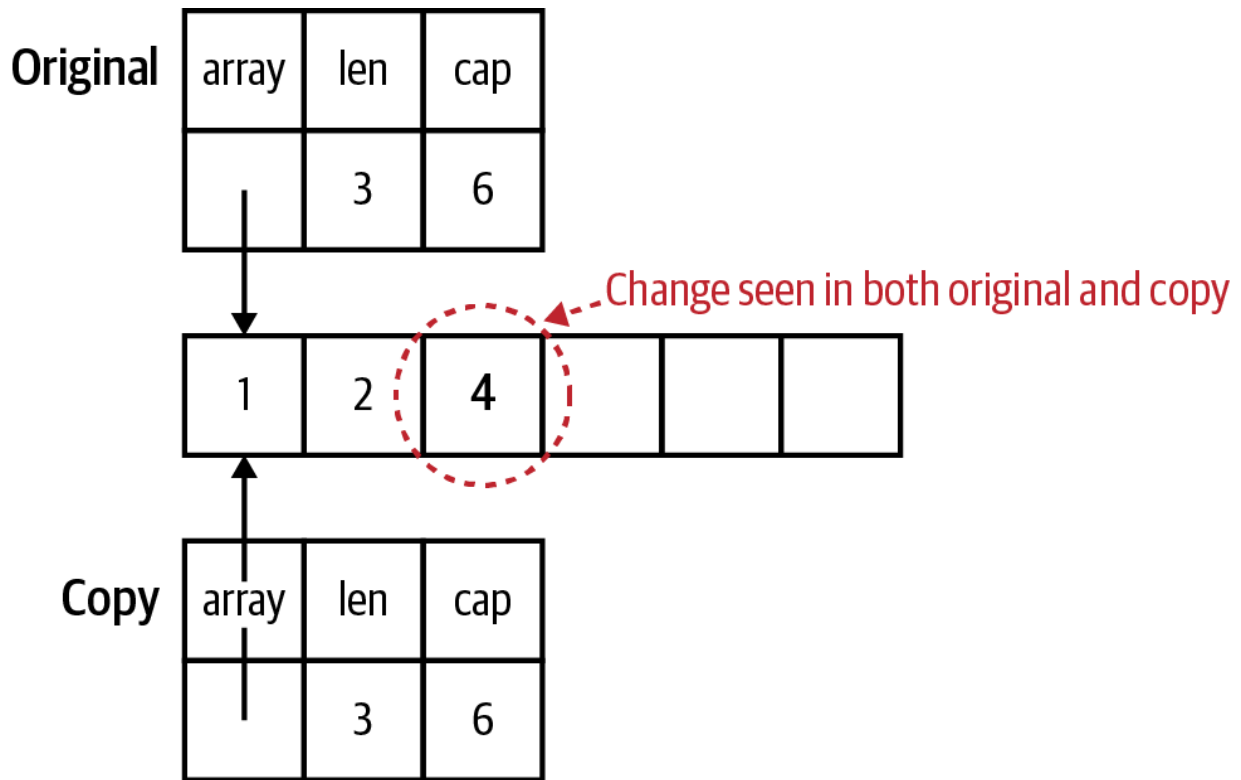


Figure 6-7. Modifying the contents of a slice

图 6-7. 修改切片的内容

If the slice copy is appended to and there *is* enough capacity in the slice for the new values, the length changes in the copy, and the new values are stored in the block of memory that's shared by the copy and the original. However, the length in the original slice remains unchanged. The Go runtime prevents the original slice from seeing those values since they are beyond the length of the original slice. [Figure 6-8](#) highlights the values that are visible in one slice variable but not in the other.

如果向切片副本追加数据，且副本中有足够的空间容纳新值，则副本的长度会发生变化，新值会存储在副本和原始切片共享的内存块中。

然而，原始切片的长度保持不变。由于这些新值超出了原始切片的长度，Go 运行时会阻止原始切片访问它们。图 6-8 突出显示了在一个切片变量中可见但在另一个切片变量中不可见的值。

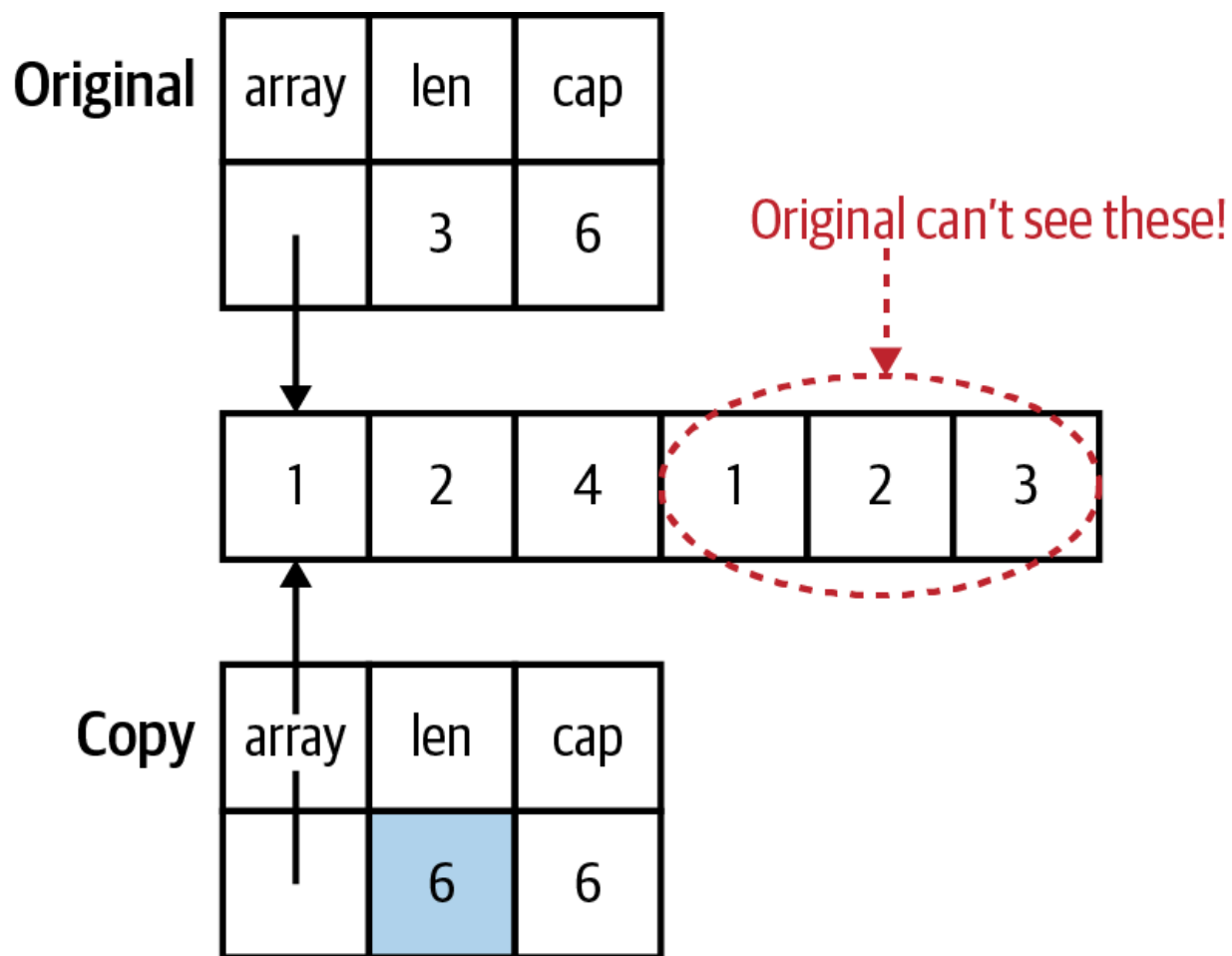


Figure 6-8. Changing the length is invisible in the original

图 6-8。长度的变化在原图中是不可见的。

If the slice copy is appended to and there *isn't* enough capacity in the slice for the new values, a new, bigger block of memory is allocated, values are copied over, and the pointer, length, and capacity fields in the copy are updated. Changes to the pointer, length, and capacity are not reflected in the

original, because they are only in the copy. [Figure 6-9](#) shows how each slice variable now points to a different memory block.

如果向切片副本追加数据，而切片容量不足以容纳新值，则会分配一块更大的新内存块，将值复制到该内存块中，并更新副本中的指针、长度和容量字段。指针、长度和容量的更改不会反映在原始切片中，因为这些更改仅存在于副本中。图 6-9 显示了每个切片变量现在如何指向不同的内存块。

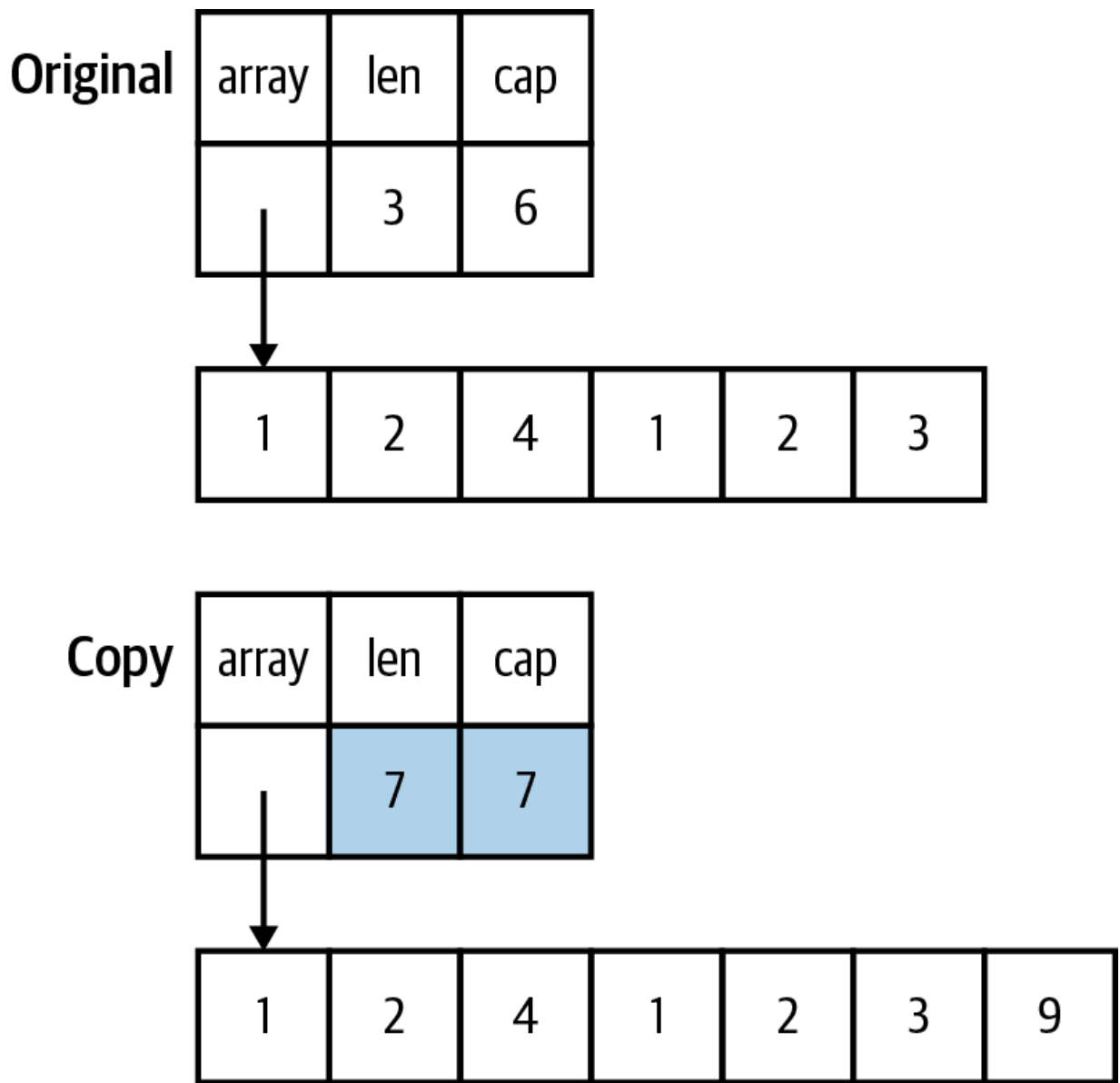


Figure 6-9. Changing the capacity changes the storage

图 6-9. 改变容量会改变存储

The result is that a slice that's passed to a function can have its contents modified, but the slice can't be resized. As the only usable linear data structure, slices are frequently passed around in Go programs. By default, you should assume that a slice is not modified by a function. Your

function's documentation should specify whether it modifies the slice's contents.

结果是，传递给函数的切片的内容可以被修改，但切片本身的大小不能被调整。作为唯一可用的线性数据结构，切片在 Go 程序中经常被传递。默认情况下，你应该假设切片不会被函数修改。你的函数文档应该明确说明它是否会修改切片的内容。

NOTE

笔记

The reason you can pass a slice of any size to a function is that the data type that's passed to the function is the same for any size slice: a struct of two `int` values and a pointer. The reason you can't write a function that takes an array of any size is that the entire array is passed to the function, not just a pointer to the data.

之所以可以将任意大小的切片传递给函数，是因为传递给函数的数据类型对于任何大小的切片都是相同的：一个包含两个 `int` 值和一个指针的结构体。之所以不能编写接受任意大小数组的函数，是因为传递给函数的是整个数组，而不仅仅是指向数据的指针。

There is one situation where the ability to modify the contents (but not the size) of a slice input parameter is very useful. This makes them ideal for reusable buffers.

在某些情况下，能够修改切片输入参数的内容（但不能修改其大小）非常有用。这使得它们非常适合用于可重用缓冲区。

Slices as Buffers

将切片作为缓冲区

When reading data from an external resource (like a file or a network connection), many languages use code like this:

当从外部资源（例如文件或网络连接）读取数据时，许多语言使用类似这样的代码：

```
r = open_resource()
while r.has_data() {
    data_chunk = r.next_chunk()
    process(data_chunk)
}
close(r)
```

The problem with this pattern is that every time you iterate through that `while` loop, you allocate another `data_chunk` even though each one is used only once. This creates lots of unnecessary memory allocations, just as I discussed in [“Pointers Are a Last Resort”](#) when looking at the `Unmarshal` function. Garbage-collected languages handle those allocations for you automatically, but the work still needs to be done to clean them up when you are done processing.

这种模式的问题在于，每次遍历 `while` 循环时，都会分配一个新的 `data_chunk` 对象，即使每个对象只使用一次。这会造成大量不必要的内存分配，正如我在“指针是最后的手段”一文中讨论 `Unmarshal` 函数时所提到的那样。虽然有垃圾回收机制的语言会自动处理这些内存分配，但程序处理完毕后，仍然需要手动清理这些内存。

Even though Go is a garbage-collected language, writing idiomatic Go means avoiding unneeded allocations. Rather than returning a new allocation each time you read from a data source, you create a slice of bytes once and use it as a buffer to read data from the data source:

尽管 Go 是一种有垃圾回收机制的语言，但编写地道的 Go 代码意味着避免不必要的内存分配。与其每次从数据源读取数据时都分配新的内存，不如一次性创建一个字节切片，并将其用作缓冲区来从数据源读取数据：

```
file, err := os.Open(fileName)
if err != nil {
    return err
}
defer file.Close()
data := make([]byte, 100)
for {
    count, err := file.Read(data)
    process(data[:count])
    if err != nil {
```



```
        if errors.Is(err, io.EOF) {  
            return nil  
        }  
        return err  
    }  
}
```

Remember that you can't change the length or capacity of a slice when it is passed into a function, but you can change the contents up to the current length. In this code, you create a buffer of 100 bytes, and each time through the loop, you copy the next block of bytes (up to 100) into the slice. You then pass the populated portion of the buffer to `process`. If an error happens (other than `io.EOF`, a special-case error that indicates there is no more data to read), it is returned. When `io.EOF` is returned as the error, there is no more data and the function returns `nil`. You'll look at more details about I/O in [“io and Friends”](#) and error handling is covered in [Chapter 9](#).

请记住，将切片传递给函数时，您无法更改切片的长度或容量，但可以更改其内容，直到达到当前长度。在这段代码中，您创建了一个 100 字节的缓冲区，每次循环时，都将下一个字节块（最多 100 字节）复制到切片中。然后，您将填充好的缓冲区部分传递给 `process`。如果发生错误（`io.EOF` 除外，这是一个特殊情况的错误，表示没有更多数据可读），则会返回该错误。当返回 `io.EOF` 作

为错误时，表示没有更多数据，函数将返回 `nil`。您将在“io and Friends”中了解更多关于 I/O 的细节，错误处理将在第 9 章中介绍。

Reducing the Garbage Collector's Workload

减轻垃圾收集员的工作量

Using buffers is just one example of how you reduce the work done by the garbage collector. When programmers talk about “garbage,” what they mean is “data that has no more pointers pointing to it.” Once there are no more pointers pointing to some data, the memory that this data takes up can be reused. If the memory isn't recovered, the program's memory usage would continue to grow until the computer ran out of RAM. The job of a garbage collector is to automatically detect unused memory and recover it so it can be reused. It is fantastic that Go has a garbage collector, because decades of experience have shown that it is very difficult for people to properly manage memory manually. But just because you have a garbage collector doesn't mean you should create lots of garbage.

使用缓冲区只是减少垃圾回收器工作量的一个例子。程序员所说的“垃圾”指的是“不再有指针指向的数据”。一旦不再有指针指向某个数据，该数据占用的内存就可以被重新利用。如果内存没有被回收，程序的

内存使用量就会持续增长，直到计算机耗尽内存。垃圾回收器的作用就是自动检测未使用的内存并将其回收，以便再次利用。Go 语言拥有垃圾回收器非常棒，因为几十年的经验表明，手动管理内存非常困难。但是，有了垃圾回收器并不意味着你可以随意制造垃圾。

If you've spent time learning how programming languages are implemented, you've probably learned about the *heap* and the *stack*. If you're unfamiliar, here's how a stack works. A *stack* is a consecutive block of memory. Every function call in a thread of execution shares the same stack. Allocating memory on the stack is fast and simple. A *stack pointer* tracks the last location where memory was allocated. Allocating additional memory is done by changing the value of the stack pointer. When a function is invoked, a new *stack frame* is created for the function's data. Local variables are stored on the stack, along with parameters passed into a function. Each new variable moves the stack pointer by the size of the value. When a function exits, its return values are copied back to the calling function via the stack, and the stack pointer is moved back to the beginning of the stack frame for the exited function, deallocating all the stack memory that was used by that function's local variables and parameters.

如果你花时间学习过编程语言的实现方式，你可能已经了解过堆和栈。如果你还不熟悉，这里解释一下栈的工作原理。栈是一块连续的内存块。执行线程中的每个函数调用都共享同一个栈。在栈上分配内存既快速又简单。栈指针跟踪上次分配内存的位置。分配额外的内存只需改变栈指针的值即可。当一个函数被调用时，会为函数的数据创

建一个新的栈帧。局部变量和传递给函数的参数都存储在栈上。每个新变量都会使栈指针移动其值的大小。当一个函数退出时，它的返回值会通过栈复制回调用函数，栈指针会移回已退出函数的栈帧的开头，从而释放该函数局部变量和参数使用的所有栈内存。

NOTE

笔记

Since version 1.17, Go uses a combination of registers (a small set of very high-speed memory that's directly on the CPU) and the stack to pass values into and out of functions. It's faster and more complicated, but the general concepts of stack-only function calls still apply.

自 1.17 版本起，Go 语言开始结合使用寄存器（CPU 上直接存储的一小块高速内存）和栈来传递函数之间的值。这种方式速度更快，也更复杂，但仅使用栈进行函数调用的基本概念仍然适用。

To store something on the stack, you have to know exactly how big it is at compile time. When you look at the value types in Go (primitive values, arrays, and structs), they all have one thing in common: you know exactly how much memory they take at compile time. This is why the size is considered part of the type for an array. Because their sizes are known, they can be allocated on the stack instead of the heap. The size of a pointer type is also known, and it is also stored on the stack.

要在栈上存储数据，必须在编译时精确知道它的大小。Go 语言中的值类型（基本类型、数组和结构体）都有一个共同点：它们在编译时占

用的内存大小是已知的。这就是为什么数组的大小被视为类型的一部分。由于数组的大小已知，因此可以将其分配到栈上而不是堆上。指针类型的大小也是已知的，并且也存储在栈上。

NOTE

笔记

Go is unusual in that it can increase the size of a stack while the program is running. This is possible because each goroutine has its own stack and goroutines are managed by the Go runtime, not by the underlying operating system (I discuss goroutines when I talk about concurrency in [Chapter 12](#)). This has advantages (Go stacks start small and use less memory) and disadvantages (when the stack needs to grow, all data on the stack needs to be copied, which is slow). It's also possible to write worst-case-scenario code that causes the stack to grow and shrink over and over.

Go 语言的独特之处在于它允许程序运行时增加栈的大小。这是因为每个 goroutine 都有自己的栈，并且 goroutine 由 Go 运行时管理，而不是由底层操作系统管理（我在第 12 章讨论并发时会详细介绍 goroutine）。这既有优点（Go 栈初始大小较小，占用内存更少），也有缺点（当栈需要增长时，栈上的所有数据都需要复制，这很慢）。此外，还可以编写一些极端情况下的代码，导致栈不断地增长和缩小。

The rules are more complicated when it comes to the data that the pointer points to. In order for Go to allocate the data the pointer points to on the stack, several conditions must be true. The data must be a local variable whose data size is known at compile time. The pointer cannot be returned from the function. If the pointer is passed into a function, the compiler must be able to ensure that these conditions still hold. If the size isn't known, you can't make space for it by moving the stack pointer. If the pointer variable

is returned, the memory that the pointer points to will no longer be valid when the function exits. When the compiler determines that the data can't be stored on the stack, we say that the data the pointer points to *escapes* the stack, and the compiler stores the data on the heap.

当涉及到指针指向的数据时，规则就复杂得多。为了让 Go 语言在栈上分配指针指向的数据，必须满足几个条件。首先，数据必须是编译时已知大小的局部变量。其次，指针不能从函数返回。如果将指针传递给函数，编译器必须确保这些条件仍然成立。如果数据大小未知，就无法通过移动栈指针来腾出空间。如果指针变量被返回，那么当函数退出时，指针指向的内存将不再有效。当编译器判断数据无法存储在栈上时，我们称之为“逃逸栈”，编译器会将数据存储在堆上。

The heap is the memory that's managed by the garbage collector (or by hand in languages like C and C++). I won't discuss the details of garbage collector algorithm implementation, but they are much more complicated than moving a stack pointer. Any data that's stored on the heap is valid as long as it can be tracked back to a pointer type variable on a stack. Once there are no more variables on the stack pointing to that data, either directly or via a chain of pointers, the data becomes *garbage*, and it's the job of the garbage collector to clear it out. This program on [The Go Playground](#) demonstrates when data on the heap becomes garbage.

堆是由垃圾回收器（或者在 C 和 C++ 等语言中由人工管理）管理的内存。我不会讨论垃圾回收器算法实现的细节，但它们远比移动栈指针

复杂得多。只要能够追溯到栈上的指针类型变量，存储在堆上的任何数据都是有效的。一旦栈上没有任何变量直接或通过指针链指向该数据，该数据就变成了垃圾，而垃圾回收器的职责就是将其清除。Go Playground 上的这个程序演示了堆上的数据何时会变成垃圾。

NOTE

笔记

A common source of bugs in C programs is returning a pointer to a local variable. In C, this results in a pointer pointing to invalid memory. The Go compiler is smarter. When it sees that a pointer to a local variable is returned, the local variable's value is stored on the heap.

C 程序中常见的 bug 来源之一是返回指向局部变量的指针。在 C 语言中，这会导致指针指向无效内存。Go 编译器则更智能。当它检测到返回的是指向局部变量的指针时，会将该局部变量的值存储在堆内存中。

The *escape analysis* done by the Go compiler isn't perfect. In some cases, data that could be stored on the stack escapes to the heap. However, the compiler has to be conservative; it can't take the chance of leaving a value on the stack when it might need to be on the heap because leaving a reference to invalid data causes memory corruption. Newer Go releases improve escape analysis.

Go 编译器的逃逸分析并不完美。在某些情况下，原本应该存储在栈上的数据会逃逸到堆上。然而，编译器必须保持保守；它不能冒险将本

应存储在堆上的值留在栈上，因为对无效数据的引用会导致内存损坏。较新的 Go 版本改进了逃逸分析。

You might be wondering: what's so bad about storing things on the heap? Two problems arise related to performance. First, the garbage collector takes time to do its work. It isn't trivial to keep track of all available chunks of free memory on the heap or to track which used blocks of memory still have valid pointers. This is time that's taken away from doing the processing that your program is written to do. Many garbage-collection algorithms have been written, and they can be placed into two rough categories: those that are designed for higher throughput (find the most garbage possible in a single scan) or lower latency (finish the garbage scan as quickly as possible). [Jeffrey Dean](#), the genius behind many of Google's engineering successes, cowrote a paper in 2013 called [“The Tail at Scale”](#). It argues that systems should be optimized for latency, to keep response times low. The garbage collector used by the Go runtime favors low latency. Each garbage-collection cycle is designed to “stop the world” (i.e., pause your program) for fewer than 500 microseconds. However, if your Go program creates lots of garbage, the garbage collector won't be able to find all the garbage during a cycle, slowing the collector and increasing memory usage.

你可能会问：把数据存储在堆上有什么不好呢？主要有两个与性能相关的问题。首先，垃圾回收器需要时间来执行其工作。跟踪堆上所有可用的空闲内存块，或者跟踪哪些已使用的内存块仍然拥有有效的指

针，并非易事。这些时间会被占用，从而影响程序执行其主要处理任务。目前已经开发出许多垃圾回收算法，它们大致可以分为两类：一类旨在提高吞吐量（在一次扫描中尽可能多地找到垃圾），另一类旨在降低延迟（尽快完成垃圾扫描）。谷歌众多工程成功背后的天才工程师杰弗里·迪恩 (Jeffrey Dean) 在 2013 年与人合著了一篇名为《规模化的尾部》(The Tail at Scale) 的论文。该论文认为，系统应该针对延迟进行优化，以保持较低的响应时间。Go 运行时使用的垃圾回收器也倾向于低延迟。每个垃圾回收周期都旨在“停止一切”（即暂停程序），时间不超过 500 微秒。然而，如果你的 Go 程序产生大量垃圾，垃圾回收器将无法在一个周期内找到所有垃圾，从而降低回收速度并增加内存使用量。

NOTE

笔记

If you are interested in the implementation details, you may want to listen to the talk Rick Hudson gave at the International Symposium on Memory Management in 2018, describing the [history and implementation](#) of the Go garbage collector.

如果您对实现细节感兴趣，不妨听听 Rick Hudson 在 2018 年国际内存管理研讨会上发表的演讲，其中介绍了 Go 垃圾回收器的历史和实现。

The second problem deals with the nature of computer hardware. RAM might mean “random access memory,” but the fastest way to read from memory is to read it sequentially. A slice of structs in Go has all the data laid out sequentially in memory. This makes it fast to load and fast to

process. A slice of pointers to structs (or structs whose fields are pointers) has its data scattered across RAM, making it far slower to read and process. Forrest Smith wrote an in-depth [blog.post](#) that explores how much this can affect performance. His numbers indicate that it's roughly two orders of magnitude slower to access data via pointers stored randomly in RAM.

第二个问题与计算机硬件的特性有关。RAM 可能代表“随机存取存储器”，但从内存中读取数据最快的方式是顺序读取。在 Go 语言中，结构体切片的所有数据都是按顺序排列在内存中的，这使得它加载和处理速度都很快。而指向结构体的指针切片（或字段为指针的结构体切片）的数据则分散在 RAM 中，导致读取和处理速度慢得多。Forrest Smith 写了一篇深入的博客文章，探讨了这会对性能造成多大的影响。他的数据显示，通过随机存储在 RAM 中的指针访问数据的速度大约慢两个数量级。

This approach of writing software that's aware of the hardware it's running on is called *mechanical sympathy*. The term comes from the world of car racing, where the idea is that a driver who understands what the car is doing can best squeeze the last bits of performance out of it. In 2011, Martin Thompson began applying the term to software development. Following best practices in Go gives it to you automatically.

这种编写能够感知运行硬件的软件的方法被称为“机械感知”。这个术语源于赛车领域，其理念是，了解赛车运行状态的车手能够最大限度

地发挥赛车的性能。2011年，Martin Thompson开始将这个术语应用于软件开发。遵循Go语言的最佳实践，你就能自动获得这种感知能力。

Compare Go's approach to Java's. In Java, local variables and parameters are stored in the stack, just as in Go. However, as discussed earlier, objects in Java are implemented as pointers. For every object variable instance, only the pointer to it is allocated on the stack; the data within the object is allocated on the heap. Only primitive values (numbers, booleans, and chars) are stored entirely on the stack. This means that the garbage collector in Java has to do a great deal of work. It also means that implementations of the `List` interface in Java are built using a pointer to an array of pointers. Even though they *look* like a linear data structure, reading from them actually involves bouncing through memory, which is highly inefficient. There are similar behaviors for the sequential data types in Python, Ruby, and JavaScript. To work around all this inefficiency, the Java Virtual Machine includes some very clever garbage collectors that do lots of work, some optimized for throughput, some for latency, and all with configuration settings to tune them for the best performance. The virtual machines for Python, Ruby, and JavaScript are less optimized, and their performance suffers accordingly.

比较 Go 和 Java 的方法。在 Java 中，局部变量和参数都存储在栈上，就像 Go 一样。然而，正如前面讨论的，Java 中的对象是以指针的形式实现的。对于每个对象变量实例，只有指向它的指针会被分配到栈上；对象内部的数据则被分配到堆上。只有基本类型（数字、布尔值

和字符) 才会完全存储在栈上。这意味着 Java 中的垃圾回收器需要执行大量的工作。这也意味着 Java 中 `List` 接口的实现是使用指向指针数组的指针来构建的。尽管它们看起来像是线性数据结构, 但从中读取数据实际上需要在内存中来回跳转, 效率非常低下。Python、Ruby 和 JavaScript 中的顺序数据类型也存在类似的行为。为了解决所有这些效率问题, Java 虚拟机包含了一些非常智能的垃圾回收器, 它们执行大量的工作, 有些针对吞吐量进行了优化, 有些针对延迟进行了优化, 并且所有垃圾回收器都提供配置设置, 以便进行最佳性能调整。Python、Ruby 和 JavaScript 的虚拟机优化程度较低, 因此其性能也相应受到影响。

Now you can see why Go encourages you to use pointers sparingly. You reduce the garbage collector's workload by making sure that as much as possible is stored on the stack. Slices of structs or primitive types have their data lined up sequentially in memory for rapid access. And when the garbage collector does do work, it is optimized to return quickly rather than gather the most garbage. The key to making this approach work is to simply create less garbage in the first place. While focusing on optimizing memory allocations can feel like premature optimization, the idiomatic approach in Go is also the most efficient.

现在你应该明白为什么 Go 鼓励你谨慎使用指针了。通过确保尽可能多的数据存储在栈上, 可以减轻垃圾回收器的负担。结构体切片或基本类型的数据在内存中按顺序排列, 以便快速访问。而且, 当垃圾回收器确实需要执行回收任务时, 它会进行优化, 以快速返回结果, 而

不是尽可能多地回收垃圾。这种方法的关键在于从一开始就减少垃圾的产生。虽然专注于优化内存分配可能感觉像是过早优化，但 Go 的惯用方法也是最有效的。

If you want to learn more about heap versus stack allocation and escape analysis in Go, excellent blog posts cover the topic, including ones by [Bill Kennedy of Ardan Labs](#) and [Achille Roussel and Rick Branson of Segment](#).

如果您想了解更多关于 Go 中堆与栈分配和逃逸分析的知识，可以阅读一些优秀的博客文章，其中包括 Ardan Labs 的 Bill Kennedy 和 Segment 的 Achille Roussel 和 Rick Branson 的文章。

Tuning the Garbage Collector

调整垃圾回收器

A garbage collector doesn't immediately reclaim memory as soon as it is no longer referenced. Doing so would seriously impact performance. Instead, it lets the garbage pile up for a bit. The heap almost always contains both live data and memory that's no longer needed. The Go runtime provides users a couple of settings to control the heap's size. The first is the `GOGC` environment variable. The garbage collector looks at the heap size at the end of a garbage-collection cycle and uses the formula

`CURRENT_HEAP_SIZE + CURRENT_HEAP_SIZE * GOGC / 100` to

calculate the heap size that needs to be reached to trigger the next garbage-collection cycle.

垃圾回收器不会在内存不再被引用时立即回收它。这样做会严重影响性能。相反，它会让垃圾堆积一段时间。堆几乎总是同时包含活动数据和不再需要的内存。Go 运行时为用户提供了一些设置来控制堆的大小。第一个是环境变量 `GOGC`。垃圾回收器会在垃圾回收周期结束时检查堆的大小，并使用公式 $\text{CURRENT_HEAP_SIZE} + \text{CURRENT_HEAP_SIZE} * \text{GOGC} / 100$ 来计算触发下一个垃圾回收周期所需的堆大小。

NOTE

笔记

The `GOGC` heap size calculation is a little more complicated than just described. It takes into account not just the heap size, but the sizes of all the stacks of all the goroutines and the memory set aside to hold package-level variables. Most of the time, the heap size is far bigger than the size of these other memory areas, but in some situations they do have an effect.

`GOGC` 堆大小的计算比刚才描述的要复杂一些。它不仅考虑堆的大小，还考虑所有 goroutine 的栈的大小以及用于存储包级变量的内存。大多数情况下，堆的大小远大于这些其他内存区域的大小，但在某些情况下，它们确实会产生影响。

By default, `GOGC` is set to `100`, which means that the heap size that triggers the next collection is roughly double the heap size at the end of the current collection. Setting `GOGC` to a smaller value will decrease the target heap size, and setting it to a larger value will increase it. As a rough

estimate, doubling the value of `GOGC` will halve the amount of CPU time spent on GC.

默认情况下, `GOGC` 设置为 `100`, 这意味着触发下一次垃圾回收的堆大小大约是当前垃圾回收结束时堆大小的两倍。将 `GOGC` 设置为较小的值会减小目标堆大小, 而将其设置为较大的值则会增加目标堆大小。粗略估计, 将 `GOGC` 的值加倍, 将使垃圾回收所花费的 CPU 时间减半。

Setting `GOGC` to `off` disables garbage collection. This will make your programs run faster. However, turning off garbage collection on a long-running process will potentially use all available memory on your computer. This is not usually considered optimal behavior.

将 `GOGC` 设置为 `off` 会禁用垃圾回收。这将加快程序运行速度。但是, 对长时间运行的进程禁用垃圾回收可能会耗尽计算机上的所有可用内存。这通常不被认为是最佳做法。

The second garbage-collection setting specifies a limit on the total amount of memory your Go program is allowed to use. Java developers are likely familiar with the `-Xmx` JVM argument, and `GOMEMLIMIT` is similar. By default, it is disabled (technically, it is set to `math.MaxInt64`, but it's unlikely that your computer has that much memory). The value for `GOMEMLIMIT` is specified in bytes, but you can optionally use the suffixes

B, KiB, MiB, GiB, and TiB. For example, `GOMEMLIMIT=3GiB` sets the memory limit to 3 gibibytes (which is equal to 3,221,225,472 bytes).

第二个垃圾回收设置指定了 Go 程序可以使用的总内存量限制。Java 开发人员可能熟悉 `-Xmx` JVM 参数，而 `GOMEMLIMIT` 与之类似。默认情况下，它处于禁用状态（技术上，它被设置为 `math.MaxInt64`，但您的计算机不太可能拥有如此大的内存）。`GOMEMLIMIT` 的值以字节为单位指定，但您可以选择使用后缀 B、KiB、MiB、GiB 和 TiB。例如，`GOMEMLIMIT=3GiB` 将内存限制设置为 3 GiB（等于 3,221,225,472 字节）。

NOTE

笔记

If you haven't seen these suffixes before, they are the official power-of-two analogues of the more commonly used power-of-ten KB, MB, GB, and TB. KiB is equal to 2^{10} , MiB is equal to 2^{20} , and so on. It is technically correct to use KiB, MiB, and friends when dealing with computers.

如果您之前没见过这些后缀，它们是常用的十的幂次方单位 KB、MB、GB 和 TB 的官方二的幂次方对应单位。KiB 等于 2^{1000} ，MiB 等于 2^{1000} ，以此类推。在计算机领域，使用 KiB、MiB 等单位在技术上是正确的。

It might seem counterintuitive that limiting the maximum amount of memory could improve a program's performance, but there are good reasons this flag was added. The primary reason is that computers (or virtual machines or containers) don't have infinite RAM. If a sudden,

temporary spike occurs in memory usage, relying on `GOGC` alone might result in the maximum heap size exceeding the amount of available memory. This can cause memory to swap to disk, which is very slow. Depending on your operating system and its settings, it might crash your program. Specifying a maximum memory limit prevents the heap from growing beyond the computer's resources.

限制最大内存量反而能提升程序性能，这看似有悖常理，但添加此标志是有充分理由的。主要原因是计算机（或虚拟机、容器）的内存并非无限。如果内存使用量突然出现短暂的峰值，仅依靠 `GOGC` 可能会导致最大堆大小超过可用内存。这会导致内存交换到磁盘，而磁盘交换速度非常慢。根据您的操作系统及其设置，这甚至可能导致程序崩溃。指定最大内存限制可以防止堆大小超出计算机资源。

`GOMEMLIMIT` is a *soft* limit that can be exceeded in certain circumstances. A common problem occurs in garbage-collected systems when the collector is unable to free up enough memory to get within a memory limit or the garbage-collection cycles are rapidly being triggered because a program is repeatedly hitting the limit. Called *thrashing*, this results in a program that does nothing other than run the garbage collector. If the Go runtime detects that thrashing is starting to happen, it chooses to end the current garbage-collection cycle and exceed the limit. This does mean that you should set `GOMEMLIMIT` below the absolute maximum amount of available memory so you have spare capacity.

`GOMEMLIMIT` 是一个软限制，在某些情况下可能会被突破。在启用垃圾回收的系统中，常见的问题是垃圾回收器无法释放足够的内存以使其符合内存限制，或者由于程序反复达到限制而导致垃圾回收周期频繁触发。这种情况被称为抖动，会导致程序除了运行垃圾回收器之外什么也不做。如果 Go 运行时检测到抖动开始发生，它会选择结束当前的垃圾回收周期并突破限制。这意味着您应该将 `GOMEMLIMIT` 设置为低于可用内存的绝对最大值，以便留有余量。

If you specify a value for `GOMEMLIMIT`, you could set `GOGC` to `off` and not run out of memory, but this may not produce the desired performance effect. You are likely to find yourself trading frequent, very short pauses for infrequent, longer pauses. If you are running a web service, this produces inconsistent response times, which was one of the behaviors that Go's garbage collection was designed to avoid.

如果为 `GOMEMLIMIT` 指定一个值，您可以将 `GOGC` 设置为 `off` 而不会耗尽内存，但这可能无法达到预期的性能效果。您可能会发现自己为了避免频繁的短暂暂停而牺牲了不频繁的较长暂停时间。如果您运行的是 Web 服务，这将导致响应时间不稳定，而 Go 的垃圾回收机制正是为了避免这种情况而设计的。

The best option is to use these two environment variables together to ensure both a reasonable pace for garbage collection and a maximum that should be respected. You can learn more about how to use `GOGC` and

GOMEMLIMIT by reading [“A Guide to the Go Garbage Collector”](#) from Go’s development team.

最佳方案是同时使用这两个环境变量，以确保垃圾回收速度合理，并设定一个必须遵守的最大值。您可以阅读 Go 开发团队编写的《Go 垃圾回收器指南》，了解更多关于如何使用 GOGC 和 GOMEMLIMIT 的信息。

Exercises

练习

Now that you have learned about pointers and memory in Go, work through these exercises to reinforce using pointers effectively. You can find answers to these exercises in the [Chapter 6 repository](#).

现在你已经学习了 Go 语言中的指针和内存相关知识，请完成以下练习以巩固指针的有效使用。你可以在第 6 章的文档库中找到这些练习的答案。

1. Create a struct named `Person` with three fields: `FirstName` and `LastName` of type `string` and `Age` of type `int`. Write a function called `MakePerson` that takes in `firstName`, `lastName`, and `age` and returns a `Person`. Write a second

function `MakePersonPointer` that takes in `firstName`, `lastName`, and `age` and returns a `*Person`. Call both from `main`. Compile your program with `go build -gcflags="-m"`. This both compiles your code and prints out which values escape to the heap. Are you surprised about what escapes?

创建一个名为 `Person` 的结构体，包含三个字段：类型为 `string` 的 `FirstName` 和 `LastName`，以及类型为 `int` 的 `Age`。编写一个名为 `MakePerson` 的函数，该函数接收 `firstName`、`lastName` 和 `age` 作为参数，并返回一个 `Person`。编写第二个函数 `MakePersonPointer`，该函数接收 `firstName`、`lastName` 和 `age` 作为参数，并返回一个 `*Person`。从 `main` 调用这两个函数。使用 `go build -gcflags="-m"` 编译你的程序。这将编译你的代码并打印出哪些值逃逸到堆中。你对逃逸的值感到惊讶吗？

2. Write two functions. The `UpdateSlice` function takes in a `[]string` and a `string`. It sets the last position in the passed-in slice to the passed-in `string`. At the end of `UpdateSlice`, print the slice after making the change. The `GrowSlice` function also takes in a `[]string` and a `string`. It appends the `string` onto the slice. At the end of `GrowSlice`, print the slice after making the change. Call these functions from `main`. Print out the slice before each function is called and after each function is called. Do you understand why some changes are visible in `main` and why some changes are not?

编写两个函数。`UpdateSlice` 函数接收 `[]string` 和 `string` 两个参数。它将传入的切片的最后一个位置设置为传入的 `string`。在 `UpdateSlice` 的末尾，打印修改后的切片。`GrowSlice` 函数也接收 `[]string` 和 `string` 两个参数。它将 `string` 追加到切片末尾。在 `GrowSlice` 的末尾，打印修改后的切片。从 `main` 调用这两个函数。在每次调用函数之前和之后，打印出切片。你明白为什么有些更改在 `main` 中可见，而有些更改不可见吗？

3. Write a program that builds a `[]Person` with 10,000,000 entries (they could all be the same names and ages). See how long it takes to run. Change the value of `GOGC` and see how that affects the time it takes for the program to complete. Set the environment variable `GODEBUG=gctrace=1` to see when garbage collections happen and see how changing `GOGC` changes the number of garbage collections. What happens if you create the slice with a capacity of 10,000,000?

编写一个程序，创建一个包含 10,000,000 条记录的 `[]Person` 表（这些记录可以全部具有相同的姓名和年龄）。查看程序运行所需时间。更改 `GOGC` 的值，观察程序运行时间的变化。设置环境变量 `GODEBUG=gctrace=1`，观察垃圾回收的发生时间，并观察更改 `GOGC` 的值如何影响垃圾回收的次数。如果创建容量为 10,000,000 的切片会发生什么？

Wrapping Up

总结

This chapter peeked under the covers a bit to help you understand pointers, what they are, how to use them, and, most importantly, when to use them. In the next chapter, you'll take a look at Go's implementation of methods, interfaces, and types, how they differ from other languages, and the power they possess.

本章将带你深入了解指针，包括指针的定义、使用方法以及最重要的——何时使用指针。下一章，你将了解 Go 语言对方法、接口和类型的实现，它们与其他语言的区别以及它们所蕴含的强大功能。

Chapter 7. Types, Methods, and Interfaces

第七章 类型、方法和接口

As you saw in earlier chapters, Go is a statically typed language with both built-in types and user-defined types. Like most modern languages, Go allows you to attach methods to types. It also has type abstraction, allowing you to write code that invokes methods without explicitly specifying the implementation.

正如你在前面的章节中所看到的，Go 是一种静态类型语言，它既有内置类型，也有用户自定义类型。与大多数现代语言一样，Go 允许你将方法附加到类型上。它还具有类型抽象，允许你编写调用方法的代码，而无需显式指定方法的具体实现。

However, Go's approach to methods, interfaces, and types is very different from that of most other languages in common use today. Go is designed to encourage the best practices advocated by software engineers, avoiding inheritance while encouraging composition. In this chapter, you'll take a look at types, methods, and interfaces, and see how to use them to build testable and maintainable programs.

然而，Go 语言在方法、接口和类型方面的处理方式与当今大多数常用语言截然不同。Go 的设计旨在鼓励软件工程师倡导的最佳实践，避免继承，同时鼓励组合。在本章中，您将了解类型、方法和接口，并学习如何使用它们来构建可测试和可维护的程序。

Types in Go

Go 语言中的类型

Back in [“Structs”](#), you saw how to define a struct type:

在“结构体”部分，你已经了解了如何定义结构体类型：

```
type Person struct {  
    FirstName string  
    LastName  string  
    Age       int  
}
```

This should be read as declaring a user-defined type with the name **Person** to have the *underlying type* of the struct literal that follows. In addition to struct literals, you can use any primitive type or compound type literal to define a concrete type. Here are a few examples:

这应该理解为声明一个名为 `Person` 的用户自定义类型，该类型将具有其后结构体字面量所对应的底层类型。除了结构体字面量之外，您还可以使用任何原始类型或复合类型字面量来定义具体类型。以下是一些示例：

```
type Score int
type Converter func(string)Score
type TeamScores map[string]Score
```

Go allows you to declare a type at any block level, from the package block down. However, you can access the type only from within its scope. The only exceptions are types exported from other packages. I'll talk more about those in [Chapter 10](#).

Go 允许你在任何代码块级别（从包代码块到更底层的代码块）声明类型。但是，你只能在其作用域内访问该类型。唯一的例外是其他包导出的类型。我将在第 10 章详细讨论这些类型。

NOTE

笔记

To make it easier to talk about types, I'll define a couple of terms. An *abstract type* is one that specifies *what* a type should do but not *how* it is done. A *concrete type* specifies what and how. This means that the type has a specified way to store its data and provides an implementation of any methods declared on the type. While all types in Go are either abstract or concrete, some languages allow hybrid types, such as abstract classes or interfaces with default methods in Java.

为了便于讨论类型，我先定义几个术语。抽象类型只规定了类型应该做什么，而不规定如何做。具体类型则规定了做什么以及如何做。这意味着类型有特定的数据存储方式，并且提供了类型上声明的所有方法的实现。虽然 Go 语言中的所有类型要么是抽象的，要么是具体的，但有些语言允许混合类型，例如 Java 中的抽象类或带有默认方法的接口。

Methods

方法

Like most modern languages, Go supports methods on user-defined types.

与大多数现代语言一样，Go 支持用户自定义类型上的方法。

The methods for a type are defined at the package block level:

类型的方法是在包块级别定义的：

```
type Person struct {  
    FirstName string  
    LastName  string  
    Age       int  
}  
  
func (p Person) String() string {  
    return fmt.Sprintf("%s %s, age %d",  
p.FirstName, p.LastName, p.Age)  
}
```

Method declarations look like function declarations, with one addition: the *receiver* specification. The receiver appears between the keyword `func` and the name of the method. Like all other variable declarations, the receiver name appears before the type. By convention, the receiver name is a short abbreviation of the type's name, usually its first letter. It is nonidiomatic to use `this` or `self`.

方法声明与函数声明类似，但多了一个步骤：接收者规范。接收者出现在关键字 `func` 和方法名之间。与其他变量声明一样，接收者名称出现在类型之前。按照惯例，接收者名称是类型名称的缩写，通常是其首字母。使用 `this` 或 `self` 是不规范的。

There is one key difference between declaring methods and functions: methods can be defined *only* at the package block level, while functions can be defined inside any block.

声明方法和声明函数之间有一个关键区别：方法只能在包块级别定义，而函数可以在任何块内定义。

Just like functions, method names cannot be overloaded. You can use the same method names for different types, but you can't use the same method name for two different methods on the same type. While this philosophy feels limiting when coming from languages that have method overloading, not reusing names is part of Go's philosophy of making clear what your code is doing.

就像函数一样，方法名不能重载。你可以对不同的类型使用相同的方法名，但不能对同一类型的两个不同方法使用相同的方法名。虽然这种理念对于习惯于方法重载的语言来说可能显得有些局限，但名称不能重用正是 Go 语言清晰表达代码功能的核心理念之一。

I'll talk more about packages in [Chapter 10](#), but be aware that methods must be declared in the same package as their associated type; Go doesn't allow you to add methods to types you don't control. While you can define a method in a different file within the same package as the type declaration, it is best to keep your type definition and its associated methods together so that it's easy to follow the implementation.

我会在第十章详细讨论包，但请注意，方法必须与其关联的类型声明在同一个包中；Go 不允许你向你无法控制的类型添加方法。虽然你可

以在与类型声明相同的包中的不同文件中定义方法，但最好将类型定义及其关联方法放在一起，这样更容易理解实现。

Method invocations should look familiar to those who have used methods in other languages:

对于使用过其他语言方法的人来说，方法调用应该看起来很熟悉：

```
p := Person{
    FirstName: "Fred",
    LastName:  "Fredson",
    Age:       52,
}
output := p.String()
```

Pointer Receivers and Value Receivers

指针接收器和值接收器

As I covered in [Chapter 6](#), Go uses parameters of pointer type to indicate that a parameter might be modified by the function. The same rules apply for method receivers too. They can be *pointer receivers* (the type is a pointer) or *value receivers* (the type is a value type). The following rules help you determine when to use each kind of receiver:

正如我在第 6 章中所述，Go 使用指针类型的参数来表示该参数可能会被函数修改。同样的规则也适用于方法接收器。它们可以是指针接收器（类型为指针）或值接收器（类型为值类型）。以下规则可帮助您确定何时使用哪种接收器：

- If your method modifies the receiver, you *must* use a pointer receiver.
如果你的方法会修改接收器，则必须使用指针接收器。
- If your method needs to handle `nil` instances (see [“Code Your Methods for nil Instances”](#)), then it *must* use a pointer receiver.
如果你的方法需要处理 `nil` 个实例（参见“为 nil 实例编写方法”），那么它必须使用指针接收器。
- If your method doesn’t modify the receiver, you *can* use a value receiver.
如果你的方法不修改接收器，你可以使用值接收器。

Whether you use a value receiver for a method that doesn’t modify the receiver depends on the other methods declared on the type. When a type has *any* pointer receiver methods, a common practice is to be consistent and use pointer receivers for *all* methods, even the ones that don’t modify the receiver.

对于不修改接收器的方法，是否使用值接收器取决于类型上声明的其他方法。当类型包含指针接收器方法时，通常的做法是保持一致，对所有方法都使用指针接收器，即使是那些不修改接收器的方法。

Here's some simple code to demonstrate pointer and value receivers. It starts with a type that has two methods on it, one using a value receiver, the other with a pointer receiver:

以下是一些简单的代码，用于演示指针接收器和值接收器。代码首先定义了一个类型，该类型有两个方法，一个使用值接收器，另一个使用指针接收器：

```
type Counter struct {
    total      int
    lastUpdated time.Time
}

func (c *Counter) Increment() {
    c.total++
    c.lastUpdated = time.Now()
}

func (c Counter) String() string {
    return fmt.Sprintf("total: %d, last updated: %v", c.total, c.lastUpdated)
}
```

You can then try out these methods with the following code. You can run it yourself on [The Go Playground](#) or use the code in the *sample_code/pointer_value* directory in the [Chapter 7 repository](#):

然后，您可以使用以下代码尝试这些方法。您可以自行在 Go Playground 上运行，也可以使用第 7 章代码库中 `sample_code/pointer_value` 目录下的代码：

```
var c Counter
fmt.Println(c.String())
c.Increment()
fmt.Println(c.String())
```

You should see the following output:

您应该看到以下输出：

```
total: 0, last updated: 0001-01-01 00:00:00 +0000
UTC
total: 1, last updated: 2009-11-10 23:00:00 +0000
UTC m=+0.000000001
```

One thing you might notice is that you were able to call the pointer receiver method even though `c` is a value type. When you use a pointer receiver with a local variable that's a value type, Go automatically takes the address of the local variable when calling the method. In this case, `c.Increment()` is converted to `(&c).Increment()`.

您可能会注意到，即使 `c` 是值类型，您仍然可以调用指针接收器方法。当您将指针接收器与值类型的局部变量一起使用时，Go 会在调用

方法时自动获取局部变量的地址。在本例中，`c.Increment()` 被转换为 `(&c).Increment()`。

If you call a value receiver on a pointer variable, Go automatically dereferences the pointer when calling the method. In the code:

如果对指针变量调用值接收器，Go 会在调用该方法时自动解引用该指针。代码如下：

```
c := &Counter{}  
fmt.Println(c.String())  
c.Increment()  
fmt.Println(c.String())
```

the call `c.String()` is silently converted to `(*c).String()`.

调用 `c.String()` 被静默地转换为 `(*c).String()`。

WARNING

警告

If you call a value receiver method with pointer instance whose value is `nil`, your code will compile but will panic at runtime (I discuss panics in [“panic and recover”](#)).

如果你用值为 `nil` 的指针实例调用值接收器方法，你的代码可以编译，但在运行时会发生 panic（我在“panic and recover”中讨论了 panic）。

Be aware that the rules for passing values to functions still apply. If you pass a value type to a function and call a pointer receiver method on the passed value, you are invoking the method on a *copy*. You can try out the following code on [The Go Playground](#) or use the code in the `sample_code/update_wrong` directory in the [Chapter 7 repository](#):

请注意，向函数传递值的规则仍然适用。如果您将值类型传递给函数，并对传递的值调用指针接收器方法，则您是在对副本调用该方法。您可以在 Go Playground 上尝试以下代码，或者使用第 7 章代码库中 `sample_code/update_wrong` 目录下的代码：

```
func doUpdateWrong(c Counter) {
    c.Increment()
    fmt.Println("in doUpdateWrong:", c.String())
}

func doUpdateRight(c *Counter) {
    c.Increment()
    fmt.Println("in doUpdateRight:", c.String())
}

func main() {
    var c Counter
    doUpdateWrong(c)
    fmt.Println("in main:", c.String())
    doUpdateRight(&c)
```

```
    fmt.Println("in main:", c.String())  
}
```

When you run this code, you'll get the following output:

运行这段代码后，你会得到以下输出：

```
in doUpdateWrong: total: 1, last updated: 2009-11-  
10 23:00:00 +0000 UTC  
    m=+0.000000001  
in main: total: 0, last updated: 0001-01-01  
00:00:00 +0000 UTC  
in doUpdateRight: total: 1, last updated: 2009-11-  
10 23:00:00 +0000 UTC  
    m=+0.000000001  
in main: total: 1, last updated: 2009-11-10  
23:00:00 +0000 UTC m=+0.000000001
```

The parameter in `doUpdateRight` is of type `*Counter`, which is a pointer instance. As you can see, you can call both `Increment` and `String` on it. Go considers both pointer and value receiver methods to be in the *method set* for a pointer instance. For a value instance, only the value receiver methods are in the method set. This seems like a pedantic detail right now, but I'll come back to it when talking about interfaces in just a bit.

`doUpdateRight` 中的参数类型为 `*Counter`，它是一个指针实例。如您所见，您可以对其调用 `Increment` 和 `String`。Go 认为指针实例的方法集中包含了指针方法和值接收方法。而对于值实例，方法集中只包含了值接收方法。这现在看起来似乎是一个无关紧要的细节，但稍后在讨论接口时我会再提到它。

NOTE

笔记

This can be confusing to new Go programmers (and, to be honest, to not-so-new Go programmers), but Go's automatic conversion from pointer types to value types, and vice versa, is purely syntactic sugar. It is independent of the method set concept. Alexey Gronskiy has written a [blog post](#) that explores in detail why the method set of pointer instances have both pointer and value receiver methods, but the method set of value instances has only value receiver methods.

这可能会让 Go 新手程序员感到困惑（说实话，对一些经验丰富的程序员来说也是如此），但 Go 中指针类型和值类型之间的自动转换纯粹是语法糖，与方法集概念无关。Alexey Gronskiy 写了一篇博文，详细探讨了为什么指针实例的方法集既包含指针接收方法也包含值接收方法，而值实例的方法集却只有值接收方法。

One final note: do not write getter and setter methods for Go structs unless you need them to meet an interface (I'll start covering interfaces in [“A Quick Lesson on Interfaces”](#)). Go encourages you to directly access a field. Reserve methods for business logic. The exceptions are when you need to update multiple fields as a single operation or when the update isn't a straightforward assignment of a new value. The `Increment` method defined earlier demonstrates both of these properties.

最后一点：除非为了满足接口要求（我会在“接口速成课”中讲解接口），否则不要为 Go 结构体编写 getter 和 setter 方法。Go 鼓励直接访问字段。将方法留给业务逻辑。例外情况是需要一次性更新多个字段，或者更新并非简单的赋值操作。前面定义的 `Increment` 方法就体现了这两种特性。

Code Your Methods for nil Instances

为空实例编写方法

The previous section covered pointer receivers, which might make you wonder what happens when you call a method on a `nil` instance. In most languages, this produces some sort of error. (Objective-C allows you to call a method on a `nil` instance, but it always does nothing.)

上一节介绍了指针接收器，这可能会让你好奇，当你在 `nil` 实例上调用方法时会发生什么。在大多数语言中，这会产生某种错误。

(Objective-C 允许你在 `nil` 实例上调用方法，但它始终不会执行任何操作。)

Go does something a little different. It actually tries to invoke the method. As mentioned earlier, if it's a method with a value receiver, you'll get a panic, since there is no value being pointed to by the pointer. If it's a method with a pointer receiver, it can work if the method is written to handle the possibility of a `nil` instance.

Go 的处理方式略有不同。它实际上会尝试调用该方法。如前所述，如果该方法接收的是值，则会引发 panic，因为指针没有指向任何值。如果该方法接收的是指针，并且该方法被编写为能够处理 nil 实例的情况，则可以正常工作。

In some cases, expecting a nil receiver makes the code simpler. Here's an implementation of a binary tree that takes advantage of nil values for the receiver:

在某些情况下，预期接收者为 nil 可以简化代码。以下是一个利用接收者为 nil 值的二叉树实现示例：

```
type IntTree struct {
    val      int
    left, right *IntTree
}

func (it *IntTree) Insert(val int) *IntTree {
    if it == nil {
        return &IntTree{val: val}
    }
    if val < it.val {
        it.left = it.left.Insert(val)
    } else if val > it.val {
        it.right = it.right.Insert(val)
    }
    return it
}
```

```
}

func (it *IntTree) Contains(val int) bool {
    switch {
    case it == nil:
        return false
    case val < it.val:
        return it.left.Contains(val)
    case val > it.val:
        return it.right.Contains(val)
    default:
        return true
    }
}
```

NOTE

笔记

The `Contains` method doesn't modify the `*IntTree`, but it is declared with a pointer receiver. This demonstrates the rule mentioned previously about supporting a `nil` receiver. A method with a value receiver can't check for `nil` and, as mentioned earlier, panics if invoked with a `nil` receiver.

`Contains` 方法不会修改 `*IntTree`，但它声明时使用了指针接收器。这说明了之前提到的关于支持 `nil` 接收器的规则。使用值接收器的方法无法检查 `nil`，并且如前所述，如果使用 `nil` 接收器调用，则会引发 panic。

The following code uses the tree. You can try it out on [The Go Playground](#) or use the code in the *sample_code/tree* directory in the [Chapter 7 repository](#):

以下代码使用了树结构。您可以在 Go Playground 上尝试运行，或者使用第 7 章代码库中 *sample_code/tree* 目录下的代码：

```
func main() {
    var it *IntTree
    it = it.Insert(5)
    it = it.Insert(3)
    it = it.Insert(10)
    it = it.Insert(2)
    fmt.Println(it.Contains(2)) // true
    fmt.Println(it.Contains(12)) // false
}
```

It's very clever that Go allows you to call a method on a `nil` receiver, and there are situations where it is useful, like the previous tree node example. However, most of the time it's not very useful. Pointer receivers work like pointer function parameters; it's a copy of the pointer that's passed into the method. Just like `nil` parameters passed to functions, if you change the copy of the pointer, you haven't changed the original. This means you can't write a pointer receiver method that handles `nil` and makes the original pointer non-nil.

Go 允许你调用 `nil` 接收器上的方法，这非常巧妙，在某些情况下确实很有用，例如前面的树节点示例。然而，大多数情况下，它并不实用。指针接收器的工作方式类似于指针函数参数；传递给方法的是指针的副本。就像传递给函数的 `nil` 参数一样，如果你修改了指针的副本，原始指针并没有改变。这意味着你不能编写一个指针接收器方法来处理 `nil` 并将原始指针变为非空值。

If your method has a pointer receiver and won't work for a `nil` receiver, you have to decide how your method should handle a `nil` receiver. One choice is to treat it as a fatal flaw, like trying to access a position in a slice beyond its length. In that case, don't do anything and let the code panic. (Also make sure you write good tests, as discussed in [Chapter 15](#).) If a `nil` receiver is something that is recoverable, check for `nil` and return an error (I discuss errors in [Chapter 9](#)).

如果你的方法有一个指针接收器，但无法处理 `nil` 接收器，那么你需要决定你的方法应该如何处理 `nil` 接收器。一种方法是将其视为致命缺陷，就像试图访问切片中超出其长度的位置一样。在这种情况下，什么都不做，让代码崩溃。（还要确保编写良好的测试，如第 15 章所述。）如果 `nil` 接收器是可恢复的，则检查 `nil` 并返回错误（我在第 9 章讨论了错误）。

Methods Are Functions Too

方法也是函数

Methods in Go are so much like functions that you can use a method as a replacement for a function anytime there's a variable or parameter of a function type.

Go 中的方法与函数非常相似，因此，只要存在函数类型的变量或参数，就可以使用方法来代替函数。

Let's start with this simple type:

我们先从这种简单的类型开始：

```
type Adder struct {  
    start int  
}  
  
func (a Adder) AddTo(val int) int {  
    return a.start + val  
}
```

You create an instance of the type in the usual way and invoke its method:

您可以按常规方式创建该类型的实例并调用其方法：

```
myAdder := Adder{start: 10}  
fmt.Println(myAdder.AddTo(5)) // prints 15
```

You can also assign the method to a variable or pass it to a parameter of type `func(int)int`. This is called a *method value*:

您还可以将方法赋值给一个变量，或将其传递给类型为 `func(int)int` 的参数。这称为方法值：

```
f1 := myAdder.AddTo  
fmt.Println(f1(10))           // prints 20
```

A method value is a bit like a closure, since it can access the values in the fields of the instance from which it was created.

方法值有点像闭包，因为它可以访问创建它的实例的字段中的值。

You can also create a function from the type itself. This is called a *method expression*:

你还可以从类型本身创建函数。这称为方法表达式：

```
f2 := Adder.AddTo  
fmt.Println(f2(myAdder, 15)) // prints 25
```

In a method expression, the first parameter is the receiver for the method; the function signature is `func(Adder, int) int`.

在方法表达式中，第一个参数是方法的接收者；函数签名是 `func(Adder, int) int`。

Method values and method expressions aren't clever corner cases. You'll see one way to use them when you look at dependency injection in ["Implicit Interfaces Make Dependency Injection Easier"](#).

方法值和方法表达式并非什么巧妙的特殊情况。在“隐式接口使依赖注入更容易”一文中，您将看到使用它们的一种方法。

Functions Versus Methods

函数与方法

Since you can use a method as a function, you might wonder when you should declare a function and when you should use a method.

既然你可以将方法用作函数，你可能会想知道何时应该声明函数，何时应该使用方法。

The differentiator is whether your function depends on other data. As I've covered several times, package-level state should be effectively immutable. Anytime your logic depends on values that are configured at startup or

changed while your program is running, those values should be stored in a struct, and that logic should be implemented as a method. If your logic depends only on the input parameters, it should be a function.

区别在于你的函数是否依赖于其他数据。正如我多次提到的，包级状态应该是不可变的。任何时候，如果你的逻辑依赖于启动时配置的值或程序运行时更改的值，这些值都应该存储在一个结构体中，并且该逻辑应该实现为一个方法。如果你的逻辑仅依赖于输入参数，那么它应该是一个函数。

Type Declarations Aren't Inheritance

类型声明不等于继承

In addition to declaring types based on built-in Go types and struct literals, you can also declare a user-defined type based on another user-defined type:

除了基于 Go 内置类型和结构字面量声明类型之外，你还可以基于另一个用户定义类型声明用户定义类型：

```
type HighScore Score
type Employee Person
```

Many concepts can be considered “object-oriented,” but one stands out: *inheritance*. With inheritance, the state and methods of a *parent* type are

declared to be available on a *child* type, and values of the child type can be substituted for the parent type.¹

许多概念都可以被认为是“面向对象的”，但其中一个尤为突出：继承。通过继承，父类型的状态和方法可以被声明为对子类型可用，并且子类型的值可以替换父类型的值。

Declaring a type based on another type looks a bit like inheritance but isn't. The two types have the same underlying type, but that's all. The types have no hierarchy. In languages with inheritance, a child instance can be used anywhere the parent instance is used. The child instance also has all the methods and data structures of the parent instance. That's not the case in Go. You can't assign an instance of type `HighScore` to a variable of type `Score`, or vice versa, without a type conversion, nor can you assign either of them to a variable of type `int` without a type conversion.

Furthermore, any methods defined on `Score` aren't defined on `HighScore`:

基于另一个类型声明一个类型看起来有点像继承，但实际上并非如此。这两个类型具有相同的底层类型，仅此而已。它们之间没有层级关系。在支持继承的语言中，子实例可以在任何使用父实例的地方使用。子实例也拥有父实例的所有方法和数据结构。但在 Go 语言中并非如此。你不能将类型为 `HighScore` 的实例赋值给类型为 `Score` 的变量，反之亦然，除非进行类型转换；同样，你也不能将它们中的

任何一个赋值给类型为 `int` 的变量，除非进行类型转换。此外，在 `Score` 上定义的任何方法都不会在 `HighScore` 上定义。

```
// assigning untyped constants is valid
var i int = 300
var s Score = 100
var hs HighScore = 200
hs = s // compilation error!
s = i // compilation error!
s = Score(i) // ok
hs = HighScore(s) // ok
```

User-defined types whose underlying types are built-in types can be assigned literals and constants compatible with the underlying type. They can also be used with the operators for those types:

用户自定义类型（其底层类型为内置类型）可以赋值与底层类型兼容的字面量和常量。它们还可以与这些类型的运算符一起使用：

```
var s Score = 50
scoreWithBonus := s + 100 // type of
scoreWithBonus is Score
```

TIP

提示

A type conversion between types that share an underlying type keeps the same underlying storage but associates different methods.

共享底层类型的类型之间的类型转换会保持相同的底层存储，但关联不同的方法。

Types Are Executable Documentation

类型是可执行文档

While it's well understood that you should declare a struct type to hold a set of related data, it's less clear when you should declare a user-defined type based on other built-in types or one user-defined type that's based on another user-defined type. The short answer is that types are documentation. They make code clearer by providing a name for a concept and describing the kind of data that is expected. It's clearer for someone reading your code when a method has a parameter of type `Percentage` than of type `int`, and it's harder for it to be invoked with an invalid value.

虽然大家都明白应该声明一个结构体类型来保存一组相关数据，但何时应该基于其他内置类型声明用户自定义类型，或者何时应该声明一个基于另一个用户自定义类型的用户自定义类型，这一点却不太明确。简而言之，类型就是文档。它们通过为概念命名并描述预期的数据类型，使代码更清晰易懂。对于阅读代码的人来说，方法参数类型

为 `Percentage` 比类型为 `int` 更清晰，而且也更不容易被无效值调用。

The same logic applies when declaring one user-defined type based on another user-defined type. When you have the same underlying data, but different sets of operations to perform, make two types. Declaring one as being based on the other avoids some repetition and makes it clear that the two types are related.

当基于另一个用户自定义类型声明一个用户自定义类型时，也适用同样的逻辑。如果底层数据相同，但需要执行的操作集不同，则应创建两个类型。声明其中一个类型基于另一个类型，可以避免重复代码，并明确表明这两个类型之间的关系。

iota Is for Enumerations—Sometimes

iota 代表枚举——有时

Many programming languages have the concept of enumerations, which allow you to specify that a type can have only a limited set of values. Go doesn't have an enumeration type. Instead, it has `iota`, which lets you assign an increasing value to a set of constants.

许多编程语言都有枚举的概念，允许你指定某种类型只能拥有有限的值。Go 语言没有枚举类型，而是使用了 `iota`，允许你为一组常量赋值，且值递增。

NOTE

笔记

The concept of `iota` comes from the programming language APL (which stood for “A Programming Language”). To generate a list of the first three positive integers in APL, you write `⍋3`, where `⍋` is the lowercase Greek letter iota.

`iota` 的概念源自编程语言 APL（全称为“A Programming Language”）。在 APL 中，要生成前三个正整数的列表，需要编写 `⍋3`，其中 `⍋` 是小写希腊字母 iota。

APL is famous for being so reliant on its own custom notation that it required computers with a special keyboard. For example, $(\sim R \in R \circ . \times R) / R \leftarrow 1 \downarrow \iota R$ is an APL program to find all the prime numbers up to the value of the variable R.

APL 因其高度依赖自身特有的符号系统而闻名，以至于需要配备特殊键盘的计算机才能运行。例如， $(\sim R \in R \circ . \times R) / R \leftarrow 1 \downarrow \iota R$ 是一个 APL 程序，用于查找小于等于变量 R 的所有质数。

It may seem ironic that a language as focused on readability as Go would borrow a concept from a language that is concise to a fault, but this is why you should learn multiple programming languages: you can find inspiration everywhere.

Go 语言非常注重可读性，但它却借鉴了以简洁著称的语言的概念，这似乎有些讽刺意味。但这正是你应该学习多种编程语言的原因：你可以在任何地方找到灵感。

When using `iota`, the best practice is to first define a type based on `int` that will represent all the valid values:

使用 `iota` 时，最佳实践是首先基于 `int` 定义一个类型，该类型将表示所有有效值：

```
type MailCategory int
```

Next, use a `const` block to define a set of values for your type:

接下来，使用 `const` 代码块为您的类型定义一组值：

```
const (  
    Uncategorized MailCategory = iota  
    Personal  
    Spam  
    Social  
    Advertisements  
)
```

The first constant in the `const` block has the type specified, and its value is set to `iota`. Every subsequent line has neither the type nor a value assigned to it. When the Go compiler sees this, it repeats the type and the assignment to all the subsequent constants in the block, which is `iota`. The value of `iota` increments for each constant defined in the `const` block, starting with `0`. This means that `0` is assigned to the first constant (`Uncategorized`), 1 to the second constant (`Personal`), and so on. When a new `const` block is created, `iota` is set back to 0.

`const` 代码块中的第一个常量具有指定的类型，其值被设置为 `iota`。后续每一行都没有指定类型或赋值。Go 编译器看到这种情况后，会将该类型和赋值重复应用于代码块中所有后续的常量，即 `iota`。`iota` 的值会随着 `const` 代码块中定义的所有常量递增，从 0 开始。这意味着 0 被赋值给第一个常量 (`Uncategorized`)，1 被赋值给第二个常量 (`Personal`)，依此类推。当创建一个新的 `const` 代码块时，`iota` 会被重置为 0。

The value of `iota` increments for each constant in the `const` block, whether or not `iota` is used to define the value of a constant. The following code demonstrates what happens when `iota` is used intermittently in a `const` block:

无论是否使用 `iota` 来定义常量的值，`iota` 的值都会在 `const` 代码块中的每个常量处递增。以下代码演示了在 `const` 代码块中间歇性使用 `iota` 时会发生什么：

```
const (  
    Field1 = 0  
    Field2 = 1 + iota  
    Field3 = 20  
    Field4  
    Field5 = iota  
)  
  
func main() {
```

```
    fmt.Println(Field1, Field2, Field3, Field4,  
Field5)  
}
```

You can run this code on [The Go Playground](#) and see the (perhaps unexpected) result:

你可以在 Go Playground 上运行这段代码，看看（或许出乎意料的）结果：

```
0 2 20 20 4
```

`Field2` is assigned `2` because `iota` has a value of `1` on the second line in the `const` block. `Field4` is assigned `20` because it has no type or value explicitly assigned, so it gets the value of the previous line with a type and assignment. Finally, `Field5` gets the value `4` because it is the fifth line and `iota` starts counting from `0`.

`Field2` 被赋值为 `2`，因为 `iota` 在 `const` 代码块的第二行中值为 `1`。`Field4` 被赋值为 `20`，因为它没有显式地赋值或类型，所以它获取了上一行已赋值或类型明确的代码块的值。最后，`Field5` 被赋值为 `4`，因为它是第五行，而 `iota` 从 `0` 开始计数。

This is the best advice I've seen on `iota`:

这是我在 `iota` 上看到的最好的建议：

Don't use iota for defining constants where its values are explicitly defined (elsewhere). For example, when implementing parts of a specification and the specification says which values are assigned to which constants, you should explicitly write the constant values. Use iota for "internal" purposes only. That is, where the constants are referred to by name rather than by value. That way, you can optimally enjoy iota by inserting new constants at any moment in time / location in the list without the risk of breaking everything.

—[Danny van Heumen](#)

不要使用 `iota` 来定义那些其值已在其他地方明确定义的常量。例如，在实现规范的某些部分时，如果规范中已明确规定了哪些值分配给哪些常量，则应显式地写出这些常量值。`iota` 仅用于“内部”用途。也就是说，常量应通过名称而非值来引用。这样，您就可以随时在列表中插入新的常量，而无需担心破坏整个规范，从而最大限度地发挥 `iota` 的优势。——*Danny van Heumen*

The important thing to understand is that nothing in Go will stop you (or anyone else) from creating additional values of your type. Furthermore, if you insert a new identifier in the middle of your list of literals, all subsequent ones will be renumbered. This will break your application in a subtle way if those constants represented values in another system or in a

database. Given these two limitations, `iota`-based enumerations make sense only when you care about being able to differentiate between a set of values and don't particularly care what the value is behind the scenes. If the actual value matters, specify it explicitly.

需要理解的关键是，Go 语言不会阻止你（或任何人）创建其他类型的值。此外，如果你在字面量列表中间插入一个新的标识符，所有后续的标识符都会被重新编号。如果这些常量代表的是其他系统或数据库中的值，这将以一种不易察觉的方式破坏你的应用程序。鉴于这两个限制，基于 `iota` 的枚举仅在你关心区分一组值，而不太关心其底层值是什么时才有意义。如果实际值很重要，请显式地指定它。

WARNING

警告

Because you can assign a literal expression to a constant, you'll see sample code that suggests you should use `iota` for cases like this:

因为您可以将字面表达式赋值给常量，所以您会看到示例代码建议您在类似情况下使用 `iota`：

```
type BitField int

const (
    Field1 BitField = 1 << iota // assigned 1
    Field2                // assigned 2
    Field3                // assigned 4
    Field4                // assigned 8
)
```

While this is clever, be careful when using this pattern. If you do so, document what you are doing. As mentioned previously, using `iota` with constants is fragile when you care about the value. You don't want a future maintainer to insert a new constant in the middle of the list and break your code.

虽然这种方法很巧妙，但使用时务必谨慎。如果一定要用，请务必记录你的操作。如前所述，当你需要关注常量的值时，使用 `iota` 来存储常量是不安全的。你肯定不希望未来的维护者在列表中间插入一个新的常量，从而破坏你的代码。

Be aware that `iota` starts numbering from 0. If you are using your set of constants to represent different configuration states, the zero value might be useful. You saw this earlier in the `MailCategory` type. When mail first arrives, it is uncategorized, so the zero value makes sense. If there isn't a

sensical default value for your constants, a common pattern is to assign the first `iota` value in the constant block to `_` or to a constant that indicates the value is invalid. This makes it easy to detect when a variable has not been properly initialized.

请注意，`iota` 的编号从 0 开始。如果您使用一组常量来表示不同的配置状态，那么零值可能很有用。您之前在 `MailCategory` 类型中已经看到过这一点。邮件刚到达时是未分类的，因此零值是合理的。如果您的常量没有合理的默认值，一种常见的做法是将常量块中的第一个 `iota` 值赋给 `_` 或一个指示该值无效的常量。这样可以轻松检测到变量何时未正确初始化。

Use Embedding for Composition

使用嵌入进行组合

The software engineering advice “Favor object composition over class inheritance” dates back to at least the 1994 book *Design Patterns* by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (Addison-Wesley), better known as the Gang of Four book. While Go doesn’t have inheritance, it encourages code reuse via built-in support for composition and promotion:

软件工程建议“优先使用对象组合而非类继承”至少可以追溯到 1994 年由 Erich Gamma、Richard Helm、Ralph Johnson 和 John Vlissides 合著的《设计模式》（Addison-Wesley 出版社），这本书也被称为“四人帮”之作。虽然 Go 语言没有继承机制，但它通过内置的组合和提升机制鼓励代码重用：

```
type Employee struct {
    Name      string
    ID        string
}

func (e Employee) Description() string {
    return fmt.Sprintf("%s (%s)", e.Name, e.ID)
}

type Manager struct {
    Employee
    Reports []Employee
}

func (m Manager) FindNewEmployees() []Employee {
    // do business logic
}
```

Note that `Manager` contains a field of type `Employee`, but no name is assigned to that field. This makes `Employee` an *embedded field*. Any

fields or methods declared on an embedded field are *promoted* to the containing struct and can be invoked directly on it. That makes the following code valid:

请注意，`Manager` 包含一个类型为 `Employee` 的字段，但该字段未赋值。这使得 `Employee` 成为一个嵌入字段。在嵌入字段上声明的任何字段或方法都会被提升到包含该字段的结构体中，并可以直接在其上调用。因此，以下代码有效：

```
m := Manager{
    Employee: Employee{
        Name: "Bob Bobson",
        ID:   "12345",
    },
    Reports: []Employee{},
}
fmt.Println(m.ID)           // prints 12345
fmt.Println(m.Description()) // prints Bob Bobson
                             (12345)
```

NOTE

笔记

You can embed any type within a struct, not just another struct. This promotes the methods on the embedded type to the containing struct.

你可以将任何类型的元素嵌入到结构体中，而不仅仅是另一个结构体。这样做会将嵌入类型的方法提升到包含该结构体的结构体中。

If the containing struct has fields or methods with the same name as an embedded field, you need to use the embedded field's type to refer to the obscured fields or methods. If you have types defined like this:

如果包含结构体中的字段或方法与嵌入字段同名，则需要使用嵌入字段的类型来引用这些被隐藏的字段或方法。例如，如果类型定义如下：

```
type Inner struct {  
    X int  
}  
  
type Outer struct {  
    Inner  
    X int  
}
```

you can access the `X` on `Inner` only by specifying `Inner` explicitly:

只有明确指定 `Inner` , 才能访问 `Inner` 上的 `x` :

```
o := Outer{
    Inner: Inner{
        X: 10,
    },
    X: 20,
}
fmt.Println(o.X)           // prints 20
fmt.Println(o.Inner.X)    // prints 10
```

Embedding Is Not Inheritance

嵌入并非继承

Built-in embedding support is rare in programming languages (I'm not aware of another popular language that supports it). Many developers who are familiar with inheritance (which is available in many languages) try to understand embedding by treating it as inheritance. That way lies tears. You cannot assign a variable of type `Manager` to a variable of type `Employee` . If you want to access the `Employee` field in `Manager` , you must do so explicitly. You can run the following code on [The Go Playground](#) or use the code in the `sample_code/embedding` directory in the [Chapter 7 repository](#):

编程语言中内置嵌入支持的情况很少见（据我所知，目前还没有其他流行的语言支持它）。许多熟悉继承（许多语言都支持继承）的开发者试图通过将其视为继承来理解嵌入。这样做会遇到很多问题。你不能将类型为 `Manager` 的变量赋值给类型为 `Employee` 的变量。如果你想访问 `Manager` 中的 `Employee` 字段，则必须显式地进行操作。你可以在 Go Playground 上运行以下代码，或者使用第 7 章代码库中 `sample_code/embedding` 目录下的代码：

```
var eFail Employee = m           // compilation
error!
var eOK Employee = m.Employee // ok!
```

You'll get the error:

你会收到以下错误信息：

```
cannot use m (type Manager) as type Employee in
assignment
```

Furthermore, Go has no *dynamic dispatch* for concrete types. The methods on the embedded field have no idea they are embedded. If you have a method on an embedded field that calls another method on the embedded field, and the containing struct has a method of the same name, the method on the embedded field is invoked, not the method on the containing struct. This behavior is demonstrated in the following code, which you can run on

[The Go Playground](#), or use the code in the *sample_code/no_dispatch* directory in the [Chapter 7 repository](#):

此外，Go 语言对具体类型没有动态分派机制。嵌入字段上的方法并不知道它们被嵌入了。如果一个嵌入字段上的方法调用了该字段上的另一个方法，而包含该字段的结构体也有一个同名方法，那么调用的是嵌入字段上的方法，而不是包含该字段的结构体上的方法。以下代码演示了这种行为，您可以在 Go Playground 上运行此代码，或者使用第 7 章代码库中 *sample_code/no_dispatch* 目录下的代码：

```
type Inner struct {
    A int
}

func (i Inner) IntPrinter(val int) string {
    return fmt.Sprintf("Inner: %d", val)
}

func (i Inner) Double() string {
    return i.IntPrinter(i.A * 2)
}

type Outer struct {
    Inner
    S string
}
```

```
func (o Outer) IntPrinter(val int) string {
    return fmt.Sprintf("Outer: %d", val)
}

func main() {
    o := Outer{
        Inner: Inner{
            A: 10,
        },
        S: "Hello",
    }
    fmt.Println(o.Double())
}
```

Running this code produces the following output:

运行此代码会产生以下输出：

```
Inner: 20
```

While embedding one concrete type inside another won't allow you to treat the outer type as the inner type, the methods on an embedded field do count toward the *method set* of the containing struct. This means they can make the containing struct implement an interface.

虽然将一个具体类型嵌入到另一个具体类型中并不会让你将外部类型视为内部类型，但嵌入字段上的方法确实会计入包含结构体的方法集。这意味着它们可以使包含结构体实现某个接口。

A Quick Lesson on Interfaces

接口速成课

While Go's concurrency model (which I cover in [Chapter 12](#)) gets all the publicity, the real star of Go's design is its implicit interfaces, the only abstract type in Go. Let's see what makes them so great.

虽然 Go 的并发模型（我在第 12 章会详细介绍）备受瞩目，但 Go 设计的真正亮点在于其隐式接口，这是 Go 中唯一的抽象类型。让我们来看看它们究竟有何过人之处。

Let's start by taking a quick look at how to declare interfaces. At their core, interfaces are simple. Like other user-defined types, you use the `type` keyword.

我们先快速了解一下如何声明接口。接口的核心很简单。与其他用户自定义类型一样，您可以使用 `type` 关键字。

Here's the definition of the `Stringer` interface in the `fmt` package:

以下是 `fmt` 包中 `Stringer` 接口的定义：

```
type Stringer interface {  
    String() string  
}
```

In an interface declaration, an interface literal appears after the name of the interface type. It lists the methods that must be implemented by a concrete type to meet the interface. The methods defined by an interface are the method set of the interface. As I covered in [“Pointer Receivers and Value Receivers”](#), the method set of a pointer instance contains the methods defined with both pointer and value receivers, while the method set of a value instance contains only the methods with value receivers. Here’s a quick example using the `Counter` struct defined previously:

在接口声明中，接口字面量出现在接口类型名称之后。它列出了具体类型必须实现的、才能满足接口要求的方法。接口定义的方法构成了接口的方法集。正如我在“指针接收器和值接收器”一节中所述，指针实例的方法集包含同时定义了指针接收器和值接收器的方法，而值实例的方法集仅包含定义了值接收器的方法。以下是一个使用之前定义的 `Counter` 结构体的简单示例：

```
type Incrementer interface {  
    Increment()  
}
```

```
var myStringer fmt.Stringer
var myIncrementer Incrementer
pointerCounter := &Counter{}
valueCounter := Counter{}

myStringer = pointerCounter    // ok
myStringer = valueCounter      // ok
myIncrementer = pointerCounter // ok
myIncrementer = valueCounter   // compile-time
error!
```

Trying to compile this code results in the error `cannot use valueCounter (variable of type Counter) as Incrementer value in assignment: Counter does not implement Incrementer (method Increment has pointer receiver)`.

尝试编译此代码时出现错误 `cannot use valueCounter (variable of type Counter) as Incrementer value in assignment: Counter does not implement Incrementer (method Increment has pointer receiver)`.

You can try this code on [The Go Playground](#) or use the code in the `sample_code/method_set` directory in the [Chapter 7 repository](#).

您可以在 Go Playground 上尝试这段代码，或者使用第 7 章存储库中的 `sample_code/method_set` 目录中的代码。

Like other types, interfaces can be declared in any block.

与其他类型一样，接口可以在任何代码块中声明。

Interfaces are usually named with “er” endings. You’ve already seen `fmt.Stringer`, but there are many more, including `io.Reader`, `io.Closer`, `io.ReadCloser`, `json.Marshaler`, and `http.Handler`.

接口名称通常以“er”结尾。您已经见过 `fmt.Stringer`，但还有很多，包括 `io.Reader`、`io.Closer`、`io.ReadCloser`、`json.Marshaler` 和 `http.Handler`。

Interfaces Are Type-Safe Duck Typing

接口是类型安全的鸭子类型

So far, nothing that’s been said about the Go interface is much different from interfaces in other languages. What makes Go’s interfaces special is that they are implemented *implicitly*. As you’ve seen with the `Counter` struct type and the `Incrementer` interface type that you’ve used in previous examples, a concrete type does not declare that it implements an

interface. If the method set for a concrete type contains all the methods in the method set for an interface, the concrete type implements the interface. Therefore, that the concrete type can be assigned to a variable or field declared to be of the type of the interface.

到目前为止，我们讨论的 Go 接口与其他语言的接口并没有太大区别。Go 接口的特殊之处在于它们是隐式实现的。正如你在之前的示例中看到的 `Counter` 结构体类型和 `Incrementer` 接口类型，具体类型并不需要声明它实现了某个接口。如果一个具体类型的方法集包含了接口方法集中的所有方法，那么该具体类型就实现了该接口。因此，可以将具体类型赋值给声明为接口类型的变量或字段。

This implicit behavior makes interfaces the most interesting thing about types in Go, because they enable both type safety and decoupling, bridging the functionality in both static and dynamic languages.

这种隐式行为使得接口成为 Go 语言类型中最有趣的部分，因为它们既实现了类型安全，又实现了解耦，从而连接了静态语言和动态语言的功能。

To understand why, let's talk about why languages have interfaces. Earlier I mentioned that *Design Patterns* taught developers to favor composition over inheritance. Another piece of advice from the book is “Program to an interface, not an implementation.” Doing so allows you to depend on behavior, not on implementation, allowing you to swap implementations as

needed. This allows your code to evolve over time, as requirements inevitably change.

为了理解其中的原因，我们先来谈谈编程语言为什么需要接口。我之前提到过，设计模式教导开发者优先选择组合而非继承。书中另一条建议是“面向接口编程，而不是面向实现编程”。这样做可以让你依赖于行为而非实现，从而允许你根据需要切换实现。这使得你的代码能够随着需求的变化而不断演进。

Dynamically typed languages like Python, Ruby, and JavaScript don't have interfaces. Instead, those developers use *duck typing*, which is based on the expression “If it walks like a duck and quacks like a duck, it's a duck.” The concept is that you can pass an instance of a type as a parameter to a function as long as the function can find a method to invoke that it expects:

像 Python、Ruby 和 JavaScript 这样的动态类型语言没有接口。相反，这些开发者使用鸭子类型，它基于“如果它走起来像鸭子，叫起来也像鸭子，那它就是鸭子”这句谚语。其概念是，只要函数能找到它所期望的调用方法，你就可以将某个类型的实例作为参数传递给函数：

```
class Logic:
    def process(self, data):
        # business logic

def program(logic):
    # get data from somewhere
```

```
logic.process(data)

logicToUse = Logic()
program(logicToUse)
```

Duck typing might sound weird at first, but it's been used to build large and successful systems. If you program in a statically typed language, this sounds like utter chaos. Without an explicit type being specified, it's hard to know exactly what functionality should be expected. As new developers move on to a project or the existing developers forget what the code is doing, they have to trace through the code to identify the actual dependencies.

鸭子类型乍听起来可能很奇怪，但它已被用于构建大型且成功的系统。如果你使用静态类型语言编程，这听起来简直是一团糟。如果没有明确指定类型，就很难确切地知道应该实现什么功能。当新开发人员加入项目或现有开发人员忘记代码的功能时，他们必须逐行检查代码才能确定实际的依赖关系。

Java developers use a different pattern. They define an interface, create an implementation of the interface, but refer to the interface only in the client code:

Java 开发人员使用不同的模式。他们定义一个接口，创建该接口的实现，但仅在客户端代码中引用该接口：

```
public interface Logic {
    String process(String data);
}

public class LogicImpl implements Logic {
    public String process(String data) {
        // business logic
    }
}

public class Client {
    private final Logic logic;
    // this type is the interface, not the
    implementation

    public Client(Logic logic) {
        this.logic = logic;
    }

    public void program() {
        // get data from somewhere
        this.logic.process(data);
    }
}

public static void main(String[] args) {
    Logic logic = new LogicImpl();
    Client client = new Client(logic);
}
```



```
    client.program();  
}
```

Dynamic language developers look at the explicit interfaces in Java and don't see how you can possibly refactor your code over time when you have explicit dependencies. Switching to a new implementation from a different provider means rewriting your code to depend on a new interface.

动态语言开发者看到 Java 中显式的接口，会觉得在存在显式依赖关系的情况下，代码很难随着时间推移进行重构。切换到不同提供商的新实现意味着需要重写代码以依赖于新的接口。

Go's developers decided that both groups are right. If your application is going to grow and change over time, you need flexibility to change implementation. However, in order for people to understand what your code is doing (as new people work on the same code over time), you also need to specify what the code depends on. That's where implicit interfaces come in. Go code is a blend of the previous two styles:

Go 的开发者们认为双方的观点都有道理。如果你的应用程序会随着时间推移而发展变化，那么你需要灵活的实现方式。然而，为了让其他人能够理解你的代码（尤其是在不同的人不断修改同一段代码的情况下），你还需要明确指出代码的依赖关系。这就是隐式接口的用武之地。Go 代码融合了前两种风格：

```
type LogicProvider struct {}

func (lp LogicProvider) Process(data string)
string {
    // business logic
}

type Logic interface {
    Process(data string) string
}

type Client struct{
    L Logic
}

func(c Client) Program() {
    // get data from somewhere
    c.L.Process(data)
}

main() {
    c := Client{
        L: LogicProvider{},
    }
    c.Program()
}
```

The Go code provides an interface, but only the caller (`Client`) knows about it; nothing is declared on `LogicProvider` to indicate that it meets the interface. This is sufficient to both allow a new logic provider in the future and provide executable documentation to ensure that any type passed into the client will match the client's need.

Go 代码提供了一个接口，但只有调用者 (`Client`) 知道它的存在；`LogicProvider` 上没有任何声明表明它符合该接口。这既足以允许将来添加新的逻辑提供程序，又能提供可执行的文档，以确保传递给客户端的任何类型都能满足客户端的需求。

TIP

提示

Interfaces specify what callers need. The client code defines the interface to specify what functionality it requires.

接口规定了调用者需要什么。客户端代码定义接口，以指定它需要哪些功能。

This doesn't mean that interfaces can't be shared. You've already seen several interfaces in the standard library that are used for input and output. Having a standard interface is powerful; if you write your code to work with `io.Reader` and `io.Writer`, it will function correctly whether it is writing to a file on local disk or a value in memory.

这并不意味着接口不能共享。您已经在标准库中看到过几个用于输入和输出的接口。拥有标准接口非常强大；如果您编写的代码能够同时处理 `io.Reader` 和 `io.Writer`，那么无论它是写入本地磁盘上的文件还是内存中的值，都能正常运行。

Furthermore, using standard interfaces encourages the *decorator pattern*. It is common in Go to write factory functions that take in an instance of an interface and return another type that implements the same interface. For example, say you have a function with the following definition:

此外，使用标准接口有助于推广装饰器模式。在 Go 语言中，编写工厂函数很常见，这些函数接收一个接口实例作为参数，并返回一个实现了同一接口的实例类型。例如，假设你有一个定义如下的函数：

```
func process(r io.Reader) error
```

You can process data from a file with the following code:

您可以使用以下代码处理文件中的数据：

```
r, err := os.Open(fileName)
if err != nil {
    return err
}
```

```
defer r.Close()
return process(r)
```

The `os.File` instance returned by `os.Open` meets the `io.Reader` interface and can be used in any code that reads in data. If the file is gzip-compressed, you can wrap the `io.Reader` in another `io.Reader`:

由 `os.Open` 返回的 `os.File` 实例符合 `io.Reader` 接口，可以在任何读取数据的代码中使用。如果文件经过 gzip 压缩，您可以将 `io.Reader` 包装在另一个 `io.Reader` 中：

```
r, err := os.Open(fileName)
if err != nil {
    return err
}
defer r.Close()
gz, err := gzip.NewReader(r)
if err != nil {
    return err
}
defer gz.Close()
return process(gz)
```

Now the exact same code that was reading from an uncompressed file is reading from a compressed file instead.

现在，原本读取未压缩文件的代码，改为读取压缩文件。

TIP

提示

If an interface in the standard library describes what your code needs, use it! Commonly used interfaces include `io.Reader`, `io.Writer`, and `io.Closer`.

如果标准库中的接口描述了你的代码需要的功能，那就使用它！常用的接口包括 `io.Reader`、`io.Writer` 和 `io.Closer`。

It's perfectly fine for a type that meets an interface to specify additional methods that aren't part of the interface. One set of client code may not care about those methods, but others do. For example, the `io.File` type also meets the `io.Writer` interface. If your code cares only about reading from a file, use the `io.Reader` interface to refer to the file instance and ignore the other methods.

对于符合接口的类型来说，指定一些接口本身不包含的额外方法完全没问题。一部分客户端代码可能不需要这些方法，但另一部分则需要。例如，`io.File` 类型也符合 `io.Writer` 接口。如果你的代码只关心从文件中读取数据，那么可以使用 `io.Reader` 接口来引用文件实例，并忽略其他方法。

Embedding and Interfaces

嵌入与接口

Embedding is not only for structs. You can also embed an interface in an interface. For example, the `io.ReadCloser` interface is built out of an `io.Reader` and an `io.Closer`:

嵌入不仅适用于结构体。你还可以将接口嵌入到另一个接口中。例如，`io.ReadCloser` 接口是由 `io.Reader` 和 `io.Closer` 构建而成的：

```
type Reader interface {  
    Read(p []byte) (n int, err error)  
}  
  
type Closer interface {  
    Close() error  
}  
  
type ReadCloser interface {  
    Reader  
    Closer  
}
```

NOTE

笔记

Just as you can embed a concrete type in a struct, you can also embed an interface in a struct. You'll see a use for this in [“Using Stubs in Go”](#).

就像你可以在结构体中嵌入具体类型一样，你也可以在结构体中嵌入接口。你将在“在 Go 中使用存根”一节中看到它的用途。

Accept Interfaces, Return Structs

接受接口，返回结构体

You'll often hear experienced Go developers say that your code should “Accept interfaces, return structs.” This phrase was most likely coined by Jack Lindamood in his 2016 blog post [“Preemptive Interface Anti-Pattern in Go”](#). It means that the business logic invoked by your functions should be invoked via interfaces, but the output of your functions should be a concrete type. I've already covered why functions should accept interfaces: they make your code more flexible and explicitly declare the exact functionality being used.

你经常会听到经验丰富的 Go 开发者说，你的代码应该“接受接口，返回结构体”。这句话很可能是 Jack Lindamood 在他 2016 年的博文“Go 中的抢占式接口反模式”中提出的。它的意思是，函数调用的业务逻辑

应该通过接口来实现，但函数的输出应该是具体的类型。我已经解释过为什么函数应该接受接口：它们使你的代码更灵活，并明确声明所使用的具体功能。

The primary reason your functions should return concrete types is they make it easier to gradually update a function's return values in new versions of your code. When a concrete type is returned by a function, new methods and fields can be added without breaking existing code that calls the function, because the new fields and methods are ignored. The same is not true for an interface. Adding a new method to an interface means that all existing implementations of that interface must be updated, or your code breaks. In semantic versioning terms, this is the difference between a minor release that is backward compatible and a major release, which is backward breaking. If you are exposing an API that's consumed by other people (either inside your organization or as part of an open source project), avoiding backward-breaking changes keeps your users happy.

函数应该返回具体类型的主要原因是，这样可以更轻松地在代码的新版本中逐步更新函数的返回值。当函数返回具体类型时，可以添加新的方法和字段而不会破坏调用该函数的现有代码，因为新添加的字段和方法会被忽略。接口则不然。向接口添加新方法意味着必须更新该接口的所有现有实现，否则代码就会出错。从语义化版本控制的角度来看，这就是向后兼容的小版本和向后破坏的大版本之间的区别。如果您公开的 API 会被其他人使用（无论是在组织内部还是作为开源项目的一部分），避免向后破坏的更改可以确保用户满意。

In some rare situations, the least bad option is to have your functions return interfaces. For example, the `database/sql/driver` package in the standard library defines a set of interfaces that define what a database driver must provide. It is the responsibility of the database driver author to provide concrete implementations of these interfaces, so almost all methods on all interfaces defined in `database/sql/driver` return interfaces. Starting in Go 1.8, database drivers are expected to support additional features. The standard library has a compatibility promise, so the existing interfaces can't be updated with new methods, and the existing methods on these interfaces can't be updated to return different types. The solution is to leave the existing interfaces alone, define new interfaces that describe the new functionality, and tell database driver authors that they should implement both the old and new methods on their concrete types.

在某些特殊情况下，让函数返回接口是相对不那么糟糕的选择。例如，标准库中的 `database/sql/driver` 包定义了一组接口，用于定义数据库驱动程序必须提供的功能。数据库驱动程序作者有责任提供这些接口的具体实现，因此 `database/sql/driver` 中定义的几乎所有接口上的方法都返回接口。从 Go 1.8 开始，数据库驱动程序需要支持更多功能。由于标准库具有兼容性承诺，因此现有接口不能更新为包含新方法，也不能更新这些接口上的现有方法以返回不同的类型。解决方案是保持现有接口不变，定义描述新功能的新接口，并告知数据库驱动程序作者，他们需要在具体类型上同时实现新旧方法。

This leads to the question of how to check whether these new methods are present and how to access them if they are. You'll learn how in [“Type Assertions and Type Switches”](#).

这就引出了一个问题：如何检查这些新方法是否存在，以及如果存在，如何访问它们？您将在“类型断言和类型切换”一节中学习相关内容。

Rather than writing a single factory function that returns different instances behind an interface based on input parameters, try to write separate factory functions for each concrete type. In some situations (such as a parser that can return one or more kinds of tokens), it's unavoidable and you have no choice but to return an interface.

与其编写一个基于输入参数返回不同接口实例的单一工厂函数，不如尝试为每种具体类型编写单独的工厂函数。在某些情况下（例如，解析器可以返回一种或多种类型的标记），返回接口是不可避免的，你别无选择。

Errors are an exception to this rule. As you'll see in [Chapter 9](#), Go functions and methods can declare a return parameter of the `error` interface type. In the case of `error`, it's quite likely that different implementations of the interface could be returned, so you need to use an interface to handle all possible options, as interfaces are the only abstract type in Go.

错误是这条规则的例外。正如你将在第 9 章中看到的，Go 函数和方法可以声明一个 `error` 接口类型的返回参数。对于 `error` 来说，很可能返回该接口的不同实现，因此你需要使用接口来处理所有可能的选项，因为接口是 Go 中唯一的抽象类型。

This pattern has one potential drawback. As I discussed in [“Reducing the Garbage Collector’s Workload”](#), reducing heap allocations improves performance by reducing the amount of work for the garbage collector. Returning a struct avoids a heap allocation, which is good. However, when invoking a function with parameters of interface types, a heap allocation occurs for each interface parameter. Figuring out the trade-off between better abstraction and better performance should be done over the life of your program. Write your code so that it is readable and maintainable. If you find that your program is too slow *and* you have profiled it *and* you have determined that the performance problems are due to a heap allocation caused by an interface parameter, then you should rewrite the function to use a concrete type parameter. If multiple implementations of an interface are passed into the function, this will mean creating multiple functions with repeated logic.

这种模式有一个潜在的缺点。正如我在“减少垃圾回收器的负载”一文中讨论的那样，减少堆内存分配可以降低垃圾回收器的工作量，从而提高性能。返回结构体可以避免堆内存分配，这很好。但是，当调用带有接口类型参数的函数时，每个接口参数都会进行一次堆内存分配。在程序的整个生命周期中，应该权衡更好的抽象和更好的性能。

编写代码时，要确保其可读性和可维护性。如果发现程序运行速度过慢，并且已经进行了性能分析，确定性能问题是由接口参数引起的堆内存分配造成的，那么应该重写该函数，使其使用具体的类型参数。如果将接口的多个实现传递给函数，则意味着需要创建多个逻辑重复的函数。

NOTE

笔记

Developers who come from a C++ or Rust background might try using generics as a way to get the compiler to generate specialized functions. As of Go 1.21, this will probably not produce faster code. I'll cover why in [“Idiomatic Go and Generics”](#).

来自 C++ 或 Rust 背景的开发者的可能会尝试使用泛型来让编译器生成专门的函数。但从 Go 1.21 开始，这样做可能不会提高代码速度。我将在“惯用的 Go 语言和泛型”一文中解释原因。

Interfaces and nil

接口和 nil

When discussing pointers in [Chapter 6](#), I also talked about `nil`, the zero value for pointer types. You can also use `nil` to represent the zero value for an interface instance, but it's not as simple as it is for concrete types.

在第六章讨论指针时，我也提到了 `nil`，它是指针类型的零值。你也可以使用 `nil` 来表示接口实例的零值，但这不像具体类型那样简单。

Understanding the relationship between interfaces and `nil` requires understanding a little bit about how interfaces are implemented. In the Go runtime, interfaces are implemented as a struct with two pointer fields, one for the value and one for the type of the value. As long as the type field is non-nil, the interface is non-nil. (Since you cannot have a variable without a type, if the value pointer is non-nil, the type pointer is always non-nil.)

要理解接口和 `nil` 之间的关系，需要对接口的实现方式有所了解。在 Go 运行时中，接口被实现为一个结构体，该结构体包含两个指针字段：一个用于存储值，另一个用于存储值的类型。只要类型字段不为 `nil`，接口就不为 `nil`。（因为变量必须有类型，所以如果值指针不为 `nil`，类型指针也总是不为 `nil`。）

In order for an interface to be considered `nil`, *both* the type and the value must be `nil`. The following code prints out `true` on the first two lines and `false` on the last:

要使接口被视为 `nil`，其类型和值都必须为 `nil`。以下代码在前两行输出 `true`，在最后一行输出 `false`：

```
var pointerCounter *Counter
fmt.Println(pointerCounter == nil) // prints true
```

```
var incremter Incrementer
fmt.Println(incremter == nil) // prints true
incremter = pointerCounter
fmt.Println(incremter == nil) // prints false
```

You can run this code for yourself on [The Go Playground](#) or use the code in the *sample_code/interface_nil* directory in the [Chapter 7 repository](#).

您可以自行在 Go Playground 上运行此代码，或者使用第 7 章存储库中的 *sample_code/interface_nil* 目录中的代码。

What `nil` indicates for a variable with an interface type is whether you can invoke methods on it. As I covered earlier, you can invoke methods on `nil` concrete instances, so it makes sense that you can invoke methods on an interface variable that was assigned a `nil` concrete instance. If an interface variable is `nil`, invoking any methods on it triggers a panic (which I'll discuss in [“panic and recover”](#)). If an interface variable is non-`nil`, you can invoke methods on it. (But note that if the value is `nil` and the methods of the assigned type don't properly handle `nil`, you could still trigger a panic.)

对于具有接口类型的变量，`nil` 表示是否可以调用其方法。正如我之前提到的，可以调用 `nil` 个具体实例的方法，因此，如果接口变量被赋值为 `nil` 个具体实例，那么调用其方法也是合理的。如果接口变量为 `nil`，则调用其任何方法都会触发 panic（我将在“panic and

recover”部分讨论)。如果接口变量不为 nil，则可以调用其方法。

(但请注意，如果值为 nil，并且赋值类型的方法无法正确处理 nil，则仍然可能触发 panic。)

Since an interface instance with a non-nil type is not equal to nil, it is not straightforward to tell whether the value associated with the interface is nil when the type is non-nil. You must use reflection (which I’ll discuss in [“Use Reflection to Check If an Interface’s Value Is nil”](#)) to find out.

由于具有非空类型的接口实例不等于 nil，因此当类型非空时，很难直接判断与该接口关联的值是否为 nil。您必须使用反射（我将在“使用反射检查接口的值是否为 nil”一节中讨论）来确定这一点。

Interfaces Are Comparable

界面具有可比性

In [Chapter 3](#), you learned about comparable types, the ones that can be checked for equality with ==. You might be surprised to learn that interfaces are comparable. Just as an interface is equal to nil only if its type and value fields are both nil, two instances of an interface type are equal only if their types are equal and their values are equal. This suggests a question: what happens if the type isn’t comparable? Let’s use a simple

example to explore this concept. Start with an interface definition and a couple of implementations of that interface:

在第三章中，你学习了可比较类型，也就是可以与 `==` 进行比较的类型。你可能会惊讶地发现，接口也是可比较的。正如一个接口仅当其类型和值字段都等于 `nil` 时才与 `nil` 相等一样，接口类型的两个实例也仅当它们的类型和值都相等时才相等。这就引出了一个问题：如果类型不可比较会怎样？让我们用一个简单的例子来探讨这个概念。首先，定义一个接口，并给出该接口的几个实现：

```
type Doubler interface {
    Double()
}

type DoubleInt int

func (d *DoubleInt) Double() {
    *d = *d * 2
}

type DoubleIntSlice []int

func (d DoubleIntSlice) Double() {
    for i := range d {
        d[i] = d[i] * 2
    }
}
```

```
    }  
}
```

The `Double` method on `DoubleInt` is declared with a pointer receiver because you are modifying the value of the `int`. You can use a value receiver for the `Double` method on `DoubleIntSlice` because, as covered in [“The Difference Between Maps and Slices”](#), you can change the value of an item in a parameter that is a slice type. The `*DoubleInt` type is comparable (all pointer types are), and the `DoubleIntSlice` type is not comparable (slices aren’t comparable).

`DoubleInt` 的 `Double` 方法声明为指针接收器，因为您要修改 `int` 的值。您可以为 `DoubleIntSlice` 的 `Double` 方法使用值接收器，因为如“映射和切片的区别”一节所述，您可以更改切片类型参数中的项的值。`*DoubleInt` 类型是可比较的（所有指针类型都是可比较的），而 `DoubleIntSlice` 类型是不可比较的（切片是不可比较的）。

You also have a function that takes in two parameters of type `Doubler` and prints if they are equal:

你还有一个函数，它接受两个类型为 `Doubler` 的参数，并打印它们是否相等：

```
func DoublerCompare(d1, d2 Doubler) {  
    fmt.Println(d1 == d2)  
}
```

You now define four variables:

现在您需要定义四个变量：

```
var di DoubleInt = 10  
var di2 DoubleInt = 10  
var dis = DoubleIntSlice{1, 2, 3}  
var dis2 = DoubleIntSlice{1, 2, 3}
```

Now, you're going to call this function three times. The first call is as follows:

现在，你要调用这个函数三次。第一次调用如下：

```
DoublerCompare(&di, &di2)
```

This prints out `false`. The types match (both are `*DoubleInt`), but you are comparing pointers, not values, and the pointers point to different instances.

这段代码会输出 `false`。类型匹配（都是 `*DoubleInt`），但你比较的是指针，而不是值，而且指针指向不同的实例。

Next, you compare a `*DoubleInt` with a `DoubleIntSlice`:

接下来，将 `*DoubleInt` 与 `DoubleIntSlice` 进行比较：

```
DoublerCompare(&di, dis)
```

This prints out `false`, because the types do not match.

这将输出 `false`，因为类型不匹配。

Finally, you get to the problematic case:

最后，你遇到了问题所在：

```
DoublerCompare(dis, dis2)
```

This code compiles without issue, but triggers a panic at runtime:

这段代码编译没有问题，但在运行时会引发 panic：

```
panic: runtime error: comparing uncomparable type  
main.DoubleIntSlice
```

The entire program is available in the *sample_code/comparable* directory in the [Chapter 7 repository](#).

整个程序可以在第 7 章存储库的 `sample_code/comparable` 目录中找到。

Also be aware that the key of a map must be comparable, so a map can be defined to have an interface as a key:

还要注意，映射的键必须是可比较的，因此可以将映射定义为以接口作为键：

```
m := map[Doubler]int{}
```

If you add a key-value pair to this map and the key isn't comparable, that will also trigger a panic.

如果向此映射中添加键值对，但键不可比较，也会触发 panic。

Given this behavior, be careful when using `==` or `!=` with interfaces or using an interface as a map key, as it's easy to accidentally generate a panic that will crash your program. Even if all your interface implementations are currently comparable, you don't know what will happen when someone else uses or modifies your code, and there's no way to specify that an interface can be implemented only by comparable types. If you want to be extra safe, you can use the `Comparable` method on `reflect.Value` to inspect an

interface before using it with `==` or `!=`. (You'll learn more about reflection in [“Reflection Lets You Work with Types at Runtime”](#)).

鉴于这种行为，在使用 `==` 或 `!=` 处理接口，或将接口用作映射键时务必小心，因为很容易意外触发 panic，导致程序崩溃。即使所有接口实现当前都是可比较的，你也无法预知其他人使用或修改你的代码时会发生什么，而且也无法指定接口只能由可比较的类型实现。为了更加安全，你可以在使用 `==` 或 `!=` 之前，对 `reflect.Value` 调用 `Comparable` 方法来检查接口。（你可以在“反射让你在运行时处理类型”一文中了解更多关于反射的内容）。

The Empty Interface Says Nothing

空荡荡的界面什么也不说

Sometimes in a statically typed language, you need a way to say that a variable could store a value of any type. Go uses an *empty interface*, `interface{}`, to represent this:

在静态类型语言中，有时需要一种方法来表示一个变量可以存储任何类型的值。Go 使用一个空接口 `interface{}` 来表示这一点：

```
var i interface{}  
i = 20
```

```
i = "hello"
i = struct {
    FirstName string
    LastName  string
} {"Fred", "Fredson"}
```

NOTE

笔记

`interface{}` isn't special case syntax. An empty interface type simply states that the variable can store any value whose type implements zero or more methods. This just happens to match every type in Go.

`interface{}` 并非特殊语法。空接口类型仅仅表示该变量可以存储任何实现了零个或多个方法的类型的值。这恰好匹配 Go 中的所有类型。

To improve readability, Go added `any` as a type alias for `interface{}`. Legacy code (written before `any` was added in Go 1.18) used `interface{}`, but stick with `any` for new code.

为了提高可读性，Go 添加了 `any` 作为 `interface{}` 的类型别名。旧代码（在 Go 1.18 添加 `any` 之前编写）使用 `interface{}`，但新代码请使用 `any`。

Because an empty interface doesn't tell you anything about the value it represents, you can't do a lot with it. One common use of `any` is as a

placeholder for data of uncertain schema that's read from an external source, like a JSON file:

因为空接口不会告诉你它代表什么值，所以无法用它做很多事情。

`any` 的一个常见用途是作为从外部源（例如 JSON 文件）读取的不确定模式数据的占位符：

```
data := map[string]any{}
contents, err :=
os.ReadFile("testdata/sample.json")
if err != nil {
    return err
}
json.Unmarshal(contents, &data)
// the contents are now in the data map
```

NOTE

笔记

User-created data containers that were written before generics were added to Go use an empty interface to store a value. (I'll talk about generics in [Chapter 8](#).) An example in the standard library is [container/list](#). Now that generics are part of Go, please use them for any newly created data containers.

在 Go 语言引入泛型之前，用户创建的数据容器使用空接口来存储值。（我将在第 8 章讨论泛型。）标准库中的一个例子是 `container/list`。现在 Go 语言已经支持泛型，请在所有新创建的数据容器中使用泛型。

If you see a function that takes in an empty interface, it's likely using reflection (which I'll talk about in [Chapter 16](#)) to either populate or read the value. In the preceding example, the second parameter of the `json.Unmarshal` function is declared to be of type `any`.

如果你看到一个函数接收一个空接口作为参数，它很可能使用了反射（我将在第16章讨论）来填充或读取该值。在前面的例子中，`json.Unmarshal` 函数的第二个参数被声明为 `any` 类型。

These situations should be relatively rare. Avoid using `any`. As you've seen, Go is designed as a strongly typed language and attempts to work around this are unidiomatic.

这种情况应该比较少见。请避免使用 `any`。正如你所见，Go 被设计成一种强类型语言，试图绕过这一限制的做法是不规范的。

If you find yourself in a situation where you had to store a value into an empty interface, you might be wondering how to read the value back again. To do that, you need to learn about type assertions and type switches.

如果你遇到需要将值存储到空接口的情况，你可能会想知道如何再次读取该值。要做到这一点，你需要学习类型断言和类型切换。

Type Assertions and Type Switches

类型断言和类型切换

Go provides two ways to see if a variable of an interface type has a specific concrete type or if the concrete type implements another interface. Let's start by looking at type assertions. A *type assertion* names the concrete type that implemented the interface, or names another interface that is also implemented by the concrete type whose value is stored in the interface. You can try it out on [The Go Playground](#) or in the `typeAssert` function in `main.go` in the `sample_code/type_assertions` directory in the [Chapter 7 repository](#):

Go 提供了两种方法来判断接口类型的变量是否具有特定的具体类型，或者该具体类型是否实现了另一个接口。我们先来看看类型断言。类型断言会指出实现了该接口的具体类型，或者指出另一个也由该具体类型实现的接口，该具体类型的值存储在该接口中。您可以在 Go Playground 上尝试，也可以在第七章代码库的 `sample_code/type_assertions` 目录下的 `main.go` 文件中的 `typeAssert` 函数中尝试：

```
type MyInt int

func main() {
```

```
var i any
var mine MyInt = 20
i = mine
i2 := i.(MyInt)
fmt.Println(i2 + 1)
}
```

In the preceding code, the variable `i2` is of type `MyInt`.

前面的代码中，变量 `i2` 的类型为 `MyInt`。

You might wonder what happens if a type assertion is wrong. In that case, your code panics. You can try it out on [The Go Playground](#) or in the `typeAssertPanicWrongType` function in `main.go` in the `sample_code/type_assertions` directory in the [Chapter 7 repository](#):

你可能想知道如果类型断言错误会发生什么。在这种情况下，你的代码会崩溃。你可以在 Go Playground 上尝试，或者在第七章代码库的 `sample_code/type_assertions` 目录下的 `main.go` 文件中的 `typeAssertPanicWrongType` 函数中尝试：

```
i2 := i.(string)
fmt.Println(i2)
```

Running this code produces the following panic:

运行此代码会产生以下 panic:

```
panic: interface conversion: interface {} is  
main.MyInt, not string
```

As you've already seen, Go is very careful about concrete types. Even if two types share an underlying type, a type assertion must match the type of the value stored in the interface. The following code panics. You can try it out on [The Go Playground](#) or in the

`typeAssertPanicTypeNotIdentical` function in *main.go* in the *sample_code/type_assertions* directory in the [Chapter 7 repository](#):

正如你所见，Go 对具体类型非常谨慎。即使两个类型共享一个底层类型，类型断言也必须与接口中存储的值的类型匹配。以下代码会导致 panic。你可以在 Go Playground 上尝试，或者在 Chapter 7 代码库的 *sample_code/type_assertions* 目录下的 *main.go* 文件中的 `typeAssertPanicTypeNotIdentical` 函数中尝试：

```
i2 := i.(int)  
fmt.Println(i2 + 1)
```

Obviously, crashing is not desired behavior. You avoid this by using the comma ok idiom, just as you saw in [“The comma ok Idiom”](#) when detecting whether a zero value was in a map. You can see this in the

`typeAssertCommaOK` function in `main.go` in the `sample_code/type_assertions` directory in the [Chapter 7 repository](#):

显然，崩溃并非我们所期望的行为。你可以使用逗号 `ok` 惯用法来避免这种情况，就像你在“逗号 `ok` 惯用法”一节中看到的那样，它用于检测映射中是否存在零值。你可以在第 7 章代码库的 `sample_code/type_assertions` 目录下的 `main.go` 文件中的 `typeAssertCommaOK` 函数中看到这一点：

```
i2, ok := i.(int)
if !ok {
    return fmt.Errorf("unexpected type for %v", i)
}
fmt.Println(i2 + 1)
```

The boolean `ok` is set to `true` if the type conversion was successful. If it was not, `ok` is set to `false` and the other variable (in this case `i2`) is set to its zero value. You then handle the unexpected condition within an `if` statement. I'll talk more about error handling in [Chapter 9](#).

如果类型转换成功，则布尔值 `ok` 被设置为 `true`。如果转换失败，则 `ok` 被设置为 `false`，另一个变量（在本例中为 `i2`）被设置为其零值。然后，您需要在 `if` 语句中处理意外情况。我将在第 9 章详细讨论错误处理。

NOTE

笔记

A type assertion is very different from a type conversion. Conversions change a value to a new type, while assertions reveal the type of the value stored in the interface. Type conversions can be applied to both concrete types and interfaces. Type assertions can be applied only to interface types. All type assertions are checked at runtime, so they can fail at runtime with a panic if you don't use the comma ok idiom. Most type conversions are checked at compile time, so if they are invalid, your code won't compile. (Type conversions between slices and array pointers can fail at runtime and don't support the comma ok idiom, so be careful when using them!)

类型断言与类型转换截然不同。类型转换将值更改为新类型，而类型断言则揭示存储在接口中的值的类型。类型转换可以应用于具体类型和接口。类型断言只能应用于接口类型。所有类型断言都在运行时进行检查，因此如果您不使用逗号 ok 语法，它们可能会在运行时失败并引发 panic。大多数类型转换在编译时进行检查，因此如果它们无效，您的代码将无法编译。（切片和数组指针之间的类型转换可能会在运行时失败，并且不支持逗号 ok 语法，因此在使用它们时要格外小心！）

Even if you are absolutely certain that your type assertion is valid, use the comma ok idiom version. You don't know how other people (or you in six months) will reuse your code. Sooner or later, your unvalidated type assertions will fail at runtime.

即使你绝对确信你的类型断言有效，也请使用逗号“ok”的惯用写法。你无法预知其他人（或者六个月后的你自己）会如何重用你的代码。迟早，未经验证的类型断言会在运行时失败。

When an interface could be one of multiple possible types, use a *type switch* instead:

当接口可能有多种类型时，请改用类型开关：

```
func doThings(i any) {  
    switch j := i.(type) {  
    case nil:  
        // i is nil, type of j is any  
    case int:  
        // j is of type int  
    case MyInt:  
        // j is of type MyInt  
    case io.Reader:  
        // j is of type io.Reader  
    case string:  
        // j is a string  
    case bool, rune:  
        // i is either a bool or rune, so j is of  
type any  
    default:  
        // no idea what i is, so j is of type any  
    }  
}
```

A type `switch` looks a lot like the `switch` statement that you saw way back in [“switch”](#). Instead of specifying a boolean operation, you specify a variable of an interface type and follow it with `.(type)`. Usually, you assign the variable being checked to another variable that’s valid only within the `switch`.

类型 `switch` 看起来很像你在“switch”语句中看到的 `switch` 语句。不同之处在于，它不是指定布尔运算，而是指定一个接口类型的变量，并在其后加上 `.(type)`。通常，你会将要检查的变量赋值给另一个仅在 `switch` 内有效的变量。

NOTE

笔记

Since the purpose of a type `switch` is to derive a new variable from an existing one, it is idiomatic to assign the variable being switched on to a variable of the same name (`i := i.(type)`), making this one of the few places where shadowing is a good idea. To make the comments more readable, this example doesn't use shadowing.

由于类型 `switch` 的目的是从现有变量派生出一个新变量，因此惯用的写法是将要切换的变量赋值给一个同名变量 (`i := i.(type)`)，这使得它成为少数几个适合使用变量遮蔽的地方之一。为了使注释更易读，本示例未使用变量遮蔽。

The type of the new variable depends on which case matches. You can use `nil` for one case to see if the interface has no associated type. If you list more than one type on a case, the new variable is of type `any`. Just as with a `switch` statement, you can have a `default` case that matches when no specified type does. Otherwise, the new variable has the type of the case that matches.

新变量的类型取决于匹配的 `case`。您可以使用 `nil` 来判断接口是否没有关联的类型。如果在一个 `case` 中列出了多个类型，则新变量的类型为 `any`。与 `switch` 语句类似，您可以设置 `default` case，当没

有指定的类型匹配时，该 case 也会匹配。否则，新变量的类型将与匹配的 case 的类型相同。

While the examples so far have used the `any` interface with type assertions and type switches, you can uncover the concrete type from all interface types.

虽然到目前为止的示例都使用了 `any` 接口以及类型断言和类型切换，但您可以从所有接口类型中发现具体类型。

TIP

提示

If you *don't* know the type of the value stored in an interface, you need to use reflection. I'll talk more about reflection in [Chapter 16](#).

如果你不知道接口中存储的值的类型，就需要使用反射。我将在第16章详细讲解反射。

Use Type Assertions and Type Switches Sparingly

谨慎使用类型断言和类型切换

While being able to extract the concrete implementation from an interface variable might seem handy, you should use these techniques infrequently.

For the most part, treat a parameter or return value as the type that was supplied and not what else it could be. Otherwise, your function's API isn't accurately declaring the types it needs to perform its task. If you needed a different type, it should be specified.

虽然从接口变量中提取具体实现看似方便，但你应该尽量少用这种技巧。大多数情况下，参数或返回值应该被视为其提供的类型，而不是其他可能的类型。否则，你的函数 API 就无法准确声明执行任务所需的类型。如果需要其他类型，则应该明确指定。

That said, type assertions and type switches are useful in some use cases. One common use of a type assertion is to see if the concrete type behind the interface also implements another interface. This allows you to specify optional interfaces. For example, the standard library uses this technique to allow more efficient copies when the `io.Copy` function is called. This function has two parameters of types `io.Writer` and `io.Reader` and calls the `io.copyBuffer` function to do its work. If the `io.Writer` parameter also implements `io.WriterTo`, or the `io.Reader` parameter also implements `io.ReaderFrom`, most of the work in the function can be skipped:

尽管如此，类型断言和类型切换在某些情况下仍然很有用。类型断言的一个常见用途是检查接口背后的具体类型是否也实现了另一个接口。这允许您指定可选接口。例如，标准库使用此技术在调用 `io.Copy` 函数时实现更高效的复制。此函数有两个参数，类型分别

为 `io.Writer` 和 `io.Reader` , 并调用 `io.copyBuffer` 函数来执行其工作。如果 `io.Writer` 参数也实现了 `io.WriterTo` , 或者 `io.Reader` 参数也实现了 `io.ReaderFrom` , 则可以跳过函数中的大部分工作:

```
// copyBuffer is the actual implementation of Copy
and CopyBuffer.
// if buf is nil, one is allocated.
func copyBuffer(dst Writer, src Reader, buf
[]byte) (written int64, err error) {
    // If the reader has a WriteTo method, use it
to do the copy.
    // Avoids an allocation and a copy.
    if wt, ok := src.(WriterTo); ok {
        return wt.WriteTo(dst)
    }
    // Similarly, if the writer has a ReadFrom
method, use it to do the copy.
    if rt, ok := dst.(ReaderFrom); ok {
        return rt.ReadFrom(src)
    }
    // function continues...
}
```

Another place optional interfaces are used is when evolving an API. As was covered in [“Accept Interfaces, Return Structs”](#), the API for the database drivers has changed over time. One of the reasons for this change is the

addition of the context, which is discussed in [Chapter 14](#). Context is a parameter that's passed to functions that provides, among other things, a standard way to manage cancellation. It was added to Go in version 1.7, which means older code doesn't support it. This includes older database drivers.

可选接口的另一个用途是在 API 演进过程中。正如“接受接口，返回结构体”一节中所述，数据库驱动程序的 API 随着时间的推移而发生了变化。造成这种变化的原因之一是上下文的引入，这在第 14 章中进行了讨论。上下文是一个传递给函数的参数，它提供了一种管理取消操作的标准方法。上下文是在 Go 1.7 版本中添加的，这意味着旧版本代码不支持它，包括旧版本的数据库驱动程序。

In Go 1.8, new context-aware analogues of existing interfaces were defined in the `database/sql/driver` package. For example, the `StmtExecContext` interface defines a method called `ExecContext`, which is a context-aware replacement for the `Exec` method in `Stmt`. When an implementation of `Stmt` is passed into standard library database code, it checks whether it also implements `StmtExecContext`. If it does, `ExecContext` is invoked. If not, the Go standard library provides a fallback implementation of the cancellation support provided by newer code:

在 Go 1.8 中，`database/sql/driver` 包中定义了现有接口的新的上下文感知版本。例如，`StmtExecContext` 接口定义了一个名为

`ExecContext` 的方法，它是 `Stmt` 中 `Exec` 方法的上下文感知替代方法。当 `Stmt` 的实现被传递给标准库数据库代码时，它会检查该实现是否也实现了 `StmtExecContext`。如果实现了，则调用 `ExecContext`。否则，Go 标准库会提供一个回退实现，以支持新代码提供的取消操作：

```
func ctxDriverStmtExec(ctx context.Context, si
driver.Stmt,
                        nvdargs
[]driver.NamedValue) (driver.Result, error) {
    if siCtx, is := si.(driver.StmtExecContext);
is {
        return siCtx.ExecContext(ctx, nvdargs)
    }
    // fallback code is here
}
```

This optional interface technique has one drawback. You saw earlier that it is common for implementations of interfaces to use the decorator pattern to wrap other implementations of the same interface to layer behavior. The problem is that if an optional interface is implemented by one of the wrapped implementations, you cannot detect it with a type assertion or type switch. For example, the standard library includes a `bufio` package that provides a buffered reader. You can buffer any other `io.Reader` implementation by passing it to the `bufio.NewReader` function and

using the returned `*bufio.Reader`. If the passed-in `io.Reader` also implemented `io.ReaderFrom`, wrapping it in a buffered reader prevents the optimization.

这种可选接口技术有一个缺点。您之前已经看到，接口的实现通常会使用装饰器模式来包装同一接口的其他实现，从而实现行为分层。问题在于，如果某个可选接口被包装的实现所实现，您就无法通过类型断言或类型切换来检测到它。例如，标准库包含一个 `bufio` 包，它提供了一个缓冲读取器。您可以将任何其他 `io.Reader` 实现传递给 `bufio.NewReader` 函数，并使用返回的 `*bufio.Reader` 来缓冲它。如果传入的 `io.Reader` 也实现了 `io.ReaderFrom`，那么将其包装在缓冲读取器中会阻止优化。

You also see this when handling errors. As mentioned earlier, they implement the `error` interface. Errors can include additional information by wrapping other errors. A type switch or type assertion cannot detect or match wrapped errors. If you want different behaviors to handle different concrete implementations of a returned error, use the `errors.Is` and `errors.As` functions to test for and access the wrapped error.

在处理错误时也会遇到这种情况。如前所述，它们实现了 `error` 接口。错误可以通过包装其他错误来包含额外信息。类型切换或类型断言无法检测或匹配包装后的错误。如果您希望针对返回错误的不同具体实现使用不同的行为，请使用 `errors.Is` 和 `errors.As` 函数来检测和访问包装后的错误。

Type `switch` statements provide the ability to differentiate between multiple implementations of an interface that require different processing. They are most useful when only certain possible valid types can be supplied for an interface. Be sure to include a `default` case in the type `switch` to handle implementations that aren't known at development time. This protects you if you forget to update your type `switch` statements when adding new interface implementations:

类型 `switch` 语句能够区分接口的多个实现，这些实现需要不同的处理方式。当接口只能提供某些有效的类型时，它们最为有用。请务必在类型 `switch` 中包含 `default` 的情况，以处理开发时未知的实现。这样，即使您在添加新的接口实现时忘记更新类型 `switch` 语句，也能避免问题。

```
func walkTree(t *treeNode) (int, error) {
    switch val := t.val.(type) {
    case nil:
        return 0, errors.New("invalid expression")
    case number:
        // we know that t.val is of type number,
        so return the
        // int value
        return int(val), nil
    case operator:
        // we know that t.val is of type operator,
        so
```

```

        // find the values of the left and right
        children, then
        // call the process() method on operator
        to return the
        // result of processing their values.
        left, err := walkTree(t.lchild)
        if err != nil {
            return 0, err
        }
        right, err := walkTree(t.rchild)
        if err != nil {
            return 0, err
        }
        return val.process(left, right), nil
    default:
        // if a new treeVal type is defined, but
        walkTree wasn't updated
        // to process it, this detects it
        return 0, errors.New("unknown node type")
    }
}

```

You can see the complete implementation on [The Go Playground](#) or in the *sample_code/type_switch* directory in the [Chapter 7 repository](#).

您可以在 Go Playground 或第 7 章存储库的 *sample_code/type_switch* 目录中查看完整的实现。

NOTE

笔记

You can further protect yourself from unexpected interface implementations by making the interface unexported and at least one method unexported. If the interface is exported, it can be embedded in a struct in another package, making the struct implement the interface. I'll talk more about packages and exporting identifiers in [Chapter 10](#).

为了进一步避免意外的接口实现，您可以将接口及其至少一个方法设为非导出状态。如果接口已导出，则可以将其嵌入到另一个包中的结构体中，使该结构体实现该接口。我将在第 10 章详细讨论包和导出标识符。

Function Types Are a Bridge to Interfaces

函数类型是连接接口的桥梁

I haven't talked about one last thing with type declarations. Once you understand the concept of declaring a method on a struct, you can start to see how a user-defined type with an underlying type of `int` or `string` can have a method as well. After all, a method provides business logic that interacts with the state of an instance, and integers and strings have state as well.

关于类型声明，我还有最后一点没谈到。一旦你理解了在结构体上声明方法的概念，你就能明白，像 `int` 或 `string` 这样底层类型的用

户自定义类型也可以拥有方法。毕竟，方法提供的是与实例状态交互的业务逻辑，而整数和字符串也都有状态。

Go, however, allows methods on *any* user-defined type, including user-defined function types. This sounds like an academic corner case, but they are actually very useful. They allow functions to implement interfaces. The most common usage is for HTTP handlers. An HTTP handler processes an HTTP server request. It's defined by an interface:

然而，Go 语言允许在任何用户自定义类型（包括用户自定义函数类型）上定义方法。这听起来像是一个学术上的特例，但实际上非常有用。它们允许函数实现接口。最常见的用途是用于 HTTP 处理程序。HTTP 处理程序处理 HTTP 服务器请求。它由一个接口定义：

```
type Handler interface {  
    ServeHTTP(http.ResponseWriter, *http.Request)  
}
```

By using a type conversion to `http.HandlerFunc`, any function that has the signature `func(http.ResponseWriter, *http.Request)` can be used as an `http.Handler`:

通过使用类型转换到 `http.HandlerFunc`，任何具有 `func(http.ResponseWriter, *http.Request)` 签名的函数都可以用作 `http.Handler`：

```
type HandlerFunc func(http.ResponseWriter,  
*http.Request)  
  
func (f HandlerFunc) ServeHTTP(w  
http.ResponseWriter, r *http.Request) {  
    f(w, r)  
}
```

This lets you implement HTTP handlers using functions, methods, or closures using the exact same code path as the one used for other types that meet the `http.Handler` interface.

这样，您就可以使用函数、方法或闭包来实现 HTTP 处理程序，其代码路径与满足 `http.Handler` 接口的其他类型的代码路径完全相同。

Functions in Go are first-class concepts, and as such, they are often passed as parameters into functions. Meanwhile, Go encourages small interfaces, and an interface of only one method could easily replace a parameter of a function type. The question becomes: when should your function or method specify an input parameter of a function type, and when should you use an interface?

在 Go 语言中，函数是一等公民，因此它们通常作为参数传递给函数。同时，Go 鼓励使用小型接口，一个仅包含一个方法的接口就可以

轻松替代函数类型的参数。问题就变成了：何时应该使用函数类型的输入参数，何时又应该使用接口？

If your single function is likely to depend on many other functions or other state that's not specified in its input parameters, use an interface parameter and define a function type to bridge a function to the interface. That's what's done in the `http` package; a `Handler` is likely just the entry point for a chain of calls that needs to be configured. However, if it's a simple function (like the one used in `sort.Slice`), then a parameter of a function type is a good choice.

如果你的单个函数可能依赖于许多其他函数或未在其输入参数中指定的其他状态，请使用接口参数并定义一个函数类型来将函数与接口连接起来。`http` 包就是这么做的；`Handler` 很可能只是一个需要配置的调用链的入口点。但是，如果它是一个简单的函数（例如 `sort.Slice` 中使用的函数），那么使用函数类型的参数是一个不错的选择。

Implicit Interfaces Make Dependency

Injection Easier

隐式接口使依赖注入更容易

In the preface, I compared writing software to building bridges. One of the things that software has in common with physical infrastructure is that any program used for a lengthy period of time by multiple people will need maintenance. While programs don't wear out, developers are often asked to update programs to fix bugs, add features, and run in new environments. Therefore, you should structure your programs in ways that make them easier to modify. Software engineers talk about *decoupling* code, so that changes to different parts of a program have no effect on one another.

在前言中，我将编写软件比作建造桥梁。软件与物理基础设施的共同之处在于，任何被多人长时间使用的程序都需要维护。虽然程序本身不会损耗，但开发人员经常需要更新程序以修复漏洞、添加功能以及使其在新环境中运行。因此，你应该以便于修改的方式构建程序。软件工程师常说代码解耦，这样程序不同部分的更改就不会相互影响。

One of the techniques that has been developed to ease decoupling is called *dependency injection*. Dependency injection is the concept that your code should explicitly specify the functionality it needs to perform its task. It's quite a bit older than you might think; in 1996, Robert Martin wrote an article called ["The Dependency Inversion Principle"](#).

为了简化解耦过程，人们开发了一种名为依赖注入的技术。依赖注入的核心思想是，代码应该明确指定完成任务所需的功能。它的历史比你想象的要悠久得多；早在1996年，罗伯特·马丁就发表了一篇题为《依赖倒置原则》的文章。

One of the surprising benefits of Go's implicit interfaces is that they make dependency injection an excellent way to decouple your code. While developers in other languages often use large, complicated frameworks to inject their dependencies, the truth is that it is easy to implement dependency injection in Go without any additional libraries. Let's work through a simple example to see how to use implicit interfaces to compose applications via dependency injection.

Go 语言的隐式接口带来的一个意想不到的好处是，它使得依赖注入成为一种极佳的代码解耦方式。其他语言的开发者通常需要使用庞大而复杂的框架来注入依赖项，但事实上，在 Go 中实现依赖注入非常简单，无需任何额外的库。让我们通过一个简单的示例来了解如何使用隐式接口，通过依赖注入来构建应用程序。

To understand this concept better and see how to implement dependency injection in Go, you'll build a very simple web application. (I'll talk more about Go's built-in HTTP server support in [“The Server”](#); consider this a preview.) Start by writing a small utility function, a logger:

为了更好地理解这个概念，并了解如何在 Go 中实现依赖注入，你将构建一个非常简单的 Web 应用程序。（我将在“服务器”一节中详细讨论 Go 内置的 HTTP 服务器支持；这里只是个预览。）首先编写一个简单的实用函数，即日志记录器：

```
func LogOutput(message string) {  
    fmt.Println(message)  
}
```

Another thing your app needs is a data store. Let's create a simple one:

你的应用还需要一个数据存储。我们来创建一个简单的数据存储：

```
type SimpleDataStore struct {  
    userData map[string]string  
}  
  
func (sds SimpleDataStore) UserNameForID(userID  
string) (string, bool) {  
    name, ok := sds.userData[userID]  
    return name, ok  
}
```

Also define a factory function to create an instance of a
SimpleDataStore :

同时定义一个工厂函数来创建 `SimpleDataStore` 的实例：

```
func NewSimpleDataStore() SimpleDataStore {
    return SimpleDataStore{
        userData: map[string]string{
            "1": "Fred",
            "2": "Mary",
            "3": "Pat",
        },
    }
}
```

Next, write some business logic that looks up a user and says hello or goodbye. Your business logic needs some data to work with, so it requires a data store. You also want your business logic to log when it is invoked, so it depends on a logger. However, you don't want to force it to depend on `LogOutput` or `SimpleDataStore`, because you might want to use a different logger or data store later. What your business logic needs are interfaces to describe what it depends on:

接下来，编写一些业务逻辑，用于查找用户并向其发送问候或告别信息。您的业务逻辑需要一些数据才能运行，因此需要一个数据存储。您还希望业务逻辑在被调用时记录日志，因此它依赖于一个日志记录器。但是，您不希望强制它依赖于 `LogOutput` 或 `SimpleDataStore`，因为您以后可能需要使用不同的日志记录器或数据存储。您的业务逻辑需要的是描述其依赖项的接口：


```
type DataStore interface {
    UserNameForID(userID string) (string, bool)
}

type Logger interface {
    Log(message string)
}
```

To make your `LogOutput` function meet this interface, you define a function type with a method on it:

为了使您的 `LogOutput` 函数符合此接口，您需要定义一个带有方法的函数类型：

```
type LoggerAdapter func(message string)

func (lg LoggerAdapter) Log(message string) {
    lg(message)
}
```

By a stunning coincidence, `LoggerAdapter` and `SimpleDataStore` happen to meet the interfaces needed by your business logic, but neither type has any idea that it does.

令人惊讶的是，`LoggerAdapter` 和 `SimpleDataStore` 恰好满足您的业务逻辑所需的接口，但这两个类型都不知道这一点。

Now that you have the dependencies defined, let's look at the implementation of your business logic:

现在依赖关系已经定义好了，让我们来看看业务逻辑的实现：

```
type SimpleLogic struct {
    l  Logger
    ds DataStore
}

func (sl SimpleLogic) SayHello(userID string)
(string, error) {
    sl.l.Log("in SayHello for " + userID)
    name, ok := sl.ds.UserNameForID(userID)
    if !ok {
        return "", errors.New("unknown user")
    }
    return "Hello, " + name, nil
}

func (sl SimpleLogic) SayGoodbye(userID string)
(string, error) {
    sl.l.Log("in SayGoodbye for " + userID)
    name, ok := sl.ds.UserNameForID(userID)
    if !ok {
        return "", errors.New("unknown user")
    }
}
```

```
    return "Goodbye, " + name, nil
}
```

You have a `struct` with two fields: one a `Logger`, the other a `DataStore`. Nothing in `SimpleLogic` mentions the concrete types, so there's no dependency on them. There's no problem if you later swap in new implementations from an entirely different provider, because the provider has nothing to do with your interface. This is very different from explicit interfaces in languages like Java. Even though Java uses an interface to decouple the implementation from the interface, the explicit interfaces bind the client and the provider together. This makes replacing a dependency in Java (and other languages with explicit interfaces) far more difficult than it is in Go.

你有一个 `struct` 接口，它有两个字段：一个是 `Logger`，另一个是 `DataStore`。`SimpleLogic` 中没有提及具体的类型，因此不存在对它们的依赖。即使你之后从完全不同的提供程序中替换掉新的实现，也不会有问题，因为提供程序与你的接口无关。这与 Java 等语言中的显式接口截然不同。尽管 Java 使用接口来将实现与接口解耦，但显式接口将客户端和提供程序绑定在一起。这使得在 Java（以及其他使用显式接口的语言）中替换依赖项比在 Go 中困难得多。

When you want a `SimpleLogic` instance, you call a factory function, passing in interfaces and returning a struct:

当您需要 `SimpleLogic` 实例时，您可以调用工厂函数，传入接口并返回一个结构体：

```
func NewSimpleLogic(l Logger, ds DataStore)
SimpleLogic {
    return SimpleLogic{
        l:    l,
        ds: ds,
    }
}
```

NOTE

笔记

The fields in `SimpleLogic` are unexported. This means they can be accessed only by code within the same package as `SimpleLogic`. You can't enforce immutability in Go, but limiting which code can access these fields makes their accidental modification less likely. I'll talk more about exported and unexported identifiers in [Chapter 10](#).

`SimpleLogic` 中的字段未导出。这意味着只有与 `SimpleLogic` 位于同一包中的代码才能访问它们。虽然 Go 无法强制执行不可变性，但限制哪些代码可以访问这些字段可以降低意外修改它们的可能性。我将在第 10 章详细讨论导出和未导出的标识符。

Now you get to your API. You're going to have only a single endpoint, `/hello`, which says hello to the person whose user ID is supplied. (Please do not use query parameters in your real applications for authentication

information; this is just a quick sample.) Your controller needs business logic that says hello, so you define an interface for that:

现在轮到你的 API 了。你只会会有一个端点 `/hello`，它会向提供用户 ID 的用户发送问候信息。（请不要在实际应用中使用查询参数来传递身份验证信息；这只是一个简单的示例。）你的控制器需要业务逻辑来发送问候信息，因此你需要为此定义一个接口：

```
type Logic interface {  
    SayHello(userID string) (string, error)  
}
```

This method is available on your `SimpleLogic` struct, but once again, the concrete type is not aware of the interface. Furthermore, the other method on `SimpleLogic`, `SayGoodbye`, is not in the interface because your controller doesn't care about it. The interface is owned by the client code, so its method set is customized to the needs of the client code:

此方法在您的 `SimpleLogic` 结构体中可用，但具体类型仍然无法识别此接口。此外，`SimpleLogic` 结构体上的另一个方法 `SayGoodbye` 也不在接口中，因为您的控制器并不关心它。此接口由客户端代码拥有，因此其方法集是根据客户端代码的需求进行定制的。

```

type Controller struct {
    l      Logger
    logic Logic
}

func (c Controller) SayHello(w
http.ResponseWriter, r *http.Request) {
    c.l.Log("In SayHello")
    userID := r.URL.Query().Get("user_id")
    message, err := c.logic.SayHello(userID)
    if err != nil {
        w.WriteHeader(http.StatusBadRequest)
        w.Write([]byte(err.Error()))
        return
    }
    w.Write([]byte(message))
}

```

Just as you have factory functions for your other types, let's write one for the `Controller`:

就像你为其他类型编写了工厂函数一样，让我们为 `Controller` 编写一个：

```

func NewController(l Logger, logic Logic)
Controller {
    return Controller{

```

```
        l:      l,  
        logic: logic,  
    }  
}
```

Again, you accept interfaces and return structs.

同样，你接受接口并返回结构体。

Finally, you wire up all your components in your `main` function and start your server:

最后，在 `main` 函数中连接所有组件并启动服务器：

```
func main() {  
    l := LoggerAdapter(LogOutput)  
    ds := NewSimpleDataStore()  
    logic := NewSimpleLogic(l, ds)  
    c := NewController(l, logic)  
    http.HandleFunc("/hello", c.SayHello)  
    http.ListenAndServe(":8080", nil)  
}
```

You can find the complete code for this application in the *sample_code/dependency_injection* directory in the [Chapter 7 repository](#).

您可以在第 7 章存储库的 `sample_code/dependency_injection` 目录中找到此应用程序的完整代码。

The `main` function is the only part of the code that knows what all the concrete types actually are. If you want to swap in different implementations, this is the only place that needs to change. Externalizing the dependencies via dependency injection means that you limit the changes that are needed to evolve your code over time.

`main` 函数是代码中唯一知道所有具体类型的部分。如果要替换不同的实现，只需修改这里即可。通过依赖注入将依赖项外部化，可以减少代码随着时间推移而演进所需的更改。

Dependency injection is also a great pattern for making testing easier. It shouldn't be surprising, since writing unit tests is effectively reusing your code in a different environment, one that constrains the inputs and outputs to validate functionality. For example, you can validate the logging output in a test by injecting a type that captures the log output and meets the `Logger` interface. I'll talk about this more in [Chapter 15](#).

依赖注入也是简化测试的绝佳模式。这并不奇怪，因为编写单元测试实际上是在不同的环境中重用代码，该环境限制了输入和输出，从而验证功能。例如，您可以通过注入一个捕获日志输出并满足 `Logger` 接口的类型，在测试中验证日志输出。我将在第 15 章中详细讨论这一点。

NOTE

笔记

The line `http.HandleFunc("/hello", c.SayHello)` demonstrates two points I talked about earlier.

行 `http.HandleFunc("/hello", c.SayHello)` 说明了我之前提到的两点。

First, you are treating the `SayHello` method as a function.

首先，您将 `SayHello` 方法视为一个函数。

Second, the `http.HandleFunc` function takes in a function and converts it to an `http.HandlerFunc` function type, which declares a method to meet the `http.Handler` interface, which is the type used to represent a request handler in Go. You took a method from one type and converted it into another type with its own method. That's pretty neat.

其次，`http.HandleFunc` 函数接收一个函数并将其转换为 `http.HandlerFunc` 函数类型，该类型声明了一个符合 `http.Handler` 接口的方法，而 `http.Handler` 接口是 Go 中用于表示请求处理程序的类型。你从一个类型中提取了一个方法，并将其转换为另一个类型及其自身的方法。这非常巧妙。

Wire

金属丝

If you feel that writing dependency injection code by hand is too much work, you can use [Wire](#), a dependency injection helper written by Google.

Wire uses code generation to automatically create the concrete type declarations that you wrote yourself in `main`.

如果您觉得手动编写依赖注入代码过于繁琐，可以使用 Google 开发的依赖注入辅助工具 Wire。Wire 使用代码生成功能，自动创建您在 `main` 中编写的具体类型声明。

Go Isn't Particularly Object-Oriented (and That's Great)

Go 语言并非严格意义上的面向对象语言（这很棒）

Now that you've taken a look at the idiomatic use of types in Go, you can see that categorizing Go as a particular style of language is difficult. It clearly isn't a strictly procedural language. At the same time, Go's lack of method overriding, inheritance, or, well, objects means that it is also not a particularly object-oriented language. Go has function types and closures, but it isn't a functional language either. If you attempt to shoehorn Go into one of these categories, the result is nonidiomatic code.

现在你已经了解了 Go 语言中类型的惯用用法，你会发现很难将 Go 归类为某种特定的语言风格。它显然不是一门严格的过程式语言。同

时，Go 缺乏方法重写、继承，或者说对象，这意味着它也不是一门典型的面向对象语言。Go 有函数类型和闭包，但它也不是函数式语言。如果你试图将 Go 硬塞进这些类别中的任何一个，最终只会得到不符合惯用语言习惯的代码。

If you had to label Go's style, the best word to use is *practical*. It borrows concepts from many places with the overriding goal of creating a language that is simple, readable, and maintainable by large teams for many years.

如果要用一个词来形容 Go 的风格，那就是实用。它借鉴了许多领域的理念，其首要目标是创造一种简单、易读且易于大型团队长期维护的语言。

Exercises

练习

In these exercises, you're going to build a program that uses what you've learned about types, methods, and interfaces. Answers are available in the *exercise_solutions* directory in the [Chapter 7 repository](#).

在这些练习中，你将编写一个程序，运用你所学的类型、方法和接口知识。答案位于第 7 章代码库的 `exercise_solutions` 目录中。

1. You have been asked to manage a basketball league and are going to write a program to help you. Define two types. The first one, called `Team`, has a field for the name of the team and a field for the player names. The second type is called `League` and has a field called `Teams` for the teams in the league and a field called `Wins` that maps a team's name to its number of wins.

你被要求管理一个篮球联赛，并需要编写一个程序来协助你。请定义两种数据类型。第一种数据类型名为 `Team`，包含一个用于存储球队名称的字段和一个用于存储球员姓名的字段。第二种数据类型名为 `League`，包含一个名为 `Teams` 的字段，用于存储联赛中的球队信息；以及一个名为 `Wins` 的字段，用于将球队名称与其胜场数关联起来。

2. Add two methods to `League`. The first method is called `MatchResult`. It takes four parameters: the name of the first team, its score in the game, the name of the second team, and its score in the game. This method should update the `Wins` field in `League`. Add a second method to `League` called `Ranking` that returns a slice of the team names in order of wins. Build your data structures and call these methods from the `main` function in your program using some sample data.

向 `League` 添加两个方法。第一个方法名为 `MatchResult`，它接受四个参数：第一支球队的名称、比赛得分、第二支球队的名称以及比赛得分。此方法应更新 `League` 中的 `Wins` 字段。向 `League` 添加第二个方法，名为 `Ranking`，该方法返回按胜负

顺序排列的球队名称切片。使用一些示例数据，构建数据结构，并在程序中的 `main` 函数中调用这些方法。

3. Define an interface called `Ranker` that has a single method called `Ranking` that returns a slice of strings. Write a function called `RankPrinter` with two parameters, the first of type `Ranker` and the second of type `io.Writer`. Use the `io.WriteString` function to write the values returned by `Ranker` to the `io.Writer`, with a newline separating each result. Call this function from `main`.
定义一个名为 `Ranker` 的接口，该接口包含一个名为 `Ranking` 的方法，该方法返回一个字符串切片。编写一个名为 `RankPrinter` 的函数，该函数有两个参数，第一个参数的类型为 `Ranker`，第二个参数的类型为 `io.Writer`。使用 `io.WriteString` 函数将 `Ranker` 返回的值写入 `io.Writer`，每个结果之间用换行符分隔。从 `main` 调用此函数。

Wrapping Up

总结

In this chapter, you learned about types, methods, interfaces, and their best practices. In the next chapter, you'll learn about generics, which improve readability and maintainability by allowing you to reuse logic and custom-written containers with different types.

本章学习了类型、方法、接口及其最佳实践。下一章将学习泛型，它允许你重用不同类型的逻辑和自定义容器，从而提高代码的可读性和可维护性。

- For the computer scientists in the audience, I realize that subtyping is not inheritance. However, most programming languages use inheritance to implement subtyping, so the definitions are often conflated in popular usage.
- 对于在座的计算机科学家，我明白子类型并非继承。然而，大多数编程语言都使用继承来实现子类型，因此在日常使用中，这两个概念经常被混淆。

Chapter 8. Generics

第八章 通用术语

“Don’t repeat yourself” is common software engineering advice. It’s better to reuse a data structure or a function than it is to re-create it, because it’s hard to keep code changes in sync between duplicated code. In a strongly typed language like Go, the type of every function parameter and every struct field must be known at compile time. This strictness enables the compiler to help validate that your code is correct, but sometimes when you’ll want to reuse the logic in a function or the fields in a struct with different types. Go provides this functionality via type parameters, which are colloquially referred to as *generics*. In this chapter you’ll learn why people want generics, what Go’s implementation of generics can do, what generics can’t do, and how to use them properly.

“不要重复自己”是软件工程中一条常见的建议。重用数据结构或函数比重新创建要好得多，因为重复代码之间的更改很难保持同步。在像 Go 这样的强类型语言中，每个函数参数和每个结构体字段的类型都必须在编译时确定。这种严格的类型要求编译器能够帮助验证代码的正确性，但有时你可能希望重用函数中的逻辑或结构体中的字段，而这些字段的类型可能不同。Go 通过类型参数（俗称泛型）提供了这种功能。在本章中，你将学习人们为什么需要泛型、Go 的泛型实现可以做什么、泛型不能做什么，以及如何正确使用泛型。

Generics Reduce Repetitive Code and Increase Type Safety

泛型减少重复代码并提高类型安全性

Go is a statically typed language, which means that the types of variables and parameters are checked when the code is compiled. Built-in types (maps, slices, channels) and functions (such as `len`, `cap`, or `make`) are able to accept and return values of different concrete types, but until Go 1.18, user-defined Go types and functions could not.

Go 是一种静态类型语言，这意味着变量和参数的类型会在代码编译时进行检查。内置类型（map、slice、channel）和函数（例如 `len`、`cap` 或 `make`）能够接受和返回不同具体类型的值，但在 Go 1.18 之前，用户自定义的 Go 类型和函数还不能做到这一点。

If you are accustomed to dynamically typed languages, where types are not evaluated until the code runs, you might not understand what the fuss is about generics, and you might be a bit unclear on what they are. It helps to think of them as “type parameters.” So far in this book, you’ve seen functions that take in parameters whose values are specified when the function is called. In [“Multiple Return Values”](#), the function

`divAndRemainder` has two `int` parameters and returns two `int` values:

如果您习惯了动态类型语言（在动态类型语言中，类型直到代码运行时才会被求值），您可能不理解泛型的魅力所在，也可能不太清楚泛型是什么。您可以将泛型理解为“类型参数”。到目前为止，本书中您看到的函数都接受参数，这些参数的值在函数调用时指定。在“多个返回值”一节中，函数 `divAndRemainder` 有两个 `int` 个参数，并返回两个 `int` 个值：

```
func divAndRemainder(num, denom int) (int, int,
error) {
    if denom == 0 {
        return 0, 0, errors.New("cannot divide by
zero")
    }
    return num / denom, num % denom, nil
}
```

Similarly, you create structs by specifying the type for the fields when the struct is declared. Here, `Node` has a field of type `int` and another field of type `*Node`:

类似地，创建结构体时，需要在声明结构体时指定字段的类型。例如，`Node` 包含一个类型为 `int` 的字段和另一个类型为 `*Node` 的字段：

```
type Node struct {  
    val int  
    next *Node  
}
```

In some situations, however, it's useful to write functions or structs that leave the specific *type* of a parameter or field unspecified until it is used.

但在某些情况下，编写函数或结构体时，将参数或字段的具体类型留空，直到使用时再指定，会很有用。

The case for generic types is easy to understand. In [“Code Your Methods for nil Instances”](#), you looked at a binary tree for `int`s. If you want a binary tree for strings or float64s and want type safety, you have a few options. The first possibility is writing a custom tree for each type, but having that much duplicated code is verbose and error-prone.

泛型类型的优势很容易理解。在“为空实例编写方法”一节中，你了解了用于表示 `int` 的二叉树。如果你想要一个用于表示字符串或 `float64` 类型的二叉树，并且希望保证类型安全，你有几种选择。第一种方法是为每种类型编写一个自定义的树，但这样会编写大量重复代码，既冗长又容易出错。

Without generics, the only way to avoid duplicated code would be to modify your tree implementation so that it uses an interface to specify how

to order values. The interface would look like this:

如果没有泛型，避免代码重复的唯一方法是修改树的实现，使其使用接口来指定值的排序方式。该接口如下所示：

```
type Orderable interface {  
    // Order returns:  
    // a value < 0 when the Orderable is less than  
the supplied value,  
    // a value > 0 when the Orderable is greater  
than the supplied value,  
    // and 0 when the two values are equal.  
    Order(any) int  
}
```

Now that you have `Orderable`, you can modify your `Tree` implementation to support it:

现在您已经有了 `Orderable`，您可以修改您的 `Tree` 实现以支持它：

```
type Tree struct {  
    val      Orderable  
    left, right *Tree  
}  
  
func (t *Tree) Insert(val Orderable) *Tree {
```

```

    if t == nil {
        return &Tree{val: val}
    }

    switch comp := val.Order(t.val); {
    case comp < 0:
        t.left = t.left.Insert(val)
    case comp > 0:
        t.right = t.right.Insert(val)
    }
    return t
}

```

With an `OrderableInt` type, you can then insert `int` values:

使用 `OrderableInt` 类型，您可以插入 `int` 个值：

```

type OrderableInt int

func (oi OrderableInt) Order(val any) int {
    return int(oi - val.(OrderableInt))
}

func main() {
    var it *Tree
    it = it.Insert(OrderableInt(5))
    it = it.Insert(OrderableInt(3))
}

```

```
    // etc...  
}
```

While this code works correctly, it doesn't allow the compiler to validate that the values inserted into your data structure are all the same. If you also had an `OrderableString` type:

虽然这段代码运行正常，但它无法让编译器验证插入到数据结构中的值是否全部相同。如果您还有 `OrderableString` 类型的值：

```
type OrderableString string  
  
func (os OrderableString) Order(val any) int {  
    return strings.Compare(string(os), val.  
        (string))  
}
```

the following code compiles:

以下代码可以编译：

```
var it *Tree  
it = it.Insert(OrderableInt(5))  
it = it.Insert(OrderableString("nope"))
```

The `Order` function uses `any` to represent the value that's passed in. This effectively short-circuits one of Go's primary advantages: checking compile-time type safety. When you compile code that attempts to insert an `OrderableString` into a `Tree` that already contains an `OrderableInt`, the compiler accepts the code. However, the program panics when run:

`Order` 函数使用 `any` 来表示传入的值。这实际上破坏了 Go 的一个主要优势：编译时类型安全检查。当你编译一段尝试将 `OrderableString` 插入到已经包含 `OrderableInt` 的 `Tree` 中的代码时，编译器会接受这段代码。然而，程序运行时会崩溃：

```
panic: interface conversion: interface {} is
main.OrderableInt, not string
```

You can try out this code in the *sample_code/non_generic_tree* directory in the [Chapter 8 repository](#).

您可以在第 8 章存储库的 *sample_code/non_generic_tree* 目录中尝试此代码。

With generics, there's a way to implement a data structure once for multiple types and detect incompatible data at compile time. You'll see how to properly use them in just a bit.

利用泛型，可以一次实现一个数据结构，使其适用于多种类型，并在编译时检测不兼容的数据。稍后您将看到如何正确使用它们。

While data structures without generics are inconvenient, the real limitation is in writing functions. Several implementation decisions in Go's standard library were made because generics weren't originally part of the language. For example, rather than write multiple functions to handle different numeric types, Go implements functions like `math.Max`, `math.Min`, and `math.Mod` using `float64` parameters, which have a range big enough to represent nearly every other numeric type exactly. (The exceptions are an `int`, `int64`, or `uint` with a value greater than $2^{53} - 1$ or less than $-2^{53} - 1$.)

虽然没有泛型的数据结构不太方便，但真正的限制在于编写函数。Go 标准库中的一些实现决策正是基于泛型最初并非该语言的一部分这一事实。例如，Go 并没有编写多个函数来处理不同的数值类型，而是使用 `math.Max`、`math.Min` 和 `math.Mod` 等函数，它们使用 `float64` 个参数，这些参数的范围足够大，几乎可以精确地表示所有其他数值类型。（例外情况是 `int`、`int64` 或 `uint` 的值大于 $2^{53} - 1$ 或小于 $-2^{53} - 1$ 。）

Some other things are impossible without generics. You cannot create a new instance of a variable that's specified by an interface, nor can you specify that two parameters of the same interface type are also of the same concrete type. Without generics, you cannot write a function to process a slice of any

type without resorting to reflection and giving up some performance along with compile-time type safety (this is how `sort.Slice` works). This meant that before generics were introduced to Go, functions that operate on slices (like `map`, `reduce`, and `filter`) would be repeatedly implemented for each type of slice. While simple algorithms are easy enough to copy, many (if not most) software engineers find it grating to duplicate code simply because the compiler isn't smart enough to do it automatically.

如果没有泛型，有些事情是不可能实现的。你无法创建由接口指定的变量的新实例，也无法指定同一接口类型的两个参数也具有相同的具体类型。如果没有泛型，你就无法编写处理任意类型切片的函数，而只能借助反射，牺牲一些性能以及编译时类型安全（这就是 `sort.Slice` 的工作原理）。这意味着在 Go 引入泛型之前，操作切片的函数（例如 `map`、`reduce` 和 `filter`）需要针对每种切片类型重复实现。虽然简单的算法很容易复制，但许多（如果不是大多数）软件工程师都觉得仅仅因为编译器不够智能而重复编写代码令人厌烦。

Introducing Generics in Go

在 Go 语言中引入泛型

Since the first announcement of Go, people have called for generics to be added to the language. Russ Cox, the development lead for Go, wrote a [blog.post](#) in 2009 to explain why generics weren't initially included. Go emphasizes a fast compiler, readable code, and good execution time, and none of the generics implementations that they were aware of would allow them to include all three. After a decade studying the problem, the Go team has a workable approach, which is outlined in the [Type Parameters Proposal](#).

自 Go 语言首次发布以来，人们就一直呼吁为其添加泛型。Go 的开发负责人 Russ Cox 在 2009 年撰写了一篇博文，解释了为何最初没有包含泛型。Go 强调快速编译、可读性强的代码和良好的执行速度，而当时他们所知的任何泛型实现都无法同时满足这三点。经过十年的研究，Go 团队找到了一个可行的方案，该方案概述于类型参数提案中。

You can see how generics work in Go by looking at a stack. If you don't have a computer science background, a stack is a data type where values are added and removed in LIFO order. It's like a pile of dishes waiting to be washed; the ones that were placed first are at the bottom, and you get to

them only by working through the ones that were added later. Let's see how to make a stack using generics:

你可以通过观察栈来了解 Go 语言中泛型的工作原理。如果你没有计算机科学背景，栈是一种数据类型，其中值的添加和移除遵循后进先出（LIFO）的顺序。它就像一堆待洗的碗碟；最先放入的碗碟在最底部，只有依次清洗后面放入的碗碟才能到达它们。让我们看看如何使用泛型创建一个栈：

```
type Stack[T any] struct {
    vals []T
}

func (s *Stack[T]) Push(val T) {
    s.vals = append(s.vals, val)
}

func (s *Stack[T]) Pop() (T, bool) {
    if len(s.vals) == 0 {
        var zero T
        return zero, false
    }
    top := s.vals[len(s.vals)-1]
    s.vals = s.vals[:len(s.vals)-1]
    return top, true
}
```

There are a few things to note. First, you have `[T any]` after the type declaration. Type parameter information is placed within brackets and has two parts. The first part is the type parameter name. You can pick any name for the type parameter, but using capital letters is customary. The second part is the *type constraint*, which uses a Go interface to specify which types are valid. If any type is usable, this is specified with the universe block identifier `any`, which you first saw in [“The Empty Interface Says Nothing”](#). Inside the `Stack` declaration, you declare `vals` to be of type `[]T`.

有几点需要注意。首先，类型声明之后是 `[T any]`。类型参数信息放在方括号内，分为两部分。第一部分是类型参数名称。您可以为类型参数选择任何名称，但通常使用大写字母。第二部分是类型约束，它使用 Go 接口来指定哪些类型有效。如果任何类型可用，则使用 universe 块标识符 `any` 来指定，您在“空接口什么也不说”一文中首次看到了它。在 `Stack` 声明中，您声明 `vals` 的类型为 `[]T`。

Next, look at the method declarations. Just as you used `T` in your `vals` declaration, you do the same here. You also refer to the type in the receiver section with `Stack[T]` instead of `Stack`.

接下来，看一下方法声明。就像你在 `vals` 声明中使用了 `T` 一样，这里也一样。此外，你在接收器部分引用类型时使用了 `Stack[T]` 而不是 `Stack`。

Finally, generics make zero value handling a little interesting. In `Pop`, you can't just return `nil`, because that's not a valid value for a value type, like `int`. The easiest way to get a zero value for a generic is to simply declare a variable with `var` and return it, since by definition, `var` always initializes its variable to the zero value if no other value is assigned.

最后，泛型使得零值处理变得有些复杂。在 `Pop` 中，你不能直接返回 `nil`，因为它不是值类型的有效值，例如 `int`。获取泛型零值的最简单方法是声明一个变量，其值为 `var` 并返回它，因为根据定义，如果没有其他值赋给变量，`var` 总是将其初始化为零值。

Using a generic type is similar to using a nongeneric one:

使用泛型类型与使用非泛型类型类似：

```
func main() {  
    var intStack Stack[int]  
    intStack.Push(10)  
    intStack.Push(20)  
    intStack.Push(30)  
    v, ok := intStack.Pop()  
    fmt.Println(v, ok)  
}
```

The only difference is that when you declare your variable, you include the type that you want to use with your `Stack`—in this case, `int`. If you try

to push a string onto your stack, the compiler will catch it. Adding the line:

唯一的区别在于，声明变量时，你需要包含你想与 `Stack` 一起使用的类型——在本例中是 `int`。如果你尝试将字符串压入栈中，编译器会捕获到错误。添加以下代码行：

```
intStack.Push("nope")
```

produces the compiler error:

产生编译器错误：

```
cannot use "nope" (untyped string constant) as int  
value  
    in argument to intStack.Push
```

You can try out the generic stack on [The Go Playground](#) or in the *sample_code/stack* directory in the [Chapter 8 repository](#).

您可以在 Go Playground 上或第 8 章存储库中的 *sample_code/stack* 目录中尝试通用堆栈。

Add another method to your stack to tell you if the stack contains a value:

在你的栈中添加另一个方法，用来判断栈中是否包含值：

```
func (s Stack[T]) Contains(val T) bool {  
    for _, v := range s.vals {  
        if v == val {  
            return true  
        }  
    }  
    return false  
}
```

Unfortunately, this does not compile. It gives an error:

很遗憾，这段代码无法编译，并出现错误：

```
invalid operation: v == val (type parameter T is  
not comparable with ==)
```

Just as `interface{}` doesn't say anything, neither does `any`. You can only store values of `any` type and retrieve them. To use `==`, you need a different type. Since nearly all Go types can be compared with `==` and `!=`, a new built-in interface called `comparable` is defined in the universe block. If you change the definition of `Stack` to use `comparable`:

正如 `interface{}` 不表示任何内容一样，`any` 也不表示任何内容。你只能存储和检索 `any` 类型的值。要使用 `==`，你需要一个不同的类

型。由于几乎所有 Go 类型都可以与 `==` 和 `!=` 进行比较，因此在 `universe` 代码块中定义了一个名为 `comparable` 的新内置接口。如果你将 `Stack` 的定义更改为使用 `comparable`：

```
type Stack[T comparable] struct {  
    vals []T  
}
```

you can then use your new method:

然后你就可以使用你的新方法了：

```
func main() {  
    var s Stack[int]  
    s.Push(10)  
    s.Push(20)  
    s.Push(30)  
    fmt.Println(s.Contains(10))  
    fmt.Println(s.Contains(5))  
}
```

This prints out the following:

输出结果如下：

```
true  
false
```

You can try out this updated stack on [The Go Playground](#) or in the `sample_code/comparable_stack` directory in the [Chapter 8 repository](#).

您可以在 Go Playground 上或第 8 章存储库中的 `sample_code/comparable_stack` 目录中尝试此更新后的堆栈。

Later, you'll see how to make a generic binary tree. First, I'll cover some additional concepts: *generic functions*, how generics work with interfaces, and *type terms*.

稍后，您将学习如何创建通用二叉树。首先，我将介绍一些附加概念：泛型函数、泛型如何与接口协同工作以及类型术语。

Generic Functions Abstract Algorithms

通用函数抽象算法

As I have hinted, you can also write generic functions. Earlier I mentioned that not having generics made it difficult to write `map`, `reduce`, and `filter` implementations that work for all types. Generics make it easy. Here are implementations from the type parameters proposal:

正如我之前提到的，你也可以编写泛型函数。我之前提到过，如果没有泛型，就很难编写适用于所有类型的 map、reduce 和 filter 实现。泛型让这一切变得简单。以下是类型参数提案中的一些实现：

```
// Map turns a []T1 to a []T2 using a mapping
function.
// This function has two type parameters, T1 and
T2.
// This works with slices of any type.
func Map[T1, T2 any](s []T1, f func(T1) T2) []T2 {
    r := make([]T2, len(s))
    for i, v := range s {
        r[i] = f(v)
    }
    return r
}

// Reduce reduces a []T1 to a single value using a
reduction function.
func Reduce[T1, T2 any](s []T1, initializer T2, f
func(T2, T1) T2) T2 {
    r := initializer
    for _, v := range s {
        r = f(r, v)
    }
    return r
}
```

```

// Filter filters values from a slice using a
// filter function.
// It returns a new slice with only the elements
// of s
// for which f returned true.
func Filter[T any](s []T, f func(T) bool) []T {
    var r []T
    for _, v := range s {
        if f(v) {
            r = append(r, v)
        }
    }
    return r
}

```

Functions place their type parameters after the function name and before the variable parameters. `Map` and `Reduce` have two type parameters, both of `any` type, while `Filter` has one. When you run the code:

函数将其类型参数放在函数名之后、变量参数之前。`Map` 和 `Reduce` 各有两个类型参数，均为 `any` 类型，而 `Filter` 只有一个类型参数。运行代码时：

```

words := []string{"One", "Potato", "Two",
    "Potato"}
filtered := Filter(words, func(s string) bool {

```

```
        return s != "Potato"
    })
    fmt.Println(filtered)
    lengths := Map(filtered, func(s string) int {
        return len(s)
    })
    fmt.Println(lengths)
    sum := Reduce(lengths, 0, func(acc int, val int)
    int {
        return acc + val
    })
    fmt.Println(sum)
```

you get the output:

您将得到以下输出：

```
[One Two]
[3 3]
6
```

Try it for yourself on [The Go Playground](#) or in the `sample_code/map_filter_reduce` directory in the [Chapter 8 repository](#).

您可以在 Go Playground 或第 8 章存储库中的 `sample_code/map_filter_reduce` 目录中亲自尝试一下。

Generics and Interfaces

泛型和接口

You can use any interface as a type constraint, not just `any` and `comparable`. For example, say you wanted to make a type that holds any two values of the same type, as long as the type implements `fmt.Stringer`. Generics make it possible to enforce this at compile time:

你可以使用任何接口作为类型约束，而不仅仅是 `any` 和 `comparable`。例如，假设你想创建一个类型，该类型可以保存任意两个相同类型的值，只要该类型实现了 `fmt.Stringer` 接口。泛型允许在编译时强制执行此约束：

```
type Pair[T fmt.Stringer] struct {  
    Val1 T  
    Val2 T  
}
```

You can also create interfaces that have type parameters. For example, here's an interface with a method that compares against a value of the specified type and returns a `float64`. It also embeds `fmt.Stringer`:

你还可以创建带有类型参数的接口。例如，以下接口包含一个方法，该方法将值与指定类型的值进行比较并返回 `float64`。它还嵌入了 `fmt.Stringer`：

```
type Differ[T any] interface {  
    fmt.Stringer  
    Diff(T) float64  
}
```

You'll use these two types to create a comparison function. The function takes in two `Pair` instances that have fields of type `Differ`, and returns the `Pair` with the closer values:

您将使用这两种类型来创建一个比较函数。该函数接收两个 `Pair` 实例，它们的字段类型为 `Differ`，并返回值更接近的 `Pair`：

```
func FindCloser[T Differ[T]](pair1, pair2 Pair[T])  
Pair[T] {  
    d1 := pair1.Val1.Diff(pair1.Val2)  
    d2 := pair2.Val1.Diff(pair2.Val2)  
    if d1 < d2 {  
        return pair1  
    }  
    return pair2  
}
```

Note that `FindCloser` takes in `Pair` instances that have fields that meet the `Differ` interface. `Pair` requires that its fields are both of the same type and that the type meets the `fmt.Stringer` interface; this function is more selective. If the fields in a `Pair` instance don't meet `Differ`, the compiler will prevent you from using that `Pair` instance with `FindCloser`.

请注意，`FindCloser` 接受 `Pair` 个实例，这些实例的字段必须符合 `Differ` 接口。`Pair` 要求其所有字段类型相同，且该类型符合 `fmt.Stringer` 接口；此函数的筛选条件更为严格。如果 `Pair` 实例中的字段不符合 `Differ` 接口，编译器将阻止您将该 `Pair` 实例与 `FindCloser` 一起使用。

Now define a couple of types that meet the `Differ` interface:

现在定义几个符合 `Differ` 接口的类型：

```
type Point2D struct {
    X, Y int
}

func (p2 Point2D) String() string {
    return fmt.Sprintf("%d,%d", p2.X, p2.Y)
}

func (p2 Point2D) Diff(from Point2D) float64 {
```

```
    x := p2.X - from.X
    y := p2.Y - from.Y
    return math.Sqrt(float64(x*x) + float64(y*y))
}

type Point3D struct {
    X, Y, Z int
}

func (p3 Point3D) String() string {
    return fmt.Sprintf("{%d,%d,%d}", p3.X, p3.Y,
p3.Z)
}

func (p3 Point3D) Diff(from Point3D) float64 {
    x := p3.X - from.X
    y := p3.Y - from.Y
    z := p3.Z - from.Z
    return math.Sqrt(float64(x*x) + float64(y*y) +
float64(z*z))
}
```

And here's what it looks like to use this code:

以下是使用这段代码的效果：

```
func main() {  
    pair2Da := Pair[Point2D]{Point2D{1, 1},  
Point2D{5, 5}}  
    pair2Db := Pair[Point2D]{Point2D{10, 10},  
Point2D{15, 5}}  
    closer := FindCloser(pair2Da, pair2Db)  
    fmt.Println(closer)  
  
    pair3Da := Pair[Point3D]{Point3D{1, 1, 10},  
Point3D{5, 5, 0}}  
    pair3Db := Pair[Point3D]{Point3D{10, 10, 10},  
Point3D{11, 5, 0}}  
    closer2 := FindCloser(pair3Da, pair3Db)  
    fmt.Println(closer2)  
}
```

Run it for yourself on [The Go Playground](#) or in the *sample_code/generic_interface* directory in the [Chapter 8 repository](#).

您可以在 Go Playground 或第 8 章存储库中的 *sample_code/generic_interface* 目录中自行运行它。

Use Type Terms to Specify Operators

使用类型术语来指定运算符。

One more thing needs to be represented with generics: operators. The `divAndRemainder` function works fine with `int`, but using it with other integer types requires type casting, and `uint` allows you to represent values that are too big for an `int`. If you want to write a generic version of `divAndRemainder`, you need a way to specify that you can use `/` and `%`. Go generics do that with a *type element*, which is composed of one or more *type terms* within an interface:

还有一点需要用泛型来表示：运算符。`divAndRemainder` 函数可以与`int`一起使用，但将其用于其他整数类型时需要进行类型转换；而`uint`则允许你表示超出`int`范围的值。如果你想编写`divAndRemainder`的泛型版本，你需要一种方法来指定可以使用`/`和`%`。Go 泛型通过类型元素来实现这一点，该元素由接口中的一个或多个类型项组成：

```
type Integer interface {  
    int | int8 | int16 | int32 | int64 |  
    uint | uint8 | uint16 | uint32 | uint64 |  
}
```

```
uintptr  
}
```

In “[Embedding and Interfaces](#)”, you learned about embedding interfaces to indicate that the method set of the containing interface includes the methods of the embedded interface. Type elements specify which types can be assigned to a type parameter and which operators are supported. They list concrete types separated by `|`. The allowed operators are the ones that are valid for *all* of the listed types. The modulus (`%`) operator is valid only for integers, so we list all integer types. (You can leave off `byte` and `rune` because they are type aliases for `uint8` and `int32`, respectively.)

在“嵌入与接口”一节中，您学习了如何嵌入接口，以表明包含接口的方法集包含被嵌入接口的方法。类型元素指定哪些类型可以分配给类型参数以及支持哪些运算符。它们列出具体类型，并以 `|` 分隔。允许的运算符是对所有列出的类型都有效的运算符。取模运算符 (`%`) 仅对整数有效，因此我们列出所有整数类型。（您可以省略 `byte` 和 `rune`，因为它们分别是 `uint8` 和 `int32` 的类型别名。）

Be aware that interfaces with type elements are valid only as type constraints. It is a compile-time error to use them as the type for a variable, field, return value, or parameter.

请注意，带有类型元素的接口仅可用作类型约束。如果将它们用作变量、字段、返回值或参数的类型，则会产生编译时错误。

Now you can write your generic version of `divAndRemainder` and use it with the built-in `uint` type (or any of the other types listed in `Integer`):

现在您可以编写 `divAndRemainder` 的通用版本，并将其与内置的 `uint` 类型（或 `Integer` 中列出的任何其他类型）一起使用：

```
func divAndRemainder[T Integer](num, denom T) (T,
T, error) {
    if denom == 0 {
        return 0, 0, errors.New("cannot divide by
zero")
    }
    return num / denom, num % denom, nil
}

func main() {
    var a uint = 18_446_744_073_709_551_615
    var b uint = 9_223_372_036_854_775_808
    fmt.Println(divAndRemainder(a, b))
}
```

By default, type terms match exactly. If you try to use `divAndRemainder` with a user-defined type whose underlying type is one of the types listed in `Integer`, you'll get an error. This code:

默认情况下，类型术语必须完全匹配。如果您尝试将 `divAndRemainder` 与用户自定义类型一起使用，而该自定义类型的

底层类型是 `Integer` 中列出的类型之一，则会收到错误。此代码：

```
type MyInt int
var myA MyInt = 10
var myB MyInt = 20
fmt.Println(divAndRemainder(myA, myB))
```

produces the following error:

产生以下错误：

```
MyInt does not satisfy Integer (possibly missing ~
for int in Integer)
```

The error text gives a hint for how to solve this problem. If you want a type term to be valid for any type that has the type term as its underlying type, put a `~` before the type term. This changes the definition of `Integer` as follows:

错误信息提示了如何解决此问题。如果您希望某个类型术语对任何以该类型术语为基础类型的类型都有效，请在类型术语前添加 `~`。这将按如下方式更改 `Integer` 的定义：

```
type Integer interface {
    ~int | ~int8 | ~int16 | ~int32 | ~int64 |
    ~uint | ~uint8 | ~uint16 | ~uint32 |
```

```
~uint64 | ~uintptr  
}
```

You can look at the generic version of the `divAndRemainder` function on [The Go Playground](#) or in the `sample_code/type_terms` directory in the [Chapter 8 repository](#).

您可以在 Go Playground 上查看 `divAndRemainder` 函数的通用版本，或者在第 8 章存储库的 `sample_code/type_terms` 目录中查看。

The addition of type terms allows you to define a type that lets you write generic comparison functions:

添加类型项后，您可以定义一种类型，从而编写通用的比较函数：

```
type Ordered interface {  
    ~int | ~int8 | ~int16 | ~int32 | ~int64 |  
        ~uint | ~uint8 | ~uint16 | ~uint32 |  
~uint64 | ~uintptr |  
        ~float32 | ~float64 |  
        ~string  
}
```

The `Ordered` interface lists all types that support the `==`, `!=`, `<`, `>`, `<=`, and `>=` operators. Having a way to specify that a variable represents an orderable type is so useful that Go 1.21 added the [`cmp` package](#), which

defines this `Ordered` interface. The package also defines two comparison functions. The `Compare` function returns either `-1`, `0`, or `1`, depending on whether its first parameter is less than, equal to, or greater than its second parameter, and the `Less` function returns `true` if its first parameter is less than its second parameter.

`Ordered` 接口列出了所有支持 `==`、`!=`、`<`、`>`、`<=` 和 `>=` 运算符的类型。能够指定变量表示可排序类型非常有用，因此 Go 1.21 添加了 `cmp` 包，该包定义了 `Ordered` 接口。该包还定义了两个比较函数。`Compare` 函数根据其第一个参数小于、等于或大于其第二个参数的值返回 `-1`、`0` 或 `1`，而 `Less` 函数在第一个参数小于其第二个参数时返回 `true`。

It is legal to have both type elements and method elements in an interface used for a type parameter. For example, you could specify that a type must have an underlying type of `int` and a `String() string` method:

在用于类型参数的接口中，同时包含类型元素和方法元素是合法的。例如，您可以指定一个类型必须具有 `int` 的底层类型和一个 `String() string` 方法：

```
type PrintableInt interface {
    ~int
    String() string
}
```

Be aware that Go will let you declare a type parameter interface that is impossible to actually instantiate. If you had used `int` instead of `~int` in `PrintableInt`, there would be no valid type that meets it, since `int` has no methods. This might seem bad, but the compiler still comes to your rescue. If you declare a type or function with an impossible type parameter, any attempt to use it causes a compiler error. Assume you declare these types:

请注意，Go 允许您声明一个实际上无法实例化的类型参数接口。例如，如果您在 `PrintableInt` 中使用 `int` 而不是 `~int`，则不存在符合条件的有效类型，因为 `int` 没有方法。这看起来很糟糕，但编译器会帮您解决问题。如果您声明了一个带有无法实例化的类型参数的类型或函数，任何尝试使用它的操作都会导致编译器错误。假设您声明了以下类型：

```
type ImpossiblePrintableInt interface {
    int
    String() string
}

type ImpossibleStruct[T ImpossiblePrintableInt]
struct {
    val T
}

type MyInt int
```

```
func (mi MyInt) String() string {  
    return fmt.Sprintf(mi)  
}
```

Even though you cannot instantiate `ImpossibleStruct`, the compiler has no problem with any of these declarations. However, once you try using `ImpossibleStruct`, the compiler complains. This code:

即使你无法实例化 `ImpossibleStruct`，编译器也不会对这些声明提出异议。然而，一旦你尝试使用 `ImpossibleStruct`，编译器就会报错。以下代码：

```
s := ImpossibleStruct[int]{10}  
s2 := ImpossibleStruct[MyInt]{10}
```

produces the following compile-time errors:

会产生以下编译时错误：

```
int does not implement ImpossiblePrintableInt  
(missing String method)  
MyInt does not implement ImpossiblePrintableInt  
(possibly missing ~ for  
int in constraint ImpossiblePrintableInt)
```


Try this on [The Go Playground](#) or in the *sample_code/impossible* directory in the [Chapter 8 repository](#).

请在 Go Playground 或第 8 章存储库中的 *sample_code/impossible* 目录中尝试此操作。

In addition to built-in primitive types, type terms can also be slices, maps, arrays, channels, structs, or even functions. They are most useful when you want to ensure that a type parameter has a specific underlying type and one or more methods.

除了内置的基本类型之外，类型项还可以是切片、映射、数组、通道、结构体，甚至是函数。当需要确保类型参数具有特定的底层类型以及一个或多个方法时，它们最为有用。

Type Inference and Generics

类型推断和泛型

Just as Go supports type inference when using the `:=` operator, it also supports type inference to simplify calls to generic functions. You can see this in the calls to `Map`, `Filter`, and `Reduce` earlier. In some situations, type inference isn't possible (for example, when a type parameter is used only as a return value). When that happens, all type arguments must be

specified. Here's a slightly silly bit of code that demonstrates a situation where type inference doesn't work:

就像 Go 语言在使用 `:=` 运算符时支持类型推断一样，它也支持类型推断来简化对泛型函数的调用。您可以在前面对 `Map`、`Filter` 和 `Reduce` 的调用中看到这一点。在某些情况下，类型推断是不可能的（例如，当类型参数仅用作返回值时）。发生这种情况时，所有类型参数都必须指定。以下是一段略显简单的代码，演示了类型推断不起作用的情况：

```
type Integer interface {
    int | int8 | int16 | int32 | int64 | uint |
    uint8 | uint16 | uint32 | uint64
}

func Convert[T1, T2 Integer](in T1) T2 {
    return T2(in)
}

func main() {
    var a int = 10
    b := Convert[int, int64](a) // can't infer the
    return type
    fmt.Println(b)
}
```

Try it out on [The Go Playground](#) or in the *sample_code/type_inference* directory in the [Chapter 8 repository](#).

您可以在 Go Playground 或第 8 章代码库的 *sample_code/type_inference* 目录中尝试一下。

Type Elements Limit Constants

类型元素限制常量

Type elements also specify which constants can be assigned to variables of the generic type. Like operators, the constants need to be valid for all the type terms in the type element. There are no constants that can be assigned to every listed type in `Ordered`, so you cannot assign a constant to a variable of that generic type. If you use the `Integer` interface, the following code will not compile, because you cannot assign the value 1,000 to an 8-bit integer:

类型元素还指定哪些常量可以赋值给泛型类型的变量。与运算符类似，常量必须对类型元素中的所有类型项都有效。`Ordered` 中列出的所有类型都没有可以赋值的常量，因此您不能将常量赋值给该泛型类型的变量。如果您使用 `Integer` 接口，以下代码将无法编译，因为您不能将值 1000 赋值给一个 8 位整数：

```
// INVALID!  
func PlusOneThousand[T Integer](in T) T {  
    return in + 1_000  
}
```

However, this is valid:

然而，这是有效的：

```
// VALID  
func PlusOneHundred[T Integer](in T) T {  
    return in + 100  
}
```

Combining Generic Functions with Generic Data Structures

将通用函数与通用数据结构相结合

Let's return to the binary tree example and see how to combine everything you've learned to make a single tree that works for any concrete type.

让我们回到二叉树的例子，看看如何将你所学到的所有内容结合起来，创建一个适用于任何具体类型的单一树。

The secret is to realize that what your tree needs is a single generic function that compares two values and tells you their order:

秘诀在于认识到你的树需要的只是一个通用的函数，该函数可以比较两个值并告诉你它们的顺序：

```
type OrderableFunc [T any] func(t1, t2 T) int
```

Now that you have `OrderableFunc`, you can modify your tree implementation slightly. First, you're going to split it into two types, `Tree` and `Node`:

现在你有了 `OrderableFunc`，你可以稍微修改一下你的树结构实现。首先，你要把它分成两种类型，`Tree` 和 `Node`：

```
type Tree[T any] struct {
    f    OrderableFunc[T]
    root *Node[T]
}

type Node[T any] struct {
    val T
```

```
    left, right *Node[T]
}
```

You construct a new `Tree` with a constructor function:

您可以使用构造函数创建一个新的 `Tree` :

```
func NewTree[T any](f OrderableFunc[T]) *Tree[T] {
    return &Tree[T]{
        f: f,
    }
}
```

`Tree` 's methods are very simple, because they just call `Node` to do all the real work:

`Tree` 的方法非常简单，因为它们只是调用 `Node` 来完成所有实际工作：

```
func (t *Tree[T]) Add(v T) {
    t.root = t.root.Add(t.f, v)
}

func (t *Tree[T]) Contains(v T) bool {
```

```
    return t.root.Contains(t.f, v)
}
```

The `Add` and `Contains` methods on `Node` are very similar to what you've seen before. The only difference is that the function you are using to order your elements is passed in:

`Add` 和 `Contains` 方法在 `Node` 上与您之前看到的非常相似。唯一的区别在于，您用于对元素进行排序的函数是传入的：

```
func (n *Node[T]) Add(f OrderableFunc[T], v T)
*Node[T] {
    if n == nil {
        return &Node[T]{val: v}
    }
    switch r := f(v, n.val); {
    case r <= -1:
        n.left = n.left.Add(f, v)
    case r >= 1:
        n.right = n.right.Add(f, v)
    }
    return n
}

func (n *Node[T]) Contains(f OrderableFunc[T], v
T) bool {
    if n == nil {
```

```

        return false
    }
    switch r := f(v, n.val); {
    case r <= -1:
        return n.left.Contains(f, v)
    case r >= 1:
        return n.right.Contains(f, v)
    }
    return true
}

```

Now you need a function that matches the `OrderedFunc` definition. Luckily, you've already seen one: `Compare` in the `cmp` package. When you use it with your `Tree`, it looks like this:

现在你需要一个与 `OrderedFunc` 定义匹配的函数。幸运的是，你已经见过一个：`cmp` 包中的 `Compare`。当你将它与你的 `Tree` 一起使用时，它看起来像这样：

```

t1 := NewTree(cmp.Compare[int])
t1.Add(10)
t1.Add(30)
t1.Add(15)
fmt.Println(t1.Contains(15))
fmt.Println(t1.Contains(40))

```


For structs, you have two options. You can write a function:

对于结构体，你有两种选择。你可以编写一个函数：

```
type Person struct {  
    Name string  
    Age int  
}  
  
func OrderPeople(p1, p2 Person) int {  
    out := cmp.Compare(p1.Name, p2.Name)  
    if out == 0 {  
        out = cmp.Compare(p1.Age, p2.Age)  
    }  
    return out  
}
```

Then you can pass that function in when you create your tree:

然后，你可以在创建树的时候传入这个函数：

```
t2 := NewTree(OrderPeople)  
t2.Add(Person{"Bob", 30})  
t2.Add(Person{"Maria", 35})  
t2.Add(Person{"Bob", 50})
```

```
fmt.Println(t2.Contains(Person{"Bob", 30}))
fmt.Println(t2.Contains(Person{"Fred", 25}))
```

Instead of using a function, you can also supply a method to `NewTree`. As I talked about in [“Methods Are Functions Too”](#), you can use a method expression to treat a method like a function. Let’s do that here. First, write the method:

除了使用函数之外，您还可以为 `NewTree` 提供一个方法。正如我在“方法也是函数”一文中提到的，您可以使用方法表达式将方法视为函数。我们在这里演示一下。首先，编写该方法：

```
func (p Person) Order(other Person) int {
    out := cmp.Compare(p.Name, other.Name)
    if out == 0 {
        out = cmp.Compare(p.Age, other.Age)
    }
    return out
}
```

And then use it:

然后使用它：

```
t3 := NewTree(Person.Order)
t3.Add(Person{"Bob", 30})
t3.Add(Person{"Maria", 35})
t3.Add(Person{"Bob", 50})
fmt.Println(t3.Contains(Person{"Bob", 30}))
fmt.Println(t3.Contains(Person{"Fred", 25}))
```

You can find the code for this tree on [The Go Playground](#) or in the `sample_code/generic_tree` directory in the [Chapter 8 repository](#).

您可以在 Go Playground 或第 8 章存储库的 `sample_code/generic_tree` 目录中找到此树的代码。

More on comparable

更多相关比较信息

As you saw in [“Interfaces Are Comparable”](#), interfaces are one of the comparable types in Go. This means that you need to be careful when using `==` and `!=` with variables of the interface type. If the underlying type of the interface is not comparable, your code panics at runtime.

正如你在“接口是可比较的”一节中看到的，接口是 Go 语言中可比较的类型之一。这意味着，当使用 `==` 和 `!=` 来处理接口类型的变量

时，你需要格外小心。如果接口的底层类型不可比较，你的代码会在运行时崩溃。

This pothole still exists when using the `comparable` interface with generics. Say you've defined an interface and a couple of implementations:

在使用泛型时，`comparable` 接口仍然存在这个问题。假设你定义了一个接口和几个实现：

```
type Thinger interface {
    Thing()
}

type ThingerInt int

func (t ThingerInt) Thing() {
    fmt.Println("ThingInt:", t)
}

type ThingerSlice []int

func (t ThingerSlice) Thing() {
    fmt.Println("ThingSlice:", t)
}
```

You also define a generic function that accepts only values that are `comparable` :

你还定义了一个通用函数，该函数只接受 `comparable` 类型的值：

```
func Comparer[T comparable](t1, t2 T) {  
    if t1 == t2 {  
        fmt.Println("equal!")  
    }  
}
```

It's legal to call this function with variables of type `int` or `ThingerInt` :

可以使用 `int` 或 `ThingerInt` 类型的变量来调用此函数：

```
var a int = 10  
var b int = 10  
Comparer(a, b) // prints true  
  
var a2 ThingerInt = 20  
var b2 ThingerInt = 20  
Comparer(a2, b2) // prints true
```

The compiler won't allow you to call this function with variables of type `ThingerSlice` (or `[]int`):

编译器不允许您使用 `ThingerSlice`（或 `[]int`）类型的变量调用此函数：

```
var a3 ThingerSlice = []int{1, 2, 3}
var b3 ThingerSlice = []int{1, 2, 3}
Comparer(a3, b3) // compile fails: "ThingerSlice
does not satisfy comparable"
```

However, it's perfectly legal to call it with variables of type `Thinger`. If you use `ThingerInt`, the code compiles and works as expected:

但是，使用 `Thinger` 类型的变量调用它是完全合法的。如果您使用 `ThingerInt`，代码可以编译并按预期运行：

```
var a4 Thinger = a2
var b4 Thinger = b2
Comparer(a4, b4) // prints true
```

But you can also assign `ThingerSlice` to variables of type `Thinger`. That's where things go wrong:

但是，你也可以将 `ThingerSlice` 赋值给类型为 `Thinger` 的变量。问题就出在这里：

```
a4 = a3
b4 = b3
Comparer(a4, b4) // compiles, panics at runtime
```

The compiler won't stop you from building this code, but if you run it, your program will panic with the message `panic: runtime error: comparing uncomparable type main.ThingerSlice` (see [“panic and recover”](#) for more information). You can try this code yourself on [The Go Playground](#) or in the `sample_code/more_comparable` directory in the [Chapter 8 repository](#).

编译器不会阻止你编译这段代码，但如果你运行它，程序会崩溃并显示错误信息 `panic: runtime error: comparing uncomparable type main.ThingerSlice`（更多信息请参阅“panic and recover”）。你可以在 Go Playground 或第 8 章代码库的 `sample_code/more_comparable` 目录中亲自尝试这段代码。

For more technical details on how comparable types and generics interact and why this design decision was made, read the blog post [“All Your Comparable Types”](#) from Robert Griesemer on the Go team.

有关可比较类型和泛型如何交互以及为什么做出此设计决策的更多技术细节，请阅读 Go 团队 Robert Griesemer 的博客文章“所有可比较类型”。

Things That Are Left Out

被遗漏的事项

Go remains a small, focused language, and the generics implementation for Go doesn't include many features that are found in generics implementations in other languages. This section describes some of the features that are not in the initial implementation of Go generics.

Go 仍然是一门小而精悍的语言，其泛型实现并不包含许多其他语言泛型实现中常见的特性。本节将介绍 Go 泛型初始实现中缺少的一些特性。

While you can build a single tree that works with both user-defined and built-in types, languages like Python, Ruby, and C++ solve this problem in a different way. They include *operator overloading*, which allows user-defined types to specify implementations for operators. Go will not be adding this feature. This means that you can't use `range` to iterate over user-defined container types or `[]` to index into them.

虽然你可以构建一个同时支持用户自定义类型和内置类型的单一树，但像 Python、Ruby 和 C++ 这样的语言以不同的方式解决了这个问题。它们引入了运算符重载，允许用户自定义类型指定运算符的实

现。Go 语言不会添加此功能。这意味着你不能使用 `range` 来遍历用户自定义的容器类型，也不能使用 `[]` 来对其进行索引。

There are good reasons for leaving out operator overloading. For one thing, Go has a surprisingly large number of operators. Go also doesn't have function or method overloading, and you'd need a way to specify different operator functionality for different types. Furthermore, operator overloading can lead to code that's harder to follow as developers invent clever meanings for symbols (in C++, `<<` means “shift bits left” for some types and “write the value on the right to the value on the left” for others). These are the sorts of readability issues that Go tries to avoid.

不采用运算符重载是有充分理由的。首先，Go 的运算符数量非常庞大。其次，Go 不支持函数或方法重载，而你需要一种方法来为不同类型的数据指定不同的运算符功能。此外，运算符重载会导致代码难以阅读，因为开发者会为符号赋予各种巧妙的含义（例如，在 C++ 中，`<<` 对于某些类型表示“左移位”，而对于其他类型则表示“将右侧的值写入左侧的值”）。这些正是 Go 试图避免的可读性问题。

Another useful feature that's been left out of the initial Go generics implementation is additional type parameters on methods. Looking back on the `Map/Reduce/Filter` functions, you might think they'd be useful as methods, like this:

另一个在最初的 Go 泛型实现中被遗漏的实用特性是方法上的附加类型参数。回顾 `Map/Reduce/Filter` 函数，你可能会觉得它们作为方法会很有用，就像这样：

```
type functionalSlice[T any] []T

// THIS DOES NOT WORK
func (fs functionalSlice[T]) Map[E any](f func(T)
E) functionalSlice[E] {
    out := make(functionalSlice[E], len(fs))
    for i, v := range fs {
        out[i] = f(v)
    }
    return out
}

// THIS DOES NOT WORK
func (fs functionalSlice[T]) Reduce[E any](start
E, f func(E, T) E) E {
    out := start
    for _, v := range fs {
        out = f(out, v)
    }
    return out
}
```

which you could use like this:

你可以这样使用：

```
var numStrings = functionalSlice[string>{"1", "2",  
"3"}  
sum := numStrings.Map(func(s string) int {  
    v, _ := strconv.Atoi(s)  
    return v  
}).Reduce(0, func(acc int, cur int) int {  
    return acc + cur  
})
```

Unfortunately for fans of functional programming, this does not work. Rather than chaining method calls together, you need to either nest function calls or use the much more readable approach of invoking the functions one at a time and assigning the intermediate values to variables. The type parameter proposal goes into detail on the reasons for excluding parameterized methods.

对于函数式编程的爱好者来说，很遗憾，这种方法行不通。你不能将方法调用串联起来，而是需要嵌套函数调用，或者采用更易读的方式，即一次调用一个函数，并将中间值赋给变量。类型参数提案详细解释了排除参数化方法的原因。

Go also has no variadic type parameters. As discussed in [“Variadic Input Parameters and Slices”](#), to implement a function that takes in a varying number of parameters, you specify that the last parameter type starts with

... . For example, there's no way to specify some sort of type pattern to those variadic parameters, such as alternating `string` and `int` . All variadic variables must match a single declared type, which can be generic or not.

Go 语言也没有可变参数类型。正如“可变输入参数和切片”一节所述，要实现一个接受数量可变参数的函数，你需要指定最后一个参数的类型以 `...` 开头。例如，无法为这些可变参数指定某种类型模式，例如交替使用 `string` 和 `int` 。所有可变变量必须匹配一个已声明的类型，该类型可以是泛型，也可以是非泛型。

Other features left out of Go generics are more esoteric. These include the following:

Go 泛型中省略的其他一些特性则更为深奥，其中包括以下几点：

Specialization

专业化

A function or method can be overloaded with one or more type-specific versions in addition to the generic version. Since Go doesn't have overloading, this feature is not under consideration.

除了通用版本之外，函数或方法还可以重载一个或多个特定类型的版本。由于 Go 语言不支持重载，因此此功能暂不考虑。

Currying

咖喱

Allows you to partially instantiate a function or type based on another generic function or type by specifying some of the type parameters.

允许您通过指定一些类型参数，基于另一个泛型函数或类型部分实例化一个函数或类型。

Metaprogramming

元编程

Allows you to specify code that runs at compile time to produce code that runs at runtime.

允许您指定在编译时运行的代码，以生成在运行时运行的代码。

Idiomatic Go and Generics

惯用 Go 语言和泛型

Adding generics clearly changes some of the advice for how to use Go idiomatically. The use of `float64` to represent any numeric type will end. You should use `any` instead of `interface{}` to represent an unspecified type in a data structure or function parameter. You can handle different slice

types with a single function. But don't feel the need to switch all your code over to using type parameters immediately. Your old code will still work as new design patterns are invented and refined.

添加泛型显然会改变一些 Go 语言的惯用写法。使用 `float64` 来表示任何数值类型的做法将不再适用。你应该使用 `any` 而不是 `interface{}` 来表示数据结构或函数参数中未指定的类型。你可以使用单个函数处理不同的切片类型。但不必立即将所有代码都切换到使用类型参数。随着新的设计模式的出现和完善，你之前的代码仍然可以正常工作。

It's still too early to judge the long-term impact of generics on performance. As of this writing, there's no impact on compilation time. The Go 1.18 compiler was slower than previous versions, but the compiler in Go 1.20 resolved this issue.

现在判断泛型对性能的长期影响还为时尚早。截至撰写本文时，编译时间尚未受到影响。Go 1.18 编译器比之前的版本慢，但 Go 1.20 中的编译器解决了这个问题。

Some research has also been done on the runtime impact of generics. Vicent Marti wrote a [detailed blog post](#) exploring cases where generics result in slower code and the implementation details that explain why this is so. Conversely, Eli Bendersky wrote a [blog post](#) that shows that generics make sorting algorithms faster.

一些研究也探讨了泛型对运行时性能的影响。Vicent Marti 撰写了一篇详细的博文，分析了泛型导致代码运行速度变慢的情况，并解释了造成这种现象的实现细节。相反，Eli Bendersky 也撰写了一篇博文，表明泛型可以加快排序算法的运行速度。

In particular, do not change a function that has an interface parameter into a function with a generic type parameter in hopes of improving performance. For example, converting this trivial function:

尤其不要为了提升性能而将带有接口参数的函数改为带有泛型类型参数的函数。例如，将以下这个简单的函数：

```
type Ager interface {  
    age() int  
}  
  
func doubleAge(a Ager) int {  
    return a.age() * 2  
}
```

into:

进入：

```
func doubleAgeGeneric[T Ager](a T) int {  
    return a.age() * 2  
}
```

```
}
```

makes the function call about 30% slower in Go 1.20. (For a nontrivial function, there's no significant performance difference.) You can run the benchmarks using the code in the *sample_code/perf* directory in the [Chapter 8 repository](#).

在 Go 1.20 中，函数调用速度会降低约 30%。（对于非平凡函数，性能差异并不显著。）您可以使用第 8 章存储库中的 *sample_code/perf* 目录中的代码运行基准测试。

This might be surprising for developers experienced with generics in other languages. In C++, for example, the compiler uses generics on abstract data types to turn what is normally a runtime operation (figuring out which concrete type is being used) into a compile-time one, generating a unique function for each concrete type. This makes the resulting binary larger but also faster. As Vicent explains in his blog post, the current Go compiler generates only unique functions for different underlying types. Furthermore, all pointer types share a single generated function. To differentiate between the different types that are passed in to shared generated functions, the compiler adds additional runtime lookups. This causes the slowdown in performance.

对于熟悉其他语言泛型的开发者来说，这可能会令人惊讶。例如，在 C++ 中，编译器会利用抽象数据类型的泛型，将通常需要在运行时进

行的操作（确定具体类型）转换为编译时操作，从而为每个具体类型生成一个唯一的函数。这虽然会增加生成的二进制文件的大小，但也会提高速度。正如 Vicent 在他的博客文章中所解释的，当前的 Go 编译器仅为不同的底层类型生成唯一的函数。此外，所有指针类型共享同一个生成的函数。为了区分传递给共享函数的不同类型，编译器会添加额外的运行时查找。这导致了性能下降。

Again, as the generics implementation matures in future versions of Go, expect runtime performance to improve. As always, the goal is to write maintainable programs that are fast enough to meet your needs. Use the benchmarking and profiling tools that are discussed in [“Using Benchmarks”](#) to measure and improve your code.

再次强调，随着 Go 语言未来版本中泛型实现的不断完善，运行时性能有望得到提升。一如既往，我们的目标是编写易于维护且速度足以满足需求的程序。请使用“使用基准测试”一节中讨论的基准测试和性能分析工具来衡量和改进您的代码。

Adding Generics to the Standard Library

向标准库添加泛型

The initial release of generics in Go 1.18 was very conservative. It added the new interfaces `any` and `comparable` to the universe block, but no

API changes occurred in the standard library to support generics. A stylistic change has been made; virtually all uses of `interface{}` in the standard library were replaced with `any`.

Go 1.18 中泛型的初始版本非常保守。它向 `universe` 代码块添加了新的接口 `any` 和 `comparable`，但标准库的 API 并未进行任何更改以支持泛型。不过，代码风格上做了一些调整：标准库中几乎所有 `interface{}` 的使用都被替换成了 `any`。

Now that the Go community is more comfortable with generics, we are starting to see more changes. Starting with Go 1.21, the standard library includes functions that use generics to implement common algorithms for slices, maps, and concurrency. In [Chapter 3](#), I covered the `Equal` and `EqualFunc` functions in the `slices` and `maps` packages. Other functions in these packages simplify slice and map manipulation. The `Insert`, `Delete`, and `DeleteFunc` functions in the `slices` package allow developers to avoid building some surprisingly tricky slice-handling code. The `maps.Clone` function takes advantage of the Go runtime to provide a faster way to create a shallow copy of a map. In [“Run Code Exactly Once”](#), you’ll learn about `sync.OnceValue` and `sync.OnceValues`, which use generics to build functions that are invoked only once and return one or two values. Prefer using functions in these packages over writing your own implementations. Future versions of the standard library will likely include additional new functions and types that take advantage of generics.

随着 Go 社区对泛型的接受度越来越高，我们开始看到更多变化。从 Go 1.21 开始，标准库中包含了一些使用泛型的函数，用于实现切片、映射和并发等常用算法。在第三章中，我介绍了 `slices` 和 `maps` 包中的 `Equal` 和 `EqualFunc` 函数。这些包中的其他函数简化了切片和映射的操作。`slices` 包中的 `Insert`、`Delete` 和 `DeleteFunc` 函数使开发者能够避免编写一些非常复杂的切片处理代码。

`maps.Clone` 函数利用 Go 运行时的特性，提供了一种更快速地创建映射浅拷贝的方法。在“仅运行一次代码”一节中，您将学习 `sync.OnceValue` 和 `sync.OnceValues`，它们使用泛型来构建仅调用一次并返回一个或两个值的函数。建议您优先使用这些包中的函数，而不是编写自己的实现。标准库的未来版本可能会包含更多利用泛型的新函数和类型。

Future Features Unlocked

未来功能已解锁

Generics might be the basis for other future features. One possibility is *sum types*. Just as type elements are used to specify the types that can be substituted for a type parameter, they could also be used for interfaces in variable parameters. This would enable some interesting features. Today, Go has a problem with a common situation in JSON: a field that can be a single value or a list of values. Even with generics, the only way to handle

this is with a field of type `any`. Adding sum types would allow you to create an interface specifying that a field could be a string, a slice of strings, and nothing else. A type switch could then completely enumerate every valid type, improving type safety. This ability to specify a bounded set of types allows many modern languages (including Rust and Swift) to use sum types to represent enums. Given the weakness of Go's current enum features, this might be an attractive solution, but it will take time for these ideas to be evaluated and explored.

泛型或许会成为未来其他特性的基础。其中一种可能性是和类型。正如类型元素用于指定可替换类型参数的类型一样，它们也可以用于可变参数中的接口。这将带来一些有趣的特性。目前，Go 在处理 JSON 中常见的情况时遇到了问题：一个字段可以是单个值，也可以是值列表。即使使用泛型，处理这种情况的唯一方法是使用 `any` 类型的字段。添加和类型将允许你创建一个接口，指定一个字段可以是字符串、字符串切片，仅此而已。然后，类型开关可以完整枚举所有有效类型，从而提高类型安全性。这种指定有限类型集的能力使得许多现代语言（包括 Rust 和 Swift）可以使用和类型来表示枚举。鉴于 Go 当前枚举特性的不足，这可能是一个很有吸引力的解决方案，但这些想法需要时间来评估和探索。

Exercises

练习

Now that you've seen how generics work, apply them to solve the following problems. Solutions are available in the *exercise_solutions* directory in the [Chapter 8 repository](#).

现在你已经了解了泛型的工作原理，请运用它们来解决以下问题。答案位于第 8 章代码库的 `exercise_solutions` 目录中。

1. Write a generic function that doubles the value of any integer or float that's passed in to it. Define any needed generic interfaces.

编写一个通用函数，该函数可以将传入的任何整数或浮点数的值翻倍。定义任何需要的通用接口。

2. Define a generic interface called `Printable` that matches a type that implements `fmt.Stringer` and has an underlying type of `int` or `float64`. Define types that meet this interface. Write a function that takes in a `Printable` and prints its value to the screen using `fmt.Println`.

定义一个名为 `Printable` 的通用接口，该接口匹配实现了 `fmt.Stringer` 接口且底层类型为 `int` 或 `float64` 的类型。定义满足此接口的类型。编写一个函数，该函数接收一个

`Printable` 类型的值，并使用 `fmt.Println` 将其值打印到屏幕上。

3. Write a generic singly linked list data type. Each element can hold a comparable value and has a pointer to the next element in the list. The methods to implement are as follows:

编写一个通用的单链表数据类型。每个元素可以存储可比较的值，并包含一个指向列表中下一个元素的指针。需要实现的方法如下：

```
// adds a new element to the end of the linked list
Add(T)
// adds an element at the specified position
in the linked list
Insert(T, int)
// returns the position of the supplied value,
-1 if it's not present
Index (T) int
```

Wrapping Up

总结

In this chapter, you took a look at generics and how to use them to simplify your code. It's still early days for generics in Go. It will be exciting to see how they help grow the language while still maintaining the spirit that makes Go special.

在本章中，你了解了泛型以及如何使用它们来简化代码。Go 语言中的泛型仍处于发展初期。令人兴奋的是，我们将看到它们如何在保持 Go 语言独特精神的同时，促进语言的发展。

In the next chapter, you are going to learn how to properly use one of Go's most controversial features: errors.

在下一章中，你将学习如何正确使用 Go 语言最具争议的特性之一：错误。

Chapter 9. Errors

第九章 错误

Error handling is one of the biggest challenges for developers moving to Go from other languages. For those used to exceptions, Go's approach feels anachronistic. But solid software engineering principles underlie Go's approach. In this chapter, you'll learn how to work with errors in Go. You'll also take a look at `panic` and `recover`, Go's system for handling errors that should stop execution.

对于从其他语言迁移到 Go 的开发者来说，错误处理是最大的挑战之一。对于习惯使用异常的开发者的而言，Go 的错误处理方式似乎有些过时。但 Go 的错误处理方式背后有着扎实的软件工程原则。在本章中，你将学习如何在 Go 中处理错误。你还将了解 `panic` 和 `recover`，它们是 Go 用于处理需要停止程序执行的错误的系统。

How to Handle Errors: The Basics

如何处理错误：基础知识

As was covered briefly in [Chapter 5](#), Go handles errors by returning a value of type `error` as the last return value for a function. This is entirely by

convention, but it is such a strong convention that it should never be breached. When a function executes as expected, `nil` is returned for the error parameter. If something goes wrong, an error value is returned instead. The calling function then checks the error return value by comparing it to `nil`, handling the error, or returning an error of its own. A simple function with error handling looks like this:

正如第五章简要介绍的那样，Go 语言通过在函数的最后一个返回值中返回 `error` 类型的值来处理错误。这完全是约定俗成的做法，但这种约定非常重要，绝不应该被打破。当函数按预期执行时，错误参数会返回 `nil`。如果出现错误，则会返回一个错误值。调用函数后会检查错误返回值是否与 `nil` 进行比较，并处理该错误，或者返回一个自身的错误。一个带有错误处理的简单函数如下所示：

```
func calcRemainderAndMod(numerator, denominator
int) (int, int, error) {
    if denominator == 0 {
        return 0, 0, errors.New("denominator is
0")
    }
    return numerator / denominator, numerator %
denominator, nil
}
```

A new error is created from a string by calling the `New` function in the `errors` package. Error messages should not be capitalized nor should

they end with punctuation or a newline. In most cases, you should set the other return values to their zero values when a non-nil error is returned. You'll see an exception to this rule when I cover sentinel errors.

通过调用 `errors` 包中的 `New` 函数，可以从字符串创建一个新的错误。错误信息不应使用大写字母，也不应以标点符号或换行符结尾。大多数情况下，当返回非空错误时，应将其他返回值设置为零。在讲解哨兵错误时，您会看到此规则的例外情况。

Unlike languages with exceptions, Go doesn't have special constructs to detect if an error was returned. When a function returns an error, use an `if` statement to check the error variable to see if it is non-nil:

与带有异常处理的语言不同，Go 没有专门的结构来检测是否返回了错误。当函数返回错误时，可以使用 `if` 语句来检查错误变量是否为非空值：

```
func main() {
    numerator := 20
    denominator := 3
    remainder, mod, err :=
    calcRemainderAndMod(numerator, denominator)
    if err != nil {
        fmt.Println(err)
        os.Exit(1)
    }
}
```

```
    fmt.Println(remainder, mod)
}
```

You can try out this code in the *sample_code/error_basics* directory in the [Chapter 9 repository](#).

您可以在第 9 章存储库的 *sample_code/error_basics* 目录中尝试此代码。

`error` is a built-in interface that defines a single method:

`error` 是一个内置接口，它定义了一个方法：

```
type error interface {
    Error() string
}
```

Anything that implements this interface is considered an error. The reason you return `nil` from a function to indicate that no error occurred is that `nil` is the zero value for any interface type.

任何实现了此接口的对象都被视为错误。之所以从函数返回 `nil` 以表明未发生错误，是因为 `nil` 是任何接口类型的零值。

There are two very good reasons that Go uses a returned error instead of thrown exceptions. First, exceptions add at least one new code path through

the code. These paths are sometimes unclear, especially in languages whose functions don't include a declaration that an exception is possible. This produces code that crashes in surprising ways when exceptions aren't properly handled, or, even worse, code that doesn't crash but whose data is not properly initialized, modified, or stored.

Go 使用返回错误而不是抛出异常有两个非常重要的原因。首先，异常至少会增加一条新的代码路径。这些路径有时并不清晰，尤其是在那些函数没有声明可能抛出异常的语言中。这会导致代码在异常处理不当时以意想不到的方式崩溃，或者更糟糕的是，代码虽然没有崩溃，但其数据却没有被正确初始化、修改或存储。

The second reason is more subtle but demonstrates how Go's features work together. The Go compiler requires all variables to be read. Making errors returned values forces developers to either check and handle error conditions or make it explicit that they are ignoring errors by using an underscore (`_`) for the returned error value.

第二个原因比较微妙，但也体现了 Go 语言特性之间的协同作用。Go 编译器要求读取所有变量。如果将错误作为返回值，开发者就必须检查并处理错误情况，或者通过使用下划线（例如 `_`）来明确表示忽略错误。

Exception handling may produce shorter code, but having fewer lines doesn't necessarily make code easier to understand or maintain. As you've

seen, idiomatic Go favors clear code, even if it takes more lines.

异常处理或许能减少代码量，但代码行数减少并不一定意味着代码更容易理解或维护。正如你所见，惯用的 Go 语言风格倾向于编写清晰的代码，即使这意味着代码行数更多。

Another thing to note is how code flows in Go. The error handling is indented inside an `if` statement. The business logic is not. This gives a quick visual clue to which code is along the “golden path” and which code is the exceptional condition.

另一点需要注意的是 Go 代码的流程。错误处理代码缩进在 `if` 语句中，而业务逻辑则没有缩进。这样可以快速直观地看出哪些代码位于“黄金路径”上，哪些代码是异常情况处理代码。

NOTE

笔记

The second situation is a reused `err` variable. The Go compiler requires every variable to be read at least once. It doesn't require that *every* write to a variable is read. If you use an `err` variable multiple times, you have to read it only once to make the compiler happy. In “[staticcheck](#)”, you'll see a way to detect this.

第二种情况是重复使用 `err` 变量。Go 编译器要求每个变量至少被读取一次，但并不要求每次写入变量都必须读取一次。如果您多次使用 `err` 变量，只需读取一次即可满足编译器的要求。在“staticcheck”中，您可以找到检测这种情况的方法。

Use Strings for Simple Errors

使用字符串表示简单错误

Go's standard library provides two ways to create an error from a string.

The first is the `errors.New` function. It takes a `string` and returns an `error`. This string is returned when you call the `Error` method on the returned error instance. If you pass an error to `fmt.Println`, it calls the `Error` method automatically:

Go 标准库提供了两种从字符串创建错误的方法。第一种是 `errors.New` 函数。它接收一个 `string` 并返回一个 `error`。当你对返回的错误实例调用 `Error` 方法时，就会返回这个字符串。如果你将错误传递给 `fmt.Println`，它会自动调用 `Error` 方法：

```
func doubleEven(i int) (int, error) {
    if i % 2 != 0 {
        return 0, errors.New("only even numbers
are processed")
    }
    return i * 2, nil
}

func main() {
```

```

    result, err := doubleEven(1)
    if err != nil {
        fmt.Println(err) // prints "only even
numbers are processed"
    }
    fmt.Println(result)
}

```

The second way is to use the `fmt.Errorf` function. This function allows you to include runtime information in the error message by using the `fmt.Printf` verbs to format an error string. Like `errors.New`, this string is returned when you call the `Error` method on the returned error instance:

第二种方法是使用 `fmt.Errorf` 函数。此函数允许您使用 `fmt.Printf` 动词格式化错误字符串，从而在错误消息中包含运行时信息。与 `errors.New` 类似，当您对返回的错误实例调用 `Error` 方法时，将返回此字符串：

```

func doubleEven(i int) (int, error) {
    if i % 2 != 0 {
        return 0, fmt.Errorf("%d isn't an even
number", i)
    }
}

```

```
    return i * 2, nil  
}
```

You can find this code in the *sample_code/string_error* directory in the [Chapter 9 repository](#).

您可以在第 9 章存储库的 *sample_code/string_error* 目录中找到此代码。

Sentinel Errors

哨兵错误

Some errors are meant to signal that processing cannot continue because of a problem with the current state. In his blog post [“Don’t Just Check Errors, Handle Them Gracefully”](#), Dave Cheney, a developer who has been active in the Go community for many years, coined the term *sentinel errors* to describe these:

有些错误旨在表明由于当前状态存在问题，处理无法继续。在题为“不要只是检查错误，要优雅地处理它们”的博文中，活跃于 Go 社区多年的开发者 Dave Cheney 创造了“哨兵错误”一词来描述这些错误：

The name descends from the practice in computer programming of using a specific value to signify that no further processing is possible. So too with Go, we use specific values to signify an error.

这个名称源于计算机编程中用特定值表示无法继续处理的做法。
在 Go 语言中，我们也使用特定值来表示错误。

Sentinel errors are one of the few variables that are declared at the package level. By convention, their names start with `Err` (with the notable exception of `io.EOF`). They should be treated as read-only; there's no way for the Go compiler to enforce this, but it is a programming error to change their value.

哨兵错误是少数几个在包级别声明的变量之一。按照惯例，它们的名称以 `Err` 开头（值得注意的是 `io.EOF`）。它们应该被视为只读变量；Go 编译器无法强制执行此操作，但更改它们的值属于编程错误。

Sentinel errors are usually used to indicate that you cannot start or continue processing. For example, the standard library includes a package for processing ZIP files, `archive/zip`. This package defines several sentinel errors, including `ErrFormat`, which is returned when data that doesn't represent a ZIP file is passed in. Try out this code on [The Go Playground](#) or in the `sample_code/sentinel_error` directory in the [Chapter 9 repository](#):

哨兵错误通常用于指示您无法开始或继续处理。例如，标准库包含一个用于处理 ZIP 文件的包 `archive/zip`。该包定义了几个哨兵错

误，包括 `ErrFormat`，当传入的数据不代表 ZIP 文件时，将返回此错误。您可以在 Go Playground 或第 9 章代码库的 `sample_code/sentinel_error` 目录中尝试此代码：

```
func main() {
    data := []byte("This is not a zip file")
    notAZipFile := bytes.NewReader(data)
    _, err := zip.NewReader(notAZipFile,
int64(len(data)))
    if err == zip.ErrFormat {
        fmt.Println("Told you so")
    }
}
```

Another example of a sentinel error in the standard library is `rsa.ErrMessageTooLong` in the `crypto/rsa` package. It indicates that a message cannot be encrypted because it is too long for the provided public key. The sentinel error `context.Canceled` is covered in [Chapter 14](#).

标准库中另一个哨兵错误示例是 `crypto/rsa` 包中的 `rsa.ErrMessageTooLong`。它表示消息长度超过提供的公钥所能加密的范围，因此无法加密。哨兵错误 `context.Canceled` 将在第 14 章中介绍。

Be sure you need a sentinel error before you define one. Once you define one, it is part of your public API, and you have committed to it being available in all future backward-compatible releases. It's far better to reuse one of the existing ones in the standard library or to define an error type that includes information about the condition that caused the error to be returned (you'll see how to do that in the next section). But if you have an error condition that indicates a specific state has been reached in your application where no further processing is possible and no additional information needs to be used to explain the error state, a sentinel error is the correct choice.

在定义哨兵错误之前，请务必确认您确实需要它。一旦定义了哨兵错误，它就成为公共 API 的一部分，并且您承诺它将在所有未来的向后兼容版本中可用。更好的做法是重用标准库中现有的哨兵错误，或者定义一个包含导致错误返回的条件信息的错误类型（您将在下一节中看到如何操作）。但是，如果您有一个错误条件，表明您的应用程序已达到特定状态，无法进行进一步处理，并且不需要使用任何额外信息来解释错误状态，那么哨兵错误是正确的选择。

How do you test for a sentinel error? As you can see in the preceding code sample, use `==` to test whether the error was returned when calling a function whose documentation explicitly says it returns a sentinel error. In [“Is and As”](#), I discuss how to check for sentinel errors in other situations.

如何检测哨兵错误？如前面的代码示例所示，可以使用 `==` 来检测调用一个文档明确指出会返回哨兵错误的函数时是否返回了该错误。在

“Is and As”一节中，我将讨论如何在其他情况下检查哨兵错误。

USING CONSTANTS FOR SENTINEL ERRORS

使用常量表示哨兵错误

In [“Constant Errors”](#), Dave Cheney proposes that constants would make useful sentinel errors. You’d have a type like this in a package (I’ll talk about creating packages in [Chapter 10](#)):

在《常量错误》一文中，Dave Cheney 提出常量可以作为有用的哨兵错误类型。你可以在一个包中使用类似这样的类型（我将在第 10 章讨论如何创建包）：

```
package consterr

type Sentinel string

func(s Sentinel) Error() string {
    return string(s)
}
```

Then you’d use it like this:

然后你可以这样使用它：

```
package mypkg

const (
    ErrFoo = consterr.Sentinel("foo error")
)
```

```
ErrBar = consterr.Sentinel("bar error")
)
```

This looks like a function call, but it's actually casting a string literal to a type that implements the `error` interface. Changing the values of `ErrFoo` and `ErrBar` would be impossible. At first glance, this looks like a good solution.

这看起来像是一个函数调用，但实际上是将字符串字面量强制转换为实现了 `error` 接口的类型。这样就无法更改 `ErrFoo` 和 `ErrBar` 的值。乍一看，这似乎是一个不错的解决方案。

However, this practice isn't considered idiomatic. If you used the same type to create constant errors across packages, two errors would be equal if their error strings are equal. They'd also be equal to a string literal with the same value. Meanwhile, an error created with `errors.New` is equal only to itself or to variables explicitly assigned its value. You almost certainly do not want to make errors in different packages equal to each other; otherwise, why declare two different errors? (You could avoid this by creating a nonpublic error type in every package, but that's a lot of boilerplate.)

然而，这种做法并不符合惯用规范。如果使用相同的类型在不同包中创建常量错误，那么两个错误如果其错误字符串相同，则它们将被视为相等。它们也等同于具有相同值的字符串字面量。而使用

`errors.New` 创建的错误仅等同于它自身或显式赋值为该值的变量。你几乎肯定不希望不同包中的错误彼此相等；否则，为什么要声明两种不同的错误呢？（你可以通过在每个包中创建一个非公共的错误类型来避免这种情况，但这会造成大量的样板代码。）

The sentinel error pattern is another example of the Go design philosophy. Sentinel errors should be rare, so they can be handled by convention instead of language rules. Yes, they are public package-level variables. This makes them mutable, but it's highly unlikely someone would accidentally reassign a public variable in a package. In short, it's a corner case that is handled by other features and patterns. The Go philosophy is that it's better to keep the language simple and trust the developers and tooling than it is to add features.

哨兵错误模式是 Go 设计理念的另一个例证。哨兵错误应该很少见，因此可以通过约定而非语言规则来处理。没错，它们是包级别的公共变量。这使得它们可变，但几乎不可能有人会意外地重新赋值包中的公共变量。简而言之，这是一个特殊情况，可以通过其他特性和模式来处理。Go 的理念是，与其添加新功能，不如保持语言的简洁性并信任开发者和工具。

So far, all the errors that you've seen are strings. But Go errors can contain more information. Let's see how.

到目前为止，你看到的所有错误都是字符串。但 Go 错误可以包含更多信息。让我们来看看如何实现。

Errors Are Values

错误也是价值

Since `error` is an interface, you can define your own errors that include additional information for logging or error handling. For example, you might want to include a status code as part of the error to indicate the kind of error that should be reported back to the user. This lets you avoid string comparisons (whose text might change) to determine error causes. Let's see how this works. First, define your own enumeration to represent the status codes:

由于 `error` 是一个接口，您可以定义自己的错误，其中包含用于日志记录或错误处理的附加信息。例如，您可能希望在错误中包含状态码，以指示应向用户报告的错误类型。这样，您就可以避免使用字符串比较（其文本可能会更改）来确定错误原因。让我们看看它是如何工作的。首先，定义您自己的枚举来表示状态码：

```
type Status int
```

```
const (
```



```
        InvalidLogin Status = iota + 1  
        NotFound  
    )
```

Next, define a `StatusErr` to hold this value:

接下来，定义一个 `StatusErr` 来保存这个值：

```
type StatusErr struct {  
    Status    Status  
    Message   string  
}  
  
func (se StatusErr) Error() string {  
    return se.Message  
}
```

Now you can use `StatusErr` to provide more details about what went wrong:

现在您可以使用 `StatusErr` 提供有关出错原因的更多详细信息：

```
func LoginAndGetData(uid, pwd, file string)  
([]byte, error) {  
    token, err := login(uid, pwd)  
    if err != nil {
```

```

        return nil, StatusErr{
            Status:    InvalidLogin,
            Message: fmt.Sprintf("invalid
credentials for user %s", uid),
        }
    }
    data, err := getData(token, file)
    if err != nil {
        return nil, StatusErr{
            Status:    NotFound,
            Message: fmt.Sprintf("file %s not
found", file),
        }
    }
    return data, nil
}

```

You can find this code in the *sample_code/custom_error* directory in the [Chapter 9 repository](#).

您可以在第 9 章存储库的 *sample_code/custom_error* 目录中找到此代码。

Even when you define your own custom error types, always use `error` as the return type for the error result. This allows you to return different types of errors from your function and allows callers of your function to choose not to depend on the specific error type.

即使您定义了自定义错误类型，也请始终使用 `error` 作为错误结果的返回类型。这样，您可以从函数返回不同类型的错误，并允许函数调用者选择不依赖于特定的错误类型。

If you are using your own error type, be sure you don't return an uninitialized instance. Let's see what happens if you do. Try out the following code on [The Go Playground](#) or in the *sample_code/return_custom_error* directory in the [Chapter 9 repository](#):

如果您使用自定义错误类型，请确保不要返回未初始化的实例。我们来看看如果这样做会发生什么。请在 Go Playground 或第 9 章代码库的 *sample_code/return_custom_error* 目录中尝试以下代码：

```
func GenerateErrorBroken(flag bool) error {
    var genErr StatusErr
    if flag {
        genErr = StatusErr{
            Status: NotFound,
        }
    }
    return genErr
}

func main() {
    err := GenerateErrorBroken(true)
    fmt.Println("GenerateErrorBroken(true) returns non-nil error:", err != nil)
```

```
    err = GenerateErrorBroken(false)
    fmt.Println("GenerateErrorBroken(false)
returns non-nil error:", err != nil)
}
```

Running this program produces the following output:

运行此程序会产生以下输出：

```
true
true
```

This isn't a pointer type versus value type issue; if you declared `genErr` to be of type `*StatusErr`, you'd see the same output. The reason `err` is non-nil is that `error` is an interface. As I discussed in [“Interfaces and nil”](#), for an interface to be considered `nil`, both the underlying type and the underlying value must be `nil`. Whether or not `genErr` is a pointer, the underlying type part of the interface is not `nil`.

这并非指针类型与值类型之争；如果您将 `genErr` 声明为 `*StatusErr` 类型，也会看到相同的输出。`err` 不为 `nil` 的原因是 `error` 是一个接口。正如我在“接口与 `nil`”一文中讨论的，要使一个接口被视为 `nil`，其底层类型和底层值都必须是 `nil`。无论 `genErr` 是否是指针，接口的底层类型部分都不是 `nil`。

You can fix this in two ways. The most common approach is to explicitly return `nil` for the error value when a function completes successfully:

您可以通过两种方式解决这个问题。最常见的方法是在函数成功执行时，显式地返回 `nil` 作为错误值：

```
func GenerateErrorOKReturnNil(flag bool) error {  
    if flag {  
        return StatusErr{  
            Status: NotFound,  
        }  
    }  
    return nil  
}
```

This has the advantage of not requiring you to read through code to make sure that the error variable on the `return` statement is correctly defined.

这样做的好处是，您无需通读代码来确保 `return` 语句中的错误变量已正确定义。

Another approach is to make sure that any local variable that holds an `error` is of type `error` :

另一种方法是确保任何保存 `error` 的局部变量都是 `error` 类型：

```
func GenerateErrorUseErrorVar(flag bool) error {  
    var genErr error  
    if flag {  
        genErr = StatusErr{  
            Status: NotFound,  
        }  
    }  
    return genErr  
}
```

WARNING

警告

When using custom errors, never define a variable to be of the type of your custom error. Either explicitly return `nil` when no error occurs or define the variable to be of type `error`.

使用自定义错误时，切勿将变量定义为自定义错误的类型。要么在未发生错误时显式返回 `nil`，要么将变量定义为 `error` 类型。

As was covered in [“Use Type Assertions and Type Switches Sparingly”](#), don’t use a type assertion or a type switch to access the fields and methods of a custom error. Instead, use `errors.As`, which is discussed in [“Is and As”](#).

正如“谨慎使用类型断言和类型开关”一节中所述，不要使用类型断言或类型开关来访问自定义错误的字段和方法。相反，请使用

`errors.As`，这在“Is 和 As”一节中有所讨论。

Wrapping Errors

包装错误

When an error is passed back through your code, you often want to add information to it. This can be the name of the function that received the error or the operation it was trying to perform. When you preserve an error while adding information, it is called *wrapping* the error. When you have a series of wrapped errors, it is called an *error tree*.

当错误被传递回代码时，你通常需要添加一些信息。这些信息可以是接收错误的函数名称，也可以是它尝试执行的操作。在添加信息的同时保留错误本身，这称为错误包装。一系列包装后的错误构成了一个错误树。

A function in the Go standard library wraps errors, and you've already seen it. The `fmt.Errorf` function has a special verb, `%w`. Use this to create an error whose formatted string includes the formatted string of another error and which contains the original error as well. The convention is to write `:` `%w` at the end of the error format string and to make the error to be wrapped the last parameter passed to `fmt.Errorf`.

Go 标准库中有一个函数可以包装错误，你已经见过它了。

`fmt.Errorf` 函数有一个特殊的动词 `%w`。使用它可以创建一个错误，该错误格式化后的字符串包含另一个错误的格式化字符串，并且还包含原始错误。约定是在错误格式化字符串的末尾添加 `: %w`，并将要包装的错误作为传递给 `fmt.Errorf` 的最后一个参数。

The standard library also provides a function for unwrapping errors, the `Unwrap` function in the `errors` package. You pass it an error, and it returns the wrapped error if there is one. If there isn't, it returns `nil`.

Here's a quick program that demonstrates wrapping with `fmt.Errorf` and unwrapping with `errors.Unwrap`. You can run it on [The Go Playground](#) or in the `sample_code/wrap_error` directory in the [Chapter 9 repository](#):

标准库还提供了一个用于解包错误的函数，即 `errors` 包中的 `Unwrap` 函数。你向它传递一个错误，如果存在已包装的错误，它会返回该包装后的错误。如果没有，它会返回 `nil`。以下是一个演示如何使用 `fmt.Errorf` 进行包装以及如何使用 `errors.Unwrap` 进行解包的简单程序。你可以在 Go Playground 上运行它，也可以在第九章代码库的 `sample_code/wrap_error` 目录中运行它：

```
func fileChecker(name string) error {
    f, err := os.Open(name)
    if err != nil {
        return fmt.Errorf("in fileChecker: %w",
```



```

err)
    }
    f.Close()
    return nil
}

func main() {
    err := fileChecker("not_here.txt")
    if err != nil {
        fmt.Println(err)
        if wrappedErr := errors.Unwrap(err);
wrappedErr != nil {
            fmt.Println(wrappedErr)
        }
    }
}

```

When you run this program, you see the following output:

运行此程序后，您将看到以下输出：

```

in fileChecker: open not_here.txt: no such file or
directory
open not_here.txt: no such file or directory

```

NOTE

笔记

You don't usually call `errors.Unwrap` directly. Instead, you use `errors.Is` and `errors.As` to find a specific wrapped error. I'll talk about these two functions in the next section.

通常情况下，你不会直接调用 `errors.Unwrap`。而是使用 `errors.Is` 和 `errors.As` 来查找特定的包装错误。我将在下一节中讨论这两个函数。

If you want to wrap an error with your custom error type, your error type needs to implement the method `Unwrap`. This method takes in no parameters and returns an `error`. Here's an update to the error that you defined earlier to demonstrate how this works. You can find it in the *sample_code/custom_wrapped_error* directory in the [Chapter 9 repository](#):

如果您想使用自定义错误类型包装错误，则您的错误类型需要实现 `Unwrap` 方法。此方法不接受任何参数，并返回一个 `error`。以下是您之前定义的错误的更新示例，用于演示其工作原理。您可以在第 9 章代码库的 `sample_code/custom_wrapped_error` 目录中找到它：

```
type StatusErr struct {
    Status Status
    Message string
    Err     error
}

func (se StatusErr) Error() string {
```

```

    return se.Message
}

func (se StatusErr) Unwrap() error {
    return se.Err
}

```

Now you can use `StatusErr` to wrap underlying errors:

现在您可以使用 `StatusErr` 来包装底层错误:

```

func LoginAndGetData(uid, pwd, file string)
([]byte, error) {
    token, err := login(uid,pwd)
    if err != nil {
        return nil, StatusErr {
            Status: InvalidLogin,
            Message: fmt.Sprintf("invalid
credentials for user %s",uid),
            Err: err,
        }
    }
    data, err := getData(token, file)
    if err != nil {
        return nil, StatusErr {
            Status: NotFound,
            Message: fmt.Sprintf("file %s not

```

```
found",file),  
        Err: err,  
    }  
}  
return data, nil  
}
```

Not all errors need to be wrapped. A library can return an error that means processing cannot continue, but the error message contains implementation details that aren't needed in other parts of your program. In this situation, it is perfectly acceptable to create a brand-new error and return that instead. Understand the situation and determine what needs to be returned.

并非所有错误都需要包装。库可能会返回一个错误，表明处理无法继续，但该错误信息包含程序其他部分不需要的实现细节。在这种情况下，完全可以创建一个全新的错误并返回它。理解具体情况，确定需要返回什么。

TIP

提示

If you want to create a new error that contains the message from another error, but don't want to wrap it, use `fmt.Errorf` to create an error but use the `%v` verb instead of `%w`:

如果您想创建一个包含另一个错误消息的新错误，但不想将其包装起来，请使用 `fmt.Errorf` 创建错误，但使用 `%v` 动词而不是 `%w`：

```
err := internalFunction()
if err != nil {
    return fmt.Errorf("internal failure: %v", err)
}
```

Wrapping Multiple Errors

包装多个错误

Sometimes a function generates multiple errors that should be returned. For example, if you wrote a function to validate the fields in a struct, it would be better to return an error for each invalid field. Since the standard function signature returns `error` and not `[]error`, you need to merge multiple errors into a single error. That's what the `errors.Join` function is for:

有时，一个函数会生成多个需要返回的错误。例如，如果您编写了一个函数来验证结构体中的字段，最好为每个无效字段返回一个错误。由于标准函数签名返回的是 `error` 而不是 `[]error`，因此您需要将多个错误合并为一个错误。这就是 `errors.Join` 函数的作用：

```
type Person struct {
    FirstName string
    LastName  string
    Age       int
}

func ValidatePerson(p Person) error {
    var errs []error
    if len(p.FirstName) == 0 {
        errs = append(errs, errors.New("field
FirstName cannot be empty"))
    }
    if len(p.LastName) == 0 {
        errs = append(errs, errors.New("field
LastName cannot be empty"))
    }
    if p.Age < 0 {
        errs = append(errs, errors.New("field Age
cannot be negative"))
    }
    if len(errs) > 0 {
        return errors.Join(errs...)
    }
}
```

```
    }  
    return nil  
}
```

You can find this code in the *sample_code/join_error* directory in the [Chapter 9 repository](#).

您可以在第 9 章存储库的 *sample_code/join_error* 目录中找到此代码。

Another way to merge multiple errors is to pass multiple `%w` verbs to `fmt.Errorf`:

合并多个错误的另一种方法是将多个 `%w` 个动词传递给 `fmt.Errorf`:

```
err1 := errors.New("first error")  
err2 := errors.New("second error")  
err3 := errors.New("third error")  
err := fmt.Errorf("first: %w, second: %w, third:  
%w", err1, err2, err3)
```

You can implement your own `error` type that supports multiple wrapped errors. To do so, implement the `Unwrap` method but have it return `[]error` instead of `error`:

您可以实现自己的 `error` 类型，以支持多个包装错误。为此，请实现 `Unwrap` 方法，但使其返回 `[]error` 而不是 `error`：

```
type MyError struct {
    Code    int
    Errors []error
}

type (m MyError) Error() string {
    return errors.Join(m.Errors...).Error()
}

func (m MyError) Unwrap() []error {
    return m.Errors
}
```

Go doesn't support method overloading, so you can't create a single type that provides both implementations of `Unwrap`. Also note that the `errors.Unwrap` function will return `nil` if you pass it an error that implements the `[]error` variant of `Unwrap`. This is another reason you shouldn't call the `errors.Unwrap` function directly.

Go 语言不支持方法重载，因此您无法创建一个类型来同时提供 `Unwrap` 的两种实现。另请注意，如果您向 `errors.Unwrap` 函数传递一个实现了 `Unwrap` 的 `[]error` 变体的错误，则该函数将返回

`nil`。这也是您不应该直接调用 `errors.Unwrap` 函数的另一个原因。

If you need to handle errors that may wrap zero, one, or multiple errors, use this code as a basis. You can find it in the *sample_code/custom_wrapped_multi_error* directory in the [Chapter 9 repository](#):

如果您需要处理可能包含零个、一个或多个错误的情况，请使用以下代码作为基础。您可以在第 9 章代码库的 *sample_code/custom_wrapped_multi_error* 目录中找到它：

```
var err error
err = funcThatReturnsAnError()
switch err := err.(type) {
case interface {Unwrap() error}:
    // handle single error
    innerErr := err.Unwrap()
    // process innerErr
case interface {Unwrap() []error}:
    //handle multiple wrapped errors
    innerErrs := err.Unwrap()
    for _, innerErr := range innerErrs {
        // process each innerErr
    }
default:
```

```
// handle no wrapped error  
}
```

Since the standard library doesn't define interfaces to represent errors with either `Unwrap` variant, this code uses anonymous interfaces in a type switch to match the methods and access the wrapped errors. Before writing your own code, see if you can use `errors.Is` and `errors.As` to examine your error trees. Let's see how they work.

由于标准库没有定义用于表示 `Unwrap` 变体错误的接口，这段代码使用类型切换中的匿名接口来匹配方法并访问包装后的错误。在编写自己的代码之前，请尝试使用 `errors.Is` 和 `errors.As` 来检查您的错误树。让我们看看它们是如何工作的。

Is and As

是和作为

Wrapping errors is a useful way to get additional information about an error, but it introduces problems. If a sentinel error is wrapped, you cannot use `==` to check for it, nor can you use a type assertion or type switch to match a wrapped custom error. Go solves this problem with two functions in the `errors` package, `Is` and `As`.

包装错误是获取错误附加信息的一种有效方法，但它也引入了一些问题。如果哨兵错误被包装，则无法使用 `==` 来检查它，也无法使用类型断言或类型切换来匹配包装后的自定义错误。Go 通过 `errors` 包中的两个函数 `Is` 和 `As` 解决了这个问题。

To check whether the returned error or any errors that it wraps match a specific sentinel error instance, use `errors.Is`. It takes in two parameters: the error being checked and the instance you are comparing it against. The `errors.Is` function returns `true` if any error in the error tree matches the provided sentinel error. You'll write a short program to see `errors.Is` in action. You can run it yourself on [The Go Playground](#) or in the `sample_code/is_error` directory in the [Chapter 9 repository](#):

要检查返回的错误或其包装的任何错误是否与特定的哨兵错误实例匹配，请使用 `errors.Is`。它接受两个参数：要检查的错误和要与之比较的实例。如果错误树中的任何错误与提供的哨兵错误匹配，则 `errors.Is` 函数返回 `true`。您将编写一个简短的程序来查看 `errors.Is` 的实际应用。您可以在 Go Playground 上自行运行它，也可以在第九章代码库的 `sample_code/is_error` 目录中运行它：

```
func fileChecker(name string) error {
    f, err := os.Open(name)
    if err != nil {
        return fmt.Errorf("in fileChecker: %w",
err)
    }
}
```

```
f.Close()
return nil
}

func main() {
    err := fileChecker("not_here.txt")
    if err != nil {
        if errors.Is(err, os.ErrNotExist) {
            fmt.Println("That file doesn't exist")
        }
    }
}
```

Running this program produces the following output:

运行此程序会产生以下输出：

```
That file doesn't exist
```

By default, `errors.Is` uses `==` to compare each wrapped error with the specified error. If this does not work for an error type that you define (for example, if your error is a noncomparable type), implement the `Is` method on your error:

默认情况下，`errors.Is` 使用 `==` 将每个包装的错误与指定的错误进行比较。如果此方法对您定义的错误类型不起作用（例如，如果您

的错误是不可比较的类型)，请在您的错误上实现 `Is` 方法：

```
type MyErr struct {
    Codes []int
}

func (me MyErr) Error() string {
    return fmt.Sprintf("codes: %v", me.Codes)
}

func (me MyErr) Is(target error) bool {
    if me2, ok := target.(MyErr); ok {
        return slices.Equal(me.Codes, me2.Codes)
    }
    return false
}
```

(The `slices.Equal` function was mentioned back in [“Slices”](#).)

(`slices.Equal` 函数在“切片”一节中已经提到过。)

Another use for defining your own `Is` method is to allow comparisons against errors that aren't identical instances. You might want to pattern match your errors, specifying a filter instance that matches errors that have some of the same fields. Define a new error type, `ResourceErr` :

定义自定义的 `Is` 方法的另一个用途是允许与非完全相同的错误实例进行比较。您可能需要对错误进行模式匹配，指定一个筛选实例来匹配具有某些相同字段的错误。定义一个新的错误类型

`ResourceErr` :

```
type ResourceErr struct {
    Resource    string
    Code        int
}

func (re ResourceErr) Error() string {
    return fmt.Sprintf("%s: %d", re.Resource,
re.Code)
}
```

If you want two `ResourceErr` instances to match when either field is set, you can do so by writing a custom `Is` method:

如果您希望两个 `ResourceErr` 实例在任一字段设置时匹配，您可以通过编写自定义的 `Is` 方法来实现：

```
func (re ResourceErr) Is(target error) bool {
    if other, ok := target.(ResourceErr); ok {
        ignoreResource := other.Resource == ""
        ignoreCode := other.Code == 0
        matchResource := other.Resource ==
```

```
re.Resource
    matchCode := other.Code == re.Code
    return matchResource && matchCode ||
        matchResource && ignoreCode ||
        ignoreResource && matchCode
}
return false
}
```

Now you can find, for example, all errors that refer to the database, no matter the code:

现在，您可以找到例如所有与数据库相关的错误，无论错误代码是什么：

```
if errors.Is(err, ResourceErr{Resource:
    "Database"}) {
    fmt.Println("The database is broken:", err)
    // process the codes
}
```

You can see this code on [The Go Playground](#) or in the *sample_code/custom_is_error_pattern_match* directory in the [Chapter 9 repository](#).

您可以在 Go Playground 或第 9 章存储库的 `sample_code/custom_is_error_pattern_match` 目录中看到此代码。

The `errors.As` function allows you to check whether a returned error (or any error it wraps) matches a specific type. It takes in two parameters. The first is the error being examined, and the second is a pointer to a variable of the type that you are looking for. If the function returns `true`, an error in the error tree was found that matched, and that matching error is assigned to the second parameter. If the function returns `false`, no match was found in the error tree. Try it out with `MyErr`:

`errors.As` 函数允许您检查返回的错误（或其包装的任何错误）是否与特定类型匹配。它接受两个参数。第一个参数是要检查的错误，第二个参数是指向您要查找类型的变量的指针。如果函数返回 `true`，则表示在错误树中找到了一个匹配的错误，并将该匹配的错误赋值给第二个参数。如果函数返回 `false`，则表示在错误树中未找到匹配项。请使用 `MyErr` 进行尝试：

```
err := AFunctionThatReturnsAnError()
var myErr MyErr
if errors.As(err, &myErr) {
    fmt.Println(myErr.Codes)
}
```


Note that you use `var` to declare a variable of a specific type set to the zero value. You then pass a pointer to this variable into `errors.As`.

请注意，您可以使用 `var` 声明一个特定类型且值设为零的变量。然后，您需要将指向该变量的指针传递给 `errors.As`。

You don't have to pass a pointer to a variable of an error type as the second parameter to `errors.As`. You can pass a pointer to an interface to find an error that meets the interface:

您不必将指向错误类型变量的指针作为第二个参数传递给 `errors.As`。您可以传递指向接口的指针来查找符合该接口的错误：

```
err := AFunctionThatReturnsAnError()
var coder interface {
    CodeVals() []int
}
if errors.As(err, &coder) {
    fmt.Println(coder.CodeVals())
}
```

The example uses an anonymous interface, but any interface type is acceptable. You can find both `errors.As` examples in the *sample_code/custom_as_error* directory in the [Chapter 9 repository](#).

示例使用了匿名接口，但任何接口类型都是可以接受的。您可以在第 9 章代码库的 `sample_code/custom_as_error` 目录中找到这两个 `errors.As` 示例。

WARNING

警告

If the second parameter to `errors.As` is anything other than a pointer to an error or a pointer to an interface, the method panics.

如果 `errors.As` 的第二个参数不是指向错误的指针或指向接口的指针，则该方法会引发 panic。

Just as you can override the default `errors.Is` comparison with an `Is` method, you can override the default `errors.As` comparison with an `As` method on your error. Implementing an `As` method is nontrivial and requires reflection (I will talk about reflection in Go in [Chapter 16](#)). You should do it only in unusual circumstances, such as when you want to match an error of one type and return another.

就像你可以用 `Is` 方法覆盖默认的 `errors.Is` 比较一样，你也可以用 `As` 方法覆盖错误处理中的默认 `errors.As` 比较。实现 `As` 方法并非易事，需要用到反射（我将在第 16 章讨论 Go 语言中的反射）。你只有在特殊情况下才应该这样做，例如当你想要匹配一种类型的错误并返回另一种类型时。

TIP

提示

Use `errors.Is` when you are looking for a specific *instance* or specific *values*. Use `errors.As` when you are looking for a specific *type*.

查找特定实例或特定值时，请使用 `errors.Is`。查找特定类型时，请使用 `errors.As`。

Wrapping Errors with defer

使用 defer 包装错误

Sometimes you find yourself wrapping multiple errors with the same message:

有时你会发现自己用同一条消息包裹了多个错误：

```
func DoSomeThings(val1 int, val2 string) (string,
error) {
    val3, err := doThing1(val1)
    if err != nil {
        return "", fmt.Errorf("in DoSomeThings:
%w", err)
    }
    val4, err := doThing2(val2)
    if err != nil {
```

```

        return "", fmt.Errorf("in DoSomeThings:
%w", err)
    }
    result, err := doThing3(val3, val4)
    if err != nil {
        return "", fmt.Errorf("in DoSomeThings:
%w", err)
    }
    return result, nil
}

```

You can simplify this code by using `defer` :

您可以使用 `defer` 简化此代码：

```

func DoSomeThings(val1 int, val2 string) (_
string, err error) {
    defer func() {
        if err != nil {
            err = fmt.Errorf("in DoSomeThings:
%w", err)
        }
    }()
    val3, err := doThing1(val1)
    if err != nil {
        return "", err
    }
}

```

```
    val4, err := doThing2(val2)
    if err != nil {
        return "", err
    }
    return doThing3(val3, val4)
}
```

You have to name the return values so that you can refer to `err` in the deferred function. If you name a single return value, you must name all of them, so you use an underscore here for the string return value that isn't explicitly assigned.

您必须为返回值命名，以便在延迟函数中引用 `err`。如果您只命名了一个返回值，则必须命名所有返回值，因此对于未显式赋值的字符串返回值，此处使用下划线。

In the `defer` closure, the code checks whether an error was returned. If so, it reassigns the error to a new error that wraps the original error with a message indicating which function detected the error.

在 `defer` 闭包中，代码会检查是否返回了错误。如果返回了错误，则会将错误重新赋值给一个新的错误，该错误会包装原始错误并附带一条消息，指示哪个函数检测到了该错误。

This pattern works well when you are wrapping every error with the same message. If you want to customize the wrapping error with more detail, put

both the specific and the general message in every `fmt.Errorf` .

当您使用相同的消息包裹所有错误时，此模式效果很好。如果您想使用更详细的信息自定义包裹错误，请在每个 `fmt.Errorf` 中同时放入特定消息和通用消息。

panic and recover

恐慌和恢复

Previous chapters have mentioned panics in passing without going into any details on what they are. A *panic* is similar to an `Error` in Java or Python. It is a state generated by the Go runtime whenever it is unable to figure out what should happen next. This is almost always due to a programming error, like an attempt to read past the end of a slice or passing a negative size to `make` . The Go runtime also panics if it detects bugs in itself, such as the garbage collector misbehaving. However, I've never seen this happen. If there's a panic, blame the runtime last.

前面的章节只是顺带提到了 `panic`，并没有详细解释它是什么。`panic` 类似于 Java 或 Python 中的 `Error`。当 Go 运行时无法确定下一步该做什么时，就会生成 `panic` 状态。这几乎总是由于编程错误造成的，例如尝试读取切片末尾之外的数据，或者向 `make` 传递负值大小。如果 Go 运行时检测到自身存在 bug，例如垃圾回收器行为异常，也会触

发 panic。不过，我从未遇到过这种情况。如果出现 panic，请最后再考虑运行时的问題。

As soon as a panic happens, the current function exits immediately, and any defer s attached to the current function start running. When those defer s complete, the defer s attached to the calling function run, and so on, until main is reached. The program then exits with a message and a stack trace.

一旦发生 panic，当前函数会立即退出，所有附加到当前函数的 defer 都会开始运行。当这些 defer 执行完毕后，附加到调用函数的 defer 会运行，依此类推，直到遇到 main。程序随后退出，并输出一条消息和堆栈跟踪信息。

NOTE

笔记

If there is a panic in a goroutine other than the main goroutine (goroutines are covered in [“Goroutines”](#)), the chain of defers ends at the function used to launch the goroutine. A program exits if *any* goroutine panics without being recovered.

如果除主 goroutine 之外的其他 goroutine 发生 panic（goroutine 的相关内容请参见“Goroutines”章节），则 defer 链会在启动该 goroutine 的函数处终止。如果任何 goroutine 发生 panic 且未被恢复，程序将退出。

If any situations in your programs are unrecoverable, you can create your own panics. The built-in function `panic` takes one parameter, which can

be of any type. Usually, it is a string. Following is a trivial program that panics; you can run it on [The Go Playground](#) or in the *sample_code/panic* directory in the [Chapter 9 repository](#):

如果你的程序中存在无法恢复的情况，你可以创建自己的 panic。内置函数 `panic` 接受一个参数，该参数可以是任何类型。通常情况下，它是一个字符串。以下是一个简单的 panic 示例程序；你可以在 Go Playground 或第 9 章代码库的 *sample_code/panic* 目录中运行它：

```
func doPanic(msg string) {  
    panic(msg)  
}  
  
func main() {  
    doPanic(os.Args[0])  
}
```

Running this code produces the following output:

运行此代码会产生以下输出：

```
panic: /tmpfs/play  
  
goroutine 1 [running]:  
main.doPanic(...)   
    /tmp/sandbox567884271/prog.go:6
```



```
main.main()
/tmp/sandbox567884271/prog.go:10 +0x5f
```

As you can see, a `panic` prints out its message followed by a stack trace.

如您所见，`panic` 会打印出其消息，然后是堆栈跟踪。

Go provides a way to capture a `panic` to provide a more graceful shutdown or to prevent shutdown at all. The built-in `recover` function is called from within a `defer` to check whether a `panic` happened. If there was a `panic`, the value assigned to the `panic` is returned. Once a `recover` happens, execution continues normally. Let's take a look with another sample program. Run it on [The Go Playground](#) or in the `sample_code/panic_recover` directory in the [Chapter 9 repository](#):

Go 提供了一种捕获 `panic` 的方法，以便更优雅地关闭程序或完全阻止程序关闭。内置的 `recover` 函数会在 `defer` 内部被调用，以检查是否发生了 `panic`。如果发生了 `panic`，则返回分配给 `panic` 的值。一旦发生了 `recover`，程序将正常执行。让我们通过另一个示例程序来看一下。您可以在 Go Playground 或第 9 章代码库的 `sample_code/panic_recover` 目录中运行它：

```
func div60(i int) {
    defer func() {
        if v := recover(); v != nil {
```

```

        fmt.Println(v)
    }
}()
fmt.Println(60 / i)
}

func main() {
    for _, val := range []int{1, 2, 0, 6} {
        div60(val)
    }
}

```

There's a specific pattern for using `recover`. You register a function with `defer` to handle a potential `panic`. You call `recover` within an `if` statement and check whether a non-nil value was found. You must call `recover` from within a `defer` because once a `panic` happens, only deferred functions are run.

使用 `recover` 有特定的模式。您需要使用 `defer` 注册一个函数来处理潜在的 `panic` 事件。您需要在 `if` 语句中调用 `recover`，并检查是否找到了非空值。您必须在 `defer` 中调用 `recover`，因为一旦发生 `panic` 事件，只会运行延迟函数。

Running this code produces the following output:

运行此代码会产生以下输出：

```
60
30
runtime error: integer divide by zero
10
```

Since `recover` uses a non-nil value to detect whether a `panic` happened, a clever reader might raise the question: *What happens if you call `panic(nil)` and there's a `recover`?* In code compiled with Go versions before 1.21, the answer was “nothing great.” In those versions, `recover` stops a `panic` from propagating, but no message or data indicates what happened. Starting with Go 1.21, a `panic(nil)` call is identical to `panic(new(runtime.PanicNilError))`.

由于 `recover` 使用非空值来检测 `panic` 是否发生，聪明的读者可能会提出这样的问题：如果调用 `panic(nil)` 时已经存在 `recover`，会发生什么？在 Go 1.21 之前的版本中，答案是“没什么特别的”。在这些版本中，`recover` 会阻止 `panic` 的传播，没有任何消息或数据表明发生了什么。从 Go 1.21 开始，`panic(nil)` 调用与 `panic(new(runtime.PanicNilError))` 完全相同。

While `panic` and `recover` look a lot like exception handling in other languages, they are not intended to be used that way. Reserve panics for fatal situations and use `recover` as a way to gracefully handle these situations. If your program panics, be careful about trying to continue executing after the panic. You'll rarely want to keep your program running

after a `panic` occurs. If the `panic` was triggered because the computer is out of a resource like memory or disk space, the safest thing to do is use `recover` to log the situation to monitoring software and shut down with `os.Exit(1)`. If a programming error caused the panic, you can try to continue, but you'll likely hit the same problem again. In the preceding sample program, it would be idiomatic to check for division by zero and return an error if one was passed in.

虽然 `panic` 和 `recover` 看起来很像其他语言中的异常处理，但它们的用途并非如此。请将 `panic` 保留用于致命情况，并使用 `recover` 来优雅地处理这些情况。如果您的程序发生 `panic`，请谨慎尝试在 `panic` 后继续执行。您很少会在发生 `panic` 后继续运行程序。如果 `panic` 是由于计算机资源（例如内存或磁盘空间）不足而触发的，最安全的做法是使用 `recover` 将情况记录到监控软件，然后使用 `os.Exit(1)` 关闭程序。如果 `panic` 是由编程错误引起的，您可以尝试继续执行，但很可能会再次遇到相同的问题。在前面的示例程序中，通常的做法是检查除以零的情况，如果传入了除以零的语句，则返回错误。

The reason you don't rely on `panic` and `recover` is that `recover` doesn't make clear *what* could fail. It just ensures that *if* something fails, you can print out a message and continue. Idiomatic Go favors code that explicitly outlines the possible failure conditions over shorter code that handles anything while saying nothing.

之所以不依赖 `panic` 和 `recover`，是因为 `recover` 并没有明确指出哪些操作可能会失败。它只是确保如果出现错误，你可以打印出一条消息并继续执行。Go 语言的惯用写法倾向于使用明确列出所有可能失败情况的代码，而不是使用简短但什么都不说明的、处理所有情况的代码。

Using `recover` is recommended in one situation. If you are creating a library for third parties, do not let panics escape the boundaries of your public API. If a `panic` is possible, a public function should use `recover` to convert the `panic` into an error, return it, and let the calling code decide what to do with them.

在一种情况下建议使用 `recover`。如果您正在为第三方创建库，请不要让 `panic` 超出您的公共 API 的范围。如果 `panic` 是可能的，则公共函数应该使用 `recover` 将 `panic` 转换为错误并返回，然后让调用代码决定如何处理。

NOTE

笔记

While the HTTP server built into Go recovers from panics in handlers, David Symonds said in a [GitHub comment](#) that as of 2015, this is now considered a mistake by the Go team.

虽然 Go 内置的 HTTP 服务器可以从处理程序中的 `panic` 中恢复，但 David Symonds 在 GitHub 评论中表示，截至 2015 年，Go 团队已将此视为一个错误。

Getting a Stack Trace from an Error

从错误中获取堆栈跟踪

One of the reasons that new Go developers are tempted to use `panic` and `recover` is they want to get a stack trace when something goes wrong. By default, Go doesn't provide that. As I've shown, you can use error wrapping to build a call stack by hand, but some third-party libraries with error types generate those stacks automatically (see [Chapter 10](#) to learn how to incorporate third-party code in your program). Cockroachdb provides a [third-party library](#) with functions for wrapping errors with stack traces.

新手 Go 开发者之所以倾向于使用 `panic` 和 `recover`，其中一个原因是他们希望在出错时获取堆栈跟踪信息。默认情况下，Go 并不提供此功能。正如我之前展示的，你可以使用错误包装手动构建调用堆栈，但一些第三方库的错误类型可以自动生成这些堆栈（请参阅第 10 章了解如何将第三方代码集成到你的程序中）。CockroachDB 提供了一个第三方库，其中包含用于将错误包装成堆栈跟踪信息的函数。

By default, the stack trace is not printed out. If you want to see the stack trace, use `fmt.Printf` and the verbose output verb (`%+v`). Check the [documentation](#) to learn more.

默认情况下，不会打印堆栈跟踪信息。如果需要查看堆栈跟踪信息，请使用 `fmt.Printf` 和详细输出动词 (`%+v`)。请查阅文档了解更多信息。

NOTE

笔记

When you have a stack trace in your error, the output includes the full path to the file on the computer where the program was compiled. If you don't want to expose the path, use the `-trimpath` flag when building your code. This replaces the full path with the package.

当错误信息中包含堆栈跟踪时，输出会包含程序编译所在计算机上文件的完整路径。如果您不想暴露该路径，请在构建代码时使用 `-trimpath` 标志。该标志会将完整路径替换为包名。

Exercises

练习

Look at the code in the *sample_code/exercise* directory in the [Chapter 9 repository](#). You are going to modify this code in each of these exercises. It works correctly, but improvements should be made to its error handling.

请查看第 9 章代码库中 `sample_code/exercise` 目录下的代码。在接下来的练习中，你需要修改这段代码。它目前运行正常，但错误处理部分需要改进。

1. Create a sentinel error to represent an invalid ID. In `main`, use `errors.Is` to check for the sentinel error, and print a message when it is found.

创建一个哨兵错误来表示无效 ID。在 `main` 中，使用 `errors.Is` 检查哨兵错误，并在找到错误时打印一条消息。

2. Define a custom error type to represent an empty field error. This error should include the name of the empty `Employee` field. In `main`, use `errors.As` to check for this error. Print out a message that includes the field name.

定义一个自定义错误类型来表示空字段错误。此错误应包含空字段 `Employee` 的名称。在 `main` 中，使用 `errors.As` 检查此错误。打印一条包含字段名称的消息。

3. Rather than returning the first error found, return back a single error that contains all errors discovered during validation. Update the code in `main` to properly report multiple errors.

不要返回第一个发现的错误，而是返回一个包含验证过程中发现的所有错误的单一错误。请更新 `main` 中的代码，以正确报告多个错误。

Wrapping Up

总结

This chapter covered errors in Go, what they are, how to define your own, and how to examine them. You also took a look at `panic` and `recover`. The next chapter discusses packages and modules, how to use third-party code in your programs, and how to publish your own code for others to use.

本章介绍了 Go 语言中的错误，包括错误的定义、如何定义错误以及如何检查错误。您还了解了 `panic` 和 `recover`。下一章将讨论包和模块、如何在程序中使用第三方代码以及如何发布自己的代码供他人使用。

Chapter 10. Modules, Packages, and Imports

第十章 模块、包和导入

Most modern programming languages have a system for organizing code into namespaces and libraries, and Go is no exception. As you've seen while exploring other features, Go introduces some new approaches to this old idea. In this chapter, you'll learn how to organize code with packages and modules, how to import them, how to work with third-party libraries, and how to create libraries of your own.

大多数现代编程语言都有一套将代码组织成命名空间和库的系统，Go 也不例外。正如你在探索其他特性时所看到的，Go 为这个古老的概念引入了一些新的方法。在本章中，你将学习如何使用包和模块来组织代码，如何导入它们，如何使用第三方库，以及如何创建自己的库。

Repositories, Modules, and Packages

存储库、模块和软件包

Library management in Go is based around three concepts: repositories, modules, and packages. A *repository* is familiar to all developers. It is a place in a version control system where the source code for a project is stored. A *module* is a bundle of Go source code that's distributed and versioned as a single unit. Modules are stored in a repository. Modules consist of one or more *packages*, which are directories of source code. Packages give a module organization and structure.

Go 语言的库管理基于三个概念：仓库（repository）、模块（modules）和包（packages）。仓库对于所有开发者来说都很熟悉。它是版本控制系统中用于存储项目源代码的地方。模块是 Go 源代码的捆绑包，作为一个整体进行分发和版本控制。模块存储在仓库中。模块由一个或多个包组成，包是源代码的目录。包赋予模块组织和结构。

NOTE

笔记

While you can store more than one module in a repository, it is discouraged. Everything within a module is versioned together. Maintaining two modules in one repository requires you to track separate versions for two different modules in a single repository.

虽然你可以在一个代码仓库中存储多个模块，但并不建议这样做。模块内的所有内容都是统一版本控制的。在同一个代码仓库中维护两个模块意味着你需要分别跟踪这两个模块的版本。

Unfortunately, different programming languages use these words to represent different concepts. While packages in Java and Go are similar, a Java repository is a centralized place to store multiple *artifacts* (the analogue of a Go module). Node.js and Go swap the meanings of the terms: a Node.js package is similar to what Go calls a module, and a Go package is similar to a Node.js module. The terminology certainly can be confusing at first, but as you get more comfortable with Go, the terms will seem more familiar.

遗憾的是，不同的编程语言使用这些术语来表示不同的概念。虽然 Java 和 Go 中的包很相似，但 Java 仓库是一个集中存储多个工件的地方（相当于 Go 模块）。Node.js 和 Go 则互换了这些术语的含义：Node.js 中的包类似于 Go 中的模块，而 Go 中的包则类似于 Node.js 中的模块。这些术语一开始可能会让人感到困惑，但随着你对 Go 越来越熟悉，这些术语就会变得更容易理解。

Before using code from packages outside the standard library, you need to make sure that you have a properly created module. Every module has a globally unique identifier. This is not unique to Go. Java defines globally unique package declarations by using the reverse domain name convention (`com.companyname.projectname.library`).

在使用标准库之外的包中的代码之前，需要确保已正确创建模块。每个模块都有一个全局唯一的标识符。这并非 Go 语言独有。Java 使用反向域名约定 (`com.companyname.projectname.library`) 来定义全局唯一的包声明。

In Go, this name is called a *module path*. It is usually based on the repository where the module is stored. For example, you can find Proteus, a module I wrote to simplify relational database access in Go, at <https://github.com/jonbodner/proteus>. It has a module path of `github.com/jonbodner/proteus`.

在 Go 语言中，这个名称被称为模块路径。它通常基于模块所在的仓库。例如，您可以在 <https://github.com/jonbodner/proteus> 找到我编写的用于简化 Go 中关系数据库访问的模块 Proteus。它的模块路径是 `github.com/jonbodner/proteus`。

NOTE

笔记

Back in [“Your First Go Program”](#), you created a module whose name was `hello_world`, which is obviously not globally unique. This is fine if you are creating a module for local use only. If you put a module with a nonunique name into a source code repository, it cannot be imported by another module.

在“你的第一个 Go 程序”中，你创建了一个名为 `hello_world` 的模块，显然这个名称不是全局唯一的。如果你创建的模块仅供本地使用，这没问题。但如果你将一个名称不唯一的模块放入源代码仓库，其他模块就无法导入它。

Using go.mod

使用 go.mod

A directory tree of Go source code becomes a module when there's a valid *go.mod* file in it. Rather than create this file manually, use the subcommands of the `go mod` command to manage modules. The command `go mod init MODULE_PATH` creates the *go.mod* file that makes the current directory the root of a module. The *MODULE_PATH* is the globally unique name that identifies your module. The module path is case-sensitive. To reduce confusion, do not use uppercase letters within it.

当 Go 源代码目录树中存在有效的 *go.mod* 文件时，该目录树就成为一个模块。与其手动创建此文件，不如使用 `go mod` 命令的子命令来管

理模块。命令 `go mod init MODULE_PATH` 会创建 `go.mod` 文件，并将当前目录设置为模块的根目录。`MODULE_PATH` 是全局唯一标识模块的名称。模块路径区分大小写。为避免混淆，请勿在路径中使用大写字母。

Take a look at the contents of a *go.mod* file:

请查看 `go.mod` 文件的内容：

```
module github.com/learning-go-book-2e/money

go 1.21

require (
    github.com/learning-go-book-2e/formatter
    v0.0.0-20220918024742-18...
    github.com/shopspring/decimal v1.3.1
)

require (
    github.com/fatih/color v1.13.0 // indirect
    github.com/mattn/go-colorable v0.1.9 //
indirect
    github.com/mattn/go-isatty v0.0.14 // indirect
    golang.org/x/sys v0.0.0-20210630005230-
```

```
0f9fa26af87c // indirect
)
```

Every *go.mod* file starts with a `module` directive that consists of the word `module` and the module's unique path. Next, the *go.mod* file specifies the minimum compatible version of Go with the `go` directive. All source code within the module must be compatible with the specified version. For example, if you specify the (rather old) version `1.12`, the compiler will not let you use underscores within numeric literals, because that feature was added in Go 1.13.

每个 *go.mod* 文件都以 `module` 指令开头，该指令由 `module` 和模块的唯一路径组成。接下来，*go.mod* 文件使用 `go` 指令指定 Go 的最低兼容版本。模块中的所有源代码都必须与指定的版本兼容。例如，如果您指定了（较旧的）版本 `1.12`，编译器将不允许您在数字字面量中使用下划线，因为该特性是在 Go 1.13 中添加的。

Use the go Directive to Manage Go Build Versions

使用 go 指令管理 Go 构建版本

What happens if the `go` directive specifies a version of Go that's newer than what's installed? If you have Go 1.20 or earlier installed, the newer Go version is ignored and you get the features of the installed version. If you have Go 1.21 or newer installed, the default behavior is to download the

newer version of Go and use it to build your code. You can control this behavior in Go 1.21 and newer with the `toolchain` directive and the `GOTOOLCHAIN` environment variable. You can assign them the following values:

如果 `go` 指令指定的 Go 版本比已安装的版本更新，会发生什么情况？如果您安装的是 Go 1.20 或更早版本，则会忽略更新的 Go 版本，您将获得已安装版本的功能。如果您安装的是 Go 1.21 或更新版本，则默认行为是下载更新的 Go 版本并使用它来构建代码。在 Go 1.21 及更高版本中，您可以使用 `toolchain` 指令和 `GOTOOLCHAIN` 环境变量来控制此行为。您可以为它们分配以下值：

- `auto` , which downloads newer versions of Go. (This is the default behavior of Go 1.21 and later.)
`auto` , 用于下载更新版本的 Go。（这是 Go 1.21 及更高版本的默认行为。）
- `local` , which restores the behavior of Go releases before 1.21.
`local` , 恢复 Go 1.21 版本之前的行为。
- A specific Go version (such as `go1.20.4`), which means that specific version will be downloaded and used to build the program.
指定一个 Go 版本（例如 `go1.20.4` ），这意味着将下载该特定版本并用于构建程序。

For example, the command line `GOTOOLCHAIN=go1.18 go build` will build your Go program with Go 1.18, downloading it if necessary.

例如，命令行 `GOTOOLCHAIN=go1.18 go build` 将使用 Go 1.18 构建您的 Go 程序，并在必要时下载它。

If both the `GOTOOLCHAIN` environment variable and the `toolchain` directive are set, the value assigned to `GOTOOLCHAIN` is used.

如果同时设置了 `GOTOOLCHAIN` 环境变量和 `toolchain` 指令，则使用分配给 `GOTOOLCHAIN` 的值。

Complete details on the `go` directive, the `toolchain` directive, and the `GOTOOLCHAIN` environment variable are found in the [official Go toolchain documentation](#).

有关 `go` 指令、`toolchain` 指令和 `GOTOOLCHAIN` 环境变量的完整详细信息，请参阅 Go 官方工具链文档。

As discussed in [“The for-range value is a copy”](#), Go 1.22 introduces the first backward-breaking change to the language. When using Go 1.22 or later, if the `go` directive is set to `1.22` or higher, a `for` loop creates a new index and value variable on each iteration. This behavior is applied per module. The value of the `go` directive in each imported module determines the language level for that module. ([“Importing Third-Party Code”](#) describes how to use and manage multiple modules in your programs.)

正如“for 循环的值是副本”一文中所述，Go 1.22 引入了该语言的一个向后破坏性变更。在使用 Go 1.22 或更高版本时，如果 `go` 指令的

值设置为 1.22 或更高，则 `for` 循环会在每次迭代时创建一个新的索引和值变量。此行为应用于每个模块。每个导入模块中的 `go` 指令的值决定了该模块的语言级别。（“导入第三方代码”一文介绍了如何在程序中使用和管理多个模块。）

You can see this difference with a short example. You can find the code in the *sample_code/loop_test* directory in the [Chapter 10 repository](#).

你可以通过一个简单的例子来看出这种区别。你可以在第 10 章代码库的 *sample_code/loop_test* 目录中找到代码。

The code in *loop.go* is simple:

loop.go 中的代码很简单：

```
func main() {  
    x := []int{1, 2, 3, 4, 5}  
    for _, v := range x {  
        fmt.Printf("%p\n", &v)  
    }  
}
```

If you haven't seen it before, the `%p` verb in the `fmt` formatting language returns the memory location of a pointer. The repository has the `go` directive in its *go.mod* file set to 1.21. Building and running the program gives the following output:

如果您之前没见过，`%p` 动词在 `fmt` 格式化语言中返回指针的内存地址。该仓库的 `go.mod` 文件中的 `go` 指令已设置为 `1.21`。构建并运行程序会得到以下输出：

```
140000140a8
140000140a8
140000140a8
140000140a8
140000140a8
```

When built using older versions of Go, the program prints out the same memory address five times. (The memory addresses might differ from what's shown here, but all will be the same value.)

如果使用旧版本的 Go 语言编写，程序会打印出相同的内存地址五次。（实际的内存地址可能与此处显示的有所不同，但值都相同。）

Change the value of the `go` directive in `go.mod` to `1.22` and rebuild and rerun the program. You will now get this output:

将 `go.mod` 文件中 `go` 指令的值更改为 `1.22`，然后重新编译并运行程序。现在您应该会看到以下输出：

```
1400000e0b0
1400000e0b8
1400000e0d0
```

```
1400000e0d8  
1400000e0e0
```

Notice that every memory address value is different, indicating that a new variable was created on each iteration. (The memory addresses might differ from what's shown here, but each one will be a different value.)

请注意，每个内存地址值都不同，这表明每次迭代都创建了一个新变量。（内存地址可能与此处显示的有所不同，但每个地址对应的值都不同。）

The require Directive

指令要求

The next sections in a *go.mod* file are the `require` directives. The `require` directives are present only if your module has dependencies. They list the modules that your module depends on and the minimum version required for each one. The first `require` section lists the direct dependencies of your module. The second one lists the dependencies of the dependencies of your module. Each line in this section ends with an `// indirect` comment. There's no functional difference between modules marked as indirect and those that aren't; it's just documentation for people when they look at the *go.mod* file. Direct dependencies are marked as

indirect in one situation, and I will discuss it when I talk about the ways to use `go get`. In [“Importing Third-Party Code”](#), you’ll learn more about adding and managing the dependencies of your module.

`go.mod` 文件中接下来的部分是 `require` 指令。只有当你的模块有依赖项时，才会出现 `require` 指令。它们列出了你的模块所依赖的模块以及每个模块所需的最低版本。第一个 `require` 部分列出了你的模块的直接依赖项。第二个部分列出了你的模块依赖项的依赖项。该部分中的每一行都以 `// indirect` 注释结尾。标记为间接依赖的模块和未标记为间接依赖的模块在功能上没有区别；这只是为了方便其他人查看 `go.mod` 文件时进行文档说明。在一种情况下，直接依赖项会被标记为间接依赖，我将在讨论 `go get` 的使用方法时进行阐述。在“导入第三方代码”中，你将了解更多关于添加和管理模块依赖项的信息。

While the `module`, `go`, and `require` directives are the ones most commonly used in a `go.mod` file, there are three others directives as well. I will cover the `replace` and `exclude` directives in [“Overriding Dependencies”](#), and I’ll cover the `retract` directive in [“Retracting a Version of Your Module”](#).

虽然 `module`、`go` 和 `require` 指令是 `go.mod` 文件中最常用的指令，但还有其他三个指令。我将在“覆盖依赖项”一节中介绍 `replace` 和 `exclude` 指令，并在“撤回模块版本”一节中介绍 `retract` 指令。

Building Packages

建筑套餐

Now that you've learned how to make your directory of code into a module, it's time to start using packages to organize your code. You'll start by seeing how `import` works, move on to creating and organizing packages, and then look at some of the features of Go's packages, both good and bad.

既然你已经学会了如何将代码目录创建为模块，现在就该开始使用包来组织代码了。你将首先了解 `import` 的工作原理，然后学习如何创建和组织包，最后还会了解 Go 包的一些特性，包括优点和缺点。

Importing and Exporting

进出口

The example programs have been using the `import` statement in Go even though I haven't discussed what it does and how importing in Go differs from other languages. Go's `import` statement allows you to access exported constants, variables, functions, and types in another package. A package's exported identifiers (an *identifier* is the name of a variable, constant, type, function, method, or a field in a struct) cannot be accessed from another current package without an `import` statement.

示例程序一直在使用 Go 语言中的 `import` 语句，尽管我还没有讨论它的作用以及 Go 语言的导入方式与其他语言有何不同。Go 语言的 `import` 语句允许你访问另一个包中导出的常量、变量、函数和类型。如果没有 `import` 语句，就无法从另一个当前包访问一个包的导出标识符（标识符是变量、常量、类型、函数、方法或结构体字段的名称）。

This leads to the question, how do you export an identifier in Go? Rather than use a special keyword, Go uses *capitalization* to determine whether a package-level identifier is visible outside the package where it is declared. An identifier whose name starts with an uppercase letter is *exported*. Conversely, an identifier whose name starts with a lowercase letter or underscore can be accessed only from within the package where it is declared. (Identifiers in Go can't start with a digit but can contain them.)

这就引出了一个问题：如何在 Go 语言中导出标识符？Go 语言并没有使用特殊的关键字，而是通过大小写来判断包级标识符是否在其声明的包之外可见。名称以大写字母开头的标识符会被导出。相反，名称以小写字母或下划线开头的标识符只能在其声明的包内部访问。（Go 语言中的标识符不能以数字开头，但可以包含数字。）

Anything you export is part of your package's API. Before you export an identifier, be sure that you intend to expose it to clients. Document all exported identifiers and keep them backward compatible unless you are

intentionally making a major version change (see [“Versioning Your Module”](#) for more information).

任何导出的内容都属于您软件包的 API。在导出标识符之前，请确保您打算将其暴露给客户端。记录所有导出的标识符，并保持其向后兼容，除非您有意进行主要版本更改（更多信息请参阅“模块版本控制”）。

Creating and Accessing a Package

创建和访问软件包

Making packages in Go is easy. Let's look at a small program to demonstrate this. You can find it in the [package_example directory for this book](#). Inside *package_example*, you'll see two additional directories, *math* and *do-format*. In *math*, there's a file called *math.go* with the following contents:

在 Go 中创建包非常简单。我们来看一个简单的程序来演示这一点。你可以在本书的 `package_example` 目录中找到它。在 `package_example` 目录下，你会看到另外两个目录：`math` 和 `do-format`。在 `math` 目录中，有一个名为 `math.go` 的文件，其内容如下：

```
package math
```

```
func Double(a int) int {  
    return a * 2  
}
```

The first line of the file is called the *package clause*. It consists of the keyword `package` and the name for the package. The package clause is always the first nonblank, noncomment line in a Go source file.

文件的第一行称为包声明（package clause）。它由关键字 `package` 和包名组成。在 Go 源文件中，包声明始终是第一个非空且非注释的行。

In the *do-format* directory is a file called *formatter.go* with the following contents:

在 *do-format* 目录中有一个名为 *formatter.go* 的文件，其内容如下：

```
package format  
  
import "fmt"  
  
func Number(num int) string {  
    return fmt.Sprintf("The number is %d", num)  
}
```

Note that the package name is `format` in the package clause, but it's in the *do-format* directory. How to interact with this package will be covered very shortly.

请注意，包名在 `package` 子句中为 `format`，但它位于 `do-format` 目录中。稍后将介绍如何与此包交互。

Finally, the following contents are in the file *main.go* in the root directory:

最后，根目录下的 `main.go` 文件包含以下内容：

```
package main

import (
    "fmt"

    "github.com/learning-go-book-2e/package_example/do-format"
    "github.com/learning-go-book-2e/package_example/math"
)

func main() {
    num := math.Double(2)
    output := format.Number(num)
```

```
    fmt.Println(output)
}
```

The first line of this file is familiar. All the programs before this chapter have put `package main` as the first line in the code. I'll talk more about what this means in a bit.

这个文件的第一行很熟悉。本章之前的所有程序都将 `package main` 作为代码的第一行。稍后我会详细解释它的含义。

Next is the import section. It imports three packages. The first is `fmt`, which is in the standard library. You've done this in previous chapters. The next two imports refer to the packages within the program. You must specify an *import path* when importing from anywhere besides the standard library. The import path consists of the module path followed by the path to the package within the module. For example, the import

```
"github.com/learning-go-book-2e/package_example/math"
```

has the module path `github.com/learning-go-book-2e/package_example`, while `/math` is the path to the package within the module.

接下来是导入部分。它导入了三个包。第一个是 `fmt`，它位于标准库中。您在之前的章节中已经导入过它。接下来的两个导入语句指向程序内部的包。从标准库以外的任何位置导入时，都必须指定导入路径。导入路径由模块路径和模块内包的路径组成。例如，导入语句

`"github.com/learning-go-book-2e/package_example/math"`
的模块路径是 `github.com/learning-go-book-`
`2e/package_example` , 而 `/math` 则是模块内包的路径。

It is a compile-time error to import a package but not use any of the identifiers exported by the package. This ensures that the binary produced by the Go compiler includes only code that's actually used in the program.

导入一个包但未使用该包导出的任何标识符，会导致编译时错误。这确保 Go 编译器生成的二进制文件仅包含程序实际使用的代码。

WARNING

警告

You might come across obsolete documentation on the web that mentions relative path import paths. They do not work with modules. (And were a bad idea, anyway.)

你可能会在网上看到一些过时的文档，其中提到了相对路径导入。它们不适用于模块。（而且无论如何，这本来就是个糟糕的设计。）

When you run this program, you'll see the following output:

运行此程序后，您将看到以下输出：

```
$ go build
```

```
$ ./package_example  
The number is 4
```

The `main` function called the `Double` function in the `math` package by prefixing the function name with the package name. You’ve seen this in previous chapters when calling functions in the standard library. You also call the `Number` function in the `format` package. You might wonder where this `format` package came from, since the import says `github.com/learning-go-book-2e/package_example/do-format`.

`main` 函数通过在函数名前加上包名，调用了 `math` 包中的 `Double` 函数。在之前的章节中，调用标准库函数时，你已经见过这种写法。你还会调用 `format` 包中的 `Number` 函数。你可能会好奇 `format` 包是从哪里来的，因为导入语句中写的是 `github.com/learning-go-book-2e/package_example/do-format`。

Every Go file in a directory must have an identical package clause. (You’ll see one tiny exception to this rule in [“Testing Your Public API”](#).) You imported the `format` package with the import path `github.com/learning-go-book-2e/package_example/do-format`. That’s because *the name of a package is determined by its package clause, not its import path*.

目录中的每个 Go 文件都必须具有相同的 `package` 子句。（在“测试你的公共 API”部分你会看到一个很小的例外。）你导入了 `format`

包，导入路径为 `github.com/learning-go-book-2e/package_example/do-format`。这是因为包的名称由其 `package` 子句决定，而不是由其导入路径决定。

As a general rule, you should make the name of the package match the name of the directory that contains the package. It is hard to discover a package's name if it does not match the containing directory. However, in a few situations you'll use a different name for the package than for the directory.

一般来说，软件包名称应该与其所在目录的名称保持一致。如果软件包名称与其所在目录的名称不一致，则很难找到该软件包。但是，在某些情况下，软件包名称和目录名称可能会不同。

The first is something you have been doing all along without realizing it. You declare a package to be a starting point for a Go application by using the special package name `main`. Since you cannot import the `main` package, this doesn't produce confusing import statements.

第一种方法你一直都在做，只是自己没有意识到。你可以使用特殊的包名 `main` 来声明一个包作为 Go 应用程序的起点。由于你无法导入 `main` 包，因此不会产生令人困惑的导入语句。

The other reasons for having a package name not match your directory name are less common. If your directory name contains a character that's not valid in a Go identifier, then you must choose a package name that's

different from your directory name. In your case, `do-format` is not a valid identifier name, so it's replaced with `format`. It's better to avoid this by never creating a directory with a name that's not a valid identifier.

包名与目录名不匹配的其他原因不太常见。如果目录名包含 Go 标识符中无效的字符，则必须选择一个与目录名不同的包名。在您的示例中，`do-format` 不是有效的标识符名称，因此会被替换为 `format`。为了避免这种情况，最好不要创建名称无效的目录。

The final reason for creating a directory whose name doesn't match the package name is to support versioning using directories. I'll talk about this more in [“Versioning Your Module”](#).

创建与包名不匹配的目录的最后一个原因是支持使用目录进行版本控制。我将在“模块版本控制”一节中详细讨论这一点。

As was discussed in [“Blocks”](#), package names in `import` statements are in the *file block*. If you use exported symbols from one package in two different files in another package, you must import the first package in both files in the second package.

如“代码块”部分所述，`import` 语句中的包名位于文件代码块中。如果您在一个包的两个不同文件中使用了来自另一个包的导出符号，则必须在第二个包的两个文件中都导入第一个包。

Naming Packages

命名软件包

Package names should be descriptive. Rather than have a package called `util`, create a package name that describes the functionality provided by the package. For example, say you have two helper functions: one to extract all names from a string and another to format names properly. Don't create two functions in a `util` package called `ExtractNames` and `FormatNames`. If you do, every time you use these functions, they will be referred to as `util.ExtractNames` and `util.FormatNames`, and that `util` package tells you nothing about what the functions do.

包名应该具有描述性。不要使用 `util` 这样的包名，而应该创建一个能够描述包所提供功能的名称。例如，假设你有两个辅助函数：一个用于从字符串中提取所有名称，另一个用于正确格式化名称。不要在 `util` 包中创建两个名为 `ExtractNames` 和 `FormatNames` 的函数。否则，每次使用这些函数时，它们都会被引用为 `util.ExtractNames` 和 `util.FormatNames`，而 `util` 包则无法告诉你这些函数的具体功能。

One option is to create one function called `Names` in a package called `extract` and a second function called `Names` in a package called `format`. It's OK for these two functions to have the same name, because they will always be disambiguated by their package names. The first will be

referred to as `extract.Names` when imported, and the second will be referred to as `format.Names`.

一种方法是在名为 `extract` 的包中创建一个名为 `Names` 的函数，并在名为 `format` 的包中创建第二个名为 `Names` 的函数。这两个函数可以同名，因为它们始终可以通过包名来区分。导入时，第一个函数将被引用为 `extract.Names`，第二个函数将被引用为 `format.Names`。

An even better option is to think about parts of speech. A function or method does something, so it should be a verb or action word. A package would be a noun, a name for the kind of item that is created or modified by the functions in the package. By following these rules, you would create a package called `names` with two functions, `Extract` and `Format`. The first would then be referred to as `names.Extract` and the second would be `names.Format`.

更好的方法是考虑词性。函数或方法执行某种操作，因此它应该是动词或表示动作的词。包则是一个名词，用来指代包内函数创建或修改的那类对象。遵循这些规则，您可以创建一个名为 `names` 的包，其中包含两个函数 `Extract` 和 `Format`。第一个函数将被命名为 `names.Extract`，第二个函数将被命名为 `names.Format`。

You should also avoid repeating the name of the package in the names of functions and types within the package. Don't name your function

`ExtractNames` when it is in the `names` package. The exception to this rule occurs when the name of the identifier is the same as the name of the package. For example, the package `sort` in the standard library has a function called `Sort`, and the `context` package defines the `Context` interface.

此外，您还应避免在包内的函数和类型名称中重复使用包名。例如，如果函数位于 `names` 包中，则不要将其命名为 `ExtractNames`。但如果标识符的名称与包名相同，则不受此规则限制。例如，标准库中的包 `sort` 有一个名为 `Sort` 的函数，而 `context` 包定义了 `Context` 接口。

Overriding a Package's Name

覆盖软件包名称

Sometimes you'll find yourself importing two packages whose names collide. For example, the standard library includes two packages for generating random numbers; one is cryptographically secure (`crypto/rand`), and the other is not (`math/rand`). The regular generator is fine when you aren't generating random numbers for encryption, but you need to seed it with an unpredictable value. A common pattern is to seed a regular random number generator with a value from a cryptographic generator. In Go, both packages have the same name (`rand`). When that happens, you provide an alternate name for one

package within the current file. Try out this code on [The Go Playground](#) or in the *sample_code/package_name_override* directory in the [Chapter 10 repository](#). First, look at the import section:

有时你会发现导入的两个包名称冲突。例如，标准库包含两个用于生成随机数的包；一个是加密安全的（`crypto/rand`），另一个则不是（`math/rand`）。如果你不需要生成用于加密的随机数，那么普通的随机数生成器就足够了，但你需要用一个不可预测的值来初始化它。一种常见的做法是用加密生成器的值来初始化普通的随机数生成器。在 Go 语言中，这两个包的名称相同（`rand`）。遇到这种情况，你需要在当前文件中为其中一个包指定一个别名。你可以在 Go Playground 或第 10 章代码库的 *sample_code/package_name_override* 目录中尝试这段代码。首先，请查看导入部分：

```
import (  
    crand "crypto/rand"  
    "encoding/binary"  
    "fmt"  
    "math/rand"  
)
```

You import `crypto/rand` with the name `crand`. This overrides the name `rand` that's declared within the package. You then import `math/rand` normally. When you look at the `seedRand` function, you

see that you access identifiers in `math/rand` with the `rand` prefix, and use the `crand` prefix with the `crypto/rand` package:

您导入 `crypto/rand` 时使用了名称 `crand`。这会覆盖包内声明的名称 `rand`。然后，您正常导入 `math/rand`。查看 `seedRand` 函数时，您会发现您使用 `rand` 前缀访问 `math/rand` 中的标识符，并使用 `crand` 前缀访问 `crypto/rand` 包：

```
func seedRand() *rand.Rand {  
    var b [8]byte  
    _, err := crand.Read(b[:])  
    if err != nil {  
        panic("cannot seed with cryptographic  
random number generator")  
    }  
    r :=  
    rand.New(rand.NewSource(int64(binary.LittleEndian.Uint64(b))))  
    return r  
}
```

NOTE

笔记

You can use two other symbols as a package name. The package name `.` (dot) places all the exported identifiers in the imported package into the current package's namespace; you don't need a prefix to refer to them. This is discouraged because it makes your source code less clear. You can no longer tell whether something is defined in the current package or whether it was imported by simply looking at its name.

您可以使用另外两个符号作为包名。包名 `.`（点）会将导入包中所有导出的标识符放入当前包的命名空间；您无需添加前缀即可引用它们。不建议这样做，因为它会降低源代码的清晰度。您将无法仅通过查看包名来判断某个内容是在当前包中定义的，还是从其他包中导入的。

You can also use `_` (underscore) as the package name. You'll explore what this does when I talk about `init` in [“Avoiding the init Function if Possible”](#).

您也可以使用 `_`（下划线）作为包名。在“尽可能避免使用 `init` 函数”一节中，我将讨论 `init`，届时您将了解它的作用。

As I discussed in [“Shadowing Variables”](#), package names can be shadowed. Declaring variables, types, or functions with the same name as a package makes the package inaccessible within the block with that declaration. If this is unavoidable (for example, a newly imported package has a name that conflicts with an existing identifier), override the package's name to resolve the conflict.

正如我在“变量遮蔽”一文中讨论的那样，包名可能会被遮蔽。如果声明的变量、类型或函数与某个包名相同，则在该声明所在的代码块

内，该包将无法访问。如果这种情况不可避免（例如，新导入的包名与现有标识符冲突），则需要覆盖包名以解决冲突。

Documenting Your Code with Go Doc Comments

使用 Go Doc 注释记录你的代码

An important part of creating a module for others to use is documenting it properly. Go has its own format for writing comments that are automatically converted into documentation. It's called *Go Doc* format, and it's very simple. Here are the rules:

为他人创建模块时，编写完善的文档至关重要。Go 语言有其独特的注释格式，这些注释会自动转换为文档。这种格式称为 Go Doc，非常简单。以下是其规则：

- Place the comment directly before the item being documented, with no blank lines between the comment and the declaration of the item.
将注释直接放在被注释项之前，注释和被注释项的声明之间不要有空行。
- Start each line of the comment with double slashes (`//`), followed by a space. While it's legal to use `/*` and `*/` to mark your comment block, it is idiomatic to use double slashes.
注释的每一行都以双斜杠 (`//`) 开头，后跟一个空格。虽然可以使用 `/*` 和 `*/` 来标记注释块，但使用双斜杠是更规范的做法。

- The first word in the comment for a symbol (a function, type, constant, variable, or method) should be the name of the symbol. You can also use “A” or “An” before the symbol name to help make the comment text grammatically correct.

符号（函数、类型、常量、变量或方法）注释中的第一个词应该是该符号的名称。您也可以在符号名称前使用“A”或“An”，以确保注释文本的语法正确。

- Use a blank comment line (double slashes and a newline) to break your comment into multiple paragraphs.

使用空白注释行（双斜杠加换行符）将注释分成多个段落。

As I’ll talk about in [“Using pkg.go.dev”](#), you can view public documentation online in HTML format. If you want to make your documents look a little snazzier, there are a couple of ways to format it:

正如我将在“使用 pkg.go.dev”一节中提到的，您可以在线查看 HTML 格式的公开文档。如果您想让文档看起来更美观，有几种方法可以对其进行格式化：

- If you want your comment to contain some preformatted content (such as a table or source code), put an additional space after the double slashes to indent the lines with the content.

如果您希望注释中包含一些预格式化的内容（例如表格或源代码），请在双斜杠后添加一个空格，以缩进包含内容的行。

- If you want a header in your comment, put a `#` and a space after the double slashes. Unlike with Markdown, you cannot use multiple `#` characters to make different levels of headers.

如果你想在评论中添加标题，请在双斜杠后输入 `#` 并加一个空格。与 Markdown 不同，你不能使用多个 `#` 字符来创建不同级别的标题。

- To make a link to another package (whether or not it is in the current module), put the package path within brackets (`[` and `]`).

要链接到另一个软件包（无论它是否在当前模块中），请将软件包路径放在方括号内（ `[` 和 `]` ）。

- To link to an exported symbol, place its name in brackets. If the symbol is in another package, use `[pkgName.SymbolName]` .

要链接到已导出的符号，请将其名称放在方括号中。如果该符号位于另一个包中，请使用 `[pkgName.SymbolName]` 。

- If you include a raw URL in your comment, it will be converted into a link.

如果你在评论中包含原始网址，它将被转换成链接。

- If you want to include text that links to a web page, put the text within brackets (`[` and `]`). At the end of the comment block, declare the mappings between your text and their URLs with the format `//`

`[TEXT]: URL` . You'll see a sample of this in a moment.

如果您想添加指向网页的链接文本，请将文本放在方括号内（例如 `[` 和 `]` ）。在注释块的末尾，使用 `// [TEXT]: URL` 的格式声明文本与其 URL 之间的映射关系。稍后您将看到示例。

Comments before the package declaration create package-level comments. If you have lengthy comments for the package (such as the extensive formatting documentation in the `fmt` package), the convention is to put the comments in a file in your package called *doc.go*.

在包声明之前的注释会创建包级别的注释。如果包的注释很长（例如 `fmt` 包中的大量格式说明文档），通常的做法是将这些注释放在包下名为 `doc.go` 的文件中。

Let's go through a well-commented file, starting with the package-level comment in [Example 10-1](#).

让我们来看一个注释完善的文件，从示例 10-1 中的包级注释开始。

Example 10-1. A package-level comment

示例 10-1. 包级注释

```
// Package convert provides various utilities to
// make it easy to convert money from one currency
// to another.
package convert
```

Next, place a comment on an exported struct (see [Example 10-2](#)). Notice that it starts with the name of the struct.

接下来，给导出的结构体添加注释（参见示例 10-2）。请注意，注释以结构体的名称开头。

Example 10-2. A struct comment

例 10-2. 结构体注释

```
// Money represents the combination of an amount
// of money
// and the currency the money is in.
//
// The value is stored using a
// [github.com/shopspring/decimal.Decimal]
type Money struct {
    Value    decimal.Decimal
    Currency string
}
```

Finally, there is a comment on a function (see [Example 10-3](#)).

最后，对一个函数进行了评论（参见示例 10-3）。

Example 10-3. A well-commented function

例 10-3. 一个注释详尽的函数

```
// Convert converts the value of one currency to
// another.
```

```
//  
// It has two parameters: a Money instance with  
// the value to convert,  
// and a string that represents the currency to  
// convert to. Convert returns  
// the converted currency and any errors  
// encountered from unknown or unconvertible  
// currencies.  
//  
// If an error is returned, the Money instance is  
// set to the zero value.  
//  
// Supported currencies are:  
//   - USD - US Dollar  
//   - CAD - Canadian Dollar  
//   - EUR - Euro  
//   - INR - Indian Rupee  
//  
// More information on exchange rates can be found  
// at [Investopedia].  
//  
// [Investopedia]:  
https://www.investopedia.com/terms/e/exchangerate.asp  
func Convert(from Money, to string) (Money, error)  
{  
    // ...  
}
```

Go includes a command-line tool called `go doc` that displays godocs. The command `go doc PACKAGE_NAME` displays the documentation for the specified package and a list of the identifiers in the package. Use `go doc PACKAGE_NAME.IDENTIFIER_NAME` to display the documentation for a specific identifier in the package.

Go 语言包含一个名为 `go doc` 的命令行工具，用于显示 godocs 文档。命令 `go doc PACKAGE_NAME` 会显示指定包的文档以及该包中所有标识符的列表。使用 `go doc PACKAGE_NAME.IDENTIFIER_NAME` 可以显示包中特定标识符对应的文档。

If you want to preview your documentation's HTML formatting before it is published on the web, use `pkgsite`. This is the same program that powers `pkg.go.dev` (which I will talk about later in the chapter). To install `pkgsite`, use this command:

如果您想在文档发布到网络之前预览其 HTML 格式，请使用 `pkgsite`。它与 `pkg.go.dev`（我将在本章后面部分详细介绍）使用相同的程序。要安装 `pkgsite`，请使用以下命令：

```
$ go install
golang.org/x/pkgsite/cmd/pkgsite@latest
```

(I will talk more about `go install` in [“Adding Third-Party Tools with `go install`”](#)).

(我将在“使用 `go install` 添加第三方工具”一文中详细讨论 `go install`)。

To view your source code’s comments rendered as HTML, go to the root of your module and run this:

要查看以 HTML 格式呈现的源代码注释，请转到模块根目录并运行以下命令：

```
$ pkgsite
```

Then go to <http://localhost:8080> to view your project and its source code.

然后访问 <http://localhost:8080> 查看您的项目及其源代码。

You can find even more details about comments and potential pitfalls by reading through the official [Go Doc Comments documentation](#).

您可以通过阅读官方的 Go Doc 注释文档，了解更多关于注释和潜在陷阱的详细信息。

TIP

提示

Make sure you comment your code properly. At the very least, any exported identifier should have a comment. In [“Using Code-Quality Scanners”](#), you will look at some third-party tools that can report missing comments on exported identifiers.

请务必正确注释代码。至少，任何导出的标识符都应该有注释。在“使用代码质量扫描器”一节中，您将了解一些可以报告导出标识符缺少注释的第三方工具。

Using the internal Package

使用内部包

Sometimes you want to share a function, type, or constant among packages in your module, but you don’t want to make it part of your API. Go supports this via the special `internal` package name.

有时，你可能希望在模块的不同包之间共享函数、类型或常量，但又不想将其作为 API 的一部分。Go 通过特殊的 `internal` 包名来支持这一点。

When you create a package called `internal`, the exported identifiers in that package and its subpackages are accessible only to the direct parent package of `internal` and the sibling packages of `internal`. Let’s look at an example to see how this works. You can find the code on [GitHub](#). The directory tree is shown in [Figure 10-1](#).

创建名为 `internal` 的包时，该包及其子包中导出的标识符仅对 `internal` 的直接父包和 `internal` 的同级包可见。我们来看一个示例来了解其工作原理。您可以在 GitHub 上找到代码。目录树如图 10-1 所示。

You've declared a simple function in the *internal.go* file in the `internal` package:

您在 `internal` 包的 *internal.go* 文件中声明了一个简单的函数：

```
func Doubler(a int) int {  
    return a * 2  
}
```

You can access this function from *foo.go* in the `foo` package and from *sibling.go* in the `sibling` package.

你可以从 `foo` 包中的 *foo.go* 以及 `sibling` 包中的 *sibling.go* 访问此函数。



Figure 10-1. The file tree for `internal_package_example`

图 10-1. `internal_package_example` 的文件树

Be aware that attempting to use the `internal` function from `bar.go` in the `bar` package or from `example.go` in the root package results in a compilation error:

请注意，尝试使用 `bar` 包中的 `bar.go` 文件或根包中的 `example.go` 文件内的内部函数会导致编译错误：

```
$ go build ./...
package github.com/learning-go-book-
```

```
2e/internal_example
example.go:3:8: use of internal package
github.com/learning-go-book-
2e/internal_example/foo/internal not allowed

package github.com/learning-go-book-
2e/internal_example/bar
bar/bar.go:3:8: use of internal package
github.com/learning-go-book-
2e/internal_example/foo/internal not allowed
```

Avoiding Circular Dependencies

避免循环依赖

Two of the goals of Go are a fast compiler and easy-to-understand source code. To support this, Go does not allow you to have a *circular dependency* between packages. If package A imports package B, directly or indirectly, package B cannot import package A, directly or indirectly.

Go 语言的两个目标是提供快速的编译器和易于理解的源代码。为了实现这些目标，Go 不允许包之间存在循环依赖。如果包 A 直接或间接地导入了包 B，那么包 B 就不能直接或间接地导入包 A。

Let's look at a quick example to explain the concept. You can find the code in the *sample_code/circular_dependency_example* directory in the [Chapter](#)

10 repository. There are two packages, `pet` and `person`. In `pet.go` in the `pet` package, you have this:

让我们来看一个简单的例子来解释这个概念。你可以在第 10 章代码库的 `sample_code/circular_dependency_example` 目录中找到代码。这里有两个包，`pet` 和 `person`。在 `pet` 包的 `pet.go` 文件中，你会看到以下代码：

```
import "github.com/learning-go-book-2e/ch10/sample_code/circular_dependency_example/person"

var owners = map[string]person.Person{
    "Bob":    {"Bob", 30, "Fluffy"},
    "Julia": {"Julia", 40, "Rex"},
}
```

While in `person.go` in the `person` package, you have this:

在 `person.go` 的 `person` 包中，您会看到以下内容：

```
import "github.com/learning-go-book-2e/ch10/sample_code/circular_dependency_example/pet"

var pets = map[string]pet.Pet{
```

```
    "Fluffy": {"Fluffy", "Cat", "Bob"},  
    "Rex":    {"Rex", "Dog", "Julia"},  
}
```

If you try to build this program, you'll get an error:

如果你尝试运行这个程序，你会收到一个错误提示：

```
$ go build  
./sample_code/circular_dependency_example  
package github.com/learning-go-book-  
2e/ch10/sample_code/  
    circular_dependency_example  
        imports github.com/learning-go-book-  
2e/ch10/sample_code/  
            circular_dependency_example/person  
                imports github.com/learning-go-book-  
2e/ch10/sample_code/  
                    circular_dependency_example/pet  
                        imports github.com/learning-go-book-  
2e/ch10/sample_code/  
                            circular_dependency_example/person: import  
cycle not allowed
```

If you find yourself with a circular dependency, you have a few options. In some cases, this is caused by splitting up packages too finely. If two packages depend on each other, there's a good chance they should be merged into a single package. You can merge the `person` and `pet` packages into a single package, and that solves your problem.

如果遇到循环依赖，您可以尝试以下几种方法。在某些情况下，循环依赖是由于包的拆分过于细小造成的。如果两个包相互依赖，它们很可能应该合并成一个包。您可以将 `person` 和 `pet` 这两个包合并成一个包，这样就能解决您的问题。

If you have a good reason to keep your packages separated, it may be possible to move just the items that cause the circular dependency to one of the two packages or to a new package.

如果您有充分的理由将软件包分开存放，则可以将导致循环依赖的项目移动到两个软件包中的一个或一个新软件包中。

Organizing Your Module

组织你的模块

There's no official way to structure the Go packages in your module, but several patterns have emerged over the years. They are guided by the

principle that you should focus on making your code easy to understand and maintain.

虽然没有官方的 Go 模块包结构规范，但多年来已经涌现出几种模式。这些模式都遵循一个原则：你应该专注于让代码易于理解和维护。

When your module is small, keep all your code in a single package. As long as no other modules depend on your module, there is no harm in delaying organization.

当你的模块很小时，将所有代码放在一个包里。只要没有其他模块依赖于你的模块，延迟组织代码并无坏处。

As your module grows, you'll want to impose some order to make your code more readable. The first question to ask is what type of module you are creating. You can group modules into two broad categories: those that are intended as a single application and those that are primarily intended as libraries. If you are sure that your module is intended to be used only as an application, make the root of the project the `main` package. The code in the `main` package should be minimal; place all your logic in an *internal* directory, and the code in the `main` function will simply invoke code within *internal*. This way, you can ensure that no one is going to create a module that depends on your application's implementation.

随着模块的增长，你需要对代码进行一些规范化处理，以提高代码的可读性。首先要问自己的是要创建的模块类型。模块可以大致分为两类：一类是作为独立应用程序使用的模块，另一类是主要作为库使用的模块。如果你确定你的模块仅作为应用程序使用，那么项目根目录应设为 `main` 包。`main` 包中的代码应该尽可能简洁；将所有逻辑放在一个内部目录中，`main` 函数中的代码只需调用内部目录中的代码即可。这样，就可以确保不会有人创建依赖于你应用程序实现的模块。

If you want your module to be used as a library, the root of your module should have a package name that matches the repository name. This makes sure that the import name matches the package name. To make this work, you must ensure that your repository name is a valid Go identifier. In particular, you cannot use a hyphen as a word separator in your repository name, because a hyphen is not a valid character in a package name.

如果您希望将模块用作库，则模块根目录的包名应与仓库名称一致。这样可以确保导入名称与包名匹配。为了实现这一点，您必须确保仓库名称是有效的 Go 标识符。尤其需要注意的是，仓库名称中不能使用连字符作为单词分隔符，因为连字符在包名称中不是有效字符。

It's not uncommon for library modules to have one or more applications included with them as utilities. In this case, create a directory called *cmd* at the root of your module. Within *cmd*, create one directory for each binary built from your module. For example, you might have a module that

contains both a web application and a command-line tool that analyzes data in the web application's database. Use `main` as the package name within each of these directories.

库模块通常会包含一个或多个实用程序应用程序。在这种情况下，请在模块根目录创建一个名为 `cmd` 的目录。在 `cmd` 目录下，为每个由该模块构建的二进制文件创建一个子目录。例如，您的模块可能包含一个 Web 应用程序和一个命令行工具，该命令行工具用于分析 Web 应用程序数据库中的数据。在每个子目录中，请使用 `main` 作为包名。

For more detailed information, this [blog post by Eli Bendersky](#) provides good advice on how you should structure a simple Go module.

如需了解更多详细信息，Eli Bendersky 的这篇博客文章提供了关于如何构建一个简单的 Go 模块的良好建议。

As projects get more complicated, you'll be tempted to break up your packages. Make sure to organize your code to limit the dependencies among packages. One common pattern is to organize your code by slices of functionality. For example, if you wrote a shopping site in Go, you might place all the code for customer management in one package and all the code for inventory management in another. This style limits the dependencies among packages, which makes it easier to later refactor a single web application into multiple microservices. This style is in contrast to the way that many Java applications are organized, with all the business logic in one

package, all the database logic in another package, and the data transfer objects in a third.

随着项目变得越来越复杂，你可能会想要拆分代码包。务必组织好代码，以减少包之间的依赖关系。一种常见的模式是按功能切片来组织代码。例如，如果你用 Go 编写了一个购物网站，你可以将所有客户管理代码放在一个包中，将所有库存管理代码放在另一个包中。这种方式可以减少包之间的依赖关系，从而更容易将单个 Web 应用程序重构为多个微服务。这种风格与许多 Java 应用程序的组织方式截然不同，后者通常会将所有业务逻辑放在一个包中，将所有数据库逻辑放在另一个包中，并将数据传输对象放在第三个包中。

When developing a library, take advantage of the `internal` package. If you create multiple packages within a module and they are outside an `internal` package, exporting a symbol so it can be used by another package in your module means it can also be used by *anyone* who imports your module. There's a principle in software engineering called [Hyrum's law](#): “With a sufficient number of users of an API, it does not matter what you promise in the contract: all observable behaviors of your system will be depended on by somebody.” Once something is part of your API, you have a responsibility to continue supporting it until you decide to make a new version that's not backward compatible. You will learn how to do this in [“Updating to Incompatible Versions”](#). If you have some symbols that you want to share only within your module, put them in `internal`. If you

change your mind, you can always move the package out of `internal` later.

开发库时，请充分利用 `internal` 包。如果您在一个模块中创建了多个包，并且它们位于 `internal` 包之外，那么导出符号以供模块中其他包使用，意味着任何导入您模块的人都可以使用该符号。软件工程中有一条名为海勒姆定律的原则：“当 API 的用户数量足够多时，您在合同中做出的任何承诺都无关紧要：系统的所有可观察行为都将受到某些人的依赖。”一旦某个功能成为 API 的一部分，您就有责任继续支持它，直到您决定发布一个不向后兼容的新版本。您将在“更新到不兼容版本”中学习如何做到这一点。如果您有一些只想在模块内部共享的符号，请将它们放在 `internal` 中。如果您改变主意，可以随时将该包从 `internal` 中移出。

For a good overview of Go project structure advice, watch Kat Zien’s talk from GopherCon 2018, [“How Do You Structure Your Go Apps”](#).

想要全面了解 Go 项目结构建议，请观看 Kat Zien 在 2018 年 GopherCon 大会上的演讲“如何构建你的 Go 应用”。

WARNING

警告

The “golang-standards” GitHub repository claims to be the “standard” module layout. Russ Cox, the development lead for Go, has [publicly stated](#) that it is not endorsed by the Go team and that the structure it recommends is in fact an antipattern. Please do not cite this repository as a way to organize your code.

“golang-standards”GitHub 仓库声称是“标准”模块布局。Go 语言的开发负责人 Russ Cox 已公开表示，Go 团队并未认可该仓库，并且其推荐的结构实际上是一种反模式。请不要引用该仓库作为组织代码的方式。

Gracefully Renaming and Reorganizing Your API

优雅地重命名和重组您的 API

After using a module for a while, you might realize that its API is not ideal. You might want to rename some of the exported identifiers or move them to another package within your module. To avoid a backward-breaking change, don’t remove the original identifiers; provide an alternate name instead.

使用某个模块一段时间后，您可能会发现它的 API 并不理想。您可能希望重命名一些导出的标识符，或者将它们移动到模块内的另一个包中。为了避免造成向后兼容性问题，请不要删除原始标识符，而是提供一个替代名称。

This is easy with a function or method. You declare a function or method that calls the original. For a constant, simply declare a new constant with the same type and value, but a different name.

使用函数或方法很容易做到这一点。你只需声明一个调用原函数或方法的函数或方法即可。对于常量，只需声明一个具有相同类型和值但名称不同的新常量即可。

If you want to rename or move an exported type, you use an alias. Quite simply, an *alias* is a new name for a type. You saw in [Chapter 7](#) how to use the `type` keyword to declare a new type based on an existing one. You also use the `type` keyword to declare an alias. Let's say you have a type called `Foo`:

如果要重命名或移动已导出的类型，可以使用别名。简而言之，别名就是类型的新名称。您在第 7 章中已经了解了如何使用 `type` 关键字基于现有类型声明新类型。您还可以使用 `type` 关键字声明别名。假设您有一个名为 `Foo` 的类型：

```
type Foo struct {  
    x int  
    S string  
}  
  
func (f Foo) Hello() string {  
    return "hello"  
}
```

```
}  
  
func (f Foo) goodbye() string {  
    return "goodbye"  
}
```

If you want to allow users to access `Foo` by the name `Bar`, all you need to do is this:

如果您想允许用户通过名称 `Bar` 访问 `Foo`，您只需执行以下操作：

```
type Bar = Foo
```

To create an alias, use the `type` keyword, the name of the alias, an equals sign, and the name of the original type. The alias has the same fields and methods as the original type.

要创建别名，请使用 `type` 关键字、别名名称、等号和原始类型名称。别名与原始类型具有相同的字段和方法。

The alias can even be assigned to a variable of the original type without a type conversion:

甚至可以将别名分配给原始类型的变量，而无需进行类型转换：

```
func MakeBar() Bar {  
    bar := Bar{  
        x: 20,  
        S: "Hello",  
    }  
    var f Foo = bar  
    fmt.Println(f.Hello())  
    return bar  
}
```

One important point to remember: an alias is just another name for a type. If you want to add new methods or change the fields in an aliased struct, you must add them to the original type.

需要记住的一点是：别名只是类型的另一个名称。如果要向别名结构体添加新方法或更改字段，必须将其添加到原始类型中。

You can alias a type that's defined in the same package as the original type or in a different package. You can even alias a type from another module. There is one drawback to an alias in another package: you cannot use an alias to refer to the unexported methods and fields of the original type. This limitation makes sense, as aliases exist to allow a gradual change to a package's API, and the API consists only of the exported parts of the package. To work around this limitation, call code in the type's original package to manipulate unexported fields and methods.

你可以为定义在与原始类型相同的包中或不同包中的类型创建别名。你甚至可以为来自其他模块的类型创建别名。在其他包中创建别名有一个缺点：你不能使用别名来引用原始类型的未导出方法和字段。这种限制是合理的，因为别名的存在是为了允许包的 API 逐步更改，而 API 仅包含包的导出部分。为了绕过这个限制，你需要调用类型原始包中的代码来操作未导出的字段和方法。

Two kinds of exported identifiers can't have alternate names. The first is a package-level variable. The second is a field in a struct. Once you choose a name for an exported struct field, there's no way to create an alternate name.

有两种导出标识符不能有别名。第一种是包级变量。第二种是结构体中的字段。一旦为导出的结构体字段选择了名称，就无法创建别名。

Avoiding the init Function if Possible

尽可能避免使用初始化函数

When you read Go code, it is usually clear which methods and functions are called. One of the reasons Go doesn't have method overriding or function overloading is to make it easier to understand what code is running.

However, there is a way to set up state in a package without explicitly calling anything: the `init` function. When you declare a function named `init` that takes no parameters and returns no values, it runs the first time

the package is referenced by another package. Since `init` functions do not have any inputs or outputs, they can work only by side effect, interacting with package-level functions and variables.

阅读 Go 代码时，通常可以清楚地看到哪些方法和函数被调用。Go 没有方法重写或函数重载的原因之一就是为了让代码更容易理解。然而，有一种方法可以在不显式调用任何函数的情况下设置包的状态：`init` 函数。当你声明一个名为 `init` 的函数，它不接受任何参数也不返回任何值时，它会在包首次被其他包引用时运行。由于 `init` 函数没有任何输入或输出，它们只能通过副作用来工作，与包级别的函数和变量进行交互。

The `init` function has another unique feature. Go allows you to declare multiple `init` functions in a single package, or even in a single file in a package. There's a documented order for running multiple `init` functions in a single package, but rather than remembering it, it's better to simply avoid them.

`init` 函数还有另一个独特的特性。Go 允许你在单个包中，甚至在包中的单个文件中声明多个 `init` 函数。虽然在单个包中运行多个 `init` 函数有一个文档说明的顺序，但与其记住它，不如直接避免这样做。

Some packages, like database drivers, use `init` functions to register the database driver. However, you don't use any of the identifiers in the

package. As mentioned earlier, Go doesn't allow you to have unused imports. To work around this, Go allows *blank imports*, where the name assigned to an import is the underscore (`_`). Just as an underscore allows you to skip an unused return value from a function, a blank import triggers the `init` function in a package but doesn't give you access to any of the exported identifiers in the package:

有些包，例如数据库驱动程序，会使用 `init` 函数来注册数据库驱动程序。但是，您并没有使用包中的任何标识符。如前所述，Go 不允许导入未使用的包。为了解决这个问题，Go 允许空导入，导入的名称为下划线 (`_`)。就像下划线允许您跳过函数中未使用的返回值一样，空导入会触发包中的 `init` 函数，但不会让您访问包中任何已导出的标识符。

```
import (  
    "database/sql"  
    _ "github.com/lib/pq"  
)
```

This pattern is considered obsolete because it's unclear that a registration operation is being performed. Go's compatibility guarantee for its standard library means that you are stuck using it to register database drivers and image formats, but if you have a registry pattern in your own code, register your plug-ins explicitly.

这种模式被认为已经过时，因为它无法明确表明是否正在执行注册操作。Go 对其标准库的兼容性保证意味着您只能使用它来注册数据库驱动程序和图像格式，但如果您自己的代码中存在注册模式，则需要显式地注册您的插件。

The primary use of `init` functions today is to initialize package-level variables that can't be configured in a single assignment. It's a bad idea to have mutable state at the top level of a package, since it makes it harder to understand how data flows through your application. That means that any package-level variables configured via `init` should be *effectively immutable*. While Go doesn't provide a way to enforce that their value does not change, you should make sure that your code does not change them. If you have package-level variables that need to be modified while your program is running, see if you can refactor your code to put that state into a struct that's initialized and returned by a function in the package.

`init` 函数目前的主要用途是初始化无法通过单个赋值操作配置的包级变量。在包的顶层拥有可变状态是一个糟糕的做法，因为它会使应用程序的数据流变得更加难以理解。这意味着任何通过 `init` 配置的包级变量都应该是实际不可变的。虽然 Go 没有提供强制其值不改变的方法，但您应该确保您的代码不会修改它们。如果您需要在程序运行时修改包级变量，请尝试重构代码，将该状态放入一个结构体中，并由包内的函数初始化和返回该结构体。

The nonexplicit invocation of `init` functions means that you should document their behavior. For example, a package with an `init` function that loads files or accesses the network should call this out in a package-level comment so that security-conscious users of your code aren't surprised by unexpected I/O.

非显式调用 `init` 函数意味着您应该记录它们的行为。例如，如果一个包中包含一个用于加载文件或访问网络的 `init` 函数，则应在包级注释中说明这一点，以免注重安全的用户因意外的 I/O 操作而感到意外。

Working with Modules

使用模块

You've seen how to work with packages within a single module, and now it's time to see how to integrate with third-party modules and the packages within them. After that, you'll learn about publishing and versioning your own modules and Go's centralized services: `pkg.go.dev`, the module proxy, and the checksum database.

您已经了解了如何在单个模块中使用包，现在是时候学习如何集成第三方模块及其中的包了。之后，您将学习如何发布和管理您自己的模

块版本，以及 Go 的集中式服务：pkg.go.dev、模块代理和校验和数据库。

Importing Third-Party Code

导入第三方代码

So far, you've imported packages from the standard library like `fmt`, `errors`, `os`, and `math`. Go uses the same import system to integrate packages from third parties. Unlike many other compiled languages, Go always builds applications from source code into a single binary file. This includes the source code of your module and the source code of all the modules on which your module depends. (The Go compiler is smart enough to not include unreferenced packages in the binary it produces.) Just as you saw when you imported a package from within your own module, when you import a third-party package, you specify the location in the source code repository where the package is located.

到目前为止，您已经从标准库导入了 `fmt`、`errors`、`os` 和 `math` 等包。Go 使用相同的导入系统来集成第三方包。与其他许多编译型语言不同，Go 始终将应用程序从源代码构建成一个单独的二进制文件。这包括您的模块的源代码以及您的模块所依赖的所有模块的源代码。

（Go 编译器足够智能，不会将未引用的包包含在其生成的二进制文件中。）正如您在自己的模块中导入包时所看到的，当您导入第三方包时，您需要指定该包在源代码仓库中的位置。

Let's look at an example. I mentioned back in [Chapter 2](#) that you should never use floating-point numbers when you need an exact representation of a decimal number. If you do need an exact representation, one good option is the `decimal` module from [ShopSpring](#). You are also going to look at a simple [formatting module](#) that I've written for this book. Both of these modules are used in a small program in the [money_repository](#) for the book. This program calculates the price of an item with the tax included and prints the output in a neat format.

我们来看一个例子。我在第二章提到过，当你需要精确表示十进制数时，千万不要使用浮点数。如果你确实需要精确表示，ShopSpring 的 `decimal` 模块是一个不错的选择。你还会看到我为本书编写的一个简单的格式化模块。这两个模块都用在本书货币库中的一个小程序里。这个程序会计算含税商品的价格，并以简洁的格式输出结果。

The following code is in *main.go*:

以下代码位于 `main.go` 文件中：

```
package main

import (
    "fmt"
    "log"
    "os"
```

```

    "github.com/learning-go-book-2e/formatter"
    "github.com/shopspring/decimal"
)

func main() {
    if len(os.Args) < 3 {
        fmt.Println("Need two parameters: amount
and percent")
        os.Exit(1)
    }
    amount, err :=
decimal.NewFromString(os.Args[1])
    if err != nil {
        log.Fatal(err)
    }
    percent, err :=
decimal.NewFromString(os.Args[2])
    if err != nil {
        log.Fatal(err)
    }
    percent = percent.Div(decimal.NewFromInt(100))
    total :=
amount.Add(amount.Mul(percent)).Round(2)
    fmt.Println(formatter.Space(80, os.Args[1],
os.Args[2],

```

```
total.StringFixed(2)))  
}
```

The two imports `github.com/learning-go-book-2e/formatter` and `github.com/shopspring/decimal` specify third-party imports. Note that they include the location of the package in the repository. Once they're imported, you access the exported items in these packages just like any other imported package.

导入语句 `github.com/learning-go-book-2e/formatter` 和 `github.com/shopspring/decimal` 指定了第三方包的导入。请注意，它们包含了包在代码仓库中的位置。导入完成后，您可以像访问其他任何已导入的包一样访问这些包中导出的内容。

Before building the application, look at the *go.mod* file. Its contents should be as follows:

在构建应用程序之前，请查看 `go.mod` 文件。其内容应如下所示：

```
module github.com/learning-go-book-2e/money  
  
go 1.20
```

If you try to do a build, you get the following message:

如果尝试进行构建，则会收到以下消息：

```
$ go build
main.go:8:2: no required module provides package
    github.com/learning-go-book-2e/formatter; to
add it:
    go get github.com/learning-go-book-
2e/formatter
main.go:9:2: no required module provides package
    github.com/shopspring/decimal; to add it:
    go get github.com/shopspring/decimal
```

As the errors indicate, you cannot build the program until you add references to the third-party modules to your *go.mod* file. The `go get` command downloads modules and updates the *go.mod* file. You have two options when using `go get`. The simplest option is to tell `go get` to scan your module's source code and add any modules that are found in `import` statements to *go.mod*:

正如错误提示所示，您必须将第三方模块的引用添加到 *go.mod* 文件中才能构建程序。`go get` 命令用于下载模块并更新 *go.mod* 文件。使用 `go get` 时，您有两种选择。最简单的选择是让 `go get` 扫描模块的源代码，并将 `import` 语句中找到的任何模块添加到 *go.mod* 文件中：


```
$ go get ./...
go: downloading github.com/shopspring/decimal
v1.3.1
go: downloading github.com/learning-go-book-
2e/formatter
v0.0.0-20220918024742-1835a89362c9
go: downloading github.com/fatih/color v1.13.0
go: downloading github.com/mattn/go-colorable
v0.1.9
go: downloading github.com/mattn/go-isatty v0.0.14
go: downloading golang.org/x/sys v0.0.0-
20210630005230-0f9fa26af87c
go: added github.com/fatih/color v1.13.0
go: added github.com/learning-go-book-2e/formatter
v0.0.0-20220918024742-1835a89362c9
go: added github.com/mattn/go-colorable v0.1.9
go: added github.com/mattn/go-isatty v0.0.14
go: added github.com/shopspring/decimal v1.3.1
go: added golang.org/x/sys v0.0.0-20210630005230-
0f9fa26af87c
```

Because the location of the package is in the source code, `go get` is able to get the package's module and download it. If you look in the *go.mod* file now, you'll see this:

由于软件包的位置信息位于源代码中，因此 `go get` 可以获取该软件包的模块并将其下载。如果您现在查看 `go.mod` 文件，您会看到以下内容：

```
module github.com/learning-go-book-2e/money

go 1.20

require (
    github.com/learning-go-book-2e/formatter
    v0.0.0-20220918024742-1835a89362c9
    github.com/shopspring/decimal v1.3.1
)

require (
    github.com/fatih/color v1.13.0 // indirect
    github.com/mattn/go-colorable v0.1.9 //
indirect
    github.com/mattn/go-isatty v0.0.14 // indirect
    golang.org/x/sys v0.0.0-20210630005230-
0f9fa26af87c // indirect
)
```

The first `require` section of the `go.mod` file lists the modules that you’ve imported into your module. After the module name is a version number. In the case of the `formatter` module, it doesn’t have a version tag, so Go makes up a *pseudoversion*.

`go.mod` 文件的第一个 `require` 部分列出了您已导入到模块中的模块。模块名称后面是版本号。对于 `formatter` 模块，它没有版本标签，因此 Go 会生成一个伪版本号。

You also see a second `require` directive section that has modules with an *indirect* comment. One of these modules (`github.com/fatih/color`) is directly used by `formatter`. It, in turn, depends on the other three modules in the second `require` directive section. All the modules used by all your module's dependencies (and your dependencies' dependencies, and so on) are included in the *go.mod* file for your module. The ones that are used only in dependencies are marked as indirect.

您还会看到第二个 `require` 指令部分，其中包含带有间接注释的模块。其中一个模块 (`github.com/fatih/color`) 被 `formatter` 直接使用。而 `formatter` 又依赖于第二个 `require` 指令部分中的其他三个模块。您的模块的所有依赖项（以及依赖项的依赖项，依此类推）所使用的所有模块都包含在您模块的 `go.mod` 文件中。仅在依赖项中使用的模块会被标记为间接依赖。

In addition to updating *go.mod*, a *go.sum* file is created. For each module in the dependency tree of your project, the *go.sum* file has one or two entries: one with the module, its version, and a hash of the module; the other with the hash of the *go.mod* file for the module. Here's what the *go.sum* file looks like:

除了更新 go.mod 文件之外，还会创建一个 go.sum 文件。对于项目依赖树中的每个模块，go.sum 文件都包含一个或两个条目：一个条目包含模块名称、版本号和模块的哈希值；另一个条目包含该模块对应的 go.mod 文件的哈希值。以下是 go.sum 文件的示例：

```
github.com/fatih/color v1.13.0
h1:8LOYc1KYPPmyKMun8QV2DNRWNbLo6LZ0iLs...
github.com/fatih/color v1.13.0/go.mod
h1:kLAiJbzzSOZDVNGyDpe0xJ47H46q...
github.com/learning-go-book-2e/formatter v0.0.0-
20220918024742-1835a8...
github.com/learning-go-book-2e/formatter v0.0.0-
20220918024742-1835a8...
github.com/mattn/go-colorable v0.1.9
h1:sqDoxXbdeALODt0DAeJCVp38ps9Zo...
github.com/mattn/go-colorable v0.1.9/go.mod
h1:u6P/XSegPjTcexA+o6vUJr...
github.com/mattn/go-isatty v0.0.12/go.mod
h1:cbi80IDigv2wuxKPP5v1RcQ1...
github.com/mattn/go-isatty v0.0.14
h1:yVuAays6BHfxijgZPzw+3Zlu5yQgKGP...
github.com/mattn/go-isatty v0.0.14/go.mod
h1:7GGIvUiUoEMVVmxf/4nioHXj...
github.com/shopspring/decimal v1.3.1
h1:2Usl1nmF/WZucqkFZhnfFYxxxu8LG...
github.com/shopspring/decimal v1.3.1/go.mod
h1:DKyhrW/HYNuLGql+MJL6WC...
```

```
golang.org/x/sys v0.0.0-20200116001909-  
b77594299b42/go.mod h1:h1NjWce...  
golang.org/x/sys v0.0.0-20200223170610-  
d5e6a3e2c0ae/go.mod h1:h1NjWce...  
golang.org/x/sys v0.0.0-20210630005230-  
0f9fa26af87c h1:F1jZWGFhYfh0Ci...  
golang.org/x/sys v0.0.0-20210630005230-  
0f9fa26af87c/go.mod h1:oPkhp1M...
```

You'll see what these hashes are used for in [“Module Proxy Servers”](#). You might also notice that there are multiple versions of some of the dependencies. I'll talk about that in [“Minimal Version Selection”](#).

您将在“模块代理服务器”部分了解这些哈希值的用途。您可能还会注意到某些依赖项存在多个版本。我将在“最小版本选择”部分讨论这一点。

Let's validate that your modules are now set up correctly. Run `go build` again, and then run the `money` binary and pass it some arguments:

让我们验证一下您的模块是否已正确设置。再次运行 `go build`，然后运行 `money` 二进制文件并向其传递一些参数：

```
$ go build  
$ ./money 99.99 7.25
```

99.99	7.25
107.24	

NOTE

笔记

This sample program was checked in without *go.sum* and with an incomplete *go.mod*. This was done so you could see what happens when these files are populated. When committing your own modules to source control, always include up-to-date *go.mod* and *go.sum* files. Doing so specifies exactly what versions of your dependencies are being used. This enables *repeatable builds*; when anyone else (including your future self) builds this module, they will get the exact same binary.

这个示例程序提交时缺少 *go.sum* 文件，并且 *go.mod* 文件也不完整。这样做是为了方便您了解填充这些文件后会发生什么。将您自己的模块提交到源代码控制系统时，请务必包含最新的 *go.mod* 和 *go.sum* 文件。这样做可以精确指定所使用的依赖项版本。这可以确保构建的可重复性；当其他人（包括未来的您自己）构建此模块时，他们将获得完全相同的二进制文件。

As I mentioned, there's another way to use `go get`. Instead of telling it to scan your source code to discover modules, you can pass the module paths to `go get`. To see this work, roll back the changes to your *go.mod* file and remove the *go.sum* file. On Unix-like systems, the following commands will do this:

正如我之前提到的，还有另一种使用 `go get` 的方法。除了让它扫描源代码来发现模块之外，你还可以将模块路径传递给 `go get`。要查

看此方法的效果，请回滚对 `go.mod` 文件的更改并删除 `go.sum` 文件。
在类 Unix 系统中，可以使用以下命令：

```
$ git restore go.mod  
$ rm go.sum
```

Now, pass the module paths to `go get` directly:

现在，直接将模块路径传递给 `go get`：

```
$ go get github.com/learning-go-book-  
2e/formatter  
go: added github.com/learning-go-book-2e/  
formatter v0.0.0-20200921021027-5abc380940ae  
$ go get github.com/shopspring/decimal  
go: added github.com/shopspring/decimal v1.3.1
```

NOTE

笔记

Sharp-eyed readers might have noticed that when we used `go get` a second time, the `go: downloading` messages weren't displayed. The reason is that Go maintains a *module cache* on your local computer. Once a version of a module is downloaded, a copy is kept in the cache. Source code is pretty compact, and drives are pretty large, so this isn't usually a concern. However, if you want to delete the module cache, use the command `go clean -modcache`.

眼尖的读者可能已经注意到，当我们第二次使用 `go get` 时，`go: downloading` 条消息没有显示出来。这是因为 Go 会在本地计算机上维护一个模块缓存。一旦下载了某个模块的版本，缓存中就会保留一份副本。源代码通常很紧凑，而硬盘空间又很大，所以这通常不会造成问题。但是，如果您想删除模块缓存，请使用命令 `go clean -modcache`。

Take a look at the contents of *go.mod*, and they'll look a little different than before:

查看 *go.mod* 的内容，你会发现它和以前略有不同：

```
module github.com/learning-go-book-2e/money

go 1.20

require (
    github.com/fatih/color v1.13.0 // indirect
    github.com/learning-go-book-2e/
    formatter v0.0.0-20220918024742-1835a89362c9
    // indirect
    github.com/mattn/go-colorable v0.1.9 //
```



```
indirect
    github.com/mattn/go-isatty v0.0.14 // indirect
    github.com/shopspring/decimal v1.3.1 //
indirect
    golang.org/x/sys v0.0.0-20210630005230-
    0f9fa26af87c // indirect
)
```

Notice that all the imports are marked as *indirect*, not just the ones that came from `formatter`. When you run `go get` and pass it a module name, it doesn't check your source code to see whether the module you specified is used within your main module. Just to be safe, it adds an indirect comment.

请注意，所有导入都被标记为间接导入，而不仅仅是来自 `formatter` 的导入。当您运行 `go get` 并传入模块名称时，它不会检查您的源代码来确认您指定的模块是否在您的主模块中使用。为了安全起见，它会添加一个间接导入注释。

If you want to fix this label automatically, use the command `go mod tidy`. It scans your source code and synchronizes the `go.mod` and `go.sum` files with your module's source code, adding and removing module references. It also makes sure that the indirect comments are correct.

如果您想自动修复此标签，请使用命令 `go mod tidy`。它会扫描您的源代码，并将 `go.mod` 和 `go.sum` 文件与模块的源代码同步，添加和

删除模块引用。它还会确保间接注释正确无误。

You might be wondering why you would want to bother using `go get` with a module name. The reason is that it allows you to update the version of an individual module.

您可能想知道为什么要在模块名称后使用 `go get`。原因在于，它允许您更新单个模块的版本。

Working with Versions

使用版本

Let's see how Go's module system uses versions. I've written a [simple module](#) that you're going to use in another [tax-collection program](#). In *main.go*, there are the following third-party imports:

我们来看看 Go 的模块系统是如何使用版本的。我编写了一个简单的模块，你将在另一个税务征收程序中使用它。在 *main.go* 文件中，有以下第三方导入：

```
"github.com/learning-go-book-2e/simpletax"  
"github.com/shopspring/decimal"
```

As before, the sample program wasn't checked in with *go.mod*, and *go.sum* updated, so you could see what happens. When the program is built, you see the following:

和之前一样，示例程序没有提交到 *go.mod* 文件中，*go.sum* 文件也已更新，因此可以看到运行结果。程序编译完成后，你会看到以下内容：

```
$ go get ./...
go: downloading github.com/learning-go-book-2e/simpletax v1.1.0
go: added github.com/learning-go-book-2e/simpletax v1.1.0
go: added github.com/shopspring/decimal v1.3.1
$ go build
```

The *go.mod* file has been updated:

go.mod 文件已更新：

```
module github.com/learning-go-book-2e/region_tax

go 1.20

require (
```

```
github.com/learning-go-book-2e/simpletax  
v1.1.0  
github.com/shopspring/decimal v1.3.1  
)
```

There is also a *go.sum* file with hashes for your dependencies. Run the code and see whether it's working:

还有一个 *go.sum* 文件，其中包含依赖项的哈希值。运行代码，看看是否有效：

```
$ ./region_tax 99.99 12345  
2022/09/19 22:04:38 unknown zip: 12345
```

That looks like an unexpected answer. The latest version of the module might have a bug. By default, Go picks the latest version of a dependency when you add it to your module. However, one of the things that makes versioning useful is that you can specify an earlier version of a module. First, you can see what versions of the module are available with the `go list` command:

这看起来是个意料之外的答案。模块的最新版本可能存在 bug。默认情况下，Go 会在你将依赖项添加到模块时选择其最新版本。然而，版

本控制的优势之一在于你可以指定模块的早期版本。首先，你可以使用 `go list` 命令查看模块的可用版本：

```
$ go list -m -versions github.com/learning-go-book-2e/simpletax
github.com/learning-go-book-2e/simpletax v1.0.0
v1.1.0
```

By default, the `go list` command lists the packages that are used in your module. The `-m` flag changes the output to list the modules instead, and the `-versions` flag changes `go list` to report on the available versions for the specified module. In this case, you see that there are two versions, v1.0.0 and v1.1.0. Let's downgrade to version v1.0.0 and see if that fixes your problem. You do that with the `go get` command:

默认情况下，`go list` 命令会列出模块中使用的软件包。`-m` 标志会将输出更改为列出模块，而 `-versions` 标志会将 `go list` 更改为报告指定模块的可用版本。在本例中，您可以看到有两个版本：v1.0.0 和 v1.1.0。让我们降级到 v1.0.0 版本，看看是否能解决您的问题。您可以使用 `go get` 命令来完成此操作：

```
$ go get github.com/learning-go-book-2e/simpletax@v1.0.0
```

```
go: downloading github.com/learning-go-book-2e/simpletax v1.0.0
go: downgraded github.com/learning-go-book-2e/simpletax v1.1.0 => v1.0.0
```

The `go get` command lets you work with modules, updating the versions of your dependencies.

`go get` 命令允许您使用模块，更新依赖项的版本。

Now if you look at *go.mod*, you'll see that the version has been changed:

现在如果你查看 *go.mod* 文件，你会发现版本号已经更改了：

```
module github.com/learning-go-book-2e/region_tax

go 1.20

require (
    github.com/learning-go-book-2e/simpletax
    v1.0.0
    github.com/shopspring/decimal v1.3.1
)
```

You also see in *go.sum* that it contains both versions of `simpletax`:

您还可以在 `go.sum` 中看到，它包含了 `simpletax` 的两个版本：

```
github.com/learning-go-book-2e/simpletax v1.0.0
h1:KZU8aXRCHkvgFmBWkV...
github.com/learning-go-book-2e/simpletax
v1.0.0/go.mod h1:lR4YYZwbDTI...
github.com/learning-go-book-2e/simpletax v1.1.0
h1:sG83gscauX/b8yKKY9...
github.com/learning-go-book-2e/simpletax
v1.1.0/go.mod h1:lR4YYZwbDTI...
```

This is fine; if you change a module's version or even remove a module from your module, an entry for it might still remain in *go.sum*. This doesn't cause problems.

这没问题；即使你更改了某个模块的版本，甚至从你的模块中删除了某个模块，它仍然可能保留在 `go.sum` 文件中。这不会造成任何问题。

When you build and run the code again, the bug is fixed:

当你再次编译并运行代码时，该错误就被修复了：

```
$ go build
$ ./region_tax 99.99 12345
107.99
```

SEMANTIC VERSIONING

语义版本控制

Software has had version numbers from time immemorial, but there has been little consistency in what version numbers mean. The version numbers attached to Go modules follow the rules of *semantic versioning*, also known as *SemVer*. By requiring semantic versioning for modules, Go makes its module management code simpler while ensuring that users of a module understand what a new release promises.

软件版本号由来已久，但版本号的含义却一直缺乏统一的标准。Go 模块的版本号遵循语义化版本控制（SemVer）的规则。通过要求模块采用语义化版本控制，Go 简化了模块管理代码，同时确保模块用户能够理解新版本带来的功能变化。

If you aren't familiar with SemVer, check out the [full specification](#). The very short explanation is that semantic versioning divides a version number into three parts: the *major* version, the *minor* version, and the *patch* version, which are written as `major.minor.patch` and preceded by a `v`. The patch version number is incremented when fixing a bug; the minor version number is incremented (and the patch version is set back to 0) when a new, backward-compatible feature is added; and the major version number is incremented (and minor and patch are set back to 0) when making a change that breaks backward compatibility.

如果您不熟悉语义化版本控制 (SemVer) , 请查看完整的规范。简而言之, 语义化版本控制将版本号分为三部分: 主版本号、次版本号和补丁版本号, 它们分别以 `major.minor.patch` 的形式表示, 并在前面加上 `v`。修复错误时, 补丁版本号递增; 添加新的、向后兼容的功能时, 次版本号递增 (补丁版本号重置为 0) ; 进行破坏向后兼容性的更改时, 主版本号递增 (次版本号和补丁版本号重置为 0) 。

Minimal Version Selection

最小版本选择

At some point, your module will depend on two or more modules that all depend on the same module. As often happens, these modules declare that they depend on different minor or patch versions of that module. How does Go resolve this?

在某些情况下, 你的模块会依赖于两个或多个模块, 而这些模块又都依赖于同一个模块。通常情况下, 这些模块会声明它们依赖于该模块的不同次版本或补丁版本。Go 是如何解决这个问题的呢?

The module system uses the principle of *minimal version selection*: you will always get the lowest version of a dependency that is declared to work in all the *go.mod* files across all your dependencies. Let's say that your module directly depends on modules A, B, and C. All three of these modules depend on module D. The *go.mod* file for module A declares that it depends

on v1.1.0, module B declares that it depends on v1.2.0, and module C declares that it depends on v1.2.3. Go will import module D only once, and it will choose version v1.2.3, as that, in the words of the [Go Modules Reference](#), is the minimum version that satisfies all requirements.

模块系统采用最小版本选择原则：在所有依赖项的 go.mod 文件中，你总是会获取到声明为可正常工作的最低依赖版本。假设你的模块直接依赖于模块 A、B 和 C。这三个模块都依赖于模块 D。模块 A 的 go.mod 文件声明它依赖于 v1.1.0，模块 B 声明它依赖于 v1.2.0，模块 C 声明它依赖于 v1.2.3。Go 只会导入模块 D 一次，并且会选择 v1.2.3 版本，因为正如 Go 模块参考文档中所述，这是满足所有要求的最低版本。

You can see this in action with your sample program from [“Importing Third-Party Code”](#). The command `go mod graph` shows the dependency graph of your module and all its dependencies. Here are a few lines of its output:

您可以通过“导入第三方代码”中的示例程序来查看此功能。命令 `go mod graph` 会显示您的模块及其所有依赖项的依赖关系图。以下是其输出的部分内容：

```
github.com/learning-go-book-2e/money
github.com/fatih/color@v1.13.0
github.com/learning-go-book-2e/money
github.com/mattn/go-colorable@v0.1.9
```

```
github.com/learning-go-book-2e/money
github.com/mattn/go-isatty@v0.0.14
github.com/fatih/color@v1.13.0
github.com/mattn/go-colorable@v0.1.9
github.com/fatih/color@v1.13.0
github.com/mattn/go-isatty@v0.0.14
github.com/mattn/go-colorable@v0.1.9
github.com/mattn/go-isatty@v0.0.12
```

Each line lists two modules, the first being the parent and the second being the dependency and its version. You'll notice the

`github.com/fatih/color` module is declared to depend on version `v0.0.14` of `github.com/mattn/go-isatty`, while `github.com/mattn/go-colorable` depends on `v0.0.12`. The Go compiler selects version `v0.0.14` to use, because it is the minimal version that meets all requirements. This occurs even though, as of this writing, the latest version of `github.com/mattn/go-isatty` is `v0.0.16`. Your minimal version requirement is met with `v0.0.14`, so that's what is used.

每一行都列出了两个模块，第一个是父模块，第二个是依赖模块及其版本。你会注意到 `github.com/fatih/color` 模块声明依赖于 `github.com/mattn/go-isatty` 的 `v0.0.14` 版本，而 `github.com/mattn/go-colorable` 依赖于 `v0.0.12` 版本。Go 编译器选择使用 `v0.0.14` 版本，因为它是满足所有要求的最低版本。即使截至撰写本文时，`github.com/mattn/go-isatty` 的最新版本是

v0.0.16，也会出现这种情况。由于 v0.0.14 满足了你的最低版本要求，因此使用了该版本。

This system isn't perfect. You might find that while module A works with version v1.1.0 of module D, it does not work with version v1.2.3. What do you do then? Go's answer is that you need to contact the module authors to fix their incompatibilities. The *import compatibility rule* says, "If an old package and a new package have the same import path, the new package must be backward compatible with the old package." All minor and patch versions of a module must be backward compatible. If they aren't, it's a bug. In our hypothetical example, either module D needs to be fixed because it broke backward compatibility, or module A needs to be fixed because it made a faulty assumption about the behavior of module D.

这个系统并不完美。你可能会发现，模块 A 可以与模块 D 的 v1.1.0 版本兼容，但却无法与 v1.2.3 版本兼容。这时该怎么办呢？Go 的答案是，你需要联系模块作者来修复兼容性问题。导入兼容性规则规定：“如果一个旧包和一个新包具有相同的导入路径，则新包必须向后兼容旧包。”模块的所有次要版本和补丁版本都必须向后兼容。如果不兼容，那就是一个 bug。在我们假设的例子中，要么是模块 D 需要修复，因为它破坏了向后兼容性；要么是模块 A 需要修复，因为它对模块 D 的行为做出了错误的假设。

You might not find this answer satisfying, but it's honest. Some build systems, like npm, will include multiple versions of the same package. This

can introduce its own set of bugs, especially when there is package-level state. It also increases the size of your application. In the end, some things are better solved by community than code.

你可能不会对这个答案感到满意，但这是实话。有些构建系统，比如 npm，会包含同一个包的多个版本。这可能会引入一系列新的 bug，尤其是在存在包级状态的情况下。此外，它还会增加应用程序的体积。归根结底，有些问题由社区解决比编写代码解决得更好。

Updating to Compatible Versions

更新至兼容版本

What if you explicitly want to upgrade a dependency? Let's assume that after writing the initial program, there are three more versions of `simpletax`. The first fixes problems in the initial v1.1.0 release. Since it's a bug patch release with no new functionality, it would be released as v1.1.1. The second keeps the current functionality but also adds a new function. It would get the version number v1.2.0. Finally, the third fixes a bug that was found in version v1.2.0. It has the version number v1.2.1.

如果您明确想要升级某个依赖项该怎么办？假设在编写初始程序之后，`simpletax` 又有三个版本。第一个版本修复了初始版本 v1.1.0 中的问题。由于这是一个修复 bug 的版本，没有新增功能，因此版本号为 v1.1.1。第二个版本保留了现有功能，并添加了一个新功能。它

的版本号为 v1.2.0。最后，第三个版本修复了版本 v1.2.0 中发现的一个 bug。它的版本号为 v1.2.1。

To upgrade to the bug patch release for the current minor version, use the command `go get -u=patch github.com/learning-go-book-2e/simpletax`. Since you had downgraded to v1.0.0, you would remain on that version, because there is no patch version with the same minor version.

要升级到当前次版本号的错误修复版本，请使用命令 `go get -u=patch github.com/learning-go-book-2e/simpletax`。由于您已降级到 v1.0.0，因此您将停留在该版本，因为没有与该次版本号相同的补丁版本。

Upgrade to version v1.1.0 by using `go get github.com/learning-go-book-2e/simpletax@v1.1.0` and then run `go get -u=patch github.com/learning-go-book-2e/simpletax`. This upgrades the version to v1.1.1.

使用 `go get github.com/learning-go-book-2e/simpletax@v1.1.0` 升级到 v1.1.0 版本，然后运行 `go get -u=patch github.com/learning-go-book-2e/simpletax`。这将会把版本升级到 v1.1.1。

Finally, use the command `go get -u github.com/learning-go-book-2e/simpletax` to get the most recent version of `simpletax`.

That upgrades you to version v1.2.1.

最后，使用命令 `go get -u github.com/learning-go-book-2e/simpletax` 获取 `simpletax` 的最新版本。这将使您升级到 v1.2.1 版本。

Updating to Incompatible Versions

更新到不兼容的版本

Let's go back to the program. You're expanding to Canada, and luckily, a version of the `simpletax` module handles both the US and Canada. However, this version has a slightly different API than the previous one, so its version is v2.0.0.

我们回到程序本身。您正在拓展到加拿大市场，幸运的是，`simpletax` 模块的一个版本可以同时支持美国和加拿大。但是，这个版本的 API 与之前的版本略有不同，因此它的版本号是 v2.0.0。

To handle incompatibility, Go modules follow the *semantic import versioning* rule. This rule has two parts:

为了处理兼容性问题，Go 模块遵循语义化导入版本控制规则。该规则包含两个部分：

- The major version of the module must be incremented.

模块的主版本号必须递增。

- For all major versions besides 0 and 1, the path to the module must end in vN , where N is the major version.

除了 0 和 1 之外的所有主要版本，模块路径必须以 vN 结尾，其中 N 为主版本。

The path changes because an import path uniquely identifies a package. By definition, incompatible versions of a package are not the same package.

Using different paths means that you can import two incompatible versions of a package into different parts of your program, allowing you to upgrade gracefully.

路径之所以会改变，是因为导入路径能够唯一标识一个包。根据定义，包的不同版本并非同一个包。使用不同的路径意味着您可以将包的两个不兼容版本导入到程序的不同部分，从而实现优雅的升级。

Let's see how this changes the program. First, change the import of `simpletax` to this:

我们来看看这会对程序产生什么影响。首先，将 `simpletax` 的导入语句改为：

```
"github.com/learning-go-book-2e/simpletax/v2"
```

This changes your import to refer to the `v2` module.

这将更改您的导入语句，使其引用 v2 模块。

Next, change the code in `main` to the following:

接下来，将 `main` 中的代码更改为以下内容：

```
func main() {  
    amount, err :=  
decimal.NewFromString(os.Args[1])  
    if err != nil {  
        log.Fatal(err)  
    }  
    zip := os.Args[2]  
    country := os.Args[3]  
    percent, err :=  
simpletax.ForCountryPostalCode(country, zip)  
    if err != nil {  
        log.Fatal(err)  
    }  
    total :=  
amount.Add(amount.Mul(percent)).Round(2)  
    fmt.Println(total)  
}
```

The program is now reading a third parameter from the command line, which is the country code. The program also calls a different function in the

`simpletax` package. When you run `go get ./...`, the dependency is automatically updated:

程序现在从命令行读取第三个参数，即国家/地区代码。程序还会调用 `simpletax` 包中的另一个函数。运行 `go get ./...` 时，依赖项会自动更新：

```
$ go get ./...
go: downloading github.com/learning-go-book-
2e/simpletax/v2 v2.0.0
go: added github.com/learning-go-book-
2e/simpletax/v2 v2.0.0
```

Build and run the program to see the new output:

编译并运行程序，查看新的输出结果：

```
$ go build
$ ./region_tax 99.99 M4B1B4 CA
112.99
$ ./region_tax 99.99 12345 US
107.99
```

When you look at the *go.mod* file, you'll see that the new version of `simpletax` is included:

查看 `go.mod` 文件，你会发现其中包含了新版本的 `simpletax`：

```
module github.com/learning-go-book-2e/region_tax

go 1.20

require (
    github.com/learning-go-book-2e/simpletax
    v1.0.0
    github.com/learning-go-book-2e/simpletax/v2
    v2.0.0
    github.com/shopspring/decimal v1.3.1
)
```

And *go.sum* has been updated as well:

`go.sum` 也已更新：

```
github.com/learning-go-book-2e/simpletax v1.0.0
h1:KZU8aXRCHkvgFmBWkV...
github.com/learning-go-book-2e/simpletax
v1.0.0/go.mod h1:lR4YYZwbDTI...
github.com/learning-go-book-2e/simpletax v1.1.0
h1:sG83gscauX/b8yKKY9...
```

```
github.com/learning-go-book-2e/simpletax
v1.1.0/go.mod h1:lR4YYZwbDTI...
github.com/learning-go-book-2e/simpletax/v2 v2.0.0
h1:EUFWy1BBA2omgkm...
github.com/learning-go-book-2e/simpletax/v2
v2.0.0/go.mod h1:yGLh6ngH...
github.com/shopspring/decimal v1.3.1
h1:2Us11nmF/WZucqkFZhnfFYxxxu8LG...
github.com/shopspring/decimal v1.3.1/go.mod
h1:DKyhrW/HYNuLGql+MJL6WC...
```

The old versions of `simpletax` are still referenced, even though they are no longer used. Use `go mod tidy` to remove those unused versions. Then you'll see only v2.0.0 of `simpletax` referenced in *go.mod* and *go.sum*.

即使不再使用，旧版本的 `simpletax` 仍然被引用。请使用 `go mod tidy` 移除这些未使用的版本。移除后，*go.mod* 和 *go.sum* 中将只引用 `simpletax` 的 v2.0.0 版本。

Vendoring

供应商

To ensure that a module always builds with identical dependencies, some organizations like to keep copies of their dependencies inside their module.

This is known as *vendoring*. It's enabled by running the command `go mod vendor`. This creates a directory called *vendor* at the top level of your module that contains all your module's dependencies. These dependencies are used in place of the module cache stored on your computer.

为了确保模块始终使用相同的依赖项进行构建，一些组织喜欢在模块内部保留依赖项的副本。这称为 vendoring（依赖项管理）。可以通过运行命令 `go mod vendor` 来启用此功能。该命令会在模块的根目录下创建一个名为 *vendor* 的目录，其中包含模块的所有依赖项。这些依赖项将取代存储在计算机上的模块缓存。

If new dependencies are added to *go.mod* or versions of existing dependencies are upgraded with `go get`, you need to run `go mod vendor` again to update the *vendor* directory. If you forget to do this, `go build`, `go run`, and `go test` will display an error message and refuse to run.

如果使用 `go get` 向 *go.mod* 添加了新的依赖项，或者升级了现有依赖项的版本，则需要再次运行 `go mod vendor` 来更新 *vendor* 目录。如果您忘记执行此操作，则 `go build`、`go run` 和 `go test` 将显示错误消息并拒绝运行。

Older Go dependency management systems required vendoring, but with the advent of Go modules and proxy servers (see [“Module Proxy Servers”](#) for details), the practice is falling out of favor. One reason you might still

want to vendor is that it can make building your code faster and more efficient when working with some CI/CD (continuous integration/continuous delivery) pipelines. If a pipeline's build servers are ephemeral, the module cache may not be preserved. Vendoring dependencies allows these pipelines to avoid making multiple network calls to download dependencies every time a build is triggered. The downside is that it dramatically increases the size of your codebase in version control.

早期的 Go 依赖管理系统需要使用 vendor 目录，但随着 Go 模块和代理服务器（详见“模块代理服务器”）的出现，这种做法已逐渐被淘汰。不过，你仍然可能需要使用 vendor 目录的一个原因是，它可以让代码构建速度更快、效率更高，尤其是在使用某些 CI/CD（持续集成/持续交付）流水线时。如果流水线的构建服务器是临时的，模块缓存可能无法保留。使用 vendor 目录可以让这些流水线避免每次触发构建时都进行多次网络请求来下载依赖项。缺点是它会显著增加版本控制系统中的代码库大小。

Using pkg.go.dev

使用 pkg.go.dev

While there isn't a single centralized repository of Go modules, there is a single service that gathers together documentation on Go modules. The Go team has created a site called [*pkg.go.dev*](https://pkg.go.dev) that automatically indexes open source Go modules. For each module, the package index publishes the

godocs, the license used, the *README*, the module's dependencies, and what open source modules depend on the module. You can see the info that *pkg.go.dev* has on your `simpletax` module in [Figure 10-2](#).

虽然 Go 模块没有统一的集中式仓库，但有一个服务专门收集 Go 模块的文档。Go 团队创建了一个名为 `pkg.go.dev` 的网站，它会自动索引开源的 Go 模块。对于每个模块，该包索引会发布其 godoc、许可证、README 文件、依赖项以及依赖于该模块的开源模块。您可以在图 10-2 中查看 `pkg.go.dev` 上关于您的 `simpletax` 模块的信息。

The screenshot shows the pkg.go.dev website interface. At the top is a dark blue header with the 'GO' logo, a search bar containing 'Search packages or symbols', and a menu icon. Below the header, the breadcrumb 'Discover Packages > github.com/learning-go-book-2e/simpletax' is visible. The package name 'simpletax' is prominently displayed, with 'package' and 'module' tabs. A link 'Go to main page' is present. The 'Versions in this module' section shows a timeline with two versions: 'v1.1.0' and 'v1.0.0', both dated 'Sep 18, 2022'. Below this, the 'Other modules containing this package' section lists 'github.com/learning-go-book-2e/simpletax/v2'.

Discover Packages > github.com/learning-go-book-2e/simpletax

simpletax package module

← Go to main page

Versions in this module Expand all

v1

v1.1.0	•	Sep 18, 2022
v1.0.0	• ▶	Sep 18, 2022

Other modules containing this package

github.com/learning-go-book-2e/simpletax/v2

Figure 10-2. Use `pkg.go.dev` to find and learn about third-party modules

图 10-2. 使用 `pkg.go.dev` 查找和了解第三方模块

Publishing Your Module

发布您的模块

Making your module available to other people is as simple as putting it in a version control system. This is true whether you are releasing your module

as open source on a public version control system like GitHub or a private one that's hosted by you or within a company. Since Go programs build from source code and use a repository path to identify themselves, there's no need to explicitly upload your module to a central library repository, as you do for Maven Central or npm. Make sure you check in both your *go.mod* file and your *go.sum* file.

将你的模块提供给其他人使用非常简单，只需将其放入版本控制系统即可。无论你是将模块开源发布到像 GitHub 这样的公共版本控制系统，还是发布到你自已或公司内部托管的私有版本控制系统，情况都是如此。由于 Go 程序是从源代码构建的，并使用仓库路径来标识自身，因此无需像 Maven Central 或 npm 那样显式地将模块上传到中央库仓库。请确保同时提交 *go.mod* 文件和 *go.sum* 文件。

NOTE

笔记

While most Go developers use Git for version control, Go also supports Subversion, Mercurial, Bazaar, and Fossil. By default, Git and Mercurial can be used for public repositories, and any of the supported version control systems can be used for private repositories. For details, check out the [version control system documentation](#) for Go modules.

虽然大多数 Go 开发者使用 Git 进行版本控制，但 Go 也支持 Subversion、Mercurial、Bazaar 和 Fossil。默认情况下，Git 和 Mercurial 可用于公共仓库，而任何受支持的版本控制系统都可用于私有仓库。更多详情，请参阅 Go 模块的版本控制系统文档。

When releasing an open source module, you should include a file named *LICENSE* in the root of your repository that specifies the open source license under which you are releasing your code. [It's FOSS](#) is a good resource for learning more about the various kinds of open source licenses.

发布开源模块时，您应该在代码仓库的根目录下包含一个名为 *LICENSE* 的文件，其中指定您发布代码所依据的开源许可证。It's FOSS 是一个很好的资源，可以帮助您了解各种开源许可证。

Roughly speaking, you can divide open source licenses into two categories: permissive (which allows users of your code to keep their code private) and nonpermissive (which requires users of your code to make their code open source). While the license you choose is up to you, the Go community favors permissive licenses, such as BSD, MIT, and Apache. Since Go compiles third-party code directly into every application, the use of a nonpermissive license like the GPL would require people who use your code to release their code as open source as well. For many organizations, this is not acceptable.

粗略地说，开源许可证可以分为两类：宽松型（允许代码用户将其代码保密）和非宽松型（要求代码用户将其代码开源）。虽然选择哪种许可证完全取决于您，但 Go 社区更倾向于宽松型许可证，例如 BSD、MIT 和 Apache。由于 Go 会将第三方代码直接编译到每个应用程序中，因此使用像 GPL 这样的非宽松型许可证会要求使用您代码的人也将其代码开源。对于许多组织来说，这是不可接受的。

One final note: do not write your own license. Few people will trust that it has been properly reviewed by a lawyer, and they can't tell what claims you are making on their module.

最后一点：不要自行撰写许可证。很少有人会相信它已经过律师审核，他们也无法判断你对他们的模块提出了哪些主张。

Versioning Your Module

模块版本控制

Whether your module is public or private, you should properly version your module so that it works correctly with Go's module system. As long as you are adding functionality or patching bugs, the process is simple. Store your changes in your source code repository, then apply a tag that follows the semantic versioning rules I discussed in [“Semantic Versioning”](#).

无论你的模块是公开的还是私有的，都应该正确地进行版本控制，以确保它能与 Go 的模块系统兼容。如果你是在添加新功能或修复 bug，这个过程很简单。只需将你的更改存储在源代码仓库中，然后应用一个符合我在“语义化版本控制”一文中讨论的语义化版本控制规则的标签即可。

Go's semantic versioning supports the concept of *pre-releases*. Let's assume that the current version of your module is tagged `v1.3.4`. You are working on version 1.4.0, which is not quite done, but you want to try importing it into another module. What you should do is append a hyphen (-) to the end of your version tag, followed by an identifier for the pre-release build. In this case, use a tag like `v1.4.0-beta1` to indicate beta 1 of version 1.4.0 or `v1.4.0-rc2` to indicate release candidate 2. If you want to depend on a pre-release candidate, you must specify its version explicitly in `go get`, as Go will not automatically select a pre-release version.

Go 的语义化版本控制支持预发布版本（pre-release）的概念。假设你的模块当前版本标记为 `v1.3.4`。你正在开发 1.4.0 版本，该版本尚未完全完成，但你想尝试将其导入到另一个模块中。你应该在版本标签末尾添加一个连字符 (-)，后跟预发布版本的标识符。例如，可以使用 `v1.4.0-beta1` 来表示 1.4.0 版本的 beta 1，或者使用 `v1.4.0-rc2` 来表示候选发布版本 2。如果你想依赖于某个预发布版本，则必须在 `go get` 中显式指定其版本，因为 Go 不会自动选择预发布版本。

If you reach a point where you need to break backward compatibility, the process is more complicated. As you saw when importing version 2 of the `simpletax` module, a backward-breaking change requires a different import path. There are a few steps to take.

如果需要打破向后兼容性，流程会更加复杂。正如您在导入 `simpletax` 模块的第 2 版时所看到的，破坏向后兼容性的更改需要不同的导入路径。这需要几个步骤。

First you need to choose a way to store your new version. Go supports two ways for creating the different import paths:

首先，您需要选择一种存储新版本的方法。Go 支持两种创建不同导入路径的方法：

- Create a subdirectory within your module named vN , where N is the major version of your module. For example, if you are creating version 2 of your module, call this directory $v2$. Copy your code into this subdirectory, including the *README* and *LICENSE* files.

在你的模块目录下创建一个名为 vN 的子目录，其中 N 是模块的主版本号。例如，如果你创建的是模块的第 2 版，则将此目录命名为 $v2$ 。将你的代码（包括 *README* 和 *LICENSE* 文件）复制到这个子目录中。

- Create a branch in your version control system. You can put either the old code or the new code on the new branch. Name the branch vN if you are putting the new code on the branch, or $vN-1$ if you are putting the old code there. For example, if you are creating version 2 of your module and want to put version 1 code on the branch, name the branch $v1$.

在版本控制系统中创建一个分支。您可以将旧代码或新代码放在新分支上。如果要将新代码放在分支上，请将分支命名为 vN ；如果要将旧代码放在分支上，请将分支命名为 $vN-1$ 。例如，如果您正在创建模块的第 2 版，并希望将第 1 版的代码放在分支上，则将分支命名为 $v1$ 。

After you decide how to store your new code, you need to change the import path in the code in your subdirectory or branch. The module path in your *go.mod* file must end with $/vN$, and all the imports within your module must use $/vN$ as well. Going through all your code can be tedious, but Marwan Sulaiman has created a [tool that automates](#) the work. Once the paths are fixed, go ahead and implement your changes.

确定新代码的存储方式后，您需要修改子目录或分支代码中的导入路径。*go.mod* 文件中的模块路径必须以 $/vN$ 结尾，模块内的所有导入语句也必须使用 $/vN$ 。逐行检查所有代码可能很繁琐，但 Marwan Sulaiman 开发了一个可以自动完成这项工作的工具。路径修改完成后，即可开始实施您的更改。

NOTE

笔记

Technically, you could just change *go.mod* and your import statements, tag your main branch with the latest version, and not bother with a subdirectory or versioned branch. However, this is not a good practice, as it makes it unclear where to find older major versions of your module.

理论上，你可以直接修改 *go.mod* 文件和导入语句，将主分支标记为最新版本，而无需创建子目录或版本分支。然而，这种做法并不好，因为它会导致难以找到模块的旧版本。

When you are ready to publish your new code, place a tag on your repository that looks like *vN.0.0*. If you are using the subdirectory system or keeping the latest code on your main branch, tag the main branch. If you are placing your new code on a different branch, tag that branch instead.

当您准备发布新代码时，请在您的代码仓库中添加一个类似 *vN.0.0* 的标签。如果您使用子目录系统或将最新代码保存在主分支上，请标记主分支。如果您将新代码放在其他分支上，请标记该分支。

You can find more details on updating your code to an incompatible version in the post [“Go Modules: v2 and Beyond”](#) on The Go Blog and in [“Developing a Major Version Update”](#) on the Go developer website.

您可以在 Go 博客上的文章“Go Modules: v2 及更高版本”和 Go 开发者网站上的“开发主要版本更新”中找到有关将代码更新到不兼容版本的更多详细信息。

Overriding Dependencies

覆盖依赖项

Forks happen. While there's a bias against forking in the open source community, sometimes a module stops being maintained or you want to experiment with changes that aren't accepted by the module's owner. A `replace` directive redirects all references to a module across all your module's dependencies and replaces them with the specified fork of the module. It looks like this:

分支（fork）是不可避免的。虽然开源社区普遍对分支持反对态度，但有时某个模块会停止维护，或者你想尝试一些模块所有者不接受的更改。`replace` 指令会将你模块所有依赖项中对目标模块的引用全部重定向到指定的分支。它看起来像这样：

```
replace github.com/jonbodner/proteus =>  
github.com/someone/my_proteus v1.0.0
```

The original module location is specified on the left side of the `=>` and the replacement on the right. The right side must have a version specified, but specifying a version is optional for the left side. If a version is specified on the left side, only that specific version will be replaced. If the version is not specified, any version of the original module will be replaced with the specific version of the fork.

=> 左侧指定原始模块位置，右侧指定替换模块位置。右侧必须指定版本，而左侧版本指定是可选的。如果左侧指定了版本，则只会替换该特定版本。如果未指定版本，则原始模块的任何版本都将被替换为分支的特定版本。

A `replace` directive can also refer to a path on your local filesystem:

`replace` 指令也可以指代本地文件系统上的路径：

```
replace github.com/jonbodner/proteus =>
../projects/proteus
```

With a local `replace` directive, the module version must be omitted from the right side.

使用本地 `replace` 指令时，必须从右侧省略模块版本。

WARNING

警告

Avoid using local `replace` directives. They provided a way to modify multiple modules simultaneously before the invention of Go workspaces, but now they are a potential source of broken modules. (I will cover workspaces shortly.) If you share your module via version control, a module with local references in `replace` directives will probably not build for anyone else, since you cannot guarantee that other people will have the replacement modules in the same locations on their drive.

避免使用本地 `replace` 指令。在 Go 工作区出现之前，它们提供了一种同时修改多个模块的方法，但现在它们可能导致模块损坏。（我稍后会介绍工作区。）如果您通过版本控制共享模块，那么在 `replace` 指令中包含本地引用的模块可能无法被其他人编译，因为您无法保证其他人的硬盘驱动器上相同位置也有替换模块。

You also might want to block a specific version of a module from being used. Perhaps it has a bug or is incompatible with your module. Go provides the `exclude` directive to prevent a specific version of a module from being used:

您可能还希望阻止使用特定版本的模块。也许该版本存在 bug 或与您的模块不兼容。Go 提供了 `exclude` 指令来阻止使用特定版本的模块：

```
exclude github.com/jonbodner/proteus v0.10.1
```

When a module version is excluded, any mentions of that module version in any dependent module are ignored. If a module version is excluded and it's

the only version of that module that's required in your module's dependencies, use `go get` to add an indirect import of a different version of the module to your module's *go.mod* file so that your module still compiles.

如果某个模块版本被排除，则所有依赖模块中对该模块版本的引用都将被忽略。如果某个模块版本被排除，并且它是该模块依赖项中唯一需要的版本，请使用 `go get` 将该模块的不同版本间接导入到模块的 *go.mod* 文件中，以确保模块仍然可以编译。

Retracting a Version of Your Module

撤回您的模块版本

Sooner or later, you will accidentally publish a version of your module that you don't want anyone to use. Perhaps it was released by accident before testing was complete. Maybe after its release, a critical vulnerability is discovered and no one should use it any more. No matter the reason, Go provides a way for you to indicate that certain versions of a module should be ignored. This is done by adding a `retract` directive to the *go.mod* file of your module. It consists of the word `retract` and the semantic version that should no longer be used. If a range of versions shouldn't be used, you can exclude all versions in that range by placing the upper and lower bounds within brackets, separated by a comma. While it's not required, you

should include a comment after a version or version range to explain the reason for the retraction.

迟早有一天，你会不小心发布了一个你不希望任何人使用的模块版本。也许是在测试完成之前误发的，又或许是发布后发现了严重漏洞，导致该版本被弃用。无论原因如何，Go 都提供了一种方法来指示应该忽略模块的某些版本。这可以通过在模块的 `go.mod` 文件中添加 `retract` 指令来实现。该指令由 `retract` 和不应再使用的语义版本号组成。如果应该排除某个版本范围，你可以将该范围内的版本号用逗号分隔，并用方括号括起来。虽然并非强制要求，但你应该在版本号或版本范围后添加注释，解释撤回的原因。

If you wish to retract multiple nonsequential versions, you can specify them with multiple `retract` directives. In the examples shown, version 1.5.0 is excluded, as are all versions from 1.7.0 to 1.8.5, inclusive:

如果您希望撤销多个非连续版本，可以使用多个 `retract` 指令来指定它们。在所示示例中，版本 1.5.0 以及从 1.7.0 到 1.8.5（含 1.7.0 和 1.8.5）的所有版本均已排除在外：

```
retract v1.5.0 // not fully tested
retract [v1.7.0, v.1.8.5] // posts your cat photos
to LinkedIn w/o permission
```

Adding a `retract` directive to `go.mod` requires you to create a new version of your module. If the new version contains only the retraction, you should retract it as well.

向 `go.mod` 添加 `retract` 指令需要您创建模块的新版本。如果新版本仅包含撤回声明，您也应该将其撤回。

When a version is retracted, existing builds that specified the version will continue to work, but `go get` and `go mod tidy` will not upgrade to them. They will no longer appear as options when you use the `go list` command. If the most recent version of a module is retracted, it will no longer be matched with `@latest`; the highest unretracted version will match instead.

当某个版本被撤回时，已指定该版本的现有构建将继续运行，但 `go get` 和 `go mod tidy` 将不会升级到该版本。当您使用 `go list` 命令时，它们将不再显示为选项。如果模块的最新版本被撤回，它将不再与 `@latest` 匹配；取而代之的是匹配最高未撤回的版本。

NOTE

笔记

While `retract` can be confused with `exclude`, there's a very important difference. You use `retract` to prevent others from using specific versions of *your* module. An `exclude` directive blocks you from using versions of another module.

虽然 `retract` 容易与 `exclude` 混淆，但它们之间存在一个非常重要的区别。您可以使用 `retract` 来阻止其他人使用您模块的特定版本。而 `exclude` 指令则可以阻止您使用其他模块的版本。

Using Workspaces to Modify Modules Simultaneously

使用工作区同时修改模块

There's one drawback to using your source code repository and its tags as a way to track your dependencies and their versions. If you want to make changes to two (or more) modules simultaneously, and you want to experiment with those changes across modules, you need a way to use a local copy of a module instead of the version in the source code repository.

使用源代码仓库及其标签来跟踪依赖项及其版本存在一个缺点。如果您想同时修改两个（或多个）模块，并且希望跨模块测试这些修改，则需要一种方法来使用模块的本地副本，而不是源代码仓库中的版本。

WARNING

警告

You can find obsolete advice online to try to solve this issue with temporary `replace` directives in `go.mod` that point to local directories. Do not do this! It's too easy to forget to undo these changes before committing and pushing your code. Workspaces were introduced to avoid this antipattern.

网上有一些过时的建议，试图通过在 `go.mod` 中使用指向本地目录的临时 `replace` 指令来解决这个问题。千万不要这样做！提交并推送代码之前，很容易忘记撤销这些更改。工作区正是为了避免这种反模式而引入的。

Go uses *workspaces* to address this issue. A workspace allows you to have multiple modules downloaded to your computer, and references between those modules will automatically resolve to the local source code instead of the code hosted in your repository.

Go 使用工作区来解决这个问题。工作区允许你将多个模块下载到本地计算机，这些模块之间的引用会自动解析为本地源代码，而不是托管在代码仓库中的代码。

NOTE

笔记

This section assumes that you have a GitHub account. If you don't, you can still follow along. I'm going to use the organization name `learning-go-book-2e`, but you should replace it with your GitHub account name or organization.

本节假设您拥有 GitHub 帐户。如果您没有帐户，仍然可以跟随本教程操作。我将使用组织名称 `learning-go-book-2e`，但您应该将其替换为您的 GitHub 帐户名称或组织名称。

Let's start with two sample modules. Create a *my_workspace* directory and in that directory, create two more directories, *workspace_lib* and *workspace_app*. In the *workspace_lib* directory, run `go mod init github.com/learning-go-book-2e/workspace_lib`. Create a file called *lib.go* with the following content:

我们先来看两个示例模块。创建一个名为 *my_workspace* 的目录，并在该目录下创建两个子目录：*workspace_lib* 和 *workspace_app*。在 *workspace_lib* 目录中，运行 `go mod init github.com/learning-go-book-2e/workspace_lib`。创建一个名为 *lib.go* 的文件，内容如下：

```
package workspace_lib

func AddNums(a, b int) int {
    return a + b
}
```

In the *workspace_app* directory, run `go mod init github.com/learning-go-book-2e/workspace_app`. Create a file called *app.go* with the following contents:

在 *workspace_app* 目录下，运行 `go mod init github.com/learning-go-book-2e/workspace_app`。创建一个名为 *app.go* 的文件，内容如下：


```
package main

import (
    "fmt"
    "github.com/learning-go-book-2e/workspace_lib"
)

func main() {
    fmt.Println(workspace_lib.AddNums(2, 3))
}
```

In previous sections, you used `go get ./...` to add `require` directives to `go.mod`. Let's see what happens if you try it here:

在前面的章节中，您使用 `go get ./...` 将 `require` 指令添加到 `go.mod` 文件中。让我们看看在这里尝试会发生什么：

```
$ go get ./...
github.com/learning-go-book-2e/workspace_app
imports
    github.com/learning-go-book-2e/workspace_lib:
cannot find module
    providing package github.com/learning-go-
book-2e/workspace_lib
```

Since *workspace_lib* hasn't been pushed to GitHub yet, you can't pull it in. If you try to run `go build`, you will get a similar error:

由于 *workspace_lib* 尚未推送到 GitHub, 因此您无法拉取它。如果您尝试运行 `go build`, 将会收到类似的错误:

```
$ go build
app.go:5:2: no required module provides
    package github.com/learning-go-book-
2e/workspace_lib; to add it:
    go get github.com/learning-go-book-
2e/workspace_lib
```

Let's take advantage of workspaces to allow *workplace_app* to see the local copy of *workspace_lib*. Go to the *my_workspace* directory and run the following commands:

让我们利用工作区功能, 让 *workplace_app* 能够访问 *workspace_lib* 的本地副本。进入 *my_workspace* 目录并运行以下命令:

```
$ go work init ./workspace_app
$ go work use ./workspace_lib
```

This creates a *go.work* file in *my_workspace* with the following contents:

这会在 `my_workspace` 目录下创建一个名为 `go.work` 的文件，内容如下：

```
go 1.20

use (
    ./workspace_app
    ./workspace_lib
)
```

WARNING

警告

The `go.work` file is meant for your local computer only. Do not commit it to source control!

`go.work` 文件仅供本地计算机使用。请勿将其提交到版本控制系统！

Now if you build `workspace_app`, everything works:

现在，如果您构建 `workspace_app`，一切正常：

```
$ cd workspace_app
$ go build
```

```
$ ./workspace_app  
5
```

Now that you are sure that *workspace_lib* does the right thing, it can be pushed to GitHub. In GitHub, create an empty public repository called `workspace_lib` and then run the following commands in the *workspace_lib* directory:

既然您已确认 `workspace_lib` 运行正常，就可以将其推送到 GitHub 了。在 GitHub 中，创建一个名为 `workspace_lib` 的空公共仓库，然后在 `workspace_lib` 目录中运行以下命令：

```
$ git init  
$ git add .  
$ git commit -m "first commit"  
$ git remote add origin  
git@github.com:learning-go-book-  
2e/workspace_lib.git  
$ git branch -M main  
$ git push -u origin main
```

After running these commands, go to https://github.com/learning-go-book-2e/workspace_lib/releases/new (replacing “learning-go-book-2e” with your account or organization), and create a new release with the tag v0.1.0.

运行这些命令后，请访问 https://github.com/learning-go-book-2e/workspace_lib/releases/new（将“learning-go-book-2e”替换为您的帐户或组织），并创建一个带有标签 v0.1.0 的新版本。

Now if you go back to the *workspace_app* directory and run `go get ./...`, the `require` directive is added, because there is a public module that can be downloaded:

现在，如果您返回 *workspace_app* 目录并运行 `go get ./...`，则会添加 `require` 指令，因为有一个可以下载公共模块：

```
$ go get ./...
go: downloading github.com/learning-go-book-2e/workspace_lib v0.1.0
go: added github.com/learning-go-book-2e/workspace_lib v0.1.0
$ cat go.mod
module github.com/learning-go-book-2e/workspace_app

go 1.20

require github.com/learning-go-book-2e/workspace_lib v0.1.0
```

You can validate that your code is working with the public module by setting the environment variable `GOWORK=off` and building your application:

您可以通过设置环境变量 `GOWORK=off` 并构建应用程序来验证您的代码是否与公共模块一起使用：

```
$ rm workspace_app
$ GOWORK=off go build
$ ./workspace_app
5
```

Even though there is now a `require` directive referring to the public module, you can continue to make updates in our local workspace and they will be used instead. In *workspace_lib*, modify the *lib.go* file and add the following function:

即使现在有一个指向公共模块的 `require` 指令，您仍然可以在本地工作区进行更新，更新后的代码将会生效。在 *workspace_lib* 中，修改 *lib.go* 文件并添加以下函数：

```
func SubNums(a, b int) int {
    return a - b
}
```

In *workspace_app*, modify the *app.go* file and add the following line to the end of the `main` function:

在 *workspace_app* 目录中，修改 *app.go* 文件，并在 `main` 函数的末尾添加以下行：

```
fmt.Println(workspace_lib.SubNums(2,3))
```

Now run `go build` and see it use the local module instead of the public one:

现在运行 `go build`，看看它是否使用了本地模块而不是公共模块：

```
$ go build
$ ./workspace_app
5
-1
```

Once you have made the edits and want to release your software, you need to update the version information in your modules' *go.mod* files to refer to the updated code. This requires you to commit your modules to source control in dependency order:

完成修改并准备发布软件后，您需要更新模块的 `go.mod` 文件中的版本信息，使其指向更新后的代码。这要求您按照依赖顺序将模块提交到源代码控制系统中：

1. Choose a modified module that has no dependencies on any of the modified modules in your workspace.

选择一个不依赖于工作区中任何其他已修改模块的已修改模块。

2. Commit this module to your source code repository.

将此模块提交到您的源代码仓库。

3. Create a new version tag on the newly committed module in your source code repository.

在源代码仓库中，为新提交的模块创建一个新的版本标签。

4. Use `go get` to update the version specified in `go.mod` in the modules that depend on the newly committed module.

使用 `go get` 更新依赖于新提交模块的模块中 `go.mod` 指定的版本。

5. Repeat the first four steps until all modified modules are committed.

重复前四个步骤，直到所有修改过的模块都提交为止。

If you have to make changes to *workspace_lib* in the future and want to test them in *workspace_app* without pushing back to GitHub and creating lots of temporary versions, you can `git pull` the latest versions of the modules into your workspace again and make your updates.

如果您将来需要对 `workspace_lib` 进行更改，并且想要在 `workspace_app` 中进行测试，而无需将其推送回 GitHub 并创建大量临时版本，您可以再次将模块的最新版本 `git pull` 添加到您的工作区并进行更新。

Module Proxy Servers

模块代理服务器

Rather than relying on a single, central repository for libraries, Go uses a hybrid model. Every Go module is stored in a source code repository, like GitHub or GitLab. But by default, `go get` doesn't fetch code directly from source code repositories. Instead, it sends requests to a *proxy server* run by Google. When the proxy server receives the `go get` request, it checks its cache to see if there has been a request for this module version before. If so, it returns the cached information. If a module or a version of a module isn't cached on the proxy server, it downloads the module from the module's repository, stores a copy, and returns the module. This allows the proxy server to keep copies of every version of virtually all public Go modules.

Go 语言并非依赖单一的中央库，而是采用混合模型。每个 Go 模块都存储在源代码仓库中，例如 GitHub 或 GitLab。但默认情况下，`go get` 并不会直接从源代码仓库获取代码。相反，它会将请求发送到由

Google 运行的代理服务器。当代理服务器收到 `go get` 请求时，它会检查缓存，查看之前是否已有对该模块版本的请求。如果有，则返回缓存的信息。如果代理服务器没有缓存某个模块或某个模块的版本，则会从模块的仓库下载该模块，存储一份副本，然后返回该模块。这使得代理服务器能够保存几乎所有公共 Go 模块的每个版本的副本。

In addition to the proxy server, Google also maintains a *checksum database*. It stores information on every version of every module cached by the proxy server. Just as the proxy server protects you from a module or a version of a module being removed from the internet, the checksum database protects you against modifications to a version of a module. This could be malicious (someone has hijacked a module and slipped in malicious code), or it could be inadvertent (a module maintainer fixes a bug or adds a new feature and reuses an existing version tag). In either case, you don't want to use a module version that has changed because you won't be building the same binary and don't know what the effects are on your application.

除了代理服务器之外，Google 还维护着一个校验和数据库。它存储着代理服务器缓存的每个模块的每个版本的信息。正如代理服务器保护您免受模块或模块版本从互联网上移除的影响一样，校验和数据库保护您免受模块版本被修改的影响。这种修改可能是恶意的（有人劫持了模块并植入了恶意代码），也可能是无意的（模块维护者修复了错误或添加了新功能，并重用了现有的版本标签）。无论哪种情况，您都不希望使用已更改的模块版本，因为这样您将构建不同的二进制文件，并且无法预知这会对您的应用程序产生什么影响。

Every time you download a module via `go get` or `go mod tidy`, the Go tools calculate a hash for the module and contact the checksum database to compare the calculated hash to the hash stored for that module's version. If they don't match, the module isn't installed.

每次通过 `go get` 或 `go mod tidy` 下载模块时，Go 工具都会计算该模块的哈希值，并联系校验和数据库，将计算出的哈希值与存储的该模块版本的哈希值进行比较。如果不匹配，则不会安装该模块。

Specifying a Proxy Server

指定代理服务器

Some people object to sending requests for third-party libraries to Google. There are a few options:

有些人反对向谷歌发送第三方库的请求。有几种替代方案：

- You can disable proxying entirely by setting the `GOPROXY` environment variable to `direct`. You'll download modules directly from their repositories, but if you depend on a version that's removed from the repository, you won't be able to access it.

您可以通过将环境变量 `GOPROXY` 设置为 `direct` 来完全禁用代理。您将直接从其存储库下载模块，但如果您依赖的版本已从存储库中移除，则您将无法访问它。

- You can run your own proxy server. Both Artifactory and Sonatype have Go proxy server support built into their enterprise repository products. The [Athens Project](#) provides an open source proxy server. Install one of these products on your network and then point `GOPROXY` to the URL.

您可以运行自己的代理服务器。Artifactory 和 Sonatype 的企业级代码库产品都内置了 Go 代理服务器支持。Athens 项目提供了一个开源代理服务器。在您的网络上安装这些产品之一，然后将 `GOPROXY` 指向相应的 URL。

Using Private Repositories

使用私有仓库

Most organizations keep their code in private repositories. If you want to use a private module in another Go module, you can't request it from Google's proxy server. Go will fall back to checking the private repository directly, but you might not want to leak the names of private servers and repositories to external services.

大多数组织会将代码保存在私有仓库中。如果您想在另一个 Go 模块中使用私有模块，则不能通过 Google 的代理服务器请求它。Go 会回退到直接检查私有仓库，但您可能不希望将私有服务器和仓库的名称泄露给外部服务。

If you are using your own proxy server, or if you have disabled proxying, this isn't an issue. Running a private proxy server has some additional benefits. First, it speeds up downloading of third-party modules, as they are cached in your company's network. If accessing your private repositories requires authentication, using a private proxy server means that you don't have to worry about exposing authentication information in your CI/CD pipeline. The private proxy server is configured to authenticate to your private repositories (see the [authentication configuration documentation](#) for Athens), while the calls to the private proxy server are unauthenticated.

如果您使用自己的代理服务器，或者已经禁用代理，则不会出现此问题。运行私有代理服务器还有一些额外的好处。首先，它可以加快第三方模块的下载速度，因为它们会被缓存到您公司的网络中。如果访问您的私有仓库需要身份验证，使用私有代理服务器意味着您无需担心在 CI/CD 管道中暴露身份验证信息。私有代理服务器配置为对您的私有仓库进行身份验证（请参阅 Athens 的身份验证配置文档），而对私有代理服务器的调用则无需身份验证。

If you are using a public proxy server, you can set the `GOPRIVATE` environment variable to a comma-separated list of your private repositories. For example, if you set `GOPRIVATE` to:

如果您使用的是公共代理服务器，可以将 `GOPRIVATE` 环境变量设置为以逗号分隔的私有仓库列表。例如，如果您将 `GOPRIVATE` 设置为：

```
GOPRIVATE=*.example.com,company.com/repo
```

any module stored in a repository that's located at any subdomain of *example.com* or at a URL that starts with *company.com/repo* will be downloaded directly.

存储在 *example.com* 的任何子域或以 *company.com/repo* 开头的 URL 的存储库中的任何模块都将直接下载。

Additional Details

更多详情

The Go Team has a complete [Go Modules Reference](#) available online. In addition to the content in this chapter, the Module Reference also covers topics like using version control systems besides Git, the structure and API of the module cache, additional environment variables for controlling module lookup behavior, and the REST API for the module proxy and checksum database.

Go 团队提供了一份完整的 Go 模块参考手册，可在网上查阅。除了本章内容外，该模块参考手册还涵盖了诸如使用 Git 以外的版本控制系统、模块缓存的结构和 API、用于控制模块查找行为的其他环境变量，以及模块代理和校验和数据库的 REST API 等主题。

Exercises

练习

1. Create a module in your own public repository. This module has a single function named `Add` with two `int` parameters and one `int` return value. This function adds the two parameters together and returns them. Make this version v1.0.0.

在您自己的公共仓库中创建一个模块。该模块包含一个名为 `Add` 的函数，该函数有两个 `int` 个参数和一个 `int` 个返回值。此函数将两个参数相加并返回结果。将此模块的版本设置为 v1.0.0。

2. Add godoc comments to your module that describe the package and the `Add` function. Be sure to include a link to

<https://www.mathsisfun.com/numbers/addition.html> in your `Add`

function godoc. Make this version v1.0.1.

请在模块中添加 godoc 注释，描述该包和 `Add` 函数。请务必在 `Add` 函数的 godoc 中包含指向

<https://www.mathsisfun.com/numbers/addition.html> 的链接。将此版本设为 v1.0.1。

3. Change `Add` to make it generic. Import the `golang.org/x/exp/constraints` package. Combine the `Integer` and `Float` types in that package to create an interface called `Number`. Rewrite `Add` to take in two parameters of type

`Number` and return a value of type `Number`. Version your module again. Because this is a backward-breaking change, this should be v2.0.0 of your module.

将 `Add` 改为泛型。导入 `golang.org/x/exp/constraints` 包。合并该包中的 `Integer` 和 `Float` 类型，创建一个名为 `Number` 的接口。重写 `Add`，使其接受两个 `Number` 类型的参数，并返回一个 `Number` 类型的值。再次更新模块版本。由于这是一个不兼容旧版本的更改，因此模块版本应为 v2.0.0。

Wrapping Up

总结

In this chapter, you've learned how to organize code and interact with the ecosystem of Go source code. You've seen how modules work, how to organize your code into packages, how to use third-party modules, and how to release modules of your own. In the next chapter, you're going to take a look at more of the development tools that are included with Go, learn about some essential third-party tools, and explore some techniques to give you better control over your build process.

在本章中，您学习了如何组织代码以及如何与 Go 源代码生态系统交互。您了解了模块的工作原理、如何将代码组织成包、如何使用第三方模块以及如何发布自己的模块。在下一章中，您将了解更多 Go 自

带的开发工具，学习一些重要的第三方工具，并探索一些技巧，以便更好地控制构建过程。

Chapter 11. Go Tooling

第十一章 Go Tooling

A programming language doesn't exist in isolation. For it to be useful, tools must help the developer turn source code into an executable. Since Go is intended to address the problems that software engineers face today and to help them build quality software, careful thought has been put into tooling that simplifies tasks that are often difficult with other development platforms. This includes improvements in how you build, format, update, validate, and distribute, and even how your users will install your code.

编程语言并非孤立存在。为了使其发挥作用，工具必须能够帮助开发者将源代码转换为可执行文件。Go 语言旨在解决软件工程师当前面临的问题，并帮助他们构建高质量的软件，因此，开发团队在工具方面投入了大量精力，力求简化其他开发平台通常难以完成的任务。这包括改进代码的构建、格式化、更新、验证和分发方式，甚至包括用户安装代码的方式。

I have already covered many of the bundled Go tools: `go vet`, `go fmt`, `go mod`, `go get`, `go list`, `go work`, `go doc`, and `go build`. The testing support provided by the `go test` tool is so extensive, it is covered by itself in [Chapter 15](#). In this chapter, you will explore additional

tools that make Go development great, both from the Go team and from third parties.

我已经介绍过许多 Go 内置工具： `go vet`、`go fmt`、`go mod`、`go get`、`go list`、`go work`、`go doc` 和 `go build`。`go test` 工具提供的测试支持非常强大，因此在第 15 章中单独介绍。在本章中，您将探索更多使 Go 开发更加强大的工具，这些工具既有 Go 团队开发的，也有第三方开发的。

Using go run to Try Out Small Programs

使用 go run 来尝试运行小型程序。

Go is a compiled language, which means that before Go code is run, it must be converted into an executable file. This is in contrast to interpreted languages like Python or JavaScript, which allow you to write a quick script to test an idea and execute it immediately. Having that rapid feedback cycle is important, so Go provides similar functionality via the `go run` command. It builds and executes a program in one step. Let's go back to the first program from [Chapter 1](#). Put it in a file called *hello.go*:

Go 是一种编译型语言，这意味着 Go 代码在运行之前必须转换成可执行文件。这与 Python 或 JavaScript 等解释型语言形成对比，后者允许你编写一个快速脚本来测试想法并立即执行。快速反馈循环非常重

要，因此 Go 通过 `go run` 命令提供了类似的功能。它一步完成程序的构建和执行。让我们回到第一章的第一个程序。把它放在一个名为 `hello.go` 的文件中：

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, world!")
}
```

(You can also find this code in the [Chapter 11 repository](#) in the *sample_code/gorun* directory.)

(您也可以在第 11 章代码库的 `sample_code/gorun` 目录中找到这段代码。)

Once the file is saved, use the `go run` command to build and execute it:

文件保存后，使用 `go run` 命令构建并执行它：

```
go run hello.go
Hello, world!
```

If you look inside the directory after running the `go run` command, you see that no binary has been saved there; the only file in the directory is the *hello.go* file you just created. Where did the executable go (no pun intended)?

运行 `go run` 命令后，如果您查看目录，会发现其中并未保存任何二进制文件；目录中唯一的文件是您刚刚创建的 *hello.go* 文件。可执行文件去哪儿了（此处并无双关之意）？

The `go run` command does, in fact, compile your code into a binary. However, the binary is built in a temporary directory. The `go run` command builds the binary, executes the binary from that temporary directory, and then deletes the binary after your program finishes. This makes the `go run` command useful for testing out small programs or using Go like a scripting language.

`go run` 命令确实会将你的代码编译成二进制文件。但是，这个二进制文件是在一个临时目录中生成的。`go run` 命令会生成二进制文件，从该临时目录执行该二进制文件，然后在程序运行结束后删除它。这使得 `go run` 命令非常适合测试小型程序或将 Go 用作脚本语言。

TIP

提示

Use `go run` when you want to treat a Go program like a script and run the source code immediately.

当您想将 Go 程序视为脚本并立即运行源代码时，请使用 `go run`。

Adding Third-Party Tools with `go install`

使用 `go install` 添加第三方工具

While some people choose to distribute their Go programs as precompiled binaries, tools written in Go can also be built from source and installed on your computer via the `go install` command.

虽然有些人选择将他们的 Go 程序作为预编译二进制文件分发，但也可以透过 `go install` 命令从源代码构建并安装到您的计算机上。

As you saw in [“Publishing Your Module”](#), Go modules are identified via their source code repositories. The `go install` command takes an argument, which is the path to the main package in a module’s source code repository, followed by an `@` and the version of the tool you want (if you just want to get the latest version, use `@latest`). It then downloads, compiles, and installs the tool.

正如您在“发布模块”一节中所看到的，Go 模块通过其源代码仓库进行标识。`go install` 命令接受一个参数，即模块源代码仓库中主包的路径，后跟 `@` 和您想要的工具版本（如果您只想获取最新版本，请使用 `@latest`）。然后，它会下载、编译并安装该工具。

WARNING

警告

Always be sure to include the `@version` or `@latest` after the name of the package that you are installing! If you don't, it triggers a variety of confusing behaviors and is almost certainly not what you want to do. You either get an error message (if the current directory is not in a module, or if the current directory is a module, but the package isn't referenced in the module's `go.mod` file), or it installs the package version mentioned in `go.mod`.

安装软件包时，务必在软件包名称后添加 `@version` 或 `@latest`！否则，会触发各种令人困惑的行为，而且几乎肯定不是您想要的结果。要么您会收到错误消息（如果当前目录不在任何模块中，或者当前目录是模块，但该软件包未在模块的 `go.mod` 文件中引用），要么它会安装 `go.mod` 文件中指定的软件包版本。

By default, `go install` places binaries into the `go/bin` directory within your home directory. Set the `GOBIN` environment variable to change this location. It is strongly recommended that you add the `go install` directory to your executable search path (this is done by modifying the `PATH` environment variable on both Unix and Windows). For simplicity, all the examples in this chapter assume that you've added this directory.

默认情况下，`go install` 会将二进制文件放置在您主目录下的 `go/bin` 目录中。您可以设置 `GOBIN` 环境变量来更改此位置。强烈建议您将 `go install` 目录添加到可执行文件搜索路径中（在 Unix 和 Windows 系统中，这可以通过修改 `PATH` 环境变量来实现）。为简单起见，本章中的所有示例均假设您已添加此目录。

NOTE

笔记

Other environment variables are recognized by the `go` tool. You can get a complete list, along with a brief description of each variable, using the `go help environment` command. Many of them control low-level behavior that can be safely ignored. I'll point out the relevant ones as needed.

`go` 工具可以识别其他环境变量。您可以使用 `go help environment` 命令获取完整列表以及每个变量的简要说明。其中许多变量控制着可以忽略的底层行为。我会根据需要指出相关的变量。

Some online resources tell you to set the `GOROOT` or `GOPATH` environment variables. `GOROOT` specifies the location where your Go development environment is installed, and `GOPATH` was used to store all Go source code, both your own and third-party dependencies. Setting these variables is no longer necessary; the `go` tool figures out `GOROOT` automatically, and `GOPATH`-based development has been superseded by modules.

有些在线资源会建议你设置 `GOROOT` 或 `GOPATH` 环境变量。`GOROOT` 指定 Go 开发环境的安装位置，而 `GOPATH` 用于存储所有 Go 源代码，包括你自己的代码和第三方依赖项。现在不再需要设置这些变量；`go` 工具会自动计算 `GOROOT`，并且基于 `GOPATH` 的开发方式已被模块取代。

Let's look at a quick example. Jaana Dogan created a great Go tool called `hey` that load tests HTTP servers. You can point it at the website of your choosing or an application that you've written. Here's how to install `hey` with the `go install` command:

我们来看一个简单的例子。Jaana Dogan 创建了一个很棒的 Go 工具 `hey`，它可以对 HTTP 服务器进行负载测试。你可以将它指向你选择的网站或你编写的应用程序。以下是如何使用 `go install` 命令安装 `hey`：

```
$ go install github.com/rakyll/hey@latest
go: downloading github.com/rakyll/hey v0.1.4
go: downloading golang.org/x/net v0.0.0-20181017193950-04a2e542c03f
go: downloading golang.org/x/text v0.3.0
```

This downloads `hey` and all its dependencies, builds the program, and installs the binary in your Go binary directory.

这会下载 `hey` 及其所有依赖项，构建程序，并将二进制文件安装到您的 Go 二进制目录中。

NOTE

笔记

As I covered in [“Module Proxy Servers”](#), the contents of Go repositories are cached in proxy servers. Depending on the repository and the values in your `GOPROXY` environment variable, `go install` may download from a proxy or directly from a repository. If `go install` downloads directly from a repository, it relies on command-line tools being installed on your computer. For example, you must have Git installed to download from GitHub.

正如我在“模块代理服务器”一文中提到的，Go 代码仓库的内容会被缓存到代理服务器中。根据代码仓库以及 `GOPROXY` 环境变量的值，`go install` 可能会从代理服务器下载，也可能直接从代码仓库下载。如果 `go install` 直接从代码仓库下载，则需要您的计算机上安装了命令行工具。例如，您必须安装 Git 才能从 GitHub 下载。

Now that you have built and installed `hey`, you can run it with the following:

现在您已经构建并安装了 `hey`，可以使用以下命令运行它：

```
$ hey https://go.dev
```

Summary:

Total:	2.1272 secs
Slowest:	1.4227 secs
Fastest:	0.0573 secs
Average:	0.3467 secs
Requests/sec:	94.0181

If you have already installed a tool and want to update it to a newer version, rerun `go install` with the newer version specified or with `@latest` :

如果您已经安装了某个工具并想将其更新到新版本，请重新运行 `go install` 并指定新版本号或使用 `@latest` :

```
$ go install github.com/rakyll/hey@latest
```

Of course, you don't need to leave programs installed via `go install` in the `go/bin` directory; they are regular executable binaries and can be stored anywhere on your computer. Likewise, you don't have to distribute programs written in Go using `go install`; you can put a binary up for download. However, `go install` is convenient for Go developers, and it has become the method of choice for distributing third-party developer tools.

当然，您无需将通过 `go install` 安装的程序保留在 `go/bin` 目录中；它们是普通的二进制可执行文件，可以存储在计算机上的任何位置。同样，您也不必使用 `go install` 分发用 Go 编写的程序；您可以提供一个可供下载的二进制文件。然而，`go install` 对 Go 开发者来说非常方便，并且已成为分发第三方开发者工具的首选方法。

Improving Import Formatting with goimports

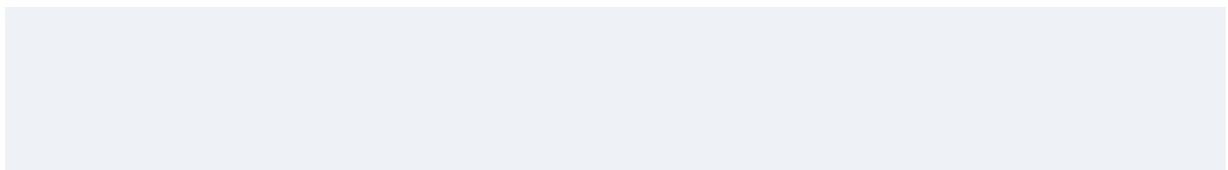
使用 goimports 改进导入格式

An enhanced version of `go fmt` called `goimports` also cleans up your import statements. It puts them in alphabetical order, removes unused imports, and attempts to guess any unspecified imports. Its guesses are sometimes inaccurate, so you should insert imports yourself.

`go fmt` 的增强版 `goimports` 也能清理你的导入语句。它会按字母顺序排列导入语句，移除未使用的导入，并尝试猜测任何未指定的导入。它的猜测有时并不准确，因此你应该手动添加导入语句。

You can download `goimports` with the command `go install golang.org/x/tools/cmd/goimports@latest`. You run it across your project with this command:

您可以使用命令 `go install golang.org/x/tools/cmd/goimports@latest` 下载 `goimports`。您可以使用以下命令在您的项目中运行它：



```
$ goimports -l -w .
```

The `-l` flag tells `goimports` to print the files with incorrect formatting to the console. The `-w` flag tells `goimports` to modify the files in place. The `.` specifies the files to be scanned: everything in the current directory and all its subdirectories.

`-l` 标志指示 `goimports` 将格式错误的文件打印到控制台。`-w` 标志指示 `goimports` 就地修改文件。`.` 指定要扫描的文件：当前目录及其所有子目录中的所有内容。

NOTE

笔记

The packages under `golang.org/x` are part of the Go Project but outside the main Go tree. While useful, they are developed under looser compatibility requirements than the Go standard library and may introduce backward-breaking changes. Some packages in the standard library, such as the `context` package that is covered in [Chapter 14](#), started out in `golang.org/x`. The `pkgsite` tool that was covered in [“Documenting Your Code with Go Doc Comments”](#) is also located there. You can see the other packages in the [“Sub-repositories” section](#).

`golang.org/x` 下的包属于 Go 项目，但位于 Go 主库之外。虽然它们很有用，但其开发兼容性要求比 Go 标准库宽松，可能会引入不兼容的变更。标准库中的一些包，例如第 14 章中介绍的 `context` 包，最初位于 `golang.org/x` 下。在“使用 Go 文档注释编写代码”一文中介绍的 `pkgsite` 工具也位于此处。您可以在“子仓库”部分查看其他包。

Using Code-Quality Scanners

使用条形码质量扫描器

Back in [“go vet”](#), you looked at the built-in tool `go vet`, which scans source code for common programming errors. Many third-party tools can check code style and scan for potential bugs that are missed by `go vet`. These tools are often called *linters*.¹ In addition to likely programming errors, some of the changes suggested by these tools include properly naming variables, formatting error messages, and placing comments on public methods and types. These aren't errors since they don't keep your programs from compiling or make your program run incorrectly, but they do flag situations where you are writing nonidiomatic code.

在“go vet”中，您查看了内置工具 `go vet`，它可以扫描源代码以查找常见的编程错误。许多第三方工具可以检查代码风格并扫描 `go vet` 遗漏的潜在错误。这些工具通常被称为代码检查器（linter）。¹ 除了常见的编程错误之外，这些工具还会建议进行一些更改，例如正确命名变量、格式化错误消息以及在公共方法和类型上添加注释。这些并非错误，因为它们不会导致程序无法编译或运行错误，但它们确实会标记出您编写不符合惯用语法规范的代码的情况。

When you add linters to your build process, follow the old maxim “trust, but verify.” Since the kinds of issues that linters find are fuzzier, they

sometimes have false positives and false negatives. This means you don't *have* to make the changes that they suggest, but you should take the suggestions seriously. Go developers expect code to look a certain way and follow certain rules, and if your code does not, it sticks out.

在构建过程中添加代码检查工具时，请遵循“信任，但要验证”这条古老的格言。由于代码检查工具发现的问题类型比较模糊，因此有时会出现误报和漏报。这意味着您不必完全按照它们建议的修改，但应该认真对待这些建议。Go 开发者期望代码符合特定的格式和规则，如果您的代码不符合这些要求，就会显得格格不入。

If you find a linter's suggestion to be unhelpful, each linting tool allows you to add a comment to your source code that blocks the errant result (the format of the comment varies from linter to linter; check each tool's documentation to learn what to write). Your comment should also include an explanation of why you are ignoring the linter's finding, so code reviewers (and future you, when you look back on your source code in six months) understand your reasoning.

如果您发现代码检查工具的建议没有帮助，每个检查工具都允许您在源代码中添加注释来阻止错误结果（注释格式因检查工具而异；请查阅各工具的文档以了解具体编写方式）。您的注释还应包含您忽略检查工具建议的原因的解释，以便代码审查人员（以及六个月后您自己回顾源代码时）能够理解您的理由。

staticcheck

静态检查

If you had to pick one third-party scanner, use `staticcheck`. It is supported by many companies that are active in the Go community, includes more than 150 code-quality checks, and tries to produce few to no false positives. It is installed via `go install honnef.co/go/tools/cmd/staticcheck@latest`. Invoke it with `staticcheck ./...` to examine your module.

如果必须选择一款第三方扫描器，请使用 `staticcheck`。它得到了许多活跃于 Go 社区的公司的支持，包含超过 150 项代码质量检查，并且力求将误报率降至最低。它可以通过 `go install honnef.co/go/tools/cmd/staticcheck@latest` 安装。使用 `staticcheck ./...` 调用它来检查您的模块。

Here's an example of something that `staticcheck` finds that `go vet` does not:

以下是 `staticcheck` 发现而 `go vet` 没有发现的内容示例：

```
package main

import "fmt"
```



```
func main() {  
    s := fmt.Sprintf("Hello")  
    fmt.Println(s)  
}
```

(You can also find this code in the *sample_code/staticcheck_test* directory in the [Chapter 11 repository](#).)

(您也可以在第 11 章代码库的 *sample_code/staticcheck_test* 目录中找到这段代码。)

If you run `go vet` on this code, it doesn't find anything wrong. But, `staticcheck` notices a problem:

如果使用 `go vet` 对这段代码进行测试，它不会发现任何问题。但是，`staticcheck` 会发现一个问题：

```
$ staticcheck ./...  
main.go:6:7: unnecessary use of fmt.Sprintf  
(S1039)
```

Pass the code in parentheses to `staticcheck` with the `-explain` flag for an explanation of the issue:

将括号中的代码传递给 `staticcheck`，并加上 `-explain` 标志，以获取问题说明：

```
$ staticcheck -explain S1039
```

```
Unnecessary use of fmt.Sprintf
```

```
Calling fmt.Sprintf with a single string argument  
is unnecessary  
and identical to using the string directly.
```

```
Available since  
2020.1
```

```
Online documentation  
https://staticcheck.io/docs/checks#S1039
```

Another common issue that `staticcheck` finds is unused assignments to variables. While the Go compiler requires all variables to be read *once*, it doesn't check that *every* value assigned to a variable is read. It is a common practice to reuse an `err` variable when there are multiple function calls within a function. If you forget to write `if err != nil` after one of those function invocations, the compiler won't be able to help you. However, `staticcheck` can. This code compiles without a problem:

`staticcheck` 发现的另一个常见问题是未使用的变量赋值。虽然 Go 编译器要求所有变量只能读取一次，但它不会检查是否读取了赋给变量的每个值。当一个函数内部有多个函数调用时，重用 `err` 变量是一种常见的做法。如果您忘记在其中一个函数调用后写入 `if err != nil`，编译器将无法帮您解决问题。但是，`staticcheck` 可以。以下代码可以编译通过：

```
func main() {  
    err := returnErr(false)  
    if err != nil {  
        fmt.Println(err)  
    }  
    err = returnErr(true)  
    fmt.Println("end of program")  
}
```

(This code is in the [Chapter 11 repository](#) in the *sample_code/check_err* directory.)

(这段代码位于第 11 章代码库的 `sample_code/check_err` 目录中。)

Running `staticcheck` finds the mistake:

运行 `staticcheck` 可以发现错误：

```
$ staticcheck ./...  
main.go:13:2: this value of err is never used  
(SA4006)  
main.go:13:8: returnErr doesn't have side effects  
and its return value is  
ignored (SA4017)
```

There are two related issues on line 13. The first is that the error returned by `returnErr` is never read. The second is that the `returnErr` function's output (the error) is being ignored.

第 13 行存在两个相关问题。第一个问题是 `returnErr` 返回的错误信息没有被读取。第二个问题是 `returnErr` 函数的输出（即错误信息）被忽略了。

revive

复活

Another good linting option is [revive](#). It is based on `golint`, a tool that used to be maintained by the Go team. Install `revive` with the command `go install github.com/mgechev/revive@latest`. By default, it enables only the rules that were present in `golint`. It can find style and code-quality issues like exported identifiers that don't have comments,

variables that don't follow naming conventions, or error return values that aren't the last return value.

另一个不错的代码检查工具是 `revive`。它基于 `golint`，后者曾由 Go 团队维护。使用命令 `go install github.com/mgechev/revive@latest` 安装 `revive`。默认情况下，它仅启用 `golint` 中存在的规则。它可以检测代码风格和质量问题，例如导出的标识符缺少注释、变量命名不符合规范，或者错误返回值不是最后一个返回值。

With a configuration file, you can turn on many more rules. For example, to enable a check for shadowing of universe block identifiers, create a file named `built_in.toml` with the following contents:

通过配置文件，您可以启用更多规则。例如，要启用对 Universe 块标识符的遮蔽检查，请创建一个名为 `built_in.toml` 的文件，并添加以下内容：

```
[rule.redefines-builtin-id]
```

If you scan the following code:

如果您扫描以下二维码：

```
package main
```

```
import "fmt"

func main() {
    true := false
    fmt.Println(true)
}
```

you'll get this warning:

你会收到以下警告：

```
$ revive -config built_in.toml ./...
main.go:6:2: assignment creates a shadow of built-
in identifier true
```

(You can also find this code in the [Chapter 11 repository](#) in the *sample_code/revive_test* directory.)

(您也可以在第 11 章代码库的 *sample_code/revive_test* 目录中找到这段代码。)

Other rules that can be enabled are focused on opinionated code organization, like limiting the number of lines in a function or number of public structs in a file. There are even rules for evaluating the complexity of

the logic in a function. Check out the [revive documentation](#) and its [supported rules](#).

其他可启用的规则侧重于规范代码组织，例如限制函数中的行数或文件中公共结构体的数量。甚至还有一些规则用于评估函数逻辑的复杂度。请查看 `revive` 文档及其支持的规则。

golangci-lint

golangci-lint

Finally, if you'd rather take the buffet approach to tool selection, there's [golangci-lint](#). It is designed to make it as efficient as possible to configure and run over 50 code-quality tools, including `go vet`, `staticcheck`, and `revive`.

最后，如果您更喜欢自助餐式的工具选择方式，那么还有 `golangci-lint`。它的设计旨在尽可能高效地配置和运行 50 多种代码质量工具，包括 `go vet`、`staticcheck` 和 `revive`。

While you can use `go install` to install `golangci-lint`, it is recommended that you download a binary version instead. Follow the installation instructions on the [website](#). Once it is installed, you run `golangci-lint`:

虽然您可以使用 `go install` 安装 `golangci-lint`，但建议您下载二进制版本。请按照网站上的安装说明进行操作。安装完成后，运行 `golangci-lint`：

```
$ golangci-lint run
```

Back in [“Unused Variables”](#), you looked at a program with variables set to values that were never read, and I mentioned that `go vet` and the go compiler were unable to detect these issues. Neither `staticcheck` nor `revive` catches this problem. However, one of the tools bundled with `golangci-lint` does:

在“未使用的变量”一节中，你看到了一个程序，其中变量被设置为从未读取的值，我提到过 `go vet` 和 Go 编译器都无法检测到这些问题。`staticcheck` 和 `revive` 也无法捕获此问题。但是，`golangci-lint` 附带的某个工具可以：

```
$ golangci-lint run
main.go:6:2: ineffectual assignment to x
(ineffassign)
    x := 10
    ^
main.go:9:2: ineffectual assignment to x
(ineffassign)
```



```
x = 30
^
```

You can also use `golangci-lint` to provide shadowing checks that go beyond what `revive` can do. Configure `golangci-lint` to detect shadowing of both universe block identifiers and identifiers within your own code by putting the following configuration into a file named `.golangci.yml` in the directory where you run `golangci-lint`:

您还可以使用 `golangci-lint` 来提供超出 `revive` 功能的遮蔽检查。要配置 `golangci-lint` 以检测 universe 块标识符和您自身代码中的标识符的遮蔽情况，请将以下配置添加到运行 `golangci-lint` 的目录中名为 `.golangci.yml` 的文件中：

```
linters:
  enable:
    - govet
    - predeclared

linters-settings:
  govet:
    check-shadowing: true
  settings:
    shadow:
```

```
strict: true
enable-all: true
```

With these settings, running `golangci-lint` on this code

使用这些设置，运行 `golangci-lint` 此代码

```
package main

import "fmt"

var b = 20

func main() {
    true := false
    a := 10
    b := 30
    if true {
        a := 20
        fmt.Println(a)
    }
    fmt.Println(a, b)
}
```

detects the following issues:

检测到以下问题:

```
$ golangci-lint run
main.go:5:5: var `b` is unused (unused)
var b = 20
    ^

main.go:10:2: shadow: declaration of "b" shadows
declaration at line 5 (govet)
    b := 30
    ^

main.go:12:3: shadow: declaration of "a" shadows
declaration at line 9 (govet)
    a := 20
    ^

main.go:8:2: variable true has same name as
predeclared identifier (predeclared)
    true := false
    ^
```

(You can find both `golangci-lint` code samples in the [Chapter 11 repository](#) in the `sample_code/golangci-lint_test` directory.)

(您可以在第 11 章代码库的 `sample_code/golangci-lint_test` 目录中找到这两个 `golangci-lint` 代码示例。)

Because `golangci-lint` runs so many tools (as of this writing, it runs 7 different tools by default and allows you to enable more than 50 more), it's inevitable that your team may disagree with some of its suggestions.

Review the [documentation](#) to understand what each tool can do. Once you come to agreement on which linters to enable, update the `.golangci.yml` file at the root of your module and commit it to source control. Check out the [documentation](#) for the file format.

由于 `golangci-lint` 运行着许多工具（截至撰写本文时，它默认运行 7 种不同的工具，并允许您启用 50 多种其他工具），您的团队难免会对它的一些建议持有不同意见。请查阅文档以了解每个工具的功能。一旦就要启用哪些代码检查工具达成一致，请更新模块根目录下的 `.golangci.yml` 文件并将其提交到源代码控制系统。有关文件格式，请参阅相关文档。

NOTE

笔记

While `golangci-lint` allows you to have a configuration file in your home directory, don't put one there if you are working with other developers. Unless you enjoy adding hours of silly arguments to your code reviews, you want to make sure that everyone is using the same code-quality tests and formatting rules.

虽然 `golangci-lint` 允许你在主目录中放置配置文件，但如果你要与其他开发人员合作，请不要这样做。除非你喜欢在代码审查中加入数小时的无谓争论，否则你需要确保每个人都使用相同的代码质量测试和格式规则。

I recommend that you start using `go vet` as a required part of your automated build process. Add `staticcheck` next since it produces few false positives. When you are interested in configuring tools and setting

code-quality standards, look at `revive`, but be aware that it might have false positives and false negatives, so you can't require your team to fix every issue it reports. Once you are used to their recommendations, try out `golangci-lint` and tweak its settings until it works for your team.

我建议您将 `go vet` 作为自动化构建流程的必要组成部分。接下来添加 `staticcheck`, 因为它误报率较低。如果您对配置工具和设置代码质量标准感兴趣, 可以考虑使用 `revive`, 但请注意, 它可能存在误报和漏报, 因此您不能要求团队修复它报告的每个问题。熟悉其建议后, 可以尝试使用 `golangci-lint` 并调整其设置, 直到它适合您的团队为止。

Using govulncheck to Scan for Vulnerable Dependencies

使用 govulncheck 扫描易受攻击的依赖项

One kind of code quality isn't enforced by the tools you've looked at so far: software vulnerabilities. Having a rich ecosystem of third-party modules is fantastic, but clever hackers find security vulnerabilities in libraries and exploit them. Developers patch these bugs when they are reported, but how

do you ensure that the software that uses a vulnerable version of a library is updated to the fixed version?

有一种代码质量问题，你目前接触的工具无法有效解决：软件漏洞。拥有丰富的第三方模块生态系统固然很好，但聪明的黑客总能找到库中的安全漏洞并加以利用。开发者会在漏洞被报告后进行修复，但如何确保使用存在漏洞的库版本的软件能够及时更新到修复后的版本呢？

The Go team has released a tool called `govulncheck` to address this situation. It scans through your dependencies and finds known vulnerabilities in both the standard library and in third-party libraries imported into your module. These vulnerabilities are reported in a [public database](#) maintained by the Go team. You can install it with this:

Go 团队发布了一个名为 `govulncheck` 的工具来解决这个问题。它会扫描你的依赖项，并查找标准库和导入到模块中的第三方库中已知的漏洞。这些漏洞会被记录在一个由 Go 团队维护的公共数据库中。你可以使用以下命令安装它：

```
$ go install  
golang.org/x/vuln/cmd/govulncheck@latest
```

Let's take a look at a small program to see the vulnerability checker in action. First, download the [repository](#). The source code in *main.go* is very simple. It imports a third-party YAML library and uses it to load a small YAML string into a struct:

让我们来看一个小程序，了解一下漏洞检测器是如何工作的。首先，下载代码仓库。main.go 中的源代码非常简单。它导入了一个第三方 YAML 库，并使用它来将一小段 YAML 字符串加载到一个结构体中：

```
func main() {
    info := Info{}

    err := yaml.Unmarshal([]byte(data), &info)
    if err != nil {
        fmt.Printf("error: %v\n", err)
        os.Exit(1)
    }
    fmt.Printf("%+v\n", info)
}
```

The *go.mod* file contains the required modules and their versions:

go.mod 文件包含所需的模块及其版本：

```
module github.com/learning-go-book-2e/vulnerable
```

```
go 1.20
```

```
require gopkg.in/yaml.v2 v2.2.7
```

```
require gopkg.in/check.v1 v1.0.0-20201130134442-  
10cb98267c6c // indirect
```

Let's see what happens when you run `govulncheck` on this project:

让我们看看在这个项目上运行 `govulncheck` 会发生什么：

```
$ govulncheck ./...
```

```
Using go1.21 and govulncheck@v1.0.0 with  
vulnerability data from  
    https://vuln.go.dev (last modified 2023-07-27  
20:09:46 +0000 UTC).
```

```
Scanning your code and 49 packages across 1  
dependent module  
    for known vulnerabilities...
```

```
Vulnerability #1: GO-2020-0036
```

```
    Excessive resource consumption in YAML parsing  
in gopkg.in/yaml.v2
```

```
    More info: https://pkg.go.dev/vuln/GO-2020-0036
```

```
    Module: gopkg.in/yaml.v2
```



```
Found in: gopkg.in/yaml.v2@v2.2.7
Fixed in: gopkg.in/yaml.v2@v2.2.8
Example traces found:
    #1: main.go:25:23: vulnerable.main calls
yaml.Unmarshal

Your code is affected by 1 vulnerability from 1
module.
```

This module is using an old and vulnerable version of the YAML package. `govulncheck` helpfully gives the exact line in the codebase that calls the problematic code.

此模块使用了旧版本且存在安全漏洞的 YAML 包。 `govulncheck` 可以清晰地指出代码库中调用问题代码的确切行号。

NOTE

笔记

If `govulncheck` knows there a vulnerability in a module that your code uses, but can't find an explicit call to the buggy part of the module, you'll get a less severe warning. The message informs you of the library's vulnerability and what version resolves the issue, but it will also let you know that your module is likely not affected.

如果 `govulncheck` 检测到您的代码使用的某个模块存在漏洞，但找不到对该模块漏洞部分的显式调用，您将收到一个不太严重的警告。该消息会告知您库的漏洞以及修复该问题的版本，同时也会告知您您的模块可能不受影响。

Let's update to a fixed version and see if that solves the problem:

我们更新到修复版本，看看能否解决问题：

```
$ go get -u=patch gopkg.in/yaml.v2
go: downloading gopkg.in/yaml.v2 v2.2.8
go: upgraded gopkg.in/yaml.v2 v2.2.7 => v2.2.8
$ govulncheck ./...
Using go1.21 and govulncheck@v1.0.0 with
vulnerability data from
    https://vuln.go.dev (last modified 2023-07-27
20:09:46 +0000 UTC).

Scanning your code and 49 packages across 1
dependent module
    for known vulnerabilities...

No vulnerabilities found.
```

Remember, you should always strive for the smallest possible change to your project's dependencies since that makes it less likely that a change in a dependency breaks your code. For that reason, update to the most recent patch version for v2.2.x, which is v2.2.8. When `govulncheck` is run again, there are no known issues.

请记住，您应该始终力求对项目依赖项进行尽可能小的更改，因为这样可以降低依赖项更改导致代码出错的可能性。因此，请将 v2.2.x 更新到最新的补丁版本，即 v2.2.8。再次运行 `govulncheck` 时，未发现任何已知问题。

While `govulncheck` currently requires a `go install` to download it, it likely will be added to the standard toolset eventually. In the meantime, be sure to install and run it against your projects as a regular part of your builds. You can learn more about it in the [blog.post](#) that announced it.

虽然目前下载 `govulncheck` 需要 `go install`，但它最终可能会被添加到标准工具集中。在此期间，请务必将其安装并作为构建流程的常规步骤运行。您可以在发布该工具的博文中了解更多信息。

Embedding Content into Your Program

将内容嵌入到您的程序中

Many programs are distributed with directories of support files; you might have web page templates or some standard data that's loaded when a program starts. If a Go program needs support files, you could include a directory of files, but this takes away one of the advantages of Go: its ability to compile to a single binary that's easy to ship and distribute.

However, there's another option. You can embed the contents of the files within your Go binary by using `go:embed` comments.

许多程序都会附带支持文件目录；例如网页模板或程序启动时加载的标准数据。如果 Go 程序需要支持文件，您可以包含一个文件目录，但这会失去 Go 的一个优势：它能够编译成易于打包和分发的单个二进制文件。不过，还有另一种选择。您可以使用 `go:embed` 注释将这些文件的内容嵌入到 Go 二进制文件中。

You can find a program demonstrating embedding on GitHub in the `sample_code/embed_passwords` directory in the [Chapter 11 repository](#). It checks to see if a password is one of the 10,000 most commonly used passwords. Rather than write that list of passwords directly into the source code, you're going to embed it.

您可以在 GitHub 上第 11 章代码库的 `sample_code/embed_passwords` 目录中找到一个演示嵌入功能的程序。该程序会检查密码是否属于最常用的 10,000 个密码之一。您无需将密码列表直接写入源代码，而是将其嵌入到代码中。

The code in `main.go` is straightforward:

`main.go` 中的代码很简单：

```
package main
```

```

import (
    _ "embed"
    "fmt"
    "os"
    "strings"
)

//go:embed passwords.txt
var passwords string

func main() {
    pwds := strings.Split(passwords, "\n")
    if len(os.Args) > 1 {
        for _, v := range pwds {
            if v == os.Args[1] {
                fmt.Println("true")
                os.Exit(0)
            }
        }
        fmt.Println("false")
    }
}

```

You must do two things to enable embedding. First, the `embed` package must be imported. The Go compiler uses this import as a flag to indicate that embedding should be enabled. Because this sample code isn't referring to anything exported from the `embed` package, you use a blank import,

which was discussed in [“Avoiding the init Function if Possible”](#). The only symbol exported from `embed` is `FS`. You’ll see it in the next example.

要启用嵌入，您必须执行两项操作。首先，必须导入 `embed` 包。Go 编译器会将此导入用作启用嵌入的标志。由于此示例代码没有引用 `embed` 包中导出的任何内容，因此您可以使用空导入，这在“尽可能避免使用 `init` 函数”一节中已讨论过。`embed` 包中导出的唯一符号是 `FS`。您将在下一个示例中看到它。

Next, you place a magic comment directly before each package-level variable that holds the contents of a file. This comment must start with `go:embed`, with no space between the slashes and `go:embed`. The comment must also be on the line directly before the variable. (Technically, it is legal to have blank lines or other, nonmagic comments between the embedding comment and the variable declaration, but don’t do it.) In this sample, you are embedding the contents of *passwords.txt* into the package-level variable named `passwords`. It is idiomatic to treat a variable with an embedded value as immutable. As mentioned earlier, you can embed into only a package-level variable. The variable must be of type `string`, `[]byte`, or `embed.FS`. If you have a single file, it’s simplest to use `string` or `[]byte`.

接下来，您需要在每个用于存储文件内容的包级变量前添加一个魔法注释。该注释必须以 `go:embed` 开头，斜杠和 `go:embed` 之间不能有空格。注释还必须位于变量声明的前一行。（理论上，嵌入注释和

变量声明之间可以有空行或其他非魔法注释，但请不要这样做。）在本示例中，您将 `passwords.txt` 的内容嵌入到名为 `passwords` 的包级变量中。通常的做法是将包含嵌入值的变量视为不可变变量。如前所述，您只能将内容嵌入到包级变量中。该变量的类型必须是 `string`、`[]byte` 或 `embed.FS`。如果只有一个文件，最简单的方法是使用 `string` 或 `[]byte`。

If you need to place one or more directories of files into your program, use a variable of type `embed.FS`. This type implements three interfaces defined in the `io/fs` package: `FS`, `ReadDirFS`, and `ReadFileFS`. This allows an instance of `embed.FS` to represent a virtual filesystem. The following program provides a simple command-line help system. If you don't provide a help file, it lists all available files. If you specify a file that's not present, it returns an error:

如果您需要在程序中放置一个或多个文件目录，请使用 `embed.FS` 类型的变量。此类型实现了 `io/fs` 包中定义的三个接口：`FS`、`ReadDirFS` 和 `ReadFileFS`。这使得 `embed.FS` 的实例可以表示一个虚拟文件系统。以下程序提供了一个简单的命令行帮助系统。如果您没有提供帮助文件，它将列出所有可用的文件。如果您指定了一个不存在的文件，它将返回错误：

```
package main
```

```
import (
```

```

    "embed"
    "fmt"
    "io/fs"
    "os"
    "strings"
)

//go:embed help
var helpInfo embed.FS

func main() {
    if len(os.Args) == 1 {
        printHelpFiles()
        os.Exit(0)
    }
    data, err := helpInfo.ReadFile("help/" +
os.Args[1])
    if err != nil {
        fmt.Println(err)
        os.Exit(1)
    }
    fmt.Println(string(data))
}

```

You can find this code, along with the sample help files, in the *sample_code/help_system* directory in the [Chapter 11 repository](#).

您可以在第 11 章存储库的 `sample_code/help_system` 目录中找到此代码以及示例帮助文件。

Here's the output when you build and run this program:

以下是构建并运行此程序后的输出结果：

```
$ go build
$ ./help_system
contents:
advanced/topic1.txt
advanced/topic2.txt
info.txt

$ ./help_system advanced/topic1.txt
This is advanced topic 1.

$ ./help_system advanced/topic3.txt
open help/advanced/topic3.txt: file does not exist
```

You should notice a couple of things. First, you no longer need to use a blank import for `embed`, since you are using `embed.FS`. Second, the directory name is part of the filesystem that's embedded. The users of this program don't enter the "help/" prefix, so you have to prepend it in the call to `ReadFile`.

您应该注意以下几点。首先，由于您使用的是 `embed.FS`，因此不再需要为 `embed` 进行空白导入。其次，目录名是嵌入式文件系统的一部分。该程序的用户不会输入“help/”前缀，因此您需要在调用 `ReadFile` 时添加该前缀。

The `printHelpFiles` function shows how you can treat an embedded virtual filesystem just like a real one:

`printHelpFiles` 函数展示了如何像处理真实文件系统一样处理嵌入式虚拟文件系统：

```
func printHelpFiles() {
    fmt.Println("contents:")
    fs.WalkDir(helpInfo, "help",
        func(path string, d fs.DirEntry, err
error) error {
        if !d.IsDir() {
            _, fileName, _ :=
strings.Cut(path, "/")
            fmt.Println(fileName)
        }
        return nil
    })
}
```

You use the `WalkDir` function in `io/fs` to walk through the embedded filesystem. `WalkDir` takes in an instance of `fs.FS`, a path to start at, and a function. This function is called for every file and directory in the filesystem, starting from the specified path. If the `fs.DirEntry` is not a directory, you print out its full pathname, removing the `help/` prefix by using `strings.Cut`.

您可以使用 `io/fs` 中的 `WalkDir` 函数遍历嵌入式文件系统。
`WalkDir` 函数接受一个 `fs.FS` 实例、一个起始路径和一个函数作为参数。该函数会从指定的路径开始，对文件系统上的每个文件和目录调用一次。如果 `fs.DirEntry` 不是一个目录，则打印出其完整路径名，并使用 `strings.Cut` 移除 `help/` 前缀。

There are a few more things to know about file embedding. While all the examples have been text files, you can embed binary files as well. You can also embed multiple files or directories into a single `embed.FS` variable by specifying their names, separated by spaces. When embedding a file or directory that has a space in its name, put the name in quotes.

关于文件嵌入，还有一些需要了解的内容。虽然所有示例都是文本文件，但您也可以嵌入二进制文件。您还可以通过指定文件名（以空格分隔）将多个文件或目录嵌入到单个 `embed.FS` 变量中。如果文件或目录名称中包含空格，请将其用引号括起来。

In addition to exact file and directory names, you can use wildcards and ranges to specify the names of the files and directories you want to embed. The syntax is defined in the [documentation for the Match function in the path package in the standard library](#), but it follows common conventions. For example, `*` matches 0 or more characters, and `?` matches a single character.

除了精确的文件和目录名之外，您还可以使用通配符和范围来指定要嵌入的文件和目录的名称。语法定义在标准库 `path` 包的 `Match` 函数文档中，但遵循一些常见的约定。例如，`*` 匹配 0 个或多个字符，而 `?` 匹配单个字符。

All embedding specifications, whether or not they use match patterns, are checked by the compiler. If they aren't valid, compilation fails. Here are the ways a pattern can be invalid:

所有嵌入规范，无论是否使用匹配模式，都会由编译器进行检查。如果无效，编译将失败。以下是模式无效的几种情况：

- If the specified name or pattern doesn't match a file or directory
如果指定的名称或模式与文件或目录不匹配
- If you specify multiple filenames or patterns for a `string` or `[]byte` variable
如果您为 `string` 或 `[]byte` 变量指定多个文件名或模式

- If you specify a pattern for a `string` or `[]byte` variable and it matches more than one file

如果您为 `string` 或 `[]byte` 变量指定了一个模式，并且该模式匹配多个文件，则需要指定一个文件。

Embedding Hidden Files

嵌入隐藏文件

Including files in a directory tree that start with `.` or `_` is a little complicated. Many operating systems consider these to be hidden files, so they are not included by default when a directory name is specified.

However, you can override this behavior in two ways. The first is to put `/*` after the name of a directory you want to embed. This will include all hidden files within the root directory, but it will not include hidden files in its subdirectories. To include all hidden files in all subdirectories, put `all:` before the name of the directory.

包含目录树中以 `.` 或 `_` 开头的文件有点复杂。许多操作系统将这些文件视为隐藏文件，因此默认情况下，指定目录名时不会包含它们。但是，您可以通过两种方式覆盖此行为。第一种方法是在要嵌入的目录名称后添加 `/*`。这将包含根目录中的所有隐藏文件，但不会包含其子目录中的隐藏文件。要包含所有子目录中的所有隐藏文件，请在目录名称前添加 `all:`。

This sample program (which you can find in the *sample_code/embed_hidden* directory in the [Chapter 11 repository](#)) makes this easier to understand. In the sample, the directory *parent_dir* contains two files, *.hidden* and *visible*, and one subdirectory, *child_dir*. The *child_dir* subdirectory contains two files, *.hidden* and *visible*.

这个示例程序（您可以在第 11 章代码库的 *sample_code/embed_hidden* 目录中找到）有助于理解这一点。在示例中，目录 *parent_dir* 包含两个文件：*.hidden* 和 *visible*，以及一个子目录 *child_dir*。*child_dir* 子目录也包含两个文件：*.hidden* 和 *visible*。

Here is the code for the program:

以下是程序代码：

```
//go:embed parent_dir
var noHidden embed.FS

//go:embed parent_dir/*
var parentHiddenOnly embed.FS

//go:embed all:parent_dir
var allHidden embed.FS

func main() {
    checkForHidden("noHidden", noHidden)
    checkForHidden("parentHiddenOnly",
```

```

parentHiddenOnly)
    checkForHidden("allHidden", allHidden)
}

func checkForHidden(name string, dir embed.FS) {
    fmt.Println(name)
    allFileNames := []string{
        "parent_dir/.hidden",
        "parent_dir/child_dir/.hidden",
    }
    for _, v := range allFileNames {
        _, err := dir.Open(v)
        if err == nil {
            fmt.Println(v, "found")
        }
    }
    fmt.Println()
}

```

The output of the program is shown here:

程序的输出结果如下所示:

```

noHidden

parentHiddenOnly
parent_dir/.hidden found

```

```
allHidden  
parent_dir/.hidden found  
parent_dir/child_dir/.hidden found
```

Using go generate

使用 go generate

The `go generate` tool is a little different, because it doesn't do anything by itself. When you run `go generate`, it looks for specially formatted comments in your source code and runs programs specified in those comments. While you could use `go generate` to run anything at all, it is most commonly used by developers to run tools that (unsurprisingly, given the name) generate source code. This could be from analyzing existing code and adding functionality or processing schemas and making source code out of it.

`go generate` 工具略有不同，因为它本身并不执行任何操作。运行 `go generate` 时，它会在源代码中查找特定格式的注释，并运行这些注释中指定的程序。虽然您可以使用 `go generate` 运行任何程序，但它最常用于开发人员运行生成源代码的工具（顾名思义）。这

些工具可以分析现有代码并添加功能，也可以处理模式并将其生成源代码。

A good example of something that can be automatically converted to code are [Protocol Buffers](#), sometimes called *protobufs*. Protobuf is a popular binary format that is used by Google to store and transmit data. When working with protobufs, you write a *schema*, which is a language-independent description of the data structure. Developers who want to write programs to interact with data in protobuf format run tools that process the schema and produce language-specific data structures to hold the data and language-specific functions to read and write data in protobuf format.

Protocol Buffers（有时也称为 protobuf）就是一个很好的例子，它可以自动转换为代码。Protobuf 是一种流行的二进制格式，被 Google 用于存储和传输数据。使用 protobuf 时，需要编写模式（schema），它是对数据结构的语言无关描述。想要编写程序来与 protobuf 格式的数据交互的开发者，需要运行一些工具来处理模式，生成特定语言的数据结构来存储数据，以及用于读写 protobuf 格式数据的语言特定函数。

Let's see how this works in Go. You can find a sample module in the [proto_generate repo](#). The module contains a protobuf schema file called *person.proto*:

让我们看看这在 Go 语言中是如何实现的。你可以在 proto_generate 代码库中找到一个示例模块。该模块包含一个名为 person.proto 的

protobuf schema 文件:

```
syntax = "proto3";

message Person {
    string name = 1;
    int32 id = 2;
    string email = 3;
}
```

While making a struct that implements `Person` would be easy, writing the code to convert back and forth from the binary format is difficult. Let's use tools from Google to do the hard work and invoke them with `go generate`. You need to install two things. The first is the `protoc` binary for your computer (see the [installation instructions](#)). Next, use `go install` to install the Go protobuf plug-ins:

创建一个实现 `Person` 的结构体很容易，但编写代码来实现二进制格式与二进制格式之间的双向转换却很困难。让我们使用 Google 的工具来完成这项繁重的工作，并使用 `go generate` 来调用它们。你需要安装两项内容。首先是适用于你计算机的 `protoc` 二进制文件（请参阅安装说明）。接下来，使用 `go install` 安装 Go protobuf 插件：

```
$ go install
```

```
google.golang.org/protobuf/cmd/protoc-gen-go@v1.28
```

In *main.go*, there is the magic comment that's processed by `go generate`:

在 *main.go* 文件中，有一条由 `go generate` 处理的神奇注释：

```
//go:generate protoc -I=. --go_out=.  
--go_opt=module=github.com/learning-go-book-  
2e/proto_generate  
--go_opt=Mperson.proto=github.com/learning-go-  
book-2e/proto_generate/data  
person.proto
```

(If you look at the source code in GitHub, you'll see this should be a single line. It's wrapped to fit the constraints of a printed page.)

(如果你查看GitHub上的源代码，你会发现这原本应该是一行代码。它被换行是为了适应打印页面的尺寸限制。)

Run `go generate` by typing the following:

输入以下内容运行 `go generate`：

```
$ go generate ./...
```

After running `go generate`, you'll see a new directory called *data* that contains a file named *person.pb.go*. It contains the source code for the `Person` struct, and some methods and functions that are used by the `Marshal` and `Unmarshal` functions in the `google.golang.org/protobuf/proto` module. You call these functions in your `main` function:

运行 `go generate` 后，你会看到一个名为 `data` 的新目录，其中包含一个名为 `person.pb.go` 的文件。它包含了 `Person` 结构体的源代码，以及 `Marshal` 和 `Unmarshal` 模块中的一些方法和函数。你可以在 `main` 函数中调用这些函数：

```
func main() {
    p := &data.Person{
        Name:  "Bob Bobson",
        Id:    20,
        Email: "bob@bobson.com",
    }
    fmt.Println(p)
    protoBytes, _ := proto.Marshal(p)
    fmt.Println(protoBytes)
    var p2 data.Person
    proto.Unmarshal(protoBytes, &p2)
```

```
    fmt.Println(&p2)
}
```

Build and run the program as usual:

像往常一样构建并运行程序：

```
$ go build
$ ./proto_generate
name:"Bob Bobson" id:20 email:"bob@bobson.com"
[10 10 66 111 98 32 66 111 98 115 111 110 16 20 26
14 98
    111 98 64 98 111 98 115 111 110 46 99 111 109]
name:"Bob Bobson" id:20 email:"bob@bobson.com"
```

Another tool commonly used with `go generate` is `stringer`. As I discussed in [“iota Is for Enumerations—Sometimes”](#), enumerations in Go lack many of the features that are found in other languages with enumerations. One of those features is automatically generating a printable name for each value in the enumeration. The `stringer` tool is used with `go generate` to add a `String` method to your enumeration’s values so they can be printed.

另一个常与 `go generate` 一起使用的工具是 `stringer`。正如我在“iota 用于枚举——有时”一文中讨论的那样，Go 语言中的枚举缺少许

多其他语言枚举所具备的功能。其中一个功能就是自动为枚举中的每个值生成可打印的名称。 `stringer` 工具与 `go generate` 一起使用，为枚举值添加 `String` 方法，以便可以打印这些值。

Install `stringer` with `go install golang.org/x/tools/cmd/stringer@latest`. The `sample_code/stringer_demo` directory in the [Chapter 11 repository](#) provides a very simple example of how to use `stringer`. Here's the source in `main.go`:

使用 `go install golang.org/x/tools/cmd/stringer@latest` 安装 `stringer`。第 11 章代码库中的 `sample_code/stringer_demo` 目录提供了一个非常简单的示例，演示如何使用 `stringer`。以下是 `main.go` 中的源代码：

```
type Direction int

const (
    _ Direction = iota
    North
    South
    East
    West
)

//go:generate stringer -type=Direction
```

```
func main() {  
    fmt.Println(North.String())  
}
```

Run `go generate ./...` and you'll see a new file generated called `direction_string.go`. Use `go build` to build the `string_demo` binary and when you run it, you'll get the output:

运行 `go generate ./...`，你会看到生成了一个名为 `direction_string.go` 的新文件。使用 `go build` 构建 `string_demo` 二进制文件，运行后你会得到以下输出：

```
North
```

You can configure `stringer` and its output in multiple ways. Arjun Mahishi has written a great [blog post](#) describing how to use `stringer` and customize its output.

您可以通过多种方式配置 `stringer` 及其输出。Arjun Mahishi 写了一篇很棒的博文，详细介绍了如何使用 `stringer` 并自定义其输出。

Working with go generate and Makefiles

使用 go generate 和 Makefiles

Since the job of `go generate` is to run other tools, you might wonder if it's worth using when you have a perfectly good Makefile in your project. The advantage of `go generate` is that it creates a separation of responsibilities. Use `go generate` commands to mechanically create source code, and use the Makefile to validate and compile source code.

由于 `go generate` 的作用是运行其他工具，您可能会疑惑，既然项目中已经有一个完善的 Makefile，是否还有必要使用它。`go generate` 的优势在于它实现了职责分离。使用 `go generate` 命令自动生成源代码，而使用 Makefile 来验证和编译源代码。

It is a best practice to commit the source code created by `go generate` to version control. (The sample projects in the [Chapter 11 repository](#) don't include generated source code so you can see `go generate` work.) This allows people browsing your source code to see everything that's invoked, even the generated parts. It also means they don't need to have tools like `protoc` installed in order to build your code.

最佳实践是将 `go generate` 生成的源代码提交到版本控制系统中。

(第 11 章代码库中的示例项目不包含生成的源代码，因此您可以查看

`go generate` 的运行情况。) 这样, 浏览您源代码的人可以看到所有被调用的部分, 包括生成的代码。这也意味着他们无需安装 `protoc` 之类的工具即可构建您的代码。

Checking in your generated source code technically means that you don't *need* to run `go generate` unless it will produce different output, such as processing a modified protobuf definition or an updated enumeration.

However, it's still a good idea to automate calling `go generate` before `go build`. Relying on a manual process is asking for trouble. Some generator tools, like `stringer`, include clever tricks to block compilation if you forget to rerun `go generate`, but that's not universal. You'll inevitably waste time during testing trying to understand why a change didn't show up before realizing that you forgot to invoke `go generate`. (I made this mistake multiple times before I learned my lesson.) Given this, it is best to add a `generate` step to your Makefile and make it a dependency of your `build` step.

从技术上讲, 提交生成的源代码意味着除非会产生不同的输出 (例如处理修改后的 protobuf 定义或更新后的枚举), 否则无需运行 `go generate`。但是, 最好还是在 `go build` 之前自动调用 `go generate`。依赖手动操作很容易出问题。一些生成器工具 (例如 `stringer`) 包含一些巧妙的技巧, 可以在忘记重新运行 `go generate` 时阻止编译, 但这并非普遍现象。在测试过程中, 你肯定会浪费时间去弄清楚为什么某个更改没有生效, 最后才意识到自己忘记调用 `go generate`。(我犯过好几次这样的错误才吸取教训。)

鉴于此，最好在 Makefile 中添加一个 `generate` 步骤，并将其作为 `build` 步骤的依赖项。

However, I would disregard this advice in two situations. The first is if invoking `go generate` on identical input produces source files with minor differences, such as a timestamp. A well-written `go generate` tool should produce identical output every time it's run on the same input, but there are no guarantees that every tool you need to use is well written. You don't want to keep on checking in new versions of files that are functionally identical, as they will clutter your version control system and make your code reviews noisier.

然而，在两种情况下，我会忽略这条建议。第一种情况是，如果对相同的输入调用 `go generate` 会生成一些只有细微差别的源文件，例如时间戳。一个编写良好的 `go generate` 工具每次对相同的输入运行时都应该产生完全相同的输出，但并不能保证你需要使用的每个工具都编写良好。你肯定不想不断提交功能完全相同的文件的新版本，因为这会使你的版本控制系统变得混乱，并增加代码审查的噪音。

The second situation is if `go generate` takes a very long time to complete. Fast builds are a feature of Go, because they allow developers to stay focused and get rapid feedback. If you are noticeably slowing down a build to generate identical files, the loss in developer productivity is not worth it. In both cases, all you can do is leave lots of comments to remind

people to rebuild when things change and hope that everyone on your team is diligent.

第二种情况是 `go generate` 构建耗时过长。快速构建是 Go 语言的一项特性，因为它可以让开发者保持专注并快速获得反馈。如果为了生成相同的文件而显著降低构建速度，那么由此造成的开发者效率损失是不值得的。在这两种情况下，你唯一能做的就是留下大量的注释，提醒大家在代码变更时重新构建，并寄希望于团队中的每个人都能认真负责。

Reading the Build Info Inside a Go Binary

读取 Go 二进制文件中的构建信息

As companies develop more of their own software, it is becoming increasingly common for them to want to understand exactly what they have deployed to their data centers and cloud environments, down to the version and the dependencies. You might wonder why you'd want to get this information from compiled code. After all, a company already has this information in version control.

随着企业自主开发软件的增多，他们越来越希望了解已部署到数据中心和云环境中的软件的具体版本和依赖关系。你可能会疑惑，为什么需要从编译后的代码中获取这些信息？毕竟，企业已经在版本控制系统中掌握了这些信息。

Companies with mature development and deployment pipelines can capture this information right before deploying a program, allowing them to be sure that the information is accurate. However, many, if not most, companies don't track the exact version of internal software that's deployed. In some cases, software can be deployed for years without being replaced, and no one remembers much about it. If a vulnerability is reported in a version of a third-party library, you need to either find some way to scan your deployed software and figure out what versions of third-party libraries are deployed, or redeploy everything just to be safe. In the Java world, this exact problem happened when a serious vulnerability was discovered in the popular Log4j library.

拥有成熟开发和部署流程的公司可以在部署程序之前获取这些信息，从而确保信息的准确性。然而，许多公司（如果不是大多数）并没有跟踪已部署的内部软件的确切版本。在某些情况下，软件可能部署多年而无需更新，以至于没有人记得它的具体情况。如果某个第三方库的某个版本被报告存在漏洞，您需要找到某种方法来扫描已部署的软件，并确定已部署的第三方库的版本，或者为了安全起见，重新部署所有内容。在 Java 世界中，当流行的 Log4j 库被发现存在严重漏洞时，就出现了这种情况。

Luckily, Go solves this problem for you. Every Go binary you create with `go build` automatically contains build information on what versions of what modules make up the binary, and also what build commands were used, what version control system was used, and what revision the code was at in your version control system. You can view this information with the `go version -m` command. The following shows the output for the `vulnerable` program when built on an Apple Silicon Mac:

幸运的是，Go 语言已经帮你解决了这个问题。你使用 `go build` 创建的每个 Go 二进制文件都会自动包含构建信息，包括构成该二进制文件的模块版本、使用的构建命令、使用的版本控制系统以及代码在你的版本控制系统中的修订版本。你可以使用 `go version -m` 命令查看这些信息。以下显示了在 Apple Silicon Mac 上构建 `vulnerable` 程序时的输出：

```
$ go build
go: downloading gopkg.in/yaml.v2 v2.2.7
$ go version -m vulnerable
vulnerable: go1.20
    path      github.com/learning-go-book-
2e/vulnerable
    mod      github.com/learning-go-book-
2e/vulnerable    (devel)
    dep      gopkg.in/yaml.v2  v2.2.7
h1:VUgggvou5XRW9mHwD/yXxIYSMtY0zoKQf/v...
```

```
build    -compiler=gc
build    CGO_ENABLED=1
build    CGO_CFLAGS=
build    CGO_CPPFLAGS=
build    CGO_CXXFLAGS=
build    CGO_LDFLAGS=
build    GOARCH=arm64
build    GOOS=darwin
build    vcs=git
build
vcs.revision=623a65b94fd02ea6f18df53afaaea3510cd1e611
build    vcs.time=2022-10-02T03:31:05Z
build    vcs.modified=false
```

Because this information is embedded into every binary, `govulncheck` is capable of scanning Go programs to check for libraries with known vulnerabilities:

由于此信息嵌入到每个二进制文件中，因此 `govulncheck` 能够扫描 Go 程序以检查是否存在已知漏洞的库：

```
$ govulncheck -mode binary vulnerable
Using govulncheck@v1.0.0 with vulnerability data
from
https://vuln.go.dev (last modified 2023-07-27)
```

```
20:09:46 +0000 UTC).
```

```
Scanning your binary for known vulnerabilities...
```

```
Vulnerability #1: GO-2020-0036
```

```
    Excessive resource consumption in YAML parsing  
in gopkg.in/yaml.v2
```

```
More info: https://pkg.go.dev/vuln/GO-2020-0036
```

```
Module: gopkg.in/yaml.v2
```

```
Found in: gopkg.in/yaml.v2@v2.2.7
```

```
Fixed in: gopkg.in/yaml.v2@v2.2.8
```

```
Example traces found:
```

```
    #1: yaml.Unmarshal
```

```
Your code is affected by 1 vulnerability from 1  
module.
```

Be aware that `govulncheck` can't track down exact lines of code when inspecting a binary. If `govulncheck` finds a problem in a binary, use `go version -m` to find out the exact deployed version, check the code out of version control, and run it again against the source code to pinpoint the issue.

请注意，`govulncheck` 在检查二进制文件时无法追踪到确切的代码行。如果 `govulncheck` 在二进制文件中发现问题，请使用 `go`

`version -m` 查找确切的已部署版本，将代码从版本控制系统中检出，然后再次针对源代码运行该工具以精确定位问题。

If you want to build your own tools to read the build information, look at the `debug/buildinfo` package in the standard library.

如果你想构建自己的工具来读取构建信息，请查看标准库中的 `debug/buildinfo` 包。

Building Go Binaries for Other Platforms

为其他平台构建 Go 二进制文件

One of the advantages of a VM-based language like Java, JavaScript, or Python is that you can take your code and get it to run on any computer where the virtual machine has been installed. This portability makes it easy for developers using these languages to build programs on a Windows or Mac computer and deploy it on a Linux server, even though the operating system and possibly the CPU architecture are different.

像 Java、JavaScript 或 Python 这样的基于虚拟机的语言的优势之一在于，你可以将代码移植到任何安装了虚拟机的计算机上运行。这种可移植性使得使用这些语言的开发者能够轻松地在 Windows 或 Mac 计

计算机上构建程序，并将其部署到 Linux 服务器上，即使操作系统甚至 CPU 架构可能有所不同。

Go programs are compiled to native code, so the generated binary is compatible with only a single operating system and CPU architecture. However, that doesn't mean that Go developers need to maintain a menagerie of machines (virtual or otherwise) to release on multiple platforms. The `go build` command makes it easy to *cross-compile*, or create a binary for a different operating system and/or CPU. When `go build` is run, the target operating system is specified by the `GOOS` environment variable. Similarly, the `GOARCH` environment variable specifies the CPU architecture. If you don't set them explicitly, `go build` defaults to using the values for your current computer, which is why you've never had to worry about these variables before.

Go 程序会被编译成本地代码，因此生成的二进制文件只能兼容单一的操作系统和 CPU 架构。但这并不意味着 Go 开发者需要维护大量的机器（无论是虚拟机还是其他机器）才能在多个平台上发布程序。`go build` 命令可以轻松实现交叉编译，即为不同的操作系统和/或 CPU 创建二进制文件。运行 `go build` 时，目标操作系统由环境变量 `GOOS` 指定。类似地，环境变量 `GOARCH` 指定 CPU 架构。如果您没有显式设置这些变量，`go build` 将默认使用您当前计算机的值，这就是为什么您以前无需担心这些变量的原因。

You can find the valid values and combinations for `GOOS` and `GOARCH` (sometimes pronounced “GOOSE” and “GORCH”) in the [installation documentation](#). Some of the supported operating systems and CPUs are a bit esoteric, and others might require some translation. For example, `darwin` refers to macOS (Darwin is the name of the macOS kernel), and `amd64` means 64-bit Intel-compatible CPUs.

您可以在安装文档中找到 `GOOS` 和 `GOARCH`（有时读作“GOOSE”和“GORCH”）的有效值和组合。某些支持的操作系统和 CPU 可能比较冷门，其他一些可能需要一些解释。例如，`darwin` 指的是 macOS（Darwin 是 macOS 内核的名称），而 `amd64` 指的是 64 位 Intel 兼容 CPU。

Let’s go back to the `vulnerable` program one last time. When using an Apple Silicon Mac (which has an ARM64 CPU), running `go build` defaults to `darwin` for `GOOS` and `arm64` for `GOARCH`. You can confirm this using the `file` command:

让我们最后再回顾一下 `vulnerable` 程序。在使用 Apple Silicon Mac（配备 ARM64 CPU）时，运行 `go build` 会默认执行 `GOOS` 对应的 `darwin` 和 `GOARCH` 对应的 `arm64`。您可以使用 `file` 命令来确认这一点：

```
$ go build
```

```
$ file vulnerable
vulnerable: Mach-O 64-bit executable arm64
```

Here is how to build a binary for Linux on 64-bit Intel CPUs:

以下是如何在 64 位 Intel CPU 上构建 Linux 二进制文件的方法：

```
$ GOOS=linux GOARCH=amd64 go build
$ file vulnerable
vulnerable: ELF 64-bit LSB executable, x86-64,
version 1 (SYSV),
    statically linked, Go
BuildID=IDHVCE8XQPpWluGpMXpX/4VU3GpRZEifN
    8TzUrT_6/1c30VcDYNVPfSSN-
zCkz/JsZSLAbWkxqIVhPkC5p5, with debug_info,
    not stripped
```

Using Build Tags

使用构建标签

When writing programs that need to run on multiple operating systems or CPU architectures, you sometimes need different code for different

platforms. You also might want to write a module that takes advantage of the latest Go features but is still backward compatible with older Go compilers.

编写需要在多种操作系统或 CPU 架构上运行的程序时，有时需要针对不同的平台编写不同的代码。此外，您可能还希望编写一个模块，既能利用最新的 Go 特性，又能向后兼容旧版本的 Go 编译器。

You can create targeted code in two ways. The first is to use the name of the file to indicate when the file should be included in the build. You do this by adding the target *GOOS* and *GOARCH*, separated by `_`, to the filename before `.go`. For example, if you have a file that you want to be compiled only on Windows, you'd name the file *something_windows.go*, but if you wanted it to be compiled only when building for ARM64 Windows, name the file *something_windows_arm64.go*.

您可以通过两种方式创建目标代码。第一种方式是使用文件名来指示何时应将该文件包含在构建过程中。具体做法是在文件名中的 `.go` 之前添加目标操作系统 (GOOS) 和架构 (GOARCH)，并用 `_` 分隔。例如，如果您希望某个文件仅在 Windows 系统上编译，则将其命名为 `something_windows.go`；如果您希望该文件仅在为 ARM64 Windows 构建时编译，则将其命名为 `something_windows_arm64.go`。

A *build tag* (also called a *build constraint*) is the other option you can use to specify when a file is compiled. Like embedding and generating, build tags

take advantage of a magic comment. In this case, it's `//go:build`. This comment must be placed on the line before the package declaration in your file.

构建标签（也称为构建约束）是另一种可用于指定文件编译时间的选项。与嵌入和生成类似，构建标签也利用了魔法注释。在本例中，魔法注释是 `//go:build`。此注释必须放在文件中包声明之前的行。

Build tags use boolean operators (`||`, `&&`, and `!`) and parentheses to specify exact build rules for architectures, operating systems, and Go versions. The build tag `//go:build (!darwin && !linux) || (darwin && !go1.12)`—which really appears in the Go standard library—specifies that the file should not be compiled on Linux or macOS, except it's OK to compile it on macOS if the Go version is 1.11 or earlier.

构建标签使用布尔运算符（`||`、`&&` 和 `!`）和括号来指定针对架构、操作系统和 Go 版本的具体构建规则。构建标签 `//go:build (!darwin && !linux) || (darwin && !go1.12)`（实际上出现在 Go 标准库中）指定该文件不应在 Linux 或 macOS 上编译，但如果 Go 版本为 1.11 或更早版本，则可以在 macOS 上编译。

Some meta build constraints are also available. The constraint `unix` matches any Unix-ish platform, and `cgo` matches if `cgo` is supported by the current platform and is enabled. (I cover `cgo` in [“Cgo Is for Integration, Not Performance”](#).)

还有一些元构建约束可用。约束 `unix` 匹配任何类 Unix 平台，而 `cgo` 则匹配当前平台支持且已启用的 `cgo`。（我在“Cgo 用于集成，而非性能”一文中介绍了 `cgo`。）

The question becomes when you should use filenames to indicate where to run code and when you should use build tags. Because build tags allow binary operators, you can specify a more specific set of platforms with them. The Go standard library sometimes takes a belt-and-suspenders approach. The package `internal/cpu` in the standard library has platform-specific source code for CPU feature detection. The file `internal/cpu/cpu_arm64_darwin.go` has a name that indicates that it is meant only for computers using Apple CPUs. It also has a `//go:build arm64 && darwin && !ios` line in the file to indicate that it should be compiled only when building for Apple Silicon Macs and not for iPhones or iPads. The build tags are able to specify the target platform with more detail, but following the filename convention makes it easy for a person to find the right file for a given platform.

问题在于何时应该使用文件名来指定代码运行位置，何时应该使用构建标签。由于构建标签支持二元运算符，因此可以使用它们来指定更具体的平台。Go 标准库有时会采用双保险的做法。标准库中的包 `internal/cpu` 包含用于 CPU 特性检测的平台特定源代码。文件 `internal/cpu/cpu_arm64_darwin.go` 的名称表明它仅适用于使用 Apple CPU 的计算机。该文件中还有一行 `//go:build arm64 && darwin && !ios`，表明它应该仅在为 Apple Silicon Mac 构建时编译，而不是

为 iPhone 或 iPad 构建。构建标签能够更详细地指定目标平台，但遵循文件名约定可以方便用户找到特定平台所需的文件。

In addition to the built-in build tags that represent Go versions, operating systems and CPU architectures, you can also use any string at all as a custom build tag. You can then control compilation of that file with the `-tags` command-line flag. For example, if you put `//go:build gopher` on the line before the package declaration in a file, it will not be compiled unless you include a `-tags gopher` flag as part of the `go build`, `go run`, or `go test` command.

除了代表 Go 版本、操作系统和 CPU 架构的内置构建标签之外，您还可以使用任何字符串作为自定义构建标签。然后，您可以使用 `-tags` 命令行标志来控制该文件的编译。例如，如果您在文件中包声明之前的行中添加 `//go:build gopher`，则除非您在 `go build`、`go run` 或 `go test` 命令中包含 `-tags gopher` 标志，否则该文件将不会被编译。

Custom build tags are surprisingly handy. If you have a source file that you don't want to build right now (perhaps it doesn't compile yet, or it's an experiment that's not ready to be included), it is idiomatic to skip over the file by putting `//go:build ignore` on the line before the package declaration. You will see another use for custom build tags when looking at integration tests in [“Using Integration Tests and Build Tags”](#).

自定义构建标签非常实用。如果您有一个源文件，但暂时不想构建（例如，它可能无法编译，或者它是一个尚未准备好包含的实验项目），通常的做法是在包声明之前添加 `//go:build ignore` 来跳过该文件。在“使用集成测试和构建标签”一节中，您将看到自定义构建标签的另一个用途。

WARNING

警告

When writing your build tags, make sure there isn't any whitespace between the `//` and `go:build`. If there is, Go will not consider it a build tag.

编写构建标签时，请确保 `//` 和 `go:build` 之间没有任何空格。如果有空格，Go 将不会将其视为构建标签。

Testing Versions of Go

测试 Go 语言版本

Despite Go's strong backward-compatibility guarantees, bugs do happen. It's natural to want to make sure that a new release doesn't break your programs. You also might get a bug report from a user of your library saying that your code doesn't work as expected on an older version of Go. One option is to install a secondary Go environment. For example, if you wanted to try out version 1.19.2, you would use the following commands:

尽管 Go 语言提供了强大的向后兼容性保证，但 bug 仍然难以避免。确保新版本不会破坏你的程序是很自然的。你也可能收到库用户提交的 bug 报告，指出你的代码在旧版本的 Go 上无法正常工作。一种方法是安装一个备用的 Go 环境。例如，如果你想尝试 1.19.2 版本，可以使用以下命令：

```
$ go install golang.org/dl/go1.19.2@latest
$ go1.19.2 download
```

You can then use the command `go1.19.2` instead of the `go` command to see if version 1.19.2 works for your programs:

然后，您可以使用命令 `go1.19.2` 代替 `go` 命令，查看版本 1.19.2 是否适用于您的程序：

```
$ go1.19.2 build
```

Once you have validated that your code works, you can uninstall the secondary environment. Go stores secondary Go environments in the *sdk* directory within your home directory. To uninstall, delete the environment from the *sdk* directory and the binary from the *go/bin* directory. Here's how to do that on macOS, Linux, and BSD:

验证代码运行正常后，即可卸载辅助 Go 环境。Go 将辅助 Go 环境存储在用户主目录下的 `sdk` 目录中。要卸载，请从 `sdk` 目录中删除该环境，并从 `go/bin` 目录中删除相应的二进制文件。以下是在 macOS、Linux 和 BSD 系统上执行此操作的方法：

```
$ rm -rf ~/sdk/go.19.2
$ rm ~/go/bin/go1.19.2
```

Using go help to Learn More About Go Tooling

使用 go help 了解更多关于 Go 工具的信息

You can learn more about Go's tooling and runtime environment with the `go help` command. It contains exhaustive information about all the commands mentioned here, as well as things like modules, import path syntax, and working with nonpublic source code. For example, you can get information on import path syntax by typing `go help importpath`.

您可以使用 `go help` 命令了解更多关于 Go 的工具和运行时环境的信息。它包含了此处提到的所有命令的详尽信息，以及模块、导入路径语法和处理非公开源代码等内容。例如，您可以通过输入 `go help importpath` 来获取有关导入路径语法的信息。

Exercises

练习

These exercises cover some of the tools that you've learned about in this chapter. You can find the solutions in the *exercise_solutions* directory in the [Chapter 11 repository](#).

这些练习涵盖了你在本章中学到的一些工具。你可以在第 11 章代码库的 `exercise_solutions` 目录中找到答案。

1. Go to the [UN's Universal Declaration of Human Rights \(UDHR\) page](#) and copy the text of the UDHR into a text file called *english_rights.txt*. Click the Other Languages link and copy the document text in a few additional languages into files named *LANGUAGE_rights.txt*. Create a program that embeds these files into package-level variables. Your program should take in one command-line parameter, the name of a language. It should then print out the UDHR in that language.

访问联合国《世界人权宣言》（UDHR）网站，将宣言文本复制到名为 `english_rights.txt` 的文本文件中。点击“其他语言”链接，将宣言文本复制到名为 `LANGUAGE_rights.txt` 的文件中，分别对应几种其他语言。创建一个程序，将这些文件嵌入到包级变量中。程序应接收一个命令行参数，即语言名称。然后，程序应输出该语言版本的《世界人权宣言》。

2. Use `go install` to install `staticcheck`. Run it against your program and fix any problems it finds.

使用 `go install` 安装 `staticcheck`。运行它来检查你的程序，并修复它发现的任何问题。

3. Cross-compile your program for ARM64 on Windows. If you are using an ARM64 Windows computer, cross-compile for AMD64 on Linux. 在 Windows 系统上将程序交叉编译为 ARM64 架构。如果您使用的是 ARM64 Windows 计算机，则需要在 Linux 系统上交叉编译为 AMD64 架构。

Wrapping Up

总结

In this chapter, you learned about the tools that Go provides to improve software engineering practices and third-party code-quality tools. In the

next chapter, you're going to explore one of the signature features in Go: concurrency.

在本章中，你学习了 Go 提供的用于改进软件工程实践的工具以及第三方代码质量工具。在下一章中，你将探索 Go 的一个标志性特性：并发。

- The term “linter” comes from the original `lint` program written by Steve Johnson when he was on the Unix team at Bell Labs and described in his [1978 paper](#). The name comes from the tiny bits of fabric that come off clothes in a dryer and are captured by a filter. He saw his program as being like a filter capturing small errors.
- “linter”一词源于史蒂夫·约翰逊 (Steve Johnson) 在贝尔实验室 Unix 团队工作期间编写的 `lint` 程序，该程序在其 1978 年的论文中有所描述。其名称来源于烘干机中衣物上脱落的细小纤维，这些纤维会被过滤器捕获。约翰逊将他的程序视为一种过滤器，用于捕获微小的错误。

Chapter 12. Concurrency in Go

第十二章 Go语言中的并发

Concurrency is the computer science term for breaking up a single process into independent components and specifying how these components safely share data. Most languages provide concurrency via a library using operating system–level threads that share data by attempting to acquire locks. Go is different. Its main concurrency model, arguably Go’s most famous feature, is based on Communicating Sequential Processes (CSP). This style for concurrency was described in 1978 in a [paper by Tony Hoare](#), the man who invented the Quicksort algorithm. The patterns implemented with CSP are just as powerful as the standard ones but are far easier to understand.

并发是计算机科学中将单个进程分解为独立组件并指定这些组件如何安全地共享数据的术语。大多数编程语言通过库来实现并发，这些库使用操作系统级别的线程，通过尝试获取锁来共享数据。Go 语言则不同。它的主要并发模型（可以说是 Go 语言最著名的特性）基于通信顺序进程（CSP）。这种并发方式由快速排序算法的发明者 Tony Hoare 在 1978 年的一篇论文中提出。使用 CSP 实现的模式与标准模式一样强大，但更容易理解。

In this chapter, you are going to quickly review the features that are the backbone of concurrency in Go: goroutines, channels, and the `select` keyword. Then you are going to look at some common Go concurrency patterns and learn about the situations where lower-level techniques are a better approach.

本章将快速回顾 Go 语言并发的核心特性：goroutine、channel 和 `select` 关键字。然后，我们将探讨一些常见的 Go 并发模式，并了解在哪些情况下使用底层技术更为合适。

When to Use Concurrency

何时使用并发

Let's start with a word of caution. Be sure that your program benefits from concurrency. When new Go developers start experimenting with concurrency, they tend to go through a series of stages:

首先要提醒一句：务必确保你的程序能从并发中获益。Go 新手开发者在尝试并发时，通常会经历以下几个阶段：

1. This is *amazing*; I'm going to put everything in goroutines!
这太棒了；我要把所有东西都放到 goroutine 里！
2. My program isn't any faster. I'm adding buffers to my channels.

我的程序运行速度并没有提升。我正在给通道添加缓冲区。

3. My channels are blocking and I'm getting deadlocks. I'm going to use buffered channels with *really* big buffers.

我的通道出现阻塞，导致死锁。我打算使用带缓冲且缓冲区大小非常大的通道。

4. My channels are still blocking. I'm going to use mutexes.

我的频道仍然被屏蔽。我打算使用互斥锁。

5. Forget it, I'm giving up on concurrency.

算了，我放弃并发了。

People are attracted to concurrency because they believe concurrent programs run faster. Unfortunately, that's not always the case. More concurrency doesn't automatically make things faster, and it can make code harder to understand. The key is understanding that *concurrency is not parallelism*. Concurrency is a tool to better structure the problem you are trying to solve.

人们之所以青睐并发，是因为他们认为并发程序运行速度更快。然而，事实并非总是如此。更多的并发并不会自动提升速度，反而会使代码更难理解。关键在于理解并发并非并行。并发是一种工具，可以帮助你更好地构建要解决的问题。

Whether concurrent code runs in parallel (at the same time) depends on the hardware and whether the algorithm allows it. In 1967, Gene Amdahl, one

of the pioneers of computer science, derived Amdahl's law. It is a formula for figuring out how much parallel processing can improve performance, given how much of the work must be performed sequentially. If you want to dive into the details on Amdahl's law, you can learn more in [*The Art of Concurrency*](#) by Clay Breshears (O'Reilly). For our purposes, all you need to understand is that more concurrency does not mean more speed.

并发代码能否并行运行（同时运行）取决于硬件以及算法是否支持。1967 年，计算机科学先驱之一的吉恩·阿姆达尔 (Gene Amdahl) 推导出了阿姆达尔定律。该定律是一个公式，用于计算在给定大量工作必须顺序执行的情况下，并行处理能够提升多少性能。如果您想深入了解阿姆达尔定律的细节，可以阅读克莱·布雷谢尔斯 (Clay Breshears) 的著作《并发的艺术》(The Art of Concurrency, O'Reilly 出版社)。就本文而言，您只需理解一点：更高的并发性并不意味着更快的速度。

Broadly speaking, all programs follow the same three-step process: they take data, transform it, and then output the result. Whether you should use concurrency in your program depends on how data flows through the steps in your program. Sometimes two steps can be concurrent because the data from one is not required for the other to proceed, and at other times two steps must happen in series because one depends on the other's output. Use concurrency when you want to combine data from multiple operations that can operate independently.

总的来说，所有程序都遵循相同的三步流程：获取数据、转换数据，然后输出结果。是否应该在程序中使用并发取决于数据在程序各步骤中的流动方式。有时，两个步骤可以并发执行，因为一个步骤的数据并非另一个步骤执行的必要条件；而有时，两个步骤必须串行执行，因为一个步骤依赖于另一个步骤的输出。当需要合并来自多个可以独立运行的操作的数据时，请使用并发。

Another important thing to note is that concurrency isn't worth using if the process that's running concurrently doesn't take a lot of time. Concurrency isn't free; many common in-memory algorithms are so fast that the overhead of passing values via concurrency overwhelms any potential time savings you'd gain by running concurrent code in parallel. This is why concurrent operations are often used for I/O; reading or writing to a disk or network is thousands of times slower than all but the most complicated in-memory processes. If you are not sure if concurrency will help, first write your code serially and then write a benchmark to compare performance with a concurrent implementation. (See [“Using Benchmarks”](#) for information on how to benchmark your code.)

另一点需要注意的是，如果并发运行的进程耗时不多，那么使用并发就没什么意义。并发并非没有代价；许多常见的内存算法速度极快，通过并发传递值的开销会远远超过并行运行并发代码所带来的任何潜在时间节省。这就是为什么并发操作通常用于 I/O 操作；读写磁盘或网络的速度比除极少数复杂内存进程之外的所有操作慢数千倍。如果您不确定并发是否有效，请先编写串行代码，然后编写基准测试程序

来比较其与并发实现的性能。（有关如何编写基准测试程序的信息，请参阅“使用基准测试”。）

Let's consider an example. Say you are writing a web service that calls three other web services. Your program sends data to two of those services, and then takes the results of those two calls and sends them to the third, returning the result. The entire process must take less than 50 milliseconds, or an error should be returned. This is a good use of concurrency, because there are parts of the code that need to perform I/O that can run without interacting with one another, there's a part where the results are combined, and there's a limit on how long the code needs to run. At the end of this chapter, you'll see how to implement this code.

我们来看一个例子。假设你正在编写一个 Web 服务，它会调用另外三个 Web 服务。你的程序会向其中两个服务发送数据，然后获取这两个服务调用的结果，并将结果发送给第三个服务并返回。整个过程必须在 50 毫秒内完成，否则应该返回错误。这是一个并发性很好的应用，因为代码中有些部分需要执行 I/O 操作，这些操作可以独立运行；有些部分需要合并结果；而且代码的运行时间也有限制。在本章末尾，你将看到如何实现这段代码。

Goroutines

goroutines

The goroutine is the core concept in Go's concurrency model. To understand goroutines, let's define a couple of terms. The first is *process*. A process is an instance of a program that's being run by a computer's operating system. The operating system associates some resources, such as memory, with the process and makes sure that other processes can't access them. A process is composed of one or more *threads*. A thread is a unit of execution that is given some time to run by the operating system. Threads within a process share access to resources. A CPU can execute instructions from one or more threads at the same time, depending on the number of cores. One of the jobs of an operating system is to schedule threads on the CPU to make sure that every process (and every thread within a process) gets a chance to run.

goroutine 是 Go 并发模型的核心概念。为了理解 goroutine，我们先来定义几个术语。首先是进程。进程是计算机操作系统运行的程序实例。操作系统会将一些资源（例如内存）与进程关联起来，并确保其他进程无法访问这些资源。一个进程由一个或多个线程组成。线程是操作系统分配运行时间的执行单元。进程内的线程共享资源。CPU 可以同时执行来自一个或多个线程的指令，具体数量取决于核心数。操

作系统的一项重要任务就是在 CPU 上调度线程，以确保每个进程（以及进程内的每个线程）都有机会运行。

Think of a goroutine as a lightweight thread, managed by the Go runtime. When a Go program starts, the Go runtime creates a number of threads and launches a single goroutine to run your program. All the goroutines created by your program, including the initial one, are assigned to these threads automatically by the Go runtime scheduler, just as the operating system schedules threads across CPU cores. This might seem like extra work, since the underlying operating system already includes a scheduler that manages threads and processes, but it has several benefits:

可以将 goroutine 理解为由 Go 运行时管理的轻量级线程。当 Go 程序启动时，Go 运行时创建若干线程，并启动一个 goroutine 来运行你的程序。程序创建的所有 goroutine（包括初始 goroutine）都会由 Go 运行时调度器自动分配给这些线程，就像操作系统在 CPU 核心之间调度线程一样。这看起来似乎是额外的工作，因为底层操作系统已经包含一个管理线程和进程的调度器，但它有以下几个好处：

- Goroutine creation is faster than thread creation, because you aren't creating an operating system-level resource.
创建 Goroutine 比创建线程更快，因为你不需要创建操作系统级别的资源。
- Goroutine initial stack sizes are smaller than thread stack sizes and can grow as needed. This makes goroutines more memory efficient.

goroutine 的初始栈大小小于线程栈大小，并且可以根据需要增长。这使得 goroutine 更节省内存。

- Switching between goroutines is faster than switching between threads because it happens entirely within the process, avoiding operating system calls that are (relatively) slow.

在 goroutine 之间切换比在线程之间切换更快，因为它完全在进程内进行，避免了（相对）缓慢的操作系统调用。

- The goroutine scheduler is able to optimize its decisions because it is part of the Go process. The scheduler works with the network poller, detecting when a goroutine can be unscheduled because it is blocking on I/O. It also integrates with the garbage collector, making sure that work is properly balanced across all the operating system threads assigned to your Go process.

由于 goroutine 调度器是 Go 进程的一部分，因此它能够优化其决策。该调度器与网络轮询器协同工作，检测何时 goroutine 因 I/O 阻塞而可以取消调度。它还与垃圾回收器集成，确保分配给 Go 进程的所有操作系统线程之间的工作负载得到适当的均衡。

These advantages allow Go programs to spawn hundreds, thousands, even tens of thousands of simultaneous goroutines. If you try to launch thousands of threads in a language with native threading, your program will slow to a crawl.

这些优势使得 Go 程序能够同时生成成百上千甚至数万个 goroutine。如果你尝试用一种本身就支持线程的语言启动数千个线程，你的程序

运行速度将会变得极其缓慢。

TIP

提示

If you are interested in learning more about how the scheduler does its work, watch the talk Kavya Joshi gave at GopherCon 2018 called [“The Scheduler Saga”](#).

如果您有兴趣了解更多关于调度程序如何工作的信息，请观看 Kavya Joshi 在 2018 年 GopherCon 大会上发表的题为“调度程序传奇”的演讲。

A goroutine is launched by placing the `go` keyword before a function invocation. Just as with any other function, you can pass it parameters to initialize its state. However, any values returned by the function are ignored.

通过在函数调用前加上 `go` 关键字来启动 goroutine。与其他函数一样，你可以向其传递参数来初始化其状态。但是，函数返回的任何值都会被忽略。

Any function can be launched as a goroutine. This is different from JavaScript, where a function runs asynchronously only if the author of the function declared it with the `async` keyword. However, it is customary in Go to launch goroutines with a closure that wraps business logic. The closure takes care of the concurrent bookkeeping. The following sample code demonstrates the concept:

任何函数都可以作为 goroutine 启动。这与 JavaScript 不同，在 JavaScript 中，只有当函数作者使用 `async` 关键字声明函数时，函数才会异步运行。然而，在 Go 中，通常使用一个封装了业务逻辑的闭包来启动 goroutine。闭包负责处理并发事务。以下示例代码演示了这一概念：

```
func process(val int) int {
    // do something with val
}

func processConcurrently(inVals []int) []int {
    // create the channels
    in := make(chan int, 5)
    out := make(chan int, 5)
    // launch processing goroutines
    for i := 0; i < 5; i++ {
        go func() {
            for val := range in {
                out <- process(val)
            }
        }()
    }
    // load the data into the in channel in
    // another goroutine
    // read the data from the out channel
```



```
// return the data  
}
```

In this code, the `processConcurrently` function creates a closure, which reads values out of a channel and passes them to the business logic in the `process` function. The `process` function is completely unaware that it is running in a goroutine. The result of `process` is then written back to a different channel by the closure. (I'll do a brief overview of channels in the next section.) This separation of responsibility makes your programs modular and testable, and keeps concurrency out of your APIs. The decision to use a thread-like model for concurrency means that Go programs avoid the “function coloring” problem described by Bob Nystrom in his famous blog post [“What Color Is Your Function?”](#)

在这段代码中，`processConcurrently` 函数创建了一个闭包，该闭包从一个通道中读取值，并将其传递给 `process` 函数中的业务逻辑。`process` 函数完全不知道它运行在一个 goroutine 中。然后，`process` 的结果由闭包写回另一个通道。（我将在下一节简要概述通道。）这种职责分离使你的程序模块化且易于测试，并将并发性排除在你的 API 之外。使用类似线程的并发模型意味着 Go 程序避免了 Bob Nystrom 在他著名的博文“你的函数是什么颜色？”中描述的“函数着色”问题。

You can find a complete example on [The Go Playground](#) or in the `sample_code/goroutine` directory in the [Chapter 12 repository](#).

您可以在 Go Playground 或第 12 章存储库的 `sample_code/goroutine` 目录中找到完整的示例。

Channels

频道

Goroutines communicate using *channels*. Like slices and maps, channels are a built-in type created using the `make` function:

Goroutine 使用通道进行通信。与切片和映射类似，通道是一种内置类型，使用 `make` 函数创建：

```
ch := make(chan int)
```

Like maps, channels are reference types. When you pass a channel to a function, you are really passing a pointer to the channel. Also like maps and slices, the zero value for a channel is `nil`.

与映射（Map）类似，通道（Channel）也是引用类型。当你将一个通道传递给一个函数时，实际上是在传递指向该通道的指针。同样，与映射和切片类似，通道的零值是 `nil`。

Reading, Writing, and Buffering

阅读、写作和缓冲

Use the `<-` operator to interact with a channel. You read from a channel by placing the `<-` operator to the left of the channel variable, and you write to a channel by placing it to the right:

使用 `<-` 运算符与通道进行交互。要从通道读取数据，请将 `<-` 运算符放在通道变量的左侧；要向通道写入数据，请将其放在右侧：

```
a := <-ch // reads a value from ch and assigns it
to a
ch <- b   // write the value in b to ch
```

Each value written to a channel can be read only once. If multiple goroutines are reading from the same channel, a value written to the channel will be read by only one of them.

写入通道的每个值只能被读取一次。如果多个 goroutine 从同一个通道读取数据，则写入通道的值只会被其中一个 goroutine 读取。

A single goroutine rarely reads and writes to the same channel. When assigning a channel to a variable or field, or passing it to a function, use an arrow before the `chan` keyword (`ch <-chan int`) to indicate that the goroutine only *reads* from the channel. Use an arrow after the `chan`

keyword (`ch chan<- int`) to indicate that the goroutine only *writes* to the channel. Doing so allows the Go compiler to ensure that a channel is only read from or written to by a function.

单个 goroutine 很少会同时读取和写入同一个通道。当将通道赋值给变量或字段，或将其传递给函数时，请在 `chan` 关键字前使用箭头（例如 `ch <-chan int`），以表明 goroutine 仅从该通道读取数据。在 `chan` 关键字后使用箭头（例如 `ch chan<- int`），以表明 goroutine 仅向该通道写入数据。这样做可以让 Go 编译器确保通道仅由函数读取或写入。

By default, channels are *unbuffered*. Every write to an open, unbuffered channel causes the writing goroutine to pause until another goroutine reads from the same channel. Likewise, a read from an open, unbuffered channel causes the reading goroutine to pause until another goroutine writes to the same channel. This means you cannot write to or read from an unbuffered channel without at least two concurrently running goroutines.

默认情况下，通道是无缓冲的。每次向已打开的无缓冲通道写入数据都会导致写入 goroutine 暂停，直到另一个 goroutine 从同一通道读取数据。同样，每次从已打开的无缓冲通道读取数据都会导致读取 goroutine 暂停，直到另一个 goroutine 向同一通道写入数据。这意味着，如果没有至少两个并发运行的 goroutine，您无法对无缓冲通道进行写入或读取操作。

Go also has *buffered* channels. These channels buffer a limited number of writes without blocking. If the buffer fills before there are any reads from the channel, a subsequent write to the channel pauses the writing goroutine until the channel is read. Just as writing to a channel with a full buffer blocks, reading from a channel with an empty buffer also blocks.

Go 也支持缓冲通道。这些通道可以缓冲有限数量的写入操作，而不会阻塞。如果通道在读取操作发生之前缓冲区已满，则后续的写入操作会暂停写入 goroutine，直到通道中的数据被读取完毕。正如向缓冲区已满的通道写入数据会阻塞一样，从缓冲区为空的通道读取数据也会阻塞。

A buffered channel is created by specifying the capacity of the buffer when creating the channel:

创建缓冲通道时，需要指定缓冲区的容量：

```
ch := make(chan int, 10)
```

The built-in functions `len` and `cap` return information about a buffered channel. Use `len` to find out how many values are currently in the buffer and use `cap` to find out the maximum buffer size. The capacity of the buffer cannot be changed.

内置函数 `len` 和 `cap` 返回有关已缓冲通道的信息。使用 `len` 可以查看缓冲区中当前有多少个值，使用 `cap` 可以查看缓冲区的最大大小。缓冲区的容量无法更改。

NOTE

笔记

Passing an unbuffered channel to both `len` and `cap` returns 0. This makes sense because, by definition, an unbuffered channel doesn't have a buffer to store values.

将无缓冲通道同时传递给 `len` 和 `cap` 都会返回 0。这是合理的，因为根据定义，无缓冲通道没有缓冲区来存储值。

Most of the time, you should use unbuffered channels. In [“Know When to Use Buffered and Unbuffered Channels”](#), I'll talk about the situations where buffered channels are useful.

大多数情况下，你应该使用非缓冲通道。在“何时使用缓冲通道和非缓冲通道”一文中，我将讨论缓冲通道的适用场景。

Using for-range and Channels

使用范围和信道

You can also read from a channel by using a `for-range` loop:

您还可以使用 `for-range` 循环从频道读取数据：

```
for v := range ch {  
    fmt.Println(v)  
}
```

Unlike other `for-range` loops, there is only a single variable declared for the channel, which is the value. If the channel is open and a value is available on the channel, it is assigned to `v` and the body of the loop executes. If no value is available on the channel, the goroutine pauses until a value is available or the channel is closed. The loop continues until the channel is closed, or until a `break` or `return` statement is reached.

与其他 `for-range` 循环不同，此循环仅声明了一个通道变量，即值。如果通道已打开且有值可用，则将该值赋给 `v`，并执行循环体。如果通道没有值可用，则 goroutine 会暂停，直到有值可用或通道关闭。循环会一直持续，直到通道关闭，或直到遇到 `break` 或 `return` 语句。

Closing a Channel

关闭频道

When you're done writing to a channel, you close it using the built-in `close` function:

完成向频道写入数据后，可以使用内置的 `close` 函数将其关闭：

```
close(ch)
```

Once a channel is closed, any attempts to write to it or close it again will panic. Interestingly, attempting to read from a closed channel always succeeds. If the channel is buffered and some values haven't been read yet, they will be returned in order. If the channel is unbuffered or the buffered channel has no more values, the zero value for the channel's type is returned.

通道一旦关闭，任何写入或再次关闭通道的尝试都会导致程序崩溃。有趣的是，尝试从已关闭的通道读取数据总是会成功。如果通道已缓冲且仍有一些值尚未读取，则会按顺序返回这些值。如果通道未缓冲或已缓冲的通道中没有更多值，则返回该通道类型的零值。

This leads to a question that might sound familiar from your experience with maps: when your code reads from a channel, how do you tell the difference between a zero value that was written and a zero value that was returned because the channel is closed? Since Go tries to be a consistent language, there is a familiar answer—use the comma ok idiom to detect whether a channel has been closed:

这引出了一个你可能在使用 Map 时会感到熟悉的问题：当你的代码从一个通道读取数据时，如何区分写入的零值和由于通道关闭而返回的零值？由于 Go 力求保持一致性，因此有一个常见的答案——使用逗号 ok 惯用法来检测通道是否已关闭：


```
v, ok := <-ch
```

If `ok` is set to `true`, the channel is open. If it is set to `false`, the channel is closed.

如果 `ok` 设置为 `true`，则通道已打开；如果设置为 `false`，则通道已关闭。

TIP

提示

Anytime you are reading from a channel that might be closed, use the comma ok idiom to ensure that the channel is still open.

当你在阅读可能已关闭的频道时，请使用逗号 ok 来确认该频道是否仍然开放。

The responsibility for closing a channel lies with the goroutine that writes to the channel. Be aware that closing a channel is required only if a goroutine is waiting for the channel to close (such as one using a `for-range` loop to read from the channel). Since a channel is just another variable, Go's runtime can detect channels that are no longer referenced and garbage collect them.

关闭通道的责任在于写入通道的 goroutine。请注意，只有当 goroutine 等待通道关闭时（例如使用 `for-range` 循环从通道读取数据的

goroutine) , 才需要关闭通道。由于通道本质上只是一个变量, Go 运行时可以检测到不再被引用的通道并将其回收。

Channels are one of the two things that set apart Go's concurrency model. They guide you into thinking about your code as a series of stages and making data dependencies clear, which makes it easier to reason about concurrency. Other languages rely on global shared state to communicate between threads. This mutable shared state makes it hard to understand how data flows through a program, which in turn makes it difficult to understand whether two threads are actually independent.

通道是 Go 并发模型的两大特色之一。它引导你将代码视为一系列阶段, 并清晰地展现数据依赖关系, 从而更容易理解并发性。其他语言依赖全局共享状态来实现线程间的通信。这种可变的共享状态使得理解数据在程序中的流动方式变得困难, 进而难以判断两个线程是否真正独立。

Understanding How Channels Behave

了解渠道行为

Channels have many states, each with a different behavior when reading, writing, or closing. Use [Table 12-1](#) to keep them straight.

通道有多种状态，每种状态在读取、写入或关闭时都有不同的行为。
请使用表 12-1 来区分它们。

Table 12-1. How channels behave
表 12-1. 信道行为

	Unbuffered, open 无缓冲，开放	Unbuffered, closed 无缓冲，已关闭	Buffered, open 缓冲，开放	Buffered, closed 缓冲，已关闭
Read 读	Pause until something is written 暂停，直到写完为止。	Return zero value (use comma ok to see if closed) 返回零值（使用逗号可以判断是否已闭合）	Pause if buffer is empty 如果缓冲区为空，则暂停。	Return remaining value buffered buffered empty zero (use comma ok to close) 返回 剩余的 结果缓冲 空， 值（ 可以 区分是 闭）

	Unbuffered, open 无缓冲，开放	Unbuffered, closed 无缓冲，已关闭	Buffered, open 缓冲，开放	Buffered, closed 缓冲，已关闭
Write 写	Pause until something is read 暂停，直到读完某些内容。	PANIC 恐慌	Pause if buffer is full 如果缓冲区已满，则暂停。	PANIC 恐慌
Close 关闭	Works 作品	PANIC 恐慌	Works, remaining values still there 作品，以及剩余的价值依然存在。	PANIC 恐慌

You must avoid situations that cause Go programs to panic. As mentioned earlier, the standard pattern is to make the writing goroutine responsible for closing the channel when there's nothing left to write. When multiple goroutines are writing to the same channel, this becomes more complicated, as calling `close` twice on the same channel causes a panic. Furthermore,

if you close a channel in one goroutine, a write to the channel in another goroutine triggers a panic as well. The way to address this is to use a `sync.WaitGroup`. You'll see an example in [“Use WaitGroups”](#).

你必须避免导致 Go 程序崩溃的情况。如前所述，标准模式是让写入 goroutine 负责在无内容可写时关闭通道。当多个 goroutine 向同一个通道写入数据时，情况会变得更加复杂，因为对同一个通道调用两次 `close` 会导致崩溃。此外，如果在一个 goroutine 中关闭了通道，另一个 goroutine 向该通道写入数据也会触发崩溃。解决这个问题方法是使用 `sync.WaitGroup`。你可以在“使用 WaitGroups”一节中找到示例。

A `nil` channel can be dangerous as well, but it is useful in some cases. You'll learn more about them in [“Turn Off a case in a select”](#).

`nil` 通道也可能存在风险，但在某些情况下也很有用。您将在“关闭 SELECT 语句中的 case”一文中了解更多相关信息。

select

选择

The `select` statement is the other thing that sets apart Go's concurrency model. It is the control structure for concurrency in Go, and it elegantly

solves a common problem: if you can perform two concurrent operations, which one do you do first? You can't favor one operation over others, or you'll never process some cases. This is called *starvation*.

`select` 语句是Go并发模型的另一大特色。它是Go中并发的控制结构，巧妙地解决了一个常见问题：如果可以执行两个并发操作，应该先执行哪个？你不能优先执行某个操作，否则某些情况将永远无法处理。这被称为饥饿。

The `select` keyword allows a goroutine to read from or write to one of a set of multiple channels. It looks a great deal like a blank `switch` statement:

`select` 关键字允许 goroutine 从一组多个通道中的一个读取数据或向其中一个通道写入数据。它看起来很像是一个空白的 `switch` 语句：

```
select {
case v := <-ch:
    fmt.Println(v)
case v := <-ch2:
    fmt.Println(v)
case ch3 <- x:
    fmt.Println("wrote", x)
case <-ch4:
    fmt.Println("got value on ch4, but ignored")
}
```

```
it")  
}
```

Each `case` in a `select` is a read or a write to a channel. If a read or write is possible for a `case`, it is executed along with the body of the `case`. Like a `switch`, each `case` in a `select` creates its own block.

`select` 中的每个 `case` 都代表对通道的读取或写入操作。如果 `case` 可以进行读取或写入操作，则会与 `case` 的主体一起执行。与 `switch` 类似，`select` 中的每个 `case` 都会创建自己的数据块。

What happens if multiple cases have channels that can be read or written? The `select` algorithm is simple: it picks randomly from any of its cases that can go forward; order is unimportant. This is very different from a `switch` statement, which always chooses the first `case` that resolves to `true`. It also cleanly resolves the starvation problem, as no `case` is favored over another and all are checked at the same time.

如果多个案例都具有可读写的通道会发生什么？`select` 算法很简单：它从所有可以继续执行的案例中随机选择一个；顺序无关紧要。这与 `switch` 语句截然不同，后者总是选择第一个解析为 `true` 的 `case`。它也完美地解决了饥饿问题，因为没有哪个 `case` 比其他的更有优势，所有案例都会同时被检查。

Another advantage of `select` choosing at random is that it prevents one of the most common causes of deadlocks: acquiring locks in an inconsistent order. If you have two goroutines that both access the same two channels, they must be accessed in the same order in both goroutines, or they will *deadlock*. This means that neither one can proceed because they are waiting on each other. If every goroutine in your Go application is deadlocked, the Go runtime kills your program (see [Example 12-1](#)).

`select` 随机选择的另一个优点是，它可以防止死锁最常见的原因之一：以不一致的顺序获取锁。如果两个 goroutine 都访问相同的两个通道，则这两个 goroutine 必须以相同的顺序访问它们，否则就会发生死锁。这意味着它们都无法继续执行，因为它们都在等待对方。如果 Go 应用程序中的每个 goroutine 都发生死锁，Go 运行时会终止程序（参见示例 12-1）。

Example 12-1. Deadlocking goroutines

例 12-1. goroutine 死锁

```
func main() {
    ch1 := make(chan int)
    ch2 := make(chan int)
    go func() {
        inGoroutine := 1
        ch1 <- inGoroutine
        fromMain := <-ch2
```

```
        fmt.Println("goroutine:", inGoroutine,
fromMain)
    }()
    inMain := 2
    ch2 <- inMain
    fromGoroutine := <-ch1
    fmt.Println("main:", inMain, fromGoroutine)
}
```

If you run this program on [The Go Playground](#) or in the *sample_code/deadlock* directory in the [Chapter 12 repository](#), you'll see the following error:

如果你在 Go Playground 或第 12 章代码库的 *sample_code/deadlock* 目录中运行此程序，你会看到以下错误：

```
fatal error: all goroutines are asleep - deadlock!
```

Remember that `main` is running on a goroutine that is launched at startup by the Go runtime. The goroutine that is explicitly launched cannot proceed until `ch1` is read, and the main goroutine cannot proceed until `ch2` is read.

请记住，`main` 运行在 Go 运行时启动的 goroutine 中。显式启动的 goroutine 必须先读取 `ch1` 才能继续执行，而主 goroutine 必须先读取 `ch2` 才能继续执行。

If the channel read and the channel write in the main goroutine are wrapped in a `select`, deadlock is avoided (see [Example 12-2](#)).

如果在主 goroutine 中对通道读取和通道写入进行包装，则可避免死锁（参见示例 12-2）。

Example 12-2. Using `select` to avoid deadlocks

示例 12-2. 使用 `select` 避免死锁

```
func main() {
    ch1 := make(chan int)
    ch2 := make(chan int)
    go func() {
        inGoroutine := 1
        ch1 <- inGoroutine
        fromMain := <-ch2
        fmt.Println("goroutine:", inGoroutine,
fromMain)
    }()
    inMain := 2
    var fromGoroutine int
    select {
    case ch2 <- inMain:
    case fromGoroutine = <-ch1:
    }
}
```

```
    fmt.Println("main:", inMain, fromGoroutine)
}
```

If you run this program on [The Go Playground](#) or in the *sample_code/select* directory in the [Chapter 12 repository](#), you'll get the output:

如果你在 Go Playground 或第 12 章代码库的 *sample_code/select* 目录中运行此程序，你将得到以下输出：

```
main: 2 1
```

Because a `select` checks whether any of its cases can proceed, the deadlock is avoided. The goroutine that is launched explicitly wrote the value 1 into `ch1`, so the read from `ch1` into `fromGoroutine` in the main goroutine is able to succeed.

由于 `select` 会检查其任何分支是否能够继续执行，因此避免了死锁。启动的 goroutine 已显式地将值 1 写入 `ch1`，因此主 goroutine 中从 `ch1` 读取到 `fromGoroutine` 的操作能够成功。

Although this program doesn't deadlock, it still doesn't do the right thing. The `fmt.Println` statement in the launched goroutine never executes, because that goroutine is paused, waiting for a value to read from `ch2`. When the main goroutine exits, the program exits and kills any remaining goroutines, which does technically resolve the pause. However, you should

make sure that all your goroutines exit properly so that you don't *leak* them. I talk about this in more detail in [“Always Clean Up Your Goroutines”](#).

虽然这个程序没有死锁，但它仍然没有正确执行。启动的 goroutine 中的 `fmt.Println` 语句永远不会执行，因为该 goroutine 处于暂停状态，等待从 `ch2` 读取值。当主 goroutine 退出时，程序会退出并终止所有剩余的 goroutine，这在技术上确实解决了暂停问题。但是，您应该确保所有 goroutine 都正确退出，以避免内存泄漏。我在“始终清理您的 goroutine”一文中对此进行了更详细的讨论。

NOTE

笔记

Making this program behave properly requires a few techniques that you'll learn about later in the chapter. You can find a working solution on [The Go Playground](#).

要让这个程序正常运行，需要一些技巧，你将在本章后面学习到。你可以在 Go Playground 上找到一个可行的解决方案。

Since `select` is responsible for communicating over a number of channels, it is often embedded within a `for` loop:

由于 `select` 负责通过多个通道进行通信，因此它通常嵌入在 `for` 循环中：

```
for {
    select {
        case <-done:
            return
        case v := <-ch:
            fmt.Println(v)
    }
}
```

This is so common that the combination is often referred to as a `for-select` loop. When using a `for-select` loop, you must include a way to exit the loop. You'll see one way to do this in [“Use the Context to Terminate Goroutines”](#).

这种组合非常常见，因此通常被称为 `for-select` 循环。使用 `for-select` 循环时，必须包含退出循环的方法。您可以在“使用上下文终止 Goroutine”一节中看到一种方法。

Just like `switch` statements, a `select` statement can have a `default` clause. Also just like `switch`, `default` is selected when there are no cases with channels that can be read or written. If you want to implement a nonblocking read or write on a channel, use a `select` with a `default`. The following code does not wait if there's no value to read in `ch`; it immediately executes the body of the `default`:

与 `switch` 语句类似，`select` 语句也可以包含 `default` 子句。同样，与 `switch` 类似，当没有可读写通道时，会选择 `default`。如果要对通道实现非阻塞读写操作，请使用 `select` 和 `default`。以下代码在 `ch` 中没有值可读时不会等待，而是立即执行 `default` 的主体：

```
select {
case v := <-ch:
    fmt.Println("read from ch:", v)
default:
    fmt.Println("no value written to ch")
}
```

You'll take a look at a use for `default` in [“Implement Backpressure”](#).

在“实现反压”中，您将看到 `default` 的用法。

NOTE

笔记

Having a `default` case inside a `for-select` loop is almost always the wrong thing to do. It will be triggered every time through the loop when there's nothing to read or write for any of the cases. This makes your `for` loop run constantly, which uses a great deal of CPU.

在 `for-select` 循环中放入 `default` 个 case 几乎总是错误的。即使循环中没有任何 case 需要读取或写入，它也会每次都被触发。这会导致 `for` 循环持续运行，从而占用大量 CPU 资源。

Concurrency Practices and Patterns

并发实践与模式

Now that you've seen the basic tools that Go provides for concurrency, let's take a look at some concurrency best practices and patterns.

现在你已经了解了 Go 提供的并发基本工具，让我们来看看一些并发最佳实践和模式。

Keep Your APIs Concurrency-Free

保持您的 API 无并发性

Concurrency is an implementation detail, and good API design should hide implementation details as much as possible. This allows you to change how your code works without changing how your code is invoked.

并发性是实现细节，优秀的 API 设计应该尽可能地隐藏实现细节。这样，你就可以在不改变代码调用方式的情况下，更改代码的运行方式。

Practically, this means that you should never expose channels or mutexes in your API's types, functions, and methods (I'll talk about mutexes in [“When to Use Mutexes Instead of Channels”](#)). If you expose a channel, you put the

responsibility of channel management on the users of your API. The users then have to worry about concerns like whether a channel is buffered or closed or `nil`. They can also trigger deadlocks by accessing channels or mutexes in an unexpected order.

实际上，这意味着你永远不应该在 API 的类型、函数和方法中暴露通道或互斥锁（我会在“何时使用互斥锁而不是通道”一节中讨论互斥锁）。如果你暴露了通道，就等于把通道管理的责任转嫁给了 API 的用户。用户需要担心诸如通道是否已缓冲、是否已关闭或是否存在 `nil` 之类的问题。他们还可能因为以非预期的顺序访问通道或互斥锁而触发死锁。

NOTE

笔记

This doesn't mean that you shouldn't ever have channels as function parameters or struct fields. It means that they shouldn't be exported.

这并不意味着你绝对不能将通道作为函数参数或结构体字段。而是说它们不应该被导出。

This rule has some exceptions. If your API is a library with a concurrency helper function, channels are going to be part of its API.

这条规则也有一些例外。如果你的 API 是一个带有并发辅助函数的库，那么通道将成为其 API 的一部分。

Goroutines, for Loops, and Varying Variables

goroutine、for循环和变量

Most of the time, the closure that you use to launch a goroutine has no parameters. Instead, it captures values from the environment where it was declared. Before Go 1.22, there was one common situation where this didn't work: when trying to capture the index or value of a `for` loop. As mentioned in [“The for-range value is a copy”](#) and [“Using go.mod”](#), a backward-breaking change was introduced in Go 1.22 that changed the behavior of a `for` loop so that it creates new variables for the index and value on each iteration instead of reusing a single variable.

大多数情况下，用于启动 goroutine 的闭包没有参数。它会从声明它的环境中获取值。在 Go 1.22 之前，有一种常见情况会导致这种方法失效：尝试获取 `for` 循环的索引或值时。正如“for 循环的值是副本”和“使用 go.mod”中所述，Go 1.22 引入了一个向后破坏性的更改，改变了 `for` 循环的行为，使其在每次迭代中为索引和值创建新变量，而不是重用单个变量。

The following code demonstrates the reason this change was worthwhile. You can find it in the [goroutine for loop repository](#) in the Learning Go 2nd Edition organization on GitHub.

以下代码演示了这项更改的必要性。您可以在 GitHub 上 Learning Go 2nd Edition 组织的 `goroutine_for_loop` 代码库中找到它。

If you run the following code on Go 1.21 or earlier (or on Go 1.22 or later with the Go version set to 1.21 or earlier in the `go` directive in the `go.mod` file), you'll see a subtle bug:

如果在 Go 1.21 或更早版本上运行以下代码（或者在 Go 1.22 或更高版本上运行，但需要在 `go.mod` 文件中的 `go` 指令中将 Go 版本设置为 1.21 或更早版本），您会发现一个不易察觉的错误：

```
func main() {  
    a := []int{2, 4, 6, 8, 10}  
    ch := make(chan int, len(a))  
    for _, v := range a {  
        go func() {  
            ch <- v * 2  
        }()  
    }  
    for i := 0; i < len(a); i++ {  
        fmt.Println(<-ch)  
    }  
}
```

One goroutine is launched for each value in `a`. It looks like a different value is passed in to each goroutine, but running the code shows something

different:

针对 `a` 中的每个值启动一个 goroutine。看起来每个 goroutine 传递的值都不同，但运行代码后却显示并非如此：

```
20
20
20
20
20
```

The reason every goroutine wrote `20` to `ch` on earlier versions of Go is that the closure for every goroutine captured the same variable. The index and value variables in a `for` loop were reused on each iteration. The last value assigned to `v` was `10`. When the goroutines run, that's the value that they see.

在早期版本的 Go 中，每个 goroutine 都会将 `20` 写入 `ch` 的原因是，每个 goroutine 的闭包都捕获了同一个变量。在 `for` 循环中，索引和值变量在每次迭代中都会被重用。`v` 的最后一个赋值为 `10`。当 goroutine 运行时，它们看到的就是这个值。

Upgrading to Go 1.22 or later and changing the value of the `go` directive in `go.mod` to 1.22 or later changes the behavior of `for` loops so they create a new index and value variable on each iteration. This gives you the expected result, with a different value passed to each goroutine:

升级到 Go 1.22 或更高版本，并将 go.mod 文件中 `go` 指令的值更改为 1.22 或更高版本，即可改变 `for` 循环的行为，使其在每次迭代时创建一个新的索引和值变量。这样就能得到预期的结果，即传递给每个 goroutine 的值都不同：

```
20
8
4
12
16
```

If you cannot upgrade to Go 1.22, you can resolve this issue in two ways. The first is to make a copy of the value by shadowing the value within the loop:

如果无法升级到 Go 1.22，可以通过两种方式解决此问题。第一种方法是在循环内部复制该值，即使用 `'shadow'` 语句覆盖原值：

```
for _, v := range a {
    v := v
    go func() {
        ch <- v * 2
    }()
}
```

If you want to avoid shadowing and make the data flow more obvious, you can also pass the value as a parameter to the goroutine:

如果想要避免数据遮蔽，使数据流更加清晰，也可以将该值作为参数传递给 goroutine：

```
for _, v := range a {  
    go func(val int) {  
        ch <- val * 2  
    }(v)  
}
```

While Go 1.22 prevents this issue for the index and value variables in `for` loops, you still need to be careful with other variables that are captured by closures. Anytime a closure depends on a variable whose value might change, whether or not it is used as a goroutine, you must pass the value into the closure or make sure a unique copy of the variable is created for each closure that refers to the variable.

虽然 Go 1.22 已经解决了 `for` 循环中索引和值变量的这个问题，但对于其他被闭包捕获的变量，你仍然需要格外小心。任何时候，只要闭包依赖于一个值可能会改变的变量，无论它是否被用作 goroutine，你都必须将该值传递给闭包，或者确保为每个引用该变量的闭包创建一个唯一的变量副本。

TIP

提示

Anytime a closure uses a variable whose value might change, use a parameter to pass a copy of the variable's current value into the closure.

任何时候，当闭包使用一个值可能会改变的变量时，都要使用参数将变量当前值的副本传递给闭包。

Always Clean Up Your Goroutines

定期清理你的 goroutine

Whenever you launch a goroutine function, you must make sure that it will eventually exit. Unlike variables, the Go runtime can't detect that a goroutine will never be used again. If a goroutine doesn't exit, all the memory allocated for variables on its stack remains allocated and any memory on the heap that is rooted in the goroutine's stack variables cannot be garbage collected. This is called a *goroutine leak*.

每次启动 goroutine 函数时，必须确保它最终会退出。与变量不同，Go 运行时无法检测到 goroutine 是否会被再次使用。如果 goroutine 没有退出，其栈上分配给变量的所有内存都会一直保留，并且堆上任何以 goroutine 栈变量为根的内存都无法被垃圾回收。这被称为 goroutine 内存泄漏。

It may not be obvious that a goroutine isn't guaranteed to exit. For example, say you used a goroutine as a generator:

goroutine 不一定一定会退出这一点可能并不明显。例如，假设你使用 goroutine 作为生成器：

```
func countTo(max int) <-chan int {
    ch := make(chan int)
    go func() {
        for i := 0; i < max; i++ {
            ch <- i
        }
        close(ch)
    }()
    return ch
}

func main() {
    for i := range countTo(10) {
        fmt.Println(i)
    }
}
```

NOTE

笔记

This is just a short example; don't use a goroutine to generate a list of numbers. It's too simple of an operation, which violates one of our “when to use concurrency” guidelines.

这只是一个简单的例子；不要使用 goroutine 来生成数字列表。这个操作太简单了，违反了我们的“何时使用并发”的指导原则之一。

In the common case, where you use all the values, the goroutine exits.

However, if you exit the loop early, the goroutine blocks forever, waiting for a value to be read from the channel:

通常情况下，当所有值都被使用时，goroutine 会退出。但是，如果提前退出循环，goroutine 会永远阻塞，等待从通道读取值：

```
func main() {  
    for i := range countTo(10) {  
        if i > 5 {  
            break  
        }  
        fmt.Println(i)  
    }  
}
```

Use the Context to Terminate Goroutines

使用上下文终止 Goroutine

To solve the `countTo` goroutine leak, you need a way to tell the goroutine that it's time to stop processing. You solve this in Go by using a *context*.

Here's a rewrite of `countTo` to demonstrate this technique. You can find the code in the *sample_code/context_cancel* directory in the [Chapter 12 repository](#):

要解决 `countTo` goroutine 内存泄漏问题，你需要一种方法来告诉 goroutine 该停止处理了。在 Go 中，你可以使用 `context` 来解决这个问题。以下是 `countTo` 的重写版本，用于演示这项技术。你可以在第 12 章代码库的 *sample_code/context_cancel* 目录中找到这段代码：

```
func countTo(ctx context.Context, max int) <-chan
int {
    ch := make(chan int)
    go func() {
        defer close(ch)
        for i := 0; i < max; i++ {
            select {
            case <-ctx.Done():
                return
            case ch <- i:
            }
        }
    }
}
```

```

    }
    }()
    return ch
}

func main() {
    ctx, cancel :=
context.WithCancel(context.Background())
    defer cancel()
    ch := countTo(ctx, 10)
    for i := range ch {
        if i > 5 {
            break
        }
        fmt.Println(i)
    }
}

```

The `countTo` function is modified to take a `context.Context` parameter in addition to `max`. The `for` loop in the goroutine is also changed. It is now a `for-select` loop with two cases. One tries to write to `ch`. The other case checks the channel returned by the `Done` method on the context. If it returns a value, you exit the `for-select` loop and the goroutine. Now, you have a way to prevent the goroutine from leaking when every value is read.

`countTo` 函数除了接受 `max` 参数外，还接受 `context.Context` 参数。goroutine 中的 `for` 循环也进行了修改。现在它是一个包含两个分支的 `for-select` 循环。一个分支尝试写入 `ch`。另一个分支检查上下文中 `Done` 方法返回的通道。如果返回值，则退出 `for-select` 循环和 goroutine。现在，您可以防止每次读取值时 goroutine 发生内存泄漏。

This leads to the question, how do you get the `Done` channel to return a value? It is triggered via *context cancellation*. In `main`, you create a context and a cancel function by using the `WithCancel` function in the `context` package. Next, you use `defer` to call `cancel` when the `main` function exits. This closes the channel returned by `Done`, and since a closed channel always returns a value, it ensures that the goroutine running `countTo` exits.

这就引出了一个问题：如何让 `Done` 通道返回值？这是通过上下文取消触发的。在 `main` 中，你使用 `context` 包中的 `WithCancel` 函数创建了一个上下文和一个取消函数。接下来，当 `main` 函数退出时，你使用 `defer` 调用 `cancel`。这将关闭 `Done` 返回的通道，由于关闭的通道总是会返回值，因此可以确保运行 `countTo` 的 goroutine 退出。

Using the context to terminate a goroutine is a very common pattern. It allows you to stop goroutines based on something from an earlier function

in the call stack. In [“Cancellation”](#), you’ll learn in detail how to use the context to tell one or more goroutines that it is time to shut down.

使用上下文来终止 goroutine 是一种非常常见的模式。它允许你根据调用栈中先前函数中的某些信息来停止 goroutine。在“取消”一节中，你将详细学习如何使用上下文来告知一个或多个 goroutine 该关闭了。

Know When to Use Buffered and Unbuffered Channels

了解何时使用缓冲通道和非缓冲通道

One of the most complicated techniques to master in Go concurrency is deciding when to use a buffered channel. By default, channels are unbuffered, and they are easy to understand: one goroutine writes and waits for another goroutine to pick up its work, like a baton in a relay race. Buffered channels are much more complicated. You have to pick a size, since buffered channels never have unlimited buffers. Proper use of a buffered channel means that you must handle the case where the buffer is full and your writing goroutine blocks waiting for a reading goroutine. So what is the proper use of a buffered channel?

在 Go 并发编程中，最难掌握的技巧之一就是何时使用缓冲通道。默认情况下，通道是无缓冲的，很容易理解：一个 goroutine 写入数据后，会等待另一个 goroutine 接力完成，就像接力赛中的接力棒一样。

缓冲通道则复杂得多。你必须指定缓冲区的大小，因为缓冲通道的缓冲区大小并非无限。正确使用缓冲通道意味着你必须处理缓冲区已满的情况，即写入 goroutine 会阻塞等待读取 goroutine。那么，缓冲通道的正确用法是什么呢？

The case for buffered channels is subtle. To sum it up in a single sentence: buffered channels are useful when you know how many goroutines you have launched, want to limit the number of goroutines you will launch, or want to limit the amount of work that is queued up.

缓冲通道的优势在于其微妙之处。简单来说：当你知道已启动的 goroutine 数量、想要限制要启动的 goroutine 数量，或者想要限制排队的工作量时，缓冲通道就很有用。

Buffered channels work great when you either want to gather data back from a set of goroutines that you have launched or want to limit concurrent usage. They are also helpful for managing the amount of work a system has queued up, preventing your services from falling behind and becoming overwhelmed. Here are a couple of examples to show how they can be used.

当您需要从已启动的 goroutine 集合中收集数据，或者需要限制并发使用量时，缓冲通道非常实用。它们还有助于管理系统排队的工作量，防止服务出现延迟和过载。以下是一些示例，展示了它们的使用方法。

In the first example, you are processing the first 10 results on a channel. To do this, you launch 10 goroutines, each of which writes its results to a buffered channel:

在第一个示例中，您正在处理通道上的前 10 个结果。为此，您启动了 10 个 goroutine，每个 goroutine 都将结果写入一个缓冲通道：

```
func processChannel(ch chan int) []int {
    const conc = 10
    results := make(chan int, conc)
    for i := 0; i < conc; i++ {
        go func() {
            v := <- ch
            results <- process(v)
        }()
    }
    var out []int
    for i := 0; i < conc; i++ {
        out = append(out, <-results)
    }
    return out
}
```

You know exactly how many goroutines have been launched, and you want each goroutine to exit as soon as it finishes its work. This means you can create a buffered channel with one space for each launched goroutine, and

have each goroutine write data to this goroutine without blocking. You can then loop over the buffered channel, reading out the values as they are written. When all the values have been read, you return the results, knowing that you aren't leaking any goroutines.

您确切地知道已启动了多少个 goroutine，并且希望每个 goroutine 在完成其工作后立即退出。这意味着您可以创建一个缓冲通道，每个已启动的 goroutine 占用一个空间，并让每个 goroutine 向该通道写入数据而不阻塞。然后，您可以循环遍历该缓冲通道，并在写入数据时读取它们。读取完所有值后，返回结果，这样就无需担心 goroutine 泄漏。

You can find this code in the *sample_code/buffered_channel_work* directory in the [Chapter 12 repository](#).

您可以在第 12 章存储库的 *sample_code/buffered_channel_work* 目录中找到此代码。

Implement Backpressure

实施反压

Another technique that can be implemented with a buffered channel is *backpressure*. It is counterintuitive, but systems perform better overall when their components limit the amount of work they are willing to perform. You

can use a buffered channel and a `select` statement to limit the number of simultaneous requests in a system:

另一种可以使用缓冲通道实现的技术是反压。这看似违反直觉，但当系统组件限制其愿意执行的工作量时，系统的整体性能会更好。您可以使用缓冲通道和 `select` 语句来限制系统中的并发请求数：

```
type PressureGauge struct {
    ch chan struct{}
}

func New(limit int) *PressureGauge {
    return &PressureGauge{
        ch: make(chan struct{}, limit),
    }
}

func (pg *PressureGauge) Process(f func()) error {
    select {
    case pg.ch <- struct{}{}:
        f()
        <-pg.ch
        return nil
    default:
        return errors.New("no more capacity")
    }
}
```

```
    }  
}
```

In this code, you create a struct containing a buffered channel that can hold a number of “tokens” and a function to run. Every time a goroutine wants to use the function, it calls `Process`. This is one of the rare examples of the same goroutine both reading and writing the same channel. The `select` tries to write a token to the channel. If it can, the function runs, and then a token is read to the buffered channel. If it can’t write a token, the `default` case runs, and an error is returned instead. Here’s a quick example that uses this code with the built-in HTTP server (you’ll learn more about working with HTTP in [“The Server”](#)):

在这段代码中，你创建了一个结构体，其中包含一个可以保存多个“令牌”的缓冲通道和一个要运行的函数。每次 goroutine 想要使用该函数时，它都会调用 `Process`。这是同一个 goroutine 同时读取和写入同一个通道的少数例子之一。`select` 尝试向通道写入一个令牌。如果可以写入，则函数运行，然后将一个令牌读取到缓冲通道中。如果无法写入令牌，则运行 `default` 的情况，并返回一个错误。以下是一个使用内置 HTTP 服务器的简单示例（你将在“服务器”部分了解更多关于使用 HTTP 的信息）：

```
func doThingThatShouldBeLimited() string {  
    time.Sleep(2 * time.Second)  
    return "done"  
}
```

```

}

func main() {
    pg := New(10)
    http.HandleFunc("/request", func(w
http.ResponseWriter, r *http.Request) {
        err := pg.Process(func() {

w.Write([]byte(doThingThatShouldBeLimited()))
        })
        if err != nil {

w.WriteHeader(http.StatusTooManyRequests)
            w.Write([]byte("Too many requests"))
        }
    })
    http.ListenAndServe(":8080", nil)
}

```

You can find this code in the *sample_code/backpressure* directory in the [Chapter 12 repository](#).

您可以在第 12 章存储库的 *sample_code/backpressure* 目录中找到此代码。

Turn Off a case in a select

在选定的案例中关闭某个案例

When you need to combine data from multiple concurrent sources, the `select` keyword is great. However, you need to properly handle closed channels. If one of the cases in a `select` is reading a closed channel, it will always be successful, returning the zero value. Every time that case is selected, you need to check to make sure that the value is valid and skip the case. If reads are spaced out, your program is going to waste a lot of time reading junk values. Even if there is lots of activity on the nonclosed channels, your program will still spend some portion of its time reading from the closed channel, since `select` chooses a case at random.

当您需要合并来自多个并发数据源的数据时，`select` 关键字非常有用。但是，您需要正确处理已关闭的通道。如果 `select` 中的某个用例读取的是一个已关闭的通道，则该用例总是会成功，并返回零值。每次选中该用例时，您都需要检查该值是否有效，如果无效则跳过该用例。如果读取操作间隔过长，您的程序将浪费大量时间读取无效值。即使未关闭的通道上有很多活动，您的程序仍然会花费一部分时间从已关闭的通道读取数据，因为 `select` 会随机选择一个用例。

When that happens, you rely on something that looks like an error: reading a `nil` channel. As you saw earlier, reading from or writing to a `nil`

channel causes your code to hang forever. While that is bad if it is triggered by a bug, you can use a `nil` channel to disable a `case` in a `select`. When you detect that a channel has been closed, set the channel's variable to `nil`. The associated case will no longer run, because the read from the `nil` channel never returns a value. Here is a `for-select` loop that reads from two channels until both are closed:

当这种情况发生时，你需要利用一个看似错误的操作：读取一个 `nil` 通道。正如你之前看到的，读取或写入一个 `nil` 通道会导致你的代码无限期地挂起。如果这是由 bug 触发的，那当然很糟糕，但你可以使用一个 `nil` 通道来禁用 `select` 通道中的 `case` 通道。当你检测到某个通道已关闭时，将该通道的变量设置为 `nil`。相关的 `case` 将不再运行，因为从 `nil` 通道读取数据永远不会返回任何值。以下是一个 `for-select` 循环，它会从两个通道读取数据，直到两个通道都关闭为止：

```
// in and in2 are channels
for count := 0; count < 2; {
    select {
        case v, ok := <-in:
            if !ok {
                in = nil // the case will never
succeed again!
                count++
                continue
            }
    }
}
```

```
        // process the v that was read from in
    case v, ok := <-in2:
        if !ok {
            in2 = nil // the case will never
succeed again!
            count++
            continue
        }
        // process the v that was read from in2
    }
}
```

You can try out this code on [The Go Playground](#) or in the *sample_code/close_case* directory in the [Chapter 12 repository](#).

您可以在 Go Playground 或第 12 章存储库的 *sample_code/close_case* 目录中尝试此代码。

Time Out Code

超时代码

Most interactive programs have to return a response within a certain amount of time. One of the things that you can do with concurrency in Go is manage how much time a request (or a part of a request) has to run. Other languages introduce additional features on top of promises or futures to add

this functionality, but Go's timeout idiom shows how you build complicated features from existing parts. Let's take a look:

大多数交互式程序都需要在一定时间内返回响应。在 Go 语言中，并发机制的一个作用就是控制请求（或请求的一部分）的运行时间。其他语言会在 Promise 或 Future 的基础上引入额外的特性来实现此功能，但 Go 的超时机制展示了如何利用现有组件构建复杂的功能。让我们来看一下：

```
func timeLimit[T any](worker func() T, limit
time.Duration) (T, error) {
    out := make(chan T, 1)
    ctx, cancel :=
context.WithTimeout(context.Background(), limit)
    defer cancel()
    go func() {
        out <- worker()
    }()
    select {
    case result := <-out:
        return result, nil
    case <-ctx.Done():
        var zero T
        return zero, errors.New("work timed out")
    }
}
```

Whenever you need to limit how long an operation takes in Go, you'll see a variation on this pattern. I talk about the context in [Chapter 14](#) and cover using timeouts in detail in [“Contexts with Deadlines”](#). For now, all you need to know is that reaching the timeout cancels the context. The `Done` method on the context returns a channel that returns a value when the context is canceled by either timing out or when the context's `cancel` method is called. You create a timed context by using the `WithTimeout` function in the `context` package and specify how long to wait by using constants from the `time` package (I'll talk more about the `time` package in [“time”](#)).

在 Go 语言中，每当需要限制操作耗时时，你都会看到这种模式的各种变体。我在第 14 章讨论了上下文，并在“带有截止时间的上下文”一节中详细介绍了超时的使用。现在，你只需要知道，达到超时时间会取消上下文。上下文的 `Done` 方法返回一个通道，当上下文因超时或调用其 `cancel` 方法而被取消时，该通道会返回一个值。你可以使用 `context` 包中的 `WithTimeout` 函数创建定时上下文，并使用 `time` 包中的常量指定等待时间（我将在“时间”一节中详细介绍 `time` 包）。

Once the context is set up, you run the worker in a goroutine and then use `select` to choose between two cases. The first case reads the value from the `out` channel when the work completes. The second case waits for the channel returned by the `Done` method to return a value, just as you saw in [“Use the Context to Terminate Goroutines”](#). If it does, you return a timeout

error. You write to a buffered channel of size 1 so that the channel write in the goroutine will complete even if `Done` is triggered first.

上下文设置完成后，在 goroutine 中运行 worker，然后使用 `select` 在两种情况之间进行选择。第一种情况在工作完成后从 `out` 通道读取值。第二种情况等待 `Done` 方法返回的通道返回值，就像你在“使用上下文终止 goroutine”中看到的那样。如果通道返回值，则返回超时错误。写入操作使用大小为 1 的缓冲通道，这样即使 `Done` 先被触发，goroutine 中的通道写入操作也能完成。

You can try out this code on [The Go Playground](#) or in the *sample_code/time_out* directory in the [Chapter 12 repository](#).

您可以在 Go Playground 或第 12 章存储库中的 *sample_code/time_out* 目录中尝试此代码。

NOTE

笔记

If `timeLimit` exits before the goroutine finishes processing, the goroutine continues to run, eventually writing the returned value to the buffered channel and exiting. You just don't do anything with the result that is returned. If you want to stop work in a goroutine when you are no longer waiting for it to complete, use context cancellation, which I'll discuss in [“Cancellation”](#).

如果 `timeLimit` 在 goroutine 完成处理之前退出，goroutine 会继续运行，最终将返回值写入缓冲通道并退出。只是你不会返回的结果做任何处理。如果你想在不再等待 goroutine 完成时停止其工作，请使用上下文取消，我将在“取消”部分讨论。

Use WaitGroups

使用等待组

Sometimes one goroutine needs to wait for multiple goroutines to complete their work. If you are waiting for a single goroutine, you can use the context cancellation pattern that you saw earlier. But if you are waiting on several goroutines, you need to use a `WaitGroup`, which is found in the `sync` package in the standard library. Here is a simple example, which you can run on [The Go Playground](#) or in the `sample_code/waitgroup` directory in the [Chapter 12 repository](#):

有时，一个 goroutine 需要等待多个 goroutine 完成它们的工作。如果只等待一个 goroutine，可以使用之前看到的上下文取消模式。但如果要等待多个 goroutine，则需要使用 `WaitGroup`，它位于标准库的 `sync` 包中。以下是一个简单的示例，您可以在 Go Playground 或第 12 章代码库的 `sample_code/waitgroup` 目录中运行它：

```
func main() {  
    var wg sync.WaitGroup  
    wg.Add(3)  
    go func() {  
        defer wg.Done()  
        doThing1()  
    }()  
    go func() {
```

```
        defer wg.Done()
        doThing2()
    }()
    go func() {
        defer wg.Done()
        doThing3()
    }()
    wg.Wait()
}
```

A `sync.WaitGroup` doesn't need to be initialized, just declared, as its zero value is useful. There are three methods on `sync.WaitGroup`: `Add`, which increments the counter of goroutines to wait for; `Done`, which decrements the counter and is called by a goroutine when it is finished; and `Wait`, which pauses its goroutine until the counter hits zero. `Add` is usually called once, with the number of goroutines that will be launched. `Done` is called within the goroutine. To ensure that it is called, even if the goroutine panics, you use a `defer`.

`sync.WaitGroup` 不需要初始化，只需声明即可，因为它的零值很有用。 `sync.WaitGroup` 上有三个方法：`Add`，用于递增等待的 goroutine 计数器；`Done`，用于递减计数器，并在 goroutine 完成后被调用；以及 `Wait`，用于暂停其 goroutine，直到计数器归零。 `Add` 通常只调用一次，并传入要启动的 goroutine 数量。 `Done` 在 goroutine

内部调用。为了确保即使 goroutine 发生 panic 也能调用它，可以使用 `defer`。

You'll notice that you don't explicitly pass the `sync.WaitGroup`. There are two reasons. The first is that you must ensure that every place that uses a `sync.WaitGroup` is using the same instance. If you pass the `sync.WaitGroup` to the goroutine function and don't use a pointer, then the function has a *copy* and the call to `Done` won't decrement the original `sync.WaitGroup`. By using a closure to capture the `sync.WaitGroup`, you are assured that every goroutine is referring to the same instance.

你会注意到你没有显式传递 `sync.WaitGroup`。原因有二。首先，你必须确保所有使用 `sync.WaitGroup` 的地方都指向同一个实例。如果你将 `sync.WaitGroup` 传递给 goroutine 函数，并且没有使用指针，那么该函数会持有一个副本，对 `Done` 的调用不会递减原始的 `sync.WaitGroup`。通过使用闭包来捕获 `sync.WaitGroup`，可以确保每个 goroutine 都指向同一个实例。

The second reason is design. Remember, you should keep concurrency out of your API. As you saw with channels earlier, the usual pattern is to launch a goroutine with a closure that wraps the business logic. The closure manages issues around concurrency, and the function provides the algorithm.

第二个原因是设计。记住，你应该避免在 API 中引入并发。正如你之前看到的通道，通常的做法是启动一个带有闭包的 goroutine，该闭封装了业务逻辑。闭包负责处理并发问题，而函数则提供具体的算法。

Let's take a look at a more realistic example. As I mentioned earlier, when you have multiple goroutines writing to the same channel, you need to make sure that the channel being written to is closed only once. A `sync.WaitGroup` is perfect for this. Let's see how it works in a function that processes the values in a channel concurrently, gathers the results into a slice, and returns the slice:

让我们来看一个更实际的例子。正如我之前提到的，当多个 goroutine 向同一个通道写入数据时，你需要确保该通道只被关闭一次。`sync.WaitGroup` 正好可以满足这个需求。我们来看一个函数是如何实现这一点的：该函数并发处理通道中的值，将结果收集到一个切片中，然后返回该切片。

```
func processAndGather[T, R any](in <-chan T,
processor func(T) R, num int) []R {
    out := make(chan R, num)
    var wg sync.WaitGroup
    wg.Add(num)
    for i := 0; i < num; i++ {
        go func() {
            defer wg.Done()
```

```

        for v := range in {
            out <- processor(v)
        }
    }()
}
go func() {
    wg.Wait()
    close(out)
}()
var result []R
for v := range out {
    result = append(result, v)
}
return result
}

```

In this example, you launch a monitoring goroutine that waits until all the processing goroutines exit. When they do, the monitoring goroutine calls `close` on the output channel. The `for-range` channel loop exits when `out` is closed and the buffer is empty. Finally, the function returns the processed values. You can try out this code in the *sample_code/waitgroup_close_once* directory in the [Chapter 12 repository](#).

在这个示例中，您启动了一个监控 goroutine，它会等待所有处理 goroutine 退出。当所有处理 goroutine 退出后，监控 goroutine 会调用输出通道上的 `close` 函数。当 `out` 关闭且缓冲区为空时，`for-`

`range` 通道循环退出。最后，该函数返回已处理的值。您可以在第 12 章代码库的 `sample_code/waitgroup_close_once` 目录中尝试这段代码。

While `WaitGroups` are handy, they shouldn't be your first choice when coordinating goroutines. Use them only when you have something to clean up (like closing a channel they all write to) after all your worker goroutines exit.

虽然 `WaitGroups` 很方便，但在协调 goroutine 时，它们不应该是首选。只有在所有 worker goroutine 退出后，需要清理某些东西（例如关闭它们共同写入的通道）时，才应该使用它们。

GOLANG.ORG/X AND ERRGROUP.GROUP

GOLANG.ORG/X 和 ERRGROUP.GROUP

The Go authors maintain a set of utilities that supplements the standard library. Collectively known as the `golang.org/x` packages, they include a package called `errgroup` that contains a type, `errgroup.Group`. It builds on top of `WaitGroup` to create a set of goroutines that stop processing when one of them returns an error. Read the `errgroup.Group` [documentation](#) to learn more.

Go 语言的作者维护了一组用于补充标准库的实用工具。这些工具统称为 `golang.org/x` 个包，其中包括一个名为 `errgroup` 的包，其中包含一个类型 `errgroup.Group`。它基于 `WaitGroup` 构建，用于创建一组 goroutine，当其中一个 goroutine 返回错误时，程序将停止运行。阅读 `errgroup.Group` 的文档以了解更多信息。

Run Code Exactly Once

仅运行一次代码

As I covered in [“Avoiding the init Function if Possible”](#), `init` should be reserved for initialization of effectively immutable package-level state. However, sometimes you want to *lazy load*, or call some initialization code exactly once after program launch time. This is usually because the initialization is relatively slow and may not even be needed every time your

program runs. The `sync` package includes a handy type called `Once` that enables this functionality. Let's take a quick look at how it works. Say you have some code that takes a long time to initialize:

正如我在“尽可能避免使用 `init` 函数”一文中提到的，`init` 应该保留用于初始化实际上不可变的包级状态。然而，有时您可能希望延迟加载，或者在程序启动后仅调用一次初始化代码。这通常是因为初始化过程相对较慢，甚至可能并非每次程序运行时都需要执行。`sync` 包包含一个名为 `Once` 的便捷类型，可以实现此功能。让我们快速了解一下它的工作原理。假设您有一些代码需要很长时间才能初始化：

```
type SlowComplicatedParser interface {  
    Parse(string) string  
}  
  
func initParser() SlowComplicatedParser {  
    // do all sorts of setup and loading here  
}
```

Here's how you use `sync.Once` to delay initialization of a `SlowComplicatedParser`:

以下是如何使用 `sync.Once` 来延迟 `SlowComplicatedParser` 的初始化：

```
var parser SlowComplicatedParser
var once sync.Once

func Parse(dataToParse string) string {
    once.Do(func() {
        parser = initParser()
    })
    return parser.Parse(dataToParse)
}
```

There are two package-level variables: `parser`, which is of type `SlowComplicatedParser`, and `once`, which is of type `sync.Once`. As with `sync.WaitGroup`, you do not have to configure an instance of `sync.Once`. This is an example of *making the zero value useful*, which is a common pattern in Go.

这里有两个包级变量：`parser`，其类型为 `SlowComplicatedParser`；以及 `once`，其类型为 `sync.Once`。与 `sync.WaitGroup` 一样，您无需配置 `sync.Once` 的实例。这是一个利用零值的例子，也是 Go 语言中常见的做法。

As with `sync.WaitGroup`, you must make sure not to make a copy of an instance of `sync.Once`, because each copy has its own state to indicate whether it has already been used. Declaring a `sync.Once` instance inside a function is usually the wrong thing to do, as a new instance will be

created on every function call and there will be no memory of previous invocations.

与 `sync.WaitGroup` 一样，您必须确保不要复制 `sync.Once` 的实例，因为每个副本都有自己的状态来指示它是否已被使用。在函数内部声明 `sync.Once` 实例通常是错误的做法，因为每次函数调用都会创建一个新实例，并且不会保留先前调用的记忆。

In the example, you want to make sure that `parser` is initialized only once, so you set the value of `parser` from within a closure that's passed to the `Do` method on `once`. If `Parse` is called more than once, `once.Do` will not execute the closure again.

在这个例子中，你需要确保 `parser` 只初始化一次，因此你需要在一个闭包中设置 `parser` 的值，并将这个闭包传递给 `once` 的 `Do` 方法。如果 `Parse` 被调用多次，`once.Do` 将不会再次执行该闭包。

You can try out this code on [The Go Playground](#) or in the `sample_code/sync_once` directory in the [Chapter 12 repository](#).

您可以在 Go Playground 或第 12 章存储库中的 `sample_code/sync_once` 目录中尝试此代码。

Go 1.21 added helper functions that make it easier to run a function exactly once: `sync.OnceFunc`, `sync.OnceValue`, and `sync.OnceValues`. The only difference between the three functions is the number of return

values of the passed-in function (zero, one, or two, respectively). The `sync.OnceValue` and `sync.OnceValues` functions are generic, so they adapt to the type of the original function's return values.

Go 1.21 新增了三个辅助函数，可以更方便地执行函数的单次运行：`sync.OnceFunc`、`sync.OnceValue` 和 `sync.OnceValues`。这三个函数之间的唯一区别在于传入函数的返回值数量（分别为零个、一个或两个）。`sync.OnceValue` 和 `sync.OnceValues` 函数是泛型函数，因此它们会根据原始函数的返回值类型进行调整。

Using these functions is straightforward. You pass the original function to the helper function and get back a function that calls the original function only once. The values returned by the original function are cached. Here's how you can use `sync.OnceValue` to rewrite the `Parse` function in the previous example:

使用这些函数非常简单。您只需将原始函数传递给辅助函数，即可获得一个只调用原始函数一次的函数。原始函数返回的值会被缓存。以下是如何使用 `sync.OnceValue` 重写上例中的 `Parse` 函数：

```
var initParserCached func() SlowComplicatedParser
= sync.OnceValue(initParser)

func Parse(dataToParse string) string {
    parser := initParserCached()
```

```
    return parser.Parse(dataToParse)
}
```

The package-level `initParserCached` variable is assigned the function returned by `sync.OnceValue` when `initParser` is passed to it. The first time `initParserCached` is called, `initParser` is also called, and its return value is cached. Each subsequent time `initParserCached` is called, the cached value is returned. This means you can get rid of the `parser` package-level variable.

包级变量 `initParserCached` 被赋值为当 `initParser` 传递给它时 `sync.OnceValue` 返回的函数。首次调用 `initParserCached` 时，也会调用 `initParser`，并将其返回值缓存。之后每次调用 `initParserCached` 时，都会返回缓存的值。这意味着您可以移除包级变量 `parser`。

You can try out this code on [The Go Playground](#) or in the `sample_code/sync_value` directory in the [Chapter 12 repository](#).

您可以在 Go Playground 或第 12 章存储库的 `sample_code/sync_value` 目录中尝试此代码。

Put Your Concurrent Tools Together

整合您的并发工具

Let's go back to the example from the first section in the chapter. You have a function that calls three web services. You send data to two of those services, and then take the results of those two calls and send them to the third, returning the result. The entire process must take less than 50 milliseconds, or an error is returned.

让我们回到本章第一节例子。假设你有一个函数，它会调用三个 Web 服务。你向其中两个服务发送数据，然后获取这两个调用的结果，并将它们发送给第三个服务，最后返回结果。整个过程必须在 50 毫秒内完成，否则将返回错误。

You'll start with the function you invoke:

首先，你要调用以下函数：

```
func GatherAndProcess(ctx context.Context, data
Input) (COut, error) {
    ctx, cancel := context.WithTimeout(ctx,
50*time.Millisecond)
    defer cancel()

    ab := newABProcessor()
```

```
    ab.start(ctx, data)
    inputC, err := ab.wait(ctx)
    if err != nil {
        return COut{}, err
    }

    c := newCProcessor()
    c.start(ctx, inputC)
    out, err := c.wait(ctx)
    return out, err
}
```

The first thing to do is set up a `context.Context` that times out in 50 milliseconds, as you saw in [“Time Out Code”](#).

首先要做的是设置一个 `context.Context`，使其在 50 毫秒后超时，正如你在“超时代码”中看到的那样。

After creating the context, use a `defer` to make sure the context’s `cancel` function is called. As I’ll discuss in [“Cancellation”](#), you must call this function, or resources leak.

创建上下文后，使用 `defer` 来确保调用上下文的 `cancel` 函数。正如我将在“取消”部分讨论的那样，您必须调用此函数，否则会导致资源泄漏。

You are using `A` and `B` as the names of the two services that are called in parallel, so you'll make a new `abProcessor` to call them. You then start processing with a call to the `start` method, and then wait for your results with a call to the `wait` method.

您使用 `A` 和 `B` 作为并行调用的两个服务的名称，因此您将创建一个新的 `abProcessor` 来调用它们。然后，您通过调用 `start` 方法开始处理，并通过调用 `wait` 方法等待结果。

When `wait` returns, you do a standard error check. If all is well, you call the third service, which you are calling `C`. The logic is the same as before. Processing is started with a call to the `start` method on the `cProcessor` and then you wait for the result with a call to the `wait` method on `cProcessor`. You then return the result of the `wait` method call.

当 `wait` 返回时，进行标准错误检查。如果一切正常，则调用第三个服务，即 `C`。逻辑与之前相同。首先调用 `cProcessor` 上的 `start` 方法启动处理，然后调用 `cProcessor` 上的 `wait` 方法等待结果。最后，返回 `wait` 方法调用的结果。

This looks a lot like standard sequential code without concurrency. Let's look at the `abProcessor` and `cProcessor` to see how the concurrency happens:

这看起来很像没有并发的标准顺序代码。让我们看一下 `abProcessor` 和 `cProcessor`，看看并发是如何实现的：

```
type abProcessor struct {  
    outA chan aOut  
    outB chan bOut  
    errs chan error  
}  
  
func newABProcessor() *abProcessor {  
    return &abProcessor{  
        outA: make(chan aOut, 1),  
        outB: make(chan bOut, 1),  
        errs: make(chan error, 2),  
    }  
}
```

The `abProcessor` has three fields, all of which are channels. They are `outA`, `outB`, and `errs`. You'll see how you use all these channels next. Notice that every channel is buffered, so that the goroutines that write to them can exit after writing without waiting for a read to happen. The `errs` channel has a buffer size of `2`, because it could have up to two errors written to it.

`abProcessor` 包含三个字段，它们都是通道。这三个字段分别是 `outA`、`outB` 和 `errs`。接下来您将看到如何使用所有这些通道。

请注意，每个通道都进行了缓冲，这样写入通道的 goroutine 就可以在写入后立即退出，而无需等待读取操作完成。`errs` 通道的缓冲区大小为 2，因为它最多可以写入两个错误。

Next is the implementation of the `start` method:

接下来是 `start` 方法的实现：

```
func (p *abProcessor) start(ctx context.Context,
data Input) {
    go func() {
        aOut, err := getResultA(ctx, data.A)
        if err != nil {
            p.errs <- err
            return
        }
        p.outA <- aOut
    }()
    go func() {
        bOut, err := getResultB(ctx, data.B)
        if err != nil {
            p.errs <- err
            return
        }
        p.outB <- bOut
    }()
}
```

The `start` method launches two goroutines. The first one calls `getResultA` to talk to the `A` service. If the call returns an error, you write to the `errs` channel. Otherwise, you write to the `outA` channel. Since these channels are buffered, the goroutine will not hang, no matter which channel is written to. Also notice that you are passing the context along to `getResultA`, which allows it to cancel processing if the timeout happens.

`start` 方法启动两个 goroutine。第一个 goroutine 调用 `getResultA` 与 `A` 服务通信。如果调用返回错误，则写入 `errs` 通道。否则，写入 `outA` 通道。由于这些通道都经过缓冲，因此无论写入哪个通道，goroutine 都不会挂起。另请注意，您将上下文传递给了 `getResultA`，这使得它在超时时可以取消处理。

The second goroutine is just like the first, only it calls `getResultB` and writes to the `outB` channel on success.

第二个 goroutine 与第一个 goroutine 完全相同，只是在成功时会调用 `getResultB` 并写入 `outB` 通道。

Let's see what the `wait` method for `ABProcessor` looks like:

让我们来看看 `ABProcessor` 的 `wait` 方法是什么样子的：

```
func (p *abProcessor) wait(ctx context.Context)
(cIn, error) {
```

```

var cData cIn
for count := 0; count < 2; count++ {
    select {
    case a := <-p.outA:
        cData.a = a
    case b := <-p.outB:
        cData.b = b
    case err := <-p.errs:
        return cIn{}, err
    case <-ctx.Done():
        return cIn{}, ctx.Err()
    }
}
return cData, nil
}

```

The `wait` method on `abProcessor` is the most complicated method you need to implement. It populates a struct of type `cIn`, which holds the data returned from calling the `A` service and the `B` service. You define your output variable, `cData`, as type `cIn`. You then have a `for` loop that counts to two, since you need to read from two channels to finish successfully. Inside the loop, you have a `select` statement. If you read a value on `outA`, you set the `a` field on `cData`. If you read a value on `outB`, you set the `b` field on `cData`. If you read a value on the `errs` channel, you return immediately with the error. Finally, if the context times out, you return immediately with the error from the context's `Err` method.

`abProcessor` 上的 `wait` 方法是你需要实现的最复杂的方法。它会填充一个类型为 `cIn` 的结构体，该结构体保存调用 `A` 服务和 `B` 服务返回的数据。你需要将输出变量 `cData` 定义为类型 `cIn`。然后，你需要一个计数到 2 的 `for` 循环，因为你需从两个通道读取数据才能成功完成。在循环内部，你需要一个 `select` 语句。如果你从 `outA` 读取到一个值，则将 `cData` 的 `a` 字段设置为该结构体的值。如果你从 `outB` 读取到一个值，则将 `cData` 的 `b` 字段设置为该结构体的值。如果从 `errs` 通道读取值，则立即返回错误。最后，如果上下文超时，则立即返回上下文的 `Err` 方法中的错误。

Once you have read a value from both the `p.outA` channel and the `p.outB` channel, you exit the loop and return the input that you're going to use with the `cProcessor`.

从 `p.outA` 通道和 `p.outB` 通道读取值后，退出循环并返回要与 `cProcessor` 一起使用的输入。

The `cProcessor` looks like a simpler version of the `abProcessor`:

`cProcessor` 看起来像是 `abProcessor` 的简化版本：

```
type cProcessor struct {  
    outC chan COut  
    errs chan error  
}
```

```

func newCProcessor() *cProcessor {
    return &cProcessor{
        outC: make(chan COut, 1),
        errs: make(chan error, 1),
    }
}

func (p *cProcessor) start(ctx context.Context,
inputC cIn) {
    go func() {
        cOut, err := getResultC(ctx, inputC)
        if err != nil {
            p.errs <- err
            return
        }
        p.outC <- cOut
    }()
}

func (p *cProcessor) wait(ctx context.Context)
(COut, error) {
    select {
    case out := <-p.outC:
        return out, nil
    case err := <-p.errs:
        return COut{}, err
    case <-ctx.Done():
        return COut{}, ctx.Err()
    }
}

```

```
    }  
}
```

The `cProcessor` struct has one out channel and one error channel.

`cProcessor` 结构体有一个输出通道和一个错误通道。

The `start` method on the `cProcessor` looks like the `start` method on the `abProcessor`. It launches a goroutine that calls `getResultC` with your input data, writes to the `errs` channel on an error, and writes to the `outC` channel on success.

`cProcessor` 上的 `start` 方法与 `abProcessor` 上的 `start` 方法类似。它会启动一个 goroutine，该 goroutine 会使用你的输入数据调用 `getResultC`，在出错时写入 `errs` 通道，在成功时写入 `outC` 通道。

Finally, the `wait` method on the `cProcessor` is a simple `select` statement that checks whether there's a value to read from the `outC` channel, the `errs` channel, or the context's `Done` channel.

最后，`cProcessor` 上的 `wait` 方法是一个简单的 `select` 语句，用于检查是否有值要从 `outC` 通道、`errs` 通道或上下文的 `Done` 通道读取。

By structuring code with goroutines, channels, and `select` statements, you separate the individual steps, allow independent parts to run and complete in any order, and cleanly exchange data between the dependent parts. In addition, you make sure that no part of the program hangs, and you properly handle timeouts set both within this function and from earlier functions in the call history. If you are not convinced that this is a better method for implementing concurrency, try to implement this in another language. You might be surprised at how difficult it is.

通过使用 goroutine、channel 和 `select` 语句来构建代码结构，您可以将各个步骤分离，允许独立的部分以任意顺序运行和完成，并在依赖部分之间清晰地交换数据。此外，您还可以确保程序不会挂起，并妥善处理本函数内部以及调用历史记录中先前函数设置的超时。如果您仍然不相信这是实现并发的更佳方法，不妨尝试用其他语言实现它。您可能会惊讶于它的难度。

You can find the code for this concurrent pipeline in the *sample_code/pipeline* directory in the [Chapter 12 repository](#).

您可以在第 12 章存储库的 *sample_code/pipeline* 目录中找到此并发管道的代码。

When to Use Mutexes Instead of

Channels

何时使用互斥锁而不是通道

If you've had to coordinate access to data across threads in other programming languages, you have probably used a *mutex*. This is short for *mutual exclusion*, and the job of a mutex is to limit the concurrent execution of some code or access to a shared piece of data. This protected part is called the *critical section*.

如果你曾在其他编程语言中协调过跨线程的数据访问，那么你很可能用过互斥锁（mutex）。互斥锁是“互斥”（mutual exclusion）的缩写，它的作用是限制某些代码的并发执行或对共享数据的访问。这部分受保护的代码被称为临界区（critical section）。

There are good reasons Go's creators designed channels and `select` to manage concurrency. The main problem with mutexes is that they obscure the flow of data through a program. When a value is passed from goroutine to goroutine over a series of channels, the data flow is clear. Access to the value is localized to a single goroutine at a time. When a mutex is used to protect a value, there is nothing to indicate which goroutine currently has ownership of the value, because access to the value is shared by all the concurrent processes. That makes it hard to understand the order of processing. There is a saying in the Go community to describe this

philosophy: “Share memory by communicating; do not communicate by sharing memory.”

Go 的创建者设计通道和 `select` 来管理并发是有充分理由的。互斥锁的主要问题在于它们会模糊程序中的数据流。当一个值通过一系列通道在 goroutine 之间传递时，数据流清晰明了。对该值的访问一次仅限于一个 goroutine。而当使用互斥锁来保护一个值时，由于所有并发进程共享对该值的访问，因此无法指示哪个 goroutine 当前拥有该值的所有权。这使得处理顺序难以理解。Go 社区有一句格言来描述这种理念：“通过沟通来共享内存；不要通过共享内存来沟通。”

That said, sometimes it is clearer to use a mutex, and the Go standard library includes mutex implementations for these situations. The most common case is when your goroutines read or write a shared value, but don't process the value. Let's use an in-memory scoreboard for a multiplayer game as an example. You'll first see how to implement this using channels. Here's a function that you can launch as a goroutine to manage the scoreboard:

也就是说，有时使用互斥锁（mutex）会更清晰明了，Go 标准库也包含了针对这种情况的互斥锁实现。最常见的情况是，当你的 goroutine 读取或写入一个共享值，但并不处理该值时。我们以多人游戏的内存记分板为例。首先，你将看到如何使用通道（channel）来实现它。以下是一个可以作为 goroutine 启动的函数，用于管理记分板：

```

func scoreboardManager(ctx context.Context, in <-
chan func(map[string]int)) {
    scoreboard := map[string]int{}
    for {
        select {
            case <-ctx.Done():
                return
            case f := <-in:
                f(scoreboard)
        }
    }
}

```

This function declares a map and then listens on one channel for a function that reads or modifies the map and on a context's Done channel to know when to shut down. Let's create a type with a method to write a value to the map:

此函数声明一个映射，然后监听一个通道以接收读取或修改该映射的函数，并监听上下文的 Done 通道以了解何时关闭。让我们创建一个类型，并添加一个向映射写入值的方法：

```

type ChannelScoreboardManager chan
func(map[string]int)

func NewChannelScoreboardManager(ctx

```

```

context.Context) ChannelScoreboardManager {
    ch := make(ChannelScoreboardManager)
    go scoreboardManager(ctx, ch)
    return ch
}

func (csm ChannelScoreboardManager) Update(name
string, val int) {
    csm <- func(m map[string]int) {
        m[name] = val
    }
}

```

The update method is very straightforward: just pass a function that puts a value into the map. But how about reading from the scoreboard? You need to return a value back. That means creating a channel that's written to within the passed-in function:

更新方法非常简单：只需传递一个将值放入映射表的函数即可。但是如何从记分板读取数据呢？你需要返回一个值。这意味着要创建一个通道，并在传入的函数中写入该通道：

```

func (csm ChannelScoreboardManager) Read(name
string) (int, bool) {
    type Result struct {
        out int
    }
}

```

```
        ok bool
    }
    resultCh := make(chan Result)
    csm <- func(m map[string]int) {
        out, ok := m[name]
        resultCh <- Result{out, ok}
    }
    result := <-resultCh
    return result.out, result.ok
}
```

While this code works, it's cumbersome and allows only a single reader at a time. A better approach is to use a mutex. The standard library has two mutex implementations, both in the `sync` package. The first, `Mutex`, has two methods, `Lock` and `Unlock`. Calling `Lock` causes the current goroutine to pause as long as another goroutine is currently in the critical section. When the critical section is clear, the lock is *acquired* by the current goroutine, and the code in the critical section is executed. A call to the `Unlock` method on the `Mutex` marks the end of the critical section.

虽然这段代码可以运行，但它很繁琐，而且一次只能有一个读取者。更好的方法是使用互斥锁。标准库提供了两个互斥锁实现，都在 `sync` 包中。第一个实现 `Mutex` 有两个方法，分别是 `Lock` 和 `Unlock`。调用 `Lock` 会导致当前 goroutine 暂停，直到另一个 goroutine 处于临界区为止。当临界区清空时，当前 goroutine 会获取

锁，并执行临界区中的代码。调用 `Mutex` 上的 `Unlock` 方法标志着临界区的结束。

The second mutex implementation, called `RWMutex`, allows you to have both reader locks and writer locks. While only one writer can be in the critical section at a time, reader locks are shared; multiple readers can be in the critical section at once. The writer lock is managed with the `Lock` and `Unlock` methods, while the reader lock is managed with `RLock` and `RUnlock` methods.

第二个互斥锁实现，称为 `RWMutex`，允许同时拥有读锁和写锁。虽然同一时间只能有一个写者进入临界区，但读锁是共享的；多个读者可以同时进入临界区。写锁由 `Lock` 和 `Unlock` 方法管理，而读锁由 `RLock` 和 `RUnlock` 方法管理。

Anytime you acquire a mutex lock, you must make sure that you release the lock. Use a `defer` statement to call `Unlock` immediately after calling `Lock` or `RLock`:

每次获取互斥锁后，必须确保释放该锁。使用 `defer` 语句在调用 `Lock` 或 `RLock` 后立即调用 `Unlock`：

```
type MutexScoreboardManager struct {  
    l          sync.RWMutex  
    scoreboard map[string]int  
}
```

```

func NewMutexScoreboardManager()
*MutexScoreboardManager {
    return &MutexScoreboardManager{
        scoreboard: map[string]int{},
    }
}

func (msm *MutexScoreboardManager) Update(name
string, val int) {
    msm.l.Lock()
    defer msm.l.Unlock()
    msm.scoreboard[name] = val
}

func (msm *MutexScoreboardManager) Read(name
string) (int, bool) {
    msm.l.RLock()
    defer msm.l.RUnlock()
    val, ok := msm.scoreboard[name]
    return val, ok
}

```

You can find the example in the *sample_code/mutex* directory in the [Chapter 12 repository](#).

您可以在第 12 章存储库的 *sample_code/mutex* 目录中找到示例。

Now that you've seen an implementation using mutexes, carefully consider your options before using one. Katherine Cox-Buday's excellent book [Concurrency in Go](#) (O'Reilly) includes a decision tree to help you decide whether to use channels or mutexes:

既然你已经了解了使用互斥锁的实现方式，在使用之前请仔细考虑其他选项。Katherine Cox-Buday 的优秀著作《Go 语言并发编程》（O'Reilly 出版社）中包含一个决策树，可以帮助你决定是使用通道还是互斥锁：

- If you are coordinating goroutines or tracking a value as it is transformed by a series of goroutines, use channels.
如果您要协调 goroutine 或跟踪某个值在经过一系列 goroutine 转换时的变化，请使用通道。
- If you are sharing access to a field in a struct, use mutexes.
如果要共享对结构体中某个字段的访问权限，请使用互斥锁。
- If you discover a critical performance issue when using channels (see [“Using Benchmarks”](#) to learn how to do this), and you cannot find any other way to fix the issue, modify your code to use a mutex.
如果您在使用通道时发现严重的性能问题（请参阅“使用基准测试”了解如何操作），并且您找不到任何其他方法来修复该问题，请修改您的代码以使用互斥锁。

Since your scoreboard is a field in a struct and there's no transfer of the scoreboard, using a mutex makes sense. This is a good use for a mutex only

because the data is stored in-memory. When data is stored in external services, like an HTTP server or a database, don't use a mutex to guard access to the system.

由于你的记分板数据是结构体中的一个字段，并且记分板数据本身不会发生传输，因此使用互斥锁是合理的。只有当数据存储在内存中时，互斥锁才适用。当数据存储在外部服务（例如 HTTP 服务器或数据库）中时，不要使用互斥锁来保护对系统的访问。

Mutexes require you to do more bookkeeping. For example, you must correctly pair locks and unlocks, or your programs will likely deadlock. The example both acquires and releases the locks within the same method. Another issue is that mutexes in Go aren't *reentrant*. If a goroutine tries to acquire the same lock twice, it deadlocks, waiting for itself to release the lock. This is different from languages like Java, where locks are reentrant.

互斥锁需要你做更多簿记工作。例如，你必须正确地配对锁和解锁操作，否则你的程序很可能会死锁。示例中在同一个方法中同时进行了锁的获取和释放操作。另一个问题是 Go 中的互斥锁是不可重入的。如果一个 goroutine 尝试两次获取同一个锁，它就会死锁，因为需要等待自身释放该锁。这与 Java 等语言不同，在 Java 中，锁是可重入的。

Nonreentrant locks make it tricky to acquire a lock in a function that calls itself recursively. You must release the lock before the recursive function

call. In general, be careful when holding a lock while making a function call, because you don't know what locks are going to be acquired in those calls. If your function calls another function that tries to acquire the same mutex lock, the goroutine deadlocks.

不可重入锁使得在递归调用自身的函数中获取锁变得棘手。你必须在递归函数调用之前释放锁。一般来说，在进行函数调用时持有锁要格外小心，因为你无法预知这些调用中会获取哪些锁。如果你的函数调用了另一个试图获取同一个互斥锁的函数，则会导致 goroutine 死锁。

Like `sync.WaitGroup` and `sync.Once`, mutexes must never be copied. If they are passed to a function or accessed as a field on a struct, it must be via a pointer. If a mutex is copied, its lock won't be shared.

与 `sync.WaitGroup` 和 `sync.Once` 一样，互斥锁绝对不能被复制。如果要将其传递给函数或作为结构体的字段访问，则必须使用指针。如果互斥锁被复制，其锁不会被共享。

WARNING

警告

Never try to access a variable from multiple goroutines unless you acquire a mutex for that variable first. It can cause odd errors that are hard to trace. See [“Finding Concurrency Problems with the Data Race Detector”](#) to learn how to detect these problems.

切勿尝试从多个 goroutine 访问同一个变量，除非您事先为该变量获取了互斥锁。这可能会导致难以追踪的奇怪错误。请参阅“使用数据竞争检测器查找并发问题”了解如何检测这些问题。

SYNC.MAP—THIS IS NOT THE MAP YOU ARE LOOKING FOR

同步地图——这不是您要查找的地图

When looking through the `sync` package, you'll find a type called `Map`. It provides a concurrency-safe version of Go's built-in `map`. Because of trade-offs in its implementation, `sync.Map` is appropriate only in very specific situations:

在查看 `sync` 包时，你会发现一个名为 `Map` 的类型。它提供了 Go 内置 `map` 的并发安全版本。由于其实现上的权衡，`sync.Map` 仅适用于非常特定的情况：

- When you have a shared map where key-value pairs are inserted once and read many times
当您有一个共享映射，其中键值对插入一次但读取多次时
- When goroutines share the map, but don't access each other's keys and values

当 goroutine 共享同一个 map，但彼此不访问对方的键和值时

Furthermore, because `sync.Map` was added to the standard library before the introduction of generics, `sync.Map` uses `any` as the type for its keys and values. This means the compiler cannot help you ensure that the right data types are used.

此外，由于 `sync.Map` 是在泛型引入之前添加到标准库的，因此 `sync.Map` 使用了 `any` 作为其键和值的类型。这意味着编译器无法帮助您确保使用正确的数据类型。

Given these limitations, in the rare situations where you need to share a map across multiple goroutines, use a built-in `map` protected by a `sync.RWMutex`.

鉴于这些限制，在极少数情况下，如果您需要在多个 goroutine 之间共享一个映射，请使用内置的 `map`，并由 `sync.RWMutex` 保护。

Atomics—You Probably Don’t Need These

原子操作——你可能不需要这些

In addition to mutexes, Go provides another way to keep data consistent across multiple threads. The `sync/atomic` package provides access to the *atomic variable* operations built into modern CPUs to add, swap, load, store, or compare and swap (CAS) a value that fits into a single register.

除了互斥锁之外，Go 还提供了另一种在多个线程间保持数据一致性的方法。`sync/atomic` 包提供了对现代 CPU 内置的原子变量操作的访问，可以对单个寄存器中的值进行加、交换、加载、存储或比较并交换 (CAS) 等操作。

If you need to squeeze out every last bit of performance and are an expert on writing concurrent code, you’ll be glad that Go includes atomic support.

For everyone else, use goroutines and mutexes to manage your concurrency needs.

如果你需要榨干每一分性能，并且是编写并发代码的专家，你会很高兴 Go 语言提供了原子操作支持。对于其他人来说，使用 goroutine 和互斥锁来管理并发需求就足够了。

Where to Learn More About Concurrency

哪里可以了解更多关于并发性的知识

I've covered a few simple concurrency patterns here, but there are many more. In fact, you could write an entire book on how to properly implement various concurrency patterns in Go, and, luckily, Katherine Cox-Buday has. I've already mentioned *Concurrency in Go*, when discussing how to decide between mutexes or channels, but it's an excellent resource on all things involving Go and concurrency. Check out her book if you want to learn more.

我在这里介绍了一些简单的并发模式，但实际上还有很多。事实上，关于如何在 Go 中正确实现各种并发模式，完全可以写一本书，幸运的是，Katherine Cox-Buday 已经写过一本。我在讨论如何选择互斥锁

还是通道时已经提到过《Go 中的并发》，但这本书对于所有与 Go 和并发相关的内容来说都是一本极好的参考资料。如果你想了解更多，可以看看她的书。

Exercises

练习

Using concurrency effectively is one of the most important skills for a Go developer. Work through these exercises to see if you have mastered them. The solutions are available in the *exercise_solutions* directory in the [Chapter 12 repository](#).

有效利用并发是 Go 开发者最重要的技能之一。完成以下练习，看看你是否已经掌握了这些技能。答案位于第 12 章代码库的 `exercise_solutions` 目录中。

1. Create a function that launches three goroutines that communicate using a channel. The first two goroutines each write 10 numbers to the channel. The third goroutine reads all the numbers from the channel and prints them out. The function should exit when all values have been printed out. Make sure that none of the goroutines leak. You can create additional goroutines if needed.

创建一个函数，该函数启动三个 goroutine，这些 goroutine 通过通道进行通信。前两个 goroutine 各自向通道写入 10 个数字。第三个 goroutine 从通道读取所有数字并打印出来。当所有值都打印完毕后，函数应退出。确保所有 goroutine 都不会发生内存泄漏。如有需要，您可以创建更多 goroutine。

2. Create a function that launches two goroutines. Each goroutine writes 10 numbers to its own channel. Use a `for-select` loop to read from both channels, printing out the number and the goroutine that wrote the value. Make sure that your function exits after all values are read and that none of your goroutines leak.

创建一个函数，该函数启动两个 goroutine。每个 goroutine 向其自身的通道写入 10 个数字。使用 `for-select` 循环从两个通道读取数据，并打印出数字及其对应的 goroutine。确保函数在读取完所有值后退出，并且所有 goroutine 都不会发生内存泄漏。

3. Write a function that builds a `map[int]float64` where the keys are the numbers from 0 (inclusive) to 100,000 (exclusive) and the values are the square roots of those numbers (use the `math.Sqrt` function to calculate square roots). Use `sync.OnceValue` to generate a function that caches the `map` returned by this function and use the cached value to look up square roots for every 1,000th number from 0 to 100,000.

编写一个函数，生成一个 `map[int]float64` 数组，其中键是 0（含 0）到 100,000（不含 100,000）之间的数字，值是这些数字的平方根（使用 `math.Sqrt` 函数计算平方根）。使用

`sync.OnceValue` 生成一个函数，该函数缓存此函数返回的 `map` 值，并使用缓存的值查找 0 到 100,000 之间每隔 1,000 个数字的平方根。

Wrapping Up

总结

In this chapter, you've looked at concurrency and learned why Go's approach is simpler than more traditional concurrency mechanisms. In doing so, you've also learned when you should use concurrency as well as a few concurrency rules and patterns. In the next chapter, you're going to take a quick look at Go's standard library, which embraces a "batteries included" ethos for modern computing.

在本章中，您了解了并发，并明白了为什么 Go 的并发机制比传统的并发机制更简洁。同时，您也学习了何时应该使用并发，以及一些并发规则和模式。下一章，您将快速浏览 Go 的标准库，它秉承了现代计算“开箱即用”的理念。

Chapter 13. The Standard Library

第十三章 标准图书馆

One of the best parts of developing with Go is being able to take advantage of its standard library. Like Python, it has a “batteries included” philosophy, providing many of the tools that you need to build an application. Since Go is a relatively new language, it ships with a library that is focused on problems faced in modern programming environments.

使用 Go 进行开发的最大优势之一在于能够充分利用其标准库。与 Python 类似，Go 秉持“开箱即用”的理念，提供了构建应用程序所需的大部分工具。由于 Go 是一种相对较新的语言，它自带的库专注于解决现代编程环境中遇到的问题。

I can’t cover all the standard library packages, and luckily, I don’t have to, as there are many excellent sources of information on the standard library, starting with the [documentation](#). Instead, I’ll focus on several of the most important packages and how their design and use demonstrate the principles of idiomatic Go. Some packages (`errors` , `sync` , `context` , `testing` , `reflect` , and `unsafe`) are covered in their own chapters. In this chapter, you’ll look at Go’s built-in support for I/O, time, JSON, and HTTP.

我无法涵盖所有标准库包，幸运的是，我也不需要这样做，因为有很多优秀的资源可以获取标准库的相关信息，首先就是官方文档。因此，我将重点介绍几个最重要的包，以及它们的设计和使用如何体现 Go 语言的惯用原则。部分包（`errors`、`sync`、`context`、`testing`、`reflect` 和 `unsafe`）将在各自的章节中进行介绍。在本章中，您将了解 Go 对 I/O、时间、JSON 和 HTTP 的内置支持。

io and Friends

io 和朋友们

For a program to be useful, it needs to read in and write out data. The heart of Go's input/output philosophy can be found in the `io` package. In particular, two interfaces defined in this package are probably the second and third most-used interfaces in Go: `io.Reader` and `io.Writer`.

一个程序要想发挥作用，就必须能够读取和写入数据。Go 语言输入/输出理念的核心在于 `io` 包。特别是，该包中定义的两个接口 `io.Reader` 和 `io.Writer` 可能是 Go 语言中使用频率第二和第三高的接口。

NOTE

笔记

What's number one? That'd be `error`, which you already looked at in [Chapter 9](#).

第一名是什么？那就是 `error`，你已经在第 9 章中看过它了。

Both `io.Reader` and `io.Writer` define a single method:

`io.Reader` 和 `io.Writer` 都定义了同一个方法：

```
type Reader interface {  
    Read(p []byte) (n int, err error)  
}  
  
type Writer interface {  
    Write(p []byte) (n int, err error)  
}
```

The `Write` method on the `io.Writer` interface takes in a slice of bytes, which are written to the interface implementation. It returns the number of bytes written and an error if something went wrong. The `Read` method on `io.Reader` is more interesting. Rather than return data through a return parameter, a slice is passed into the implementation and modified. Up to `len(p)` bytes will be written into the slice. The method returns the

number of bytes written. This might seem a little strange. You might expect this:

`io.Writer` 接口上的 `Write` 方法接收一个字节切片，并将这些字节写入接口实现。它返回写入的字节数，如果出现错误，则返回错误信息。`io.Reader` 接口上的 `Read` 方法更有意思。它不是通过返回参数返回数据，而是将一个字节切片传递给实现并进行修改。最多会向该切片写入 `len(p)` 个字节。该方法返回写入的字节数。这可能看起来有点奇怪。你可能会期望这样：

```
type NotHowReaderIsDefined interface {  
    Read() (p []byte, err error)  
}
```

There's a very good reason `io.Reader` is defined the way it is. Let's write a function that's representative of how to work with an `io.Reader` to illustrate:

`io.Reader` 的定义方式是有充分理由的。让我们编写一个函数来演示如何使用 `io.Reader`：

```
func countLetters(r io.Reader) (map[string]int,  
error) {  
    buf := make([]byte, 2048)  
    out := map[string]int{}  
    for {
```

```

        n, err := r.Read(buf)
        for _, b := range buf[:n] {
            if (b >= 'A' && b <= 'Z') || (b >=
'a' && b <= 'z') {
                out[string(b)]++
            }
        }
        if err == io.EOF {
            return out, nil
        }
        if err != nil {
            return nil, err
        }
    }
}

```

There are three things to note. First, you create your buffer once and reuse it on every call to `r.Read`. This allows you to use a single memory allocation to read from a potentially large data source. If the `Read` method were written to return a `[]byte`, it would require a new allocation on every single call. Each allocation would end up on the heap, which would make quite a lot of work for the garbage collector.

有三点需要注意。首先，您只需创建一次缓冲区，即可在每次调用 `r.Read` 时重复使用它。这样，您只需一次内存分配即可从可能很大的数据源读取数据。如果 `Read` 方法返回 `[]byte`，则每次调用都需

要重新分配内存。每次分配的内存都会被加载到堆上，这将给垃圾回收器带来相当大的工作量。

If you want to reduce the allocations further, you could create a pool of buffers when the program launches. You would then take a buffer out of the pool when the function starts, and return it when it ends. By passing in a slice to `io.Reader`, memory allocation is under the control of the developer.

如果想进一步减少内存分配，可以在程序启动时创建一个缓冲区池。函数开始时从池中取出一个缓冲区，函数结束时将其返回。通过向 `io.Reader` 传递一个切片，内存分配就由开发者控制了。

Second, you use the `n` value returned from `r.Read` to know how many bytes were written to the buffer and iterate over a subslice of your `buf` slice, processing the data that was read.

其次，您可以使用从 `r.Read` 返回的 `n` 值来了解写入缓冲区的字节数，并遍历 `buf` 切片的子切片，处理读取的数据。

Finally, you know that you're done reading from `r` when the error returned from `r.Read` is `io.EOF`. This error is a bit odd, in that it isn't really an error. It indicates that there's nothing left to read from the `io.Reader`. When `io.EOF` is returned, you are finished processing and return your result.

最后，当从 `r.Read` 返回的错误是 `io.EOF` 时，就表示已完成从 `r` 的读取。这个错误有点奇怪，因为它实际上并不是一个真正的错误。它表示从 `io.Reader` 中已经没有剩余数据可以读取了。当返回 `io.EOF` 时，处理就完成了，可以返回结果了。

The `Read` method in `io.Reader` has one unusual aspect. In most cases when a function or method has an error return value, you check the error before you try to process the nonerror return values. You do the opposite for `Read` because bytes might have been copied into the buffer before an error was triggered by the end of the data stream or by an unexpected condition.

`io.Reader` 中的 `Read` 方法有一个不寻常之处。通常情况下，当函数或方法返回错误值时，你会先检查错误，然后再处理非错误返回值。但对于 `Read`，情况恰恰相反，因为在数据流结束或出现意外情况触发错误之前，可能已经有一些字节被复制到缓冲区中。

TIP

提示

If you get to the end of an `io.Reader` unexpectedly, a different sentinel error is returned (`io.ErrUnexpectedEOF`). Note that it starts with the string `Err` to indicate that it is an unexpected state.

如果意外到达 `io.Reader` 的末尾，则会返回不同的哨兵错误（`io.ErrUnexpectedEOF`）。请注意，它以字符串 `Err` 开头，以表明这是一个意外状态。

Because `io.Reader` and `io.Writer` are such simple interfaces, they can be implemented many ways. You can create an `io.Reader` from a string by using the `strings.NewReader` function:

由于 `io.Reader` 和 `io.Writer` 是非常简单的接口，因此可以用多种方式实现。您可以使用 `strings.NewReader` 函数从字符串创建 `io.Reader`：

```
s := "The quick brown fox jumped over the lazy dog"
sr := strings.NewReader(s)
counts, err := countLetters(sr)
if err != nil {
    return err
}
fmt.Println(counts)
```

As I discussed in [“Interfaces Are Type-Safe Duck Typing”](#), implementations of `io.Reader` and `io.Writer` are often chained together in a decorator pattern. Because `countLetters` depends on an `io.Reader`, you can use the exact same `countLetters` function to count English letters in a gzip-compressed file. First you write a function that, when given a filename, returns a `*gzip.Reader` :

正如我在《接口是类型安全的鸭子类型》一文中讨论的那样，`io.Reader` 和 `io.Writer` 的实现通常会以装饰器模式链接在一

起。由于 `countLetters` 依赖于 `io.Reader`，因此您可以使用完全相同的 `countLetters` 函数来统计 `gzip` 压缩文件中的英文字母数量。首先，您需要编写一个函数，该函数在给定文件名时返回一个 `*gzip.Reader`：

```
func buildGZipReader(fileName string)
(*gzip.Reader, func(), error) {
    r, err := os.Open(fileName)
    if err != nil {
        return nil, nil, err
    }
    gr, err := gzip.NewReader(r)
    if err != nil {
        return nil, nil, err
    }
    return gr, func() {
        gr.Close()
        r.Close()
    }, nil
}
```

This function demonstrates the way to properly wrap types that implement `io.Reader`. You create an `*os.File` (which meets the `io.Reader` interface), and after making sure it's valid, you pass it to the `gzip.NewReader` function, which returns a `*gzip.Reader` instance. If

it is valid, you return the `*gzip.Reader` and a closure that properly cleans up your resources when it is invoked.

此函数演示了如何正确封装实现了 `io.Reader` 接口的类型。首先, 创建一个 `*os.File` 对象 (满足 `io.Reader` 接口), 确认其有效后, 将其传递给 `gzip.NewReader` 函数, 该函数返回一个 `*gzip.Reader` 实例。如果该实例有效, 则返回 `*gzip.Reader` 对象以及一个闭包, 该闭包会在调用时正确清理资源。

Since `*gzip.Reader` implements `io.Reader`, you can use it with `countLetters` just as you used the `*strings.Reader` previously:

由于 `*gzip.Reader` 实现了 `io.Reader`, 因此您可以像之前使用 `*strings.Reader` 一样, 将其与 `countLetters` 一起使用:

```
r, closer, err :=
buildGZipReader("my_data.txt.gz")
if err != nil {
    return err
}
defer closer()
counts, err := countLetters(r)
if err != nil {
    return err
}
fmt.Println(counts)
```

You can find the code for `countLetters` and `buildGZipReader` in the *sample_code/io_friends* directory in the [Chapter 13 repository](#).

您可以在第 13 章存储库的 *sample_code/io_friends* 目录中找到 `countLetters` 和 `buildGZipReader` 的代码。

Because there are standard interfaces for reading and writing, there's a standard function in the `io` package for copying from an `io.Reader` to an `io.Writer`, `io.Copy`. There are other standard functions for adding new functionality to existing `io.Reader` and `io.Writer` instances. These include the following:

由于读写操作都有标准接口，因此 `io` 包中有一个标准函数，用于将 `io.Reader` 复制到 `io.Writer` 或 `io.Copy`。此外，还有其他标准函数，用于向现有的 `io.Reader` 和 `io.Writer` 实例添加新功能。这些函数包括：

`io.MultiReader`

Returns an `io.Reader` that reads from multiple `io.Reader` instances, one after another

返回一个 `io.Reader`，该实例会依次从多个 `io.Reader` 实例读取数据。

`io.LimitReader`

Returns an `io.Reader` that reads only up to a specified number of bytes from the supplied `io.Reader`

返回一个 `io.Reader`，该对象仅从提供的 `io.Reader` 中读取指定数量的字节。

`io.MultiWriter`

Returns an `io.Writer` that writes to multiple `io.Writer` instances at the same time

返回一个 `io.Writer`，该对象同时写入多个 `io.Writer` 个实例。

Other packages in the standard library provide their own types and functions to work with `io.Reader` and `io.Writer`. You've seen a few of them already, but there are many more. These cover compression algorithms, archives, cryptography, buffers, byte slices, and strings.

标准库中的其他包提供了各自的类型和函数来处理 `io.Reader` 和 `io.Writer`。您可能已经见过其中一些，但还有更多。这些类型和函数涵盖了压缩算法、归档、密码学、缓冲区、字节切片和字符串等领域。

Other one-method interfaces are defined in `io`, such as `io.Closer` and `io.Seeker`:

在 `io` 中定义其他单方法接口，例如 `io.Closer` 和 `io.Seeker`：

```
type Closer interface {
    Close() error
}

type Seeker interface {
    Seek(offset int64, whence int) (int64,
    error)
}
```

The `io.Closer` interface is implemented by types like `os.File` that need to do cleanup when reading or writing is complete. Usually, `Close` is called via a `defer`：

`io.Closer` 接口由 `os.File` 等类型实现，这些类型需要在读取或写入完成后进行清理。通常，`Close` 通过 `defer` 调用：

```
f, err := os.Open(fileName)
if err != nil {
    return nil, err
}
defer f.Close()
// use f
```

WARNING

警告

If you are opening the resource in a loop, do not use `defer`, as it will not run until the function exits. Instead, you should call `Close` before the end of the loop iteration. If there are errors that can lead to an exit, you must call `Close` there too.

如果在循环中打开资源，请勿使用 `defer`，因为它要等到函数退出才会执行。您应该在循环迭代结束前调用 `Close`。如果出现可能导致函数退出的错误，您还必须调用 `Close`。

The `io.Seeker` interface is used for random access to a resource. The valid values for `whence` are the constants `io.SeekStart`, `io.SeekCurrent`, and `io.SeekEnd`. This should have been made clearer by using a custom type, but in a surprising design oversight, `whence` is of type `int`.

`io.Seeker` 接口用于对资源进行随机访问。`whence` 的有效值为常量 `io.SeekStart`、`io.SeekCurrent` 和 `io.SeekEnd`。使用自定义类型本应更清晰地表达这一点，但令人惊讶的是，设计上却存在一个疏忽，即 `whence` 的类型为 `int`。

The `io` package defines interfaces that combine these four interfaces in various ways. They include `io.ReadCloser`, `io.ReadSeeker`, `io.ReadWriteCloser`, `io.ReadWriteSeeker`, `io.ReadWriter`, `io.WriteCloser`, and `io.WriterSeeker`. Use these interfaces to specify what your functions expect to do with the data. For example, rather than just using an `os.File` as a parameter, use the interfaces to specify

exactly what your function will do with the parameter. Not only does it make your functions more general-purpose, it also makes your intent clearer. Furthermore, make your code compatible with these interfaces if you are writing your own data sources and sinks. In general, strive to create interfaces as simple and decoupled as the interfaces defined in `io`. They demonstrate the power of simple abstractions.

`io` 包定义了以各种方式组合这四个接口的接口。它们包括 `io.ReadCloser`、`io.ReadSeeker`、`io.ReadWriteCloser`、`io.ReadWriteSeeker`、`io.ReadWriter`、`io.WriteCloser` 和 `io.Writer`。使用这些接口来指定函数对数据的预期操作。例如，与其仅仅使用 `os.File` 作为参数，不如使用这些接口来精确地指定函数将如何处理该参数。这不仅能使函数更具通用性，还能使你的意图更加清晰。此外，如果你要编写自己的数据源和数据接收器，请确保你的代码与这些接口兼容。总的来说，尽量创建像 `io` 中定义的接口一样简单且解耦的接口。它们展现了简单抽象的强大之处。

In addition to the interfaces in the `io` package, there are several helper functions for common operations. For example, the `io.ReadAll` function reads all the data from an `io.Reader` into a byte slice. One of the more clever functions in `io` demonstrates a pattern for adding a method to a Go type. If you have a type that implements `io.Reader` but not `io.Closer` (such as `strings.Reader`) and need to pass it to a function that expects an `io.ReadCloser`, pass your `io.Reader` into

`io.NopCloser` and get back a type that implements `io.ReadCloser`.

If you look at the implementation, it's very simple:

除了 `io` 包中的接口之外，还有一些用于常见操作的辅助函数。例如，`io.ReadAll` 函数会将 `io.Reader` 中的所有数据读取到一个字节切片中。`io` 中一个比较巧妙的函数演示了如何向 Go 类型添加方法。如果您有一个实现了 `io.Reader` 但未实现 `io.Closer` 的类型（例如 `strings.Reader`），并且需要将其传递给一个期望 `io.ReadCloser` 的函数，则将您的 `io.Reader` 传递给 `io.NopCloser`，即可获得一个实现了 `io.ReadCloser` 的类型。如果您查看其实现，会发现它非常简单：

```
type nopCloser struct {  
    io.Reader  
}  
  
func (nopCloser) Close() error { return nil }  
  
func NopCloser(r io.Reader) io.ReadCloser {  
    return nopCloser{r}  
}
```

Anytime you need to add additional methods to a type so that it can meet an interface, use this embedded type pattern.

每当需要向类型添加额外方法以满足接口要求时，请使用这种嵌入式类型模式。

NOTE

笔记

The `io.NopCloser` function violates the general rule of not returning an interface from a function, but it's a simple adapter for an interface that is guaranteed to stay the same because it is part of the standard library.

`io.NopCloser` 函数违反了函数不返回接口的一般规则，但它是一个简单的接口适配器，保证接口保持不变，因为它是标准库的一部分。

Among other things, the `os` package contains functions for interacting with files. The functions `os.ReadFile` and `os.WriteFile` read an entire file into a slice of bytes and write a slice of bytes into a file, respectively. These functions (and `io.ReadAll`) are fine for small amounts of data, but they are not appropriate for large data sources. When working with larger data sources, use the `Create`, `NewFile`, `Open`, and `OpenFile` functions in the `os` package. They return an `*os.File` instance, which implements the `io.Reader` and `io.Writer` interfaces. You can use an `*os.File` instance with the `Scanner` type in the `bufio` package.

`os` 包包含用于与文件交互的函数。函数 `os.ReadFile` 和 `os.WriteFile` 分别将整个文件读取到一个字节切片中，并将该字节

切片写入文件。这些函数（以及 `io.ReadAll`）适用于少量数据，但不适用于大型数据源。处理大型数据源时，请使用 `os` 包中的 `Create`、`NewFile`、`Open` 和 `OpenFile` 函数。它们返回一个 `*os.File` 实例，该实例实现了 `io.Reader` 和 `io.Writer` 接口。您可以使用 `*os.File` 实例以及 `Scanner` 类型，该类型位于 `bufio` 包中。

time

时间

Like most languages, Go's standard library includes time support, which is found, unsurprisingly, in the `time` package. Two main types are used to represent time: `time.Duration` and `time.Time`.

与大多数语言一样，Go 的标准库也包含时间支持，不出所料，它位于 `time` 包中。时间主要使用两种类型：`time.Duration` 和 `time.Time`。

A period of time is represented with a `time.Duration`, a type based on an `int64`. The smallest amount of time that Go can represent is one nanosecond, but the `time` package defines constants of type `time.Duration` to represent a nanosecond, microsecond, millisecond,

second, minute, and hour. For example, you represent a duration of 2 hours and 30 minutes as follows:

时间段用 `time.Duration` 表示, 该类型基于 `int64`。Go 语言能够表示的最小时间单位是纳秒, 但 `time` 包定义了 `time.Duration` 类型的常量, 分别用于表示纳秒、微秒、毫秒、秒、分钟和小时。例如, 2 小时 30 分钟的持续时间可以表示如下:

```
d := 2 * time.Hour + 30 * time.Minute // d is of
type time.Duration
```

These constants make the use of a `time.Duration` both readable and type-safe. They demonstrate a good use of a typed constant.

这些常量使得使用 `time.Duration` 既可读又类型安全。它们展示了类型化常量的良好用法。

Go defines a sensible string format, a series of numbers, that can be parsed into a `time.Duration` with the `time.ParseDuration` function. This format is described in the standard library documentation:

Go 定义了一种合理的字符串格式, 即一系列数字, 可以使用 `time.ParseDuration` 函数将其解析为 `time.Duration`。此格式在标准库文档中有详细描述:

A duration string is a possibly signed sequence of decimal numbers, each with optional fraction and a unit suffix, such as “300ms”, “-1.5h” or “2h45m”. Valid time units are “ns”, “us” (or “ μ s”), “ms”, “s”, “m”, “h”.

—[Go Standard Library Documentation](#)

持续时间字符串是一个可能带符号的十进制数序列，每个数字可以带有可选的小数部分和单位后缀，例如“300ms”、“-1.5h”或“2h45m”。有效的时间单位包括“ns”、“us”（或“ μ s”）、“ms”、“s”、“m”和“h”。Go 标准库文档

Several methods are defined on `time.Duration`. It meets the `fmt.Stringer` interface and returns a formatted duration string via the `String` method. It also has methods to get the value as a number of hours, minutes, seconds, milliseconds, microseconds, or nanoseconds. The `Truncate` and `Round` methods truncate or round a `time.Duration` to the units of the specified `time.Duration`.

`time.Duration` 上定义了多个方法。它符合 `fmt.Stringer` 接口，并通过 `String` 方法返回格式化的持续时间字符串。它还提供了以小时、分钟、秒、毫秒、微秒或纳秒为单位获取值的方法。`Truncate` 和 `Round` 方法将 `time.Duration` 截断或四舍五入到指定的 `time.Duration` 单位。

A moment of time is represented with the `time.Time` type, complete with a time zone. You acquire a reference to the current time with the function `time.Now`. This returns a `time.Time` instance set to the current local time.

时间点用 `time.Time` 类型表示，并包含时区信息。您可以使用函数 `time.Now` 获取当前时间的引用。该函数返回一个 `time.Time` 实例，其值为当前本地时间。

TIP

提示

The fact that a `time.Time` instance contains a time zone means that you should not use `==` to check whether two `time.Time` instances refer to the same moment in time. Instead, use the `Equal` method, which corrects for time zone.

由于 `time.Time` 实例包含时区信息，因此不应使用 `==` 来检查两个 `time.Time` 实例是否指向同一时间点。而应使用 `Equal` 方法，该方法会自动校正时区。

The `time.Parse` function converts from a `string` to a `time.Time`, while the `Format` method converts a `time.Time` to a `string`. While Go usually adopts ideas that worked well in the past, it uses [its own date and time formatting language](#). It relies on the idea of formatting the date and time January 2, 2006 at 3:04:05PM MST (Mountain Standard Time) to specify your format.

`time.Parse` 函数将 `string` 转换为 `time.Time`，而 `Format` 方法将 `time.Time` 转换为 `string`。虽然 Go 通常会采用过去行之有效的方法，但它使用自己的日期和时间格式化语言。它依赖于将日期和时间格式化为“2006 年 1 月 2 日下午 3:04:05 MST（山区标准时间）”的思路来指定格式。

NOTE

笔记

Why that date? Because each part of it represents one of the numbers from 1 to 7 in sequence, that is, 01/02 03:04:05PM '06 -0700 (MST is 7 hours before UTC).

为什么是这个日期？因为它的每一部分都代表了 1 到 7 之间的一个数字，即 01/02 03:04:05PM '06 -0700（MST 比 UTC 早 7 小时）。

For example, the following code

例如，以下代码

```
t, err := time.Parse("2006-01-02 15:04:05 -0700",
    "2023-03-13 00:00:00 +0000")
if err != nil {
    return err
}
fmt.Println(t.Format("January 2, 2006 at 3:04:05PM
MST"))
```

prints out this output:

输出如下内容：

```
March 13, 2023 at 12:00:00AM UTC
```

While the date and time used for formatting is intended to be a clever mnemonic, I find it hard to remember and have to look it up each time I want to use it. Luckily, the most commonly used date and time formats have been given their own constants in the `time` package.

虽然用于格式化的日期和时间格式旨在方便记忆，但我发现很难记住，每次使用时都得查阅。幸运的是，`time` 包中已经为最常用的日期和时间格式提供了各自的常量。

Just as there are methods on `time.Duration` to extract portions of it, there are methods defined on `time.Time` to do the same, including `Day`, `Month`, `Year`, `Hour`, `Minute`, `Second`, `Weekday`, `Clock` (which returns the time portion of a `time.Time` as separate hour, minute, and second `int` values), and `Date` (which returns the year, month, and day as separate `int` values). You can compare one `time.Time` instance against another with the `After`, `Before`, and `Equal` methods.

就像 `time.Duration` 提供了提取其部分信息的方法一样，`time.Time` 也定义了执行相同操作的方法，包括 `Day`、`Month`、

`Year`、`Hour`、`Minute`、`Second`、`Weekday`、`Clock`（该方法将 `time.Time` 的时间部分分别以小时、分钟和秒 `int` 值返回）以及 `Date`（该方法将年、月和日分别以 `int` 值返回）。您可以使用 `After`、`Before` 和 `Equal` 方法将一个 `time.Time` 实例与另一个实例进行比较。

The `Sub` method returns a `time.Duration` that represents the elapsed time between two `time.Time` instances, while the `Add` method returns a `time.Time` that is `time.Duration` later, and the `AddDate` method returns a new `time.Time` instance that's incremented by the specified number of years, months, and days. As with `time.Duration`, there are `Truncate` and `Round` methods defined as well. All these methods are defined on a value receiver, so they do not modify the `time.Time` instance.

`Sub` 方法返回一个 `time.Duration`，表示两个 `time.Time` 实例之间的时间间隔；`Add` 方法返回一个 `time.Time`，表示时间上的差值，`time.Duration`；而 `AddDate` 方法返回一个新的 `time.Time` 实例，其索引值按指定的年、月、日递增。与 `time.Duration` 类似，也定义了 `Truncate` 和 `Round` 方法。所有这些方法都定义在值接收器上，因此它们不会修改 `time.Time` 实例。

Monotonic Time

单调时间

Most operating systems keep track of two sorts of time: the *wall clock*, which corresponds to the current time, and the *monotonic clock*, which counts up from the time the computer was booted. The reason for tracking two clocks is that the wall clock doesn't uniformly increase. Daylight Saving Time, leap seconds, and Network Time Protocol (NTP) updates can make the wall clock move unexpectedly forward or backward. This can cause problems when setting a timer or finding the amount of time that's elapsed.

大多数操作系统会跟踪两种时间：一种是墙钟，对应当前时间；另一种是自增时钟，从计算机启动时间开始计数。之所以要跟踪两种时钟，是因为墙钟的计时并非均匀递增。夏令时、闰秒以及网络时间协议 (NTP) 的更新都可能导致墙钟意外地向前或向后移动。这会在设置计时器或计算经过的时间时造成问题。

To address this potential problem, Go uses monotonic time to track elapsed time whenever a timer is set or a `time.Time` instance is created with `time.Now`. This support is invisible; timers use it automatically. The `Sub` method uses the monotonic clock to calculate the `time.Duration` if both `time.Time` instances have it set. If they don't (because one or both of the

instances was not created with `time.Now`), the `Sub` method uses the time specified in the instances to calculate the `time.Duration` instead.

为了解决这个潜在问题，Go 使用单调时间来跟踪计时器设置或使用 `time.Now` 创建 `time.Time` 实例时经过的时间。这种支持是透明的；计时器会自动使用它。如果两个 `time.Time` 实例都设置了单调时间，则 `Sub` 方法会使用单调时间来计算 `time.Duration`。如果两个实例都没有设置单调时间（因为其中一个或两个实例不是使用 `time.Now` 创建），则 `Sub` 方法会使用实例中指定的时间来计算 `time.Duration`。

NOTE

笔记

If you want to understand the sorts of problems that can occur when not handling monotonic time correctly, take a look at the Cloudflare [blog post](#) that detailed a bug caused by the lack of monotonic time support in an earlier version of Go.

如果您想了解在处理单调时间时可能会出现哪些问题，请查看 Cloudflare 的博客文章，该文章详细介绍了早期版本的 Go 中由于缺乏单调时间支持而导致的错误。

Timers and Timeouts

定时器和超时

As I covered in [“Time Out Code”](#), the `time` package includes functions that return channels that output values after a specified time. The

`time.After` function returns a channel that outputs once, while the channel returned by `time.Tick` returns a new value every time the specified `time.Duration` elapses. These are used with Go's concurrency support to enable timeouts or recurring tasks.

正如我在“超时代码”一文中提到的，`time` 包包含一些函数，这些函数返回的通道会在指定时间后输出值。`time.After` 函数返回一个只输出一次的通道，而 `time.Tick` 函数返回的通道则会在每次指定的 `time.Duration` 时间过去后返回一个新值。这些函数与 Go 的并发支持结合使用，可以实现超时或重复任务。

You can also trigger a single function to run after a specified `time.Duration` with the `time.AfterFunc` function. Don't use `time.Tick` outside trivial programs, because the underlying `time.Ticker` cannot be shut down (and therefore cannot be garbage collected). Use the `time.NewTicker` function instead, which returns a `*time.Ticker` that has the channel to listen to, as well as methods to reset and stop the ticker.

您还可以使用 `time.AfterFunc` 函数在指定的 `time.Duration` 之后触发单个函数运行。请勿在简单的程序之外使用 `time.Tick`，因为底层 `time.Ticker` 无法关闭（因此也无法被垃圾回收）。请改用 `time.NewTicker` 函数，该函数返回一个 `*time.Ticker` 对象，其中包含要监听的通道以及重置和停止计时器的方法。

encoding/json

encoding/json

REST APIs have enshrined JSON as the standard way to communicate between services, and Go's standard library includes support for converting Go data types to and from JSON. The word *marshaling* means converting from a Go data type to an encoding, and *unmarshaling* means converting to a Go data type.

REST API 已将 JSON 确立为服务间通信的标准格式，而 Go 的标准库也支持 Go 数据类型与 JSON 之间的相互转换。序列化（marshaling）是指将 Go 数据类型转换为某种编码，而反序列化（unmarshaling）是指将编码转换回 Go 数据类型。

Using Struct Tags to Add Metadata

使用结构化标签添加元数据

Let's say that you are building an order management system and have to read and write the following JSON:

假设您正在构建一个订单管理系统，并且需要读取和写入以下 JSON：

```
{
  "id": "12345",
  "date_ordered": "2020-05-01T13:01:02Z",
  "customer_id": "3",
  "items": [{ "id": "xyz123", "name": "Thing 1" },
    { "id": "abc789", "name": "Thing 2" } ]
}
```

You define types to map this data:

您可以定义类型来映射这些数据：

```
type Order struct {
    ID          string          `json:"id"`
    DateOrdered time.Time
    `json:"date_ordered"`
    CustomerID  string
    `json:"customer_id"`
    Items       []Item          `json:"items"`
}

type Item struct {
    ID   string `json:"id"`
    Name string `json:"name"`
}
```

You specify the rules for processing your JSON with *struct tags*, strings that are written after the fields in a struct. Even though struct tags are strings marked with backticks, they cannot extend past a single line. Struct tags are composed of one or more tag/value pairs, written as

`tagName:"tagValue"` and separated by spaces. Because they are just strings, the compiler cannot validate that they are formatted correctly, but `go vet` does. Also, note that all these fields are exported. Like any other package, the code in the `encoding/json` package cannot access an unexported field on a struct in another package.

您可以使用结构体标签（struct tag）来指定 JSON 的处理规则。结构体标签是写在结构体字段后面的字符串。尽管结构体标签是用反引号（```）标记的字符串，但它们不能超过一行。结构体标签由一个或多个标签/值对组成，格式为 `tagName:"tagValue"`，并用空格分隔。由于它们只是字符串，编译器无法验证其格式是否正确，但 `go vet` 可以。此外，请注意，所有这些字段都是导出的。与其他任何包一样，`encoding/json` 包中的代码无法访问另一个包中结构体上未导出的字段。

For JSON processing, use the tag `json` to specify the name of the JSON field that should be associated with the struct field. If no `json` tag is provided, the default behavior is to assume that the name of the JSON object field matches the name of the Go struct field. Despite this default behavior, it's best to use the struct tag to specify the name of the field explicitly, even if the field names are identical.

对于 JSON 处理，请使用 ``json`` 标签指定应与结构体字段关联的 JSON 字段名称。如果没有提供 ``json`` 标签，则默认行为是假定 JSON 对象字段的名称与 Go 结构体字段的名称匹配。尽管存在这种默认行为，但即使字段名称相同，最好也使用 ``<struct>`` 标签显式指定字段名称。

NOTE

笔记

When unmarshaling from JSON into a struct field with no `json` tag, the name match is case-insensitive. When marshaling a struct field with no `json` tag back to JSON, the JSON field will always have an uppercase first letter, because the field is exported.

当从 JSON 反序列化为没有 `json` 标签的结构体字段时，名称匹配不区分大小写。当将没有 `json` 标签的结构体字段序列化回 JSON 时，JSON 字段的首字母始终大写，因为该字段会被导出。

If a field should be ignored when marshaling or unmarshaling, use a dash (`-`) for the name. If the field should be left out of the output when it is empty, add `,omitempty` after the name. For example, in the `Order` struct, if you didn't want to include `CustomerID` in the output if it was set to an empty string, the struct tag would be `json:"customer_id,omitempty"`.

如果在序列化或反序列化时需要忽略某个字段，请使用短横线 (`-`) 作为字段名。如果字段为空时需要从输出中排除，请在字段名后添加

,omitempty。例如，在 Order 结构体中，如果您不希望在 CustomerID 设置为空字符串时将其包含在输出中，则结构体标签应为 json:"customer_id,omitempty"。

WARNING

警告

Unfortunately, the definition of “empty” doesn’t exactly align with the zero value, as you might expect. The zero value of a struct doesn’t count as empty, but a zero-length slice or map does.

遗憾的是，“空”的定义并不完全等同于零值，这可能与你的预期相反。结构体的零值不算作空，但长度为零的切片或映射则算作空。

Struct tags allow you to use metadata to control how your program behaves. Other languages, most notably Java, encourage developers to place annotations on various program elements to describe *how* they should be processed, without explicitly specifying *what* is going to do the processing. While declarative programming allows for more concise programs, automatic processing of metadata makes it difficult to understand how a program behaves. Anyone who has worked on a large Java project with annotations has had a moment of panic when something goes wrong and they don’t understand which code is processing a particular annotation and what changes it made. Go favors explicit code over short code. Struct tags are never evaluated automatically; they are processed when a struct instance is passed into a function.

结构体标签允许你使用元数据来控制程序的行为。其他语言，尤其是 Java，鼓励开发者在各种程序元素上添加注解来描述它们的处理方式，而无需明确指定由谁来执行处理。虽然声明式编程可以编写更简洁的程序，但元数据的自动处理使得理解程序的行为变得困难。任何参与过使用注解的大型 Java 项目的人都经历过这样的时刻：当出现问题时，他们无法理解是哪段代码在处理某个特定的注解以及它做了哪些更改，从而感到恐慌。Go 语言更倾向于使用显式代码而非简洁的代码。结构体标签永远不会自动求值；它们会在结构体实例被传递给函数时才被处理。

Unmarshaling and Marshaling

反序列化和序列化

The `Unmarshal` function in the `encoding/json` package is used to convert a slice of bytes into a struct. If you have a string named `data`, this is the code to convert `data` to a struct of type `Order`:

`encoding/json` 包中的 `Unmarshal` 函数用于将字节切片转换为结构体。如果您有一个名为 `data` 的字符串，以下代码可将 `data` 转换为 `Order` 类型的结构体：

```
var o Order
err := json.Unmarshal([]byte(data), &o)
if err != nil {
```

```
    return err  
}
```

The `json.Unmarshal` function populates data into an input parameter, just like the implementations of the `io.Reader` interface. As I discussed in [“Pointers Are a Last Resort”](#), this allows for efficient reuse of the same struct over and over, giving you control over memory usage.

`json.Unmarshal` 函数会将数据填充到输入参数中，就像 `io.Reader` 接口的实现一样。正如我在“指针是最后的手段”一文中讨论的那样，这允许高效地重复使用同一个结构体，从而更好地控制内存使用。

You use the `Marshal` function in the `encoding/json` package to write an `Order` instance back as JSON, stored in a slice of bytes:

您可以使用 `encoding/json` 包中的 `Marshal` 函数，将 `Order` 实例以 JSON 格式写回，存储在字节切片中：

```
out, err := json.Marshal(o)
```

This leads to the question: how are you able to evaluate struct tags? You might also be wondering how `json.Marshal` and `json.Unmarshal` are able to read and write a struct of any type. After all, every other method that you’ve written has worked only with types that were known when the

program was compiled (even the types listed in a type switch are enumerated ahead of time). The answer to both questions is reflection. You can find out more about reflection in [Chapter 16](#).

这就引出了一个问题：如何评估结构体标签？你可能还会好奇

`json.Marshal` 和 `json.Unmarshal` 如何能够读写任何类型的结构体。毕竟，你编写的其他所有方法都只能处理程序编译时已知的类型（即使是类型切换中列出的类型也是预先枚举的）。这两个问题的答案都是反射。你可以在第 16 章中找到更多关于反射的内容。

JSON, Readers, and Writers

JSON、读取器和写入器

The `json.Marshal` and `json.Unmarshal` functions work on slices of bytes. As you just saw, most data sources and sinks in Go implement the `io.Reader` and `io.Writer` interfaces. While you could use `io.ReadAll` to copy the entire contents of an `io.Reader` into a byte slice so it can be read by `json.Unmarshal`, this is inefficient. Similarly, you could write to an in-memory byte slice buffer using `json.Marshal` and then write that byte slice to the network or disk, but it'd be better if you could write to an `io.Writer` directly.

`json.Marshal` 和 `json.Unmarshal` 函数处理的是字节切片。正如你刚才看到的，Go 中的大多数数据源和数据接收器都实现了

`io.Reader` 和 `io.Writer` 接口。虽然你可以使用 `io.ReadAll` 将 `io.Reader` 的全部内容复制到一个字节切片中，以便 `json.Unmarshal` 可以读取它，但这效率很低。类似地，你可以使用 `json.Marshal` 写入内存中的字节切片缓冲区，然后将该字节切片写入网络或磁盘，但如果可以直接写入 `io.Writer` 会更好。

The `encoding/json` package includes two types that allow you to handle these situations. The `json.Decoder` and `json.Encoder` types read from and write to anything that meets the `io.Reader` and `io.Writer` interfaces, respectively. Let's take a quick look at how they work.

`encoding/json` 包包含两种类型，可用于处理这些情况。`json.Decoder` 和 `json.Encoder` 类型分别用于读取和写入符合 `io.Reader` 和 `io.Writer` 接口的任何对象。让我们快速了解一下它们的工作原理。

Start with your data in `toFile`, which implements a simple struct:

首先，你的数据位于 `toFile` 中，它实现了一个简单的结构体：

```
type Person struct {
    Name string `json:"name"`
    Age  int    `json:"age"`
}
toFile := Person {
```

```
    Name: "Fred",  
    Age: 40,  
}
```

The `os.File` type implements both the `io.Reader` and `io.Writer` interfaces, so it can be used to demonstrate `json.Decoder` and `json.Encoder`. First, you write `toFile` to a temp file by passing the temp file to `json.NewEncoder`, which returns a `json.Encoder` for the temp file. You then pass `toFile` to the `Encode` method:

`os.File` 类型同时实现了 `io.Reader` 和 `io.Writer` 接口, 因此可用于演示 `json.Decoder` 和 `json.Encoder`。首先, 将 `toFile` 写入临时文件, 方法是将临时文件传递给 `json.NewEncoder`, 该方法会返回一个 `json.Encoder` 类型的临时文件。然后, 将 `toFile` 传递给 `Encode` 方法:

```
tmpFile, err := os.CreateTemp(os.TempDir(),  
    "sample-")  
if err != nil {  
    panic(err)  
}  
defer os.Remove(tmpFile.Name())  
err = json.NewEncoder(tmpFile).Encode(toFile)  
if err != nil {  
    panic(err)  
}
```

```
err = tmpFile.Close()
if err != nil {
    panic(err)
}
```

Once `toFile` is written, you can read the JSON back in by passing a reference to the temp file to `json.NewDecoder` and then calling the `Decode` method on the returned `json.Decoder` with a variable of type `Person`:

一旦 `toFile` 写入完成，您可以通过将临时文件的引用传递给 `json.NewDecoder`，然后使用 `json.Decoder` 上类型为 `Person` 的变量调用 `Decode` 方法，将 JSON 读回：

```
tmpFile2, err := os.Open(tmpFile.Name())
if err != nil {
    panic(err)
}
var fromFile Person
err = json.NewDecoder(tmpFile2).Decode(&fromFile)
if err != nil {
    panic(err)
}
err = tmpFile2.Close()
if err != nil {
    panic(err)
}
```

```
}  
fmt.Printf("%+v\n", fromFile)
```

You can see a complete example on [The Go Playground](#) or in the *sample_code/json* directory in the [Chapter 13 repository](#).

您可以在 Go Playground 或第 13 章存储库的 *sample_code/json* 目录中看到完整的示例。

Encoding and Decoding JSON Streams

JSON 流的编码和解码

What do you do when you have multiple JSON structs to read or write at once? Our friends `json.Decoder` and `json.Encoder` can be used for these situations too.

当您需要同时读取或写入多个 JSON 结构体时该怎么办？我们的工具 `json.Decoder` 和 `json.Encoder` 也可用于这种情况。

Assume you have the following data:

假设您有以下数据：

```
{"name": "Fred", "age": 40}  
{"name": "Mary", "age": 21}
```



```
{"name": "Pat", "age": 30}
```

For the sake of this example, assume it's stored in a string called `streamData`, but it could be in a file or even an incoming HTTP request (you'll see how HTTP servers work in just a bit).

为了便于举例，假设它存储在名为 `streamData` 的字符串中，但它也可以存储在文件中，甚至是传入的 HTTP 请求中（稍后您就会看到 HTTP 服务器是如何工作的）。

You're going to store this data into your `t` variable, one JSON object at a time.

您将把这些数据逐个 JSON 对象存储到 `t` 变量中。

Just as before, you initialize your `json.Decoder` with the data source, but this time you use a `for` loop and run until you get an error. If the error is `io.EOF`, you have successfully read all the data. If not, there was a problem with the JSON stream. This lets you read and process the data, one JSON object at a time:

和之前一样，你需要使用数据源初始化 `json.Decoder`，但这次你需要使用 `for` 循环，直到出现错误为止。如果错误信息是 `io.EOF`，则表示你已成功读取所有数据。否则，JSON 流存在问题。这样你就可以一次读取和处理一个 JSON 对象：

```

var t struct {
    Name string `json:"name"`
    Age  int    `json:"age"`
}

dec :=
json.NewDecoder(strings.NewReader(streamData))
for {
    err := dec.Decode(&t)
    if err != nil {
        if errors.Is(err, io.EOF) {
            break
        }
        panic(err)
    }
    // process t
}

```

Writing out multiple values with the `json.Encoder` works just like using it to write out a single value. In this example, you are writing to a `bytes.Buffer`, but any type that meets the `io.Writer` interface will work:

使用 `json.Encoder` 写入多个值与写入单个值的方式相同。在本例中，您正在写入 `bytes.Buffer`，但任何符合 `io.Writer` 接口的类型都可以：

```
var b bytes.Buffer
enc := json.NewEncoder(&b)
for _, input := range allInputs {
    t := process(input)
    err = enc.Encode(t)
    if err != nil {
        panic(err)
    }
}
out := b.String()
```

You can run this example on [The Go Playground](#) or find it in the *sample_code/encode_decode* directory in the [Chapter 13 repository](#).

您可以在 Go Playground 上运行此示例，或者在 Chapter 13 存储库的 *sample_code/encode_decode* 目录中找到它。

This example has multiple JSON objects in the data stream that aren't wrapped in an array, but you can also use the `json.Decoder` to read a single object from an array without loading the entire array into memory at once. This can greatly increase performance and reduce memory usage. An example is in the [Go documentation](#).

这个例子中，数据流中包含多个未封装在数组中的 JSON 对象，但您也可以使用 `json.Decoder` 从数组中读取单个对象，而无需一次性

将整个数组加载到内存中。这可以显著提高性能并减少内存使用。Go 文档中提供了一个示例。

Custom JSON Parsing

自定义 JSON 解析

While the default functionality is often sufficient, sometimes you need to override it. While `time.Time` supports JSON fields in RFC 3339 format out of the box, you might have to deal with other time formats. You can handle this by creating a new type that implements two interfaces, `json.Marshaler` and `json.Unmarshaler`:

虽然默认功能通常足够使用，但有时您需要对其进行覆盖。

`time.Time` 本身支持 RFC 3339 格式的 JSON 字段，但您可能需要处理其他时间格式。您可以通过创建一个实现两个接口的新类型 `json.Marshaler` 和 `json.Unmarshaler` 来解决这个问题：

```
type RFC822ZTime struct {
    time.Time
}

func (rt RFC822ZTime) MarshalJSON() ([]byte,
error) {
    out := rt.Time.Format(time.RFC822Z)
    return []byte(`"` + out + `"`), nil
}
```

```

}

func (rt *RFC822ZTime) UnmarshalJSON(b []byte)
error {
    if string(b) == "null" {
        return nil
    }
    t, err := time.Parse(`"`+time.RFC822Z+`"`,
string(b))
    if err != nil {
        return err
    }
    *rt = RFC822ZTime{t}
    return nil
}

```

You embedded a `time.Time` instance into a new struct called `RFC822ZTime` so that you still have access to the other methods on `time.Time`. As was discussed in [“Pointer Receivers and Value Receivers”](#), the method that reads the time value is declared on a value receiver, while the method that modifies the time value is declared on a pointer receiver.

您将 `time.Time` 实例嵌入到名为 `RFC822ZTime` 的新结构体中，以便您仍然可以访问 `time.Time` 上的其他方法。正如“指针接收器和值

接收器”中所述，读取时间值的方法声明在值接收器上，而修改时间值的方法声明在指针接收器上。

You then change the type of your `DateOrdered` field and can work with RFC 822 formatted times instead:

然后，您可以更改 `DateOrdered` 字段的类型，并改用 RFC 822 格式的时间：

```
type Order struct {  
    ID          string      `json:"id"`  
    DateOrdered RFC822ZTime `json:"date_ordered"`  
    CustomerID  string      `json:"customer_id"`  
    Items       []Item      `json:"items"`  
}
```

You can run this code on [The Go Playground](#) or find it in the `sample_code/custom_json` directory in the [Chapter 13 repository](#).

您可以在 Go Playground 上运行此代码，或者在第 13 章存储库的 `sample_code/custom_json` 目录中找到它。

This approach has a philosophical drawback: the date format of the JSON determines the types of the fields in your data structure. This is a drawback to the `encoding/json` approach. You could have `Order` implement `json.Marshaler` and `json.Unmarshaler`, but that requires you to

write code to handle all the fields, even the ones that don't require custom support. The struct tag format does not provide a way to specify a function to parse a particular field. That leaves you with creating a custom type for the field.

这种方法存在一个逻辑上的缺陷：JSON 的日期格式决定了数据结构中字段的类型。这是 `encoding/json` 方法的一个缺点。你可以让 `Order` 实现 `json.Marshaler` 和 `json.Unmarshaler`，但这需要你编写代码来处理所有字段，即使是那些不需要自定义支持的字段。结构体标签格式没有提供指定解析特定字段的函数的方法。因此，你只能为该字段创建自定义类型。

Another option is described in a [blog post by Ukiah Smith](#). It allows you to redefine only the fields that don't match the default marshaling behavior by taking advantage of how struct embedding (which was covered in [“Use Embedding for Composition”](#)) interacts with JSON marshaling and unmarshaling. If a field on an embedded struct has the same name as the containing struct, that field is ignored when marshaling or unmarshaling JSON.

Ukiah Smith 在一篇博文中介绍了另一种方法。它利用结构体嵌入（在“使用嵌入进行组合”一文中已介绍）与 JSON 序列化和反序列化之间的交互方式，允许您仅重新定义与默认序列化行为不匹配的字段。如果嵌入结构体上的某个字段与包含该结构体的结构体同名，则在序列化或反序列化 JSON 时，该字段将被忽略。

In this example, the fields for `Order` look like this:

在这个例子中，`Order` 的字段如下所示：

```
type Order struct {  
    ID          string    `json:"id"`  
    Items       []Item    `json:"items"`  
    DateOrdered time.Time `json:"date_ordered"`  
    CustomerID  string    `json:"customer_id"`  
}
```

The `MarshalJSON` method looks like this:

`MarshalJSON` 方法如下所示：

```
func (o Order) MarshalJSON() ([]byte, error) {  
    type Dup Order  
  
    tmp := struct {  
        DateOrdered string `json:"date_ordered"`  
        Dup  
    }{  
        Dup: (Dup)(o),  
    }  
    tmp.DateOrdered =  
    o.DateOrdered.Format(time.RFC822Z)  
    b, err := json.Marshal(tmp)
```



```
    return b, err
}
```

In the `MarshalJSON` method for `Order`, you define a type `Dup` whose underlying type is `Order`. The reason for creating `Dup` is that a type based on another type has the same fields as the underlying type, but not the methods. If you didn't have `Dup`, there would be an infinite loop of calls to `MarshalJSON` when you call `json.Marshal`, eventually resulting in a stack overflow.

在 `Order` 的 `MarshalJSON` 方法中，您定义了一个类型 `Dup`，其底层类型为 `Order`。创建 `Dup` 的原因是，基于另一个类型的类型拥有与底层类型相同的字段，但方法不同。如果没有 `Dup`，当您调用 `json.Marshal` 时，将会陷入对 `MarshalJSON` 的无限循环调用，最终导致堆栈溢出。

You define an anonymous struct that has the `DateOrdered` field and an embedded `Dup`. You then assign the `Order` instance to the embedded field in `tmp`, assign the `DateOrdered` field in `tmp` the time formatted as RFC822Z, and call `json.Marshal` on `tmp`. This produces the desired JSON output.

您定义了一个匿名结构体，其中包含 `DateOrdered` 字段和一个嵌入式 `Dup` 字段。然后，您将 `Order` 实例赋值给 `tmp` 中的嵌入式字段，将 `tmp` 中的 `DateOrdered` 字段赋值给 RFC822Z 格式的时间，

并对 `tmp` 调用 `json.Marshal` 方法。这样就生成了所需的 JSON 输出。

There is similar logic in `UnmarshalJSON` :

`UnmarshalJSON` 中也存在类似的逻辑：

```
func (o *Order) UnmarshalJSON(b []byte) error {
    type Dup Order

    tmp := struct {
        DateOrdered string `json:"date_ordered"`
        *Dup
    }{
        Dup: (*Dup)(o),
    }

    err := json.Unmarshal(b, &tmp)
    if err != nil {
        return err
    }

    o.DateOrdered, err = time.Parse(time.RFC822Z,
tmp.DateOrdered)
    if err != nil {
        return err
    }
}
```

```
    return nil
}
```

In `UnmarshalJSON`, the call to `json.Unmarshal` populates the fields in `o` (except `DateOrdered`) because it's embedded into `tmp`. You then populate `DateOrdered` in `o` by using `time.Parse` to process the `DateOrdered` field in `tmp`.

在 `UnmarshalJSON` 中，对 `json.Unmarshal` 的调用会填充 `o` 中的字段（除了 `DateOrdered`），因为它嵌入在 `tmp` 中。然后，您可以使用 `time.Parse` 处理 `tmp` 中的 `DateOrdered` 字段，从而填充 `o` 中的 `DateOrdered`。

You can run this code on [The Go Playground](#) or find it in the `sample_code/custom_json2` directory in the [Chapter 13 repository](#).

您可以在 Go Playground 上运行此代码，或者在第 13 章存储库的 `sample_code/custom_json2` 目录中找到它。

While this does keep `Order` from having a field tied to the JSON format, the `MarshalJSON` and `UnmarshalJSON` methods on `Order` are coupled to the format of the time field in the JSON. You cannot reuse `Order` to support JSON that has the time formatted another way.

虽然这样可以避免 `Order` 的字段与 JSON 格式绑定，但 `Order` 上的 `MarshalJSON` 和 `UnmarshalJSON` 方法与 JSON 中时间字段的格式相关联。您无法重用 `Order` 来支持时间格式不同的 JSON。

To limit the amount of code that cares about what your JSON looks like, define two structs. Use one for converting to and from JSON and the other for data processing. Read in JSON to your JSON-aware type, and then copy it to the other. When you want to write out JSON, do the reverse. This does create some duplication, but it keeps your business logic from depending on wire protocols.

为了减少与 JSON 格式相关的代码量，可以定义两个结构体。一个用于 JSON 格式的转换，另一个用于数据处理。将 JSON 数据读入支持 JSON 格式的结构体，然后复制到另一个结构体。需要输出 JSON 数据时，反向操作即可。虽然这样做会造成一些代码重复，但可以避免业务逻辑依赖于网络协议。

You can pass a `map[string]any` to `json.Marshal` and `json.Unmarshal` to translate back and forth between JSON and Go, but save that for the exploratory phase of your coding and replace it with a concrete type when you understand what you are processing. Go uses types for a reason; they document the expected data and the types of the expected data.

你可以将 `map[string]any` 传递给 `json.Marshal` 和 `json.Unmarshal` 来进行 JSON 和 Go 之间的双向转换，但最好在编码的探索阶段使用这种方法，并在理解了要处理的数据类型之后，用具体的类型来代替它。Go 使用类型是有原因的；它们记录了预期的数据以及预期数据的类型。

While JSON is probably the most commonly used encoder in the standard library, Go ships with others, including XML and Base64. If you have a data format that you want to encode and you can't find support for it in the standard library or a third-party module, you can write one yourself. You'll learn how to implement our own encoder in [“Use Reflection to Write a Data Marshaler”](#).

虽然 JSON 可能是标准库中最常用的编码器，但 Go 还提供了其他编码器，包括 XML 和 Base64。如果您需要编码某种数据格式，但标准库或第三方模块都不支持，您可以自己编写一个编码器。您将在“使用反射编写数据编组器”一文中学习如何实现自定义编码器。

WARNING

警告

The standard library includes `encoding/gob`, which is a Go-specific binary representation that is a bit like serialization in Java. Just as Java serialization is the wire protocol for Enterprise Java Beans and Java RMI, the gob protocol is intended as the wire format for a Go-specific RPC (remote procedure call) implementation in the `net/rpc` package. Don't use either `encoding/gob` or `net/rpc`. If you want to do remote method invocation with Go, use a standard protocol like [GRPC](#) so that you aren't tied to a specific language. No matter how much you love Go, if you want your services to be useful, make them callable by developers using other languages.

标准库包含 `encoding/gob`，它是 Go 特有的二进制表示形式，有点像 Java 中的序列化。正如 Java 序列化是企业级 Java Bean 和 Java RMI 的网络协议一样，gob 协议旨在作为 `net/rpc` 包中 Go 特有的 RPC（远程过程调用）实现的网络格式。请勿使用 `encoding/gob` 或 `net/rpc`。如果您想使用 Go 进行远程方法调用，请使用像 gRPC 这样的标准协议，这样您就不会被特定语言所束缚。无论您多么喜欢 Go，如果您希望您的服务有用，请确保其他语言的开发人员可以调用它们。

net/http

net/http

Every language ships with a standard library, but the expectations of what a standard library should include have changed over time. As a language launched in the 2010s, Go's standard library includes something that other language distributions had considered the responsibility of a third party: a production-quality HTTP/2 client and server.

每种编程语言都自带标准库，但人们对标准库应包含内容的期望随着时间推移而发生了变化。作为一门在 2010 年代推出的语言，Go 的标准库包含了其他语言发行版通常认为应由第三方负责的功能：一套生产级的 HTTP/2 客户端和服务端。

The Client

客户

The `net/http` package defines a `Client` type to make HTTP requests and receive HTTP responses. A default client instance (cleverly named `DefaultClient`) is found in the `net/http` package, but you should avoid using it in production applications, because it defaults to having no timeout. Instead, instantiate your own. You need to create only a single `http.Client` for your entire program, as it properly handles multiple simultaneous requests across goroutines:

`net/http` 包定义了一个 `Client` 类型，用于发出 HTTP 请求和接收 HTTP 响应。`net/http` 包中提供了一个默认的客户端实例（巧妙地命名为 `DefaultClient`），但您应该避免在生产应用程序中使用它，因为它默认没有超时。相反，请实例化您自己的实例。您的整个程序只需要创建一个 `http.Client` 实例，因为它能够正确处理跨 goroutine 的多个并发请求：

```
client := &http.Client{
    Timeout: 30 * time.Second,
}
```

When you want to make a request, you create a new `*http.Request` instance with the `http.NewRequestWithContext` function, passing it a context, the method, and URL that you are connecting to. If you are making a `PUT`, `POST`, or `PATCH` request, specify the body of the request with the last parameter as an `io.Reader`. If there is no body, use `nil`:

当您想要发出请求时，请使用 `http.NewRequestWithContext` 函数创建一个新的 `*http.Request` 实例，并向其传递上下文、方法和要连接的 URL。如果您要发出 `PUT`、`POST` 或 `PATCH` 请求，请使用最后一个参数 `io.Reader` 指定请求正文。如果没有请求正文，请使用 `nil`。

```
req, err :=
http.NewRequestWithContext(context.Background(),
    http.MethodGet,
    "https://jsonplaceholder.typicode.com/todos/1",
    nil)
if err != nil {
    panic(err)
}
```

NOTE

笔记

I'll talk about what a context is in [Chapter 14](#).

我将在第 14 章讨论什么是语境。

Once you have an `*http.Request` instance, you can set any headers via the `Headers` field of the instance. Call the `Do` method on the `http.Client` with your `http.Request`, and the result is returned in an `http.Response`:

一旦你拥有一个 `*http.Request` 实例，你可以通过该实例的 `Headers` 字段设置任何标头。使用你的 `http.Request` 调用 `Do` 方法，结果将返回到 `http.Response` 中：

```
req.Header.Add("X-My-Client", "Learning Go")
res, err := client.Do(req)
if err != nil {
    panic(err)
}
```

The response has several fields with information on the request. The numeric code of the response status is in the `StatusCode` field, the text of the response code is in the `Status` field, the response headers are in the `Header` field, and any returned content is in a `Body` field of type

`io.ReadCloser` . This allows you to use it with `json.Decoder` to process REST API responses:

响应包含多个字段，其中包含有关请求的信息。响应状态的数字代码位于 `StatusCode` 字段中，响应代码的文本位于 `Status` 字段中，响应标头位于 `Header` 字段中，所有返回的内容都位于类型为 `io.ReadCloser` 的 `Body` 字段中。这允许您将其与 `json.Decoder` 一起使用来处理 REST API 响应：

```
defer res.Body.Close()
if res.StatusCode != http.StatusOK {
    panic(fmt.Sprintf("unexpected status: got %v",
res.Status))
}
fmt.Println(res.Header.Get("Content-Type"))
var data struct {
    UserID    int    `json:"userId"`
    ID        int    `json:"id"`
    Title     string `json:"title"`
    Completed bool   `json:"completed"`
}
err = json.NewDecoder(res.Body).Decode(&data)
if err != nil {
    panic(err)
}
fmt.Printf("%+v\n", data)
```

You can find this code in the *sample_code/client* directory in the [Chapter 13 repository](#).

您可以在第 13 章代码库的 *sample_code/client* 目录中找到此代码。

WARNING

警告

There are functions in the `net/http` package to make `GET`, `HEAD`, and `POST` calls. Avoid using these functions because they use the default client, which means they don't set a request timeout.

`net/http` 包中包含用于发起 `GET`、`HEAD` 和 `POST` 请求的函数。请避免使用这些函数，因为它们使用默认客户端，这意味着它们不会设置请求超时。

The Server

服务器

The HTTP Server is built around the concept of an `http.Server` and the `http.Handler` interface. Just as the `http.Client` sends HTTP requests, the `http.Server` is responsible for listening for HTTP requests. It is a performant HTTP/2 server that supports TLS.

HTTP 服务器基于 `http.Server` 和 `http.Handler` 接口构建。正如 `http.Client` 发送 HTTP 请求一样，`http.Server` 负责监听 HTTP 请求。它是一个高性能的 HTTP/2 服务器，支持 TLS。

A request to a server is handled by an implementation of the `http.Handler` interface that's assigned to the `Handler` field. This interface defines a single method:

对服务器的请求由分配给 `Handler` 字段的 `http.Handler` 接口的实现来处理。该接口定义了一个方法：

```
type Handler interface {  
    ServeHTTP(http.ResponseWriter, *http.Request)  
}
```

The `*http.Request` should look familiar, as it's the exact same type that's used to send a request to an HTTP server. The `http.ResponseWriter` is an interface with three methods:

`*http.Request` 应该看起来很眼熟，因为它与用于向 HTTP 服务器发送请求的类型完全相同。`http.ResponseWriter` 是一个包含三个方法的接口：

```
type ResponseWriter interface {  
    Header() http.Header  
    Write([]byte) (int, error)  
    WriteHeader(statusCode int)  
}
```

These methods must be called in a specific order. First, call `Header` to get an instance of `http.Header` and set any response headers you need. If you don't need to set any headers, you don't need to call it. Next, call `WriteHeader` with the HTTP status code for your response. (All the status codes are defined as constants in the `net/http` package. This would have been a good place to define a custom type, but that was not done; all status code constants are untyped integers.) If you are sending a response that has a 200 status code, you can skip `WriteHeader`. Finally, call the `Write` method to set the body for the response. Here's what a trivial handler looks like:

这些方法必须按特定顺序调用。首先，调用 `Header` 获取 `http.Header` 的实例，并设置所需的任何响应头。如果不需要设置任何响应头，则无需调用此方法。接下来，调用 `WriteHeader` 并传入响应的 HTTP 状态码。（所有状态码均在 `net/http` 包中定义为常量。这里原本可以定义自定义类型，但并未这样做；所有状态码常量均为无类型整数。）如果发送的响应状态码为 200，则可以跳过 `WriteHeader`。最后，调用 `Write` 方法设置响应正文。以下是一个简单的处理程序示例：

```
type HelloHandler struct{}

func (hh HelloHandler) ServeHTTP(w
http.ResponseWriter, r *http.Request) {
```

```
w.Write([]byte("Hello!\n"))
}
```

You instantiate a new `http.Server` just like any other struct:

你可以像实例化其他任何结构体一样实例化一个新的

`http.Server` :

```
s := http.Server{
    Addr:           ":8080",
    ReadTimeout:   30 * time.Second,
    WriteTimeout:  90 * time.Second,
    IdleTimeout:   120 * time.Second,
    Handler:       HelloHandler{},
}
err := s.ListenAndServe()
if err != nil {
    if err != http.ErrServerClosed {
        panic(err)
    }
}
```

The `Addr` field specifies the host and port the server listens on. If you don't specify them, your server defaults to listening on all hosts on the standard HTTP port, 80. You specify timeouts for the server's reads, writes, and idles by using `time.Duration` values. Be sure to set these to

properly handle malicious or broken HTTP clients, as the default behavior is to not time out at all. Finally, you specify the `http.Handler` for your server with the `Handler` field.

`Addr` 字段指定服务器监听的主机和端口。如果不指定，服务器默认监听所有主机上的标准 HTTP 端口 80。您可以使用 `time.Duration` 值指定服务器读取、写入和空闲的超时时间。请务必正确设置这些值，以便妥善处理恶意或损坏的 HTTP 客户端，因为默认行为是不设置超时。最后，您可以使用 `Handler` 字段指定服务器的 `http.Handler` 值。

You can find this code in the *sample_code/server* directory in the [Chapter 13 repository](#).

您可以在第 13 章存储库的 *sample_code/server* 目录中找到此代码。

A server that handles only a single request isn't terribly useful, so the Go standard library includes a request router, `*http.ServeMux`. You create an instance with the `http.NewServeMux` function. It meets the `http.Handler` interface, so it can be assigned to the `Handler` field in `http.Server`. It also includes two methods that allow it to dispatch requests. The first method is called `Handle` and takes in two parameters, a path and an `http.Handler`. If the path matches, the `http.Handler` is invoked.

仅处理单个请求的服务器用处不大，因此 Go 标准库包含一个请求路由器和 `*http.ServeMux`。您可以使用 `http.NewServeMux` 函数创建一个实例。它符合 `http.Handler` 接口，因此可以将其赋值给 `http.Server` 中的 `Handler` 字段。它还包含两个用于分发请求的方法。第一个方法名为 `Handle`，它接受两个参数：路径和 `http.Handler`。如果路径匹配，则调用 `http.Handler`。

While you could create implementations of `http.Handler`, the more common pattern is to use the `HandleFunc` method on `*http.ServeMux`:

虽然你可以创建 `http.Handler` 的实现，但更常见的做法是在 `*http.ServeMux` 上使用 `HandleFunc` 方法：

```
mux.HandleFunc("/hello", func(w
http.ResponseWriter, r *http.Request) {
    w.Write([]byte("Hello!\n"))
})
```

This method takes in a function or closure and converts it to an `http.HandlerFunc`. You explored the `http.HandlerFunc` type in [“Function Types Are a Bridge to Interfaces”](#). For simple handlers, a closure is sufficient. For more complicated handlers that depend on other business logic, use a method on a struct, as demonstrated in [“Implicit Interfaces Make Dependency Injection Easier”](#).

此方法接收一个函数或闭包，并将其转换为 `http.HandlerFunc` 类型。您在“函数类型是连接接口的桥梁”一文中探讨了 `http.HandlerFunc` 类型。对于简单的处理程序，闭包就足够了。对于依赖于其他业务逻辑的更复杂的处理程序，请使用结构体上的方法，如“隐式接口简化依赖注入”一文中所示。

Go 1.22 extends the path syntax to optionally allow HTTP verbs and path wildcard variables. The value of a wildcard variable is read using the `PathValue` method on `http.Request`:

Go 1.22 扩展了路径语法，允许选择性地使用 HTTP 谓词和路径通配符变量。通配符变量的值可以通过 `PathValue` 方法在 `http.Request` 上读取：

```
mux.HandleFunc("GET /hello/{name}", func(w
http.ResponseWriter,
                                r
*http.Request) {
    name := r.PathValue("name")
    w.Write([]byte(fmt.Sprintf("Hello, %s!\n",
name)))
})
```

WARNING

警告

The package-level functions `http.Handle`, `http.HandleFunc`, `http.ListenAndServe`, and `http.ListenAndServeTLS` that work with a package-level instance of the `*http.ServeMux` called `http.DefaultServeMux`. Don't use them outside trivial test programs. The `http.Server` instance is created in the `http.ListenAndServe` and `http.ListenAndServeTLS` functions, so you are unable to configure server properties like timeouts. Furthermore, third-party libraries could have registered their own handlers with the `http.DefaultServeMux`, and there's no way to know without scanning through all your dependencies (both direct and indirect). Keep your application under control by avoiding shared state.

包级函数 `http.Handle`、`http.HandleFunc`、`http.ListenAndServe` 和 `http.ListenAndServeTLS` 与名为 `http.DefaultServeMux` 的包级实例 `*http.ServeMux` 配合使用。请勿在简单的测试程序之外使用这些函数。`http.Server` 实例在 `http.ListenAndServe` 和 `http.ListenAndServeTLS` 函数中创建，因此您无法配置超时等服务器属性。此外，第三方库可能已向 `http.DefaultServeMux` 注册了它们自己的处理程序，而如果不扫描所有依赖项（包括直接依赖项和间接依赖项），则无法得知这一点。请避免共享状态，以确保应用程序的可控性。

Because `*http.ServeMux` dispatches requests to `http.Handler` instances, and since `*http.ServeMux` implements `http.Handler`, you can create an `*http.ServeMux` instance with multiple related requests and register it with a parent `*http.ServeMux`:

因为 `*http.ServeMux` 将请求分派给 `http.Handler` 个实例，并且由于 `*http.ServeMux` 实现了 `http.Handler`，所以您可以创建一

个包含多个相关请求的 `*http.ServeMux` 实例，并将其注册到父实例 `*http.ServeMux`：

```
person := http.NewServeMux()
person.HandleFunc("/greet", func(w
http.ResponseWriter, r *http.Request) {
    w.Write([]byte("greetings!\n"))
})
dog := http.NewServeMux()
dog.HandleFunc("/greet", func(w
http.ResponseWriter, r *http.Request) {
    w.Write([]byte("good puppy!\n"))
})
mux := http.NewServeMux()
mux.Handle("/person/", http.StripPrefix("/person",
person))
mux.Handle("/dog/", http.StripPrefix("/dog", dog))
```

In this example, a request for `/person/greet` is handled by handlers attached to `person`, while `/dog/greet` is handled by handlers attached to `dog`. When you register `person` and `dog` with `mux`, you use the `http.StripPrefix` helper function to remove the part of the path that's already been processed by `mux`. You can find this code in the `sample_code/server_mux` directory in the [Chapter 13 repository](#).

在这个例子中，对 `/person/greet` 的请求由附加到 `person` 的处理程序处理，而 `/dog/greet` 的请求由附加到 `dog` 的处理程序处理。当您将 `person` 和 `dog` 注册到 `mux` 时，您可以使用 `http.StripPrefix` 辅助函数来移除路径中已被 `mux` 处理过的部分。您可以在第 13 章代码库的 `sample_code/server_mux` 目录中找到这段代码。

Middleware

中间件

One of the most common requirements of an HTTP server is to perform a set of actions across multiple handlers, such as checking whether a user is logged in, timing a request, or checking a request header. Go handles these cross-cutting concerns with the *middleware pattern*.

HTTP 服务器最常见的需求之一是在多个处理程序中执行一系列操作，例如检查用户是否已登录、控制请求时长或检查请求头。Go 使用中间件模式来处理这些横切关注点。

Rather than using a special type, the middleware pattern uses a function that takes in an `http.Handler` instance and returns an `http.Handler`. Usually, the returned `http.Handler` is a closure that is converted to an `http.HandlerFunc`. Here are two middleware generators, one that

provides timing of requests and another that uses perhaps the worst access controls imaginable:

中间件模式不使用特殊类型，而是使用一个函数，该函数接收一个 `http.Handler` 实例并返回一个 `http.Handler`。通常，返回的 `http.Handler` 是一个闭包，它会被转换为 `http.HandlerFunc`。以下是两个中间件生成器，一个用于提供请求时序控制，另一个则使用了可能是你能想象到的最糟糕的访问控制：

```
func RequestTimer(h http.Handler) http.Handler {  
    return http.HandlerFunc(func(w  
http.ResponseWriter, r *http.Request) {  
        start := time.Now()  
        h.ServeHTTP(w, r)  
        dur := time.Since(start)  
        slog.Info("request time",  
            "path", r.URL.Path,  
            "duration", dur)  
    })  
}
```

```
var securityMsg = []byte("You didn't give the  
secret password\n")
```

```
func TerribleSecurityProvider(password string)  
func(http.Handler) http.Handler {  
    return func(h http.Handler) http.Handler {
```

```
        return http.HandlerFunc(func(w
http.ResponseWriter, r *http.Request) {
            if r.Header.Get("X-Secret-Password")
!= password {

w.WriteHeader(http.StatusUnauthorized)
            w.Write(securityMsg)
            return
            }
            h.ServeHTTP(w, r)
        })
    }
}
```

These two middleware implementations demonstrate what middleware does. First, you do setup operations or checks. If the checks don't pass, you write the output in the middleware (usually with an error code) and return. If all is well, you call the handler's `ServeHTTP` method. When that returns, you run cleanup operations.

这两个中间件实现演示了中间件的功能。首先，执行设置操作或检查。如果检查失败，则将输出（通常包含错误代码）写入中间件并返回。如果一切正常，则调用处理程序的 `ServeHTTP` 方法。该方法返回后，执行清理操作。

The `TerribleSecurityProvider` shows how to create configurable middleware. You pass in the configuration information (in this case, the password), and the function returns middleware that uses that configuration information. It is a bit of a mind bender, as it returns a closure that returns a closure.

`TerribleSecurityProvider` 展示了如何创建可配置中间件。您需要传入配置信息（在本例中是密码），该函数会返回一个使用该配置信息的中间件。这有点绕，因为它返回的是一个闭包，而这个闭包又返回另一个闭包。

NOTE

笔记

You might be wondering how to pass values through the layers of middleware. This is done via the context, which you'll look at in [Chapter 14](#).

您可能想知道如何将值传递到中间件的各个层级。这是通过上下文实现的，您将在第 14 章中看到相关内容。

You add middleware to your request handlers by chaining them:

您可以通过链式连接的方式将中间件添加到请求处理程序中：

```
terribleSecurity :=  
    TerribleSecurityProvider("GOPHER")
```

```
mux.Handle("/hello",
    terribleSecurity(RequestTimer(
        http.HandlerFunc(func(w http.ResponseWriter, r
            *http.Request) {
                w.Write([]byte("Hello!\n"))
            }))))
```

We get back your middleware from `TerribleSecurityProvider` and then wrap your handler in a series of function calls. This calls the `terribleSecurity` closure first, then calls the `RequestTimer`, which then calls your actual request handler.

我们从 `TerribleSecurityProvider` 获取您的中间件，然后将您的处理程序包装在一系列函数调用中。这首先调用 `terribleSecurity` 闭包，然后调用 `RequestTimer`，最后调用您的实际请求处理程序。

Because `*http.ServeMux` implements the `http.Handler` interface, you can apply a set of middleware to all the handlers registered with a single request router:

由于 `*http.ServeMux` 实现了 `http.Handler` 接口，因此您可以将一组中间件应用于注册到单个请求路由器的所有处理程序：

```
terribleSecurity :=
    TerribleSecurityProvider("GOPHER")
```



```
wrappedMux := terribleSecurity(RequestTimer(mux))
s := http.Server{
    Addr:      ":8080",
    Handler: wrappedMux,
}
```

You can find this code in the *sample_code/middleware* directory in the [Chapter 13 repository](#).

您可以在第 13 章存储库的 *sample_code/middleware* 目录中找到此代码。

Use third-party modules to enhance the server

使用第三方模块增强服务器性能

Just because the server is production quality doesn't mean that you shouldn't use third-party modules to improve its functionality. If you don't like the function chains for middleware, you can use a third-party module called [alice](#), which allows you to use the following syntax:

服务器达到生产级别并不意味着您不应该使用第三方模块来增强其功能。如果您不喜欢中间件的函数链，可以使用名为 `alice` 的第三方模块，它允许您使用以下语法：

```
helloHandler := func(w http.ResponseWriter, r
*http.Request) {
    w.Write([]byte("Hello!\n"))
}
chain := alice.New(terribleSecurity,
RequestTimer).ThenFunc(helloHandler)
mux.Handle("/hello", chain)
```

While `*http.ServeMux` gained some much-requested features in Go 1.22, its routing and variable support are still basic. Nesting `*http.ServeMux` instances is also a bit clunky. If you find yourself needing more advanced features, such as routing based on a header value, specifying a path variable using a regular expression, or better handler nesting, many third-party request routers are available. Two of the most popular ones are [gorilla mux](#) and [chi](#). Both are considered idiomatic because they work with `http.Handler` and `http.HandlerFunc` instances, demonstrating the Go philosophy of using composable libraries that fit together with the standard library. They also work with idiomatic middleware, and both projects provide optional middleware implementations of common concerns.

虽然 Go 1.22 版本中 `*http.ServeMux` 获得了一些用户呼声很高的功能，但它的路由和变量支持仍然比较基础。嵌套 `*http.ServeMux` 实例也略显笨拙。如果您需要更高级的功能，例如基于请求头值进行路由、使用正则表达式指定路径变量或更完善的处理程序嵌套，可以

使用许多第三方请求路由器。其中两个最流行的是 gorilla mux 和 chi。它们都被认为是符合 Go 惯用的，因为它们可以与 `http.Handler` 和 `http.HandlerFunc` 实例配合使用，体现了 Go 使用可组合库并与标准库兼容的理念。它们也支持惯用的中间件，并且这两个项目都提供了针对常见问题的可选中间件实现。

Several popular web frameworks also implement their own handler and middleware patterns. Two of the most popular are [Echo](#) and [Gin](#). They simplify web development by incorporating features like automating the binding of data in requests or responses to JSON. They also provide adapter functions that enable you to use `http.Handler` implementations, providing a migration path.

一些流行的 Web 框架也实现了自己的处理程序和中间件模式。其中最流行的两个是 Echo 和 Gin。它们通过集成诸如自动将请求或响应中的数据绑定到 JSON 等功能来简化 Web 开发。它们还提供适配器函数，使您能够使用 `http.Handler` 实现，从而提供迁移路径。

ResponseController

响应控制器

In [“Accept Interfaces, Return Structs”](#), you learned that modifying interfaces breaks backward compatibility. You also learned that you can evolve an interface over time by defining new interfaces and using type

switches and type assertions to see whether the new interfaces are implemented. The drawback to creating these additional interfaces is that it is difficult to know that they exist, and using type switches to check for them is verbose.

在“接受接口，返回结构体”一节中，你了解到修改接口会破坏向后兼容性。你也了解到，可以通过定义新接口，并使用类型开关和类型断言来检查新接口是否已实现，从而随着时间的推移不断演进接口。创建这些额外接口的缺点在于，很难知道它们是否存在，而且使用类型开关来检查它们也很冗长。

You can find an example of this in the `http` package. When the package was designed, the choice was made to make `http.ResponseWriter` an interface. This meant that additional methods could not be added to it in future releases, or the Go Compatibility Guarantee would be broken. To represent new optional functionality for an `http.ResponseWriter` instance, the `http` package contains a couple of interfaces that may be implemented by `http.ResponseWriter` implementations:

`http.Flusher` and `http.Hijacker`. The methods on these interfaces are used to control the output for a response.

您可以在 `http` 包中找到一个示例。该包在设计时，决定将其设计为一个接口。这意味着在未来的版本中无法向其添加其他方法，否则将违反 Go 兼容性保证。为了表示 `http.ResponseWriter` 实例的新增可选功能，`http` 包包含两个可供 `http.ResponseWriter` 实现使用

的接口： `http.Flusher` 和 `http.Hijacker`。这些接口中的方法用于控制响应的输出。

In Go 1.20, a new concrete type was added to the `http` package, `http.ResponseController`. It demonstrates another way to expose methods that have been added to an existing API:

在 Go 1.20 中， `http` 包中添加了一个新的具体类型 `http.ResponseController`。它展示了另一种公开已添加到现有 API 中的方法的方式：

```
func handler(rw http.ResponseWriter, req
*http.Request) {
    rc := http.NewResponseController(rw)
    for i := 0; i < 10; i++ {
        result := doStuff(i)
        _, err := rw.Write([]byte(result))
        if err != nil {
            slog.Error("error writing", "msg",
err)
            return
        }
        err = rc.Flush()
        if err != nil && !errors.Is(err,
http.ErrNotSupported) {
            slog.Error("error flushing", "msg",
err)
```

```
        return  
    }  
}  
}
```

In this example, you want to return data to the client as it is calculated, if the `http.ResponseWriter` supports `Flush`. If not, you will return all the data after all of it has been calculated. The factory function `http.NewResponseController` takes in an `http.ResponseWriter` and returns a pointer to an `http.ResponseController`. This concrete type has methods for the optional functionality of an `http.ResponseWriter`. You check whether the optional method is implemented by the underlying `http.ResponseWriter` by comparing the returned error to `http.ErrNotSupported` using `errors.Is`. You can find this code in the *sample_code/response_controller* directory in the [Chapter 13 repository](#).

在这个例子中，您希望在计算完成后立即将数据返回给客户端（如果 `http.ResponseWriter` 支持 `Flush`）。否则，您将在所有数据计算完成后返回所有数据。工厂函数 `http.NewResponseController` 接收一个 `http.ResponseWriter` 并返回一个指向 `http.ResponseController` 的指针。这个具体类型具有 `http.ResponseWriter` 的可选功能的方法。您可以通过将返回的错误与 `http.ErrNotSupported` 进行比较（使用 `errors.Is`），来

检查底层 `http.ResponseWriter` 是否实现了可选方法。您可以在第 13 章代码库的 `sample_code/response_controller` 目录中找到这段代码。

Because `http.ResponseController` is a concrete type that wraps access to an `http.ResponseWriter` implementation, new methods can be added to it over time without breaking existing implementations. This makes new functionality discoverable and provides a way to check for the presence or absence of an optional method with a standard error check. This pattern is an interesting way to handle situations where an interface needs to evolve. In fact, `http.ResponseController` contains two methods that don't have corresponding interfaces: `SetReadDeadline` and `SetWriteDeadline`. Future optional methods on `http.ResponseWriter` will likely be added via this technique.

由于 `http.ResponseController` 是一个封装了对 `http.ResponseWriter` 实现的访问的具体类型，因此可以随着时间的推移向其添加新方法而不会破坏现有实现。这使得新功能更容易被发现，并提供了一种使用标准错误检查来判断可选方法是否存在的方法。这种模式是处理接口需要演进情况的一种有效方式。事实上，`http.ResponseController` 包含两个没有对应接口的方法：`SetReadDeadline` 和 `SetWriteDeadline`。未来 `http.ResponseWriter` 上的可选方法很可能会通过这种技术添加。

Structured Logging

结构化日志

Since its initial release, the Go standard library has included a simple logging package, `log`. While it's fine for small programs, it doesn't easily produce *structured* logs. Modern web services can have millions of simultaneous users, and at that scale you need software to process log output in order to understand what's going on. A structured log uses a documented format for each log entry, making it easier to write programs that process log output and discover patterns and anomalies.

自 Go 标准库发布以来，它就包含了一个简单的日志包 `log`。虽然它对于小型程序来说足够用了，但却难以生成结构化日志。现代 Web 服务可能拥有数百万并发用户，在这种规模下，你需要软件来处理日志输出，才能了解发生了什么。结构化日志为每个日志条目使用规范的格式，这使得编写处理日志输出并发现模式和异常的程序变得更加容易。

JSON is commonly used for structured logs, but even whitespace separated key-value pairs are easier to process than unstructured logs that don't separate values into fields. While you certainly could write JSON by using the `log` package, it doesn't provide any support for simplifying structured log creation. The `log/slog` package resolves this problem.

JSON 通常用于结构化日志，但即使是使用空格分隔的键值对，也比未将值分隔到字段中的非结构化日志更容易处理。虽然您可以使用 `log` 包编写 JSON，但它并不提供任何简化结构化日志创建的支持。`log/slog` 包解决了这个问题。

Adding `log/slog` to the standard library demonstrated several good Go library design practices. The first good decision was to include structured logging in the standard library. Having a standard structured logger makes it easier to write modules that work together. Several third-party structured loggers have been released to address the shortcomings of `log`, including [zap](#), [logrus](#), [go-kit log](#), and many others. The problem with a fragmented logging ecosystem is that you want control over where log output goes and what level of messages are logged. If your code depends on third-party modules that use different loggers, this becomes impossible. The usual advice to prevent logging fragmentation is to not log in a module intended as a library, but that's impossible to enforce and makes it harder to monitor what's going on in a third-party library. The `log/slog` package was new in Go 1.21, but the fact that it solves these inconsistencies makes it likely that within a few years, it will be used in the vast majority of Go programs.

将 `log/slog` 添加到标准库体现了 Go 语言的几项优秀库设计实践。首先，将结构化日志记录功能集成到标准库中是一个明智之举。拥有一个标准的结构化日志记录器使得编写能够协同工作的模块变得更加容易。为了弥补 `log` 的不足，已经发布了许多第三方结构化日志记录器，例如 `zap`、`logrus`、`go-kit log` 等等。日志生态系统碎片化的问题

在于，开发者需要控制日志输出的位置以及记录的消息级别。如果你的代码依赖于使用不同日志记录器的第三方模块，那么这种控制就变得不可能。通常防止日志碎片化的建议是，不要在作为库的模块中使用日志记录，但这很难强制执行，而且会使监控第三方库内部的运行情况变得更加困难。 `log/slog` 包是 Go 1.21 中的新包，但它解决了这些不一致之处，因此很可能在几年内，它将在绝大多数 Go 程序中使用。

The second good decision was to make structured logging its own package and not part of the `log` package. While both packages have similar purposes, they have very different design philosophies. Trying to add structured logging into an unstructured logging package would confuse the API. By making them separate packages, you know at a glance that `slog.Info` is a structured log and `log.Print` is unstructured, even if you don't remember whether `Info` is for structured or unstructured logging.

第二个明智的决定是将结构化日志记录单独打包，而不是将其作为 `log` 包的一部分。虽然这两个包的用途相似，但它们的设计理念截然不同。如果尝试将结构化日志记录添加到非结构化日志记录包中，将会造成 API 混乱。通过将它们作为独立的包，即使您不记得 `Info` 是用于结构化日志还是非结构化日志，也能一目了然地知道 `slog.Info` 是结构化日志，`log.Print` 是非结构化日志。

The next good decision was to make the `log/slog` API scalable. It starts simply, with a default logger available via functions:

下一个明智的决定是使 `log/slog` API 具备可扩展性。它从一个简单的函数开始：提供一个可通过函数访问的默认日志记录器。

```
func main() {  
    slog.Debug("debug log message")  
    slog.Info("info log message")  
    slog.Warn("warning log message")  
    slog.Error("error log message")  
}
```

These functions allow you to log simple messages at various logging levels. The output looks like the following:

这些函数允许您以不同的日志级别记录简单的消息。输出结果如下所示：

```
2023/04/20 23:13:31 INFO info log message  
2023/04/20 23:13:31 WARN warning log message  
2023/04/20 23:13:31 ERROR error log message
```

There are two things to notice. First, the default logger suppresses debug messages by default. You'll see how to control the logging level when I discuss creating your own logger in a bit.

有两点需要注意。首先，默认日志记录器默认会抑制调试信息。稍后我会讲解如何创建自定义日志记录器，届时您将了解如何控制日志级别。

The second point is a bit more subtle. While this is plain-text output, it uses whitespace to make a structured log. The first column is the date in year/month/day format. The second column is the time in 24-hour time. The third column is the logging level. Finally, there is the message.

第二点比较微妙。虽然输出的是纯文本，但它利用空格来构建结构化日志。第一列是日期，格式为年/月/日。第二列是时间，采用24小时制。第三列是日志级别。最后一列是消息。

The power of structured logging comes from the ability to add fields with custom values. Update your logs with some custom fields:

结构化日志的强大之处在于能够添加具有自定义值的字段。使用一些自定义字段更新您的日志：

```
userID := "fred"
loginCount := 20
slog.Info("user login",
    "id", userID,
    "login_count", loginCount)
```

You use the same function as before, but now you add optional arguments. Optional arguments come in pairs. The first part is the key, which should be a string. The second part is the value. This log line prints out the following:

你使用了与之前相同的函数，但现在添加了可选参数。可选参数成对出现。第一部分是键，它应该是一个字符串。第二部分是值。这条日志输出如下：

```
2023/04/20 23:36:38 INFO user login id=fred  
login_count=20
```

After the message, you have key-value pairs, again space separated.

消息之后是键值对，同样以空格分隔。

While this text format is far easier to parse than an unstructured log, you might want to use something like JSON instead. You also might want to customize where the log is written or the logging level. To do that, you create a structured logging instance:

虽然这种文本格式比非结构化日志更容易解析，但您可能更希望使用JSON之类的格式。您可能还希望自定义日志的写入位置或日志级别。为此，您可以创建一个结构化日志实例：

```
options := &slog.HandlerOptions{Level:  
slog.LevelDebug}
```

```
handler := slog.NewJSONHandler(os.Stderr, options)
mySlog := slog.New(handler)
lastLogin := time.Date(2023, 01, 01, 11, 50, 00,
00, time.UTC)
mySlog.Debug("debug message",
    "id", userID,
    "last_login", lastLogin)
```

You are using the `slog.HandlerOptions` struct to define the minimum logging level for the new logger. You then use the `NewJSONHandler` method on `slog.HandlerOptions` to create a `slog.Handler` that writes logs using JSON to the specified `io.Writer`. In this case, you are using the standard error output. Finally, you use the `slog.New` function to create a `*slog.Logger` that wraps the `slog.Handler`. You then create a `lastLogin` value to log, along with a user ID. This gives the following output:

您使用 `slog.HandlerOptions` 结构体来定义新日志记录器的最低日志级别。然后，您使用 `NewJSONHandler` 方法对 `slog.HandlerOptions` 创建一个 `slog.Handler`，该记录器使用 JSON 格式将日志写入指定的 `io.Writer`。在本例中，您使用的是标准错误输出。最后，您使用 `slog.New` 函数创建一个 `*slog.Logger`，该记录器封装了 `slog.Handler`。然后，您创建一个 `lastLogin` 值来记录日志，并附带一个用户 ID。这将产生以下输出：

```
{"time":"2023-04-22T23:30:01.170243-04:00","level":"DEBUG",  
  "msg":"debug  
message","id":"fred","last_login":"2023-01-01T11:50:00Z"}
```

If JSON and text aren't sufficient for your output needs, you can define your own implementation of the `slog.Handler` interface and pass it to `slog.New`.

如果 JSON 和文本不足以满足您的输出需求，您可以定义自己的 `slog.Handler` 接口实现，并将其传递给 `slog.New`。

Finally, the `log/slog` package takes performance into consideration. If you aren't careful, your program might end up spending more time writing logs than doing the work it was designed to perform. You can choose to write data out to `log/slog` in a number of ways. You have already seen the simplest (but slowest) method, using alternating keys and values on the `Debug`, `Info`, `Warn`, and `Error` methods. For improved performance with fewer allocations, use the `LogAttrs` method instead:

最后，`log/slog` 包也考虑到了性能。如果不注意，程序可能会花费更多时间写入日志，而不是执行其设计的功能。您可以选择多种方式将数据写入 `log/slog`。您已经了解了最简单（但速度最慢）的方

法，即在 `Debug`、`Info`、`Warn` 和 `Error` 方法中使用键值交替。为了提高性能并减少内存分配，请改用 `LogAttrs` 方法：

```
mySlog.LogAttrs(ctx, slog.LevelInfo, "faster
logging",
               slog.String("id", userID),
               slog.Time("last_login",
lastLogin))
```

The first parameter is a `context.Context`, next comes the logging level, and then zero or more `slog.Attr` instances. There are factory functions for the most commonly used types, and you can use `slog.Any` for the ones that don't have functions already supplied.

第一个参数是 `context.Context`，接下来是日志级别，然后是零个或多个 `slog.Attr` 个实例。对于最常用的类型，我们提供了工厂函数；对于没有提供函数的类型，您可以使用 `slog.Any`。

Because of the Go Compatibility Promise, the `log` package isn't going away. Existing programs that use it will continue to work, as will programs that use third-party structured loggers. If you have code that uses a `log.Logger`, the `slog.NewLogLogger` function provides a bridge to the original `log` package. It creates a `log.Logger` instance that uses a `slog.Handler` to write its output:

由于 Go 兼容性承诺，`log` 包不会被移除。使用它的现有程序以及使用第三方结构化日志记录器的程序将继续运行。如果您有使用 `log.Logger` 的代码，`slog.NewLogLogger` 函数提供了一个与原始 `log` 包的桥梁。它创建一个 `log.Logger` 实例，该实例使用 `slog.Handler` 来写入其输出：

```
myLog := slog.NewLogLogger(mySlog.Handler(),
slog.LevelDebug)
myLog.Println("using the mySlog Handler")
```

This produces the following output:

这将产生以下输出：

```
{"time":"2023-04-22T23:30:01.170269-
04:00","level":"DEBUG",
  "msg":"using the mySlog Handler"}
```

You can find all the coding examples for `log/slog` in the *sample_code/structured_logging* directory in the [Chapter 13 repository](#).

您可以在第 13 章存储库的 `sample_code/structured_logging` 目录中找到 `log/slog` 的所有编码示例。

The `log/slog` API includes more features, including dynamic logging-level support, context support (the context is covered in [Chapter 14](#)),

grouping values, and creating a common header of values. You can learn more by looking at its [API documentation](#). Most importantly, look at how `log/slog` was put together so you can learn how to construct APIs of your own.

`log/slog` API 包含更多功能，例如动态日志级别支持、上下文支持（上下文在第 14 章中介绍）、值分组以及创建公共值标头。您可以查阅其 API 文档了解更多信息。最重要的是，了解 `log/slog` 的构建方式，以便学习如何构建自己的 API。

Exercises

练习

Now that you've learned more about the standard library, work through these exercises to reinforce what you've learned. Solutions are in the *exercise_solutions* directory of the [Chapter 13 repository](#).

现在你已经对标准库有了更深入的了解，请完成以下练习以巩固所学知识。答案位于第 13 章代码库的 `exercise_solutions` 目录中。

1. Write a small web server that returns the current time in RFC 3339 format when you send it a `GET` command. You can use a third-party module if you'd like.

编写一个小型 Web 服务器，当收到 GET 命令时，该服务器以 RFC 3339 格式返回当前时间。如果需要，可以使用第三方模块。

2. Write a small middleware component that uses JSON structured logging to log the IP address of each incoming request to your web server.

编写一个小型中间件组件，使用 JSON 结构化日志记录发送到 Web 服务器的每个请求的 IP 地址。

3. Add the ability to return the time as JSON. Use the Accept header to control whether JSON or text is returned (default to text). The JSON should be structured as follows:

新增以 JSON 格式返回时间的功能。使用 Accept 请求头控制返回 JSON 还是文本格式（默认为文本）。JSON 数据结构应如下所示：

```
{
  "day_of_week": "Monday",
  "day_of_month": 10,
  "month": "April",
  "year": 2023,
  "hour": 20,
  "minute": 15,
  "second": 20
}
```

Wrapping Up

总结

In this chapter, you looked at some of the most commonly used packages in the standard library and saw how they embody best practices that should be emulated in your code. You've also seen other sound software engineering principles: how some decisions might have been made differently given experience, and how to respect backward compatibility so you can build applications on a solid foundation.

在本章中，您了解了标准库中最常用的一些包，并看到了它们如何体现了您应该在代码中效仿的最佳实践。您还了解了其他一些优秀的软件工程原则：例如，根据经验，某些决策可能会有不同的做法，以及如何尊重向后兼容性，从而在坚实的基础上构建应用程序。

In the next chapter, you're going to look at the context, a package and pattern for passing state and timers through Go code.

下一章，我们将探讨通过 Go 代码传递状态和计时器的上下文、包和模式。

Chapter 14. The Context

第十四章 背景

Servers need a way to handle metadata on individual requests. This metadata falls into two general categories: metadata that is required to correctly process the request, and metadata on when to stop processing the request. For example, an HTTP server might want to use a tracking ID to identify a chain of requests through a set of microservices. It also might want to set a timer that ends requests to other microservices if they take too long.

服务器需要一种方法来处理单个请求的元数据。这些元数据大致分为两类：正确处理请求所需的元数据，以及何时停止处理请求的元数据。例如，HTTP 服务器可能需要使用跟踪 ID 来识别一组微服务之间的请求链。它还可能需要一个定时器，如果对其他微服务的请求耗时过长，则终止这些请求。

Many languages use *threadlocal* variables to store this kind of information, associating data to a specific operating system thread of execution. This doesn't work in Go because goroutines don't have unique identities that can be used to look up values. More importantly, threadlocals feel like magic; values go in one place and pop up somewhere else.

许多语言使用线程局部变量来存储这类信息，将数据与特定的操作系统线程关联起来。但这在 Go 中行不通，因为 goroutine 没有可用于查找值的唯一标识。更重要的是，线程局部变量给人的感觉就像魔法一样：值存储在一个地方，却出现在另一个地方。

Go solves the request metadata problem with a construct called the *context*. Let's see how to use it correctly.

Go 语言使用一种名为 context 的结构来解决请求元数据问题。让我们看看如何正确使用它。

What Is the Context?

背景是什么？

Rather than add a new feature to the language, a context is simply an instance that meets the `Context` interface defined in the `context` package. As you know, idiomatic Go encourages explicit data passing via function parameters. The same is true for the context. It is just another parameter to your function.

上下文并非为语言添加新功能，而仅仅是满足 `context` 包中定义的 `Context` 接口的实例。如您所知，Go 语言的惯用写法鼓励通过函数参数显式传递数据。上下文也是如此，它只是函数的另一个参数。

Just as Go has a convention that the last return value from a function is an `error`, Go has another convention that the context is explicitly passed through your program as the first parameter of a function. The usual name for the context parameter is `ctx`:

就像 Go 语言约定函数的最后一个返回值是 `error` 一样，Go 语言还有另一个约定，即上下文必须作为函数的第一个参数显式地传递给程序。上下文参数的常用名称是 `ctx`：

```
func logic(ctx context.Context, info string)
(string, error) {
    // do some interesting stuff here
    return "", nil
}
```

In addition to defining the `Context` interface, the `context` package contains several factory functions for creating and wrapping contexts. When you don't have an existing context, such as at the entry point to a command-line program, create an empty initial context with the function `context.Background`. This returns a variable of type `context.Context`. (Yes, this is an exception to the usual pattern of returning a concrete type from a function call.)

除了定义 `Context` 接口之外，`context` 包还包含几个用于创建和包装上下文的工厂函数。当没有现有上下文时（例如在命令程序的入

口点)，可以使用函数 `context.Background` 创建一个空的初始上下文。该函数返回一个 `context.Context` 类型的变量。（是的，这与通常函数调用返回具体类型的模式有所不同。）

An empty context is a starting point; each time you add metadata to the context, you do so by *wrapping* the existing context by using one of the factory functions in the `context` package.

空上下文是一个起点；每次向上下文添加元数据时，都是通过使用 `context` 包中的工厂函数之一来包装现有上下文。

NOTE

笔记

Another function, `context.TODO`, also creates an empty `context.Context`. It is intended for temporary use during development. If you aren't sure where the context is going to come from or how it's going to be used, use `context.TODO` to put a placeholder in your code. Production code shouldn't include `context.TODO`.

另一个函数 `context.TODO` 也会创建一个空的 `context.Context`。它用于开发期间的临时使用。如果您不确定上下文的来源或使用方式，请使用 `context.TODO` 在代码中放置一个占位符。生产代码不应包含 `context.TODO`。

When writing an HTTP server, you use a slightly different pattern for acquiring and passing the context through layers of middleware to the top-level `http.Handler`. Unfortunately, context was added to the Go APIs long after the `net/http` package was created. Because of the

compatibility promise, there was no way to change the `http.Handler` interface to add a `context.Context` parameter.

编写 HTTP 服务器时，获取上下文并将其通过中间件层层传递到顶层 `http.Handler` 的模式略有不同。遗憾的是，上下文是在 `net/http` 包创建很久之后才添加到 Go API 中的。由于兼容性问题，无法修改 `http.Handler` 接口来添加 `context.Context` 参数。

The compatibility promise does allow new methods to be added to existing types, and that's what the Go team did. Two context-related methods are on `http.Request` :

兼容性承诺允许向现有类型添加新方法，Go 团队正是这样做的。

`http.Request` 中包含两个与上下文相关的方法：

- `Context` returns the `context.Context` associated with the request.
`Context` 返回与请求关联的 `context.Context` 。
- `WithContext` takes in a `context.Context` and returns a new `http.Request` with the old request's state combined with the supplied `context.Context` .

`WithContext` 接收一个 `context.Context` ，并返回一个新的 `http.Request` ，其中包含旧请求的状态以及提供的 `context.Context` 。

Here's the general pattern:

一般模式如下：

```
func Middleware(handler http.Handler) http.Handler
{
    return http.HandlerFunc(func(rw
http.ResponseWriter, req *http.Request) {
        ctx := req.Context()
        // wrap the context with stuff -- you'll
see how soon!
        req = req.WithContext(ctx)
        handler.ServeHTTP(rw, req)
    })
}
```

The first thing you do in your middleware is extract the existing context from the request by using the `Context` method. (If you want to skip ahead, you can see how to put values into the context in [“Values”](#).) After you put values into the context, you create a new request based on the old request and the now-populated context by using the `WithContext` method. Finally, you call the `handler` and pass it your new request and the existing `http.ResponseWriter`.

在中间件中，首先要使用 `Context` 方法从请求中提取现有上下文。

（如果您想跳过这一步，可以在“值”部分查看如何将值放入上下文。）将值放入上下文后，使用 `WithContext` 方法基于旧请求和新

填充的上下文创建一个新请求。最后，调用 `handler` 方法，并将新请求和现有的 `http.ResponseWriter` 传递给它。

When you implement the handler, extract the context from the request by using the `Context` method and call your business logic with the context as the first parameter, just as you saw previously:

在实现处理程序时，使用 `Context` 方法从请求中提取上下文，并将上下文作为第一个参数调用业务逻辑，就像您之前看到的那样：

```
func handler(rw http.ResponseWriter, req
*http.Request) {
    ctx := req.Context()
    err := req.ParseForm()
    if err != nil {

rw.WriteHeader(http.StatusInternalServerError)
    rw.Write([]byte(err.Error()))
    return
    }
    data := req.FormValue("data")
    result, err := logic(ctx, data)
    if err != nil {

rw.WriteHeader(http.StatusInternalServerError)
    rw.Write([]byte(err.Error()))
    return
    }
```

```
    }  
    rw.Write([]byte(result))  
}
```

When making an HTTP call from your application to another HTTP service, use the `NewRequestWithContext` function in the `net/http` package to construct a request that includes existing context information:

当应用程序向另一个 HTTP 服务发出 HTTP 请求时，请使用 `net/http` 包中的 `NewRequestWithContext` 函数来构造包含现有上下文信息的请求：

```
type ServiceCaller struct {  
    client *http.Client  
}  
  
func (sc ServiceCaller) callAnotherService(ctx  
context.Context, data string)  
                                (string,  
error) {  
    req, err := http.NewRequestWithContext(ctx,  
http.MethodGet,  
                                "http://example.com?data="+data,  
nil)  
    if err != nil {  
        return "", err
```

```

    }
    resp, err := sc.client.Do(req)
    if err != nil {
        return "", err
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        return "", fmt.Errorf("Unexpected status
code %d",
                                resp.StatusCode)
    }
    // do the rest of the stuff to process the
response
    id, err := processResponse(resp.Body)
    return id, err
}

```

You can find these code samples in the *sample_code/context_patterns* directory in the [Chapter 14 repository](#).

您可以在第 14 章存储库的 *sample_code/context_patterns* 目录中找到这些代码示例。

Now that you know how to acquire and pass a context, let's start making them useful. You'll begin with passing values.

既然你已经知道如何获取和传递上下文，接下来我们就来学习如何让它们发挥作用。首先，我们将学习如何传递值。

Values

价值观

By default, you should prefer to pass data through explicit parameters. As has been mentioned before, idiomatic Go favors the explicit over the implicit, and this includes explicit data passing. If a function depends on some data, it should be clear what data it needs and where that data came from.

默认情况下，您应该优先使用显式参数传递数据。如前所述，Go 语言的惯用写法倾向于显式传递而非隐式传递，这其中也包括显式数据传递。如果一个函数依赖于某些数据，那么它需要哪些数据以及这些数据的来源应该非常明确。

However, in some cases you cannot pass data explicitly. The most common situation is an HTTP request handler and its associated middleware. As you have seen, all HTTP request handlers have two parameters, one for the request and one for the response. If you want to make a value available to your handler in middleware, you need to store it in the context. Possible situations include extracting a user from a JWT (JSON Web Token) or

creating a per-request GUID that is passed through multiple layers of middleware and into your handler and business logic.

然而，在某些情况下，您无法显式传递数据。最常见的情况是 HTTP 请求处理程序及其关联的中间件。如您所见，所有 HTTP 请求处理程序都有两个参数，一个用于请求，一个用于响应。如果您想在中间件中为处理程序提供某个值，则需要将其存储在上下文中。例如，从 JWT (JSON Web Token) 中提取用户，或者创建一个每个请求的 GUID，该 GUID 会经过多层中间件传递到您的处理程序和业务逻辑中。

There is a factory method for putting values into the context, `context.WithValue`. It takes in three values: a context, a key to look up the value, and the value itself. The key and the value parameters are declared to be of type `any`. The `context.WithValue` function returns a context, but it is not the same context that was passed into the function. Instead, it is a *child* context that contains the key-value pair and *wraps* the passed-in *parent* `context.Context`.

有一个工厂方法用于将值放入上下文中，即 `context.WithValue`。它接受三个参数：一个上下文、一个用于查找值的键以及值本身。键和值参数被声明为 `any` 类型。`context.WithValue` 函数返回一个上下文，但它并非传递给该函数的同一个上下文。相反，它是一个子上下文，其中包含键值对，并包装了传入的父上下文 `context.Context`。

NOTE

笔记

You'll see this wrapping pattern several times. A context is treated as an immutable instance.

Whenever you add information to a context, you do so by wrapping an existing parent context with a child context. This allows you to use contexts to pass information into deeper layers of the code. The context is never used to pass information out of deeper layers to higher layers.

你会多次看到这种包装模式。上下文被视为不可变实例。每当你向上下文添加信息时，你都是通过将现有的父上下文包装在一个子上下文中来实现的。这允许你使用上下文将信息传递到代码的更深层。上下文永远不会用于将信息从更深层传递到更高层。

The `Value` method on `context.Context` checks whether a value is in a context or any of its parent contexts. This method takes in a key and returns the value associated with the key. Again, both the key parameter and the value result are declared to be of type `any`. If no value is found for the supplied key, `nil` is returned. Use the comma ok idiom to type-assert the returned value to the correct type:

`context.Context` 上的 `Value` 方法检查一个值是否存在于某个上下文或其任何父上下文中。此方法接收一个键，并返回与该键关联的值。同样，键参数和值结果都被声明为 `any` 类型。如果找不到与提供的键对应的值，则返回 `nil`。使用逗号 ok 惯用法来断言返回值的类型是否正确：

```
ctx := context.Background()
if myVal, ok := ctx.Value(myKey).(int); !ok {
```



```
    fmt.Println("no value")
} else {
    fmt.Println("value:", myVal)
}
```

NOTE

笔记

If you are familiar with data structures, you might recognize that searching for values stored in the context chain is a *linear* search. This has no serious performance implications when there are only a few values, but it would perform poorly if you stored dozens of values in the context during a request. That said, if your program is creating a context chain with dozens of values, your program probably needs some refactoring.

如果您熟悉数据结构，您可能会意识到，在上下文链中查找存储的值实际上是一个线性搜索。当值数量很少时，这不会对性能造成严重影响；但如果一次请求中将数十个值存储在上下文中，性能就会显著下降。也就是说，如果您的程序创建了一个包含数十个值的上下文链，那么您的程序可能需要进行一些重构。

The value stored in the context can be of any type, but picking the correct key is important. Like the key for a `map`, the key for a context value must be comparable. Don't just use a `string` like `"id"`. If you use `string` or another predefined or exported type for the type of the key, different packages could create identical keys, resulting in collisions. This causes problems that are hard to debug, such as one package writing data to the context that masks the data written by another package, or reading data from the context that was written by another package.

存储在上下文中的值可以是任何类型，但选择正确的键至关重要。与 `map` 的键一样，上下文值的键必须是可比较的。不要像使用 `"id"` 那样使用 `string`。如果使用 `string` 或其他预定义或导出的类型作为键，不同的包可能会创建相同的键，从而导致冲突。这会引发难以调试的问题，例如一个包写入上下文的数据会覆盖另一个包写入的数据，或者从上下文中读取由另一个包写入的数据。

Two patterns are used to guarantee that a key is unique and comparable. The first creates a new, unexported type for the key, based on an `int`:

为确保键的唯一性和可比性，使用了两种模式。第一种模式基于 `int` 为键创建一个新的、未导出的类型：

```
type userKey int
```

After declaring your unexported key type, you then declare an unexported constant of that type:

声明未导出的键类型之后，接下来需要声明该类型的未导出常量：

```
const (  
    _ userKey = iota  
    key  
)
```

With both the type and the typed constant of the key being unexported, no code from outside your package can put data into the context that would cause a collision. If your package needs to put multiple values into the context, define a different key of the same type for each value, using the `iota` pattern you looked at in [“iota Is for Enumerations—Sometimes”](#). Since you care about the constant’s value only as a way to differentiate multiple keys, this is a perfect use for `iota`.

由于键的类型和类型化常量均未导出，因此包外部的代码无法将数据放入上下文中，从而避免冲突。如果您的包需要将多个值放入上下文中，请为每个值定义一个相同类型的不同键，使用您在“iota 用于枚举——有时”中看到的 `iota` 模式。由于您仅将常量值作为区分多个键的一种方式，因此 `iota` 非常适合这种情况。

Next, build an API to place a value into the context and to read the value from the context. Make these functions public only if code outside your package should be able to read and write your context values. The name of the function that creates a context with the value should start with `ContextWith`. The function that returns the value from the context should have a name that ends with `FromContext`. Here are the implementations of functions to set and read the user from the context:

接下来，构建一个 API，用于向上下文中放置值以及从上下文中读取值。仅当包外部的代码需要能够读取和写入上下文值时，才将这些函数设为公共函数。创建包含值的上下文的函数名称应以

ContextWith 开头。从上下文中返回值的函数名称应以 FromContext 结尾。以下是设置和读取上下文中用户的函数实现：

```
func ContextWithUser(ctx context.Context, user
string) context.Context {
    return ctx.WithValue(key, user)
}

func UserFromContext(ctx context.Context) (string,
bool) {
    user, ok := ctx.Value(key).(string)
    return user, ok
}
```

Another option is to define the unexported key type by using an empty struct:

另一种方法是使用空结构体来定义未导出的键类型：

```
type userKey struct{}
```

The functions for managing access to the context value are then changed:

然后，对用于管理对上下文值访问的函数进行更改：

```
func ContextWithUser(ctx context.Context, user
string) context.Context {
    return context.WithValue(ctx, userKey{}, user)
}

func UserFromContext(ctx context.Context) (string,
bool) {
    user, ok := ctx.Value(userKey{}).(string)
    return user, ok
}
```

How do you know which key style to use? If you have a set of related keys for storing different values in the context, use the `int` and `iota` technique. If you have only a single key, either is fine. The important thing is that you want to make it impossible for context keys to collide.

如何确定使用哪种键样式？如果您有一组相关的键用于存储上下文中的不同值，请使用 `int` 和 `iota` 的方法。如果您只有一个键，则两种方法都可以。重要的是要确保上下文键之间不会发生冲突。

Now that you've written your user-management code, let's see how to use it. You're going to write middleware that extracts a user ID from a cookie:

现在你已经编写好了用户管理代码，让我们来看看如何使用它。你将编写一个中间件，用于从 cookie 中提取用户 ID：

```
// a real implementation would be signed to make
// sure
// the user didn't spoof their identity
func extractUser(req *http.Request) (string,
error) {
    userCookie, err := req.Cookie("identity")
    if err != nil {
        return "", err
    }
    return userCookie.Value, nil
}

func Middleware(h http.Handler) http.Handler {
    return http.HandlerFunc(func(rw
http.ResponseWriter, req *http.Request) {
        user, err := extractUser(req)
        if err != nil {

rw.WriteHeader(http.StatusUnauthorized)
            rw.Write([]byte("unauthorized"))
            return
        }
        ctx := req.Context()
        ctx = ContextWithUser(ctx, user)
        req = req.WithContext(ctx)
        h.ServeHTTP(rw, req)
    })
}
```

```
    })  
}
```

In the middleware, you first get your user value. Next, you extract the context from the request with the `Context` method and create a new context that contains the user with your `ContextWithUser` function. It is idiomatic to reuse the `ctx` variable name when you wrap a context. You then make a new request from the old request and the new context by using the `WithContext` method. Finally, you call the next function in your handler chain with our new request and the supplied `http.ResponseWriter`.

在中间件中，首先获取用户值。接下来，使用 `Context` 方法从请求中提取上下文，并使用 `ContextWithUser` 函数创建一个包含用户的新上下文。在包装上下文时，通常会重用 `ctx` 变量名。然后，使用 `WithContext` 方法，根据旧请求和新上下文发起新请求。最后，使用新请求和提供的 `http.ResponseWriter` 调用处理程序链中的下一个函数。

In most cases, you want to extract the value from the context in your request handler and pass it in to your business logic explicitly. Go functions have explicit parameters, and you shouldn't use the context as a way to sneak values past the API:

大多数情况下，你需要从请求处理程序的上下文中提取值，并将其显式传递给业务逻辑。Go 函数具有显式参数，你不应该使用上下文来绕过 API 传递值：

```
func (c Controller) DoLogic(rw
http.ResponseWriter, req *http.Request) {
    ctx := req.Context()
    user, ok := identity.UserFromContext(ctx)
    if !ok {

rw.WriteHeader(http.StatusInternalServerError)
        return
    }
    data := req.URL.Query().Get("data")
    result, err := c.Logic.BusinessLogic(ctx,
user, data)
    if err != nil {

rw.WriteHeader(http.StatusInternalServerError)
        rw.Write([]byte(err.Error()))
        return
    }
    rw.Write([]byte(result))
}
```

Your handler gets the context by using the `Context` method on the request, extracts the user from the context by using the

`UserFromContext` function, and then calls the business logic. This code shows the value of separation of concerns; how the user is loaded is unknown to the `Controller`. A real user-management system could be implemented in middleware and swapped in without changing any of the controller's code.

您的处理程序通过请求中的 `Context` 方法获取上下文，使用 `UserFromContext` 函数从上下文中提取用户，然后调用业务逻辑。这段代码体现了关注点分离的价值；`Controller` 并不知道用户是如何加载的。一个真正的用户管理系统可以在中间件中实现，并且可以替换进去，而无需更改控制器中的任何代码。

The complete code for this example is in the *sample_code/context_user* directory in the [Chapter 14 repository](#).

本示例的完整代码位于第 14 章存储库的 *sample_code/context_user* 目录中。

In some situations, it's better to keep a value in the context. The tracking GUID that was mentioned earlier is one. This information is meant for management of your application; it is not part of your business state. Passing it explicitly through your code adds additional parameters and prevents integration with third-party libraries that do not know about your metainformation. By leaving a tracking GUID in the context, it passes invisibly through business logic that doesn't need to know about tracking

and is available when your program writes a log message or connects to another server.

在某些情况下，最好将值保留在上下文中。前面提到的跟踪 GUID 就是其中之一。此信息用于应用程序管理，并非业务状态的一部分。在代码中显式传递此信息会增加额外的参数，并妨碍与不了解元信息的第三方库集成。通过将跟踪 GUID 保留在上下文中，它可以隐式地传递给不需要了解跟踪信息的业务逻辑，并在程序写入日志消息或连接到其他服务器时可用。

Here is a simple context-aware GUID implementation that tracks from service to service and creates logs with the GUID included:

这是一个简单的上下文感知 GUID 实现，它可以跟踪服务之间的交互，并创建包含 GUID 的日志：

```
package tracker

import (
    "context"
    "fmt"
    "net/http"

    "github.com/google/uuid"
)

type guidKey int
```

```
const key guidKey = 1

func contextWithGUID(ctx context.Context, guid
string) context.Context {
    return ctx.WithValue(key, guid)
}

func guidFromContext(ctx context.Context) (string,
bool) {
    g, ok := ctx.Value(key).(string)
    return g, ok
}

func Middleware(h http.Handler) http.Handler {
    return http.HandlerFunc(func(rw
http.ResponseWriter, req *http.Request) {
        ctx := req.Context()
        if guid := req.Header.Get("X-GUID"); guid
!= "" {
            ctx = contextWithGUID(ctx, guid)
        } else {
            ctx = contextWithGUID(ctx,
uuid.New().String())
        }
        req = req.WithContext(ctx)
        h.ServeHTTP(rw, req)
    })
}
```

```

}

type Logger struct{}

func (Logger) Log(ctx context.Context, message
string) {
    if guid, ok := guidFromContext(ctx); ok {
        message = fmt.Sprintf("GUID: %s - %s",
guid, message)
    }
    // do logging
    fmt.Println(message)
}

func Request(req *http.Request) *http.Request {
    ctx := req.Context()
    if guid, ok := guidFromContext(ctx); ok {
        req.Header.Add("X-GUID", guid)
    }
    return req
}

```

The **Middleware** function either extracts the GUID from the incoming request or generates a new GUID. In both cases, it places the GUID into the context, creates a new request with the updated context, and continues the call chain.

`Middleware` 函数要么从传入的请求中提取 GUID，要么生成一个新的 GUID。无论哪种情况，它都会将 GUID 放入上下文中，使用更新后的上下文创建一个新请求，并继续调用链。

Next you see how this GUID is used. The `Logger` struct provides a generic logging method that takes in a context and a string. If there's a GUID in the context, it appends it to the beginning of the log message and outputs it. The `Request` function is used when this service makes a call to another service. It takes in an `*http.Request`, adds a header with the GUID if it exists in the context, and returns the `*http.Request`.

接下来您将看到 GUID 的用法。`Logger` 结构体提供了一个通用的日志记录方法，该方法接受一个上下文和一个字符串作为参数。如果上下文中存在 GUID，它会将 GUID 添加到日志消息的开头并输出。当此服务调用另一个服务时，会使用 `Request` 函数。它接受一个 `*http.Request` 参数，如果上下文中存在 GUID，则将其添加到包含该 GUID 的标头中，然后返回 `*http.Request` 参数。

Once you have this package, you can use the dependency injection techniques that I discussed in [“Implicit Interfaces Make Dependency Injection Easier”](#) to create business logic that is completely unaware of any tracking information. First, you declare an interface to represent your logger, a function type to represent a request decorator, and a business logic struct that depends on them:

有了这份包之后，你就可以使用我在“隐式接口让依赖注入更简单”一文中讨论的依赖注入技术，创建完全无需任何跟踪信息的业务逻辑。首先，你需要声明一个接口来表示你的日志记录器，一个函数类型来表示请求装饰器，以及一个依赖于它们的业务逻辑结构体：

```
type Logger interface {
    Log(context.Context, string)
}

type RequestDecorator func(*http.Request)
                        *http.Request

type LogicImpl struct {
    RequestDecorator RequestDecorator
    Logger            Logger
    Remote            string
}
```

Next, you implement your business logic:

接下来，您需要实现业务逻辑：

```
func (l LogicImpl) Process(ctx context.Context,
data string) (string, error) {
    l.Logger.Log(ctx, "starting Process with
"+data)
    req, err := http.NewRequestWithContext(ctx,
```

```

        http.MethodGet, l.Remote+"/second?
query="+data, nil)
    if err != nil {
        l.Logger.Log(ctx, "error building remote
request:"+err.Error())
        return "", err
    }
    req = l.RequestDecorator(req)
    resp, err := http.DefaultClient.Do(req)
    // process the response...
}

```

The GUID is passed through to the logger and the request decorator without the business logic being aware of it, separating the data needed for program logic from the data needed for program management. The only place that's aware of the association is the code in `main` that wires up your dependencies:

GUID 会传递给日志记录器和请求装饰器，而业务逻辑对此毫不知情，从而将程序逻辑所需的数据与程序管理所需的数据分离。唯一了解这种关联的地方是 `main` 中连接依赖项的代码：

```

controller := Controller{
    Logic: LogicImpl{
        RequestDecorator: tracker.Request,
        Logger:            tracker.Logger{}},
}

```

```
        Remote: "http://localhost:4000",  
    },  
}
```

You can find the complete code for the GUID tracker in the *sample_code/context_guid* directory in the [Chapter 14 repository](#).

您可以在第 14 章存储库的 *sample_code/context_guid* 目录中找到 GUID 跟踪器的完整代码。

TIP

提示

Use the context to pass values through standard APIs. Copy values from the context into explicit parameters when they are needed for processing business logic. System maintenance information can be accessed directly from the context.

使用上下文通过标准 API 传递值。当需要处理业务逻辑时，将上下文中的值复制到显式参数中。可以直接从上下文访问系统维护信息。

Cancellation

消除

While context values are useful for passing metadata and working around the limitations of Go's HTTP API, the context has a second use. The context also allows you to control the responsiveness of your application and coordinate concurrent goroutines. Let's see how.

虽然上下文值对于传递元数据和绕过 Go HTTP API 的限制非常有用，但它还有第二个用途。上下文还可以让你控制应用程序的响应速度并协调并发的 goroutine。让我们来看看它是如何实现的。

I discussed this briefly in [“Use the Context to Terminate Goroutines”](#).

Imagine that you have a request that launches several goroutines, each one calling a different HTTP service. If one service returns an error that prevents you from returning a valid result, there is no point in continuing to process the other goroutines. In Go, this is called *cancellation*, and the context provides the mechanism for its implementation.

我在“使用上下文终止 Goroutine”一文中简要讨论过这个问题。假设你有一个请求，它会启动多个 Goroutine，每个 Goroutine 调用不同的 HTTP 服务。如果某个服务返回错误，导致你无法返回有效结果，那

么继续处理其他 Goroutine 就毫无意义了。在 Go 语言中，这被称为取消（cancellation），而上下文则提供了实现取消的机制。

To create a cancellable context, use the `context.WithCancel` function. It takes in a `context.Context` as a parameter and returns a `context.Context` and a `context.CancelFunc`. Just like `context.WithValue`, the returned `context.Context` is a child context of the context that was passed into the function. A `context.CancelFunc` is a function that takes no parameters and *Cancels* the context, telling all the code that's listening for potential cancellation that it's time to stop processing.

要创建可取消的上下文，请使用 `context.WithCancel` 函数。它接收一个 `context.Context` 作为参数，并返回一个 `context.Context` 和一个 `context.CancelFunc`。与 `context.WithValue` 类似，返回的 `context.Context` 是传递给该函数的上下文的子上下文。`context.CancelFunc` 是一个不接受任何参数的函数，它会取消上下文，并通知所有监听潜在取消事件的代码停止处理。

Anytime you create a context that has an associated cancel function, you *must* call that cancel function when you are done processing, whether or not your processing ends in an error. If you do not, your program will leak resources (memory and goroutines) and eventually slow down or crash. No

error occurs if you call the cancel function more than once; any invocation after the first does nothing.

每当您创建了一个带有关联取消函数的上下文时，无论处理过程是否以错误结束，都必须在处理完成后调用该取消函数。否则，您的程序将发生资源泄漏（内存和 goroutine），最终导致运行速度变慢甚至崩溃。多次调用取消函数不会产生任何错误；第一次调用之后的任何调用都不会执行任何操作。

The easiest way to make sure you call the cancel function is to use `defer` to invoke it right after the cancel function is returned:

确保调用取消函数的最简单方法是在取消函数返回后立即使用 `defer` 来调用它：

```
ctx, cancelFunc :=  
context.WithCancel(context.Background())  
defer cancelFunc()
```

This leads to the question, how do you detect cancellation? The `context.Context` interface has a method called `Done`. It returns a channel of type `struct{}`. (The reason this is the chosen return type is that an empty struct uses no memory.) This channel is closed when the `cancel` function is invoked. Remember, a closed channel always immediately returns its zero value when you attempt to read it.

这就引出了一个问题：如何检测取消操作？`context.Context` 接口有一个名为 `Done` 的方法。它返回一个 `struct{}` 类型的通道。

（之所以选择这种返回类型，是因为空结构体不占用内存。）当调用 `cancel` 函数时，该通道会被关闭。请记住，尝试读取已关闭的通道时，它总是立即返回零值。

WARNING

警告

If you call `Done` on a context that isn't cancellable, it returns `nil`. As was covered in [“Turn Off a case in a select”](#), a read from a `nil` channel never returns. If this is not done inside a `case` in a `select` statement, your program will hang.

如果对不可取消的上下文调用 `Done`，它会返回 `nil`。正如“关闭 select 语句中的 case”一文中所述，从 `nil` 通道读取数据永远不会返回。如果此操作未在 `select` 语句中的 `case` 内执行，则程序将挂起。

Let's take a look at how this works. Let's say you have a program that's gathering data from a number of HTTP endpoints. If any one of them fails, you want to end processing across all of them. Context cancellation allows you to do that.

我们来看看它是如何工作的。假设你有一个程序，它从多个 HTTP 端点收集数据。如果其中任何一个端点出现故障，你都希望停止所有端点的处理。上下文取消功能可以让你做到这一点。

NOTE

笔记

In this example, you are going to take advantage of a great service called httpbin.org. You send it HTTP or HTTPS requests to test how your application responds to a variety of situations. You'll use two of its endpoints: one that delays response for a specified number of seconds, and one that will return one of the status codes that you send it.

在这个示例中，您将使用一个名为 httpbin.org 的出色服务。您将向其发送 HTTP 或 HTTPS 请求，以测试您的应用程序在各种情况下的响应。您将使用其两个端点：一个端点会将响应延迟指定的秒数，另一个端点会返回您发送的状态码之一。

First, create your cancellable context, a channel to get back data from your goroutines, and a `sync.WaitGroup` to allow you to wait until all goroutines have completed:

首先，创建可取消上下文、用于从 goroutine 获取数据的通道，以及 `sync.WaitGroup`，以便您可以等待所有 goroutine 完成：

```
ctx, cancelFunc :=
context.WithCancel(context.Background())
defer cancelFunc()
ch := make(chan string)
var wg sync.WaitGroup
wg.Add(2)
```

Next, you launch two goroutines, one that calls a URL that randomly returns a bad status, and the other that sends a canned JSON response after

a delay. First the random status goroutine:

接下来，启动两个 goroutine，一个调用一个 URL 随机返回错误状态码，另一个在延迟一段时间后发送预设的 JSON 响应。首先是随机状态码 goroutine：

```
go func() {
    defer wg.Done()
    for {
        // return one of these status code at
random
        resp, err := makeRequest(ctx,

"http://httpbin.org/status/200,200,200,500")
        if err != nil {
            fmt.Println("error in status
goroutine:", err)
            cancelFunc()
            return
        }
        if resp.StatusCode ==
http.StatusInternalServerError {
            fmt.Println("bad status, exiting")
            cancelFunc()
            return
        }
        select {
        case ch <- "success from status":
```

```

        case <- ctx.Done():
    }
    time.Sleep(1 * time.Second)
}
}()

```

The `makeRequest` function is a helper to make an HTTP request using the supplied context and URL. If you get back an OK status, you write a message to the channel and sleep for a second. When an error occurs or you get back a bad status code, you call `cancelFunc` and exit the goroutine.

`makeRequest` 函数是一个辅助函数，用于使用提供的上下文和 URL 发起 HTTP 请求。如果收到 OK 状态码，则向通道写入一条消息并休眠一秒钟。如果发生错误或收到错误状态码，则调用 `cancelFunc` 并退出 goroutine。

The delay goroutine is similar:

延迟 goroutine 类似：

```

go func() {
    defer wg.Done()
    for {
        // return after a 1 second delay
        resp, err := makeRequest(ctx,
            "http://httpbin.org/delay/1")
    }
}()

```

```

        if err != nil {
            fmt.Println("error in delay
goroutine:", err)
            cancelFunc()
            return
        }
        select {
        case ch <- "success from delay: " +
resp.Header.Get("date"):
        case <-ctx.Done():
        }
    }
}()

```

Finally, you use the for/select pattern to read data from the channel written to by the goroutines and wait for cancellation to happen:

最后，使用 for/select 模式从 goroutine 写入的通道中读取数据，并等待取消操作发生：

```

loop:
    for {
        select {
        case s := <-ch:
            fmt.Println("in main:", s)
        case <-ctx.Done():
            fmt.Println("in main: cancelled!")
        }
    }
}()

```



```
        break loop
    }
}
wg.Wait()
```

In your `select` statement, you have two cases. One reads from the message channel, and the other waits for the done channel to be closed. When it closes, you exit the loop and wait for the goroutines to exit. You can find this program in the *sample_code/cancel_http* directory in the [Chapter 14 repository](#).

在你的 `select` 语句中，你有两个 case。一个用于从消息通道读取数据，另一个用于等待 done 通道关闭。当 done 通道关闭时，你退出循环并等待 goroutine 退出。你可以在第 14 章代码库的 *sample_code/cancel_http* 目录中找到这个程序。

Here's what happens when you run the code (the results are random, so go ahead and run it a few times to see different results):

以下是运行代码后发生的情况（结果是随机的，所以请多运行几次以查看不同的结果）：

```
in main: success from status
in main: success from delay: Thu, 16 Feb 2023
03:53:57 GMT
in main: success from status
```

```
in main: success from delay: Thu, 16 Feb 2023
03:53:58 GMT
bad status, exiting
in main: cancelled!
error in delay goroutine: Get
"http://httpbin.org/delay/1": context canceled
```

There are some interesting things to note. First, you are calling `cancelFunc` multiple times. As mentioned earlier, this is perfectly fine and causes no problems. Next, notice that you got an error from the delay goroutine after cancellation was triggered. This is because the built-in HTTP client in the Go standard library respects cancellation. You created the request using the cancellable context, and when it was cancelled, the request ended. This triggers the error path in the goroutine and makes sure that it does not leak.

有几点值得注意。首先，你多次调用了 `cancelFunc`。如前所述，这完全没问题，不会造成任何问题。其次，请注意，在取消操作触发后，延迟 goroutine 抛出了一个错误。这是因为 Go 标准库中的内置 HTTP 客户端支持取消操作。你使用可取消上下文创建了请求，当请求被取消时，请求就结束了。这会触发 goroutine 中的错误处理流程，从而确保不会发生内存泄漏。

You might wonder about the error that caused the cancellation and how you can report it. An alternate version of `WithCancel`, called

`WithCancelCause` , returns a cancellation function that takes in an error as a parameter. The `Cause` function in the `context` package returns the error that was passed into the first invocation of the cancellation function.

您可能想知道导致取消的错误是什么，以及如何报告该错误。

`WithCancel` 的一个替代版本，称为 `WithCancelCause` , 返回一个取消函数，该函数接受一个错误作为参数。`context` 包中的 `Cause` 函数返回传递给第一次调用取消函数的错误。

NOTE

笔记

`Cause` is a function in the `context` package instead of a method on `context.Context` because the ability to return errors via cancellation was added to the `context` package in Go 1.20, long after the context was initially introduced. If a new method was added to the `context.Context` interface, this would have broken any third-party code that implemented it. Another option would be to define a new interface that included this method, but existing code already passes `context.Context` everywhere, and converting to a new interface with a `Cause` method would require type assertions or type switches. Adding a function is the simplest approach. There are several ways to evolve your APIs over time. You should pick the one that has the least impact on your users.

`Cause` 是 `context` 包中的一个函数，而不是 `context.Context` 上的方法，因为通过取消操作返回错误的能力是在 Go 1.20 中添加到 `context` 包的，而此时上下文早已被引入。如果向 `context.Context` 接口添加一个新方法，将会破坏任何实现了该方法的第三方代码。另一种选择是定义一个包含此方法的新接口，但现有代码已经到处传递 `context.Context` , 而转换为包含 `Cause` 方法的新接口需要类型断言或类型切换。添加一个函数是最简单的方法。随着时间的推移，有多种方式可以演进您的 API。您应该选择对用户影响最小的方式。

Let's rewrite the program to capture the error. First, you change your context creation:

让我们重写程序以捕获错误。首先，您需要更改上下文的创建方式：

```
ctx, cancelFunc :=
context.WithCancelCause(context.Background())
defer cancelFunc(nil)
```

Next, you are going to make slight modifications to your two goroutines. The body of the `for` loop in the status goroutine now looks like this:

接下来，你需要对两个 goroutine 进行一些修改。状态 goroutine 中的 `for` 循环体现在看起来像这样：

```
resp, err := makeRequest(ctx,
"http://httpbin.org/status/200,200,200,500")
if err != nil {
    cancelFunc(fmt.Errorf("in status goroutine:
%w", err))
    return
}
if resp.StatusCode ==
http.StatusInternalServerError {
    cancelFunc(errors.New("bad status"))
    return
}
```

```
ch <- "success from status"  
time.Sleep(1 * time.Second)
```

You've removed the `fmt.Println` statements and pass errors to `cancelFunc`. The body of the `for` loop in the delay goroutine now looks like this:

您已移除 `fmt.Println` 语句并将错误传递给 `cancelFunc`。延迟 goroutine 中的 `for` 循环体现在如下所示：

```
resp, err := makeRequest(ctx,  
"http://httpbin.org/delay/1")  
if err != nil {  
    fmt.Println("in delay goroutine:", err)  
    cancelFunc(fmt.Errorf("in delay goroutine:  
%w", err))  
    return  
}  
ch <- "success from delay: " +  
resp.Header.Get("date")
```

The `fmt.Println` is still there so you can show that the error is still generated and passed to `cancelFunc`.

`fmt.Println` 仍然存在，因此您可以证明错误仍然生成并传递给 `cancelFunc`。

Finally, you use `context.Cause` to print out the error both on initial cancellation and after you finish waiting for your goroutines to complete:

最后，您可以使用 `context.Cause` 在初始取消时以及等待 goroutine 完成后打印出错误信息：

```
loop:
    for {
        select {
            case s := <-ch:
                fmt.Println("in main:", s)
            case <-ctx.Done():
                fmt.Println("in main: cancelled with
error", context.Cause(ctx))
                break loop
        }
    }
    wg.Wait()
    fmt.Println("context cause:",
context.Cause(ctx))
```

You can find this code in the *sample_code/cancel_error_http* directory in the [Chapter 14 repository](#).

您可以在第 14 章存储库的 `sample_code/cancel_error_http` 目录中找到此代码。

Running your new program produces this output:

运行新程序后，将输出以下内容：

```
in main: success from status
in main: success from delay: Thu, 16 Feb 2023
04:11:49 GMT
in main: cancelled with error bad status
in delay goroutine: Get
"http://httpbin.org/delay/1": context canceled
context cause: bad status
```

You see that the error from the status goroutine is printed out both when cancellation is initially detected in the `switch` statement and after you finish waiting for the delay goroutine to complete. Notice that the delay goroutine called `cancelFunc` with an error, but that error doesn't overwrite the initial cancellation error.

您可以看到，状态 goroutine 的错误信息会在 `switch` 语句中首次检测到取消操作时以及延迟 goroutine 执行完毕后打印出来。请注意，延迟 goroutine 调用 `cancelFunc` 时抛出了一个错误，但该错误并没有覆盖最初的取消错误。

Manual cancellation is very useful when your code reaches a logical state that ends processing. Sometimes you want to cancel because a task is taking too long. In that case, you use a timer.

当代码达到逻辑状态并终止处理时，手动取消非常有用。有时，由于任务耗时过长，您可能需要取消任务。在这种情况下，您可以使用定时器。

Contexts with Deadlines

有截止日期的情境

One of the most important jobs for a server is managing requests. A novice programmer often thinks that a server should take as many requests as it possibly can and work on them for as long as it can until it returns a result for each client.

服务器最重要的任务之一就是管理请求。新手程序员常常认为服务器应该尽可能多地接收请求，并尽可能长时间地处理这些请求，直到为每个客户端返回结果为止。

The problem is that this approach does not scale. A server is a shared resource. Like all shared resources, each user wants to get as much as they can out of it and isn't terribly concerned with the needs of other users. It's the responsibility of the shared resource to manage itself so that it provides a fair amount of time to all its users.

问题在于这种方法无法扩展。服务器是一种共享资源。与所有共享资源一样，每个用户都希望尽可能多地利用它，而不太关心其他用户的需求。共享资源本身有责任进行自我管理，从而为所有用户提供公平的可用时间。

Generally, a server can do four things to manage its load:

一般来说，服务器可以通过以下四种方式来管理其负载：

- Limit simultaneous requests
限制同时请求数
- Limit the number of queued requests waiting to run
限制等待运行的排队请求数量
- Limit the amount of time a request can run
限制请求的运行时间。
- Limit the resources a request can use (such as memory or disk space)
限制请求可以使用的资源（例如内存或磁盘空间）

Go provides tools to handle the first three. You saw how to handle the first two when learning about concurrency in [Chapter 12](#). By limiting the number of goroutines, a server manages simultaneous load. The size of the waiting queue is handled via buffered channels.

Go 提供了处理前三种情况的工具。你在第 12 章学习并发时已经了解了如何处理前两种情况。服务器通过限制 goroutine 的数量来管理并发负载。等待队列的大小则通过缓冲通道来控制。

The context provides a way to control how long a request runs. When building an application, you should have an idea of your performance envelope: how long you have for your request to complete before the user has an unsatisfactory experience. If you know the maximum amount of time that a request can run, you can enforce it using the context.

上下文提供了一种控制请求运行时间的方法。在构建应用程序时，您应该了解性能范围：即在用户体验不佳之前，请求必须运行多长时间才能完成。如果您知道请求可以运行的最长时间，则可以使用上下文来强制执行该时间。

NOTE

笔记

While `GOMEMLIMIT` provides a soft way to limit the amount of memory a Go program uses, if you want to enforce constraints on memory or disk space that a single request uses, you'll have to write the code to manage that yourself. Discussion of this topic is beyond the scope of this book.

虽然 `GOMEMLIMIT` 提供了一种限制 Go 程序内存使用量的简便方法，但如果您想强制限制单个请求使用的内存或磁盘空间，则需要自行编写代码进行管理。本书不对此主题进行深入探讨。

You can use one of two functions to create a time-limited context. The first is `context.WithTimeout`. It takes two parameters: an existing context and `time.Duration` that specifies the duration until the context automatically cancels. It returns a context that automatically triggers a

cancellation after the specified duration as well as a cancellation function that is invoked to cancel the context immediately.

您可以使用以下两个函数之一来创建有时限的上下文。第一个函数是 `context.WithTimeout`。它接受两个参数：一个现有上下文和 `time.Duration`，后者指定上下文自动取消前的持续时间。它返回一个上下文，该上下文会在指定的持续时间结束后自动触发取消，同时还会返回一个取消函数，该函数会立即调用以取消上下文。

The second function is `context.WithDeadline`. This function takes in an existing context and a `time.Time` that specifies the time when the context is automatically canceled. Like `context.WithTimeout`, it returns a context that automatically triggers a cancellation after the specified time has elapsed as well as a cancellation function.

第二个函数是 `context.WithDeadline`。此函数接收一个现有上下文和一个 `time.Time`，后者指定上下文自动取消的时间。与 `context.WithTimeout` 类似，它返回一个上下文，该上下文会在指定时间过后自动触发取消操作，同时还返回一个取消函数。

TIP

提示

If you pass a time in the past to `context.WithDeadline`, the context is created already canceled.

如果将过去的某个时间传递给 `context.WithDeadline`，则上下文已创建并取消。

Just as with the cancellation function returned from `context.WithCancel` and `context.WithCancelCause`, you must make sure that the cancellation function returned by `context.WithTimeout` and `context.WithDeadline` is invoked at least once.

与 `context.WithCancel` 和 `context.WithCancelCause` 返回的取消函数一样，您必须确保 `context.WithTimeout` 和 `context.WithDeadline` 返回的取消函数至少被调用一次。

If you want to find out when a context will automatically cancel, use the `Deadline` method on `context.Context`. It returns a `time.Time` that indicates the time and a `bool` that indicates if a timeout was set. This mirrors the comma ok idiom you use when reading from maps or channels.

如果想知道某个上下文何时会自动取消，请在 `context.Context` 上使用 `Deadline` 方法。它会返回一个 `time.Time`，指示时间，以及一个 `bool`，指示是否设置了超时。这类似于从地图或频道读取数据时使用的逗号 ok 用法。

When you set a time limit for the overall duration of the request, you might want to subdivide that time. And if you call another service from your service, you might want to limit how long you allow the network call to run, reserving some time for the rest of your processing or for other network calls. You control how long an individual call takes by creating a

child context that wraps a parent context by using `context.WithTimeout` or `context.WithDeadline`.

设置请求总持续时间限制时，您可能需要将该时间细分。如果您从服务调用其他服务，则可能需要限制网络调用的运行时间，以便为后续处理或其他网络调用预留时间。您可以通过创建子上下文来控制单个调用的持续时间，该子上下文使用 `context.WithTimeout` 或 `context.WithDeadline` 来包装父上下文。

Any timeout that you set on the child context is bounded by the timeout set on the parent context; if a parent context times out in 2 seconds, you can declare that a child context times out in 3 seconds, but when the parent context times out after 2 seconds, so will the child.

您在子上下文中设置的任何超时时间都受父上下文中设置的超时时间的限制；如果父上下文在 2 秒后超时，您可以声明子上下文在 3 秒后超时，但是当父上下文在 2 秒后超时时，子上下文也会超时。

You can see this with a simple program:

你可以通过一个简单的程序来验证这一点：

```
ctx := context.Background()
parent, cancel := context.WithTimeout(ctx,
2*time.Second)
defer cancel()
```

```
child, cancel2 := context.WithTimeout(parent,
3*time.Second)
defer cancel2()
start := time.Now()
<-child.Done()
end := time.Now()
fmt.Println(end.Sub(start).Truncate(time.Second))
```

In this sample, you specify a 2-second timeout on the parent context and a 3-second timeout on the child context. You then wait for the child context to complete by waiting on the channel returned from the `Done` method on the child `context.Context`. I'll talk more about the `Done` method in the next section.

在这个示例中，您为父上下文指定了 2 秒的超时时间，为子上下文指定了 3 秒的超时时间。然后，您通过等待子上下文 `context.Context` 的 `Done` 方法返回的通道来等待子上下文完成。我将在下一节中详细介绍 `Done` 方法。

You can find this code in the *sample_code/nested_timers* directory in the [Chapter 14 repository](#) or run this code on [The Go Playground](#). You'll see the following result:

您可以在第 14 章代码库的 `sample_code/nested_timers` 目录中找到这段代码，或者在 Go Playground 上运行它。您将看到以下结果：

Since contexts with timers can cancel because of a timeout or an explicit call to the cancellation function, the context API provides a way to tell what caused cancellation. The `Err` method returns `nil` if the context is still active, or it returns one of two sentinel errors if the context has been canceled: `context.Canceled` or `context.DeadlineExceeded`. The first is returned after explicit cancellation, and the second is returned when a timeout triggered cancellation.

由于带有定时器的上下文可能会因超时或显式调用取消函数而取消，上下文 API 提供了一种方法来判断取消的原因。如果上下文仍然处于活动状态，则 `Err` 方法返回 `nil`；如果上下文已被取消，则返回以下两个哨兵错误之一：`context.Canceled` 或 `context.DeadlineExceeded`。第一个错误在显式取消后返回，第二个错误在超时触发取消时返回。

Let's see them in use. You're going to make one more change to your `httpbin` program. This time, you're adding a timeout to the context that's used to control how long the goroutines run:

让我们看看它们是如何应用的。你还要对 `httpbin` 程序做最后一处修改。这次，你要在用于控制 goroutine 运行时间的上下文中添加一个超时：

```
ctx, cancelFuncParent :=
context.WithTimeout(context.Background(),
3*time.Second)
defer cancelFuncParent()
ctx, cancelFunc := context.WithCancelCause(ctx)
defer cancelFunc(nil)
```

NOTE

笔记

If you want the option of returning the error for the cancellation cause, you need to wrap a context created by `WithTimeout` or `WithDeadline` in a context created by `WithCancelCause`. You must `defer` both cancellation functions to keep resources from being leaked. If you want to return a custom sentinel error when a context times out, use the `context.WithTimeoutCause` or `context.WithDeadlineCause` functions instead.

如果您希望返回取消原因的错误信息，则需要将由 `WithTimeout` 或 `WithDeadline` 创建的上下文包装在由 `WithCancelCause` 创建的上下文中。您必须 `defer` 这两个取消函数，以防止资源泄漏。如果您希望在上下文超时返回自定义哨兵错误，请改用 `context.WithTimeoutCause` 或 `context.WithDeadlineCause` 函数。

Now your program will exit if a 500 status code is returned or if you don't get a 500 status code within 3 seconds. The only other change you are making to the program is to print out the value returned by `Err` when cancellation happens:

现在，如果返回 500 状态码，或者在 3 秒内没有收到 500 状态码，程序将退出。您对程序所做的唯一其他更改是，在取消操作发生时，打印出 `Err` 返回的值：

```
fmt.Println("in main: cancelled with cause:",
context.Cause(ctx),
    "err:", ctx.Err())
```

You can find the code in the *sample_code/timeout_error_http* directory in the [Chapter 14 repository](#).

您可以在第 14 章存储库的 *sample_code/timeout_error_http* 目录中找到代码。

The results are random, so run the program multiple times to see different results. If you run the program and hit the timeout, you'll get output like this:

结果是随机的，所以请多次运行程序以查看不同的结果。如果程序运行超时，您将得到类似这样的输出：

```
in main: success from status
in main: success from delay: Sun, 19 Feb 2023
04:36:44 GMT
in main: success from status
in main: success from status
```

```
in main: success from delay: Sun, 19 Feb 2023
04:36:45 GMT
in main: cancelled with cause: context deadline
exceeded
    err: context deadline exceeded
in delay goroutine: Get
"http://httpbin.org/delay/1":
    context deadline exceeded
context cause: context deadline exceeded
```

Notice that the error returned by `context.Cause` is the same error that's returned by the `Err` method: `context.DeadlineExceeded`.

请注意，`context.Cause` 返回的错误与 `Err` 方法返回的错误相同：`context.DeadlineExceeded`。

If the status error happens within 3 seconds, you'll get output like this:

如果状态错误发生在 3 秒内，您将看到如下输出：

```
in main: success from status
in main: success from status
in main: success from delay: Sun, 19 Feb 2023
04:37:14 GMT
in main: cancelled with cause: bad status err:
context canceled
in delay goroutine: Get
```

```
"http://httpbin.org/delay/1": context canceled  
context cause: bad status
```

Now the error returned by `context.Cause` is `bad status`, but `Err` returns a `context.Canceled` error.

现在 `context.Cause` 返回的错误是 `bad status`，但 `Err` 返回的错误是 `context.Canceled`。

Context Cancellation in Your Own Code

在您自己的代码中取消上下文

Most of the time, you don't need to worry about timeouts or cancellation within your own code; it simply doesn't run for long enough. Whenever you call another HTTP service or the database, you should pass along the context; those libraries properly handle cancellation via the context.

大多数情况下，你无需担心代码中的超时或取消问题；因为代码运行时间通常不会太长。当你调用其他 HTTP 服务或数据库时，应该传递上下文；这些库会通过上下文正确处理取消操作。

You should think about handling cancellation for two situations. The first is when you have a function that reads or writes channels by using a `select`

statement. As shown in [“Cancellation”](#), include a `case` that checks the channel returned by the `Done` method on the context. This allows your function to exit upon context cancellation, even if the goroutines do not handle cancellation properly.

你应该考虑处理两种情况下的取消操作。第一种情况是你的函数使用 `select` 语句读取或写入通道。如“取消”部分所示，添加一个 `case` 语句，用于检查上下文中 `Done` 方法返回的通道。这样，即使 goroutine 没有正确处理取消操作，你的函数也能在上下文取消时退出。

The second situation is when you write code that runs long enough that it should be interrupted by a context cancellation. In that case, check the status of the context periodically using `context.Cause`. The `context.Cause` function returns an error if the context has been cancelled.

第二种情况是，当你编写的代码运行时间过长，以至于需要通过取消上下文来中断时。在这种情况下，请使用 `context.Cause` 定期检查上下文的状态。如果上下文已被取消，则 `context.Cause` 函数会返回错误。

Here’s the pattern for supporting context cancellation in your code:

以下是在代码中支持上下文取消的模式：

```

func longRunningComputation(ctx context.Context,
data string) (string, error) {
    for {
        // do some processing
        // insert this if statement periodically
        // to check if the context has been
cancelled
        if err := context.Cause(ctx); err != nil {
            // return a partial value if it makes
sense,
            // or a default one if it doesn't
            return "", err
        }
        // do some more processing and loop again
    }
}

```

Here's an example loop from a function that calculates π by using the inefficient Leibniz algorithm. Using context cancellation allows you to control how long it can run:

以下是一个使用效率较低的莱布尼茨算法计算 π 的函数中的循环示例。使用上下文取消功能可以控制循环的运行时间：

```

i := 0
for {
    if err := context.Cause(ctx); err != nil {

```

```

        fmt.Println("cancelled after", i,
"iterations")
        return sum.Text('g', 100), err
    }
    var diff big.Float
    diff.SetInt64(4)
    diff.Quo(&diff, &d)
    if i%2 == 0 {
        sum.Add(&sum, &diff)
    } else {
        sum.Sub(&sum, &diff)
    }
    d.Add(&d, two)
    i++
}

```

You can see the complete sample program that demonstrates this pattern in the *sample_code/own_cancellation* directory in the [Chapter 14 repository](#).

您可以在第 14 章存储库的 *sample_code/own_cancellation* 目录中查看演示此模式的完整示例程序。

Exercises

练习

Now that you've seen how to use the context, try to implement these exercises. All the answers are found in the [Chapter 14 repository](#).

既然你已经了解了如何使用上下文，请尝试完成以下练习。所有答案都可以在第 14 章的题库中找到。

1. Create a middleware-generating function that creates a context with a timeout. The function should have one parameter, which is the number of milliseconds that a request is allowed to run. It should return a `func(http.Handler) http.Handler`.

创建一个中间件生成函数，该函数创建一个带有超时设置的上下文。该函数应有一个参数，即请求允许运行的毫秒数。它应返回一个 `func(http.Handler) http.Handler`。

2. Write a program that adds randomly generated numbers between 0 (inclusive) and 100,000,000 (exclusive) together until one of two things happen: the number 1234 is generated or 2 seconds has passed. Print out the sum, the number of iterations, and the reason for ending (timeout or number reached).

编写一个程序，将 0（含 0）到 100,000,000（不含 100,000）之间随机生成的数字相加，直到发生以下两种情况之一：生成数字

1234 或 2 秒已过。打印出总和、迭代次数以及结束原因（超时或达到指定数字）。

3. Assume you have a simple logging function that looks like this:

假设你有一个简单的日志记录函数，如下所示：

```
func Log(ctx context.Context, level Level,
message string) {
    var inLevel Level
    // TODO get a logging level out of the
context and assign it to inLevel
    if level == Debug && inLevel == Debug {
        fmt.Println(message)
    }
    if level == Info && (inLevel == Debug ||
inLevel == Info) {
        fmt.Println(message)
    }
}
```

Define a type called `Level` whose underlying type is `string`.

Define two constants of this type, `Debug` and `Info`, and set them to `"debug"` and `"info"`, respectively.

定义一个名为 `Level` 的类型，其底层类型为 `string`。定义该类型的两个常量，`Debug` 和 `Info`，并分别将它们设置为 `"debug"` 和 `"info"`。

Create functions to store the log level in the context and to extract it.

创建函数以在上下文中存储日志级别并提取它。

Create a middleware function to get the logging level from a query parameter called `log_level`. The valid values for `log_level` are `debug` and `info`.

创建一个中间件函数，从名为 `log_level` 的查询参数中获取日志级别。`log_level` 的有效值为 `debug` 和 `info`。

Finally, fill in the `TODO` in `Log` to properly extract the log level from the context. If the log level is not assigned or is not a valid value, nothing should be printed.

最后，在 `Log` 中填写 `TODO`，以便从上下文中正确提取日志级别。如果未分配日志级别或日志级别值无效，则不应打印任何内容。

Wrapping Up

总结

In this chapter, you learned how to manage request metadata by using the context. You can now set timeouts, perform explicit cancellation, pass values through the context, and know when you should do each of these things. In the next chapter, you're going to see Go's built-in testing framework and learn how to use it to find bugs and diagnose performance problems in your programs.

在本章中，你学习了如何使用上下文来管理请求元数据。现在你可以设置超时、显式取消请求、通过上下文传递值，并且知道何时应该执行这些操作。在下一章中，你将了解 Go 的内置测试框架，并学习如何使用它来查找程序中的错误并诊断性能问题。

Chapter 15. Writing Tests

第十五章 编写测试题

Since the 2000s, the widespread adoption of automated testing has probably done more to improve code quality than any other software engineering technique. As a language and ecosystem focused on improving software quality, it's not surprising that Go includes testing support as part of its standard library. Go makes it so easy to test your code, there's no excuse to not do it.

自 2000 年以来，自动化测试的广泛应用对提升代码质量的贡献可能超过了任何其他软件工程技术。Go 语言及其生态系统专注于提升软件质量，因此将测试支持纳入其标准库也就不足为奇了。Go 让代码测试变得如此简单，以至于没有任何理由不进行测试。

In this chapter, you'll see how to test Go code, group tests into unit and integration tests, examine code coverage, write benchmarks, and learn how to check code for concurrency issues by using the Go data race detector. Along the way, I'll discuss how to write code that is testable and why this improves our code quality.

在本章中，你将学习如何测试 Go 代码，如何将测试分为单元测试和集成测试，如何检查代码覆盖率，如何编写基准测试，以及如何使用

Go 数据竞争检测器检查代码中的并发问题。同时，我还会讨论如何编写可测试的代码，以及为什么这能提高代码质量。

Understanding the Basics of Testing

了解测试基础知识

Go's testing support has two parts: libraries and tooling. The `testing` package in the standard library provides the types and functions to write tests, while the `go test` tool that's bundled with Go runs your tests and generates reports. Unlike many other languages, Go places its tests in the same directory and the same package as the production code. Since tests are located in the same package, they are able to access and test unexported functions and variables. You'll see in a bit how to write tests that ensure that you are testing only a public API.

Go 的测试支持包含两部分：库和工具。标准库中的 `testing` 包提供了编写测试所需的类型和函数，而 Go 自带的 `go test` 工具则负责运行测试并生成报告。与其他许多语言不同，Go 将测试代码放在与生产代码相同的目录和包中。由于测试代码位于同一包内，因此它们可以访问和测试未导出的函数和变量。稍后您将看到如何编写测试以确保仅测试公共 API。

NOTE

笔记

Complete code samples for this chapter are found in the [Chapter 15 repository](#).

本章的完整代码示例可在第 15 章代码库中找到。

Let's write a simple function and then a test to make sure the function works. In the *sample_code/adder* directory, in the file *adder.go*, you have the following:

我们来编写一个简单的函数，然后编写一个测试来确保该函数能够正常工作。在 *sample_code/adder* 目录下的 *adder.go* 文件中，你会看到以下内容：

```
func addNumbers(x, y int) int {  
    return x + x  
}
```

The corresponding test is in *adder_test.go*:

相应的测试代码位于 *adder_test.go* 文件中：

```
func Test_addNumbers(t *testing.T) {  
    result := addNumbers(2,3)  
    if result != 5 {  
        t.Error("incorrect result: expected 5,
```

```
got", result)
    }
}
```

Every test is written in a file whose name ends with `_test.go`. If you are writing tests against `foo.go`, place your tests in a file named `foo_test.go`.

所有测试都写在文件名以 `_test.go` 结尾的文件中。如果您要针对 `foo.go` 编写测试，请将测试放在名为 `foo_test.go` 的文件中。

Test functions start with the word `Test` and take in a single parameter of type `*testing.T`. By convention, this parameter is named `t`. Test functions do not return any values. The name of the test (apart from starting with the word “Test”) is meant to document what you are testing, so pick something that explains what you are testing. When writing unit tests for individual functions, the convention is to name the unit test `Test` followed by the name of the function. When testing unexported functions, some people use an underscore between the word `Test` and the name of the function.

测试函数以 `Test` 开头，并接受一个类型为 `*testing.T` 的参数。按照惯例，该参数被命名为 `t`。测试函数不返回任何值。测试名称（除了以“Test”开头之外）旨在说明您正在测试的内容，因此请选择能够清晰解释测试目标的名称。为单个函数编写单元测试时，惯例是

将单元测试命名为 `Test`，后跟函数名称。测试未导出的函数时，有些人会在 `Test` 和函数名称之间使用下划线。

Also note that you use standard Go code to call the code being tested and to validate that the responses are as expected. When there's an incorrect result, you report the error with the `t.Error` method, which works like the `fmt.Print` function. You'll see other error-reporting methods in a bit.

另请注意，您需要使用标准的 Go 代码来调用被测代码，并验证响应是否符合预期。如果结果不正确，则使用 `t.Error` 方法报告错误，其工作方式与 `fmt.Print` 函数类似。稍后您将看到其他错误报告方法。

You've just seen the library portion of Go's test support. Now let's take a look at the tooling. Just as `go build` builds a binary and `go run` runs a program, the command `go test` runs the test functions in the current directory:

您刚刚了解了 Go 测试支持的库部分。现在让我们来看看工具集。正如 `go build` 构建二进制文件，`go run` 运行程序一样，命令 `go test` 运行当前目录中的测试函数：

```
$ go test
--- FAIL: Test_addNumbers (0.00s)
    adder_test.go:8: incorrect result: expected 5,
```

```
got 4
FAIL
exit status 1
FAIL    test_examples/adder    0.006s
```

It looks like you found a bug in your code. Taking a second look at `addNumbers`, you see that you are adding `x` to `x`, not `x` to `y`. Let's change the code and rerun the test to verify that the bug is fixed:

看来你的代码中存在一个错误。仔细查看 `addNumbers`，你会发现你将 `x` 添加到了 `x`，而不是将 `x` 添加到 `y`。让我们修改代码并重新运行测试，以验证错误是否已修复：

```
$ go test
PASS
ok      test_examples/adder    0.006s
```

The `go test` command allows you to specify which packages to test. Using `./...` for the package name specifies that you want to run tests in the current directory and all subdirectories of the current directory. Include a `-v` flag to get verbose testing output.

`go test` 命令允许您指定要测试的软件包。使用 `./...` 作为软件包名称，表示您希望在当前目录及其所有子目录中运行测试。添加 `-v`

标志可获取详细的测试输出。

Reporting Test Failures

报告测试失败

There are several methods on `*testing.T` for reporting test failures. You've already seen `Error`, which builds a failure description string out of a comma-separated list of values.

`*testing.T` 上有几种报告测试失败的方法。您已经见过 `Error`，它会根据以逗号分隔的值列表构建失败描述字符串。

If you'd rather use a `Printf`-style formatting string to generate your message, use the `Errorf` method instead:

如果您希望使用 `Printf` 样式的格式字符串来生成消息，请改用 `Errorf` 方法：

```
t.Errorf("incorrect result: expected %d, got %d",  
5, result)
```

While `Error` and `Errorf` mark a test as failed, the test function continues running. If you think a test function should stop processing as soon as a failure is found, use the `Fatal` and `Fatalf` methods. The `Fatal` method works like `Error`, and the `Fatalf` method works like

`Errorf` . The difference is that the test function exits immediately after the test failure message is generated. Note that this doesn't exit *all* tests; any remaining test functions will execute after the current test function exits.

当 `Error` 和 `Errorf` 将测试标记为失败时，测试函数会继续运行。如果您认为测试函数应该在发现失败后立即停止处理，请使用 `Fatal` 和 `Fatalf` 方法。`Fatal` 方法的工作方式与 `Error` 类似，而 `Fatalf` 方法的工作方式与 `Errorf` 类似。区别在于，测试函数会在生成测试失败消息后立即退出。请注意，这不会退出所有测试；任何剩余的测试函数都会在当前测试函数退出后执行。

When should you use `Fatal` / `Fatalf` and when should you use `Error` / `Errorf` ? If the failure of a check in a test means that further checks in the same test function will always fail or cause the test to panic, use `Fatal` or `Fatalf` . If you are testing several independent items (such as validating fields in a struct), then use `Error` or `Errorf` so you can report many problems at once. This makes it easier to fix multiple problems without rerunning your tests over and over.

何时应该使用 `Fatal` / `Fatalf` , 何时应该使用 `Error` / `Errorf` ? 如果测试中的某个检查失败会导致同一测试函数中的后续检查总是失败或导致测试崩溃，请使用 `Fatal` 或 `Fatalf` 。如果您正在测试多个独立项（例如验证结构体中的字段），则请使用 `Error` 或 `Errorf` , 以便一次性报告多个问题。这样可以更轻松地修复多个问题，而无需反复运行测试。

Setting Up and Tearing Down

搭建和拆卸

Sometimes you have some common state that you want to set up before any tests run and remove when testing is complete. Use a `TestMain` function to manage this state and run your tests:

有时，您需要在运行任何测试之前设置一些通用状态，并在测试完成后移除这些状态。可以使用 `TestMain` 函数来管理此状态并运行您的测试：

```
var testTime time.Time

func TestMain(m *testing.M) {
    fmt.Println("Set up stuff for tests here")
    testTime = time.Now()
    exitVal := m.Run()
    fmt.Println("Clean up stuff after tests here")
    os.Exit(exitVal)
}

func TestFirst(t *testing.T) {
    fmt.Println("TestFirst uses stuff set up in
TestMain", testTime)
}
```

```
func TestSecond(t *testing.T) {  
    fmt.Println("TestSecond also uses stuff set up  
in TestMain", testTime)  
}
```

Both `TestFirst` and `TestSecond` refer to the package-level variable `testTime`. Assume that it needs to be initialized in order for the tests to run properly. You declare a function called `TestMain` with a parameter of type `*testing.M`. If there's a function named `TestMain` in a package, `go test` calls it instead of the test functions. It is the responsibility of the `TestMain` function to set up any state that's necessary to make the tests in the package run correctly.

`TestFirst` 和 `TestSecond` 都指向包级变量 `testTime`。假设该变量需要初始化才能使测试正常运行。您声明了一个名为 `TestMain` 的函数，其参数类型为 `*testing.M`。如果包中存在名为 `TestMain` 的函数，则 `go test` 会调用该函数而不是测试函数。`TestMain` 函数负责设置包中测试正常运行所需的任何状态。

Once the state is configured, the `TestMain` function calls the `Run` method on `*testing.M`. This runs the test functions in the package. The `Run` method returns the exit code; `0` indicates that all tests passed. Finally, the `TestMain` function must call `os.Exit` with the exit code returned from `Run`.

状态配置完成后, `TestMain` 函数会调用 `*testing.M` 上的 `Run` 方法。这将运行包中的测试函数。`Run` 方法返回退出代码; `0` 表示所有测试均已通过。最后, `TestMain` 函数必须调用 `os.Exit`, 并将 `Run` 返回的退出代码传递给它。

Running `go test` on this produces the output:

对此运行 `go test` 会产生以下输出:

```
$ go test
Set up stuff for tests here
TestFirst uses stuff set up in TestMain 2020-09-01
21:42:36.231508 -0400 EDT
    m=+0.000244286
TestSecond also uses stuff set up in TestMain
2020-09-01 21:42:36.231508 -0400
    EDT m=+0.000244286
PASS
Clean up stuff after tests here
ok      test_examples/testmain 0.006s
```

NOTE

笔记

Be aware that `TestMain` is invoked once, not before and after each individual test. Also be aware that you can have only one `TestMain` per package.

请注意，`TestMain` 只会调用一次，而不是在每个测试前后都调用一次。另请注意，每个包只能有一个 `TestMain`。

`TestMain` is useful in two common situations:

`TestMain` 在以下两种常见情况下很有用：

- When you need to set up data in an external repository, such as a database
当您需要在外部存储库（例如数据库）中设置数据时
- When the code being tested depends on package-level variables that need to be initialized
当被测代码依赖于需要初始化的包级变量时

As mentioned before (and will be again!), you should avoid package-level variables in your programs. They make it hard to understand how data flows through your program. If you are using `TestMain` for this reason, consider refactoring your code.

如前所述（以后还会再说！），您应该避免在程序中使用包级变量。它们会使程序中的数据流变得难以理解。如果您出于这个原因使用了

`TestMain` , 请考虑重构您的代码。

The `Cleanup` method on `*testing.T` is used to clean up temporary resources created for a single test. This method has a single parameter, a function with no input parameters or return values. The function runs when the test completes. For simple tests, you can achieve the same result by using a `defer` statement, but `Cleanup` is useful when tests rely on helper functions to set up sample data, as you see in [Example 15-1](#). It's fine to call `Cleanup` multiple times. Just like `defer` , the functions are invoked in last-added, first-called order.

`*testing.T` 上的 `Cleanup` 方法用于清理为单个测试创建的临时资源。此方法只有一个参数，即一个没有输入参数或返回值的函数。该函数在测试完成后运行。对于简单的测试，您可以使用 `defer` 语句实现相同的结果，但当测试依赖辅助函数来设置示例数据时（如示例 15-1 所示），`Cleanup` 非常有用。您可以多次调用 `Cleanup`。与 `defer` 一样，这些函数按照后添加、先调用的顺序执行。

Example 15-1. Using `t.Cleanup`

示例 15-1. 使用 `t.Cleanup`

```
// createFile is a helper function called from
multiple tests
func createFile(t *testing.T) (_ string, err
error) {
```

```

    f, err := os.Create("tempFile")
    if err != nil {
        return "", err
    }
    defer func() {
        err = errors.Join(err, f.Close())
    }()
    // write some data to f
    t.Cleanup(func() {
        os.Remove(f.Name())
    })
    return f.Name(), nil
}

func TestFileProcessing(t *testing.T) {
    fName, err := createFile(t)
    if err != nil {
        t.Fatal(err)
    }
    // do testing, don't worry about cleanup
}

```

If your test uses temporary files, you can avoid writing cleanup code by taking advantage of the `TempDir` method on `*testing.T`. This method creates a new temporary directory every time it is invoked and returns the full path of the directory. It also registers a handler with `Cleanup` to

delete the directory and its contents when the test has completed. You can use it to rewrite the previous example:

如果您的测试使用了临时文件，您可以利用 `*testing.T` 上的 `TempDir` 方法来避免编写清理代码。此方法每次调用都会创建一个新的临时目录，并返回该目录的完整路径。它还会向 `Cleanup` 注册一个处理程序，以便在测试完成后删除该目录及其内容。您可以使用它来重写前面的示例：

```
// createFile is a helper function called from
multiple tests
func createFile(tempDir string) (_ string, error) {
    f, err := os.CreateTemp(tempDir, "tempFile")
    if err != nil {
        return "", err
    }
    defer func() {
        err = errors.Join(err, f.Close())
    }()
    // write some data to f
    return f.Name(), nil
}

func TestFileProcessing(t *testing.T) {
    tempDir := t.TempDir()
    fName, err := createFile(tempDir)
```

```
    if err != nil {
        t.Fatal(err)
    }
    // do testing, don't worry about cleanup
}
```

Testing with Environment Variables

使用环境变量进行测试

It's a common (and very good) practice to configure applications with environment variables. To help you test your environment-variable-parsing code, Go provides a helper method on `testing.T`. Call `t.Setenv()` to register a value for an environment variable for your test. Behind the scenes, it calls `Cleanup` to revert the environment variable to its previous state when the test exits:

使用环境变量配置应用程序是一种常见（且非常好）的做法。为了帮助您测试环境变量解析代码，Go 在 `testing.T` 上提供了一个辅助方法。调用 `t.Setenv()` 为您的测试注册一个环境变量值。在后台，它会在测试退出时调用 `Cleanup` 将环境变量恢复到之前的状态：

```
// assume ProcessEnvVars is a function that
processes environment variables
// and returns a struct with an OutputFormat field
```

```
func TestEnvVarProcess(t *testing.T) {  
    t.Setenv("OUTPUT_FORMAT", "JSON")  
    cfg := ProcessEnvVars()  
    if cfg.OutputFormat != "JSON" {  
        t.Error("OutputFormat not set correctly")  
    }  
    // value of OUTPUT_FORMAT is reset when the  
    test function exits  
}
```

NOTE

笔记

While it is good to use environment variables to configure your application, it is also good to make sure that most of your code is entirely unaware of them. Be sure to copy the values of environment variables into configuration structs before your program starts its work, in your `main` function or soon afterward. Doing so makes it easier to reuse and test code, since *how* the code is configured has been abstracted away from *what* the code is doing.

虽然使用环境变量来配置应用程序是明智之举，但同样重要的是确保大部分代码完全无需感知它们。务必在程序启动之前，在 `main` 函数中或之后不久，将环境变量的值复制到配置结构体中。这样做可以简化代码的重用和测试，因为代码的配置方式与代码的实际运行机制已经分离。

Rather than writing this code yourself, you should strongly consider using a third-party configuration library, like [Viper](#) or [envconfig](#). Also, look at [GoDotEnv](#) as a way to store environment variables in `.env` files for development or continuous integration machines.

与其自己编写这段代码，不如强烈建议使用第三方配置库，例如 Viper 或 envconfig。此外，还可以考虑使用 GoDotEnv，它可将环境变量存储在 `.env` 文件中，适用于开发或持续集成环境。

Storing Sample Test Data

存储样本测试数据

As `go test` walks your source code tree, it uses the current package directory as the current working directory. If you want to use sample data to test functions in a package, create a subdirectory named `testdata` to hold your files. Go reserves this directory name as a place to hold test files.

When reading from *testdata*, always use a relative file reference. Since `go test` changes the current working directory to the current package, each package accesses its own *testdata* via a relative file path.

当 `go test` 遍历你的源代码树时，它会将当前包目录用作当前工作目录。如果你想使用示例数据来测试包中的函数，请创建一个名为 *testdata* 的子目录来存放你的文件。Go 保留了这个目录名作为存放测试文件的位置。从 *testdata* 读取数据时，请始终使用相对文件路径。由于 `go test` 会将当前工作目录更改为当前包，因此每个包都通过相对文件路径访问自己的 *testdata*。

TIP

提示

The `text_package` in the *sample_code* directory of the Chapter 15 repository demonstrates how to use *testdata*.

第 15 章存储库中的 *sample_code* 目录中的 `text` 包演示了如何使用测试数据。

Caching Test Results

缓存测试结果

Just as you learned in [Chapter 10](#) that Go caches compiled packages if they haven't changed, Go also caches test results when running tests across multiple packages if they have passed and their code hasn't changed. The

tests are recompiled and rerun if you change any file in the package or in the *testdata* directory. You can also force tests to always run if you pass the flag `-count=1` to `go test`.

正如你在第 10 章学到的，Go 会缓存已编译但未发生更改的包，同样，当跨多个包运行测试且测试通过且代码未更改时，Go 也会缓存测试结果。如果你更改了包或 *testdata* 目录中的任何文件，测试将被重新编译并重新运行。你还可以通过向 `-count=1` 传递标志 `go test` 来强制测试始终运行。

Testing Your Public API

测试您的公共 API

The tests that you've written are in the same package as the production code. This allows you to test both exported and unexported functions.

你编写的测试用例与生产代码位于同一个包中。这样你就可以测试已导出和未导出的函数。

If you want to test just the public API of your package, Go has a convention for specifying this. You still keep your test source code in the same directory as the production source code, but you use `package_name_test` for the package name. Let's redo the initial test case, using an exported function instead. You can find the code in the *sample_code/pubadder*

directory in the [Chapter 15 repository](#). If you have the following function in the `pubadder` package

如果您只想测试包的公共 API，Go 提供了一种约定来指定。您仍然需要将测试源代码与生产源代码放在同一目录下，但包名要使用 `packagename_test`。让我们重新执行初始测试用例，这次使用导出的函数。您可以在第 15 章代码库的 `sample_code/pubadder` 目录中找到代码。如果您在 `pubadder` 包中有以下函数，则需要注意这一点。

```
func AddNumbers(x, y int) int {  
    return x + y  
}
```

then you can test it as public API by using the following code in a file in the `pubadder` package named `adder_public_test.go`:

然后，您可以使用以下代码在 `pubadder` 包中名为 `adder_public_test.go` 的文件中对其进行公共 API 测试：

```
package pubadder_test  
  
import (  
    "github.com/learning-go-book-  
2e/ch15/sample_code/pubadder"  
    "testing"  
)
```

```
func TestAddNumbers(t *testing.T) {  
    result := pubadder.AddNumbers(2, 3)  
    if result != 5 {  
        t.Error("incorrect result: expected 5,  
got", result)  
    }  
}
```

Notice that the package name for the test file is `pubadder_test`. You have to import `github.com/learning-go-book-2e/ch15/sample_code/pubadder` even though the files are in the same directory. To follow the convention for naming tests, the test function name matches the name of the `AddNumbers` function. Also note that you use `pubadder.AddNumbers`, since you are calling an exported function in a different package.

请注意，测试文件的包名为 `pubadder_test`。即使文件位于同一目录下，您也必须导入 `github.com/learning-go-book-2e/ch15/sample_code/pubadder`。为了遵循测试命名约定，测试函数的名称应与 `AddNumbers` 函数的名称一致。另请注意，由于您调用的是另一个包中的导出函数，因此需要使用 `pubadder.AddNumbers`。

TIP

提示

If you are typing this code in by hand, you'll need to create a module with a *go.mod* file that has the module declaration:

如果您是手动输入这段代码，则需要创建一个模块，并在其中创建一个 *go.mod* 文件，并在该文件中包含模块声明：

```
module github.com/learning-go-book-2e/ch15
```

and that puts the source code in the *sample_code/pubadder* directory within the module.

这样就把源代码放在了模块内的 *sample_code/pubadder* 目录中。

Just as you can call exported functions from within a package, you can test your public API from a test that is in the same package as your source code. The advantage of using the `_test` suffix in the package name is that it lets you treat your tested package as a “black box.” You are forced to interact with it only via its exported functions, methods, types, constants, and variables. Also be aware that you can have test source files with both package names intermixed in the same source directory.

就像你可以从包内部调用导出的函数一样，你也可以在与源代码位于同一包中的测试文件中测试你的公共 API。在包名中使用 `_test` 后缀的优势在于，它允许你将待测包视为一个“黑盒”。你只能通过其导

出的函数、方法、类型、常量和变量来与其交互。另请注意，你可以在同一个源代码目录中混合使用两个包名的测试源文件。

Using go-cmp to Compare Test Results

使用 go-cmp 比较测试结果

Writing a thorough comparison of a compound type's two instances can be verbose. While you can use `reflect.DeepEqual` to compare structs, maps, and slices, there's a better way. Google released a third-party module called `go-cmp` that does the comparison for you and returns a detailed description of what does not match. Let's see how it works by defining a simple `struct` and a factory function that populates it. You can find this code in the `sample_code/cmp` directory in the [Chapter 15 repository](#):

对复合类型的两个实例进行详尽的比较可能会很冗长。虽然可以使用 `reflect.DeepEqual` 来比较结构体、映射和切片，但还有更好的方法。Google 发布了一个名为 `go-cmp` 的第三方模块，它可以自动进行比较，并返回不匹配项的详细描述。让我们通过定义一个简单的 `struct` 和一个用于填充它的工厂函数来了解它的工作原理。您可以在第 15 章代码库的 `sample_code/cmp` 目录中找到这段代码：

```
type Person struct {  
    Name    string  
    Age     int
```

```

        DateAdded time.Time
    }

    func CreatePerson(name string, age int) Person {
        return Person{
            Name:      name,
            Age:       age,
            DateAdded: time.Now(),
        }
    }
}

```

In your test file, you need to import `github.com/google/go-cmp/cmp`, and your test function looks like this:

在你的测试文件中，你需要导入 `github.com/google/go-cmp/cmp`，你的测试函数看起来像这样：

```

func TestCreatePerson(t *testing.T) {
    expected := Person{
        Name: "Dennis",
        Age:  37,
    }
    result := CreatePerson("Dennis", 37)
    if diff := cmp.Diff(expected, result); diff != "" {
        t.Error(diff)
    }
}

```

```
    }  
}
```

The `cmp.Diff` function takes in the expected output and the output that was returned by the function that you're testing. It returns a string that describes any mismatches between the two inputs. If the inputs match, it returns an empty string. You assign the output of the `cmp.Diff` function to a variable called `diff` and then check whether `diff` is an empty string. If it is not, an error occurred.

`cmp.Diff` 函数接收预期输出和被测函数返回的输出作为参数。它返回一个字符串，描述两个输入之间的任何不匹配之处。如果输入匹配，则返回空字符串。您将 `cmp.Diff` 函数的输出赋值给名为 `diff` 的变量，然后检查 `diff` 是否为空字符串。如果不是，则表示发生错误。

When you build and run the test, you'll see the output that `go-cmp` generates when two struct instances don't match:

构建并运行测试后，您将看到 `go-cmp` 在两个结构体实例不匹配时生成的输出：

```
$ go test  
--- FAIL: TestCreatePerson (0.00s)
```

```

ch13_cmp_test.go:16:   ch13_cmp.Person{
                        Name:      "Dennis",
                        Age:       37,
                        -   DateAdded: s"0001-01-01 00:00:00
+0000 UTC",
                        +   DateAdded: s"2020-03-01
22:53:58.087229 -0500 EST m=+0.001242842",
                        }

FAIL
FAIL    ch13_cmp    0.006s

```

The lines with a `-` and `+` indicate the fields whose values differ. The test failed because the dates didn't match. This is a problem because you can't control what date is assigned by the `CreatePerson` function. You have to ignore the `DateAdded` field. You do that by specifying a comparator function. Declare the function as a local variable in your test:

带有 `-` 和 `+` 的行表示值不同的字段。测试失败是因为日期不匹配。这是一个问题，因为您无法控制 `CreatePerson` 函数分配的日期。您必须忽略 `DateAdded` 字段。您可以通过指定一个比较函数来实现这一点。在您的测试中将该函数声明为局部变量：

```

comparer := cmp.Comparer(func(x, y Person) bool {
    return x.Name == y.Name && x.Age == y.Age
})

```

```
} )
```

Pass a function to the `cmp.Comparer` function to create a customer comparator. The function that's passed in must have two parameters of the same type and return a `bool`. It also must be symmetric (the order of the parameters doesn't matter), deterministic (it always returns the same value for the same inputs), and pure (it must not modify its parameters). In your implementation, you are comparing the `Name` and `Age` fields and ignoring the `DateAdded` field.

将一个函数传递给 `cmp.Comparer` 函数以创建客户比较器。传入的函数必须有两个相同类型的参数，并返回一个 `bool` 值。此外，该函数必须是对称的（参数顺序无关紧要）、确定性的（对于相同的输入，它始终返回相同的值）和纯函数（它不能修改其参数）。在您的实现中，您比较了 `Name` 和 `Age` 字段，而忽略了 `DateAdded` 字段。

Then change your call to `cmp.Diff` to include `comparer`:

然后将您的调用更改为 `cmp.Diff`，以包含 `comparer`:

```
if diff := cmp.Diff(expected, result, comparer);  
diff != "" {
```

```
t.Error(diff)
}
```

This is only a quick preview of the most useful features in `go-cmp`. Check its [documentation](#) to learn more about how to control what is compared and the output format.

这只是 `go-cmp` 中最实用功能的快速预览。请查阅其文档，了解更多关于如何控制比较对象和输出格式的信息。

Running Table Tests

运行表格测试

Most of the time, it takes more than a single test case to validate that a function is working correctly. You could write multiple test functions to validate your function or multiple tests within the same function, but you'll find that a great deal of the testing logic is repetitive. You set up supporting data and functions, specify inputs, check the outputs, and compare to see if they match your expectations.

大多数情况下，仅仅一个测试用例不足以验证一个函数是否正常工作。你可以编写多个测试函数来验证你的函数，或者在同一个函数内编写多个测试用例，但你会发现很多测试逻辑都是重复的。你需要设

置支持数据和函数，指定输入，检查输出，然后进行比较以查看它们是否符合你的预期。

Rather than writing this over and over, you can take advantage of a pattern called *table tests*. Let's take a look at a sample. You can find this code in the *sample_code/table* directory in the [Chapter 15 repository](#). Assume you have the following function in the `table` package:

与其一遍又一遍地重复编写这些代码，不如利用一种叫做表格测试的模式。我们来看一个示例。你可以在第 15 章代码库的 *sample_code/table* 目录中找到这段代码。假设你在 `table` 包中有以下函数：

```
func DoMath(num1, num2 int, op string) (int,
error) {
    switch op {
    case "+":
        return num1 + num2, nil
    case "-":
        return num1 - num2, nil
    case "*":
        return num1 + num2, nil
    case "/":
        if num2 == 0 {
            return 0, errors.New("division by
zero")
        }
    }
```



```

        return num1 / num2, nil
    default:
        return 0, fmt.Errorf("unknown operator
%s", op)
    }
}

```

To test this function, you need to check the different branches, trying out inputs that return valid results, as well as inputs that trigger errors. You could write code like this, but it's very repetitive:

要测试这个函数，你需要检查不同的分支，尝试返回有效结果的输入，以及触发错误的输入。你可以编写类似这样的代码，但这非常重复：

```

func TestDoMath(t *testing.T) {
    result, err := DoMath(2, 2, "+")
    if result != 4 {
        t.Error("Should have been 4, got", result)
    }
    if err != nil {
        t.Error("Should have been nil error, got",
err)
    }
    result2, err2 := DoMath(2, 2, "-")
    if result2 != 0 {

```

```

        t.Error("Should have been 0, got",
result2)
    }
    if err2 != nil {
        t.Error("Should have been nil error, got",
err2)
    }
    // and so on...
}

```

Let's replace this repetition with a table test. First, you declare a slice of anonymous structs. The struct contains fields for the name of the test, the input parameters, and the return values. Each entry in the slice represents another test:

让我们用表格测试来代替这种重复。首先，声明一个匿名结构体切片。该结构体包含测试名称、输入参数和返回值字段。切片中的每个条目代表一个测试：

```

data := []struct {
    name      string
    num1      int
    num2      int
    op        string
    expected  int
    errMsg    string
}

```

```
}{
    {"addition", 2, 2, "+", 4, ""},
    {"subtraction", 2, 2, "-", 0, ""},
    {"multiplication", 2, 2, "*", 4, ""},
    {"division", 2, 2, "/", 1, ""},
    {"bad_division", 2, 0, "/", 0, `division by
zero`},
}
```

Next, loop over each test case in `data`, invoking the `Run` method each time. This is the line that does the magic. You pass two parameters to `Run`, a name for the subtest and a function with a single parameter of type `*testing.T`. Inside the function, you call `DoMath` by using the fields of the current entry in `data`, using the same logic over and over. When you run these tests, you'll see that not only do they pass, but when you use the `-v` flag, each subtest also now has a name:

接下来，遍历 `data` 中的每个测试用例，每次都调用 `Run` 方法。这行代码至关重要。你需要向 `Run` 传递两个参数：子测试的名称和一个带有 `*testing.T` 类型单参数的函数。在函数内部，你使用 `data` 中当前条目的字段调用 `DoMath`，并重复相同的逻辑。运行这些测试后，你会发现它们不仅通过了，而且当你使用 `-v` 标志时，每个子测试现在都有一个名称：

```

for _, d := range data {
    t.Run(d.name, func(t *testing.T) {
        result, err := DoMath(d.num1, d.num2,
d.op)
        if result != d.expected {
            t.Errorf("Expected %d, got %d",
d.expected, result)
        }
        var errMsg string
        if err != nil {
            errMsg = err.Error()
        }
        if errMsg != d.errMsg {
            t.Errorf("Expected error message `%s`,
got `%s`",
                d.errMsg, errMsg)
        }
    })
}

```

TIP

提示

Comparing error messages can be fragile, because there may not be any compatibility guarantees on the message text. The function that you are testing uses `errors.New` and `fmt.Errorf` to make errors, so the only option is to compare the messages. If an error has a custom type or a named sentinel error, use `errors.Is` or `errors.As` to check that the correct error is returned.

比较错误消息可能存在风险，因为消息文本可能不兼容。您正在测试的函数使用 `errors.New` 和 `fmt.Errorf` 来生成错误，因此唯一的办法是比较这些消息。如果错误具有自定义类型或命名哨兵错误，请使用 `errors.Is` 或 `errors.As` 来检查是否返回了正确的错误。

Running Tests Concurrently

同时运行测试

By default, unit tests are run sequentially. Since each unit test is supposed to be independent from every other unit test, they make ideal candidates for concurrency. To make a unit test run concurrently with other tests, call the `Parallel` method on `*testing.T` as the first line in your test:

默认情况下，单元测试是顺序运行的。由于每个单元测试都应该独立于其他单元测试，因此它们非常适合并发运行。要使单元测试与其他测试并发运行，请在测试的第一行调用 `Parallel` 方法，该方法针对 `*testing.T`：

```
func TestMyCode(t *testing.T) {  
    t.Parallel()  
    // rest of test goes here  
}
```

Parallel tests run concurrently with other tests marked as parallel.

并行测试与其他标记为并行的测试同时运行。

The advantage of parallel tests is that it can speed up long-running test suites. There are some disadvantages, though. If you have multiple tests that rely on the same shared mutable state, do not mark them as parallel, because you will get inconsistent results. (But after all of my warnings, you don't have any shared mutable state in your application, right?) Also be aware that your test will panic if you mark it as parallel and use the `Setenv` method in your test function.

并行测试的优势在于可以加快长时间运行的测试套件的执行速度。但它也有一些缺点。如果多个测试依赖于同一个共享的可变状态，请不要将它们标记为并行，否则会导致结果不一致。（不过，经过我的种种警告，你的应用程序中应该没有任何共享的可变状态吧？）另外，请注意，如果你将测试标记为并行并在测试函数中使用 `Setenv` 方法，则该测试会崩溃。

Be careful when running table tests in parallel. When table tests run in parallel, it's just as you saw in [“Goroutines, for Loops, and Varying Variables”](#), where you launched multiple goroutines within a `for` loop. If you run this example using Go 1.21 or earlier (or on Go 1.22 or later with the Go version set to 1.21 or earlier in the `go` directive in the `go.mod` file), a reference to the variable `d` is shared by all the parallel tests, so they all see the same value:

并行运行表格测试时要格外小心。并行运行表格测试的情况与你在“Goroutine、for 循环和变量变化”一节中看到的情况类似，你在一个 `for` 循环中启动了多个 goroutine。如果你使用 Go 1.21 或更早版本运行此示例（或者在 `go.mod` 文件中的 `go` 指令中将 Go 版本设置为 1.21 或更早，但使用 Go 1.22 或更高版本运行），所有并行测试都会共享对变量 `d` 的引用，因此它们看到的都是相同的值：

```
func TestParallelTable(t *testing.T) {
    data := []struct {
        name    string
        input   int
        output  int
    }{
        {"a", 10, 20},
        {"b", 30, 40},
        {"c", 50, 60},
    }
    for _, d := range data {
```

```

        t.Run(d.name, func(t *testing.T) {
            t.Parallel()
            fmt.Println(d.input, d.output)
            out := toTest(d.input)
            if out != d.output {
                t.Error("didn't match", out,
d.output)
            }
        })
    }
}

```

You can run this code on [The Go Playground](#) or in the *sample_code/parallel* directory in the [Chapter 15 repository](#). Take a look at the output and you'll see that you test the last value in the table test three times:

您可以在 Go Playground 或第 15 章代码库的 *sample_code/parallel* 目录中运行此代码。查看输出，您会发现您对表 *test* 中的最后一个值进行了三次测试：

```

=== CONT TestParallelTable/a
50 60
=== CONT TestParallelTable/c
50 60

```



```
=== CONT TestParallelTable/b
50 60
```

This problem is common enough that Go 1.20 added a `go vet` check for this problem. When you run `go vet` on this code, you get the message `loop variable d captured by func literal` on each line where `d` is captured.

这个问题很常见，以至于 Go 1.20 专门为此添加了 `go vet` 检查。当你对这段代码运行 `go vet` 时，在捕获到 `d` 的每一行都会出现 `loop variable d captured by func literal` 消息。

The `for` loop changes in Go 1.22 or later resolve this issue. If you cannot use Go 1.22, you can avoid this bug by shadowing `d` within the `for` loop before invoking `t.Run`:

Go 1.22 或更高版本中 `for` 循环的更改解决了此问题。如果您无法使用 Go 1.22，可以通过在调用 `t.Run` 之前，在 `for` 循环中遮蔽 `d` 来避免此错误：

```
for _, d := range data {
    d := d // THIS IS THE LINE THAT SHADOWS d!
    t.Run(d.name, func(t *testing.T) {
        t.Parallel()
        fmt.Println(d.input, d.output)
        out := toTest(d.input)
```

```
        if out != d.output {
            t.Error("didn't match", out,
d.output)
        }
    })
}
```

Now that you have a way to run lots of tests, you'll learn about code coverage to find out what your tests are testing.

现在你有了运行大量测试的方法，接下来你将学习代码覆盖率，以了解你的测试正在测试什么。

Checking Your Code Coverage

检查代码覆盖率

Code coverage is a very useful tool for knowing whether you've missed any obvious cases. However, reaching 100% code coverage doesn't guarantee that there aren't bugs in your code for some inputs. First you'll see how `go test` displays code coverage and then you'll look at the limitations of relying on code coverage alone.

代码覆盖率是一个非常有用的工具，可以帮助你了解是否遗漏了任何明显的错误。然而，达到 100% 的代码覆盖率并不能保证你的代码在所有输入情况下都不存在 bug。首先，你将看到 `go test` 如何显示代码覆盖率，然后你会了解仅仅依赖代码覆盖率的局限性。

Adding the `-cover` flag to the `go test` command calculates coverage information and includes a summary in the test output. If you include a second flag `-coverprofile`, you can save the coverage information to a file. Let's go back to the *sample_code/table* directory in the [Chapter 15 repository](#) and gather code coverage information:

在 `go test` 命令中添加 `-cover` 标志，即可计算代码覆盖率信息并在测试输出中包含摘要。如果添加第二个标志 `-coverprofile`，则可以将代码覆盖率信息保存到文件中。让我们回到第 15 章代码库中的 *sample_code/table* 目录，并收集代码覆盖率信息：

```
$ go test -v -cover -coverprofile=c.out
```

If you run your table test with code coverage, the test output now includes a line that indicates the amount of test code coverage, 87.5%. That's good to know, but it'd be more useful if you could see what you missed. The `cover` tool included with Go generates an HTML representation of your source code with that information:

如果你运行带有代码覆盖率的表格测试，测试输出现在会包含一行，指示测试代码覆盖率，即 87.5%。知道这个信息固然不错，但如果能看到自己遗漏了什么，那就更有用了。Go 自带的 `cover` 工具会生成包含这些信息的源代码 HTML 表示形式：

```
$ go tool cover -html=c.out
```

When you run it, your web browser should open and show you a page that looks like [Figure 15-1](#).

运行该程序后，您的网络浏览器应该会打开并显示一个类似于图 15-1 的页面。

Every file that's tested appears in the combo box in the upper left. The source code is in one of three colors. Gray is used for lines of code that aren't testable, green is used for code that's been covered by a test, and red is used for code that hasn't been tested. (The reliance on color is unfortunate for readers of the print edition and those who have red-green color blindness. If you are unable to see the colors, the lighter gray is the covered lines.) From looking at this, you can see that you didn't write a test to cover the default case, when a bad operator is passed to your function. Add that case to your slice of test cases:

所有已测试的文件都会显示在左上角的组合框中。源代码以三种颜色显示：灰色表示无法测试的代码行，绿色表示已被测试覆盖的代码，红色表示尚未测试的代码。（对于纸质版读者和红绿色盲患者来说，使用颜色区分代码可能不太方便。如果您无法看到颜色，较浅的灰色表示已被测试覆盖的代码行。）由此可见，您没有编写测试来覆盖默认情况，即向函数传递错误运算符的情况。请将此情况添加到您的测试用例集中：

```
{"bad_op", 2, 2, "?", 0, `unknown operator ?`},
```

When you rerun `go test -v -cover -coverprofile=c.out` and `go tool cover -html=c.out`, you see in [Figure 15-2](#) that the final line is covered and you have 100% test code coverage.

当你重新运行 `go test -v -cover -coverprofile=c.out` 和 `go tool cover -html=c.out` 时，你会在图 15-2 中看到最后一行代码被覆盖，并且你的测试代码覆盖率为 100%。

test_examples/table/table.go (87.5%) not tracked not covered covered

```
package table

import (
    "errors"
    "fmt"
)

func DoMath(num1, num2 int, op string) (int, error) {
    switch op {
    case "+":
        return num1 + num2, nil
    case "-":
        return num1 - num2, nil
    case "*":
        return num1 + num2, nil
    case "/":
        if num2 == 0 {
            return 0, errors.New("division by zero")
        }
        return num1 / num2, nil
    default:
        return 0, fmt.Errorf("unknown operator %s", op)
    }
}
```

Figure 15-1. Initial code coverage

图 15-1. 初始代码覆盖率

```
test_examples/table/table.go (100.0%)  not tracked  not covered  covered

package table

import (
    "errors"
    "fmt"
)

func DoMath(num1, num2 int, op string) (int, error) {
    switch op {
    case "+":
        return num1 + num2, nil
    case "-":
        return num1 - num2, nil
    case "*":
        return num1 + num2, nil
    case "/":
        if num2 == 0 {
            return 0, errors.New("division by zero")
        }
        return num1 / num2, nil
    default:
        return 0, fmt.Errorf("unknown operator %s", op)
    }
}
```

Figure 15-2. Final code coverage

图 15-2. 最终代码覆盖率

Code coverage is a great thing, but it's not enough. There's actually a bug in your code, even though you have 100% coverage. Have you noticed it? If not, add another test case and rerun your tests:

代码覆盖率固然重要，但这还不够。即使覆盖率达到 100%，你的代码中也可能存在 bug。你发现了吗？如果没有，请添加另一个测试用例并重新运行测试：

```
{"another_mult", 2, 3, "*", 6, ""},
```

You should see the error:

你应该会看到以下错误信息：

```
table_test.go:57: Expected 6, got 5
```

Your case for multiplication has a typo. It adds the numbers together instead of multiplying them. (Beware the dangers of copy-and-paste coding!) Fix the code, rerun `go test -v -cover -coverprofile=c.out` and `go tool cover -html=c.out`, and you'll see that the tests pass again.

你的乘法运算代码有误。它把数字相加而不是相乘。（小心复制粘贴代码的风险！）修改代码后，重新运行 `go test -v -cover -coverprofile=c.out` 和 `go tool cover -html=c.out`，你会发现测试再次通过。

WARNING

警告

Code coverage is necessary but not sufficient. You can have 100% code coverage and still have bugs in your code!

代码覆盖率固然重要，但并非充分条件。即使代码覆盖率达到 100%，代码中仍然可能存在 bug！

Fuzzing

模糊测试

One of the most important lessons that every developer eventually learns is that all data is suspect. No matter how well specified a data format is, you will eventually have to process input that doesn't match what you expect. This doesn't happen for only malicious reasons. Data can be corrupted in transit, in storage, and even in memory. Programs that process data might have bugs, and specifications for data formats always have corner cases that can be interpreted differently by different developers.

每个开发者最终都会学到的最重要的一课就是：所有数据都值得怀疑。无论数据格式规范得多么完善，你最终总会遇到与预期不符的输入。这种情况并非总是出于恶意。数据在传输、存储甚至内存中都可能损坏。处理数据的程序本身可能存在漏洞，而且数据格式规范总会有一些特殊情况，不同的开发者可能会有不同的解读。

Even when developers do the work of writing good unit tests, it's impossible to think of everything. As you've seen, even 100% unit test code coverage is no guarantee that your code is bug free. You need to supplement unit tests with generated data that could break your program in ways you didn't anticipate. That's where fuzzing helps.

即使开发者编写了高质量的单元测试，也不可能面面俱到。正如你所见，即使单元测试代码覆盖率达到 100%，也不能保证代码完全没有 bug。你需要用生成的测试数据来补充单元测试，这些数据可能会以你意想不到的方式破坏程序。模糊测试正是在这方面发挥作用。

Fuzzing is a technique for generating random data and submitting it to code to see whether it properly handles unexpected input. The developer can provide a *seed corpus* or set of known good data, and the fuzzer uses that as a basis for generating bad input. Let's see how to use the fuzzing support in Go's testing tools to discover additional test cases.

模糊测试是一种生成随机数据并将其提交给代码以检验其是否能正确处理意外输入的技术。开发者可以提供种子语料库或一组已知良好的数据，模糊测试器会以此为基础生成异常输入。接下来，我们将探讨如何使用 Go 语言测试工具中的模糊测试功能来发现更多测试用例。

Assume that you are writing a program to process data files. You can find the code for this example [on GitHub](#). You are sending a list of strings but want to efficiently allocate memory, so the number of strings in a file is sent as the first line, while the remaining lines are the lines of text. Here's the sample function for processing this data:

假设您正在编写一个处理数据文件的程序。您可以在 GitHub 上找到此示例的代码。您发送的是一个字符串列表，但为了高效分配内存，文

件中的字符串数量作为第一行发送，其余行则发送文本内容。以下是处理此数据的示例函数：

```
func ParseData(r io.Reader) ([]string, error) {
    s := bufio.NewScanner(r)
    if !s.Scan() {
        return nil, errors.New("empty")
    }
    countStr := s.Text()
    count, err := strconv.Atoi(countStr)
    if err != nil {
        return nil, err
    }
    out := make([]string, 0, count)
    for i := 0; i < count; i++ {
        hasLine := s.Scan()
        if !hasLine {
            return nil, errors.New("too few
lines")
        }
        line := s.Text()
        out = append(out, line)
    }
    return out, nil
}
```

You use a `bufio.Scanner` to read line-by-line from an `io.Reader`. If there's no data to be read, an error is returned. You then read the first line and attempt to convert it to an int named `count`. If you can't, an error is returned. Next, you allocate the memory for your slice of strings, and read `count` number of lines from the scanner. If there aren't enough lines, an error is returned. If all goes well, you return the lines you read.

你使用 `bufio.Scanner` 从 `io.Reader` 逐行读取数据。如果没有数据可读，则返回错误。然后，你读取第一行并尝试将其转换为名为 `count` 的整数。如果转换失败，则返回错误。接下来，你为字符串切片分配内存，并从扫描器读取 `count` 行。如果行数不足，则返回错误。如果一切顺利，则返回已读取的行。

A unit test has already been written to validate the code:

已经编写了单元测试来验证代码：

```
func TestParseData(t *testing.T) {
    data := []struct {
        name    string
        in      []byte
        out     []string
        errMsg string
    }{
        {
            name:    "simple",
```

```

        in:
[]byte("3\nhello\ngoodbye\ngreetings\n"),
        out:    []string{"hello", "goodbye",
" greetings"},
        errMsg: "",
    },
    {
        name:    "empty_error",
        in:      []byte(""),
        out:     nil,
        errMsg:  "empty",
    },
    {
        name:    "zero",
        in:      []byte("0\n"),
        out:     []string{},
        errMsg:  "",
    },
    {
        name:    "number_error",
        in:
[]byte("asdf\nhello\ngoodbye\ngreetings\n"),
        out:     nil,
        errMsg:  `strconv.Atoi: parsing "asdf":
invalid syntax`,
    },
    {
        name:    "line_count_error",

```

```

        in:
[]byte("4\nhello\ngoodbye\ngreetings\n"),
        out:    nil,
        errMsg: "too few lines",
    },
}
for _, d := range data {
    t.Run(d.name, func(t *testing.T) {
        r := bytes.NewReader(d.in)
        out, err := ParseData(r)
        var errMsg string
        if err != nil {
            errMsg = err.Error()
        }
        if diff := cmp.Diff(d.out, out); diff
!= "" {
            t.Error(diff)
        }
        if diff := cmp.Diff(d.errMsg, errMsg);
diff != "" {
            t.Error(diff)
        }
    })
}
}

```

The unit test has 100% line coverage for `ParseData`, handling all its error cases. You might think the code is ready for production, but let's see if fuzzing can help you discover errors that you hadn't considered.

单元测试对 `ParseData` 的代码覆盖率达到了 100%，处理了所有错误情况。你可能认为代码已经可以投入生产环境了，但让我们看看模糊测试能否帮你发现一些你之前没有考虑到的错误。

NOTE

笔记

Be aware that fuzzing uses a lot of resources. A fuzz test can allocate (or attempt to allocate) many gigabytes of memory and might write several gigabytes of data to your local disk. If you are running something else on the same machine at the same time as a fuzz test, don't be surprised if it slows down.

请注意，模糊测试会占用大量资源。一次模糊测试可能会分配（或尝试分配）数GB的内存，并可能向本地磁盘写入数GB的数据。如果在同一台机器上同时运行其他程序和模糊测试，则测试速度变慢是正常的。

You start by writing a fuzz test:

首先，你需要编写一个模糊测试：

```
func FuzzParseData(f *testing.F) {
    testcases := [][]byte{
        []byte("3\nhello\ngoodbye\ngreetings\n"),
        []byte("0\n"),
    }
```

```

    }
    for _, tc := range testcases {
        f.Add(tc)
    }
    f.Fuzz(func(t *testing.T, in []byte) {
        r := bytes.NewReader(in)
        out, err := ParseData(r)
        if err != nil {
            t.Skip("handled error")
        }
        roundTrip := ToData(out)
        rtr := bytes.NewReader(roundTrip)
        out2, err := ParseData(rtr)
        if diff := cmp.Diff(out, out2); diff != ""
    {
        t.Error(diff)
    }
    })
}

```

A fuzz test looks similar to a standard unit test. The function name starts with `Fuzz`, the only parameter is of type `*testing.F`, and it has no return values.

模糊测试与标准单元测试类似。函数名以 `Fuzz` 开头，唯一的参数类型为 `*testing.F`，并且没有返回值。

Next, you set up a seed corpus, which is composed of one or more sets of sample data. The data can run successfully, error out, or even panic. The important things are that you know how your program behaves when this data is provided and that this behavior is accounted for by your fuzz test. These samples are mutated by the fuzzer to generate bad inputs. This example uses only a single field of data for each entry (a slice of bytes), but you can have as many fields as needed. The fields in a corpus entry are currently limited to only certain types:

接下来，你需要设置一个种子语料库，它由一个或多个样本数据集组成。这些数据可能运行成功、出错，甚至崩溃。重要的是，你要了解你的程序在提供这些数据时的行为，并且模糊测试能够解释这种行为。模糊测试器会对这些样本进行变异，生成错误的输入。本例中每个条目仅使用一个数据字段（字节切片），但你可以根据需要添加任意数量的字段。目前，语料库条目中的字段仅限于以下几种类型：

- Any integer type (including unsigned types, `rune`, and `byte`)
任何整数类型（包括无符号类型、`rune` 和 `byte`）
- Any floating-point type
任何浮点类型
- `bool`
`bool`
- `string`
`string`
- `[]byte`

```
[ ]byte
```

Each entry in the corpus is passed to the `Add` method on the `*testing.F` instance. In the example, you have a slice of bytes for each entry:

语料库中的每个条目都会传递给 `*testing.F` 实例上的 `Add` 方法。在本例中，每个条目都对应一个字节切片：

```
f.Add(tc)
```

If the function being fuzzed needed an `int` and a `string`, the call to `Add` would look like this:

如果被模糊测试的函数需要 `int` 和 `string`，则对 `Add` 的调用将如下所示：

```
f.Add(1, "some text")
```

It's a runtime error to pass a value of an invalid type to `Add`.

将无效类型的值传递给 `Add` 会导致运行时错误。

Next you call the `Fuzz` method on your `*testing.F` instance. This looks a bit like a call to `Run` when writing a table test in a standard unit test. `Fuzz` takes in a single parameter, a function whose first parameter is

of type `*testing.T` and whose remaining parameters exactly match the types, order, and count of the values that were passed in to `Add`. This also specifies the type of data generated by the fuzzing engine during fuzzing. There's no way for the Go compiler to enforce this constraint, so it's a runtime error if the convention is not followed.

接下来，您需要调用 `Fuzz` 方法，该方法作用于 `*testing.F` 实例。这看起来有点像在标准单元测试中编写表格测试时调用 `Run`。`Fuzz` 接收一个参数，该参数是一个函数，其第一个参数的类型为 `*testing.T`，其余参数的类型、顺序和数量与传递给 `Add` 的值完全匹配。这也指定了模糊测试引擎在模糊测试期间生成的数据类型。Go 编译器无法强制执行此约束，因此如果不遵循此约定，则会引发运行时错误。

Finally, let's look at the body of the fuzz target. Remember, fuzzing is used to find cases where bad input is not handled correctly. Since the input is randomly generated, you can't write tests that have knowledge of what the output should be. Instead, you have to use test conditions that will be true for all inputs. In the case of `ParseData`, you can check for two things:

最后，我们来看一下模糊测试目标的主体部分。记住，模糊测试用于查找错误输入未被正确处理的情况。由于输入是随机生成的，因此您无法编写已知输出结果的测试用例。相反，您必须使用对所有输入都成立的测试条件。对于 `ParseData`，您可以检查以下两点：

- Does the code return an error for bad input, or does it panic?
如果输入错误，代码会返回错误信息还是会崩溃？
- If you convert the slice of strings back to a slice of bytes and re-parse it, do you get the same result?
如果将字符串切片转换回字节切片并重新解析，是否会得到相同的结果？

Let's see what happens when you run the fuzz test:

让我们看看运行模糊测试会发生什么：

```
$ go test -fuzz=FuzzParseData
fuzz: elapsed: 0s, gathering baseline coverage:
0/243 completed
fuzz: elapsed: 0s, gathering baseline coverage:
243/243 completed,
    now fuzzing with 8 workers
fuzz: minimizing 289-byte failing input file
fuzz: elapsed: 3s, minimizing
fuzz: elapsed: 6s, minimizing
fuzz: elapsed: 9s, minimizing
fuzz: elapsed: 10s, minimizing
--- FAIL: FuzzParseData (10.48s)
    fuzzing process hung or terminated
unexpectedly while minimizing: EOF
    Failing input written to
```

```
testdata/fuzz/FuzzParseData/

fedbaf01dc50bf41b40d7449657cdc9af9868b1be0421c98b2910
  To re-run:
  go test -run=FuzzParseData/

fedbaf01dc50bf41b40d7449657cdc9af9868b1be0421c98b2910
FAIL
exit status 1
FAIL    file_parser    10.594s
```

If you don't specify the `-fuzz` flag, your fuzz tests will be treated like unit tests and will be run against the seed corpus. Only a single fuzz test can be fuzzed at a time.

如果您不指定 `-fuzz` 标志，您的模糊测试将被视为单元测试，并针对种子语料库运行。一次只能运行一个模糊测试。

NOTE

笔记

If you want to get the full experience, delete the contents of the *testdata/fuzz/FuzzParseData* directory. This will cause the fuzzer to generate new seed corpus entries. Since the fuzzer generates random input, your samples might be different from the ones shown. The different entries will likely produce similar errors, though maybe not in the same order.

如果想要获得完整的体验，请删除 *testdata/fuzz/FuzzParseData* 目录中的所有内容。这将导致模糊测试器生成新的种子语料库条目。由于模糊测试器生成的是随机输入，因此您的样本可能与所示样本有所不同。不同的条目可能会产生类似的错误，但错误顺序可能不同。

The fuzz test runs for several seconds and then fails. In this case, the `go` command reports that it has crashed. You don't want programs that crash, so let's look at the input that was generated. Every time a test case fails, the fuzzer writes it to the *testdata/fuzz/TESTNAME* subdirectory in the same package as the failed test, adding a new entry to the seed corpus. The new seed corpus entry in the file now becomes a new unit test, one that was automatically generated by the fuzzer. It is run anytime `go test` runs the `FuzzParseData` function, and acts as a regression test once you fix your bug.

模糊测试运行几秒钟后失败。在这种情况下，`go` 命令报告程序崩溃。程序崩溃是不可接受的，所以我们来看看生成的输入。每次测试用例失败时，模糊测试器都会将其写入与失败测试用例位于同一包下的 *testdata/fuzz/TESTNAME* 子目录中，从而向种子语料库添加一个新条目。文件中新增的种子语料库条目现在就变成了一个单元测

试，该测试由模糊测试器自动生成。每当 `go test` 运行 `FuzzParseData` 函数时，该单元测试都会运行，并在您修复错误后用作回归测试。

Here are the contents of the file:

以下是文件内容：

```
go test fuzz v1  
[]byte("300000000000")
```

The first line is a header to indicate that this is test data for a fuzz test. The subsequent lines have the data that caused the failure.

第一行是标题行，表明这是模糊测试的测试数据。后续行包含导致测试失败的数据。

The failure message tells you how to isolate this failing case when you rerun the test:

失败信息会告诉你如何在重新运行测试时隔离此失败案例：

```
$ go test -run=FuzzParseData/  
  
fedbaf01dc50bf41b40d7449657cdc9af9868b1be0421c981
```

```
signal: killed
FAIL    file_parser    15.046s
```

The problem is that you are trying to allocate a slice with the capacity to hold 300,000,000,000 strings. That requires quite a bit more RAM than my computer (and probably yours) has. You need to limit the number of expected text elements to a reasonable number. Set the maximum number of rows to 1,000 by adding the following code to `ParseData` after you parse the number of expected rows:

问题在于您试图分配一个容量为 300,000,000,000 个字符串的切片。这需要比我的电脑（可能也包括您的电脑）拥有的内存多得多。您需要将预期文本元素的数量限制在一个合理的范围内。在解析出预期行数后，将以下代码添加到 `ParseData` 中，将最大行数设置为 1,000：

```
if count > 1000 {
    return nil, errors.New("too many")
}
```

Run the fuzzer again to see whether it finds any more errors:

再次运行模糊测试程序，看看是否还能发现其他错误：

```
$ go test -fuzz=FuzzParseData
```



```
fuzz: elapsed: 0s, gathering baseline coverage:
0/245 completed
fuzz: elapsed: 0s, gathering baseline coverage:
245/245 completed,
    now fuzzing with 8 workers
fuzz: minimizing 29-byte failing input file
fuzz: elapsed: 2s, minimizing
--- FAIL: FuzzParseData (2.20s)
    --- FAIL: FuzzParseData (0.00s)
        testing.go:1356: panic: runtime error:
makeslice: cap out of range
            goroutine 23027 [running]:
            runtime/debug.Stack()

/usr/local/go/src/runtime/debug/stack.go:24 +0x104
            testing.tRunner.func1()

/usr/local/go/src/testing/testing.go:1356 +0x258
            panic({0x1003f9920, 0x10042a260})

/usr/local/go/src/runtime/panic.go:884 +0x204
            file_parser.ParseData({0x10042a7c8,
0x14006c39bc0})
            file_parser/file_parser.go:24
+0x254
[...]
    Failing input written to
testdata/fuzz/FuzzParseData/
```

```
03f81b404ad91d092a482ad1ccb4a457800599ab826ec8dae47b4
  To re-run:
  go test -run=FuzzParseData/

03f81b404ad91d092a482ad1ccb4a457800599ab826ec8dae47b4
FAIL
exit status 1
FAIL    file_parser    2.434s
```

This time you get a fuzz result that produces a panic. Looking at the file generated by `go fuzz`, you see this:

这次得到的模糊测试结果会导致系统崩溃。查看 `go fuzz` 生成的文件，可以看到以下内容：

```
go test fuzz v1
[]byte("-1")
```

The line that's generating the panic is shown here:

引发恐慌的那行代码如下所示：

```
out := make([]string, 0, count)
```

You are trying to create a slice with negative capacity, which panics. Add another condition to your code to look for negative numbers:

你试图创建一个容量为负数的切片，这会导致程序崩溃。请在代码中添加一个条件来检查是否为负数：

```
if count < 0 {  
    return nil, errors.New("no negative  
numbers")  
}
```

Run your fuzzer again, and it generates another error:

再次运行模糊测试程序，它会产生另一个错误：

```
$ go test -fuzz=FuzzParseData  
fuzz: elapsed: 0s, gathering baseline coverage:  
0/246 completed  
fuzz: elapsed: 0s, gathering baseline coverage:  
246/246 completed,  
    now fuzzing with 8 workers  
fuzz: elapsed: 3s, execs: 288734 (96241/sec), new  
interesting: 0 (total: 246)  
fuzz: elapsed: 6s, execs: 418803 (43354/sec), new  
interesting: 0 (total: 246)  
fuzz: minimizing 34-byte failing input file
```

```
fuzz: elapsed: 7s, minimizing
--- FAIL: FuzzParseData (7.43s)
    --- FAIL: FuzzParseData (0.00s)
        file_parser_test.go:89: []string{
            -   "\r",
            +   "",
            }

```

Failing input written to
testdata/fuzz/FuzzParseData/

b605c41104bf41a21309a13e90cfc6f30ecf133a2382759f2abc3

To re-run:

```
go test -run=FuzzParseData/
```

b605c41104bf41a21309a13e90cfc6f30ecf133a2382759f2abc3

FAIL

exit status 1

FAIL file_parser 7.558s

Looking at the file it created, you are generating blank lines with only `\r` (return) characters. Blank lines are not what you expect in your input, so add some code to the loop that reads lines of text from the `Scanner`. You'll check whether a line consists only of whitespace characters. If it does, return an error:

查看生成的文件，你会发现其中只包含 `\r`（回车符）字符的空行。空行并非你期望的输入内容，因此请在从 `Scanner` 读取文本行的循环中添加一些代码。你需要检查某一行是否只包含空格字符。如果是，则返回错误：

```
line = strings.TrimSpace(line)
if len(line) == 0 {
    return nil, errors.New("blank line")
}
```

Run your fuzzer yet again:

再运行一次模糊测试：

```
$ go test -fuzz=FuzzParseData
fuzz: elapsed: 0s, gathering baseline coverage:
0/247 completed
fuzz: elapsed: 0s, gathering baseline coverage:
247/247 completed,
    now fuzzing with 8 workers
fuzz: elapsed: 3s, execs: 391018 (130318/sec), new
interesting: 2 (total: 249)
fuzz: elapsed: 6s, execs: 556939 (55303/sec), new
interesting: 2 (total: 249)
fuzz: elapsed: 9s, execs: 622126 (21734/sec), new
interesting: 2 (total: 249)
```

```
[...]
fuzz: elapsed: 2m0s, execs: 2829569 (0/sec), new
interesting: 16 (total: 263)
fuzz: elapsed: 2m3s, execs: 2829569 (0/sec), new
interesting: 16 (total: 263)
^Cfuzz: elapsed: 2m4s, execs: 2829569 (0/sec), new
interesting: 16 (total: 263)
PASS
ok      file_parser      123.662s
```

After a few minutes, no more errors are found, so hit Ctrl-C to end fuzzing.

几分钟后，未发现更多错误，因此按 Ctrl-C 结束模糊测试。

Just because the fuzzer didn't find additional issues doesn't mean that the code is now bug free. However, fuzzing allowed you to automatically find some significant oversights in the original code. Writing fuzz tests takes practice, as they require a slightly different mindset than writing unit tests. Once you get the hang of them, they become an essential tool for validating how your code handles unexpected user input.

模糊测试工具没有发现其他问题并不意味着代码就完全没有bug了。但是，模糊测试确实能帮助你自动发现原始代码中的一些重大疏漏。编写模糊测试需要练习，因为它与编写单元测试的思路略有不同。一旦掌握了技巧，模糊测试就会成为验证代码如何处理意外用户输入的必备工具。

Using Benchmarks

使用基准测试

Determining how fast (or slow) code runs is surprisingly difficult. Rather than trying to figure it out yourself, you should use the benchmarking support that's built into Go's testing framework. Let's explore it with a function in the *sample_code/bench* directory in the [Chapter 15 repository](#):

确定代码运行速度快慢其实非常困难。与其自己摸索，不如使用 Go 测试框架内置的基准测试支持。让我们通过第 15 章代码库中 *sample_code/bench* 目录下的一个函数来探索一下：

```
func FileLen(f string, bufsize int) (int, error) {
    file, err := os.Open(f)
    if err != nil {
        return 0, err
    }
    defer file.Close()
    count := 0
    for {
        buf := make([]byte, bufsize)
        num, err := file.Read(buf)
        count += num
        if err != nil {
```

```
        break
    }
}
return count, nil
}
```

This function counts the number of characters in a file. It takes in two parameters, the name of the file and the size of the buffer that you are using to read the file (you'll see the reason for the second parameter in a moment).

此函数用于统计文件中的字符数。它接受两个参数：文件名和用于读取文件的缓冲区大小（稍后您就会明白第二个参数的作用）。

Before seeing how fast it is, you should test your library to make sure that it works (it does). Here's a simple test:

在测试速度之前，你应该先测试一下你的库，确保它能正常工作（它确实可以）。以下是一个简单的测试：

```
func TestFileLen(t *testing.T) {
    result, err := FileLen("testdata/data.txt", 1)
    if err != nil {
        t.Fatal(err)
    }
    if result != 65204 {
```



```
        t.Error("Expected 65204, got", result)
    }
}
```

Now you can see how long it takes your file-length function to run. Your goal is to find out what size buffer you should use to read from the file.

现在你可以看到文件长度计算函数运行需要多长时间。你的目标是找出应该使用多大的缓冲区来读取文件。

NOTE

笔记

Before you spend time going down an optimization rabbit hole, be sure that you need to optimize. If your program is already fast enough to meet your responsiveness requirements and is using an acceptable amount of memory, your time is better spent on adding features and fixing bugs. Your business requirements determine what “fast enough” and “acceptable amount of memory” mean.

在你花费大量时间钻研优化之前，务必确认你是否真的需要优化。如果你的程序速度已经足够快，满足响应速度要求，并且内存占用也在可接受的范围内，那么你应该把时间花在添加新功能和修复漏洞上。你的业务需求决定了“足够快”和“可接受的内存占用”的具体含义。

In Go, *benchmarks* are functions in your test files that start with the word **Benchmark** and take in a single parameter of type `*testing.B`. This type includes all the functionality of a `*testing.T` as well as additional support for benchmarking. Let’s start by looking at a benchmark that uses a buffer size of 1 byte:

在 Go 语言中，基准测试函数是测试文件中以 `Benchmark` 开头的函数，它接受一个 `*testing.B` 类型的参数。这种类型包含了 `*testing.T` 的所有功能，并额外支持基准测试。我们先来看一个缓冲区大小为 1 字节的基准测试示例：

```
var blackhole int

func BenchmarkFileLen1(b *testing.B) {
    for i := 0; i < b.N; i++ {
        result, err :=
FileLen("testdata/data.txt", 1)
        if err != nil {
            b.Fatal(err)
        }
        blackhole = result
    }
}
```

The `blackhole` package-level variable is interesting. You write the results from `FileLen` to this package-level variable to make sure that the compiler doesn't get too clever and decide to optimize away the call to `FileLen`, ruining your benchmark.

`blackhole` 包级变量很有意思。你需要将 `FileLen` 的结果写入这个包级变量，以确保编译器不会过于聪明地优化掉对 `FileLen` 的调用，从而破坏你的基准测试结果。

Every Go benchmark must have a loop that iterates from 0 to `b.N`. The testing framework calls your benchmark functions over and over with larger and larger values for `N` until it is sure that the timing results are accurate. You'll see this in the output in a moment.

每个 Go 基准测试都必须包含一个从 0 到 `b.N` 的循环。测试框架会不断调用你的基准测试函数，每次使用越来越大的 `N` 值，直到确认计时结果准确无误。你很快会在输出中看到这一点。

You run a benchmark by passing the `-bench` flag to `go test`. This flag expects a regular expression to describe the name of the benchmarks to run. Use `-bench=.` to run all benchmarks. A second flag, `-benchmem`, includes memory allocation information in the benchmark output. All tests are run before the benchmarks, so you can benchmark code only when tests pass.

您可以通过将 `-bench` 标志传递给 `go test` 来运行基准测试。此标志需要一个正则表达式来描述要运行的基准测试的名称。使用 `-bench=.` 可以运行所有基准测试。第二个标志 `-benchmem` 会将内存分配信息包含在基准测试的输出中。所有测试都会在基准测试之前运行，因此只有在测试通过后才能对代码进行基准测试。

Here's the output for the benchmark on my computer:

以下是我电脑上运行基准测试的输出结果：

```
BenchmarkFileLen1-12  25  47201025 ns/op  65342  
B/op  65208 allocs/op
```

Running a benchmark with memory allocation information produces output with five columns. Here's what each one means:

运行包含内存分配信息的基准测试会生成五列输出结果。以下是每一列的含义：

BenchmarkFileLen1-12

基准文件长度 1-12

The name of the benchmark, a hyphen, and the value of `GOMAXPROCS` for the benchmark.

基准测试的名称，一个连字符，以及基准测试的值 `GOMAXPROCS`。

25

The number of times that the test ran to produce a stable result.

为获得稳定结果而进行的测试运行次数。

47201025 ns/op

47201025 ns/op

The amount of time it took to run a single pass of this benchmark, in nanoseconds (there are 1,000,000,000 nanoseconds in a second).

运行此基准测试一次所需的时间，以纳秒为单位（1 秒等于 1,000,000,000 纳秒）。

65342 B/op

65342 B/op

The number of bytes allocated during a single pass of the benchmark.

基准测试单次运行期间分配的字节数。

65208 allocs/op

65208 次分配/操作

The number of times bytes had to be allocated from the heap during a single pass of the benchmark. This will always be less than or equal to the number of bytes allocated.

在基准测试单次运行期间，需要从堆中分配字节的次数。该值始终小于或等于已分配的字节数。

Now that you have results for a buffer of 1 byte, let's see what the results look like when you use buffers of different sizes:

现在你已经得到了 1 字节缓冲区的结果，让我们看看使用不同大小的缓冲区时结果会是什么样子：

```

func BenchmarkFileLen(b *testing.B) {
    for _, v := range []int{1, 10, 100, 1000,
10000, 100000} {
        b.Run(fmt.Sprintf("FileLen-%d", v), func(b
*testing.B) {
            for i := 0; i < b.N; i++ {
                result, err :=
FileLen("testdata/data.txt", v)
                if err != nil {
                    b.Fatal(err)
                }
                blackhole = result
            }
        })
    }
}

```

Just as you launched table tests using `t.Run`, you're using `b.Run` to launch benchmarks that vary based only on input. Here are the results of this benchmark on my computer:

就像您之前使用 `t.Run` 启动表格测试一样，您现在使用 `b.Run` 启动仅根据输入而变化的基准测试。以下是我电脑上该基准测试的结果：

```

BenchmarkFileLen/FileLen-1-12          25
47828842 ns/op   65342 B/op   65208 allocs/op

```

```
BenchmarkFileLen/FileLen-10-12      230
5136839 ns/op  104488 B/op   6525 allocs/op
BenchmarkFileLen/FileLen-100-12     2246
509619 ns/op   73384 B/op    657 allocs/op
BenchmarkFileLen/FileLen-1000-12    16491
71281 ns/op   68744 B/op     70 allocs/op
BenchmarkFileLen/FileLen-10000-12   42468
26600 ns/op   82056 B/op     11 allocs/op
BenchmarkFileLen/FileLen-100000-12  36700
30473 ns/op  213128 B/op      5 allocs/op
```

These results aren't surprising; as you increase the size of the buffer, you make fewer allocations and your code runs faster, until the buffer is bigger than the file. When the buffer is bigger than the size of the file, extra allocations slow the output. If you expect files of roughly this size, a buffer of 10,000 bytes would work best.

这些结果并不令人意外；随着缓冲区大小的增加，内存分配次数减少，代码运行速度加快，直到缓冲区大于文件大小。当缓冲区大于文件大小时，额外的内存分配会降低输出速度。如果预期文件大小大致如此，10,000 字节的缓冲区效果最佳。

But you can make a change that improves the numbers more. You are reallocating the buffer every time you get the next set of bytes from the file. That's unnecessary. If you move the byte slice allocation before the loop and rerun your benchmark, you see an improvement:

但你可以做一些改动来进一步提升性能。你每次从文件中获取下一组字节时都会重新分配缓冲区，这是不必要的。如果你把字节切片分配移到循环之前并重新运行基准测试，你会看到性能有所提升：

```
BenchmarkFileLen/FileLen-1-12          25
46167597 ns/op      137 B/op  4 allocs/op
BenchmarkFileLen/FileLen-10-12         261
4592019 ns/op      152 B/op  4 allocs/op
BenchmarkFileLen/FileLen-100-12        2518
478838 ns/op       248 B/op  4 allocs/op
BenchmarkFileLen/FileLen-1000-12       20059
60150 ns/op       1160 B/op  4 allocs/op
BenchmarkFileLen/FileLen-10000-12     62992
19000 ns/op      10376 B/op  4 allocs/op
BenchmarkFileLen/FileLen-100000-12   51928
21275 ns/op     106632 B/op  4 allocs/op
```

The number of allocations are now consistent and small, just four allocations for every buffer size. What is interesting is that you now can make trade-offs. If you are tight on memory, you can use a smaller buffer size and save memory at the expense of performance.

现在内存分配次数稳定且较少，每个缓冲区大小仅需分配四次。有趣的是，现在您可以进行权衡。如果内存紧张，可以使用较小的缓冲区大小来节省内存，但会牺牲一些性能。

PROFILING YOUR GO CODE

分析你的 GO 代码

If benchmarking reveals that you have a performance or memory problem, the next step is figuring out exactly what the problem is. Go includes profiling tools that gather CPU and memory usage data from a running program as well as tools that help you visualize and interpret the generated data. You can even expose a web service endpoint to gather profiling information remotely from a running Go service.

如果基准测试显示存在性能或内存问题，下一步就是找出问题的具体原因。Go 语言内置了性能分析工具，可以从运行中的程序收集 CPU 和内存使用数据，以及帮助您可视化和解读这些数据的工具。您甚至可以公开一个 Web 服务端点，以便从正在运行的 Go 服务远程收集性能分析信息。

Discussing the profiler is a topic that's beyond the scope of this book. Many great resources are available online. A good starting point is the blog post [“Profiling Go Programs with pprof”](#) by Julia Evans.

讨论性能分析器超出了本书的范围。网上有很多优秀的资源可供参考。Julia Evans 的博文“使用 pprof 分析 Go 程序”是一个很好的起点。

Using Stubs in Go

在 Go 中使用 Stubs

So far, you've written tests for functions that didn't depend on other code. This is not typical, as most code is filled with dependencies. As you saw in [Chapter 7](#), Go allows you to abstract function calls in two ways: defining a function type and defining an interface. These abstractions help you write not only modular production code but also unit tests.

到目前为止，你编写的测试都是针对不依赖于其他代码的函数。这并不常见，因为大多数代码都充满了依赖关系。正如你在第 7 章中看到的，Go 允许你通过两种方式抽象函数调用：定义函数类型和定义接口。这些抽象不仅有助于你编写模块化的生产代码，还有助于编写单元测试。

TIP

提示

When your code depends on abstractions, it's easier to write unit tests!

当你的代码依赖于抽象概念时，编写单元测试就更容易了！

Let's take a look at an example in the *sample_code/solver* directory in the [Chapter 15 repository](#). You define a type called `Processor` :

让我们来看一个位于第 15 章代码库 `sample_code/solver` 目录中的示例。您定义了一个名为 `Processor` 的类型：

```
type Processor struct {  
    Solver MathSolver  
}
```

It has a field of type `MathSolver` :

它有一个类型为 `MathSolver` 的字段：

```
type MathSolver interface {  
    Resolve(ctx context.Context, expression  
string) (float64, error)  
}
```

You'll implement and test `MathSolver` in a bit.

稍后您将实现并测试 `MathSolver` 。

`Processor` also has a method that reads an expression from an `io.Reader` and returns the calculated value:

`Processor` 还有一个方法，可以从 `io.Reader` 读取表达式并返回计算值：

```

func (p Processor) ProcessExpression(ctx
context.Context, r io.Reader)
                                (float64,
error) {
    curExpression, err := readToNewLine(r)
    if err != nil {
        return 0, err
    }
    if len(curExpression) == 0 {
        return 0, errors.New("no expression to
read")
    }
    answer, err := p.Solver.Resolve(ctx,
curExpression)
    return answer, err
}

```

Let's write the code to test `ProcessExpression`. First, you need a simple implementation of the `Resolve` method to write your test:

让我们编写代码来测试 `ProcessExpression`。首先，你需要一个简单的 `Resolve` 方法实现来编写测试：

```

type MathSolverStub struct{}

func (ms MathSolverStub) Resolve(ctx
context.Context, expr string)

```

```

                                                                    (float64, error) {
    switch expr {
    case "2 + 2 * 10":
        return 22, nil
    case "( 2 + 2 ) * 10":
        return 40, nil
    case "( 2 + 2 * 10":
        return 0, errors.New("invalid expression:
( 2 + 2 * 10")
    }
    return 0, nil
}

```

Next, you write a unit test that uses this stub (production code should test the error messages too, but for the sake of brevity, you'll leave those out):

接下来，编写一个使用此存根的单元测试（生产代码也应该测试错误消息，但为了简洁起见，这里省略这些内容）：

```

func TestProcessorProcessExpression(t *testing.T)
{
    p := Processor{MathSolverStub{}}
    in := strings.NewReader(`2 + 2 * 10
( 2 + 2 ) * 10
( 2 + 2 * 10`)
    data := []float64{22, 40, 0}
    hasErr := []bool{false, false, true}

```

```
    for i, d := range data {
        result, err :=
p.ProcessExpression(context.Background(), in)
        if err != nil && !hasErr[i] {
            t.Error(err)
        }
        if result != d {
            t.Errorf("Expected result %f, got %f",
d, result)
        }
    }
}
```

You can then run your test and see that everything works.

然后您可以运行测试，看看一切是否正常。

While most Go interfaces specify only one or two methods, this isn't always the case. You sometimes find yourself with an interface that has many methods. Let's take a look at the code in the *sample_code/stub* directory in the [Chapter 15 repository](#). Assume you have an interface that looks like this:

虽然大多数 Go 接口只定义一到两个方法，但并非总是如此。有时你会发现接口包含很多方法。让我们来看一下第 15 章代码库中 *sample_code/stub* 目录下的代码。假设你有一个如下所示的接口：

```
type Entities interface {
    GetUser(id string) (User, error)
    GetPets(userID string) ([]Pet, error)
    GetChildren(userID string) ([]Person, error)
    GetFriends(userID string) ([]Person, error)
    SaveUser(user User) error
}
```

There are two patterns for testing code that depends on large interfaces. The first is to embed the interface in a struct. Embedding an interface in a struct automatically defines all the interface's methods on the struct. It doesn't provide any implementations of those methods, so you need to implement the methods that you care about for the current test. Let's assume that `Logic` is a struct that has a field of type `Entities`:

对于依赖于大型接口的代码，有两种测试模式。第一种是将接口嵌入到结构体中。将接口嵌入结构体会自动在结构体中定义接口的所有方法。但它不会提供这些方法的任何实现，因此您需要为当前测试实现您关心的方法。假设 `Logic` 是一个结构体，它有一个类型为 `Entities` 的字段：

```
type Logic struct {
    Entities Entities
}
```

Assume you want to test this method:

假设你想测试这个方法：

```
func (l Logic) GetPetNames(userId string)
([]string, error) {
    pets, err := l.Entities.GetPets(userId)
    if err != nil {
        return nil, err
    }
    out := make([]string, len(pets))
    for _, p := range pets {
        out = append(out, p.Name)
    }
    return out, nil
}
```

This method uses only one of the methods declared on `Entities`: `GetPets`. Rather than creating a stub that implements every single method on `Entities` just to test `GetPets`, you can write a stub struct that implements only the method you need to test this method:

此方法仅使用 `Entities` 上声明的方法之一：`GetPets`。与其为了测试 `GetPets` 而创建一个实现 `Entities` 上所有方法的存根，不如编写一个仅实现测试此方法所需方法的存根结构：


```

type GetPetNamesStub struct {
    Entities

}

func (ps GetPetNamesStub) GetPets(userID string)
([]Pet, error) {
    switch userID {
    case "1":
        return []Pet{{Name: "Bubbles"}}, nil
    case "2":
        return []Pet{{Name: "Stampy"}, {Name:
"Snowball II"}}, nil
    default:
        return nil, fmt.Errorf("invalid id: %s",
userID)
    }
}

```

You then write your unit test, with your stub injected into **Logic** :

然后编写单元测试，并将存根注入到 **Logic** 中：

```

func TestLogicGetPetNames(t *testing.T) {
    data := []struct {
        name      string
        userID    string
        petNames []string
    }
}

```

```

    }{
        {"case1", "1", []string{"Bubbles"}},
        {"case2", "2", []string{"Stampy",
"Snowball II"}},
        {"case3", "3", nil},
    }
    l := Logic{GetPetNamesStub{}}
    for _, d := range data {
        t.Run(d.name, func(t *testing.T) {
            petNames, err :=
l.GetPetNames(d.userID)
            if err != nil {
                t.Error(err)
            }
            if diff := cmp.Diff(d.petNames,
petNames); diff != "" {
                t.Error(diff)
            }
        })
    }
}
}

```

(By the way, the `GetPetNames` method has a bug. Did you see it? Even simple methods can sometimes have bugs.)

(顺便说一下，`GetPetNames` 方法有个 bug。你发现了吗？即使是简单的方法有时也可能出现 bug。)

WARNING

警告

If you embed an interface in a stub struct, make sure you provide an implementation for every method that's called during your test! If you call an unimplemented method, your tests will panic.

如果将接口嵌入到存根结构体中，请确保为测试期间调用的每个方法都提供实现！如果调用了未实现的方法，测试将会崩溃。

If you need to implement only one or two methods in an interface for a single test, this technique works well. The drawback comes when you need to call the same method in different tests with different inputs and outputs. When that happens, you need to either include every possible result for every test within the same implementation or reimplement the struct for each test. This quickly becomes difficult to understand and maintain. A better solution is to create a stub struct that proxies method calls to function fields. For each method defined on `Entities`, you define a function field with a matching signature on your stub struct:

如果只需要在单个测试中实现接口中的一两个方法，这种方法效果很好。但缺点在于，当需要在不同的测试中使用不同的输入输出调用同一个方法时，要么需要在同一个实现中包含每个测试的所有可能结果，要么需要为每个测试重新实现结构体。这很快就会变得难以理解和维护。更好的解决方案是创建一个存根结构体，将方法调用代理到函数字段。对于 `Entities` 上定义的每个方法，在存根结构体中定义一个具有匹配签名的函数字段：

```

type EntitiesStub struct {
    getUser      func(id string) (User, error)
    getPets      func(userID string) ([]Pet, error)
    getChildren  func(userID string) ([]Person,
error)
    getFriends   func(userID string) ([]Person,
error)
    saveUser     func(user User) error
}

```

You then make `EntitiesStub` meet the `Entities` interface by defining the methods. In each method, you invoke the associated function field. For example:

然后，通过定义方法使 `EntitiesStub` 符合 `Entities` 接口。在每个方法中，调用关联的函数字段。例如：

```

func (es EntitiesStub) GetUser(id string) (User,
error) {
    return es.getUser(id)
}

func (es EntitiesStub) GetPets(userID string)
([]Pet, error) {
    return es.getPets(userID)
}

```

Once you create this stub, you can supply different implementations of different methods in different test cases via the fields in the data struct for a table test:

创建此存根后，您可以通过表测试的数据结构中的字段，在不同的测试用例中提供不同方法的不同实现：

```
func TestLogicGetPetNames(t *testing.T) {
    data := []struct {
        name      string
        getPets   func(userID string) ([]Pet,
error)
        userID   string
        petNames []string
        errMsg   string
    }{
        {"case1", func(userID string) ([]Pet,
error) {
            return []Pet{{Name: "Bubbles"}}, nil
        }, "1", []string{"Bubbles"}, ""},
        {"case2", func(userID string) ([]Pet,
error) {
            return nil, errors.New("invalid id:
3")
        }, "3", nil, "invalid id: 3"}},
    }
    l := Logic{}
```

```

    for _, d := range data {
        t.Run(d.name, func(t *testing.T) {
            l.Entities = EntitiesStub{getPets:
d.getPets}

            petNames, err :=
l.GetPetNames(d.userID)
            if diff := cmp.Diff(d.petNames,
petNames); diff != "" {
                t.Error(diff)
            }
            var errMsg string
            if err != nil {
                errMsg = err.Error()
            }
            if errMsg != d.errMsg {
                t.Errorf("Expected error `%s`, got
`s`, d.errMsg, errMsg)
            }
        })
    }
}

```

You add a field of function type to `data`'s anonymous struct. In each test case, you specify a function that returns the data that `GetPets` would return. When you write your test stubs this way, it's clear what the stubs should return for each test case. As each test runs, you instantiate a new

`EntitiesStub` and assign the `getPets` function field in your test data to the `getPets` function field in `EntitiesStub`.

您向 `data` 的匿名结构体添加一个函数类型的字段。在每个测试用例中，您指定一个函数，该函数返回 `GetPets` 会返回的数据。这样编写测试桩时，每个测试用例的桩应该返回什么就一目了然了。每次测试运行时，您实例化一个新的 `EntitiesStub`，并将测试数据中的 `getPets` 函数字段赋值给 `EntitiesStub` 中的 `getPets` 函数字段。

MOCKS AND STUBS

模拟稿和存根

The terms *mock* and *stub* are often used interchangeably, but they are actually two different concepts. Martin Fowler, a respected voice on all things related to software development, wrote a [blog post](#) on mocks that, among other things, covers the difference between mocks and stubs. In short, a *stub* returns a canned value for a given input, whereas a *mock* validates that a set of calls happen in the expected order with the expected inputs.

mock 和 stub 这两个术语经常被混用，但实际上它们是两个不同的概念。Martin Fowler 是一位在软件开发领域备受尊敬的专家，他撰写了一篇关于 mock 的博文，其中就阐述了 mock 和 stub 之间的区别。简而言之，stub 会针对给定的输入返回一个预设值，而 mock 则会验证一系列调用是否按照预期的顺序和输入执行。

You used stubs in the examples to return canned values to a given response. You can write your own mocks by hand, or you can use a third-party library to generate them. The two most popular options are the [gomock](#) library from Google and the [testify](#) library from Stretchr, Inc.

在示例中，你使用了存根（stub）来向给定的响应返回预设值。你可以手动编写模拟对象，也可以使用第三方库来生成它们。最常用的两个库分别是 Google 的 gomock 库和 Stretchr 公司的 testify 库。

Using httptest

使用 httptest

It can be difficult to write tests for a function that calls an HTTP service. Traditionally, this became an integration test, requiring you to stand up a test instance of the service that the function calls. The Go standard library includes the `net/http/httptest` package to make it easier to stub HTTP services. Let's go back to the `sample_code/solver` directory in the [Chapter 15 repository](#) and provide an implementation of `MathSolver` that calls an HTTP service to evaluate expressions:

为调用 HTTP 服务的函数编写测试可能比较困难。传统上，这需要进行集成测试，也就是搭建一个函数调用服务的测试实例。Go 标准库包含了 `net/http/httptest` 包，可以更轻松地模拟 HTTP 服务。让我们回到第 15 章代码库中的 `sample_code/solver` 目录，并提供一个 `MathSolver` 的实现，该实现会调用 HTTP 服务来计算表达式：

```
type RemoteSolver struct {
    MathServerURL string
    Client         *http.Client
}

func (rs RemoteSolver) Resolve(ctx
```

```

context.Context, expression string)
                                (float64, error) {
    req, err := http.NewRequestWithContext(ctx,
http.MethodGet,
    rs.MathServerURL+"?
expression="+url.QueryEscape(expression),
    nil)
    if err != nil {
        return 0, err
    }
    resp, err := rs.Client.Do(req)
    if err != nil {
        return 0, err
    }
    defer resp.Body.Close()
    contents, err := io.ReadAll(resp.Body)
    if err != nil {
        return 0, err
    }
    if resp.StatusCode != http.StatusOK {
        return 0, errors.New(string(contents))
    }
    result, err :=
strconv.ParseFloat(string(contents), 64)
    if err != nil {
        return 0, err
    }
}

```

```
    return result, nil
}
```

Now let's see how to use the `httptest` library to test this code without standing up a server. The code is in the `TestRemoteSolver_Resolve` function in `sample_code/solver/remote_solver_test.go` in the [Chapter 15 repository](#), but here are the highlights. First, you want to make sure that the data that's passed into the function arrives on the server. So in your test function, you define a type called `info` to hold your input and output and a variable called `io` that is assigned the current input and output:

现在我们来看看如何使用 `httptest` 库在不搭建服务器的情况下测试这段代码。代码位于第 15 章代码库中的 `sample_code/solver/remote_solver_test.go` 文件中的 `TestRemoteSolver_Resolve` 函数里，但这里我们来看一下重点。首先，你需要确保传递给函数的数据能够到达服务器。因此，在你的测试函数中，你需要定义一个名为 `info` 的类型来保存输入和输出，以及一个名为 `io` 的变量，该变量被赋值为当前的输入和输出：

```
type info struct {
    expression string
    code       int
    body       string
}
```

```
}  
var io info
```

Next, you set up your fake remote server and use it to configure an instance of `RemoteSolver`:

接下来，您需要设置虚拟远程服务器，并使用它来配置 `RemoteSolver` 的一个实例：

```
server := httptest.NewServer(  
    http.HandlerFunc(func(rw http.ResponseWriter,  
req *http.Request) {  
        expression :=  
req.URL.Query().Get("expression")  
        if expression != io.expression {  
            rw.WriteHeader(http.StatusBadRequest)  
            fmt.Fprintf(rw, "expected expression  
'%s', got '%s'",  
                io.expression, expression)  
            return  
        }  
        rw.WriteHeader(io.code)  
        rw.Write([]byte(io.body))  
    })  
)  
defer server.Close()  
rs := RemoteSolver{  
    MathServerURL: server.URL,
```

```
    Client:      server.Client(),  
  }
```

The `httptest.NewServer` function creates and starts an HTTP server on a random unused port. You need to provide an `http.Handler` implementation to process the request. Since this is a server, you must close it when the test completes. The `server` instance has its URL specified in the `URL` field of the `server` instance and a preconfigured `http.Client` for communicating with the test server. You pass these into `RemoteSolver`.

`httptest.NewServer` 函数会在一个随机的未使用端口上创建并启动一个 HTTP 服务器。你需要提供一个 `http.Handler` 实现来处理请求。由于这是一个服务器，测试完成后必须将其关闭。`server` 实例的 URL 已在 `server` 实例的 `URL` 字段中指定，并且预配置了一个 `http.Client` 用于与测试服务器通信。你需要将这些信息传递给 `RemoteSolver`。

The rest of the function works like every other table test that you've seen:

该函数的其余部分与您之前见过的所有其他表格测试一样：

```
data := []struct {  
    name    string  
    io      info
```

```

        result float64
    }{
        {"case1", info{"2 + 2 * 10", http.StatusOK,
"22"}, 22},
        // remaining cases
    }
    for _, d := range data {
        t.Run(d.name, func(t *testing.T) {
            io = d.io
            result, err :=
rs.Resolve(context.Background(), d.io.expression)
            if result != d.result {
                t.Errorf("io `%f`, got `%f`",
d.result, result)
            }
            var errMsg string
            if err != nil {
                errMsg = err.Error()
            }
            if errMsg != d.errMsg {
                t.Errorf("io error `%s`, got `%s`",
d.errMsg, errMsg)
            }
        })
    }
}

```

The interesting thing to note is that the variable `io` has been captured by two closures: the one for the stub server and the one for running each test. You write to it in one closure and read it in the other. This is a bad idea in production code, but it works well in test code within a single function.

值得注意的是，变量 `io` 被两个闭包捕获：一个用于存根服务器，另一个用于运行每个测试。你在一个闭包中写入它，在另一个闭包中读取它。这在生产代码中是一个糟糕的做法，但在单个函数内的测试代码中却行之有效。

Using Integration Tests and Build Tags

使用集成测试和构建标签

Even though `httptest` provides a way to avoid testing against external services, you should still write *integration tests*, automated tests that connect to other services. These validate that your understanding of the service's API is correct. The challenge is figuring out how to group your automated tests; you want to run integration tests only when the support environment is present. Also, integration tests tend to be slower than unit tests, so they are usually run less frequently.

即使 `httptest` 提供了一种避免测试外部服务的方法，您仍然应该编写集成测试，即连接到其他服务的自动化测试。这些测试可以验证您

对服务 API 的理解是否正确。难点在于如何对自动化测试进行分组；您应该仅在支持环境存在时才运行集成测试。此外，集成测试通常比单元测试慢，因此它们的运行频率通常较低。

In [“Using Build Tags”](#), I covered build tags, which are used by the Go compiler to control when a file is compiled. While they are primarily intended to allow developers to write code that’s intended only for a specific operating system, CPU, or Go version, you can take advantage of the ability to specify custom build tags to control when integration tests are compiled and run.

在“使用构建标签”一文中，我介绍了构建标签，Go 编译器使用构建标签来控制文件的编译时间。虽然构建标签的主要目的是允许开发者编写仅适用于特定操作系统、CPU 或 Go 版本的代码，但您也可以利用自定义构建标签来控制集成测试的编译和运行时间。

Let’s try this out with the math-solving project. Use [Docker](#) to download a server implementation with `docker pull jonbodner/math-server` and then run the server locally on port 8080 with `docker run -p 8080:8080 jonbodner/math-server`.

让我们用数学求解项目来尝试一下。使用 Docker 下载服务器实现 `docker pull jonbodner/math-server`，然后在本地端口 8080 上运行服务器 `docker run -p 8080:8080 jonbodner/math-server`。

NOTE

笔记

If you don't have Docker installed or if you want to build the code for yourself, you can find it on [GitHub](#).

如果您没有安装 Docker，或者您想自己构建代码，您可以在 GitHub 上找到它。

You need to write an integration test to make sure that your `Resolve` method properly communicates with the math server. The `sample_code/solver/remote_solver_integration_test.go` file in the [Chapter 15 repository](#) has a complete test in the `TestRemoteSolver_ResolveIntegration` function. The test looks like every other table test that you've written. The interesting thing is the first line of the file, separated from the package declaration by a newline:

你需要编写一个集成测试，以确保你的 `Resolve` 方法能够与数学服务器正确通信。第 15 章代码库中的 `sample_code/solver/remote_solver_integration_test.go` 文件在 `TestRemoteSolver_ResolveIntegration` 函数中包含一个完整的测试。该测试看起来与你编写的其他表格测试类似。有趣的是文件的第一行，它与包声明之间用换行符分隔：

```
//go:build integration
```

To run your integration test alongside the other tests you've written, use this:

要将集成测试与其他已编写的测试一起运行，请使用以下命令：

```
$ go test -tags integration -v ./...
```

You should be aware that some people in the Go community prefer to use environment variables rather than build tags to create groups of integration tests. Peter Bourgon describes the concept in a blog post with the uns subtle title [“Don't Use Build Tags for Integration Tests”](#). His argument is that it's hard to find out what build tags to set in order to run integration tests. An explicit check for an environment variable in each integration test, combined with a detailed message in a `t.Skip` method call, makes it clear that tests aren't being run and how to run them. In the end, it's a trade-off between verbosity and discoverability. Feel free to use either technique.

您应该知道，Go 社区中有些人更倾向于使用环境变量而不是构建标签来创建集成测试组。Peter Bourgon 在一篇标题直白的博文“不要为集成测试使用构建标签”中描述了这一概念。他的理由是，很难找到运行集成测试需要设置哪些构建标签。在每个集成测试中显式检查环境变量，并在 `t.Skip` 方法调用中提供详细信息，可以清楚地表明哪些测试没有运行以及如何运行它们。最终，这是一种在冗长性和可发现性之间的权衡。您可以随意选择使用哪种方法。

USING THE -SHORT FLAG

使用 -SHORT 标志

Yet another way to group tests is to use `go test` with the `-short` flag. If you want to skip over tests that take a long time, label your slow tests by placing the following code at the start of the test function:

另一种对测试进行分组的方法是使用 `go test` 和 `-short` 标志。如果您想跳过耗时较长的测试，请在测试函数开头添加以下代码来标记这些耗时较长的测试：

```
if testing.Short() {  
    t.Skip("skipping test in short mode.")  
}
```

When you want to run only short tests, pass the `-short` flag to `go test`.

当您只想运行短测试时，请将 `-short` 标志传递给 `go test`。

There are a few problems with the `-short` flag. If you use it, there are only two levels of testing: short tests and all tests. By using build tags, you can group your integration tests, specifying which service they need in order to run. Another argument against using the `-short` flag to indicate integration tests is philosophical. Build tags indicate a dependency, while the `-short` flag is meant to indicate only that you don't want to run tests that take a long time. Those are different concepts. Finally, I find the `-`

`short` flag unintuitive. You should run short tests all the time. It makes more sense to require a flag to *include* long-running tests, not to *exclude* them.

`-short` 标志存在一些问题。如果使用它，测试就只有两个级别：短测试和所有测试。通过使用构建标签，您可以将集成测试分组，并指定它们运行所需的特定服务。反对使用 `-short` 标志来指示集成测试的另一个理由是理念上的。构建标签表示依赖关系，而 `-short` 标志仅用于指示您不想运行耗时过长的测试。这是不同的概念。最后，我认为 `-short` 标志不够直观。您应该始终运行短测试。更合理的做法是使用一个标志来包含耗时较长的测试，而不是排除它们。

Finding Concurrency Problems with the Data Race Detector

使用数据竞争检测器查找并发问题

Even with Go's built-in support for concurrency, bugs still happen. It's easy to accidentally reference a variable from two different goroutines without acquiring a lock. The computer science term for this is a *data race*. To help find these sorts of bugs, Go includes a *race checker*. It isn't guaranteed to find every single data race in your code, but if it finds one, you should put proper locks around what it finds.

即使 Go 内置了对并发的支持，bug 仍然会发生。很容易在两个不同的 goroutine 中意外地引用同一个变量，而没有加锁。计算机科学中称之为数据竞争。为了帮助查找这类 bug，Go 包含了一个竞争检查器。它并不能保证找到代码中的所有数据竞争，但如果它发现了一个，你应该对它发现的数据竞争进行适当的加锁。

Look at a simple example in the file *sample_code/race/race.go* in the [Chapter 15 repository](#):

请查看第 15 章代码库中 *sample_code/race/race.go* 文件中的简单示例：

```
func getCounter() int {
    var counter int
    var wg sync.WaitGroup
    wg.Add(5)
    for i := 0; i < 5; i++ {
        go func() {
            for i := 0; i < 1000; i++ {
                counter++
            }
            wg.Done()
        }()
    }
    wg.Wait()
    return counter
}
```

This code launches five goroutines, has each of them update a shared `counter` variable 1,000 times, and then returns the result. You'd expect it to be 5,000, so let's verify this with a unit test in `sample_code/race/race_test.go`:

这段代码启动了五个 goroutine，每个 goroutine 都更新一个共享的 `counter` 变量 1000 次，然后返回结果。你预期结果是 5000，所以让我们在 `sample_code/race/race_test.go` 中使用单元测试来验证这一点：

```
func TestGetCounter(t *testing.T) {  
    counter := getCounter()  
    if counter != 5000 {  
        t.Error("unexpected counter:", counter)  
    }  
}
```

If you run `go test` a few times, you'll see that sometimes it passes, but most of the time it fails with an error message like this:

如果你多次运行 `go test`，你会发现有时它会通过，但大多数时候会失败，并出现类似这样的错误信息：

```
unexpected counter: 3673
```

The problem is that there's a data race in the code. In a program this simple, the cause is obvious: multiple goroutines are trying to update `counter` simultaneously, and some of their updates are lost. In more complicated programs, these sorts of races are harder to see. Let's see what the data race detector does. Use the flag `-race` with `go test` to enable it:

问题在于代码中存在数据竞争。在如此简单的程序中，原因显而易见：多个 goroutine 同时尝试更新 `counter`，导致部分更新丢失。在更复杂的程序中，这类竞争问题则更难发现。让我们看看数据竞争检测器是如何工作的。使用标志 `-race` 和 `go test` 来启用它：

```
$ go test -race
=====
WARNING: DATA RACE
Read at 0x00c000128070 by goroutine 10:
    test_examples/race.getCounter.func1()
        test_examples/race/race.go:12 +0x45

Previous write at 0x00c000128070 by goroutine 8:
    test_examples/race.getCounter.func1()
        test_examples/race/race.go:12 +0x5b
```

The traces make it clear that the line `counter++` is the source of your problems.

跟踪信息清楚地表明, `counter++` 这行代码是问题的根源。

WARNING

警告

Some people try to fix race conditions by inserting “sleeps” into their code, trying to space out access to the variable that’s being accessed by multiple goroutines. *This is a bad idea.* Doing so might eliminate the problem in some cases, but the code is still wrong and will fail in some situations.

有些人试图通过在代码中插入“sleep”语句来解决竞态条件, 试图拉开多个goroutine对同一变量访问的间隔。但这并非明智之举。这样做或许能在某些情况下消除问题, 但代码本身仍然存在缺陷, 在某些情况下仍然会失败。

You can also use the `-race` flag when you build your programs. This creates a binary that includes the data race detector and that reports any races it finds to the console. This allows you to find data races in code that doesn’t have tests.

您也可以在构建程序时使用 `-race` 标志。这将创建一个包含数据竞争检测器的二进制文件, 并将找到的任何竞争情况报告到控制台。这样, 您就可以在没有测试的代码中查找数据竞争。

If the data race detector is so useful, why isn’t it enabled all the time for testing and production? A binary with `-race` enabled runs approximately 10 times slower than a normal binary. That isn’t a problem for test suites that take a second to run, but for large test suites that take several minutes, a 10× slowdown reduces productivity.

如果数据竞争检测器如此有用，为什么不在测试和生产环境中始终启用它呢？启用 `-race` 的二进制文件运行速度比普通二进制文件慢大约 10 倍。对于只需一秒钟即可运行的测试套件来说，这不算什么问题，但对于需要几分钟才能运行的大型测试套件来说，10 倍的速度下降会降低生产效率。

For more information on the data race detector, check out its [official documentation](#).

有关数据竞争检测器的更多信息，请查看其官方文档。

Exercises

练习

Now that you've learned about writing tests and using the code-quality tools included with Go, complete these exercises to apply that knowledge to a sample application.

现在你已经学习了如何编写测试以及如何使用 Go 语言自带的代码质量工具，请完成以下练习，将这些知识应用到示例应用程序中。

1. [Download the Simple Web App program](#). Write unit tests for the program and get as close to 100% code coverage as you can. If you find any bugs, fix them.

下载 Simple Web App 程序。为该程序编写单元测试，并尽可能达到 100% 的代码覆盖率。如果发现任何错误，请修复它们。

2. Use the race detector to find a concurrency problem in the program and fix it.

使用竞争检测器查找程序中的并发问题并修复它。

3. Write a fuzz test against the `parser` function and fix any problems that you find.

针对 `parser` 函数编写模糊测试，并修复发现的任何问题。

Wrapping Up

总结

In this chapter, you've learned how to write tests and improve code quality by using Go's built-in support for testing, code coverage, benchmarking, fuzzing, and data race checking. In the next chapter, you're going to explore some Go features that allow you to break the rules: the `unsafe` package, reflection, and `cgo`.

在本章中，您学习了如何编写测试并利用 Go 内置的测试、代码覆盖率、基准测试、模糊测试和数据竞争检查功能来提高代码质量。在下一章中，您将探索一些允许您打破常规的 Go 特性：`unsafe` 包、反射和 `cgo`。

Chapter 16. Here Be Dragons: Reflect, Unsafe, and Cgo

第十六章 此处有龙：反思、不安全与 Cgo

The edges of the known world are scary. Ancient maps would fill in the unexplored areas with pictures of dragons and lions. In the previous sections, I have emphasized that Go is a safe language, with typed variables to make clear what sort of data you are using and garbage collection to manage memory. Even the pointers are tame; you can't abuse them in the way that C and C++ do.

已知世界的边缘令人恐惧。古代地图会用龙和狮子的图案来描绘未知的区域。在前面的章节中，我强调了 Go 是一种安全的语言，它使用类型化的变量来明确你正在使用的数据类型，并有垃圾回收机制来管理内存。即使是指针也很温和；你无法像 C 和 C++ 那样滥用它们。

All those things are true, and for the vast majority of the Go code that you'll write, you can be assured that the Go runtime will protect you. But there are escape hatches. Sometimes your Go programs need to venture out into less defined areas. In this chapter, you're going to look at how to handle situations that can't be solved with normal Go code. For example, when the

type of the data can't be determined at compile time, you can use the reflection support in the `reflect` package to interact with and even construct data. When you need to take advantage of the memory layout of data types in Go, you can use the `unsafe` package. And if there is functionality that can be provided only by libraries written in C, you can call into C code with `cgo`.

以上所述皆是事实，对于您编写的绝大多数 Go 代码，您可以放心，Go 运行时会为您提供保护。但并非没有例外。有时，您的 Go 程序需要探索一些定义较为模糊的领域。本章将介绍如何处理那些无法用常规 Go 代码解决的情况。例如，当编译时无法确定数据类型时，您可以使用 `reflect` 包中的反射支持来与数据交互，甚至构造数据。当您需要利用 Go 中数据类型的内存布局时，可以使用 `unsafe` 包。如果某些功能只能由 C 语言编写的库提供，您可以使用 `cgo` 包调用 C 代码来实现。

You might be wondering why these advanced concepts appear in a book targeted at those new to Go. There are two reasons. First, developers searching for a solution to a problem sometimes discover (and copy and paste) techniques they don't fully understand. It's best to know a bit about advanced techniques that can cause problems before you add them to your codebase. Second, these tools are fun. Because they allow you to do things that aren't normally possible with Go, it feels a bit exciting to play with them and see what you can do.

你或许会好奇，为什么这本面向 Go 新手的书中会出现这些高级概念。原因有二。首先，开发者在寻找问题解决方案时，有时会发现（并直接复制粘贴）一些他们并不完全理解的技术。最好在将这些高级技术添加到代码库之前，先了解一些可能导致问题的原理。其次，这些工具本身就很有趣。因为它们能让你用 Go 实现一些通常无法实现的功能，所以尝试使用它们，看看自己能做到什么，会让人感到兴奋。

Reflection Lets You Work with Types at Runtime

反射功能允许你在运行时处理类型

One of the things that people who use Go like about it is that it is a statically typed language. Most of the time, declaring variables, types, and functions in Go is pretty straightforward. When you need a type, a variable, or a function, you define it:

Go语言的一大优点是它是一种静态类型语言。大多数情况下，在Go中声明变量、类型和函数都非常简单直接。当你需要某种类型、变量或函数时，只需定义即可：

```
type Foo struct {  
    A int  
    B string  
}  
  
var x Foo  
  
func DoSomething(f Foo) {  
    fmt.Println(f.A, f.B)  
}
```

You use types to represent the data structures you know you need when you write your programs. Since types are a core part of Go, the compiler uses them to make sure that your code is correct. But sometimes, relying on only compilation-time information is a limitation. You might need to work with variables at runtime using information that didn't exist when the program was written. Maybe you're trying to map data from a file or network request into a variable. In those situations, you need to use *reflection*. Reflection allows you to examine types at runtime. It also provides the ability to examine, modify, and create variables, functions, and structs at runtime.

在编写程序时，你会使用类型来表示你已知需要的数据结构。由于类型是 Go 的核心组成部分，编译器会使用类型来确保代码的正确性。但有时，仅仅依赖编译时信息会存在局限性。你可能需要使用在程序

编写时并不存在的信息来操作运行时变量。例如，你可能试图将文件或网络请求中的数据映射到一个变量中。在这些情况下，你需要使用反射。反射允许你在运行时检查类型。它还提供了在运行时检查、修改和创建变量、函数和结构体的功能。

This leads to the question of when this functionality is needed. If you look at the Go standard library, you can get an idea. Its uses fall into one of a few general categories:

这就引出了一个问题：何时需要这种功能？如果你查看 Go 标准库，就能找到答案。它的用途大致可以分为以下几类：

- Reading and writing from a database. The `database/sql` package uses reflection to send records to databases and read data back.
从数据库读取和写入数据。`database/sql` 包使用反射将记录发送到数据库并从中读取数据。
- Go's built-in templating libraries, `text/template` and `html/template`, use reflection to process the values that are passed to the templates.
Go 的内置模板库 `text/template` 和 `html/template` 使用反射来处理传递给模板的值。
- The `fmt` package uses reflection heavily, as all those calls to `fmt.Println` and friends rely on reflection to detect the type of the provided parameters.

`fmt` 包大量使用反射，因为所有对 `fmt.Println` 及其相关函数的调用都依赖于反射来检测所提供参数的类型。

- The `errors` package uses reflection to implement `errors.Is` and `errors.As`.
`errors` 包使用反射来实现 `errors.Is` 和 `errors.As`。
- The `sort` package uses reflection to implement functions that sort and evaluate slices of any type: `sort.Slice`, `sort.SliceStable`, and `sort.SliceIsSorted`.
`sort` 包使用反射来实现对任何类型的切片进行排序和评估的函数: `sort.Slice`、`sort.SliceStable` 和 `sort.SliceIsSorted`。
- The last main usage of reflection in the Go standard library is for marshaling and unmarshaling data into JSON and XML, along with the other data formats defined in the various `encoding` packages. Struct tags (which I will talk about soon) are accessed via reflection, and the fields in structs are read and written using reflection as well. Go 标准库中反射的最后一个主要用途是将数据序列化和反序列化为 JSON 和 XML，以及各种 `encoding` 包中定义的其他数据格式。结构体标签（我稍后会详细介绍）通过反射访问，结构体中的字段也通过反射进行读写。

Most of these examples have one thing in common: they involve accessing and formatting data that is being imported into or exported out of a Go program. You'll often see reflection used at the boundaries between your program and the outside world.

这些例子大多有一个共同点：它们都涉及访问和格式化导入或导出到 Go 程序的数据。你经常会看到在程序与外部世界之间的边界处使用反射。

As you take a look at the techniques you can implement with reflection, keep in mind that this power has a price. Using reflection is quite a bit slower than performing the identical operation without it. I'll talk more about this in [“Use Reflection Only if It's Worthwhile”](#). Even more importantly, code that uses reflection is both more fragile and more verbose. Many functions and methods in the `reflect` package panic when passed the wrong type of data. Be sure to leave comments in your code to explain what you are doing so that it's clear to future reviewers (including yourself).

在了解反射的各种实现技巧时，请记住，这种强大的功能是有代价的。使用反射会比不使用反射执行相同的操作慢得多。我将在“仅在必要时使用反射”一文中详细讨论这一点。更重要的是，使用反射的代码更加脆弱且冗长。`reflect` 包中的许多函数和方法在传递错误类型的数据时会引发 panic。请务必在代码中添加注释来解释您的操作，以便未来的代码审查者（包括您自己）能够清楚地理解。

NOTE

笔记

Another use of the `reflect` package in the Go standard library is testing. In [“Slices”](#), I mentioned a function that you can find in the `reflect` package called `DeepEqual`. It’s in the `reflect` package because it takes advantage of reflection to do its work. The `reflect.DeepEqual` function checks whether two values are “deeply equal” to each other. This is a more thorough comparison than what you get if you use `==` to compare two things, and it’s used in the standard library as a way to validate test results. It can also compare things that can’t be compared using `==`, like slices and maps. Most of the time, you don’t need `DeepEqual`. Since the release of Go 1.21, it’s faster to use `slices.Equal` and `maps.Equal` to check for equality in slices and maps.

Go 标准库中的 `reflect` 包的另一个用途是测试。在“切片”一节中，我提到了 `reflect` 包中的一个名为 `DeepEqual` 的函数。它之所以位于 `reflect` 包中，是因为它利用了反射来实现其功能。`reflect.DeepEqual` 函数检查两个值是否“深度相等”。这比使用 `==` 比较两个对象更彻底，它在标准库中被用作验证测试结果的一种方式。它还可以比较一些无法使用 `==` 进行比较的对象，例如切片和映射。大多数情况下，你不需要 `DeepEqual`。自 Go 1.21 发布以来，使用 `slices.Equal` 和 `maps.Equal` 检查切片和映射中的相等性速度更快。

Types, Kinds, and Values

类型、种类和值

Now that you know what reflection is and when you might need it, let’s see how it works. The `reflect` package in the standard library is the home for the types and functions that implement reflection in Go. Reflection is built around three core concepts: types, kinds, and values.

既然您已经了解了反射的概念以及何时需要用到它，接下来让我们看看它是如何工作的。标准库中的 `reflect` 包包含了 Go 语言中实现反射的类型和函数。反射的核心概念有三个：类型、种类和值。

Types and kinds

类型和种类

A *type* is exactly what it sounds like. It defines the properties of a variable, what it can hold, and how you can interact with it. With reflection, you are able to query a type to find out about these properties using code.

类型正如其名，它定义了变量的属性、可以存储的值以及如何与它交互。通过反射，你可以使用代码查询类型，从而了解这些属性。

You get the reflection representation of the type of a variable with the `TypeOf` function in the `reflect` package:

您可以使用 `reflect` 包中的 `TypeOf` 函数获取变量类型的反射表示：

```
vType := reflect.TypeOf(v)
```

The `reflect.TypeOf` function returns a value of type `reflect.Type`, which represents the type of the variable passed into the `TypeOf` function.

The `reflect.Type` type defines methods with information about a variable's type. I can't cover all the methods, but here are a few.

`reflect.TypeOf` 函数返回一个 `reflect.Type` 类型的值，该值表示传递给 `TypeOf` 函数的变量的类型。`reflect.Type` 类型定义了包含变量类型信息的各种方法。我无法一一介绍所有方法，但这里仅列举几个。

The `Name` method returns, not surprisingly, the name of the type. Here is a quick example:

`Name` 方法返回的正是类型名称，这并不令人意外。以下是一个简单的示例：

```
var x int
xt := reflect.TypeOf(x)
fmt.Println(xt.Name())    // returns int
f := Foo{}
ft := reflect.TypeOf(f)
fmt.Println(ft.Name())    // returns Foo
xpt := reflect.TypeOf(&x)
fmt.Println(xpt.Name())   // returns an empty
string
```

You start with a variable `x` of type `int`. You pass it to `reflect.TypeOf` and get back a `reflect.Type` instance. For

primitive types like `int`, `Name` returns the name of the type, in this case the string “int” for your `int`. For a struct, the name of the struct is returned. Some types, like a slice or a pointer, don’t have names; in those cases, `Name` returns an empty string.

你首先有一个类型为 `int` 的变量 `x`。你将其传递给 `reflect.TypeOf`, 并得到一个 `reflect.Type` 实例。对于像 `int` 这样的基本类型, `Name` 返回类型名称, 在本例中, 对于你的 `int`, 返回字符串“int”。对于结构体, 则返回结构体的名称。某些类型, 例如切片或指针, 没有名称; 在这种情况下, `Name` 返回一个空字符串。

The `Kind` method on `reflect.Type` returns a value of type `reflect.Kind`, which is a constant that says what the type is made of—a slice, a map, a pointer, a struct, an interface, a string, an array, a function, an int, or some other primitive type. The difference between the kind and the type can be tricky to understand. Remember this rule: if you define a struct named `Foo`, the kind is `reflect.Struct` and the type is “Foo.”

`reflect.Type` 上的 `Kind` 方法返回一个类型为 `reflect.Kind` 的值, 这是一个常量, 用于说明该类型由什么构成——切片、映射、指针、结构体、接口、字符串、数组、函数、整数或其他原始类型。类型和种类之间的区别可能比较难理解。记住这条规则: 如果你定义了一个名为 `Foo` 的结构体, 那么它的种类是 `reflect.Struct`, 类型是“Foo”。

The kind is very important. One thing to be aware of when using reflection is that almost everything in the `reflect` package assumes that you know what you are doing. Some of the methods defined on `reflect.Type` and other types in the `reflect` package make sense for only certain kinds. For example, there's a method on `reflect.Type` called `NumIn`. If your `reflect.Type` instance represents a function, it returns the number of input parameters for the function. If your `reflect.Type` instance isn't a function, calling `NumIn` will panic your program.

类型非常重要。使用反射时需要注意的一点是，`reflect` 包中的几乎所有内容都假定您清楚自己在做什么。`reflect.Type` 和 `reflect` 包中的其他类型定义的一些方法仅对特定类型有效。例如，`reflect.Type` 中有一个名为 `NumIn` 的方法。如果您的 `reflect.Type` 实例代表一个函数，它将返回该函数的输入参数数量。如果您的 `reflect.Type` 实例不是一个函数，调用 `NumIn` 将导致程序崩溃。

WARNING

警告

In general, if you call a method that doesn't make sense for the kind of the type, the method call panics. Always remember to use the kind of the reflected type to know which methods will work and which ones will panic.

一般来说，如果调用的方法与目标类型的类型不符，则该方法调用会引发 panic。务必记住使用反射类型的类型来确定哪些方法可以正常工作，哪些方法会引发 panic。

Another important method on `reflect.Type` is `Elem`. Some types in Go have references to other types and `Elem` is how to find out the contained type. For example, let's use `reflect.TypeOf` on a pointer to an `int`:

`reflect.Type` 的另一个重要方法是 `Elem`。Go 中的某些类型会引用其他类型，而 `Elem` 用于查找被引用的类型。例如，我们可以对指向 `int` 的指针使用 `reflect.TypeOf`：

```
var x int
xpt := reflect.TypeOf(&x)
fmt.Println(xpt.Name())           // returns an empty
string
fmt.Println(xpt.Kind())           // returns
reflect.Pointer
fmt.Println(xpt.Elem().Name())    // returns "int"
fmt.Println(xpt.Elem().Kind())    // returns
reflect.Int
```

That gives you a `reflect.Type` instance with a blank name and a kind of `reflect.Pointer`. When the `reflect.Type` represents a pointer, `Elem` returns the `reflect.Type` for the type the pointer points to. In this case, the `Name` method returns “int,” and `Kind` returns `reflect.Int`. The `Elem` method also works for slices, maps, channels, and arrays.

这将生成一个名称为空、类型为 `reflect.Pointer` 的 `reflect.Type` 实例。当 `reflect.Type` 表示指针时，`Elem` 返回指针指向类型的 `reflect.Type`。在本例中，`Name` 方法返回“int”，而 `Kind` 返回 `reflect.Int`。`Elem` 方法也适用于切片、映射、通道和数组。

There are methods on `reflect.Type` for reflecting on structs. Use the `NumField` method to get the number of fields in the struct, and get the fields in a struct by index with the `Field` method. That returns each field's structure described in a `reflect.StructField`, which has the name, order, type, and struct tags on a field. Let's look at a quick example, which you can run on [The Go Playground](#) or in the `sample_code/struct_tag` directory in the [Chapter 16 repository](#):

`reflect.Type` 提供了用于反射结构体的方法。使用 `NumField` 方法可以获取结构体中的字段数量，而使用 `Field` 方法可以通过索引获取结构体中的字段。这将返回一个 `reflect.StructField`，其中包含字段的名称、顺序、类型和结构体标签。让我们来看一个简单的示例，您可以在 Go Playground 或第 16 章代码库的 `sample_code/struct_tag` 目录中运行该示例：

```
type Foo struct {  
    A int    `myTag:"value"`  
    B string `myTag:"value2"`  
}
```



```
var f Foo
ft := reflect.TypeOf(f)
for i := 0; i < ft.NumField(); i++ {
    curField := ft.Field(i)
    fmt.Println(curField.Name,
curField.Type.Name(),
        curField.Tag.Get("myTag"))
}
```

You create an instance of type `Foo` and use `reflect.TypeOf` to get the `reflect.Type` for `f`. Next you use the `NumField` method to set up a `for` loop to get the index of each field in `f`. Then you use the `Field` method to get the `reflect.StructField` struct that represents the field, and then you can use the fields on `reflect.StructField` to get more information about the field. This code prints out the following:

您创建了一个类型为 `Foo` 的实例，并使用 `reflect.TypeOf` 获取 `reflect.Type` 对应的 `f`。接下来，您使用 `NumField` 方法设置一个 `for` 循环，以获取 `f` 中每个字段的索引。然后，您使用 `Field` 方法获取表示该字段的 `reflect.StructField` 结构体，之后您可以使用 `reflect.StructField` 中的字段来获取有关该字段的更多信息。此代码输出以下内容：

```
A int value  
B string value2
```

There are many more methods in `reflect.Type`, but they all follow the same pattern, allowing you to access the information that describes the type of a variable. You can look at the [reflect.Type documentation](#) in the standard library for more information.

`reflect.Type` 中还有许多其他方法，但它们都遵循相同的模式，允许您访问描述变量类型的信息。您可以查看标准库中的 `reflect.Type` 文档以获取更多信息。

Values

价值观

In addition to examining the types of variables, you can also use reflection to read a variable's value, set its value, or create a new value from scratch.

除了检查变量的类型之外，你还可以使用反射来读取变量的值、设置变量的值，或者从头开始创建一个新值。

You use the `reflect.ValueOf` function to create a `reflect.Value` instance that represents the value of a variable:

您可以使用 `reflect.ValueOf` 函数创建一个 `reflect.Value` 实例，该实例表示变量的值：

```
vValue := reflect.ValueOf(v)
```

Since every variable in Go has a type, `reflect.Value` has a method called `Type` that returns the `reflect.Type` of the `reflect.Value`. There's also a `Kind` method, just as there is on `reflect.Type`.

由于 Go 中的每个变量都有类型，因此 `reflect.Value` 有一个名为 `Type` 的方法，该方法返回 `reflect.Value` 的 `reflect.Type`。此外，还有一个 `Kind` 方法，就像 `reflect.Type` 一样。

Just as `reflect.Type` has methods for finding out information about the type of a variable, `reflect.Value` has methods for finding out information about the value of a variable. I'm not going to cover all of them, but let's take a look at how to use a `reflect.Value` to get the value of the variable.

就像 `reflect.Type` 提供了获取变量类型信息的方法一样，`reflect.Value` 也提供了获取变量值信息的方法。我不会一一介绍所有方法，但让我们来看看如何使用 `reflect.Value` 来获取变量的值。

I'll start by demonstrating how to read your values back out of a `reflect.Value`. The `Interface` method returns the value of the variable as `any`. When you put the value returned by `Interface` into a variable, you have to use a type assertion to get back to a usable type:

我将首先演示如何从 `reflect.Value` 中读取值。`Interface` 方法返回变量的值，即 `any`。当您将 `Interface` 返回的值放入变量中时，必须使用类型断言才能将其转换为可用的类型：

```
s := []string{"a", "b", "c"}
sv := reflect.ValueOf(s)           // sv is of type
reflect.Value
s2 := sv.Interface().([]string) // s2 is of type
[]string
```

While `Interface` can be called for `reflect.Value` instances that contain values of any kind, you can use special case methods if the kind of the variable is one of the built-in, primitive types: `Bool`, `Complex`, `Int`, `Uint`, `Float`, and `String`. There's also a `Bytes` method that works if the type of the variable is a slice of bytes. If you use a method that doesn't match the type of the `reflect.Value`, your code will panic. If you are unsure about the type of the `Value`, methods can preemptively check: the `CanComplex`, `CanFloat`, `CanInt`, and `CanUint` methods validate that the `Value` is of one of the numeric types, and `CanConvert` checks for other types.

虽然可以对包含任何类型值的 `reflect.Value` 实例调用 `Interface`，但如果变量的类型是内置的基本类型之一，则可以使用特殊情况方法：`Bool`、`Complex`、`Int`、`Uint`、`Float` 和 `String`。此外，如果变量的类型是字节切片，则还有一个 `Bytes` 方法。如果使用的方法与 `reflect.Value` 的类型不匹配，则代码会引发 panic。如果您不确定 `Value` 的类型，可以使用以下方法进行预先检查：`CanComplex`、`CanFloat`、`CanInt` 和 `CanUint` 方法验证 `Value` 是否为数字类型之一，而 `CanConvert` 方法检查其他类型。

You can use reflection to set the value of a variable as well, but it's a three-step process.

你也可以使用反射来设置变量的值，但这需要三个步骤。

First, you pass a pointer to the variable into `reflect.ValueOf`. This returns a `reflect.Value` that represents the pointer:

首先，将指向变量的指针传递给 `reflect.ValueOf`。这将返回一个 `reflect.Value`，它代表该指针：

```
i := 10
iv := reflect.ValueOf(&i)
```

Next, you need to get to the actual value to set it. You use the `Elem` method on `reflect.Value` to get to the value pointed to by the pointer

that was passed into `reflect.ValueOf`. Just as `Elem` on `reflect.Type` returns the type that's pointed to by a containing type, `Elem` on `reflect.Value` returns the value that's pointed to by a pointer or the value that's stored in an interface:

接下来，你需要获取实际值并进行设置。你可以使用 `Elem` 方法在 `reflect.Value` 上获取传递给 `reflect.ValueOf` 的指针所指向的值。正如 `Elem` 方法在 `reflect.Type` 上返回包含类型所指向的类型一样，`Elem` 方法在 `reflect.Value` 上返回指针所指向的值或存储在接口中的值：

```
ivv := iv.Elem()
```

Finally, you get to the actual method that's used to set the value. Just as there are special-case methods for reading primitive types, there are special-case methods for setting primitive types: `SetBool`, `SetInt`, `SetFloat`, `SetString`, and `SetUint`. In the example, calling `ivv.SetInt(20)` changes the value of `i`. If you print out `i` now, you will get 20:

最后，你会看到用于设置值的实际方法。就像读取基本类型有特殊方法一样，设置基本类型也有特殊方法：`SetBool`、`SetInt`、`SetFloat`、`SetString` 和 `SetUint`。在示例中，调用 `ivv.SetInt(20)` 会更改 `i` 的值。如果你现在打印 `i`，你会得到 20。

```
ivv.SetInt(20)
fmt.Println(i) // prints 20
```

For all other types, you need to use the `Set` method, which takes a variable of type `reflect.Value`. The value that you are setting it to doesn't need to be a pointer, because you are just reading this value, not changing it. And just as you can use `Interface()` to read primitive types, you can use `Set` to write primitive types.

对于所有其他类型，您需要使用 `Set` 方法，该方法接受一个 `reflect.Value` 类型的变量。您要设置的值不需要是指针，因为您只是读取该值，而不是更改它。正如您可以使用 `Interface()` 读取基本类型一样，您也可以使用 `Set` 写入基本类型。

The reason you need to pass a pointer to `reflect.ValueOf` to change the value of the input parameter is that it is just like any other function in Go. As I discussed in [“Pointers Indicate Mutable Parameters”](#), you use a parameter of a pointer type to indicate that you want to modify the value of the parameter. When you modify the value, you dereference the pointer and then set the value. The following two functions follow the same process:

之所以需要传递指向 `reflect.ValueOf` 的指针来更改输入参数的值，是因为它与 Go 中的任何其他函数一样。正如我在“指针指示可变参数”一文中讨论的那样，使用指针类型的参数来表明您想要修改参数

的值。修改值时，您需要解引用指针，然后设置值。以下两个函数遵循相同的过程：

```
func changeInt(i *int) {  
    *i = 20  
}  
  
func changeIntReflect(i *int) {  
    iv := reflect.ValueOf(i)  
    iv.Elem().SetInt(20)  
}
```

Attempting to set a `reflect.Value` to a value of the wrong type will produce a panic.

尝试将 `reflect.Value` 设置为错误类型的值将导致 panic。

TIP

提示

If you don't pass a pointer to a variable to `reflect.ValueOf`, you can still read the value of the variable using reflection. But if you try to use any of the methods that can change the value of a variable, the method calls will (not surprisingly) panic. The `CanSet` method on `reflect.Value` will tell you if calling `Set` will produce a panic.

如果你不将指向变量的指针传递给 `reflect.ValueOf`，你仍然可以使用反射读取该变量的值。但是，如果你尝试使用任何可以改变变量值的方法，这些方法调用（不出所料）会引发 panic。`reflect.Value` 的 `CanSet` 方法会告诉你调用 `Set` 是否会导致 panic。

Make New Values

创造新价值观

Before you learn how to best use reflection, there's one more thing to cover: how to create a value. The `reflect.New` function is the reflection analogue of the `new` function. It takes in a `reflect.Type` and returns a `reflect.Value` that's a pointer to a `reflect.Value` of the specified type. Since it's a pointer, you can modify it and then assign the modified value to a variable by using the `Interface` method.

在学习如何最佳使用反射之前，还有一件事需要了解：如何创建值。`reflect.New` 函数是反射中 `new` 函数的对应函数。它接收一个 `reflect.Type` 对象，并返回一个指向指定类型 `reflect.Value` 对象的指针 `reflect.Value`。由于它是一个指针，你可以修改它，然后使用 `Interface` 方法将修改后的值赋给一个变量。

Just as `reflect.New` creates a pointer to a scalar type, you can also use reflection to do the same thing as the `make` keyword with the following functions:

就像 `reflect.New` 创建一个指向标量类型的指针一样，你也可以使用反射来执行与 `make` 关键字相同的操作，以下函数可以实现：

```
func MakeChan(typ Type, buffer int) Value

func MakeMap(typ Type) Value

func MakeMapWithSize(typ Type, n int) Value

func MakeSlice(typ Type, len, cap int) Value
```

Each of these functions takes in a `reflect.Type` that represents the compound type, not the contained type.

这些函数都接受一个 `reflect.Type`，它表示复合类型，而不是包含的类型。

You must always start from a value when constructing a `reflect.Type`. However, a trick lets you create a variable to represent a `reflect.Type` if you don't have a value handy:

构建 `reflect.Type` 时，必须始终从一个值开始。但是，如果您手头没有值，可以使用一个技巧来创建一个变量来表示 `reflect.Type`：

```
var stringType = reflect.TypeOf((*string)
(nil)).Elem()
```

```
var stringSliceType =  
reflect.TypeOf([]string(nil))
```

The variable `stringType` contains a `reflect.Type` that represents a `string`, and the variable `stringSliceType` contains a `reflect.Type` that represents a `[]string`. That first line can take a bit of effort to decode. What you are doing is converting `nil` to a pointer to `string`, using `reflect.TypeOf` to make a `reflect.Type` of that pointer type, and then calling `Elem` on that pointer's `reflect.Type` to get the underlying type. You have to put `*string` in parentheses because of the Go order of operations; without the parentheses, the compiler thinks that you are converting `nil` to `string`, which is illegal.

变量 `stringType` 包含一个 `reflect.Type`，它代表一个 `string`；变量 `stringSliceType` 包含一个 `reflect.Type`，它代表一个 `[]string`。第一行代码可能需要一些时间才能理解。你正在做的是将 `nil` 转换为指向 `string` 的指针，使用 `reflect.TypeOf` 创建一个该指针类型的 `reflect.Type`，然后调用该指针的 `reflect.Type` 的 `Elem` 来获取底层类型。由于 Go 语言的运算顺序，你必须将 `*string` 放在括号中；如果没有括号，编译器会认为你正在将 `nil` 转换为 `string`，这是非法的。

For the `stringSliceType`, it's a bit simpler since `nil` is a valid value for a slice. All you have to do is type conversion of `nil` to a `[]string` and pass that to `reflect.Type`.

对于 `stringSliceType`，处理起来稍微简单一些，因为 `nil` 是一个有效的切片值。你只需要将 `nil` 转换为 `[]string`，然后将其传递给 `reflect.Type` 即可。

Now that you have these types, you can see how to use `reflect.New` and `reflect.MakeSlice`:

现在您已经了解了这些类型，就可以了解如何使用 `reflect.New` 和 `reflect.MakeSlice` 了：

```
ssv := reflect.MakeSlice(stringSliceType, 0, 10)

sv := reflect.New(stringType).Elem()
sv.SetString("hello")

ssv = reflect.Append(ssv, sv)
ss := ssv.Interface().([]string)
fmt.Println(ss) // prints [hello]
```

You can try out this code for yourself on [The Go Playground](#) or in the `sample_code/reflect_string_slice` directory in the [Chapter 16 repository](#).

您可以亲自在 Go Playground 或第 16 章存储库的 `sample_code/reflect_string_slice` 目录中尝试这段代码。

Use Reflection to Check If an Interface's Value Is nil

使用反射检查接口的值是否为 nil

As I talked about in [“Interfaces and nil”](#), if a `nil` variable of a concrete type is assigned to a variable of an interface type, the variable of the interface type is not `nil`. This is because a type is associated with the interface variable. If you want to check whether the value associated with an interface is `nil`, you can do so with reflection by using two methods—`IsValid` and `IsNil`:

正如我在“接口与 nil”一文中提到的，如果将具体类型的 `nil` 变量赋值给接口类型的变量，则该接口类型的变量的值不会是 `nil`。这是因为接口变量是与特定类型关联的。如果您想检查与接口关联的值是否为 `nil`，可以使用反射，通过两种方法——`IsValid` 和 `IsNil` 来实现：

```
func hasNoValue(i any) bool {
    iv := reflect.ValueOf(i)
    if !iv.IsValid() {
        return true
    }
    switch iv.Kind() {
    case reflect.Pointer, reflect.Slice,
    reflect.Map, reflect.Func,
```

```
        reflect.Interface:
        return iv.IsNil()
    default:
        return false
    }
}
```

The `IsValid` method returns `true` if `reflect.Value` holds anything other than a `nil` interface. You need to check this first because calling any other method on `reflect.Value` will (unsurprisingly) panic if `IsValid` is `false`. The `IsNil` method returns `true` if the value of the `reflect.Value` is `nil`, but it can be called only if the `reflect.Kind` is something that *can* be `nil`. If you call it on a type whose zero value isn't `nil`, it (you guessed it) panics.

如果 `reflect.Value` 的值不是 `nil` 接口，则 `IsValid` 方法返回 `true`。你需要先检查这一点，因为如果 `IsValid` 的值是 `false`，那么对 `reflect.Value` 调用任何其他方法都会（不出所料地）引发 panic。如果 `reflect.Value` 的值为 `nil`，则 `IsNil` 方法返回 `true`，但只有当 `reflect.Kind` 的值可以是 `nil` 时才能调用此方法。如果对一个零值不是 `nil` 的类型调用此方法，则（你猜对了）会引发 panic。

You can see this function in use on [The Go Playground](#) or in the `sample_code/no_value` directory in the [Chapter 16 repository](#).

您可以在 Go Playground 或第 16 章存储库的 `sample_code/no_value` 目录中看到此函数的使用。

Even though it is possible to detect an interface with a `nil` value, strive to write your code so that it performs correctly even when the value associated with an interface is `nil`. Reserve this code for situations where you have no other options.

即使可以检测到值为 `nil` 的接口，也应尽量编写代码，使其在接口值为 `nil` 时也能正确运行。仅在别无选择的情况下才使用此代码。

Use Reflection to Write a Data Marshaler

使用反射编写数据编组器

As mentioned earlier, reflection is what the standard library uses to implement marshaling and unmarshaling. Let's see how it's done by building a data marshaler for ourselves. Go provides the `csv.NewReader` and `csv.NewWriter` functions to read a CSV file into a slice of slice of strings and to write a slice of slice of strings out to a CSV file, but nothing in the standard library can automatically map that data to the fields in a struct. The following code is going to add that missing functionality.

如前所述，反射是标准库用于实现序列化和反序列化的技术。让我们通过构建一个数据序列化器来了解它是如何实现的。Go 提供了

`csv.NewReader` 和 `csv.NewWriter` 函数，用于将 CSV 文件读取到字符串切片中，以及将字符串切片写入 CSV 文件，但标准库中没有任何函数可以自动将这些数据映射到结构体中的字段。以下代码将添加这一缺失的功能。

NOTE

笔记

The examples here have been cut down a bit to fit, reducing the number of supported types. You can find the complete code on [The Go Playground](#) or in the `sample_code/csv` directory in the [Chapter 16 repository](#).

为了节省篇幅，这里的示例代码有所精简，因此支持的类型数量有所减少。您可以在 Go Playground 或第 16 章代码库的 `sample_code/csv` 目录中找到完整的代码。

You'll start by defining your API. As with other marshalers, you'll define a struct tag that specifies the name of a field in the data to map it to a field in a struct:

首先，你需要定义 API。与其他编组器一样，你需要定义一个结构体标签，该标签指定数据中字段的名称，以便将其映射到结构体中的字段：

```
type MyData struct {  
    Name    string `csv:"name"`  
    Age     int    `csv:"age"`
```



```
    HasPet bool    `csv:"has_pet"`  
}
```

The public API consists of two functions:

公共 API 由两个函数组成：

```
// Unmarshal maps all of the rows of data in a  
// slice of slice of strings  
// into a slice of structs.  
// The first row is assumed to be the header with  
// the column names.  
func Unmarshal(data [][]string, v any) error  
  
// Marshal maps all of the structs in a slice of  
// structs to a slice of slice  
// of strings.  
// The first row written is the header with the  
// column names.  
func Marshal(v any) ([][]string, error)
```

You'll start with `Marshal`, writing the function and then looking at the two helper functions it uses:

首先，从 `Marshal` 开始，编写函数，然后查看它使用的两个辅助函数：

```

func Marshal(v any) ([][]string, error) {
    sliceVal := reflect.ValueOf(v)
    if sliceVal.Kind() != reflect.Slice {
        return nil, errors.New("must be a slice of
structs")
    }
    structType := sliceVal.Type().Elem()
    if structType.Kind() != reflect.Struct {
        return nil, errors.New("must be a slice of
structs")
    }
    var out [][]string
    header := marshalHeader(structType)
    out = append(out, header)
    for i := 0; i < sliceVal.Len(); i++ {
        row, err := marshalOne(sliceVal.Index(i))
        if err != nil {
            return nil, err
        }
        out = append(out, row)
    }
    return out, nil
}

```

Since you can marshal a struct of any type, you need to use a parameter of type `any`. This isn't a pointer to a slice of structs, because you are only reading from your slice, not modifying it.

由于你可以对任何类型的结构体进行序列化，因此你需要使用类型为 `any` 的参数。这不是指向结构体切片的指针，因为你只是从切片中读取数据，而不是修改它。

The first row of your CSV is going to be the header with the column names, so you get those column names from the struct tags on fields in the struct's type. You use the `Type` method to get the `reflect.Type` of the slice from the `reflect.Value`, and then call the `Elem` method to get the `reflect.Type` of the elements of the slice. You then pass this to `marshalHeader` and append the response to your output.

CSV 文件的第一行是包含列名的标题行，这些列名可以从结构体类型字段的结构体标签中获取。使用 `Type` 方法从 `reflect.Value` 中获取切片的 `reflect.Type`，然后调用 `Elem` 方法获取切片元素的 `reflect.Type`。最后，将此结果传递给 `marshalHeader`，并将响应附加到输出中。

Next, you iterate through each element in the struct slice using reflection, passing the `reflect.Value` of each element to `marshalOne`, appending the result to your output. When you finish iterating, you return your slice of slice of `string`.

接下来，使用反射遍历结构体切片中的每个元素，将每个元素的 `reflect.Value` 传递给 `marshalOne`，并将结果追加到输出中。遍历完成后，返回 `string` 的切片的切片。

Look at the implementation of your first helper function,
`marshalHeader` :

请查看你的第一个辅助函数 `marshalHeader` 的实现:

```
func marshalHeader(vt reflect.Type) []string {  
    var row []string  
    for i := 0; i < vt.NumField(); i++ {  
        field := vt.Field(i)  
        if curTag, ok := field.Tag.Lookup("csv");  
    ok {  
        row = append(row, curTag)  
    }  
    }  
    return row  
}
```

This function simply loops over the fields of the `reflect.Type`, reads the `csv` tag on each field, appends it into a `string` slice, and returns the slice.

此函数简单地遍历 `reflect.Type` 的各个字段，读取每个字段上的 `csv` 标签，将其附加到 `string` 切片中，并返回该切片。

The second helper function is `marshalOne` :

第二个辅助函数是 `marshalOne` :

```

func marshalOne(vv reflect.Value) ([]string,
error) {
    var row []string
    vt := vv.Type()
    for i := 0; i < vv.NumField(); i++ {
        fieldVal := vv.Field(i)
        if _, ok := vt.Field(i).Tag.Lookup("csv");
!ok {
            continue
        }
        switch fieldVal.Kind() {
        case reflect.Int:
            row = append(row,
strconv.FormatInt(fieldVal.Int(), 10))
        case reflect.String:
            row = append(row, fieldVal.String())
        case reflect.Bool:
            row = append(row,
strconv.FormatBool(fieldVal.Bool()))
        default:
            return nil, fmt.Errorf("cannot handle
field of kind %v",

fieldVal.Kind())
        }
    }
}

```

```
    return row, nil
}
```

This function takes in a `reflect.Value` and returns a `string` slice. You create the `string` slice, and for each field in the struct, you switch on its `reflect.Kind` to determine how to convert it to a `string`, and append it to the output.

此函数接收一个 `reflect.Value` 并返回一个 `string` 切片。您创建 `string` 切片，然后对于结构体中的每个字段，您根据其 `reflect.Kind` 的值来确定如何将其转换为 `string`，并将其附加到输出中。

Your simple marshaler is now complete. Let's see what you have to do to unmarshal:

你的简易序列化器已经完成。接下来我们看看如何进行反序列化：

```
func Unmarshal(data [][]string, v any) error {
    sliceValPointer := reflect.ValueOf(v)
    if sliceValPointer.Kind() != reflect.Pointer {
        return errors.New("must be a pointer to a
slice of structs")
    }
    sliceVal := sliceValPointer.Elem()
    if sliceVal.Kind() != reflect.Slice {
```

```

        return errors.New("must be a pointer to a
slice of structs")
    }
    structType := sliceVal.Type().Elem()
    if structType.Kind() != reflect.Struct {
        return errors.New("must be a pointer to a
slice of structs")
    }

    // assume the first row is a header
    header := data[0]
    namePos := make(map[string]int, len(header))
    for i, name := range header {
        namePos[name] = i
    }

    for _, row := range data[1:] {
        newVal := reflect.New(structType).Elem()
        err := unmarshalOne(row, namePos, newVal)
        if err != nil {
            return err
        }
        sliceVal.Set(reflect.Append(sliceVal,
newVal))
    }
    return nil
}

```

Since you are copying data into a slice of any kind of struct, you need to use a parameter of type `any`. Furthermore, because you are modifying the value stored in this parameter, you *must* pass in a pointer to a slice of structs. The `Unmarshal` function converts that slice of structs pointer to a `reflect.Value`, then gets the underlying slice, and then gets the type of the structs in the underlying slice.

由于您要将数据复制到任意类型结构体的切片中，因此需要使用 `any` 类型的参数。此外，由于您要修改存储在此参数中的值，因此必须传入指向结构体切片的指针。`Unmarshal` 函数会将该结构体切片指针转换为 `reflect.Value` 类型，然后获取底层切片，最后获取底层切片中结构体的类型。

As I said earlier, the code assumes that the first row of data is a header with the names of the columns. You use this information to build up a map, so you can associate the `csv` struct tag value with the correct data element.

正如我之前所说，这段代码假定第一行数据是包含列名的表头。您可以使用此信息构建一个映射，以便将 `csv` 结构标签值与正确的数据元素关联起来。

You then loop through all the remaining `string` slices, creating a new `reflect.Value` using the `reflect.Type` of the struct, call `unmarshalOne` to copy the data in the current `string` slice into the

struct, and then add the struct to your slice. After iterating through all the rows of data, you return.

然后遍历所有剩余的 `string` 个切片, 使用结构体的 `reflect.Type` 创建一个新的 `reflect.Value`, 调用 `unmarshalOne` 将当前 `string` 切片中的数据复制到结构体中, 然后将该结构体添加到新的切片中。遍历完所有数据行后, 返回。

All that remains is looking at the implementation of `unmarshalOne`:

剩下的就是研究 `unmarshalOne` 的实施情况了:

```
func unmarshalOne(row []string, namePos
map[string]int, vv reflect.Value) error {
    vt := vv.Type()
    for i := 0; i < vv.NumField(); i++ {
        typeField := vt.Field(i)
        pos, ok :=
namePos[typeField.Tag.Get("csv")]
        if !ok {
            continue
        }
        val := row[pos]
        field := vv.Field(i)
        switch field.Kind() {
        case reflect.Int:
            i, err := strconv.ParseInt(val, 10,
```

```

64)
        if err != nil {
            return err
        }
        field.SetInt(i)
    case reflect.String:
        field.SetString(val)
    case reflect.Bool:
        b, err := strconv.ParseBool(val)
        if err != nil {
            return err
        }
        field.SetBool(b)
    default:
        return fmt.Errorf("cannot handle field
of kind %v",
                                field.Kind())
    }
}
return nil
}

```

This function iterates over each field in the newly created `reflect.Value`, uses the `csv` struct tag on the current field to find its name, looks up the element in the `data` slice by using the `namePos` map, converts the value from a `string` to the correct type, and sets the value

on the current field. After all fields have been populated, the function returns.

此函数遍历新创建的 `reflect.Value` 中的每个字段，使用当前字段上的 `csv` 结构标签查找其名称，使用 `namePos` 映射在 `data` 切片中查找元素，将 `string` 类型的值转换为正确的类型，并将该值设置到当前字段。所有字段填充完毕后，函数返回。

Now that you have written your marshaler and unmarshaler, you can integrate with the existing CSV support in the Go standard library:

现在你已经编写好了序列化器和反序列化器，可以将其与 Go 标准库中现有的 CSV 支持集成：

```
data := `name,age,has_pet
Jon,"100",true
"Fred ""The Hammer"" Smith",42,false
Martha,37,"true"
`

r := csv.NewReader(strings.NewReader(data))
allData, err := r.ReadAll()
if err != nil {
    panic(err)
}
var entries []MyData
Unmarshal(allData, &entries)
fmt.Println(entries)
```

```
//now to turn entries into output
out, err := Marshal(entries)
if err != nil {
    panic(err)
}
sb := &strings.Builder{}
w := csv.NewWriter(sb)
w.WriteAll(out)
fmt.Println(sb)
```

Build Functions with Reflection to Automate Repetitive Tasks

利用反射构建函数以实现重复性任务的自动化

Another thing that Go lets you do with reflection is create a function. You can use this technique to wrap existing functions with common functionality without writing repetitive code. For example, here's a factory function that adds timing to any function that's passed into it:

Go 语言的反射机制还允许你创建函数。你可以使用这种技术来封装现有函数，使其具备常用功能，而无需编写重复代码。例如，以下是一个工厂函数，它可以为任何传入的函数添加计时功能：

```

func MakeTimedFunction(f any) any {
    ft := reflect.TypeOf(f)
    fv := reflect.ValueOf(f)
    wrapperF := reflect.MakeFunc(ft, func(in
[]reflect.Value) []reflect.Value {
        start := time.Now()
        out := fv.Call(in)
        end := time.Now()
        fmt.Println(end.Sub(start))
        return out
    })
    return wrapperF.Interface()
}

```

This function takes in any function, so the parameter is of type `any`. It then passes the `reflect.Type` that represents the function into `reflect.MakeFunc`, along with a closure that captures the start time, calls the original function using reflection, captures the end time, prints out the difference, and returns the value calculated by the original function. The value returned from `reflect.MakeFunc` is a `reflect.Value`, and you call its `Interface` method to get the value to return. Here's how you use it:

此函数接受任意函数作为参数，因此参数类型为 `any`。它将代表该函数的 `reflect.Type` 传递给 `reflect.MakeFunc`，同时传递一个闭包，该闭包捕获开始时间，使用反射调用原始函数，捕获结束时间，

打印出差值，并返回原始函数计算出的值。 `reflect.MakeFunc` 返回的值是一个 `reflect.Value`，您可以调用其 `Interface` 方法来获取要返回的值。以下是使用方法：

```
func timeMe(a int) int {
    time.Sleep(time.Duration(a) * time.Second)
    result := a * 2
    return result
}

func main() {
    timed := MakeTimedFunction(timeMe).(func(int)
int)
    fmt.Println(timed(2))
}
```

You can run a more complete version of this program on [The Go Playground](#) or in the `sample_code/timed_function` directory in the [Chapter 16 repository](#).

您可以在 Go Playground 或第 16 章存储库的 `sample_code/timed_function` 目录中运行此程序的更完整版本。

While generating functions is clever, be careful when using this feature. Make sure that it's clear when you are using a generated function and what functionality it is adding. Otherwise, you will be making it harder to

understand the flow of data through your program. Furthermore, as I'll discuss in [“Use Reflection Only if It's Worthwhile”](#), reflection makes your programs slower, so using it to generate and invoke functions seriously impacts performance unless the code you are generating is already performing a slow operation, like a network call. Remember, reflection works best when it's used to map data in and out of the edge of your programs.

生成函数虽然很巧妙，但使用时务必谨慎。确保你清楚地知道何时使用了生成的函数以及它添加了哪些功能。否则，你会让程序的数据流变得难以理解。此外，正如我将在“仅在必要时使用反射”一文中讨论的那样，反射会降低程序的运行速度，因此，除非你生成的代码本身就在执行耗时的操作（例如网络调用），否则使用它来生成和调用函数会严重影响性能。记住，反射最适用于在程序边界附近映射数据。

One project that follows these rules for generated functions is my SQL mapping library Proteus. It creates a type-safe database API by generating a function from a SQL query and a function field or variable. You can learn more about Proteus in my GopherCon 2017 talk, [“Runtime Generated, Typesafe, and Declarative: Pick Any Three,”](#) and you can find the source code on [GitHub](#).

我的 SQL 映射库 Proteus 就是一个遵循这些生成函数规则的项目。它通过 SQL 查询和函数字段或变量生成函数，从而创建类型安全的数据库 API。您可以在我 2017 年 GopherCon 大会上的演讲“运行时生成、

类型安全和声明式：任选三项”中了解更多关于 Proteus 的信息，您也可以 在 GitHub 上找到其源代码。

You Can Build Structs with Reflection, but Don't

你可以利用反射来构建结构，但不要这样做。

You can make one more thing with reflection, and it's weird. The `reflect.StructOf` function takes in a slice of `reflect.StructField` and returns a `reflect.Type` that represents a new struct type. These structs can be assigned only to variables of type `any`, and their fields can only be read and written using reflection.

你还可以利用反射实现一个很奇特的功能。`reflect.StructOf` 函数接收一个 `reflect.StructField` 的切片，并返回一个 `reflect.Type`，它代表一种新的结构体类型。这些结构体只能赋值给 `any` 类型的变量，并且它们的字段只能通过反射进行读写操作。

For the most part, this is a feature of academic interest only. To see a demo of how `reflect.StructOf` works, look at the `memoizer` function on [The Go Playground](#) or in the `sample_code/memoizer` directory in the [Chapter 16 repository](#). It uses dynamically generated structs as the keys to a map that caches the output of a function.

大多数情况下，这仅具有学术意义。要查看 `reflect.StructOf` 的工作原理演示，请查看 Go Playground 上的 `memoizer` 函数，或第 16 章代码库中的 `sample_code/memoizer` 目录。它使用动态生成的结构体作为键，指向一个缓存函数输出的映射。

Reflection Can't Make Methods

反思无法创造方法

You've seen all the things that you can do with reflection, but there's one thing you can't make. While you can use reflection to create new functions and new struct types, there's no way to use reflection to add methods to a type. This means you cannot use reflection to create a new type that implements an interface.

你已经了解了反射的各种用途，但有一件事你无法做到。虽然你可以使用反射创建新函数和新的结构体类型，但反射无法为类型添加方法。这意味着你不能使用反射来创建实现某个接口的新类型。

Use Reflection Only if It's Worthwhile

只有在值得的时候才进行反思

Although reflection is essential when converting data at the boundaries of Go, be careful using it in other situations. Reflection isn't free. To

demonstrate, let's implement `Filter` by using reflection. You looked at a generic implementation in [“Generic Functions Abstract Algorithms”](#), but let's see what a reflection-based version looks like:

虽然在 Go 语言边界处转换数据时反射至关重要，但在其他情况下使用反射要谨慎。反射并非没有代价。为了演示，让我们使用反射来实现 `Filter`。您在“泛型函数抽象算法”中已经了解了泛型实现，但现在让我们看看基于反射的版本是什么样的：

```
func Filter(slice any, filter any) any {
    sv := reflect.ValueOf(slice)
    fv := reflect.ValueOf(filter)

    sliceLen := sv.Len()
    out := reflect.MakeSlice(sv.Type(), 0,
sliceLen)
    for i := 0; i < sliceLen; i++ {
        curVal := sv.Index(i)
        values := fv.Call([]reflect.Value{curVal})
        if values[0].Bool() {
            out = reflect.Append(out, curVal)
        }
    }
    return out.Interface()
}
```

You use it like this:

使用方法如下：

```
names := []string{"Andrew", "Bob", "Clara",  
"Hortense"}  
longNames := Filter(names, func(s string) bool {  
    return len(s) > 3  
}).([]string)  
fmt.Println(longNames)  
  
ages := []int{20, 50, 13}  
adults := Filter(ages, func(age int) bool {  
    return age >= 18  
}).([]int)  
fmt.Println(adults)
```

This prints out the following:

输出结果如下：

```
[Andrew Clara Hortense]  
[20 50]
```

Your reflection-using filter function isn't difficult to understand, but it's certainly longer than a custom-written or generic function. Let's see how it performs on an Apple Silicon M1 with 16 GB of RAM on Go 1.20 when

filtering 1,000 element slices of strings and `int`s, compared to custom-written functions:

你用反射实现的过滤函数并不难理解，但它肯定比自定义函数或通用函数要长。让我们看看它在配备 16 GB 内存的 Apple Silicon M1 芯片上，使用 Go 1.20 版本过滤 1000 个字符串和 `int` 元素切片时的性能，并与自定义函数进行比较：

```
BenchmarkFilterReflectString-8      5870   203962
ns/op  46616 B/op  2219 allocs/op
BenchmarkFilterGenericString-8     294355    3920
ns/op  16384 B/op    1 allocs/op
BenchmarkFilterString-8            302636    3885
ns/op  16384 B/op    1 allocs/op
BenchmarkFilterReflectInt-8         5756   204530
ns/op  45240 B/op  2503 allocs/op
BenchmarkFilterGenericInt-8        439100    2698
ns/op   8192 B/op    1 allocs/op
BenchmarkFilterInt-8               443745    2677
ns/op   8192 B/op    1 allocs/op
```

You can find this code in the *sample_code/reflection_filter* directory in the [Chapter 16 repository](#) so you can run it yourself.

您可以在第 16 章存储库的 `sample_code/reflection_filter` 目录中找到此代码，以便您可以自己运行它。

This benchmark demonstrates the value of using generics when you can. Reflection is over 50 times slower than a custom or generic function for string filtering and roughly 75 times slower for `int` s. It uses significantly more memory and performs thousands of allocations, which creates additional work for the garbage collector. The generic version provides the same performance as the custom-written functions without having to write multiple versions.

此基准测试表明，尽可能使用泛型具有显著价值。对于字符串过滤，反射函数比自定义函数或泛型函数慢 50 多倍；对于 `int` 类型的字符串，反射函数慢约 75 倍。它占用大量内存，并执行数千次内存分配，从而增加了垃圾回收器的工作量。而泛型版本无需编写多个版本即可提供与自定义函数相同的性能。

A more serious downside to the reflection implementation is that the compiler can't stop you from passing in a wrong type for either the `slice` or the `filter` parameter. You might not mind a few thousand nanoseconds of CPU time, but if someone passes in a function or slice of the wrong type to `Filter`, your program will crash in production. The maintenance cost might be too high to accept.

反射实现的一个更严重的缺点是，编译器无法阻止你为 `slice` 或 `filter` 参数传递错误的类型。你或许可以接受几千纳秒的 CPU 时间，但如果有人将错误类型的函数或切片传递给 `Filter`，你的程序在生产环境中就会崩溃。维护成本可能太高，难以接受。

Some things can't be written using generics, and you'll need to fall back to reflection. CSV marshaling and unmarshaling requires reflection, as does the memoizing program. In both cases, you need to work with an unknown number of values of different (and unknown) types. But do make sure that reflection is essential before employing it.

有些事情不能用泛型来实现，这时就需要用到反射。CSV 的序列化和反序列化以及记忆化程序都需要反射。在这两种情况下，你都需要处理数量未知、类型各异（且未知）的值。但务必在使用反射之前确认其必要性。

unsafe Is Unsafe

不安全就是不安全

Just as the `reflect` package allows you to manipulate types and values, the `unsafe` package allows you to manipulate memory. The `unsafe` package is very small and very odd. It defines several functions and one

type, none of which act like the types and functions found in other packages.

正如 `reflect` 包允许您操作类型和值一样, `unsafe` 包允许您操作内存。 `unsafe` 包非常小, 而且非常奇怪。它定义了几个函数和一个类型, 但他们的行为方式都与其他包中的类型和函数不同。

Given Go's focus on memory safety, you might wonder why `unsafe` even exists. Just as you used `reflect` to translate text data between the outside world and Go code, you use `unsafe` to translate binary data. There are two main reasons for using `unsafe`. A 2020 paper by Diego Elias Costa et al. called [“Breaking Type-Safety in Go: An Empirical Study on the Usage of the unsafe Package”](#) surveyed 2,438 popular Go open source projects and found the following:

鉴于 Go 语言对内存安全的重视, 你可能会好奇 `unsafe` 存在的意义。正如你之前使用 `reflect` 在外部世界和 Go 代码之间转换文本数据一样, 你也使用 `unsafe` 来转换二进制数据。使用 `unsafe` 主要有两个原因。Diego Elias Costa 等人于 2020 年发表的论文《打破 Go 语言中的类型安全: 关于 `unsafe` 包使用情况的实证研究》调查了 2438 个流行的 Go 开源项目, 并发现了以下情况:

- 24% of the studied Go projects use `unsafe` at least once in their codebase.

在所研究的 Go 项目中，有 24% 的项目在其代码库中至少使用过一次 `unsafe`。

- The majority of `unsafe` usages were motivated by integration with operating systems and C code.

`unsafe` 的大多数使用都是出于与操作系统和 C 代码集成的考虑。

- Developers frequently use `unsafe` to write more efficient Go code. 开发者经常使用 `unsafe` 来编写更高效的 Go 代码。

The plurality of the uses of `unsafe` are for system interoperability. The Go standard library uses `unsafe` to read data from and write data to the operating system. You can see examples in the `syscall` package in the standard library or in the higher-level `sys` package. You can learn more about how to use `unsafe` to communicate with the operating system in [a great blog post](#) written by Matt Layher.

`unsafe` 的主要用途是实现系统互操作性。Go 标准库使用 `unsafe` 从操作系统读取数据并向其写入数据。您可以在标准库的 `syscall` 包或更高级别的 `sys` 包中找到示例。Matt Layher 撰写的一篇精彩博文详细介绍了如何使用 `unsafe` 与操作系统通信。

The `unsafe.Pointer` type is a special type that exists for a single purpose: a pointer of any type can be converted to or from `unsafe.Pointer`. In addition to pointers, `unsafe.Pointer` can also be converted to or from a special integer type, called `uintptr`. As with

any other integer type, you can do math with it. This allows you to walk into an instance of a type, extracting individual bytes. You can also perform pointer arithmetic, just as you can with pointers in C and C++. This byte manipulation changes the value of the variable.

`unsafe.Pointer` 类型是一种特殊类型，其存在的唯一目的就是：任何类型的指针都可以与 `unsafe.Pointer` 类型相互转换。除了指针之外，`unsafe.Pointer` 还可以与一种名为 `uintptr` 的特殊整数类型相互转换。与其他整数类型一样，您可以对它进行数学运算。这允许您访问该类型的实例，并提取单个字节。您还可以执行指针运算，就像在 C 和 C++ 中对指针进行运算一样。这种字节操作会改变变量的值。

There are two common patterns in `unsafe` code. The first is a conversion between two types of variables that are normally not convertible. This is performed using a series of type conversions with `unsafe.Pointer` in the middle. The second is reading or modifying the bytes in a variable by converting a variable to an `unsafe.Pointer`, converting the `unsafe.Pointer` to a pointer, and then copying or manipulating the underlying bytes. Both of these techniques require you to know the size (and possibly the location) of the data being manipulated. The `Sizeof` and `Offsetof` functions in the `unsafe` package provide this information.

`unsafe` 代码中有两种常见模式。第一种是在两种通常不可转换的变量类型之间进行转换。这种转换通过一系列类型转换来实现，其中 `unsafe.Pointer` 位于中间。第二种是读取或修改变量中的字节，方法是将变量转换为 `unsafe.Pointer`，将 `unsafe.Pointer` 转换为指针，然后复制或操作底层字节。这两种技术都需要您知道正在操作的数据的大小（以及可能的位置）。`unsafe` 包中的 `Sizeof` 和 `Offsetof` 函数提供了这些信息。

Using Sizeof and Offsetof

使用 Sizeof 和 Offsetof

Some of the functions in `unsafe` reveal how the bytes in various types are laid out in memory. The first one you'll look at is `Sizeof`. As the name implies, it returns the size in bytes of whatever is passed into it.

`unsafe` 中的一些函数揭示了各种类型字节在内存中的布局方式。首先要看的是 `Sizeof`。顾名思义，它返回传入数据的大小（以字节为单位）。

While the sizes for numeric types are fairly obvious (an `int16` is 16 bits or 2 bytes, a `byte` is 1 byte, and so on), other types are a bit more complicated. For a pointer, you get the size of the memory to store the pointer (usually 8 bytes on a 64-bit system), not the size of the data that the pointer points to. This is why `Sizeof` considers any slice to be 24 bytes

long on a 64-bit system; it's implemented as two `int` fields (for length and capacity) and a pointer to the data for the slice. Any `string` is 16 bytes long on a 64-bit system (an `int` for length and a pointer to the contents of the string). Any `map` on a 64-bit system is 8 bytes, since within the Go runtime, a `map` is implemented as a pointer to a rather complicated data structure.

数值类型的大小相当明显（例如，`int16` 为 16 位或 2 字节，`byte` 为 1 字节，依此类推），但其他类型的大小则稍微复杂一些。对于指针，其大小取决于存储指针的内存空间（在 64 位系统中通常为 8 字节），而不是指针指向的数据的大小。这就是为什么在 64 位系统中，`Sizeof` 会将任何切片视为 24 字节长；它由两个 `int` 字段（分别用于长度和容量）以及指向切片数据的指针组成。在 64 位系统中，任何 `string` 的长度为 16 字节（一个 `int` 用于长度，以及一个指向字符串内容的指针）。在 64 位系统上，任何 `map` 都是 8 字节，因为在 Go 运行时中，`map` 被实现为指向一个相当复杂的数据结构的指针。

Arrays are value types, so their size is calculated by multiplying the length of the array by the size of each element in the array.

数组是值类型，因此其大小是通过将数组的长度乘以数组中每个元素的大小来计算的。

For a struct, the size is the sum of the sizes of the fields, plus some adjustments for *alignment*. Computers like to read and write data in regular-

sized chunks and they really don't want a value to start in one chunk and end in another. To make this happen, the compiler adds padding between fields so they line up properly. The compiler also wants the entire struct to be properly aligned. On a 64-bit system, it will add padding at the end of the struct to bring its size up to a multiple of 8 bytes.

对于结构体来说，其大小是所有字段大小之和，再加上一些用于对齐的调整。计算机倾向于以规则大小的数据块进行读写操作，并且不希望一个值从一个数据块开始，在另一个数据块结束。为了实现这一点，编译器会在字段之间添加填充，使它们正确对齐。编译器还要求整个结构体正确对齐。在 64 位系统中，它会在结构体的末尾添加填充，使其大小达到 8 字节的倍数。

Another function in `unsafe`, `Offsetof`, tells you the position of a field within a struct. Let's use `Sizeof` and `Offsetof` to look at the effects of field order on the size of a struct. Say you have two structs:

`unsafe` 中的另一个函数 `Offsetof` 可以告诉你结构体中某个字段的位置。让我们使用 `Sizeof` 和 `Offsetof` 来查看字段顺序对结构体大小的影响。假设你有两个结构体：

```
type BoolInt struct {  
    b bool  
    i int64  
}
```

```
type IntBool struct {  
    i int64  
    b bool  
}
```

Running this code

运行此代码

```
fmt.Println(unsafe.Sizeof(BoolInt{}),  
            unsafe.Offsetof(BoolInt{}.b),  
            unsafe.Offsetof(BoolInt{}.i))  
  
fmt.Println(unsafe.Sizeof(IntBool{}),  
            unsafe.Offsetof(IntBool{}.i),  
            unsafe.Offsetof(IntBool{}.b))
```

produces the following output:

产生以下输出：

```
16 0 8  
16 0 8
```

The size of both types is 16 bytes. When the `bool` comes first, the compiler puts 7 bytes of padding between `b` and `i`. When the `int64`

comes first, the compiler puts 7 bytes of padding after `b` to make the struct's size a multiple of 8 bytes.

两种类型的大小均为 16 字节。当 `bool` 在前时，编译器会在 `b` 和 `i` 之间填充 7 字节。当 `int64` 在前时，编译器会在 `b` 之后填充 7 字节，以使结构体的大小为 8 字节的倍数。

You can see the effect of field order when there are more fields:

当字段数量较多时，您可以明显看出字段顺序的影响：

```
type BoolIntBool struct {
    b  bool
    i  int64
    b2 bool
}

type BoolBoolInt struct {
    b  bool
    b2 bool
    i  int64
}

type IntBoolBool struct {
    i  int64
    b  bool
}
```

```
    b2 bool
}
```

Printing out the sizes and offsets for these types produces the following output:

打印出这些类型的尺寸和偏移量，会产生以下输出：

```
24 0 8 16
16 0 1 8
16 0 8 9
```

Putting the `int64` field between the two `bool` fields produces a struct that's 24 bytes long, since both `bool` fields need to be padded to 8 bytes. Meanwhile, grouping the `bool` fields together produces a struct that's 16 bytes long, the same as the structs with only two fields. You can validate this on [The Go Playground](#) or in the `sample_code/sizeof_offsetof` directory in the [Chapter 16 repository](#).

将 `int64` 字段放在两个 `bool` 字段之间，会生成一个 24 字节长的结构体，因为两个 `bool` 字段都需要填充到 8 字节。同时，将 `bool` 字段组合在一起，会生成一个 16 字节长的结构体，与只有两个字段的结构体长度相同。您可以在 Go Playground 或第 16 章代码库中的 `sample_code/sizeof_offsetof` 目录中验证这一点。

Although only of academic interest most of the time, this information is useful in two situations. The first is in programs that manage large amounts of data. You can sometimes achieve significant savings in memory usage simply by reordering the fields in heavily used structs in order to minimize the amount of padding needed for alignment.

虽然这些信息大多时候仅具有学术价值，但在两种情况下却很有用。第一种情况是在管理大量数据的程序中。有时，只需重新排列常用结构体中的字段，即可显著节省内存使用量，从而最大限度地减少对齐所需的填充。

The second situation occurs when you want to map a sequence of bytes directly into a struct. You'll look at that next.

第二种情况是你想将一系列字节直接映射到一个结构体中。接下来我们会讨论这种情况。

Using unsafe to Convert External Binary Data

使用不安全的方式转换外部二进制数据

As was mentioned earlier, one of the main reasons people use `unsafe` is for performance, especially when reading data from a network. If you want to map the data into or out of a Go data structure, `unsafe.Pointer` gives you a very fast way to do so. You can explore this with a contrived

example. Imagine a *wire protocol* (a specification indicating which bytes are written in which order when communicating over a network) with the following structure:

如前所述，人们使用 `unsafe` 的主要原因之一是提高性能，尤其是在从网络读取数据时。如果您想将数据映射到 Go 数据结构或从中映射数据，`unsafe.Pointer` 提供了一种非常快速的方法。您可以通过一个设计的示例来探索这一点。假设有一个网络协议（一种规范，用于指示在网络通信时哪些字节按什么顺序写入），其结构如下：

- Value: 4 bytes, representing an unsigned, big-endian 32-bit int
值：4 字节，表示一个无符号、大端序的 32 位整数
- Label: 10 bytes, ASCII name for the value
标签：10 字节，值的 ASCII 名称
- Active: 1 byte, boolean flag to indicate whether the field is active
Active：1 字节，布尔标志，指示该字段是否处于活动状态
- Padding: 1 byte, because you want everything to fit into 16 bytes
填充：1 字节，因为您希望所有内容都能放入 16 字节内。

NOTE

笔记

Data sent over a network is usually sent in big-endian format (most significant bytes first), often called *network byte order*. Since most CPUs in use today are little-endian (or bi-endian running in little-endian mode), you need to be careful when reading or writing data to a network.

通过网络传输的数据通常采用大端字节序（最高有效字节在前），也称为网络字节序。由于目前大多数 CPU 都是小端字节序（或以小端模式运行的双端字节序），因此在网络读写数据时需要格外小心。

The code for this example is in the *sample_code/unsafe_data* directory in the [Chapter 16 repository](#).

本示例的代码位于第 16 章存储库的 *sample_code/unsafe_data* 目录中。

You define a data structure whose in-memory layout matches the wire protocol:

您定义一个数据结构，其内存布局与网络协议相匹配：

```
type Data struct {
    Value  uint32    // 4 bytes
    Label  [10]byte     // 10 bytes
    Active bool       // 1 byte
    // Go pads this with 1 byte to make it align
}
```

You use `unsafe.Sizeof` to define a constant that represents its size:

您可以使用 `unsafe.Sizeof` 来定义一个表示其大小的常量：

```
const dataSize = unsafe.Sizeof(Data{}) // sets  
dataSize to 16
```

(One of the weird things about `unsafe.Sizeof` and `Offsetof` is that you can use them in `const` expressions. The size and layout of a data structure in memory is known at compile time, so the results of these functions are calculated at compile time, just like a constant expression.)

(`unsafe.Sizeof` 和 `Offsetof` 的一个奇特之处在于，你可以在 `const` 表达式中使用它们。内存中数据结构的大小和布局在编译时就已经确定，因此这些函数的结果也是在编译时计算的，就像常量表达式一样。)

Say you just read the following bytes off the network:

假设你刚刚从网络读取了以下字节：

```
[0 132 95 237 80 104 111 110 101 0 0 0 0 0 1 0]
```

You're going to read these bytes into an array of length 16 and then convert that array into the `struct` described previously.

你要将这些字节读入长度为 16 的数组，然后将该数组转换为前面描述的 `struct`。

With safe Go code, you could map it like this:

使用安全的 Go 代码，你可以这样映射：

```
func DataFromBytes(b [dataSize]byte) Data {
    d := Data{}
    d.Value = binary.BigEndian.Uint32(b[:4])
    copy(d.Label[:], b[4:14])
    d.Active = b[14] != 0
    return d
}
```

Or, you could use `unsafe.Pointer` instead:

或者，您也可以使用 `unsafe.Pointer`：

```
func DataFromBytesUnsafe(b [dataSize]byte) Data {
    data := (*Data)(unsafe.Pointer(&b))
    if isLE {
        data.Value =
bits.ReverseBytes32(data.Value)
    }
}
```

```
    return data  
}
```

The first line is a little confusing, but you can take it apart and understand what's going on. First, you take the address of the byte array and convert it to an `unsafe.Pointer`. Then you convert the `unsafe.Pointer` to a `(*Data)` (you have to put `(*Data)` in parentheses because of Go's order of operations). You want to return the struct, not a pointer to it, so you dereference the pointer. Next, you check your flag to see if you are on a little-endian platform. If so, you reverse the bytes in the `Value` field. Finally, you return the value.

第一行代码可能有点令人困惑，但你可以把它拆解开来理解它的工作原理。首先，你获取字节数组的地址并将其转换为 `unsafe.Pointer`。然后，你将 `unsafe.Pointer` 转换为 `(*Data)`（由于 Go 的运算顺序，你需要将 `(*Data)` 放在括号中）。你想要返回的是结构体本身，而不是指向它的指针，所以你需要解引用指针。接下来，你检查标志位，看看你是否在小端序平台上运行。如果是，则反转 `Value` 字段中的字节。最后，你返回该值。

NOTE

笔记

Why did you use an array instead of a slice for the parameter? Remember, arrays, like structs, are value types; the bytes are allocated directly. That means you can map the values in `b` directly into the `data` struct. A slice is composed of three parts: a length, a capacity, and a pointer to the actual values. You'll see how to map a slice into a struct using `unsafe` in a bit.

为什么参数要用数组而不是切片呢？记住，数组和结构体一样，都是值类型；字节是直接分配的。这意味着你可以直接将 `b` 中的值映射到 `data` 结构体中。切片由三部分组成：长度、容量和指向实际值的指针。稍后你会看到如何使用 `unsafe` 将切片映射到结构体。

How do you know if you are on a little-endian platform? Here's the code you're using:

如何判断你是否在使用小端平台？以下是你正在使用的代码：

```
var isLE bool

func init() {
    var x uint16 = 0xFF00
    xb := *(*[2]byte)(unsafe.Pointer(&x))
    isLE = (xb[0] == 0x00)
}
```

As I discussed in [“Avoiding the init Function if Possible”](#), you should avoid using `init` functions, except when initializing a package-level value whose value is effectively immutable. Since the endianness of your

processor isn't going to change while your program is running, this is a good use case.

正如我在“尽可能避免使用 `init` 函数”一文中讨论的那样，你应该避免使用 `init` 函数，除非是初始化一个实际上不可变的包级值。由于程序运行时处理器的字节序不会改变，因此这是一个很好的使用场景。

On a little-endian platform, the bytes that represent `x` will be stored as `[00 FF]`. On a big-endian platform, `x` is stored in memory as `[FF 00]`. You use `unsafe.Pointer` to convert a number to an array of bytes, and you check what the first byte is to determine the value of `isLE`.

在小端平台上，表示 `x` 的字节将存储为 `[00 FF]`。在大端平台上，`x` 在内存中存储为 `[FF 00]`。您可以使用 `unsafe.Pointer` 将数字转换为字节数组，并通过检查第一个字节来确定 `isLE` 的值。

Likewise, if you wanted to write your `Data` back to the network, you could use safe Go:

同样地，如果您想将 `Data` 写回网络，可以使用安全的 Go 语言：

```
func BytesFromData(d Data) [dataSize]byte {
    out := [dataSize]byte{}
    binary.BigEndian.PutUint32(out[:4], d.Value)
    copy(out[4:14], d.Label[:])
    if d.Active {
        out[14] = 1
    }
}
```

```
    }  
    return out  
}
```

Or you could use `unsafe` :

或者您可以使用 `unsafe` :

```
func BytesFromDataUnsafe(d Data) [dataSize]byte {  
    if isLE {  
        d.Value = bits.ReverseBytes32(d.Value)  
    }  
    b := *(*[dataSize]byte)(unsafe.Pointer(&d))  
    return b  
}
```

If the bytes are stored in a slice, you can use the `unsafe.Slice` function to create a slice from the contents of `Data`. The `unsafe.SliceData` function is used to create a `Data` instance from the data stored in a slice:

如果字节存储在切片中，则可以使用 `unsafe.Slice` 函数从 `Data` 的内容创建一个切片。`unsafe.SliceData` 函数用于从切片中存储的数据创建 `Data` 实例：

```
func BytesFromDataUnsafeSlice(d Data) []byte {  
    if isLE {
```



```

        d.Value = bits.ReverseBytes32(d.Value)
    }
    bs := unsafe.Slice((*byte)
(unsafe.Pointer(&d)), dataSize)
    return bs
}

func DataFromBytesUnsafeSlice(b []byte) Data {
    data := *(*Data)((unsafe.Pointer)
(unsafe.SliceData(b)))
    if isLE {
        data.Value =
bits.ReverseBytes32(data.Value)
    }
    return data
}

```

The first parameter to `unsafe.Slice` requires two casts. The first cast converts a pointer of the `Data` instance to an `unsafe.Pointer`. Then you need to cast again to a pointer of the type of data that you want the slice to hold. For a slice of bytes, you use `*byte`. The second parameter is the length of the data.

`unsafe.Slice` 的第一个参数需要两次类型转换。第一次转换将指向 `Data` 实例的指针转换为 `unsafe.Pointer`。然后，需要再次转换，

将其转换为切片要存储的数据类型的指针。例如，对于字节切片，可以使用 `*byte`。第二个参数是数据的长度。

The `unsafe.SliceData` function takes in a slice and returns a pointer to the type of the data contained in the slice. In this case, you passed in a `[]byte`, so it returned a `*byte`. You then use `unsafe.Pointer` as a bridge between `*byte` and `*Data` to convert the contents of the slice into a `Data` instance.

`unsafe.SliceData` 函数接收一个切片，并返回指向该切片中所含数据类型的指针。在本例中，您传入的是 `[]byte`，因此它返回的是 `*byte`。然后，您可以使用 `unsafe.Pointer` 作为 `*byte` 和 `*Data` 之间的桥梁，将切片的内容转换为 `Data` 实例。

Is this worth it? Here are the timings on an Apple Silicon M1 (which is little-endian):

这值得吗？以下是苹果M1芯片（小端序）的时序数据：

```
BenchmarkBytesFromData-8          548607271
2.185 ns/op    0 B/op 0 allocs/op
BenchmarkBytesFromDataUnsafe-8     1000000000
0.8418 ns/op   0 B/op 0 allocs/op
BenchmarkBytesFromDataUnsafeSlice-8 91179056
13.14 ns/op   16 B/op 1 allocs/op
BenchmarkDataFromBytes-8           538443861
```

```
2.186 ns/op    0 B/op 0 allocs/op
BenchmarkDataFromBytesUnsafe-8      1000000000
1.160 ns/op    0 B/op 0 allocs/op
BenchmarkDataFromBytesUnsafeSlice-8 1000000000
0.9617 ns/op   0 B/op 0 allocs/op
```

Two things stand out in this chart. First, the conversion from a struct to a slice is by far the slowest operation, and it's the only one that allocates memory. This isn't surprising, since the slice's data needs to escape to the heap when it is returned from the function. Allocating memory on the heap is almost always slower than using memory on the stack. Converting from a slice to a struct is very fast, though.

这张图表中有两点特别突出。首先，从结构体到切片的转换是所有操作中最慢的，也是唯一需要分配内存的操作。这并不奇怪，因为切片的数据在从函数返回时需要逃逸到堆上。在堆上分配内存几乎总是比在栈上使用内存慢。不过，从切片到结构体的转换速度非常快。

If you are working with arrays, using `unsafe` is about 2–2.5 times faster than the standard approach. If you have a program with many of these kinds of conversions, or if you are trying to map a very large and complicated data structure, using these low-level techniques is worthwhile. But for the vast majority of programs, stick with the safe code.

如果你正在处理数组，使用 `unsafe` 比标准方法快 2 到 2.5 倍。如果你的程序需要进行大量此类转换，或者你正在尝试映射一个非常庞大且复杂的数据结构，那么使用这些底层技术是值得的。但对于绝大多数程序来说，坚持使用安全可靠的代码即可。

Accessing Unexported Fields

访问未导出的字段

Here's another bit of magic you can do with `unsafe`, but it's something to use only as a last resort. You can combine reflection and `unsafe` to read and modify unexported fields in structs. Let's see how.

这里还有一点可以用 `unsafe` 实现，但最好只在万不得已的情况下使用。你可以结合反射和 `unsafe` 来读取和修改结构体中未导出的字段。让我们来看看具体怎么做。

First, define a struct in one package:

首先，在一个包中定义一个结构体：

```
type HasUnexportedField struct {  
    A int  
    b bool
```

```
C string
}
```

Normally, code outside this package is unable to access `b`. However, let's see what you can do from another package by using `unsafe`:

通常情况下，此包之外的代码无法访问 `b`。但是，让我们看看如何使用 `unsafe` 从另一个包中执行哪些操作：

```
func SetBUnsafe(huf
*one_package.HasUnexportedField) {
    sf, _ :=
reflect.TypeOf(huf).Elem().FieldByName("b")
    offset := sf.Offset
    start := unsafe.Pointer(huf)
    pos := unsafe.Add(start, offset)
    b := (*bool)(pos)
    fmt.Println(*b) // read the value
    *b = true      // write the value
}
```

You use reflection to access type information about field `b`. The `FieldByName` method returns a `reflect.StructField` instance for any field on a struct, even unexported ones. The instance includes the offset of its associated field. You then convert `huf` into an `unsafe.Pointer` and use the `unsafe.Add` method to add the offset to the pointer in order

to find the location of `b` within the struct. All that remains is casting the `unsafe.Pointer` returned by `Add` to a `*bool`. Now you can read the value of `b` or set its value. You can try out this code in the *sample_code/unexported_field_access* directory in the [Chapter 16 repository](#).

您可以使用反射来访问字段 `b` 的类型信息。`FieldByName` 方法会为结构体上的任何字段（即使是未导出的字段）返回一个 `reflect.StructField` 实例。该实例包含其关联字段的偏移量。然后，您可以将 `huf` 转换为 `unsafe.Pointer`，并使用 `unsafe.Add` 方法将偏移量添加到指针，以便找到结构体中 `b` 的位置。剩下的工作就是将 `Add` 返回的 `unsafe.Pointer` 强制转换为 `*bool`。现在，您可以读取 `b` 的值或设置其值。您可以在第 16 章代码库的 *sample_code/unexported_field_access* 目录中尝试这段代码。

Using unsafe Tools

使用不安全的工具

Go is a language that values tooling, and a compiler flag can help you find misuse of `Pointer` and `unsafe.Pointer`. Run your code with the flag `-gcflags=-d=checkptr` to add additional checks at runtime. Like the race checker, it's not guaranteed to find every `unsafe` problem and it does slow your program. However, it's a good practice while testing your code.

Go 语言非常重视工具链，编译器标志可以帮助你发现 `Pointer` 和 `unsafe.Pointer` 的误用。使用标志 `-gcflags=-d=checkptr` 运行代码，可以在运行时添加额外的检查。与竞态检查器类似，它并不能保证找到所有 `unsafe` 问题，而且会降低程序运行速度。但是，在测试代码时，这是一个很好的做法。

If you want to learn more about `unsafe`, read through the package [documentation](#).

如果您想了解有关 `unsafe` 的更多信息，请阅读软件包文档。

WARNING

警告

The `unsafe` package is powerful and low-level! Avoid using it unless you know what you are doing and you need the performance improvements that it provides.

`unsafe` 包功能强大且底层！除非您清楚自己在做什么并且需要它提供的性能提升，否则请避免使用它。

Cgo Is for Integration, Not Performance

Cgo 是用于集成，而非性能

Just like reflection and `unsafe`, `cgo` is most useful at the border between Go programs and the outside world. Reflection helps integrate with external

textual data, `unsafe` is best used with operating system and network data, and `cgo` is best for integrating with C libraries.

与反射和 `unsafe` 类似, `cgo` 在 Go 程序与外部世界交互时最为有用。反射有助于集成外部文本数据, `unsafe` 最适合用于操作系统和网络数据, 而 `cgo` 最适合与 C 库集成。

Despite being nearly 50 years old, C is still the *lingua franca* of programming languages. All the major operating systems are primarily written in either C or C++, which means that they are bundled with libraries written in C. It also means that nearly every programming language provides a way to integrate with C libraries. Go calls its foreign function interface (FFI) to C `cgo`.

尽管已有近 50 年的历史, C 语言仍然是编程语言的通用语言。所有主流操作系统主要都是用 C 或 C++ 编写的, 这意味着它们都捆绑了用 C 语言编写的库。这也意味着几乎所有编程语言都提供了与 C 库集成的方法。Go 语言调用其外部函数接口 (FFI) 来与 C 语言交互 `cgo`。

As you have seen many times, Go is a language that favors explicit specification. Go developers sometimes deride automatic behaviors in other languages as “magic.” However, using `cgo` feels a bit like spending time with Merlin. Let’s take a look at this magical glue code.

正如你多次看到的, Go 语言鼓励显式定义。Go 开发者有时会嘲讽其他语言中的自动行为是“魔法”。然而, 使用 `cgo` 的感觉就像是与梅

林共度时光。让我们来看看这段神奇的粘合代码。

You'll start with a very simple program that calls C code to do some math. The source code is in GitHub in the *sample_code/call_c_from_go* directory in the [Chapter 16 repository](#). First, here's the code in *main.go*:

你将从一个非常简单的程序开始，该程序调用 C 代码进行一些数学运算。源代码位于 GitHub 上第 16 章代码库的 *sample_code/call_c_from_go* 目录中。首先，以下是 *main.go* 中的代码：

```
package main

import "fmt"

/*
#cgo LDFLAGS: -lm
#include <stdio.h>
#include <math.h>
#include "mylib.h"

int add(int a, int b) {
    int sum = a + b;
    printf("a: %d, b: %d, sum %d\n", a, b,
sum);
    return sum;
}
```

```
*/  
import "C"  
  
func main() {  
    sum := C.add(3, 2)  
    fmt.Println(sum)  
    fmt.Println(C.sqrt(100))  
    fmt.Println(C.multiply(10, 20))  
}
```

The *mylib.h* header is in the same directory as your *main.go*:

mylib.h 头文件与 main.go 文件位于同一目录下:

```
int multiply(int a, int b);
```

There is also *mylib.c*:

还有 mylib.c:

```
#include "mylib.h"  
  
int multiply(int a, int b) {  
    return a * b;  
}
```

Assuming you have a C compiler installed on your computer, all you need to do is compile your program with `go build`:

假设你的电脑上已经安装了 C 编译器，你只需要使用 `go build` 编译你的程序即可：

```
$ go build
$ ./call_c_from_go
a: 3, b: 2, sum 5
5
10
200
```

What's going on here? The standard library doesn't have a real package named `C`. Instead, `C` is an automatically generated package whose identifiers mostly come from the C code embedded in the comments that immediately precede it. In this example, you declare a C function called `add`, and `cgo` makes it available to your Go program as the name `C.add`. You can also invoke functions or global variables that are imported into the comment block from libraries via header files, as you can see when you call `C.sqrt` from `main` (imported from *math.h*) or `C.multiply` (imported from *mylib.h*).

这里发生了什么？标准库中并没有名为 `c` 的实际包。相反，`c` 是一个自动生成的包，其标识符主要来自紧邻其前面的注释中嵌入的 C 代

码。在这个例子中，你声明了一个名为 `add` 的 C 函数，而 `cgo` 则将其作为 `C.add` 提供给你的 Go 程序。你还可以调用通过头文件从库导入到注释块中的函数或全局变量，例如，当你从 `main`（从 `math.h` 导入）或 `C.multiply`（从 `mylib.h` 导入）调用 `C.sqrt` 时，就可以看到这一点。

In addition to the identifier names that appear in the comment block (or are imported into the comment block), the `C` pseudopackage also defines types like `C.int` and `C.char` to represent the built-in C types and functions, such as `C.CString` to convert a Go string to a C string.

除了出现在注释块中的标识符名称（或导入到注释块中的标识符名称）之外，`C` 伪包还定义了 `C.int` 和 `C.char` 等类型来表示内置的 C 类型和函数，例如 `C.CString` 将 Go 字符串转换为 C 字符串。

You can use more magic to call Go functions from C functions. Go functions can be exposed to C code by putting an `//export` comment before the function. You can see this in use in the *sample_code/call_go_from_c* directory in the [Chapter 16 repository](#). In *main.go*, you export the doubler function:

你可以使用更多技巧从 C 函数调用 Go 函数。通过在函数名前添加 `//export` 注释，可以将 Go 函数暴露给 C 代码。你可以在第 16 章代码库的 *sample_code/call_go_from_c* 目录中看到它的用法。在 *main.go* 文件中，你可以导出 `doubler` 函数：

```
//export doubler
func doubler(i int) int {
    return i * 2
}
```

When you export a Go function, you can no longer write C code directly in the comment before the `import "C"` statement. You can only list function headers:

导出 Go 函数时，不能再直接在 `import "C"` 语句前的注释中编写 C 代码。只能列出函数头：

```
/*
    extern int add(int a, int b);
*/
import "C"
```

Put your C code into a `.c` file in the same directory as your Go code and include the magic header `"cgo_export.h"`. You can see this in the *example.c* file:

将您的 C 代码放在与 Go 代码相同的目录下的一个 `.c` 文件中，并包含魔法头文件 `"cgo_export.h"`。您可以在 `example.c` 文件中看到这一点：

```
#include "_cgo_export.h"

int add(int a, int b) {
    int doubleA = doubler(a);
    int sum = doubleA + b;
    return sum;
}
```

Running this program with `go build` give the following output:

使用 `go build` 运行此程序，将得到以下输出：

```
$ go build
$ ./call_go_from_c
8
```

So far, this seems pretty simple, but one major stumbling block arises when using `cgo`: Go is a garbage-collected language, and C is not. This makes it difficult to integrate nontrivial Go code with C. While you can pass a pointer into C code, you cannot directly pass something that contains a pointer. This is very limiting, as things like strings, slices, and functions are implemented with pointers and therefore cannot be contained in a struct passed into a C function.

到目前为止，这看起来很简单，但在使用 `cgo` 时会出现一个主要的障碍：Go 是一种垃圾回收语言，而 C 不是。这使得将复杂的 Go 代码与 C 代码集成变得困难。虽然你可以将指针传递给 C 代码，但你不能直接传递包含指针的对象。这非常局限，因为字符串、切片和函数等都是用指针实现的，因此不能包含在传递给 C 函数的结构体中。

That's not all: a C function cannot store a copy of a Go pointer that lasts after the function returns. If you break these rules, your program will compile and run, but it may crash or behave incorrectly at runtime when the memory pointed to by the pointer is garbage collected.

不仅如此：C 函数不能存储 Go 指针的副本，使其在函数返回后仍然存在。如果违反这些规则，程序虽然可以编译运行，但当指针指向的内存被垃圾回收时，程序可能会崩溃或运行异常。

If you find yourself needing to pass an instance of a type that contains a pointer from Go to C and then back into Go, you can use a `cgo.Handle` to wrap the instance. Here's a short example. You can find the source code in the *sample_code/handle* directory in the [Chapter 16 repository](#).

如果您需要将包含指针的类型实例从 Go 传递到 C，然后再传递回 Go，可以使用 `cgo.Handle` 来包装该实例。以下是一个简短的示例。您可以在第 16 章代码库的 *sample_code/handle* 目录中找到源代码。

First you have your Go code:

首先, 你需要编写 Go 代码:

```
package main

/*
    #include <stdint.h>

    extern void in_c(uintptr_t handle);
*/
import "C"

import (
    "fmt"
    "runtime/cgo"
)

type Person struct {
    Name string
    Age  int
}

func main() {
    p := Person{
        Name: "Jon",
        Age:  21,
    }
    C.in_c(C.uintptr_t(cgo.NewHandle(p)))
}
```



```
//export processor
func processor(handle C.uintptr_t) {
    h := cgo.Handle(handle)
    p := h.Value().(Person)
    fmt.Println(p.Name, p.Age)
    h.Delete()
}
```

And this is the C code:

这是C语言代码：

```
#include <stdint.h>
#include "_cgo_export.h"

void in_c(uintptr_t handle) {
    processor(handle);
}
```

In the Go code, you're passing a `Person` instance into the C function `in_c`. This function in turn calls the Go function `processor`. You can't safely pass a `Person` into C via `cgo`, because one of its fields is a `string`, and every `string` contains a pointer. To make this work, use the `cgo.NewHandle` function to convert `p` to a `cgo.Handle`. You then cast the `Handle` to a `C.uintptr_t` so you can pass it to the C function

`in_c`. The function takes in a single parameter of type `uintptr_t`, which is a C type that is analogous to Go's `uintptr` type. The `in_c` function calls the Go function `process`, which also takes in a single parameter of type `C.uintptr_t`. It casts this parameter to a `cgo.Handle`, and then uses a type assertion to convert the `Handle` to a `Person` instance. You print out the fields in `p`. Now that you are done using the `Handle`, you call the `Delete` method to delete it.

在 Go 代码中，您将一个 `Person` 实例传递给 C 函数 `in_c`。该函数又会调用 Go 函数 `processor`。您无法通过 `cgo` 安全地将 `Person` 传递给 C，因为它的一个字段是 `string`，而每个 `string` 都包含一个指针。为了解决这个问题，请使用 `cgo.NewHandle` 函数将 `p` 转换为 `cgo.Handle`。然后，将 `Handle` 强制转换为 `C.uintptr_t`，以便将其传递给 C 函数 `in_c`。该函数接受一个 `uintptr_t` 类型的参数，这是一个类似于 Go 的 `uintptr` 类型的 C 类型。`in_c` 函数调用 Go 函数 `process`，该函数也接收一个 `C.uintptr_t` 类型的参数。它将该参数强制转换为 `cgo.Handle` 类型，然后使用类型断言将 `Handle` 转换为 `Person` 实例。您打印出 `p` 中的字段。现在您已使用完 `Handle`，因此调用 `Delete` 方法将其删除。

In addition to working with pointers, other `cgo` limitations exist. For example, you cannot use `cgo` to call a variadic C function (such as `printf`). Union types in C are converted into byte arrays. And you cannot invoke a C function pointer (but you can assign it to a Go variable and pass it back into a C function).

除了指针操作之外，还有其他一些限制。例如，您不能使用 `cgo` 来调用可变参数 C 函数（例如 `printf`）。C 中的联合类型会被转换为字节数组。您也不能直接调用 C 函数指针（但您可以将其赋值给一个 Go 变量，然后再将其传递给 C 函数）。

These rules make using `cgo` nontrivial. If you have a background in Python or Ruby, you might think that using `cgo` is worth it for performance reasons. Those developers write the performance-critical parts of their programs in C. The speed of NumPy is due to the C libraries that are wrapped by Python code.

这些规则使得使用 `cgo` 并非易事。如果您有 Python 或 Ruby 的背景，您可能会认为出于性能考虑，使用 `cgo` 是值得的。这些开发者会用 C 语言编写程序中对性能要求极高的部分。NumPy 的速度之所以如此之快，是因为它使用了被 Python 代码封装的 C 库。

In most cases, Go code is many times faster than Python or Ruby, so the need to rewrite algorithms in a lower-level language is greatly reduced. You might think that you could save `cgo` for those situations when you do need additional performance gains, but unfortunately, using `cgo` to make your code faster is difficult. Because of the mismatches in the processing and memory models, calling a C function from Go is roughly 29 times slower than a C function calling another C function.

在大多数情况下，Go 代码的运行速度比 Python 或 Ruby 快很多倍，因此大大减少了用底层语言重写算法的需求。你可能会认为可以把 `cgo` 留到真正需要提升性能的时候再用，但遗憾的是，用 `cgo` 来加速代码非常困难。由于处理模型和内存模型的不匹配，从 Go 调用 C 函数的速度大约比 C 函数调用另一个 C 函数慢 29 倍。

At CapitalGo 2018, Filippo Valsorda gave a talk, called “Why cgo is slow.” Unfortunately, the talk wasn’t recorded, but [slides are available](#). They explain why `cgo` is slow and why it will not be made appreciably faster in the future. In his blog post [“CGO Performance in Go 1.21,”](#) Shane Hansen measured the overhead of a `cgo` call in Go 1.21 at roughly 40ns on an Intel Core i7-12700H.

在 2018 年 CapitalGo 大会上，Filippo Valsorda 发表了题为“为什么 `cgo` 很慢”的演讲。遗憾的是，演讲没有录像，但幻灯片可以获取。幻灯片解释了 `cgo` 函数速度慢的原因，以及为什么未来它的速度不会有显著提升。Shane Hansen 在他的博客文章“Go 1.21 中的 CGO 性能”中，测量了在 Intel Core i7-12700H 处理器上，Go 1.21 中 `cgo` 函数调用的开销约为 40 纳秒。

TIP

提示

Since `cgo` isn't fast, and it isn't easy to use for nontrivial programs, the only reason to use `cgo` is if you must use a C library and there is no suitable Go replacement. Rather than writing `cgo` yourself, see if a third-party module already provides the wrapper. For example, if you want to embed SQLite in a Go application, look at [GitHub](#). For ImageMagick, check out [this repository](#).

由于 `cgo` 速度不快，而且对于复杂的程序来说也不容易使用，因此使用 `cgo` 的唯一理由是，您必须使用 C 库，并且没有合适的 Go 替代方案。与其自己编写 `cgo`，不如看看是否有第三方模块已经提供了封装。例如，如果您想在 Go 应用程序中嵌入 SQLite，请查看 GitHub。对于 ImageMagick，请查看此仓库。

If you find yourself needing to use an internal C library or third-party library that doesn't have a wrapper, you can find [additional details](#) on how to write your integration in the Go documentation. For information on the sorts of performance and design trade-offs that you are likely to encounter when using `cgo`, read Tobias Grieger's blog post called "[The Cost and Complexity of Cgo](#)."

如果您需要使用没有封装的内部 C 库或第三方库，可以在 Go 文档中找到有关如何编写集成代码的更多详细信息。要了解使用 `cgo` 时可能遇到的性能和设计权衡，请阅读 Tobias Grieger 的博文“Cgo 的成本和复杂性”。

Exercises

练习

Now it's time to test yourself to see if you can write small programs using reflection, `unsafe`, and `cgo`. Answers for the exercises are found in the *exercise_solutions* directory in the [Chapter 16 repository](#).

现在是时候测试一下自己，看看你是否能使用反射、`unsafe` 和 `cgo` 编写小程序了。练习答案位于第 16 章代码库的 *exercise_solutions* 目录中。

1. Use reflection to create a simple minimal string-length validator for struct fields. Write a `ValidateStringLength` function that takes in a struct and returns an error if one or more of the fields is a string, has a struct tag called `minStrlen`, and the length of the value in the field is less than the value specified in the struct tag. Nonstring fields and string fields that don't have the `minStrlen` struct tag are ignored. Use `errors.Join` to report all invalid fields. Be sure to validate that a struct was passed in. Return `nil` if all fields are of the proper length.

使用反射创建一个简单的最小字符串长度验证器，用于验证结构体字段。编写一个名为 `ValidateStringLength` 的函数，该函数接收一个结构体作为参数，如果其中一个或多个字段是字符串

类型，且具有名为 `minStrlen` 的结构体标签，并且字段值的长度小于结构体标签中指定的值，则返回错误。非字符串字段和没有 `minStrlen` 结构体标签的字符串字段将被忽略。使用 `errors.Join` 报告所有无效字段。务必验证传入的是否为结构体。如果所有字段的长度都符合要求，则返回 `nil`。

2. Use `unsafe.Sizeof` and `unsafe.Offsetof` to print out the size and offsets for the `OrderInfo` struct defined in [ch16/tree/main/sample_code/orders](#). Create a new type, `SmallOrderInfo`, that has the same fields, but reordered to use as little memory as possible.

使用 `unsafe.Sizeof` 和 `unsafe.Offsetof` 打印出 `ch16/tree/main/sample_code/orders` 中定义的 `OrderInfo` 结构体的大小和偏移量。创建一个新类型 `SmallOrderInfo`，它具有相同的字段，但重新排序以尽可能减少内存占用。

3. Copy the C code at [ch16/tree/main/sample_code/mini_calc](#) into your own module and use `cgo` to call it from a Go program.

将 `ch16/tree/main/sample_code/mini_calc` 中的 C 代码复制到您自己的模块中，并使用 `cgo` 从 Go 程序中调用它。

Wrapping Up

总结

In this chapter, you've learned about reflection, `unsafe`, and `cgo`. These features are probably the most exciting parts of Go because they allow you to break the rules that make Go a boring, type-safe, memory-safe language. More importantly, you've learned *why* you would want to break the rules and why you should avoid doing so most of the time.

在本章中，你学习了反射、`unsafe` 和 `cgo`。这些特性可能是 Go 语言中最令人兴奋的部分，因为它们允许你打破那些使 Go 成为一门枯燥乏味、类型安全、内存安全的语言的规则。更重要的是，你了解了为什么要打破这些规则，以及为什么大多数情况下应该避免这样做。

You've completed your journey through Go and how to use it idiomatically. As with any graduation ceremony, it's time for a few closing words. Let's look back at what was said in the preface. “[P]roperly written, Go is boring.... Well-written Go programs tend to be straightforward and sometimes a bit repetitive.” I hope you can now see why this leads to better software engineering.

你已经完成了 Go 语言的学习之旅，并掌握了它的惯用用法。就像任何毕业典礼一样，是时候说几句结语了。让我们回顾一下前言中的内

容。“编写得当的 Go 语言代码其实很枯燥……优秀的 Go 程序往往简洁明了，有时甚至略显重复。”我希望你现在能够理解，为什么这反而有助于提升软件工程水平。

Idiomatic Go is a set of tools, practices, and patterns that makes it easier to maintain software across time and changing teams. That's not to say the cultures around other languages don't value maintainability; it just may not be their highest priority. Instead, they emphasize things like performance, new features, or concise syntax. There is a place for these trade-offs, but in the long run, I suspect Go's focus on crafting software that lasts will win out. I wish you the best as you create the software for the next 50 years of computing.

Go 语言的惯用模式是一套工具、实践和模式，它能帮助开发者更轻松地维护软件，使其能够跨越时间和团队的更迭。这并非意味着其他语言的开发文化不重视可维护性；只是可维护性可能并非它们的首要任务。相反，它们更注重性能、新功能或简洁的语法。这些权衡取舍固然有其存在的意义，但从长远来看，我认为 Go 语言专注于打造持久耐用的软件最终会胜出。祝愿您在为未来 50 年的计算机世界开发软件的过程中一切顺利。

Index

指数

Symbols

符号

- " (double quote)
(双引号)
 - not interchangeable with single quote, [Literals](#)
不可与单引号、字面量互换
 - string literals, [Literals](#)
字符串字面量，字面量
- % (modulus) for integer types, [Integer operators](#)
 - not floating-point types, [Floating-point types](#)

%（取模）运算符适用于整数类型，整数运算符不适用于浮点类型，浮点类型

- & (ampersand) as pointer address operator, [A Quick Pointer Primer](#)
使用 &（& 符号）作为指针地址运算符，《指针快速入门》
- ' (single quote)
'（单引号）

- not interchangeable with double quote, [Literals](#)
不可与双引号、字面值互换
- rune literals, [Literals](#)
符文字面值，字面值
- * (asterisk)
 - * (星号)
 - pointer indirection operator, [A Quick Pointer Primer](#)
指针间接运算符，指针快速入门
 - pointer type declaration, [A Quick Pointer Primer](#)
指针类型声明，指针快速入门
- - (dash) for ignored field in marshaling or unmarshaling, [Using Struct Tags to Add Metadata](#)
- (短横线) 表示在序列化或反序列化过程中忽略的字段，使用结构标签添加元数据
- ... operator
 - 。 。 。 操作员
 - slice appended onto another, [append](#)
将一片切片附加到另一片切片上，附加
 - variadic input parameter, [Variadic Input Parameters and Slices](#)
可变输入参数、可变输入参数和切片
- . prefix for hidden file names, [Embedding Hidden Files](#)
隐藏文件名的 . 前缀，嵌入隐藏文件
- ./..., [go fmt](#)
./..., 去 fmt

- 0b (binary prefix), [Literals](#)

0b (二进制前缀) , 字面量

- 0o (octal prefix), [Literals](#)

- 0 alone should not be used, [Literals](#)

0o (八进制前缀) , 字面量0不应单独使用, 字面量

- 0x (hexadecimal prefix), [Literals](#)

0x (十六进制前缀) , 字面量

- := versus var, [var Versus :=-var Versus :=](#)

- shadowing variables, [Shadowing Variables-Shadowing Variables](#)

:= 与 var 比较, var 与 :=-var 与 := 遮蔽变量, 遮蔽变量-遮蔽变量

- ; (semicolon) insertion rule, [go fmt](#)

; (分号) 插入规则, go fmt

- <- for channel access, [Reading, Writing, and Buffering](#)

<- 用于通道访问、读取、写入和缓冲

- = (equal sign)

等号

- := versus var, [var Versus :=-var Versus :=](#)

:= 与 var、 var 与 :=-var 与 :=

- arithmetic operators, [Integer operators](#)

算术运算符, 整数运算符

- comparison operators

比较算子

- arrays, [Arrays—Too Rigid to Use Directly](#)
数组，数组——过于僵化而无法直接使用
- comparable interface, [Introducing Generics in Go](#), [More on comparable](#)
可比较的接口，Go 语言中的泛型引入，更多关于可比较的内容
- complex types, [Complex types \(you're probably not going to use these\)](#)
复杂类型，复杂类型（您可能不会用到这些）
- floating-point types, [Floating-point types](#)
浮点类型，浮点类型
- integer types, [Integer operators](#)
整数类型，整数运算符
- interfaces comparable, [Interfaces Are Comparable](#)
接口可比性，接口是可比的
- maps not comparable, [Maps](#)
地图不具可比性，地图
- slices not comparable, [Slices](#)
切片不具有可比性，切片
- string types, [A Taste of Strings and Runes](#)
字符串类型，字符串和符文入门
- structs comparable if fields comparable, [Comparing and Converting Structs](#)
如果字段可比较，则结构体可比较；比较和转换结构体

- time.Time methods, [time](#), [time](#)
时间。时间方法，时间，时间

- @latest

- @最新的

- installing third-party tools, [Adding Third-Party Tools with go install](#)
 - updating a tool, [Adding Third-Party Tools with go install](#)

安装第三方工具，使用 go install 添加第三方工具；更新工具，使用 go install 添加第三方工具

- retracted version not matching, [Retracting a Version of Your Module](#)

撤回版本不匹配，撤回您的模块版本

- @version

- @版本

- installing third-party tools, [Adding Third-Party Tools with go install](#)
 - updating a tool, [Adding Third-Party Tools with go install](#)

安装第三方工具，使用 go install 添加第三方工具；更新工具，使用 go install 添加第三方工具

- [] (brackets) for slices versus arrays, [Slices](#)

[]（方括号）用于区分切片和数组，切片

- `_` (underscore)
 _ (下划线)
 - blank imports, [Avoiding the init Function if Possible](#)
 空白导入，尽可能避免使用初始化函数
 - first iota value when ignored, [iota Is for Enumerations—Sometimes](#)
 第一个 iota 值被忽略时，iota 用于枚举——有时
 - floating-point literals, [Literals](#)
 浮点字面量，字面量
 - hidden file name prefix, [Embedding Hidden Files](#)
 隐藏文件名前缀，嵌入隐藏文件
 - integer literals, [Literals](#)
 整数字面量，字面量
 - return value ignored, [The for-range Statement, Ignoring Returned Values](#)
 返回值被忽略，for-range 语句，忽略返回值
 - return value nameless among named, [Named Return Values](#)
 返回值无名，但有名返回值
 - targeted code build tags, [Using Build Tags](#)
 使用构建标签进行定向代码构建标签
 - unused variable and constant names, [The for-range Statement](#)
 未使用的变量和常量名称，for-range 语句
 - variable and constant names not using, [Naming Variables and Constants](#)

未使用的变量和常量名称，变量和常量的命名

- ` (backquote)

` (反引号)

- raw string literals, [Literals](#)

原始字符串字面量，字面量

- struct tags, [Using Struct Tags to Add Metadata](#)

结构体标签，使用结构体标签添加元数据

- {} (braces)

{ } (花括号)

- defining a block, [go build](#), [Blocks](#)

定义一个代码块，然后构建，代码块

- empty interface, [The Empty Interface Says Nothing](#)

空界面，空界面什么也不说

- not fixed by go fmt, [go fmt](#)

go fmt 未修复此问题

- opening brace placement, [go fmt](#)

打开支架位置，前往 fmt

- ~ (tilde) for types with underlying type, [Use Type Terms to Specify](#)

[Operators](#)

对于具有底层类型的类型，请使用类型术语来指定运算符

(~) 。

A

一个

- abstract types, [Types in Go](#)
 - interfaces, [A Quick Lesson on Interfaces](#)

抽象类型、Go 接口中的类型、接口速成课

- addresses

地址

- pointers pointing to, [A Quick Pointer Primer](#)
指向指针的指针, 《指针快速入门》
- variable storage in memory, [A Quick Pointer Primer](#)
内存中的变量存储, 快速指针入门
- aliases, [Gracefully Renaming and Reorganizing Your API](#)
别名, 优雅地重命名和重组您的 API
- alice module for http.Server, [Use third-party modules to enhance the server](#)
alice 模块用于 http.Server, 使用第三方模块来增强服务器功能。
- Amdahl, Gene, [When to Use Concurrency](#)
Amdahl, Gene, 《何时使用并发》
- Amdahl's law, [When to Use Concurrency](#)
阿姆达尔定律, 何时使用并发

- ampersand (&) as pointer address operator, [A Quick Pointer Primer](#)
指针地址运算符：& 符号（指针）入门
- anonymous functions, [Anonymous Functions](#)
 - as closures, [Closures-Returning Functions from Functions](#)

匿名函数、匿名函数作为闭包、闭包——从函数返回函数

- anonymous structs, [Anonymous Structs](#)
 - comparing structs, [Comparing and Converting Structs](#)
 - unmarshaling and marshaling data, [Anonymous Structs](#)

匿名结构体、匿名结构体比较、结构体比较与转换、数据反序列化与序列化、匿名结构体

- Antinyan, Vard, [defer](#)
安提尼扬，瓦尔德，推迟
- any as type alias for empty interface, [The Empty Interface Says Nothing](#)
 - comparable interface, [Interfaces Are Comparable](#), [Introducing Generics in Go](#), [More on comparable](#)
 - interfaces as type constraints, [Generics and Interfaces](#)
 - generic function creating a stack, [Introducing Generics in Go](#)

any 作为空接口的类型别名，空接口什么也不说，可比较接口，接口是可比较的，Go 语言中的泛型引入，更多关于可比较接口的

内容，接口作为类型约束，泛型和接口，创建栈的泛型函数，Go 语言中的泛型引入

- APIs

蜜蜂

- concurrency-free, [Keep Your APIs Concurrency-Free](#)
无并发，保持您的 API 无并发
- database/sql/driver package evolution, [Accept Interfaces, Return Structs, Use Type Assertions and Type Switches Sparingly](#)
数据库/SQL/驱动程序包的演变，接受接口，返回结构体，谨慎使用类型断言和类型切换
- exported identifiers in package API, [Importing and Exporting](#)
包 API 中的导出标识符、导入和导出
- internal packages not part of, [Using the internal Package](#)
不属于内部包的包，使用内部包
- language servers, [Visual Studio Code](#)
语言服务器，Visual Studio Code
- log/slog package documentation URL, [Structured Logging](#)
log/slog 包文档 URL，结构化日志记录
- maps for input parameters or return values, [The Difference Between Maps and Slices](#)
用于输入参数或返回值的映射，映射和切片的区别
- optional interfaces for evolving APIs, [Use Type Assertions and Type Switches Sparingly](#)

为不断演进的 API 提供可选接口，谨慎使用类型断言和类型切换。

- panics must not escape public APIs, [panic and recover](#)
panic 不应逃逸出公共 API，应进行 panic 和恢复。
- renaming and reorganizing, [Gracefully Renaming and Reorganizing Your API](#)

重命名和重组，优雅地重命名和重组您的 API

- sentinel errors a part of public API, [Sentinel Errors](#)
哨兵错误是公共 API 的一部分，哨兵错误
- testing your public API, [Testing Your Public API](#)
测试您的公共 API，测试您的公共 API
- type assertions and type switches, [Use Type Assertions and Type Switches Sparingly](#)

类型断言和类型开关，谨慎使用类型断言和类型开关

- APL (A Programming Language) and iota, [iota Is for Enumerations—Sometimes](#)

APL（一种编程语言）和 iota，iota 用于枚举——有时

- append function

追加函数

- not assigning return value as error, [append](#)
未将返回值赋为错误，追加
- slices grown via, [append](#)
 - capacity of slices, [Capacity](#)
 - full slice expression protecting subslices, [Slicing Slices](#)

- slice for input parameter then using append, [The Difference Between Maps and Slices](#)

通过以下方式增长的切片，切片的附加容量，保护子切片的满容量切片表达式，为输入参数对切片 Sliceslice 进行切片，然后使用附加，映射和切片的区别

- arithmetic operators

算术运算符

- complex types, [Complex types \(you're probably not going to use these\)](#)

复杂类型，复杂类型（您可能不会用到这些）

- dividing by zero

除以零

- floating-point types, [Floating-point types](#)

浮点类型，浮点类型

- integer types, [Integer operators](#)

整数类型，整数运算符

- floating-point types, [Floating-point types](#)

浮点类型，浮点类型

- integer types, [Integer operators](#)

- bit manipulation operators, [Integer operators](#)

- combined with =, [Integer operators](#)

- comparison operators, [Integer operators](#)

- Go specification URL, [Integer operators](#)

整数类型、整数运算符、位操作运算符、与 = 组合的整数运算符、整数运算符、比较运算符、整数运算符 Go 规范
URL、整数运算符

- type conversion, [Explicit Type Conversion](#)
类型转换, 显式类型转换
- array literals, [Arrays—Too Rigid to Use Directly](#)
数组字面量, 数组——过于僵化, 无法直接使用
- arrays, [Arrays—Too Rigid to Use Directly](#)-[Arrays—Too Rigid to Use Directly](#)
 - converting to slices, [Converting Arrays to Slices](#)
 - declaration, [Arrays—Too Rigid to Use Directly](#)
 - function input parameters, [The Difference Between Maps and Slices](#)
 - len function for length, [Arrays—Too Rigid to Use Directly](#)
 - multidimensional arrays simulated, [Arrays—Too Rigid to Use Directly](#)
 - reading and writing, [Arrays—Too Rigid to Use Directly](#)
 - size affecting array type, [Arrays—Too Rigid to Use Directly](#)
 - converting array to slice to array, [Converting Slices to Arrays](#)
 - slices contrasted, [Slices](#)
 - stack and, [Reducing the Garbage Collector's Workload](#)
 - slices converted to, [Converting Slices to Arrays](#)

数组，数组——过于僵化而无法直接使用——数组——过于僵化而无法直接使用——转换为切片，将数组转换为切片——声明，数组——过于僵化而无法直接使用——函数输入参数，映射和切片的区别——长度的len函数，数组——过于僵化而无法直接使用——模拟多维数组，数组——过于僵化而无法直接使用——读写，数组——过于僵化而无法直接使用——大小影响数组类型，数组——过于僵化而无法直接使用——将数组转换为切片再转换为数组，将切片转换为数组——切片对比，切片堆栈和，减少垃圾回收器的工作负载——切片转换为，将切片转换为数组

- The Art of Concurrency (Breshears), [When to Use Concurrency](#)
并发的艺术（布雷谢尔斯），何时使用并发

- Artifactory proxy server support, [Specifying a Proxy Server](#)
Artifactory代理服务器支持，指定代理服务器

- asterisk (*)

星号 (*)

- pointer indirection operator, [A Quick Pointer Primer](#)
指针间接运算符，指针快速入门
- pointer type declaration, [A Quick Pointer Primer](#)
指针类型声明，指针快速入门
- Athens Project open source proxy server, [Specifying a Proxy Server](#)
 - authentication configuration documentation URL, [Using Private Repositories](#)

Athens Project 开源代理服务器，指定代理服务器身份验证配置文件 URL，使用私有仓库

- atomic variables, [Atomics—You Probably Don't Need These](#)
原子变量，原子操作——你可能不需要这些
- automatic type promotion, [Explicit Type Conversion](#)
自动类型提升，显式类型转换

B

B

- B suffix, [Tuning the Garbage Collector](#)
B 后缀，调整垃圾回收器
- backpressure in buffered channels, [Implement Backpressure](#)
缓冲通道中的背压，实现背压
- backquote (`)
反引号 (`)
 - raw string literals, [Literals](#)
原始字符串字面量，字面量
 - struct tags, [Using Struct Tags to Add Metadata](#)
结构体标签，使用结构体标签添加元数据
- backslash rune literal (`\`), [Literals](#)
反斜杠符文字面量 (`\`) ， 字面量

- backslash-escaped rune literals, [Literals](#)
反斜杠转义的符文字面量，字面量
- backward compatibility of Go, [Preface](#)
 - Go Compatibility Promise, [The Go Compatibility Promise](#),
[Accept Interfaces](#), [Return Structs](#)

Go 的向后兼容性、前言、Go 兼容性承诺、Go 兼容性承诺、接受接口、返回结构体

- Bazaar for version control, [Publishing Your Module](#)
版本控制市场，发布您的模块
- benchmarks, [Using Benchmarks](#)
基准测试，使用基准测试
- Bendersky, Eli, [Idiomatic Go and Generics](#), [Organizing Your Module](#)
Bendersky, Eli, 《惯用 Go 和泛型》，组织你的模块
- Bhargava, Aditya, [Maps](#)
Bhargava、Aditya、地图
- binary prefix (0b), [Literals](#)
 - uses for binary, [Literals](#)

二进制前缀 (0b), Literalsuses 表示二进制, Literals

- binary trees
二叉树
 - generic functions with generic data structures, [Combining Generic Functions with Generic Data Structures](#)-[Combining](#)

Generic Functions with Generic Data Structures

通用函数与通用数据结构的结合，将通用函数与通用数据结构结合起来——将通用函数与通用数据结构结合起来

- generic types for, [Generics Reduce Repetitive Code and Increase Type Safety](#)

泛型类型，泛型减少重复代码并提高类型安全性

- nil values for value receiver, [Code Your Methods for nil Instances](#)
值接收器为空值，请为空实例编写方法
- bit manipulation operators, [Integer operators](#)
位操作运算符、整数运算符
- blank imports, [Avoiding the init Function if Possible](#)
 - embed package import, [Embedding Content into Your Program](#)

空白导入，尽可能避免使用初始化函数，嵌入包导入，将内容嵌入到您的程序中

- blank returns never should be used, [Blank Returns—Never Use These!](#)
空白回车符绝对不能使用，空白回车符——千万不要使用！
- blank switches, [Blank Switches](#)
空白开关，空白开关
- blocks, [Blocks](#)
 - file block, [Blocks](#)
 - package blocks, [var Versus :=, Blocks](#)
 - universe block, [Shadowing Variables](#)

块、块文件块、块包块、变量 Versus :=、块宇宙块、变量阴影

- book supplemental material URL, [Using Code Examples](#)
 - code repository
 - Chapter 1, [Makefiles-Exercises](#)
 - Chapter 2, [Complex types \(you're probably not going to use these\)](#), [Explicit Type Conversion](#), [Using const](#), [Unused Variables-Exercises](#)
 - Chapter 3, [Capacity](#), [Slicing Slices-Converting Arrays to Slices](#), [Converting Slices to Arrays-Strings and Runes and Bytes](#), [Reading and Writing a Map](#), [Using Maps as Sets](#), [Exercises](#)
 - Chapter 4, [Shadowing Variables-if](#), [The Infinite for Statement](#), [The for-range Statement-Labeling Your for Statements](#), [switch](#), [switch-switch](#), [Choosing Between if and switch-goto—Yes,.goto](#), [Exercises](#)
 - Chapter 5, [Declaring and Calling Functions-Simulating Named and Optional Parameters](#), [Multiple Return Values](#), [Named Return Values](#), [Functions Are Values](#), [Anonymous Functions-Anonymous Functions](#), [Passing Functions as Parameters-defer](#), [defer](#), [defer-Exercises](#)
 - Chapter 6, [Don't Fear the Pointers](#), [Pointer Passing Performance](#), [Exercises](#)

- Chapter 7, [Pointer Receivers and Value Receivers](#), [Code Your Methods for nil Instances](#), [Embedding Is Not Inheritance](#), [A Quick Lesson on Interfaces](#), [Interfaces and nil](#), [Interfaces Are Comparable](#), [Type Assertions and Type Switches](#), [Use Type Assertions and Type Switches Sparingly](#), [Implicit Interfaces Make Dependency Injection Easier-Exercises](#)
- Chapter 8, [Generics Reduce Repetitive Code and Increase Type Safety](#), [Introducing Generics in Go](#), [Generic Functions Abstract Algorithms](#), [Generics and Interfaces-Type Inference and Generics](#), [Combining Generic Functions with Generic Data Structures-More on comparable](#), [Idiomatic Go and Generics-Exercises](#)
- Chapter 9, [How to Handle Errors: The Basics-Sentinel Errors](#), [Errors Are Values-Wrapping Errors](#), [Wrapping Multiple Errors-Is and As](#), [Is and As](#), [panic and recover-panic and recover](#), [Exercises](#)
- Chapter 10, [Use the go Directive to Manage Go Build Versions](#), [Overriding a Package's Name](#), [Avoiding Circular Dependencies](#)
- Chapter 11, [Using go run to Try Out Small Programs](#), [staticcheck-golangci-lint](#), [Embedding Content into Your Program](#), [Embedding Content into Your Program-](#)

[Embedding Hidden Files](#), [Using go generate-Working with go generate and Makefiles](#), [Exercises](#)

- Chapter 12, [Goroutines](#), [select-select](#), [Use the Context to Terminate Goroutines](#)-[Run Code Exactly Once](#), [Put Your Concurrent Tools Together](#), [When to Use Mutexes Instead of Channels](#), [Exercises](#)
- Chapter 13, [io and Friends](#), [JSON](#), [Readers, and Writers](#)-[Custom JSON Parsing](#), [Custom JSON Parsing](#), [The Client](#)-[The Server](#), [The Server](#), [Middleware](#), [ResponseController](#), [Structured Logging](#), [Exercises](#)
- Chapter 14, [What Is the Context?](#), [Values](#), [Values](#), [Cancellation](#), [Cancellation](#), [Contexts with Deadlines](#)-[Contexts with Deadlines](#), [Context Cancellation in Your Own Code](#)
- Chapter 15, [Understanding the Basics of Testing](#), [Storing Sample Test Data](#)-[Using go-cmp to Compare Test Results](#), [Running Table Tests](#), [Running Tests Concurrently](#)-[Checking Your Code Coverage](#), [Using Benchmarks](#), [Using Stubs in Go](#), [Using Stubs in Go](#), [Using httptest](#)-[Finding Concurrency Problems with the Data Race Detector](#)
- Chapter 16, [Types and kinds](#), [Make New Values](#)-[Use Reflection to Write a Data Marshaler](#), [Build Functions with Reflection to Automate Repetitive Tasks](#), [Use Reflection Only if It's Worthwhile](#), [Using Sizeof and Offsetof](#)-[Using](#)

[unsafe to Convert External Binary Data, Accessing Unexported Fields-Cgo Is for Integration, Not Performance, Exercises](#)

- goroutine_for_loop repository, [Goroutines, for Loops, and Varying Variables](#)
- money repository, [Importing Third-Party Code](#)
- package examples, [Creating and Accessing a Package](#)

书籍补充材料 URL, 使用代码示例代码库, 第 1 章, Makefiles - 练习, 第 2 章, 复杂类型 (您可能不会用到这些), 显式类型转换, 使用 const, 未使用的变量 - 练习, 第 3 章, 容量, 切片 - 将数组转换为切片, 将切片转换为数组 - 字符串、符文和字节, 读取和写入 Map, 将 Map 用作集合, 练习, 第 4 章, 变量遮蔽 - if, 无限循环 for 语句, for-range 语句 - 标记 for 语句, switch, switch-switch, 在 if 和 switch 之间进行选择 - goto - 是的, goto, 练习, 第 5 章, 声明和调用函数 - 模拟命名参数和可选参数, 多个返回值, 命名返回值, 函数即值, 匿名函数 - 匿名函数, 将函数作为参数传递 - defer, defer, defer-练习 第6章, 不要害怕指针, 指针传递性能, 练习 第7章, 指针接收器和值接收器, 为nil实例编写方法, 嵌入不是继承, 接口速览, 接口和nil, 接口是可比较的, 类型断言和类型切换, 谨慎使用类型断言和类型切换, 隐式接口使依赖注入更容易-练习 第8章, 泛型减少重复代码并提高类型安全性, Go语言中的泛型介绍, 泛型函数抽象算法, 泛型和接口-类型推断和泛型, 将泛型函数与泛型数据结构结合使用-

更多关于可比较的内容，惯用的Go语言和泛型-练习 第9章，如何处理错误：基础知识-哨兵错误，错误是值 - 错误包装、多个错误包装 - Is 和 As、Is 和 As、panic 和 recover、练习 第 10 章、使用 go 指令管理 Go 构建版本、覆盖包名、避免循环依赖 第 11 章、使用 go run 试运行小程序、staticcheck-golangci-lint、将内容嵌入程序、将内容嵌入程序 - 嵌入隐藏文件、使用 go generate - 使用 go generate 和 Makefile、练习 第 12 章、Goroutine、select-select、使用 Context 终止 Goroutine - 代码仅运行一次、将并发工具组合在一起、何时使用互斥锁而不是通道、练习 第 13 章、io 和相关内容、JSON、读取器和写入器 - 自定义 JSON 解析、自定义 JSON 解析、客户端 - 服务器、服务器、中间件、响应控制器结构化日志记录，练习 第14章，什么是上下文？，值，值，取消，取消，带截止时间的上下文，在自己的代码中取消上下文 第15章，理解测试基础，存储示例测试数据，使用 go-cmp 比较测试结果，运行表测试，并发运行测试，检查代码覆盖率，使用基准测试，在 Go 中使用存根，在 Go 中使用存根，使用 httptest，使用数据竞争检测器查找并发问题 第16章，类型和种类，创建新值，使用反射编写数据编组器，使用反射构造函数以自动化重复性任务，仅在值得时使用反射，使用 sizeof 和 offsetof，使用 unsafe 转换外部二进制数据，访问未导出的字段，Cgo 用于集成，而不是性能，练习 goroutine_for_loop 仓库Goroutine、for 循环和可变变量；money 存储库；导入第三方代码包示例；创建和访问包

- bool type, [Booleans](#)
 - explicit type conversion and booleans, [Explicit Type Conversion](#)

- zero value as false, [Booleans](#)

布尔类型，布尔值显式类型转换和布尔值，显式类型转换零值为 false，布尔值

- braces ({ })

花括号 ({ })

- defining a block, [go build](#), [Blocks](#)

定义一个代码块，然后构建，代码块

- empty interface, [The Empty Interface Says Nothing](#)

空界面，空界面什么也不说

- not fixed by go fmt, [go fmt](#)

go fmt 未修复此问题

- opening brace placement, [go fmt](#)

打开支架位置，前往 fmt

- brackets ([]) for slices versus arrays, [Slices](#)

切片与数组之间的区别在于方括号 ([])，切片

- Branson, Rick, [Reducing the Garbage Collector's Workload](#)

布兰森，里克，《减少垃圾收集员的工作量》

- break keyword

关键词

- for statement, [break and continue](#)

对于语句，中断并继续

- switch statement, [switch](#)

switch语句，switch

- Breshears, Clay, [When to Use Concurrency](#)
Breshears, Clay, 《何时使用并发》
- brew (see Homebrew installation of Go)
brew (参见 Homebrew 安装 Go)
- BSD installation of Go, [Installing the Go Tools](#)
 - updating Go tools, [Staying Up-to-Date](#)

在 BSD 系统下安装 Go 语言、安装 Go 工具、更新 Go 工具、保持最新

- buffers for reading data via slices, [Slices as Buffers](#)
 - reducing garbage collector's workload, [Slices as Buffers](#),
[Reducing the Garbage Collector's Workload](#)

通过切片读取数据的缓冲区，切片作为缓冲区减少垃圾回收器的工作负载，切片作为缓冲区，减少垃圾回收器的工作负载

- bufio package

bufio 包

- bufio.NewReader, [Use Type Assertions and Type Switches Sparingly](#)
bufio.NewReader, 请谨慎使用类型断言和类型切换
- bufio.Reader, [Use Type Assertions and Type Switches Sparingly](#)
bufio.Reader, 请谨慎使用类型断言和类型切换。
- *os.File instance with Scanner type, [io and Friends](#)
*os.File 实例，类型为 Scanner，包含 io 和 Friends

- bug detection via go vet, [go vet](#)
 - (see also testing)

通过 go vet 进行错误检测, go vet (另请参阅测试)

- build tags, [Using Build Tags](#)
 - integration tests and, [Using Integration Tests and Build Tags](#)

构建标签、使用构建标签集成测试以及使用集成测试和构建标签

- builds

建造

- before building

建造之前

- go fmt, [go fmt](#)
去 fmt, 去 fmt
- go vet, [go vet](#)
去看兽医, 去看兽医

- binaries for other platforms, [Building Go Binaries for Other Platforms](#)

为其他平台构建二进制文件, 为其他平台构建 Go 二进制文件

- go build, [go build](#)
去建设, 去建设
- go generate, [Using go generate-Using go generate](#)
go generate, 使用 go generate-使用 go generate

- go install, [Adding Third-Party Tools with go install](#)-[Adding Third-Party Tools with go install](#)
使用 go install 添加第三方工具 - 使用 go install 添加第三方工具
- go run to build and execute, [Using go run to Try Out Small Programs](#)
使用 `go run` 命令构建并执行程序, 使用 `go run` 尝试运行小型程序。
- Hello World, [Your First Go Program](#)-[go vet](#)
你好世界, 你的第一个 Go 程序 - go vet
- Makefiles, [Makefiles](#)
Makefiles, Makefiles
- semicolons added by compiler, [go fmt](#)
编译器添加了分号, go fmt
- single native binary, [Installing the Go Tools](#), [Importing Third-Party Code](#)
 - Go runtime compiled into, [Capacity](#)
 - Go tool updates and, [Staying Up-to-Date](#)
 - reading build info inside, [Reading the Build Info Inside a Go Binary](#)

单个原生二进制文件、安装 Go 工具、导入第三方代码、编译到 Go 运行时、容量、Go 工具更新和保持最新、读取内部构建信息、读取 Go 二进制文件中的构建信息

- byte for uint8, [The special integer types](#), [A Taste of Strings and Runes](#)
 - for statements iterating over strings, [Iterating over strings](#)
 - strings as sequences of, [Strings and Runes and Bytes](#)-[Strings and Runes and Bytes](#)
 - string bytes versus UTF-8 multi-byte code points, [Strings and Runes and Bytes](#)
 - type conversions to strings and runes, [Strings and Runes and Bytes](#)

uint8 的字节，特殊整数类型，字符串和符文入门，for 语句遍历字符串，遍历字符串，字符串作为序列，字符串、符文和字节——字符串、符文和字节，字符串字节与 UTF-8 多字节代码点，字符串、符文和字节，字符串和符文的类型转换，字符串、符文和字节

C

C

- C library integration

C 库集成

- C pseudopackage, [Cgo Is for Integration, Not Performance](#)-[Cgo Is for Integration, Not Performance](#)

C 伪包，Cgo 用于集成，而非性能 - Cgo 用于集成，而非性能

- cgo, [Cgo Is for Integration, Not Performance-Cgo Is for Integration, Not Performance](#)
cgo, cgo 用于集成, 而非性能 - cgo 用于集成, 而非性能
- calculator example code, [Functions Are Values](#)
计算器示例代码, 函数即值
- call by value language, [Go Is Call by Value-Go Is Call by Value, Pointers Indicate Mutable Parameters](#)
 - map passed to a function, [The Difference Between Maps and Slices](#)
 - pointer passed to a function, [Pointers Indicate Mutable Parameters-Pointers Indicate Mutable Parameters](#)

按值调用语言, Go 语言是按值调用的, 指针指示可变参数, 传递给函数的映射, 映射和切片的区别, 传递给函数的指针, 指针指示可变参数

- call-by-value language, [append](#)
按值调用语言, 追加
- camelCase instead of snake_case, [Naming Variables and Constants](#)
使用驼峰命名法 (camelCase) 代替蛇形命名法 (snake_case) ,
变量和常量命名
- cancellation, [Cancellation-Cancellation](#)
 - in your own code, [Context Cancellation in Your Own Code](#)

取消, 自定义代码中的取消, 自定义代码中的上下文取消

- cap function

帽子功能

- channel buffer capacity, [Reading, Writing, and Buffering](#)
通道缓冲容量、读取、写入和缓冲
- slice capacity, [Capacity](#)
 - example code, [Capacity](#)

切片容量、容量示例代码、容量

- capacity of slices, [Capacity](#)
 - never less than length, [make](#)

切片的容量，容量永远不会小于长度，制作

- capitalization determining export of package-level identifiers, [Importing and Exporting](#)
决定包级标识符导出的资本化，进出口
- case clause, [switch-switch](#)
情况条款，切换开关
- cat simplified example code, [defer](#), [defer](#)
cat 简化示例代码，defer, defer
- cgo package for C library integration, [Cgo Is for Integration, Not Performance](#)
[Cgo Is for Integration, Not Performance](#)
 - cgo.Handle, [Cgo Is for Integration, Not Performance](#)
 - cgo.NewHandle, [Cgo Is for Integration, Not Performance](#)

cgo 包用于 C 库集成, Cgo 用于集成, 而非性能 - Cgo 用于集成, 而非性能 cgo.Handle, Cgo 用于集成, 而非性能 cgo.NewHandle, Cgo 用于集成, 而非性能

- chan keyword, [Reading, Writing, and Buffering](#)

chan 关键词、阅读、写作和缓冲

- channels in concurrency

并发通道

- about, [Channels](#)
 - passed as pointer to functions, [Channels](#)

关于, 通道作为指向函数的指针传递, 通道

- backpressure in buffered channels, [Implement Backpressure](#)
缓冲通道中的背压, 实现背压
- behavior chart, [Understanding How Channels Behave](#)
行为图表, 了解渠道行为
- closing, [Closing a Channel](#)
 - selecting a closed channel, [Turn Off a case in a select](#)
 - zero value when read while closed, [Closing a Channel](#)

关闭通道, 关闭通道, 选择已关闭的通道, 关闭通道时读取值为零的情况, 关闭通道

- mutexes used instead, [When to Use Mutexes Instead of Channels](#)-
[When to Use Mutexes Instead of Channels](#)

- critical section, [When to Use Mutexes Instead of Channels](#)
- passed via pointer, [When to Use Mutexes Instead of Channels](#)

使用互斥锁代替通道，何时使用互斥锁代替通道 - 临界区，何时使用互斥锁代替通道，通过指针传递，何时使用互斥锁代替通道

- reading, writing, and buffering, [Reading, Writing, and Buffering](#)
 - read once, [Reading, Writing, and Buffering](#)
 - reading via for-range loop, [Using for-range and Channels](#)
 - zero value and comma ok idiom, [Closing a Channel](#)

读取、写入和缓冲，读取、写入和缓冲一次，读取、写入和缓冲通过 for 循环读取，使用 for 循环和通道零值和逗号的惯用法，关闭通道

- when to use buffered versus unbuffered, [Know When to Use Buffered and Unbuffered Channels](#)

何时使用缓冲通道，何时使用非缓冲通道？了解何时使用缓冲通道和非缓冲通道

- zero value as nil, [Channels](#)

零值为 nil，通道

- characters via rune type, [A Taste of Strings and Runes](#)
通过符文类型来定义角色，《弦与符文的滋味》

- checksum database, [Module Proxy Servers](#)
校验和数据库, 模块代理服务器
- Cheney, Dave, [Sentinel Errors](#), [Sentinel Errors](#)
切尼, 戴夫, 《哨兵错误》, 哨兵错误
- chi request router, [Use third-party modules to enhance the server](#)
chi 请求路由器, 使用第三方模块增强服务器
- Chocolatey
巧克力味
 - Go installation, [Installing the Go Tools](#)
Go 安装, 安装 Go 工具
 - make installation, [Makefiles](#)
进行安装, Makefiles
 - updating Go tools, [Staying Up-to-Date](#)
更新 Go 工具, 保持最新状态
- circular dependencies, [Avoiding Circular Dependencies](#)
循环依赖, 避免循环依赖
- clear function
清除功能
 - map length set to zero, [Emptying a Map](#)
映射长度设置为零, 清空映射
 - slice elements to zero value, [Emptying a Slice](#)
将切片元素的值清零, 清空切片
- Client type, [The Client-ResponseController](#)
客户端类型, 客户端响应控制器

- close function to close a channel, [Closing a Channel](#)
关闭通道的函数，关闭通道
- Close method of file via defer, [defer](#)
通过 defer 函数关闭文件
- closures, [Closures-Returning Functions from Functions](#)
 - passing functions as parameters, [Passing Functions as Parameters](#)
 - returning functions from functions, [Returning Functions from Functions](#)

闭包，闭包——从函数返回函数，将函数作为参数传递，将函数作为参数传递，从函数返回函数，从函数返回函数

- code

代码

- book code repository (see book supplemental material URL)
书籍代码库（参见书籍补充材料网址）
- documented with Go Doc comments, [Documenting Your Code with Go Doc Comments-Documenting Your Code with Go Doc Comments](#)

使用 Go Doc 注释编写文档，使用 Go Doc 注释编写代码文档
- 使用 Go Doc 注释编写代码文档

- error detection via go vet, [go vet](#)
 - (see also testing)

通过 go vet 进行错误检测，go vet（另请参阅测试）

- formatting, [go fmt](#)
格式化, 转到 fmt
- go generate, [Using go generate-Using go generate](#)
go generate, 使用 go generate-使用 go generate
- Hello World, [go build](#)
Hello World, 开始构建
- importance of error checking, [Functions Are Values](#)
错误检查的重要性, 《函数即值》
- modules, [Making a Go Module](#), [Repositories](#), [Modules](#), and [Packages](#)
 - (see also modules)

模块、创建 Go 模块、仓库、模块和包 (另请参阅模块)

- packages, [go build](#)
 - (see also packages)

packages, go build (另请参阅 packages)

- third-party code imported, [Importing Third-Party Code](#)
导入第三方代码, 导入第三方代码
- timing out in concurrency, [Time Out Code](#)
并发超时, 超时代码
- code points
代码点

- string bytes versus UTF-8 multi-byte code points, [Strings and Runes and Bytes](#)

字符串字节与 UTF-8 多字节代码点, 《字符串、符文和字节》

- Unicode using four bytes for each, [Strings and Runes and Bytes](#)
Unicode 使用四个字节来表示字符串、符文和字节。

- collaboration

合作

- code formatting for, [go fmt](#), [Unused Variables](#)
go fmt 代码格式化, 未使用的变量
- Go Playground Share button, [The Go Playground](#)
Go Playground 分享按钮, Go Playground
- unused variables, [Unused Variables](#)
未使用的变量, 未使用的变量

- comma ok idiom

逗号ok习语

- channel returning zero value, [Closing a Channel](#)
通道返回零值, 关闭通道
- handling wrong type assertion, [Type Assertions and Type Switches](#), [Type Assertions and Type Switches](#)
处理错误类型断言、类型断言和类型切换、类型断言和类型切换
- map key 0 or not in map, [The comma ok Idiom](#)
地图键 0 或不在地图中, 逗号可以省略。

- zero value versus no value indication, [The Zero Value Versus No Value](#)

零值与无值指示, 零值与无值

- comments in Go Doc format, [Documenting Your Code with Go Doc Comments](#)-[Documenting Your Code with Go Doc Comments](#)

使用 Go Doc 格式编写注释, 用 Go Doc 注释记录代码 - 使用 Go Doc 注释记录代码

- Communicating Sequential Processes (CSP), [Concurrency in Go](#)
Go语言中的通信顺序进程 (CSP) 和并发
- comparable interface, [Interfaces Are Comparable](#), [Introducing Generics in Go](#), [More on comparable](#)
 - interfaces as type constraints, [Generics and Interfaces](#)

可比较接口、接口是可比较的、在 Go 中引入泛型、更多关于可比较接口作为类型约束的内容、泛型和接口

- comparison operators

比较算子

- arrays, [Arrays—Too Rigid to Use Directly](#)
数组, 数组——过于僵化而无法直接使用
- boolean types via, [Explicit Type Conversion](#)
通过显式类型转换实现布尔类型
- comparable interface, [Interfaces Are Comparable](#), [Introducing Generics in Go](#), [More on comparable](#)

可比较接口、接口是可比较的、在 Go 中引入泛型、更多关于可比较接口的内容

- complex types, [Complex types \(you're probably not going to use these\)](#)

复杂类型, 复杂类型 (您可能不会用到这些)

- floating-point types, [Floating-point types](#)

浮点类型, 浮点类型

- integer types, [Integer operators](#)

整数类型, 整数运算符

- interfaces comparable, [Interfaces Are Comparable](#), [More on comparable](#)

接口可比性, 接口具有可比性, 更多关于可比性的信息

- maps not comparable, [Maps](#)
 - maps.Equal, [Comparing Maps](#)
 - maps.EqualFunc, [Comparing Maps](#)

地图不可比较, Mapsmaps.Equal, 比较

Mapsmaps.EqualFunc, 比较地图

- slices not comparable, [Slices](#)
 - DeepEqual in reflect package, [Slices](#), [Reflection Lets You Work with Types at Runtime](#)
 - slices.Equal, [Slices](#)
 - slices.EqualFunc, [Slices](#)

切片不可比较, reflect 包中的 SlicesDeepEqual, 切片, 反射
允许您在运行时处理类型, slices.Equal, Slices.EqualFunc,
切片

- strings, [A Taste of Strings and Runes](#)
弦乐, 弦乐与符文的品味
- structs comparable if fields comparable, [Comparing and Converting Structs](#)
 - anonymous structs, [Comparing and Converting Structs](#)

如果字段可比较, 则结构体可比较; 比较和转换结构体; 匿名结构体; 比较和转换结构体

- test results compared with go-cmp, [Using go-cmp to Compare Test Results-Using go-cmp to Compare Test Results](#)
将测试结果与 go-cmp 进行比较, 使用 go-cmp 比较测试结果
- time.Time methods, [time](#)
时间。时间方法, 时间

- compiling

编译

- before compiling

编译之前

- go fmt, [go fmt](#)
去 fmt, 去 fmt

- go vet, [go vet](#)
去看兽医，去看兽医
- cross-compiling, [Building Go Binaries for Other Platforms](#)
交叉编译，为其他平台构建 Go 二进制文件
- go build, [go build](#)
去建设，去建设
- go install, [Adding Third-Party Tools with go install-Adding Third-Party Tools with go install](#)
使用 go install 添加第三方工具 - 使用 go install 添加第三方工具
- go run to build and execute, [Using go run to Try Out Small Programs](#)
使用 `go run` 命令构建并执行程序，使用 `go run` 尝试运行小型程序。
- Makefiles, [Makefiles](#)
Makefiles, Makefiles
- semicolons added by compiler, [go fmt](#)
编译器添加了分号， go fmt
- single native binary, [Installing the Go Tools](#), [Importing Third-Party Code](#)
 - Go runtime compiled into, [Capacity](#)
 - Go tool updates and, [Staying Up-to-Date](#)
 - reading build info inside, [Reading the Build Info Inside a Go Binary](#)

单个原生二进制文件、安装 Go 工具、导入第三方代码、编译到 Go 运行时、容量、Go 工具更新和保持最新、读取内部构建信息、读取 Go 二进制文件中的构建信息

- complex types, [Complex types \(you're probably not going to use these\)](#)-[Complex types \(you're probably not going to use these\)](#)
 - arithmetic operators, [Complex types \(you're probably not going to use these\)](#)
 - complex function to declare, [Complex types \(you're probably not going to use these\)](#)
 - extracting real and imaginary portions, [Complex types \(you're probably not going to use these\)](#)
 - inexactness of, [Complex types \(you're probably not going to use these\)](#)
 - why supported, [Complex types \(you're probably not going to use these\)](#)
 - zero value as 0, [Complex types \(you're probably not going to use these\)](#)

复杂类型, 复杂类型 (您可能不会用到这些) -复杂类型 (您可能不会用到这些) 算术运算符, 复杂类型 (您可能不会用到这些) 要声明的复杂函数, 复杂类型 (您可能不会用到这些) 提取实部和虚部, 复杂类型 (您可能不会用到这些) 的不精确性, 复杂类

型（您可能不会用到这些）支持的原因，复杂类型（您可能不会用到这些）零值表示为 0，复杂类型（您可能不会用到这些）

- composite types

复合类型

- arrays, [Arrays—Too Rigid to Use Directly](#)-[Arrays—Too Rigid to Use Directly](#)

数组，数组——过于僵化而无法直接使用——数组——过于僵化而无法直接使用

- maps, [Maps-Using Maps as Sets](#)

地图，地图——将地图用作集合

- slices, [Slices-copy](#)

切片，切片复制

- strings, runes, bytes, [Strings and Runes and Bytes](#)-[Strings and Runes and Bytes](#)

字符串、符文、字节、字符串、符文和字节-字符串、符文和字节

- structs, [Structs](#)

结构体，结构体

- composition and promotion, [Use Embedding for Composition](#)

排版与推广，使用嵌入进行排版

- concrete types, [Types in Go](#)

- examples, [Types in Go](#)

- function return values, [Accept Interfaces](#), [Return Structs](#)

- interface implicit implementation, [Interfaces Are Type-Safe Duck Typing](#)
- type assertions, [Type Assertions and Type Switches](#)
 - uses for, [Use Type Assertions and Type Switches Sparingly.-
Use Type Assertions and Type Switches Sparingly.](#)

具体类型、Go 中的类型示例、Go 函数返回值中的类型、接受接口、返回结构体、接口隐式实现、接口是类型安全的鸭子类型、类型断言、类型断言和类型开关的用途、谨慎使用类型断言和类型开关

- concurrency

并发

- about, [Concurrency in Go](#)
关于 Go 中的并发
- atomic variables, [Atomics—You Probably Don't Need These](#)
原子变量，原子操作——你可能不需要这些

- channels

频道

- about, [Channels](#)
关于，频道
- backpressure in buffered channels, [Implement Backpressure](#)
缓冲通道中的背压，实现背压
- behavior chart, [Understanding How Channels Behave](#)
行为图表，了解渠道行为

- closing, [Closing a Channel](#), [Turn Off a case in a select](#)
关闭、关闭通道、在选定区域关闭案例
- mutexes used instead, [When to Use Mutexes Instead of Channels](#)-[When to Use Mutexes Instead of Channels](#)
何时使用互斥锁代替通道？
- reading via for-range loop, [Using for-range and Channels](#)
通过 for 循环读取，使用 for 循环和通道
- reading, writing, and buffering, [Reading, Writing, and Buffering](#)
阅读、写作和缓冲，阅读、写作和缓冲
- using buffered versus unbuffered, [Know When to Use Buffered and Unbuffered Channels](#)
使用缓冲通道与非缓冲通道，了解何时使用缓冲通道和非缓冲通道
- zero value as nil, [Channels](#)
零值为 nil，通道
- data race detector, [Finding Concurrency Problems with the Data Race Detector](#)
数据竞争检测器，利用数据竞争检测器查找并发问题
- goroutines, [Goroutines](#)
 - channels for communication, [Channels](#)
 - launching, [Goroutines](#), [Always Clean Up Your Goroutines](#)
 - scheduler, [Goroutines](#)

- terminating, [Use the Context to Terminate Goroutines](#), [Use the Context to Terminate Goroutines](#), [Cancellation-Cancellation](#), [Context Cancellation in Your Own Code](#)

goroutine、用于通信的 goroutine 通道、通道启动、goroutine、始终清理 goroutine 调度器、goroutine 终止、使用 Context 终止 goroutine、使用 Context 终止 goroutine、取消、在您自己的代码中取消 Context

- practices and patterns

实践和模式

- always clean up goroutines, [Always Clean Up Your Goroutines](#)
始终清理 goroutine, 始终清理你的 goroutine
- APIs concurrency-free, [Keep Your APIs Concurrency-Free](#)
保持 API 无并发性, 让您的 API 无并发性
- backpressure, [Implement Backpressure](#)
背压, 实施背压
- buffered versus unbuffered channels, [Know When to Use Buffered and Unbuffered Channels](#)
缓冲通道与非缓冲通道, 何时使用缓冲通道, 何时使用非缓冲通道
- context to terminate goroutines, [Use the Context to Terminate Goroutines](#)
使用上下文终止 goroutine

- for loop index variable, [Goroutines, for Loops, and Varying Variables](#)-[Goroutines, for Loops, and Varying Variables](#)
for 循环索引变量、Goroutine、for 循环和可变变量 -
Goroutine、for 循环和可变变量
- putting the tools together, [Put Your Concurrent Tools Together](#)-[Put Your Concurrent Tools Together](#)
将工具整合在一起，将您的并发工具整合在一起 - 将您的
的并发工具整合在一起
- running code exactly once, [Run Code Exactly Once](#)-[Run Code Exactly Once](#)
仅运行一次代码
- select case turned off, [Turn Off a case in a select](#)
选择语句中的某个情况已关闭。
- timing out code, [Time Out Code](#)
超时代码，超时代码
- waiting for goroutines to finish, [Use WaitGroups](#)-[Use WaitGroups](#)
等待 goroutine 完成，请使用 WaitGroups-Use WaitGroups
- select keyword, [select-select](#)
 - case turned off, [Turn Off a case in a select](#)
 - default clause, [select](#)
 - for loop for multiple channels, [select](#)

选择关键字，关闭 select-selectcase，在 selectdefault 子句中关

闭 case, 对多个通道使用 selectfor 循环, 选择

- sync.Map type, [When to Use Mutexes Instead of Channels](#)

同步映射类型, 何时使用互斥锁而不是通道

- when to use, [When to Use Concurrency](#)

何时使用并发

- Concurrency in Go (Cox-Buday), [When to Use Mutexes Instead of Channels](#), [Where to Learn More About Concurrency](#)

Go 语言中的并发 (Cox-Buday) , 何时使用互斥锁而不是通道, 哪里可以学习更多关于并发的知识

- const keyword, [Using const-Typed and Untyped Constants](#)
 - const block for iota set of values, [iota Is for Enumerations—Sometimes](#)
 - constants
 - generic type assignments, [Type Elements Limit Constants](#)
 - immutability of, [Using const](#)
 - io.Seeker whence constants, [io and Friends](#)
 - as sentinel errors, [Sentinel Errors](#)
 - unused constants, [Unused Variables](#)
 - naming constants, [Naming Variables and Constants](#)
 - camelCase instead of snake_case, [Naming Variables and Constants](#)
 - uppercase names not used, [Naming Variables and Constants](#)
- typed and untyped constants, [Typed and Untyped Constants](#)
 - untyped constant type conversion, [var Versus :=](#)

const 关键字, 使用 const 类型化和无类型化常量, const 块用于 iota 值集, iota 用于枚举——有时常量泛型类型赋值, 类型元素限制常量不可变性, 使用 constio.Seeker 查找常量, io 和 Friends 作为哨兵错误, 哨兵错误, 未使用的常量, 未使用的变量, 命名常量, 命名变量和常量: 驼峰式命名法而不是蛇形命名法, 命名变量和常量: 不使用大写字母, 命名变量和常量: 类型化和无类型化常量, 类型化和无类型化常量, 无类型化常量类型转换, var 与 :=

- containers with Go app image, [Installing the Go Tools](#)
 - blog post resource, [Installing the Go Tools](#)

使用 Go 应用镜像的容器, 安装 Go 工具博客文章资源, 安装 Go 工具

- content embedded into program, [Embedding Content into Your Program-Embedding Content into Your Program](#)

将内容嵌入程序中, 将内容嵌入到您的程序中 - 将内容嵌入到您的程序中

- context package

上下文包

- about metadata problem, [The Context](#)
关于元数据问题, 背景
- cancellation, [Use the Context to Terminate Goroutines, Cancellation-Cancellation](#)

- in your own code, [Context Cancellation in Your Own Code](#)

取消、使用上下文终止 Goroutine、在您自己的代码中取消、
在您自己的代码中取消上下文

- context.Background, [What Is the Context?](#)
上下文。背景，什么是上下文？
- context.CancelFunc, [Cancellation](#)
context.CancelFunc, 取消
- context.Cause, [Cancellation](#)
上下文.原因，取消
- context.TODO, [What Is the Context?](#)
context.TODO, 上下文是什么？
- context.WithCancel, [Cancellation](#)
context.WithCancel, 取消
- context.WithDeadline, [Contexts with Deadlines](#)
context.WithDeadline, 带有截止日期的上下文
- context.WithTimeout, [Contexts with Deadlines](#)
context.WithTimeout, 带有截止日期的上下文
- context.WithValue, [Values](#)
context.WithValue, 值
- contexts with deadlines, [Contexts with Deadlines-Contexts with Deadlines](#)
有截止日期的情境，有截止日期的情境-有截止日期的情境
- description of context, [What Is the Context?-What Is the Context?](#)

- first parameter passed, [What Is the Context?](#)

上下文描述, 什么是上下文? -什么是上下文? 传递的第一个参数, 什么是上下文?

- terminating goroutines, [Use the Context to Terminate Goroutines](#)
 - context cancellation, [Use the Context to Terminate Goroutines](#), [Cancellation-Cancellation](#)

终止 goroutine, 使用 Context 终止 goroutine, 上下文取消, 使用 Context 终止 goroutine, 取消-取消

- Value method, [Values](#)
 - key, [Values](#)

值方法、值键、值

- values, [Values-Values](#)

价值观, 价值观-价值观

- continue keyword, [break and continue](#)

继续关键字, 中断并继续

- control structures

控制结构

- choosing between if and switch, [Choosing Between if and switch](#)
在 if 和 switch 之间进行选择, 在 if 和 switch 之间进行选择
- for statement, [for, Four Ways-Choosing the Right for Statement](#)
 - about, [for, Four Ways](#)

- break statement, [break and continue](#)
- choosing the right statement, [Choosing the Right for Statement](#)
- complete for statement, [The Complete for Statement](#)
- condition-only for statement, [The Condition-Only for Statement](#)
- continue statement, [break and continue](#)
- for-range statement, [The for-range Statement](#)-[The for-range value is a copy](#).
- for-select loop, [select](#)
- goroutines and for loop index, [Goroutines, for Loops, and Varying Variables](#)-[Goroutines, for Loops, and Varying Variables](#)
- infinite for statement, [The Infinite for Statement](#)
- labeling, [Labeling Your for Statements](#), [switch](#)
- nested, [Labeling Your for Statements](#)
- shadowing variables, [The Complete for Statement](#)
- single-letter variable names, [Naming Variables and Constants](#)

for 语句, for, 四种方法 - 选择正确的 for 语句, 关于, for, 四种方法break 语句, break 和 continue 选择正确的语句, 选择正确的 for 语句完整的 for 语句, 完整的 for 语句仅条件 for 语句, 仅条件 for 语句continue 语句, break 和 continuefor-

range 语句, for-range 语句 - for-range 的值是副本for-select 循环, selectgoroutine 和 for 循环索引, goroutine、for 循环和可
变变量-goroutine、for 循环和可变变量无限 for 语句, 无限
for 语句标签, 为 for 语句贴标签, switch 嵌套, 为 for 语句
贴标签变量阴影, 完整的 for 语句单字母变量名, 命名变量
和常量

- goto generally not used, [goto—Yes, goto-goto—Yes, goto](#)
goto 通常不使用, goto—是, goto-goto—是, goto
- if/else if/else, [if](#)
如果/否则 如果/否则, 如果
- switch statement, [switch-switch](#)
 - blank switches, [Blank Switches](#)

switch语句, switch-switch空白开关, 空白开关

- copy function for slices, [copy](#)
切片的复制函数, 复制
- Cox, Russ, [go fmt](#), [The Go Compatibility Promise](#), [Introducing Generics in Go](#), [Organizing Your Module](#)
Cox、Russ、go fmt、《Go 兼容性承诺》、《在 Go 中引入泛型》、《组织你的模块》
- Cox-Buday, Katherine, [When to Use Mutexes Instead of Channels](#), [Where to Learn More About Concurrency](#)
Cox-Buday, Katherine, 《何时使用互斥锁而非通道》, 以及“哪里
可以了解更多关于并发的知识”

- cross-compiling, [Building Go Binaries for Other Platforms](#)
交叉编译, 为其他平台构建 Go 二进制文件
- csv package
csv包
 - csv.NewReader, [Use Reflection to Write a Data Marshaler](#)
csv.NewReader, 使用反射编写数据编组器
 - csv.NewWriter, [Use Reflection to Write a Data Marshaler](#)
csv.NewWriter, 使用反射编写数据编组器
- Ctrl-C to stop execution, [The Infinite for Statement](#)
按 Ctrl-C 停止执行, 无限循环语句

D

D

- dash (-) for ignored field in marshaling or unmarshaling, [Using Struct Tags to Add Metadata](#)
使用短横线 (-) 表示序列化或反序列化过程中忽略的字段, 使用结构标签添加元数据
- data embedded into program, [Embedding Content into Your Program-Embedding Content into Your Program](#)
将数据嵌入程序中, 将内容嵌入到您的程序中 - 将内容嵌入到您的程序中
- data race detector, [Finding Concurrency Problems with the Data Race Detector](#)

- documentation URL, [Finding Concurrency Problems with the Data Race Detector](#)

数据竞争检测器，使用数据竞争检测器查找并发问题文档 URL，
使用数据竞争检测器查找并发问题

- database/sql/driver package

数据库/SQL/驱动程序包

- database support, [defer](#)
数据库支持，延迟

- evolution of, [Accept Interfaces](#), [Return Structs](#), [Use Type Assertions and Type Switches Sparingly](#)

演进过程：接受接口、返回结构体、谨慎使用类型断言和类型切换

- interfaces defined for database driver, [Accept Interfaces](#), [Return Structs](#)

为数据库驱动程序定义的接口、接受接口、返回结构

- deadlock, [select](#)

死锁，选择

- Dean, Jeff, [Reducing the Garbage Collector's Workload](#)

迪恩、杰夫，《减轻垃圾收集员的工作量》

- decimal module (ShopSpring), [Importing Third-Party Code](#)

十进制模块（ShopSpring），导入第三方代码

- declaration list for variables, [var Versus :=](#)

变量声明列表，var Versus :=

- decorator pattern with standard interfaces, [Interfaces Are Type-Safe Duck Typing](#)

装饰器模式与标准接口，接口是类型安全的鸭子类型

- DeepEqual in reflect package, [Slices](#), [Reflection Lets You Work with Types at Runtime](#)

reflect 包中的 DeepEqual、切片和反射功能允许您在运行时处理类型。

- default keyword, [switch-switch](#)

默认关键字，switch-switch

- default type, [Literals](#)

默认类型，字面量

- defer keyword, [defer-defer](#)

- actions taken on returned error, [defer](#)
- function, method, or closure with, [defer](#)
 - multiple functions, [defer](#)
- io.Closer Close function called via, [io and Friends](#)
 - loops and, [io and Friends](#)
- Mutex Unlock called after Lock or RLock, [When to Use Mutexes Instead of Channels](#)
- named return values, [defer](#)
- returned values cannot be read, [defer](#)
- wrapping errors with, [Wrapping Errors with defer](#)

defer 关键字，defer-对返回的错误进行延迟操作，defer 函数、方

法或闭包, defer 多个函数, deferio.Closer 通过 io 和 Friendsloops 调用 Close 函数, io 和 FriendsMutex 在 Lock 或 RLock 之后调用 Unlock, 何时使用互斥锁而不是通道, defer 返回的值无法读取, defer 包装错误, 使用 defer 包装错误

- delete function for map key-value, [Deleting from Maps](#)

删除映射键值对的功能, 从映射中删除数据

- Delve debugger, [Visual Studio Code](#)

Delve调试器, Visual Studio Code

- dependencies

依赖项

- circular dependencies, [Avoiding Circular Dependencies](#)

循环依赖, 避免循环依赖

- module versions

模块版本

- dependency copies inside module, [Vendoring](#)

模块内部的依赖副本, 供应商

- resolving different module version dependencies, [Minimal](#)

[Version Selection](#)

解决不同的模块版本依赖关系, 最小版本选择

- updating to compatible versions, [Updating to Compatible](#)

[Versions](#)

更新至兼容版本

- updating to incompatible versions, [Updating to Incompatible](#)

[Versions](#), [Versioning Your Module](#)

更新到不兼容的版本、更新到不兼容的版本、模块版本控制

- vendoring, [Vendoring](#)

供应商，供应商

- overriding, [Overriding Dependencies](#)

覆盖，覆盖依赖项

- require section of go.mod, [The require Directive](#), [Importing Third-Party Code](#)

go.mod 文件中的 require 部分，require 指令，导入第三方代码

- dependency injection via implicit interfaces, [Implicit Interfaces Make Dependency Injection Easier](#)-[Implicit Interfaces Make Dependency Injection Easier](#)

- Wire helper instead, [Wire](#)

通过隐式接口进行依赖注入，隐式接口使依赖注入更容易——隐式接口使依赖注入更容易，使用 Wire 辅助函数代替 Wire

- dereferencing, [A Quick Pointer Primer](#)
 - non-nil pointers only, [A Quick Pointer Primer](#)

解引用：指针快速入门（仅限非空指针）：指针快速入门

- Design Patterns (Gamma, Helm, Johnson, and Vlissides), [Use Embedding for Composition](#), [Interfaces Are Type-Safe Duck Typing](#)

设计模式 (Gamma、Helm、Johnson 和 Vlissides) 、使用嵌入进行组合、接口是类型安全的鸭子类型

- development environment

开发环境

- go commands, [Go Tooling](#)
 - go build, [go build](#)
 - go fmt, [go fmt](#)
 - go mod, [Making a Go Module](#)
 - go version, [Installing the Go Tools](#)

Go 命令、Go 工具、go build、go fmt、go mod、创建 Go 模块、go version、安装 Go 工具

- installing Go tools, [Installing the Go Tools](#)
 - go command, [Go Tooling](#)
 - troubleshooting installation, [Troubleshooting Your Go Installation](#)
 - updating Go tools, [Staying Up-to-Date](#)
 - validating with go version, [Installing the Go Tools](#)

安装 Go 工具、安装 Go 工具、go 命令、Go 工具故障排除、Go 安装故障排除、更新 Go 工具、保持最新、使用 Go 版本验证、安装 Go 工具

- Makefiles, [Makefiles](#)
Makefiles, Makefiles

- project organization, [Go Tooling](#)

项目组织, Go Tooling

- tools

工具

- about, [Choose Your Tools](#)

关于, 选择你的工具

- Go development tools URL, [Choose Your Tools](#)

Go 开发工具 URL, 选择您的工具

- The Go Playground, [The Go Playground](#)-[The Go Playground](#)

Go Playground, Go Playground-Go Playground

- GoLand, [GoLand](#)

GoLand, GoLand

- Visual Studio Code, [Visual Studio Code](#)

Visual Studio Code, Visual Studio Code

- updating, [Staying Up-to-Date](#)

更新, 保持最新状态

- Dijkstra, Edsger, [goto—Yes, goto](#)

Dijkstra、Edsger、goto——是的, goto

- dividing by zero

除以零

- floating-point types, [Floating-point types](#)

浮点类型, 浮点类型

- integer types, [Integer operators](#)

整数类型, 整数运算符

- do loop via infinite for loop, [break and continue](#)
通过无限循环执行循环，然后跳出循环语句并继续执行。
- Docker containers with Go app image, [Installing the Go Tools](#)
 - blog post resource, [Installing the Go Tools](#)

使用 Go 应用镜像的 Docker 容器，安装 Go 工具博客文章资源，
安装 Go 工具

- Dogan, Jaana, [Adding Third-Party Tools with go install](#)
Dogan, Jaana, 《使用 go install 添加第三方工具》
- double quote (")

双引号 (")

- not interchangeable with single quote, [Literals](#)
不可与单引号、字面量互换
- string literals, [Literals](#)
字符串字面量，字面量
- duck typing, [Interfaces Are Type-Safe Duck Typing](#)
鸭子类型，接口是类型安全的鸭子类型

E

E

- Echo web framework, [Use third-party modules to enhance the server](#)

Echo Web框架，使用第三方模块增强服务器功能

- Effective Go

有效

- idiomatic Go resource, [go vet](#)

惯用语：去资源，去兽医

- semicolon insertion rule, [go fmt](#)

分号插入规则，go fmt

- else keyword, [if](#)

- variables scoped to if and else blocks, [if](#)

else 关键字，if 变量的作用域限定在 if 和 else 代码块中，如果

- embed package, [Embedding Content into Your Program](#)

嵌入包，将内容嵌入到您的程序中

- embedding an interface in an interface, [Embedding and Interfaces](#)

将接口嵌入到接口中，嵌入和接口

- embedding content into program, [Embedding Content into Your Program-Embedding Content into Your Program](#)

- hidden files, [Embedding Hidden Files](#)

- virtual filesystem, [Embedding Content into Your Program](#)

将内容嵌入程序、将内容嵌入程序、将内容嵌入程序、隐藏文件、嵌入隐藏文件、虚拟文件系统、将内容嵌入程序

- embedding for composition, [Use Embedding for Composition](#)
 - embedded field, [Use Embedding for Composition](#)
 - embedding is not inheritance, [Embedding Is Not Inheritance](#)

组合嵌入，使用组合嵌入嵌入字段，使用组合嵌入嵌入不是继承，嵌入不是继承

- encoding/json package, [encoding/json-Custom JSON Parsing](#)
 - custom JSON parsing, [Custom JSON Parsing-Custom JSON Parsing](#)
 - JSON
 - about, [encoding/json](#)
 - marshaling versus unmarshaling, [encoding/json](#)
 - struct tags to add metadata, [Using Struct Tags to Add Metadata](#)
 - zero value versus no value via pointers, [The Zero Value Versus No Value](#)
- json.Decoder, [JSON, Readers, and Writers-JSON, Readers, and Writers](#)
- json.Encoder, [JSON, Readers, and Writers-JSON, Readers, and Writers](#)

- multiple JSON structs at once, [Encoding and Decoding JSON Streams](#)
- json.Marshal, [Unmarshaling and Marshaling](#)
- json.Unmarshal, [Unmarshaling and Marshaling](#)
 - about unmarshaling, [encoding/json](#)
 - dash for name of ignored field, [Using Struct Tags to Add Metadata](#)
- populating variable from, [Pointers Are a Last Resort](#)

encoding/json 包, encoding/json - 自定义 JSON 解析, 自定义 JSON 解析, JSONabout, encoding/json 序列化与反序列化, encoding/jsonstruct 标签用于添加元数据, 使用结构标签添加元数据, 通过指针获取零值与无值, 零值与无值, json.Decoder, JSON、读取器和写入器 - JSON、读取器和写入器, json.Encoder, JSON、读取器和写入器 - JSON、读取器和写入器, 一次处理多个 JSON 结构体, 编码和解码 JSON 流, json.Marshal, 反序列化和序列化, json.Unmarshal, 反序列化和序列化, 关于反序列化, encoding/jsondash 用于忽略字段的名称, 使用结构标签添加元数据, 从填充变量, 指针是最后的手段

- enumerations via iota, [iota Is for Enumerations—Sometimes](#)
 - printable via stringer, [Using go generate](#)

通过 iota 进行枚举, iota 是用于枚举的——有时可通过 stringer 打印, 使用 go generate

- environment variables

环境变量

- go help environment to list, [Adding Third-Party Tools with go install](#)

使用 go help 环境列表，添加第三方工具（使用 go install）

- GOARCH, [Building Go Binaries for Other Platforms](#)

GOARCH, 为其他平台构建 Go 二进制文件

- GOBIN, [Adding Third-Party Tools with go install](#)

GOBIN, 使用 go install 添加第三方工具

- GOGC, [Tuning the Garbage Collector-Tuning the Garbage Collector](#)

GOGC, 调优垃圾回收器——调优垃圾回收器

- GOMEMLIMIT, [Tuning the Garbage Collector](#), [Contexts with Deadlines](#)

GOMEMLIMIT、调整垃圾回收器、带截止时间的上下文

- GOOS, [Building Go Binaries for Other Platforms](#)

GOOS, 为其他平台构建 Go 二进制文件

- GOPATH, [Adding Third-Party Tools with go install](#)

GOPATH, 使用 go install 添加第三方工具

- GOPRIVATE, [Using Private Repositories](#)

GOPRIVATE, 使用私有仓库

- GOPROXY, [Specifying a Proxy Server](#), [Adding Third-Party Tools with go install](#)

GOPROXY, 指定代理服务器, 使用 go install 添加第三方工具

- GOROOT, [Adding Third-Party Tools with go install](#)

GOROOT, 使用 go install 添加第三方工具

- GOTOOLCHAIN, [Use the go Directive to Manage Go Build Versions](#)

GOTOOLCHAIN, 使用 go 指令管理 Go 构建版本

- GOWORK, [Using Workspaces to Modify Modules Simultaneously](#)

GOWORK, 使用工作区同时修改模块

- testing using, [Testing with Environment Variables](#)
使用环境变量进行测试

- equal sign (=)

等号 (=)

- := versus var, [var Versus :=-var Versus :=](#)
:= 与 var、var 与 :=-var 与 :=

- arithmetic operators, [Integer operators](#)
算术运算符, 整数运算符

- comparison operators

比较算子

- arrays, [Arrays—Too Rigid to Use Directly](#)
数组, 数组——过于僵化而无法直接使用
- comparable interface, [Introducing Generics in Go, More on comparable](#)

可比较的接口，Go 语言中的泛型引入，更多关于可比较的内容

- complex types, [Complex types \(you're probably not going to use these\)](#)

复杂类型，复杂类型（您可能不会用到这些）

- floating-point types, [Floating-point types](#)

浮点类型，浮点类型

- integer types, [Integer operators](#)

整数类型，整数运算符

- interfaces comparable, [Interfaces Are Comparable](#)

接口可比性，接口是可比的

- maps not comparable, [Maps](#)

地图不具可比性，地图

- slices not comparable, [Slices](#)

切片不具有可比性，切片

- string types, [A Taste of Strings and Runes](#)

字符串类型，字符串和符文入门

- structs comparable if fields comparable, [Comparing and Converting Structs](#)

如果字段可比较，则结构体可比较；比较和转换结构体

- time.Time methods, [time](#), [time](#)

时间。时间方法，时间，时间

- errgroup package Group method for WaitGroup errors, [Use WaitGroups](#)

- documentation URL, [Use WaitGroups](#)

errgroup 包中的 Group 方法用于处理 WaitGroup 错误，请参阅 WaitGroups 文档 URL，了解如何使用 WaitGroups。

- error interface type, [Accept Interfaces](#), [Return Structs](#), [How to Handle Errors: The Basics](#)
 - wrapped errors, [Use Type Assertions and Type Switches Sparingly](#)

错误接口类型、接受接口、返回结构体、如何处理错误：基础知识、包装错误、谨慎使用类型断言和类型切换

- errors

错误

- actions taken on returned error via defer, [defer](#)
通过 defer 对返回的错误采取的操作，defer
- compile-time errors

编译时错误

- anonymous function name, [Anonymous Functions](#)
匿名函数名，匿名函数
- append return value not assigned, [append](#)
追加未赋值的返回值，追加
- declared variable not used, [Unused Variables](#)
已声明但未使用的变量，未使用的变量

- defer function parentheses, [defer](#)
延迟函数括号, 延迟
- importing a package but not using exported identifiers, [Creating and Accessing a Package](#)
导入包但不使用导出的标识符, 创建和访问包
- importing a package without go.mod references, [Importing Third-Party Code](#)
导入未引用 go.mod 的包, 导入第三方代码
- impossible type parameter interface instantiated, [Use Type Terms to Specify Operators](#)
无法实例化类型参数接口, 请使用类型术语来指定运算符。
- multiple return values assigned to one variable, [Multiple Return Values Are Multiple Values](#)
为同一个变量分配多个返回值, 多个返回值即多个值
- parentheses around returned values, [Multiple Return Values](#)
返回值周围加括号, 多个返回值
- slice capacity less than length, [make](#)
切片容量小于长度, 制作
- struct pointer field assigned a literal, [A Quick Pointer Primer](#)
结构体指针字段被赋予字面值, 《指针快速入门》
- type elements as other than type constraint, [Use Type Terms to Specify Operators](#)

类型元素可以用作类型约束以外的其他类型元素，使用类型术语来指定运算符。

- variadic input parameter dot placement, [Variadic Input Parameters and Slices](#)

可变参数输入参数点位置、可变参数输入参数和切片

- workspace not published, [Using Workspaces to Modify Modules Simultaneously](#)

工作区未发布，使用工作区同时修改模块

- detecting in code via go vet, [go vet](#)
 - (see also testing)

通过 go vet 在代码中检测，go vet（另请参阅测试）

- documentation URL, [Getting a Stack Trace from an Error](#)
文档 URL，如何从错误中获取堆栈跟踪
- error handling, [How to Handle Errors: The Basics](#)-[How to Handle Errors: The Basics](#)
 - error messages not capitalized or punctuated, [How to Handle Errors: The Basics](#)
 - multiple return values, [How to Handle Errors: The Basics](#)-[How to Handle Errors: The Basics](#)
 - sentinel errors, [Sentinel Errors](#)-[Sentinel Errors](#)
 - strings for simple errors, [Use Strings for Simple Errors](#)
 - values for errors, [Errors Are Values](#)-[Errors Are Values](#)

- wrong type assertion, [Type Assertions and Type Switches](#),
[Type Assertions and Type Switches](#)

错误处理，如何处理错误：基础知识 - 如何处理错误：基础知识；错误消息未大写或未加标点符号，如何处理错误：基础知识；多个返回值，如何处理错误：基础知识 - 如何处理错误：基础知识；哨兵错误，哨兵错误 - 哨兵错误；简单错误的字符串，使用字符串表示简单错误；错误的值，错误即值 - 错误即值；类型断言错误，类型断言和类型切换，类型断言和类型切换

- error interface type, [Accept Interfaces](#), [Return Structs](#), [How to Handle Errors: The Basics](#)
 - wrapped errors, [Use Type Assertions and Type Switches Sparingly](#)

错误接口类型、接受接口、返回结构体、如何处理错误：基础知识、包装错误、谨慎使用类型断言和类型切换

- Go as call-by-value language, [append](#)
Go 作为按值调用的语言，追加
- importance of error checking, [Functions Are Values](#)
错误检查的重要性，《函数即值》
- io.Reader
io.Reader

- io.EOF, [io and Friends](#)
io.EOF、io 和朋友们
- io.ErrUnexpectedEOF, [io and Friends](#)
io.ErrUnexpectedEOF、io 和 Friends
- panic and recover, [panic and recover](#)-[panic and recover](#)
 - (see also panics)

恐慌与恢复，恐慌与恢复——恐慌与恢复（另见恐慌症）

- returning errors via return value, [Multiple Return Values](#)
 - actions taken via defer, [defer](#)
 - last return value, [How to Handle Errors: The Basics](#)
 - nil returned if no error, [Multiple Return Values](#), [How to Handle Errors: The Basics](#)
 - other return values to zero values on error, [How to Handle Errors: The Basics](#)
 - why not throwing exceptions, [How to Handle Errors: The Basics](#)

通过返回值返回错误、多个返回值、通过 defer 执行的操作、defer 的最后一个返回值、如何处理错误：基础知识、如果没有错误则返回 nil、多个返回值、如何处理错误：基础知识、发生错误时其他返回值变为零、如何处理错误：基础知识、为什么不抛出异常、如何处理错误：基础知识

- stack trace from, [Getting a Stack Trace from an Error](#)
从错误中获取堆栈跟踪
- wrapping errors, [Use Type Assertions and Type Switches Sparingly](#), [Wrapping Errors](#)-[Wrapping Errors](#)
 - defer, [Wrapping Errors with defer](#)
 - sentinel errors not checkable by ==, [Is and As](#)
 - wrapping multiple errors, [Wrapping Multiple Errors](#)

错误包装，谨慎使用类型断言和类型切换，错误包装 - 使用 defer 包装错误，使用 defers 包装错误，哨兵错误无法通过 == 检查，is 和 as 包装多个错误，包装多个错误

- errors package

错误包

- documentation URL, [Getting a Stack Trace from an Error](#)
文档 URL，如何从错误中获取堆栈跟踪
- errors.As, [Use Type Assertions and Type Switches Sparingly](#), [Is and As](#)
错误。请谨慎使用类型断言和类型切换，Is 和 As
- errors.Is, [Use Type Assertions and Type Switches Sparingly](#), [Is and As](#)
错误。是，谨慎使用类型断言和类型切换，是和作为
- errors.New, [How to Handle Errors: The Basics](#), [Use Strings for Simple Errors](#)
错误处理方法：基础知识，使用字符串表示简单错误

- escape analysis, [Reducing the Garbage Collector's Workload](#)
 - URLs to more information, [Reducing the Garbage Collector's Workload](#)

逃逸分析，降低垃圾回收器的工作负载（更多信息请访问 URL），降低垃圾回收器的工作负载

- exclude directive, [Overriding Dependencies](#)
 - retract directive versus, [Retracting a Version of Your Module](#)

排除指令、覆盖依赖项、撤回指令与撤回模块版本

- exporting package-level identifiers, [Importing and Exporting](#)
 - aliases, [Gracefully Renaming and Reorganizing Your API](#)

导出包级标识符、导入和导出别名、优雅地重命名和重组 API

F

F

- fallthrough keyword avoided, [switch](#)
避免了 fallthrough 关键字，切换
- false, [Booleans](#)
 - booleans only, [Explicit Type Conversion](#)
 - zero value for bool type, [Booleans](#)

false, 布尔值 (仅限布尔值), 显式类型转换, 布尔类型的值为零, 布尔值

- file block, [Blocks](#)
文件块, 块
- first program, [Your First Go Program-go vet](#)
 - building executable, [go build](#)
 - go fmt before, [go fmt](#)
 - code, [go build](#)
 - formatting, [go fmt](#)
 - packages, [go build](#)
 - creating go module, [Making a Go Module](#)
 - nonunique name, [Repositories, Modules, and Packages](#)

第一个程序, 你的第一个 Go 程序 - 构建可执行文件, go buildgo
fmt 之前, go fmtpackages, go build 格式化, go fmtpackages, go build
创建 Go 模块, 使 Go 模块名称不唯一, 仓库、模块和包

- floating-point literals, [Literals](#)
 - 0x prefix for hexadecimal, [Literals](#)
 - default type of float64, [Floating-point types](#)
 - p indicating exponent, [Literals](#)
 - underscores for readability, [Literals](#)

浮点字面量, Literals0x 前缀表示十六进制, Literalsdefault 类型为

float64, 浮点类型 p 表示指数, Literalsunderscores 表示可读性, Literals

- floating-point types, [Floating-point types](#)
 - arithmetic operators, [Floating-point types](#)
 - dividing by zero, [Floating-point types](#)
 - division, [Floating-point types](#)
 - decimal module by ShopSpring, [Importing Third-Party Code](#)
 - The Floating Point Guide, [Floating-point types](#)
 - comparing floats, [Floating-point types](#)
 - inexactness of, [Floating-point types](#)
 - comparing floats, [Floating-point types](#)
 - using float64 simplest, [Floating-point types](#)
 - zero value as 0, [Floating-point types](#)

浮点类型、浮点类型算术运算符、浮点类型除以零、浮点类型除法、ShopSpring 的浮点类型 decimal 模块、导入第三方代码、浮点类型指南、浮点类型比较浮点数、浮点类型不精确性、浮点类型比较浮点数、使用 float64 最简单的浮点类型、浮点类型零值设为 0、浮点类型

- fmt package, [go build](#)
 - fmt.Errorf, [Use Strings for Simple Errors](#)
 - fmt.Printf, [go vet](#)
 - stack trace via verbose output, [Getting a Stack Trace from an Error](#)

- `fmt.Println`, [go build](#)
 - passing error to, [Use Strings for Simple Errors](#)
- `fmt.Stringer`, [time](#)

`fmt` 包, `go build`
`fmt.Errorf`, 使用字符串表示简单错误
`fmt.Printf`,
`go vet`通过详细输出获取堆栈跟踪, 从错误中获取堆栈跟踪
`fmt.Println`, `go build`传递错误, 使用字符串表示简单错误
`fmt.Stringer`, 时间

- for statement, [for, Four Ways-Choosing the Right for Statement](#)
 - about, [for, Four Ways](#)
 - break statement, [break and continue](#)
 - choosing the right statement, [Choosing the Right for Statement](#)
 - complete for statement, [The Complete for Statement](#)
 - condition-only for statement, [The Condition-Only for Statement](#)
 - continue statement, [break and continue](#)
 - for-range statement, [The for-range Statement-The for-range value is a copy](#)
 - channel read via, [Using for-range and Channels](#)
 - iterating over maps, [Iterating over maps](#)
 - iterating over strings, [Iterating over strings](#)
 - underscore (`_`) for unused variable name, [The for-range Statement](#)
 - value variable as a copy, [The for-range value is a copy](#)
 - value variable copy control, [The for-range value is a copy](#)

- for-select loop, [select](#)
- goroutines and for loop index, [Goroutines, for Loops, and Varying Variables-Goroutines, for Loops, and Varying Variables](#)
- infinite for statement, [The Infinite for Statement](#)
- labeling, [Labeling Your for Statements](#), [switch](#)
- nested, [Labeling Your for Statements](#)
- shadowing variables, [The Complete for Statement](#)
- single-letter variable names, [Naming Variables and Constants](#)

for 语句, for, 四种方法 - 选择正确的 for 语句, 关于, for, 四种方法
 break 语句, break 和 continue 选择正确的语句, 选择正确的
 for 语句完整的 for 语句, 完整的 for 语句仅条件 for 语句, 仅条件
 for 语句continue 语句, break 和 continuefor-range 语句, for-range
 语句 - for-range 的值是副本通过通道读取, 使用 for-range 和通道
 遍历映射, 遍历映射遍历字符串, 遍历字符串下划线 () 用于未
 使用的变量名, for-range 语句值变量作为副本, for-range 的值是
 副本值变量复制控制, for-range 的值是副本for-select 循环,
 selectgoroutine 和 for 循环索引, goroutine、for 循环和可变变量-
 goroutine、for 循环和可变变量无限 for 语句, 无限 for 语句标
 签, 为 for 语句贴标签, switch 嵌套, 为 for 语句贴标签阴影变
 量, for 语句完整指南, 单字母变量名, 变量和常量命名

- forks and overriding dependencies, [Overriding Dependencies](#)
 分支和覆盖依赖项, 覆盖依赖项
- formatting of code, [go fmt](#)

- go fmt, [go fmt](#)

代码格式化, go fmtgo fmt, go fmt

- Fossil for version control, [Publishing Your Module](#)

Fossil 版本控制, 发布您的模块

- Fowler, Martin, [Using Stubs in Go](#)

福勒, 马丁, 《在 Go 中使用存根》

- func keyword

函数关键字

- function declarations, [Declaring and Calling Functions](#)

函数声明, 声明和调用函数

- method declarations, [Methods](#)

方法声明, 方法

- new function within a function, [Anonymous Functions](#)

函数中的新函数, 匿名函数

- functions

函数

- accept interfaces, return structs, [Accept Interfaces, Return Structs](#)-

[Accept Interfaces, Return Structs](#)

接受接口, 返回结构体, 接受接口, 返回结构体-接受接口, 返回结构体

- anonymous functions, [Anonymous Functions](#)

- as closures, [Closures-Returning Functions from Functions](#)

匿名函数、匿名函数作为闭包、闭包——从函数返回函数

- automating repetitive tasks via reflection, [Build Functions with Reflection to Automate Repetitive Tasks](#)

利用反射实现重复性任务自动化，使用反射构造函数以实现重复性任务自动化

- call by value language, [Go Is Call by Value-Go Is Call by Value](#),
[Pointers Indicate Mutable Parameters](#)
 - channel passed to a function, [Channels](#)
 - map passed to a function, [The Difference Between Maps and Slices](#)
 - pointer passed to a function, [Pointers Indicate Mutable Parameters-Pointers Indicate Mutable Parameters](#)

按值调用语言，Go 语言是按值调用的，指针指示可变参数，传递给函数的通道，传递给函数的映射，映射和切片的区别，传递给函数的指针，指针指示可变参数

- call-by-value language, [append](#)
按值调用语言，追加
- calling, [Declaring and Calling Functions](#)
 - anonymous functions, [Anonymous Functions](#)
 - consecutive input parameters of same type, [Declaring and Calling Functions](#)

- emulating named and optional parameters, [Simulating Named and Optional Parameters](#)
- input parameters must be supplied, [Simulating Named and Optional Parameters](#)
- no input parameters, [Declaring and Calling Functions](#)
- stack, [Reducing the Garbage Collector's Workload](#)
- variadic input parameters, [Variadic Input Parameters and Slices](#)

调用、声明和调用函数；匿名函数；相同类型的连续输入参数；声明和调用函数；模拟命名参数和可选参数；必须提供输入参数；模拟命名参数和可选参数；无输入参数；声明和调用函数；堆栈；减少垃圾回收器的工作负载；可变输入参数；可变输入参数和切片

- closures, [Closures-Returning Functions from Functions](#)
 - passing functions as parameters, [Passing Functions as Parameters](#)
 - returning functions from functions, [Returning Functions from Functions](#)

闭包，闭包——从函数返回函数，将函数作为参数传递，将函数作为参数传递，从函数返回函数，从函数返回函数

- declaring, [go build](#), [Declaring and Calling Functions](#)
 - methods versus, [Methods](#)

- new function within a function, [Anonymous Functions](#)

声明、go build、声明和调用函数、方法与方法、函数内的新函数、匿名函数

- defer, [defer-defer](#)
推迟, 再推迟
- functions are values, [Functions Are Values-Functions Are Values](#)
 - calculator example code, [Functions Are Values](#)
 - signature of function, [Functions Are Values](#)
 - zero value of function variable as nil, [Functions Are Values](#)

函数即值, 函数即值 - 函数即值计算器示例代码, 函数即值 - 函数签名, 函数即值 - 函数变量的零值设为 nil, 函数即值

- generic functions (see generic functions)
通用函数 (参见通用函数)
- goroutines in concurrency, [Goroutines](#)
 - channels for communication, [Channels](#)
 - launching, [Goroutines](#)
 - scheduler, [Goroutines](#)
 - terminating, [Use the Context to Terminate Goroutines](#), [Use the Context to Terminate Goroutines](#), [Cancellation-Cancellation](#), [Context Cancellation in Your Own Code](#)

并发中的 goroutine、用于通信的 goroutine 通道、通道启动、

goroutine 调度器、goroutine 终止、使用 Context 终止
goroutine、使用 Context 终止 goroutine、取消、在您自己的
代码中取消 Context

- helper functions (see helper functions)
辅助函数（参见辅助函数）
- “higher-order functions” definition, [Returning Functions from Functions](#)
“高阶函数”定义，从函数返回函数
- input parameters not modified, [append](#), [Go Is Call by Value-Go Is Call by Value](#), [Pointers Indicate Mutable Parameters](#)
 - pointer passed to a function, [Pointers Indicate Mutable Parameters-Pointers Indicate Mutable Parameters](#)

输入参数未修改，追加，Go 是按值调用 - Go 是按值调用，
指针指示可变参数，传递给函数的指针，指针指示可变参数
- 指针指示可变参数

- interface implementation, [Function Types Are a Bridge to Interfaces](#)
接口实现，函数类型是连接接口的桥梁
- methods as replacements for, [Methods Are Functions Too](#)
 - when to use methods versus functions, [Functions Versus Methods](#)

方法作为函数的替代品，方法也是函数，何时使用方法而不

是函数，函数与方法的区别

- no overloading, [Preface](#), [Methods](#)
无过载，前言，方法
- passing functions as parameters, [Passing Functions as Parameters](#)
 - when to use interface instead, [Function Types Are a Bridge to Interfaces](#)

将函数作为参数传递，何时应该使用接口，函数类型是连接接口的桥梁

- programs starting with main function, [go build](#)
以 main 函数开头的程序，go build
- returning values, [Declaring and Calling Functions](#)
 - blank returns never should be used, [Blank Returns—Never Use These!](#)
 - concrete types returned, [Accept Interfaces](#), [Return Structs](#)
 - error handling, [How to Handle Errors: The Basics](#)-[How to Handle Errors: The Basics](#)
 - errors, [Multiple Return Values](#)
 - errors as last return values, [How to Handle Errors: The Basics](#)
 - goroutine return values ignored, [Goroutines](#)
 - ignoring returned values, [The for-range Statement](#), [Ignoring Returned Values](#)

- maps for return values, [The Difference Between Maps and Slices](#)
- multiple return values, [Multiple Return Values](#)
- multiple return values via multiple variables, [Multiple Return Values Are Multiple Values](#)
- named return values, [Named Return Values](#)
- named return values used by defer, [defer](#)
- no return value, [Declaring and Calling Functions](#)
- parentheses around returned values, [Multiple Return Values](#)
- return keyword, [Declaring and Calling Functions](#)
- returning functions from functions, [Returning Functions from Functions](#)
- reused err variable, [How to Handle Errors: The Basics](#)
- stack for, [Reducing the Garbage Collector's Workload](#)
- value types favored over pointers, [Pointers Are a Last Resort](#)

返回值、声明和调用函数、永远不应该使用空返回值、空返回值——永远不要使用它们！、返回具体类型、接受接口、返回结构体、错误处理、如何处理错误：基础知识、错误、多个返回值、错误作为最后一个返回值、如何处理错误：基础知识、忽略 goroutine 返回值、goroutine、指定返回值、for-range 语句、忽略返回值、使用 map 作为返回值、map 和 slice 的区别、多个返回值、多个返回值、通过多个变量实现多个返回值、多个返回值就是多个值、命名返回值、命名返

返回值、defer 使用的命名返回值、defer、无返回值、声明和调用函数、返回值周围的括号、多个返回值、return 关键字、声明和调用函数、从函数返回函数、从函数返回函数、重用 err 变量、如何处理错误：基础知识栈：减少垃圾回收器的工作负载；值类型优于指针；指针是最后的手段

- triggering to run after time duration, [Timers and Timeouts](#)
触发运行时间、定时器和超时
- type declarations, [Function Type Declarations](#)
类型声明，函数类型声明
- fuzzing, [Fuzzing-Fuzzing](#)
模糊处理，模糊处理

G

G

- Gamma, Erich, [Use Embedding for Composition](#), [Interfaces Are Type-Safe Duck Typing](#)
Gamma、Erich，《使用嵌入进行组合，接口是类型安全的鸭子类型》
- Gang of Four book, [Use Embedding for Composition](#)
《四人帮》一书，《使用嵌入进行组合创作》
- garbage collector
垃圾收集员
 - algorithm, [Reducing the Garbage Collector's Workload](#)

- implementation details talk URL, [Reducing the Garbage Collector's Workload](#)

算法，减少垃圾回收器的工作量实现细节讨论 URL，减少垃圾回收器的工作量

- Go runtime, [Capacity](#)
Go运行时，容量
- “A Guide to the Go Garbage Collector” URL, [Tuning the Garbage Collector](#)
“Go 垃圾回收器指南”网址，调整垃圾回收器
- heap, [Reducing the Garbage Collector's Workload](#)
堆，减少垃圾收集器的工作量
- pointer memory management, [A Quick Pointer Primer](#)
指针内存管理，指针快速入门
- reducing workload, [Reducing the Garbage Collector's Workload-Reducing the Garbage Collector's Workload](#)
 - buffers, [Slices as Buffers](#), [Reducing the Garbage Collector's Workload](#)
 - interface trade-off, [Accept Interfaces](#), [Return Structs](#)
 - Java versus Go, [Reducing the Garbage Collector's Workload](#)

减少工作负载，减少垃圾回收器的工作负载——减少垃圾回收器的工作负载，缓冲区，切片作为缓冲区，减少垃圾回收

器的工作负载，接口权衡，接受接口，返回结构体，Java 与 Go，减少垃圾回收器的工作负载

- slices growing, [Capacity](#)
切片增长，容量
- stack, [Reducing the Garbage Collector's Workload](#)
堆栈，减少垃圾回收器的工作负载
- thrashing, [Tuning the Garbage Collector](#)
猛烈抨击，调整垃圾收集器
- tuning, [Tuning the Garbage Collector-Tuning the Garbage Collector](#)
调优，调优垃圾回收器-调优垃圾回收器
- using values versus pointers, [Don't Fear the Pointers](#)
使用值而不是指针，不要害怕指针
- generic data structures with generic functions, [Combining Generic Functions with Generic Data Structures-Combining Generic Functions with Generic Data Structures](#)
通用数据结构与通用函数，将通用函数与通用数据结构相结合
——将通用函数与通用数据结构相结合
- generic functions
通用函数
 - abstract algorithms, [Generic Functions Abstract Algorithms-Generic Functions Abstract Algorithms](#)
抽象算法，通用函数抽象算法-通用函数抽象算法

- actions made possible by, [Generics Reduce Repetitive Code and Increase Type Safety](#)

泛型带来的操作可以减少重复代码并提高类型安全性

- adding to standard library, [Adding Generics to the Standard Library](#)

向标准库添加泛型

- features not implemented in Go, [Things That Are Left Out-Things That Are Left Out](#)

Go 中未实现的功能，被遗漏的内容

- fixing struct pointer error, [A Quick Pointer Primer](#)

修复结构体指针错误，指针快速入门

- future feature possibilities, [Future Features Unlocked](#)

未来功能可能性，未来功能已解锁

- generic data structures with, [Combining Generic Functions with Generic Data Structures-Combining Generic Functions with Generic Data Structures](#)

将通用数据结构与通用函数结合使用

- idiomatic Go and, [Idiomatic Go and Generics](#)

惯用 Go 和，惯用 Go 和泛型

- interfaces and, [Generics and Interfaces](#)

接口和泛型与接口

- introducing, [Introducing Generics in Go-Introducing Generics in Go](#)

- stack created via, [Introducing Generics in Go](#)

- type constraint, [Introducing Generics in Go](#)
- Type Parameters Proposal URL, [Introducing Generics in Go](#)
- why not initially included, [Introducing Generics in Go](#)

Go 语言引入泛型 - Go 语言引入泛型 Stack 创建, Go 语言引入泛型类型约束, Go 语言引入泛型类型参数提案 URL, Go 语言引入泛型为何最初未包含在内, Go 语言引入泛型

- operators via type terms, [Use Type Terms to Specify Operators](#)-
[Use Type Terms to Specify Operators](#)
 - comparable interface with generics, [More on comparable](#)

通过类型项使用运算符, 使用类型项指定运算符 - 使用类型项指定运算符, 具有泛型的可比较接口, 更多关于可比较的信息

- repetitive code reduced, [Generics Reduce Repetitive Code and Increase Type Safety](#)-[Generics Reduce Repetitive Code and Increase Type Safety](#)

减少重复代码, 泛型减少重复代码并提高类型安全性——泛型减少重复代码并提高类型安全性

- type inference and, [Type Inference and Generics](#)
类型推断和类型推断与泛型
- GiB suffix, [Tuning the Garbage Collector](#)
GiB 后缀, 调整垃圾回收器

- Gin web framework, [Use third-party modules to enhance the server](#)
Gin Web框架，使用第三方模块增强服务器功能
- Git for version control, [Publishing Your Module](#)
 - git restore go.mod, [Importing Third-Party Code](#)
 - needed to download from GitHub, [Adding Third-Party Tools with go install](#)

使用 Git 进行版本控制，发布模块：`git restore go.mod`，导入第三方代码（需要从 GitHub 下载），使用 `go install` 添加第三方工具

- GitHub

GitHub

- Git needed for downloading, [Adding Third-Party Tools with go install](#)

下载需要 Git，使用 go install 添加第三方工具

- pushing workspace files to, [Using Workspaces to Modify Modules Simultaneously](#)

将工作区文件推送到，使用工作区同时修改模块

- Go

去

- backward compatibility, [Preface](#)
 - Go Compatibility Promise, [The Go Compatibility Promise](#), [Accept Interfaces](#), [Return Structs](#)

向后兼容性、前言、Go 兼容性承诺、Go 兼容性承诺、接受接口、返回结构体

- boring, [Preface](#)
 - but practical, [Go Isn't Particularly Object-Oriented \(and That's Great\)](#)

枯燥乏味，前言但实用，Go 语言并非特别面向对象（而且这很棒）

- call by value language, [Go Is Call by Value-Go Is Call by Value, Pointers Indicate Mutable Parameters](#)
 - map passed to a function, [The Difference Between Maps and Slices](#)
 - pointer passed to a function, [Pointers Indicate Mutable Parameters-Pointers Indicate Mutable Parameters](#)

按值调用语言，Go 语言是按值调用的，指针指示可变参数，传递给函数的映射，映射和切片的区别，传递给函数的指针，指针指示可变参数

- call-by-value language, [append](#)
按值调用语言，追加
- cgo for C library integration, [Cgo Is for Integration, Not Performance-Cgo Is for Integration, Not Performance](#)
cgo 用于 C 库集成，cgo 注重集成而非性能。

- compiling to a single native binary, [Installing the Go Tools](#),
[Importing Third-Party Code](#)
 - Go runtime compiled into, [Capacity](#)
 - Go tool updates and, [Staying Up-to-Date](#)
 - reading build info inside, [Reading the Build Info Inside a Go Binary](#)

编译成单个本地二进制文件、安装 Go 工具、导入第三方代码、编译后的 Go 运行时、容量、Go 工具更新和保持最新、读取内部构建信息、读取 Go 二进制文件中的构建信息

- creators of, [Strings and Runes and Bytes](#)
《字符串、符文和字节》的创作者
- Effective Go
有效
 - idiomatic Go resource, [go vet](#)
惯用语：去资源，去兽医
 - semicolon insertion rule, [go fmt](#)
分号插入规则， go fmt
- errors returned instead of exceptions thrown, [How to Handle Errors: The Basics](#)
 - (see also errors)

返回错误而不是抛出异常，如何处理错误：基础知识（另请参阅错误）

- first program, [Your First Go Program](#)-[go vet](#)
 - building executable, [go build](#)
 - code, [go build](#)
 - creating go module, [Making a Go Module](#), [Repositories, Modules, and Packages](#)
 - formatting of code, [go fmt](#)
 - packages, [go build](#)

第一个程序，你的第一个 Go 程序 - 构建可执行文件，go build
code, go build 创建 Go 模块，制作 Go 模块，仓库、模块和包 代码格式化，go fmt packages, go build

- formatting of code, [go fmt](#)
代码格式化，go fmt
- Go Programming Language Specification, [The Zero Value](#)
Go 编程语言规范，零值
- improvements, [Preface](#)
改进，前言
- installing Go tools, [Installing the Go Tools](#)
 - (see also development environment)

安装 Go 工具，安装 Go 工具（另请参阅开发环境）

- project organization, [Go Tooling](#)
项目组织，Go Tooling

- requirements for learning, [Who Should Read This Book](#)
学习要求, 以及本书的读者对象
- semantic versioning, [Working with Versions](#)
 - specifications URL, [Working with Versions](#)

语义化版本控制, 使用版本规范 URL, 使用版本

- The Go Playground, [The Go Playground](#)-[The Go Playground](#)
Go Playground, Go Playground-Go Playground
- updating managed by go directive, [Use the go Directive to Manage Go Build Versions](#)

使用 go 指令管理更新, 使用 go 指令管理 Go 构建版本

- wiki Code Review Comments page, [go vet](#)
wiki 代码审查评论页面, 去兽医

- go commands

go 命令

- about, [Go Tooling](#)
 - ./..., [go fmt](#)

关于, Go Tooling./..., go fmt

- go build, [go build](#)
 - -o flag for name or path change, [go build](#)
 - reading build info inside binary, [Reading the Build Info Inside a Go Binary](#)

- third-party packages, [Importing Third-Party Code-Importing Third-Party Code](#)

go build, go build-o 标志用于更改名称或路径, go build 读取二进制文件中的构建信息, 读取 Go 二进制文件中的构建信息, 第三方包, 导入第三方代码

- go clean -modcache, [Importing Third-Party Code](#)
执行清理 -modcache 命令, 导入第三方代码
- Go Compatibility Promise and, [The Go Compatibility Promise](#)
Go兼容性承诺和Go兼容性承诺
- go fmt, [go fmt](#)
 - braces not fixed by, [go fmt](#)
 - Go Playground Format button, [The Go Playground](#)
 - goimports as enhanced version, [Improving Import Formatting with goimports](#)

go fmt、go fmtbraces 未修复、go fmtGo Playground 格式按钮、Go Playgroundgoimports 作为增强版、使用 goimports 改进导入格式

- go generate, [Using go generate-Using go generate](#)
 - Makefiles and, [Working with go generate and Makefiles](#)

go generate, 使用 go generate - 使用 go generateMakefiles, 以及使用 go generate 和 Makefiles

- go get

去拿

- go get ./..., [Importing Third-Party Code](#), [Using Workspaces to Modify Modules Simultaneously](#), [Using Workspaces to Modify Modules Simultaneously](#)

go get ./..., 导入第三方代码, 使用工作区同时修改模块, 使用工作区同时修改模块

- module path passed to, [Importing Third-Party Code](#)
传递给导入第三方代码的模块路径

- proxy server, [Module Proxy Servers](#)
代理服务器, 模块代理服务器

- updating to compatible versions, [Updating to Compatible Versions](#)

更新至兼容版本

- updating to incompatible versions, [Updating to Incompatible Versions](#)

更新到不兼容的版本, 更新到不兼容的版本

- version specified, [Working with Versions](#)
指定版本, 使用版本

- go help, [Using go help to Learn More About Go Tooling](#)

- go help environment, [Adding Third-Party Tools with go install](#)

- go help importpath, [Using go help to Learn More About Go Tooling](#)

使用 go help 了解更多关于 Go 工具的信息；使用 go help environment 了解更多关于使用 go install 添加第三方工具的信息；使用 go help importpath 了解更多关于 Go 工具的信息。

- go install, [Documenting Your Code with Go Doc Comments](#), [Adding Third-Party Tools with go install](#)-[Adding Third-Party Tools with go install](#)
 - secondary Go environment, [Testing Versions of Go](#)

使用 Go 安装、使用 Go Doc 注释编写代码文档、使用 go install 添加第三方工具、使用 go install 辅助 Go 环境添加第三方工具、测试 Go 版本

- go list, [Working with Versions](#)
执行列表，使用版本
- go mod, [Making a Go Module](#)
 - go mod init, [Using go.mod](#), [Using Workspaces to Modify Modules Simultaneously](#)
 - go mod tidy, [Importing Third-Party Code](#), [Updating to Incompatible Versions](#)
 - go mod vendor, [Vendoring](#)

go mod，创建 Go 模块；go mod init，使用 go.mod；使用工作区同时修改模块；go mod tidy，导入第三方代码；更新到不兼容版本；go mod vendor，供应商

- go run, [Using go run to Try Out Small Programs](#)
使用 go run 运行程序来尝试运行小型程序。
- go test, [Pointer Passing Performance](#), [Understanding the Basics of Testing](#)-[Understanding the Basics of Testing](#), [Setting Up and Tearing Down](#)
 - benchmarks, [Using Benchmarks](#)
 - caching test results, [Caching Test Results](#)
 - code coverage, [Checking Your Code Coverage](#)-[Checking Your Code Coverage](#)
 - data race detector, [Finding Concurrency Problems with the Data Race Detector](#)
 - fuzzing, [Fuzzing-Fuzzing](#)
 - go-cmp to compare results, [Using go-cmp to Compare Test Results](#)-[Using go-cmp to Compare Test Results](#)
 - integration tests, [Using Integration Tests and Build Tags](#)
 - -short flag, [Using Integration Tests and Build Tags](#)
 - storing sample test data, [Storing Sample Test Data](#)

go 测试、指针传递性能、理解测试基础、设置和清理基准测试、使用基准测试、缓存测试结果、代码覆盖率、检查代码覆盖率、数据竞争检测器、使用数据竞争检测器查找并发问题、模糊测试、使用 go-cmp 比较测试结果、集成测试、使用集成测试和构建标签、存储示例测试数据

- go version, [Installing the Go Tools](#)

- -m flag to specify filename, [Reading the Build Info Inside a Go Binary](#)

go 版本控制，安装 Go Tools-m 标志以指定文件名，读取 Go 二进制文件中的构建信息

- go vet, [go vet](#)
 - shadowing not reported, [Shadowing Variables](#)
 - struct tag validation, [Using Struct Tags to Add Metadata](#)
 - third-party tools, [Using Code-Quality Scanners-golangci-lint](#)

go vet, go vetshadowing 未报告，Shadowing Variablesstruct 标签验证，使用 Struct 标签添加元数据第三方工具，使用代码质量扫描器-golangci-lint

- go work, [Using Workspaces to Modify Modules Simultaneously](#)
开始工作，使用工作区同时修改模块
- Makefiles to automate, [Makefiles](#)
Makefiles 用于自动化，Makefiles
- Go Doc comments, [Documenting Your Code with Go Doc Comments-Documenting Your Code with Go Doc Comments](#)
 - documentation URL, [Documenting Your Code with Go Doc Comments](#)

Go Doc 注释，使用 Go Doc 注释编写代码文档 - 使用 Go Doc 注释编写代码文档 URL，使用 Go Doc 注释编写代码文档

- The Go Playground, [The Go Playground](#)-[The Go Playground](#)
 - Google security for content removal, [The Go Playground](#)

Go Playground, Go Playground-Go PlaygroundGoogle 内容移除安全措施, Go Playground

- Go Programming Language Specification, [The Zero Value](#)
Go编程语言规范, 零值
- Go Project golang.org/x packages, [Improving Import Formatting with goimports](#)

Go 项目 golang.org/x 包, 使用 goimports 改进导入格式

- Go runtime, [Capacity](#)
 - compiling to a single native binary, [Installing the Go Tools](#),
[Importing Third-Party Code](#)
 - Go tool updates and, [Staying Up-to-Date](#)
 - reading build info inside, [Reading the Build Info Inside a Go Binary](#)
 - error runtime information via fmt.Errorf, [Use Strings for Simple Errors](#)
 - garbage collector tuning, [Tuning the Garbage Collector](#)-[Tuning the Garbage Collector](#)
 - generic function impact on performance, [Idiomatic Go and Generics](#)
 - hash algorithms implemented, [Maps](#)
 - scheduler, [Goroutines](#)

- more information URL, [Goroutines](#)

Go 运行时、容量、编译成单个本地二进制文件、安装 Go 工具、导入第三方代码、Go 工具更新和保持最新、读取内部构建信息、读取 Go 二进制文件中的构建信息、通过 `fmt.Errorf` 获取运行时错误信息、使用字符串表示简单错误、垃圾回收器调优、调优垃圾回收器、泛型函数对性能的影响、惯用的 Go 和泛型、已实现的哈希算法、Map、调度器、Goroutine、更多信息 URL、Goroutine

- “Go To Statement Considered Harmful” (Dijkstra), [goto—Yes, goto](#)
“前往被认为有害的声明”（迪克斯特拉），前往——是的，前往
- go-cmp to compare test results, [Using go-cmp to Compare Test Results](#)
[Results-Using go-cmp to Compare Test Results](#)
使用 go-cmp 比较测试结果
- go.mod file, [Making a Go Module](#), [Using go.mod](#)
 - contents of, [Using go.mod](#)
 - exclude directive, [Overriding Dependencies](#)
 - retract directive versus, [Retracting a Version of Your Module](#)
 - go directive
 - for-range value as copy, [The for-range value is a copy, Use the go Directive to Manage Go Build Versions](#)
 - managing Go build versions, [Use the go Directive to Manage Go Build Versions](#)
 - module path versioning tool, [Versioning Your Module](#)
 - replace directive, [Overriding Dependencies](#)

- workspaces for simultaneous module modifications, [Using Workspaces to Modify Modules Simultaneously](#)
- require directives, [The require Directive](#)
 - dependencies, [The require Directive](#)
 - imports marked as indirect, [Importing Third-Party Code](#)
- retract directive, [Retracting a Version of Your Module](#)
 - exclude directive versus, [Retracting a Version of Your Module](#)
- rolling back with git restore, [Importing Third-Party Code](#)
- synchronizing with go.sum, [Importing Third-Party Code](#)
- third-party code references, [Importing Third-Party Code](#)
- toolchain directive for version control, [Use the go Directive to Manage Go Build Versions](#)

go.mod 文件，创建 Go 模块，使用 go.mod 的内容，使用 go.modexclude 指令，覆盖依赖项，retract 指令与，撤回模块的某个版本，go 指令的 for-range 值作为副本，for-range 值是副本，使用 go 指令管理 Go 构建版本，管理 Go 构建版本，使用 go 指令管理 Go 构建版本，模块路径版本控制工具，模块版本控制，replace 指令，覆盖依赖项，用于同时修改模块的工作区，使用工作区同时修改模块，require 指令，require 指令依赖项，require 指令，标记为间接的导入，导入第三方代码，retract 指令，撤回模块的某个版本，exclude 指令与，撤回模块的某个版本，使用 git restore 回滚，导入第三方代码，使用 go.sum 同步，导入第三方代

码，第三方代码引用，导入第三方代码，用于版本控制的工具链指令，使用 go 指令管理 Go 构建版本

- go.sum file, [Importing Third-Party Code](#)
 - module listed though removed, [Working with Versions](#)
 - synchronizing with go.mod, [Importing Third-Party Code](#)

go.sum 文件，导入第三方代码模块（已列出但已移除），使用版本控制（与 go.mod 同步），导入第三方代码

- go.work file, [Using Workspaces to Modify Modules Simultaneously](#)
go.work 文件，使用工作区同时修改模块
- go/bin directory, [Adding Third-Party Tools with go install](#)
go/bin 目录，使用 go install 添加第三方工具
- go:embed, [Embedding Content into Your Program-Embedding Content into Your Program](#)

go:embed，将内容嵌入到您的程序中 - 将内容嵌入到您的程序中

- GOARCH environment variable, [Building Go Binaries for Other Platforms](#)

GOARCH 环境变量，用于为其他平台构建 Go 二进制文件

- GOBIN environment variable, [Adding Third-Party Tools with go install](#)

GOBIN 环境变量，使用 go install 添加第三方工具

- GOGC environment variable, [Tuning the Garbage Collector-Tuning the Garbage Collector](#)

GOGC 环境变量，调优垃圾回收器 - 调优垃圾回收器

- goimports tool improving formatting, [Improving Import Formatting with goimports](#)

goimports 工具改进格式，使用 goimports 改进导入格式

- GoLand (JetBrains), [GoLand](#)
GoLand (JetBrains) , GoLand
- golang.org/x packages, [Improving Import Formatting with goimports](#)
 - errgroup.Group for WaitGroup errors, [Use WaitGroups](#)
 - golang.org/x/sys, [unsafe Is Unsafe](#)

golang.org/x 包，使用 goimporterrgroup.Group 改进导入格式以处理 WaitGroup 错误，使用 WaitGroupsgolang.org/x/sys，unsafe 是不安全的

- golangci-lint linter, [golangci-lint-golangci-lint](#)
golangci-lint linter、golangci-lint-golangci-lint
- GOMEMLIMIT environment variable, [Tuning the Garbage Collector, Contexts with Deadlines](#)
 - soft limit, [Tuning the Garbage Collector](#)

GOMEMLIMIT 环境变量、垃圾回收器调优、带有 Deadlinesoft 限制的上下文、垃圾回收器调优

- Gonum package for numerical computing, [Complex types \(you're probably not going to use these\)](#)
Gonum 包用于数值计算，复杂类型（您可能不会用到这些）
- Google proxy server, [Module Proxy Servers](#)

- checksum database, [Module Proxy Servers](#)

Google代理服务器，模块代理服务器校验和数据库，模块代理服务器

- Google security for Playground content, [The Go Playground](#)

Google Playground 内容安全保障，Go Playground

- Google Wire helper, [Wire](#)

Google Wire 助手，Wire

- GOOS environment variable, [Building Go Binaries for Other Platforms](#)

GOOS 环境变量，为其他平台构建 Go 二进制文件

- GOPATH environment variable, [Adding Third-Party Tools with go install](#)

GOPATH 环境变量，使用 go install 添加第三方工具

- gopls language server, [Visual Studio Code](#)

gopls 语言服务器，Visual Studio Code

- GOPRIVATE environment variable, [Using Private Repositories](#)

GOPRIVATE 环境变量，使用私有仓库

- GOPROXY environment variable, [Specifying a Proxy Server](#), [Adding Third-Party Tools with go install](#)

GOPROXY 环境变量、指定代理服务器、使用 go install 添加第三方工具

- gorilla mux request router, [Use third-party modules to enhance the server](#)

gorilla mux 请求路由器，使用第三方模块增强服务器

- GOROOT environment variable, [Adding Third-Party Tools with go install](#)

GOROOT 环境变量，使用 go install 添加第三方工具

- goroutines in concurrency, [Goroutines](#)
 - always clean up goroutines, [Always Clean Up Your Goroutines](#)
 - cancellation in your own code, [Context Cancellation in Your Own Code](#)
 - context cancellation, [Use the Context to Terminate Goroutines](#), [Cancellation-Cancellation](#)
 - context to terminate goroutines, [Use the Context to Terminate Goroutines](#)
 - goroutine leak, [Always Clean Up Your Goroutines](#)
 - channels for communication, [Channels](#)
 - for loop index variable, [Goroutines, for Loops, and Varying Variables-Goroutines, for Loops, and Varying Variables](#)
 - launching, [Goroutines](#)
 - must eventually exit, [Always Clean Up Your Goroutines](#)
 - scheduler, [Goroutines](#)
 - more information URL, [Goroutines](#)
 - terminating, [Use the Context to Terminate Goroutines](#)
 - cancellation in your own code, [Context Cancellation in Your Own Code](#)

- context cancellation, [Use the Context to Terminate Goroutines](#), [Cancellation-Cancellation](#)
- waiting for others to finish, [Use WaitGroups](#)-[Use WaitGroups](#)

并发中的 goroutine, 始终清理 goroutine, 始终清理你的 goroutine, 在你自己的代码中取消, 在你自己的代码中取消上下文, 上下文取消, 使用上下文终止 goroutine, 取消 - 取消, 使用上下文终止 goroutine, 使用上下文终止 goroutine, goroutine 泄漏, 始终清理你的 goroutine, 通信通道, 通道 for 循环索引变量, goroutine、for 循环和可变变量 - goroutine、for 循环和可变变量, 启动, goroutine 必须最终退出, 始终清理你的 goroutine, 调度器, goroutine 更多信息 URL, goroutine 终止, 使用上下文终止 goroutine, 在你自己的代码中取消, 在你自己的代码中取消上下文, 上下文取消, 使用上下文终止 goroutine, 取消 - 取消, 等待为了让其他人完成, 请使用等待组 - 使用等待组

- goto keyword generally not used, [goto—Yes,.goto-goto—Yes,.goto](#)
goto 关键字通常不使用, goto—是, goto-goto—是, goto
- GOTOOLCHAIN environment variable, [Use the go Directive to Manage Go Build Versions](#)
GOTOOLCHAIN 环境变量, 使用 go 指令管理 Go 构建版本
- govulncheck tool for security vulnerabilities, [Using.govulncheck to Scan for Vulnerable Dependencies](#)-[Using.govulncheck to Scan for Vulnerable Dependencies](#)

govulncheck 是一款用于安全漏洞检测的工具，使用 govulncheck 扫描易受攻击的依赖项。

- GOWORK environment variable, [Using Workspaces to Modify Modules Simultaneously](#)
GOWORK 环境变量，使用工作区同时修改模块
- Grokking Algorithms (Bhargava), [Maps](#)
理解算法 (Bhargava) , 地图
- “A Guide to the Go Garbage Collector” URL, [Tuning the Garbage Collector](#)
“Go 垃圾回收器指南”网址，调整垃圾回收器
- gzip package
gzip 包
 - gzip.NewReader function, [io and Friends](#)
gzip.NewReader 函数、io 和 Friends
 - gzip.Reader pointer to io.Reader, [io and Friends](#)
gzip.Reader 指向 io.Reader、io 和 Friends

H

H

- hash map implementation of maps, [Maps](#)
 - Go runtime hash algorithm implementation, [Maps](#)

Maps 的哈希映射实现，MapsGo 运行时哈希算法实现，Maps

- Haskell having higher-order functions, [Returning Functions from Functions](#)

Haskell 支持高阶函数，函数返回函数

- heap, [Reducing the Garbage Collector's Workload](#)
 - garbage collector tuning, [Tuning the Garbage Collector-Tuning the Garbage Collector](#)
 - garbage collector workload reduced, [Reducing the Garbage Collector's Workload-Reducing the Garbage Collector's Workload](#)
 - interface trade-off, [Accept Interfaces, Return Structs](#)
 - goroutine leak, [Always Clean Up Your Goroutines](#)
 - heap size control environment variable, [Tuning the Garbage Collector-Tuning the Garbage Collector](#)
 - more information on heap versus stack, [Reducing the Garbage Collector's Workload](#)

堆、减少垃圾回收器的工作负载、垃圾回收器调优、调整垃圾回收器、减少垃圾回收器的工作负载、接口权衡、接受接口、返回结构体、goroutine 泄漏、始终清理 goroutine、堆大小控制环境变量、调整垃圾回收器、更多关于堆与栈的信息、减少垃圾回收器的工作负载

- Hello World, [Your First Go Program-go vet](#)
 - building executable, [go build](#)
 - go fmt before, [go fmt](#)

- code, [go build](#)
 - formatting, [go fmt](#)
 - packages, [go build](#)
- creating go module, [Making a Go Module](#)
 - nonunique name, [Repositories, Modules, and Packages](#)

Hello World, 你的第一个 Go 程序 - go vet 构建可执行文件, go build go fmt 之前, go fmt code, go build 格式化, go fmt packages, go build 创建 Go 模块, 使 Go 模块名称不唯一, 仓库、模块和包

- Helm, Richard, [Use Embedding for Composition](#), [Interfaces Are Type-Safe Duck Typing](#)

Helm, Richard, 《使用嵌入进行组合, 接口是类型安全的鸭子类型》

- helper functions

辅助函数

- building packages, [Naming Packages](#)
构建软件包, 命名软件包
- fixing struct pointer error, [A Quick Pointer Primer](#)
修复结构体指针错误, 指针快速入门
- io package, [io and Friends](#)
io 包, io 和朋友们
- maps
地图

- maps.Equal, [Comparing Maps](#)
地图相等, 比较地图
- maps.EqualFunc, [Comparing Maps](#)
maps.EqualFunc, 比较地图
- running a function exactly once, [Run Code Exactly Once](#)
仅运行一次函数, 仅运行一次代码
- hexadecimal prefix (0x), [Literals](#)
 - uses for hexadecimal, [Literals](#)

十六进制前缀 (0x), 十六进制文字, 文字

- hey tool to load test HTTP servers, [Adding Third-Party Tools with go install](#)

嘿, 这是一个用于对 HTTP 服务器进行负载测试的工具, 使用 go install 添加第三方工具。

- hidden files in embedded content, [Embedding Hidden Files](#)
嵌入内容中的隐藏文件, 嵌入隐藏文件
- “higher-order functions” definition, [Returning Functions from Functions](#)
“高阶函数”定义, 从函数返回函数
- Hoare, Tony, [Concurrency in Go](#)
Hoare, Tony, 《Go 语言中的并发性》
- Homebrew installation of Go, [Installing the Go Tools](#)
 - updating Go tools, [Staying Up-to-Date](#)

使用 Homebrew 安装 Go, 安装 Go 工具, 更新 Go 工具, 保持最新

- http support (see net/http package)
http 支持 (参见 net/http 软件包)
- Hudson, Rick, [Reducing the Garbage Collector's Workload](#)
哈德森, 里克, 《减少垃圾收集员的工作量》
- Hyrum's law, [Organizing Your Module](#)
海勒姆定律, 组织你的模块

I

我

- IDEs, [Choose Your Tools](#)
 - Go development tools URL, [Choose Your Tools](#)
 - GoLand, [GoLand](#)
 - Visual Studio Code, [Visual Studio Code](#)

IDE, 选择你的工具, 前往开发工具网址, 选择你的工具, 前往
Land, 前往Land, Visual Studio Code, Visual Studio Code

- if keyword, [if](#)
 - choosing between switch and if, [Choosing Between if and switch](#)
 - shadowing variables, [Shadowing Variables](#)
 - variables scoped to if and else blocks, [if](#)

if关键字，如果在switch和if之间进行选择，在if和switch之间进行选择，变量阴影，变量阴影，作用于if和else代码块的变量，如果

- imag function, [Complex types \(you're probably not going to use these\)](#), 图像函数，复杂类型（您可能不会用到这些）
- ImageMagick, [Cgo Is for Integration, Not Performance](#)
ImageMagick 和 Cgo 用于集成，而非性能
- imaginary literals, [Complex types \(you're probably not going to use these\)](#), 虚构字面量，复杂类型（您可能不会用到这些）
- immutable values in Go, [Pointers Indicate Mutable Parameters](#)
 - constant immutability, [Using const](#)

Go 语言中的不可变值、指针指示可变参数、常量不可变性、使用 const

- import compatibility rule, [Minimal Version Selection](#)
导入兼容性规则，最小版本选择
- import keyword, [go build](#), [Importing and Exporting](#)
 - blank imports, [Avoiding the init Function if Possible](#)
 - import path, [Creating and Accessing a Package](#)
 - tool to automate versioning, [Versioning Your Module](#)
 - shadowing variables, [Shadowing Variables](#)
 - third-party code imported, [Importing Third-Party Code](#)

导入关键字、go build、导入和导出、空白导入、尽可能避免使用 init 函数、导入路径、创建和访问包、自动化版本控制工具、模块版本控制、变量遮蔽、变量遮蔽、导入第三方代码、导入第三方代码

- import path, [Creating and Accessing a Package](#)
 - relative path import paths not working with modules, [Creating and Accessing a Package](#)

导入路径、创建和访问包、相对路径导入路径不适用于模块、创建和访问包

- index expression for strings, [Strings and Runes and Bytes](#)
字符串、字符串、符文和字节的索引表达式
- $\pm\text{Inf}$ when dividing by zero, [Floating-point types](#)
除以零时为 $\pm$ 无穷大，浮点类型
- inheritance not a part of Go, [Type Declarations Aren't Inheritance](#)
 - code reuse via composition and promotion, [Use Embedding for Composition](#)
 - embedding is not inheritance, [Embedding Is Not Inheritance](#)

继承并非 Go 语言的一部分，类型声明并非继承，代码重用通过组合和类型提升实现，使用嵌入进行组合，嵌入并非继承，嵌入并非继承

- init function avoided, [Avoiding the init Function if Possible](#), [Run Code Exactly Once](#)
 - blank imports, [Avoiding the init Function if Possible](#)

避免使用初始化函数，尽可能避免使用初始化函数，代码只运行一次，空白导入，尽可能避免使用初始化函数

- “Inside the Map Implementation” video, [Maps](#)
“地图实现内部原理”视频，地图
- int for int32 or int64, [The special integer types](#)
 - uint as unsigned int, [The special integer types](#)

int 代表 int32 或 int64，特殊的整数类型 uint 也代表 unsigned int，特殊的整数类型

- int32 via rune type, [A Taste of Strings and Runes](#)
int32 通过符文类型，弦乐与符文的滋味
- integer literals, [Literals](#)
 - defaulting to int type, [The special integer types](#)
 - prefixes indicating bases, [Literals](#)
 - underscores for readability, [Literals](#)

整数字面量，字面量默认为 int 类型，特殊的整数类型前缀表示基数，字面量使用下划线以提高可读性，字面量

- integer types, [Integer types-Integer operators](#)
 - arithmetic operators, [Integer operators](#)

- dividing by zero, [Integer operators](#)
- choosing which to use, [Choosing which integer to use](#)
- special integer types, [The special integer types](#)
- zero value as 0, [Integer types](#)

整数类型、整数类型 - 整数运算符、算术运算符、整数运算符
(除以零)、整数运算符 (选择要使用的整数)、选择要使用的
整数 (特殊整数类型)、特殊整数类型 (零值为 0)、整数类型

- integration tests, [Using Integration Tests and Build Tags](#)
集成测试, 使用集成测试和构建标签
- interface keyword, [A Quick Lesson on Interfaces](#)
 - interface {}, [The Empty Interface Says Nothing](#)
 - any as type alias for, [The Empty Interface Says Nothing](#)

interface 关键字, 接口简述 interface {}, 空接口什么也不说 any 作
为类型别名, 空接口什么也不说

- interface literals, [A Quick Lesson on Interfaces](#)
接口字面量, 接口速成课
- interfaces, [A Quick Lesson on Interfaces](#)
 - accept interfaces, return structs, [Accept Interfaces, Return Structs](#)-
[Accept Interfaces, Return Structs](#)
 - error interface type exception, [Accept Interfaces, Return Structs](#)
 - heap allocation trade-off, [Accept Interfaces, Return Structs](#)

- as comparable, [Interfaces Are Comparable](#), [More on comparable](#)
- comparable interface for any, [Introducing Generics in Go](#), [More on comparable](#)
 - interfaces as type constraints, [Generics and Interfaces](#)
- declaring, [A Quick Lesson on Interfaces](#)
 - named with “er” endings, [A Quick Lesson on Interfaces](#)
- embedding and, [Embedding and Interfaces](#)
- empty interface for variable value of any type, [The Empty Interface Says Nothing](#)
 - any as type alias for, [The Empty Interface Says Nothing](#)
- functions implementing, [Function Types Are a Bridge to Interfaces](#)
- generic functions and, [Generics and Interfaces](#)
- implicit implementation, [Interfaces Are Type-Safe Duck Typing](#)
 - dependency injection easier, [Implicit Interfaces Make Dependency Injection Easier](#)-[Implicit Interfaces Make Dependency Injection Easier](#)
 - safety and decoupling, static and dynamic, [Interfaces Are Type-Safe Duck Typing](#)-[Interfaces Are Type-Safe Duck Typing](#)
- nil and, [Interfaces and nil](#)
 - reflection checking for nil value, [Use Reflection to Check If an Interface’s Value Is nil](#)

- optional interfaces, [Use Type Assertions and Type Switches Sparingly](#)
- passing functions as parameters or interface, [Function Types Are a Bridge to Interfaces](#)
- standard interfaces, [Interfaces Are Type-Safe Duck Typing](#)
 - interface instance returning type implementing same interface, [Interfaces Are Type-Safe Duck Typing](#)
 - io package, [Interfaces Are Type-Safe Duck Typing](#), [io](#) and [Friends](#), [io](#) and [Friends](#)
- type assertions, [Type Assertions and Type Switches](#)
 - uses for, [Use Type Assertions and Type Switches Sparingly](#)-[Use Type Assertions and Type Switches Sparingly](#)
- type elements, [Use Type Terms to Specify Operators](#)-[Use Type Terms to Specify Operators](#)
 - only valid as type constraints, [Use Type Terms to Specify Operators](#)
 - type terms, [Use Type Terms to Specify Operators](#)-[Use Type Terms to Specify Operators](#)
- type switches, [Type Assertions and Type Switches](#)
 - uses for, [Use Type Assertions and Type Switches Sparingly](#)-[Use Type Assertions and Type Switches Sparingly](#)
- zero value as nil, [Interfaces and nil](#)

接口，接口速览，接受接口，返回结构体，接受接口，返回结构

体 - 接受接口, 返回结构体, 错误接口类型异常, 接受接口, 返回结构体, 堆分配权衡, 接受接口, 返回结构体作为可比较对象, 接口是可比较的, 更多关于可比较对象的内容, 任何类型的可比较接口, Go 语言中的泛型介绍, 更多关于可比较对象的内容, 接口作为类型约束, 泛型和接口声明, 接口速览, 以“er”结尾命名的接口, 接口速览, 嵌入和, 嵌入和接口, 任何类型变量值的空接口, 空接口什么也不说, 任何类型作为别名, 空接口什么也不说, 实现的函数, 函数类型是通往接口的桥梁, 泛型函数和, 泛型和接口隐式实现, 接口是类型安全的鸭子类型, 依赖注入更容易, 隐式接口使依赖注入更容易 - 隐式接口使依赖注入更容易注入更便捷, 安全性更高, 解耦更彻底, 静态和动态, 接口是类型安全的鸭子类型——接口是类型安全的鸭子类型, nil 和, 接口和 nil, 反射检查 nil 值, 使用反射检查接口的值是否为 nil, 可选接口, 谨慎使用类型断言和类型开关, 将函数作为参数或接口传递, 函数类型是通往接口的桥梁, 标准接口, 接口是类型安全的鸭子类型, 返回实现同一接口类型的接口实例, 接口是类型安全的鸭子类型, io 包, 接口是类型安全的鸭子类型, io 及其相关包, io 及其相关包, 类型断言, 类型断言和类型开关的用途, 谨慎使用类型断言和类型开关——谨慎使用类型断言和类型开关, 类型元素, 使用类型术语指定运算符——使用类型术语指定运算符, 仅作为类型约束有效, 使用类型术语指定运算符, 类型术语, 使用类型术语指定运算符 - 使用类型术语来指定运算符类型开关、类型断言和类型开关的用途, 谨慎使用类型断言和类型开关 - 谨慎使用类型断言和类型开关零值作为 nil, 接口和 nil

- internal packages, [Using the internal Package](#)
内部包, 使用内部包
- io package, [io and Friends](#)-[io and Friends](#)
 - helper functions, [io and Friends](#)
 - io.Closer, [io and Friends](#)
 - io.Copy, [Use Type Assertions and Type Switches Sparingly](#), [io and Friends](#)
 - io.LimitReader, [io and Friends](#)
 - io.MultiReader, [io and Friends](#)
 - io.MultiWriter, [io and Friends](#)
 - io.NopCloser, [io and Friends](#)
 - io.ReadAll, [io and Friends](#)
 - io.ReadCloser, [io and Friends](#)
 - io.Reader, [io and Friends](#)-[io and Friends](#)
 - io.EOF, [io and Friends](#)
 - io.ErrUnexpectedEOF, [io and Friends](#)
 - passing to bufio.NewReader, [Use Type Assertions and Type Switches Sparingly](#)
 - standard interface, [Interfaces Are Type-Safe Duck Typing](#), [io and Friends](#)
 - strings.NewReader function, [io and Friends](#)
 - io.ReadSeeker, [io and Friends](#)
 - io.ReadWriteCloser, [io and Friends](#)
 - Io.ReadWriter, [io and Friends](#)

- io.ReadWriteSeeker, [io and Friends](#)
- io.Seeker, [io and Friends](#)
 - whence constants, [io and Friends](#)
- io.WriteCloser, [io and Friends](#)
- io.Writer, [io and Friends](#)-[io and Friends](#)
 - standard interface, [Interfaces Are Type-Safe Duck Typing](#), [io and Friends](#)
- io.WriteSeeker, [io and Friends](#)
- standard interfaces, [Interfaces Are Type-Safe Duck Typing](#), [io and Friends](#), [io and Friends](#)

io 包、io 和 Friends-io 以及 Friendshelper 函数、io 和 Friendsio.Closer、io 和 Friendsio.Copy、谨慎使用类型断言和类型切换、io 和 Friendsio.LimitReader、io 和 Friendsio.MultiReader、io 和 Friendsio.MultiWriter、io 和 Friendsio.NopCloser、io 和 Friendsio.ReadAll、io 和 Friendsio.ReadCloser、io 和 Friendsio.Reader、io 和 Friends-io 以及 Friendsio.EOF、io 和 Friendsio.ErrUnexpectedEOF、io 和 Friends 传递给 bufio.NewReader、谨慎使用类型断言和类型切换标准接口、接口是类型安全的鸭子类型、io 和 Friendsstrings.NewReader 函数、io 和 Friendsio.ReadSeeker、io 和 Friendsio.ReadWriteCloser、io 和 FriendsIo.ReadWrite、io 和 Friendsio.ReadWriteSeeker、io 和 Friendsio.Seeker、io 和 Friendswhence 常量、io 和 Friendsio.WriteCloser、io 和 Friendsio.Writer、io 和 Friends-io 以及

Friends 标准接口、接口是类型安全的鸭子类型、io 和 Friendsio.WriteSeeker、io 和 Friends 标准接口、接口是类型安全的鸭子类型、io 和 Friends、io 和 Friends

- io/fs package

io/fs 包

- fs.embed.FS type, [Embedding Content into Your Program](#)
fs.embed.FS 类型, 将内容嵌入到您的程序中
- fs.FS, [Embedding Content into Your Program](#)
fs.FS, 将内容嵌入到您的程序中
- fs.ReadDirFS, [Embedding Content into Your Program](#)
fs.ReadDirFS, 将内容嵌入到您的程序中
- fs.ReadFileFS, [Embedding Content into Your Program](#)
fs.ReadFileFS, 将内容嵌入到您的程序中
- fs.WalkDir, [Embedding Content into Your Program](#)
fs.WalkDir, 将内容嵌入到您的程序中
- iota for enumerations, [iota Is for Enumerations—Sometimes-iota Is for Enumerations—Sometimes](#)
 - advice on, [iota Is for Enumerations—Sometimes](#)
 - origin of iota, [iota Is for Enumerations—Sometimes](#)

iota 用于枚举, iota 用于枚举——有时-iota 用于枚举——有时建议, iota 用于枚举——有时 iota 的起源, iota 用于枚举——有时

- It's FOSS resource for open source licenses, [Publishing Your Module](#)
这是一个关于开源许可证的 FOSS 资源, 用于发布您的模块

J

J

- JetBrains GoLand, [GoLand](#)
JetBrains GoLand, GoLand
- Johnson, Ralph, [Use Embedding for Composition](#), [Interfaces Are Type-Safe Duck Typing](#)
约翰逊, 拉尔夫, 《使用嵌入进行组合, 接口是类型安全的鸭子类型》
- Joshi, Kavya, [Goroutines](#)
Joshi、Kavya、Goroutines
- JSON support in encoding/json package
encoding/json 包中对 JSON 的支持
 - json.Unmarshal
json.Unmarshal
 - populating variable, [Pointers Are a Last Resort](#)
填充变量时, 指针是最后的手段。
 - zero value versus no value via pointers, [The Zero Value Versus No Value](#)
通过指针表示零值与无值, 零值与无值
- JSON support in encoding/json package, [encoding/json-Custom JSON Parsing](#)
 - about JSON, [encoding/json](#)

- custom JSON parsing, [Custom JSON Parsing-Custom JSON Parsing](#)
- json.Decoder, [JSON, Readers, and Writers-JSON, Readers, and Writers](#)
 - multiple JSON structs at once, [Encoding and Decoding JSON Streams](#)
- json.Encoder, [JSON, Readers, and Writers-JSON, Readers, and Writers](#)
 - multiple JSON structs at once, [Encoding and Decoding JSON Streams](#)
- json.Marshal, [Unmarshaling and Marshaling](#)
- json.Unmarshal, [Unmarshaling and Marshaling](#)
 - about unmarshaling, [encoding/json](#)
 - dash for name of ignored field, [Using Struct Tags to Add Metadata](#)
- marshaling versus unmarshaling, [encoding/json](#)
- struct tags to add metadata, [Using Struct Tags to Add Metadata](#)

encoding/json 包中的 JSON 支持, encoding/json - 自定义 JSON 解析, 关于 JSON, encoding/json 自定义 JSON 解析, 自定义 JSON 解析, json.Decoder, JSON、读取器和写入器 - JSON、读取器和写入器, 一次处理多个 JSON 结构体, 编码和解码 JSON 流, json.Encoder, JSON、读取器和写入器 - JSON、读取器和写入器, 一次处理多个 JSON 结构体, 编码和解码 JSON 流,

json.Marshal, 反序列化和序列化, json.Unmarshal, 反序列化和序列化, 关于反序列化, encoding/json 使用破折号表示忽略字段的名称, 使用结构体标签添加元数据, 序列化与反序列化的区别, encoding/json 使用结构体标签添加元数据, 使用结构体标签添加元数据

- json tag with struct field, [Using Struct Tags to Add Metadata](#)
使用结构体字段的 JSON 标签, 使用结构体标签添加元数据

K

K

- Kennedy, Bill, [Reducing the Garbage Collector's Workload](#)

肯尼迪, 比尔, 《减轻垃圾收集员的工作量》

- keywords

关键词

- break

休息

- for statement, [break and continue](#)

对于语句, 中断并继续

- switch statement, [switch](#)

switch语句, switch

- case, [switch-switch](#)

案例, 开关

- chan, [Reading, Writing, and Buffering](#)

chan, 阅读、写作和缓冲

- const, [Using const-Typed and Untyped Constants](#)

const, 使用 const 类型化常量和非类型化常量

- continue, [break and continue](#)

继续, 中断, 然后继续

- default, [switch-switch](#)

默认, 开关

- defer, [defer-defer](#), [Wrapping Errors with defer](#)
defer、defer-defer、使用 defer 包装错误
- else, [if](#)
否则，如果
- fallthrough avoided, [switch](#)
避免了失败，切换
- for, [for, Four Ways-Choosing the Right for Statement](#)
四种方式——选择正确的陈述
- func
函数
 - function declarations, [Declaring and Calling Functions](#)
函数声明，声明和调用函数
 - method declarations, [Methods](#)
方法声明，方法
- go, [Go Tooling](#)
 - (see also concurrency; goroutines in concurrency)

go, Go 工具 (另见并发; 并发中的 goroutine)
- goto generally not used, [goto—Yes, goto-goto—Yes, goto](#)
goto 通常不使用, goto—是, goto-goto—是, goto
- if, [if](#)
如果，如果
- import, [go build](#), [Importing and Exporting](#)
导入、构建、导入和导出

- interface, [A Quick Lesson on Interfaces](#)
 - interface {}, [The Empty Interface Says Nothing](#)

接口, 接口速成课 interface {}, 空接口什么也不说

- map, [Maps](#)
地图, 地图集
- package, [Creating and Accessing a Package](#)
包、创建和访问包
- predeclared identifiers versus, [Shadowing Variables](#)
预先声明的标识符与变量遮蔽
- range, [The for-range Statement-The for-range value is a copy.](#)
范围, for-range 语句——for-range 的值是副本
- return, [Declaring and Calling Functions](#)
返回、声明和调用函数
- select, [select-select](#)
选择, 选择-选择
- struct, [Structs](#)
结构体, 结构
- switch, [switch-switch](#)
开关, 开关-开关
- type, [Types in Go](#)
 - alias declaration, [Gracefully Renaming and Reorganizing Your API](#)
 - functions, [Function Type Declarations](#)

- interface, [A Quick Lesson on Interfaces](#)
- structs, [Structs, Types in Go](#)

类型、Go 中的类型、别名声明、优雅地重命名和重组 API、函数、函数类型声明、接口、接口速成课、结构体、Go 中的结构体和类型

- var, [var Versus :=-var Versus :=](#)
var, var 与 := -var 与 :=
- KiB suffix, [Tuning the Garbage Collector](#)
KiB 后缀, 调整垃圾回收器
- Kubernetes containers with Go app image, [Installing the Go Tools](#)
 - blog post resource, [Installing the Go Tools](#)

使用 Go 应用镜像的 Kubernetes 容器, 安装 Go 工具 (博客文章资源), 安装 Go 工具

L

L

- language servers, [Visual Studio Code](#)
语言服务器, Visual Studio Code
- Layher, Matt, [unsafe Is Unsafe](#)
莱赫尔, 马特, 不安全就是不安全

- lazy load, [Run Code Exactly Once](#)

延迟加载，仅运行一次代码

- len function

len 函数

- array length, [Arrays—Too Rigid to Use Directly](#)
数组长度，数组——过于僵化而无法直接使用
- channel buffer contents, [Reading, Writing, and Buffering](#)
通道缓冲区内容、读取、写入和缓冲
- map key-value pair count, [Maps](#)
映射键值对计数，映射
- slice length, [len](#)
 - example code, [Capacity](#)

切片长度、lenexample 代码、容量

- string length in bytes, [Strings and Runes and Bytes](#)
字符串长度（以字节为单位），字符串、符文和字节
- lib.go file, [Using Workspaces to Modify Modules Simultaneously](#),
[Using Workspaces to Modify Modules Simultaneously](#)
lib.go 文件，使用工作区同时修改模块，使用工作区同时修改模块
- LICENSE file for open source modules, [Publishing Your Module](#)
 - It's FOSS resource, [Publishing Your Module](#)

开源模块的 LICENSE 文件，发布您的模块。这是 FOSS 资源，发

布您的模块

- Lindamood, Jack, [Accept Interfaces, Return Structs](#)
Lindamood, Jack, 接受接口, 返回结构体
- linters, [Using Code-Quality Scanners-golangci-lint](#)
 - golangci-lint, [golangci-lint](#)
 - revive, [revive](#)
 - staticcheck, [staticcheck-staticcheck](#)

代码检查器, 使用代码质量扫描器-golangci-lintgolangci-lint、
golangci-lintrevive、revivestaticcheck、staticcheck-staticcheck

- Linux installation of Go, [Installing the Go Tools](#)
 - updating Go tools, [Staying Up-to-Date](#)

Linux 系统下 Go 语言的安装、Go 工具的安裝、Go 工具的更新、
保持最新状态

- literals, [Literals](#)
 - array literals, [Arrays—Too Rigid to Use Directly](#)
 - constants for naming literals, [Using const](#)
 - floating-point literals, [Literals](#)
 - imaginary literals, [Complex types \(you're probably not going to use these\)](#)
 - integer literals, [Literals](#)
 - interface literals, [A Quick Lesson on Interfaces](#)
 - map literals, [Maps](#)

- rune literals, [Literals](#)
- slice literals, [Slices](#)
- string literals, [Literals](#)
 - raw string literals, [Literals](#)
- struct literals, [Structs](#)
- untyped, [Literals](#), [Literals Are Untyped](#)
 - default type, [Literals](#)
 - type conversion, [var Versus :=](#)

字面量、字面量数组字面量、数组——过于死板而无法直接使用、用于命名字面量的常量、使用 `const`、浮点字面量、字面量、虚数字面量、复杂类型（您可能不会用到这些）、整数字面量、字面量、接口字面量、接口速览、映射字面量、映射、符文字面量、字面量、切片字面量、切片、字符串字面量、字面量、原始字符串字面量、字面量结构体字面量、结构体、无类型字面量、字面量是无类型字面量、默认类型、字面量类型转换、`var` 与 `:=`

- log/slog package, [Structured Logging-Structured Logging](#)
 - API documentation URL, [Structured Logging](#)
 - `slog.Debug`, [Structured Logging](#)
 - `slog.Error`, [Structured Logging](#)
 - `slog.Info`, [Structured Logging](#)
 - `slog.NewLogLogger`, [Structured Logging](#)
 - `slog.Warn`, [Structured Logging](#)

log/slog 包、结构化日志记录 - 结构化日志记录 API 文档 URL、
结构化日志记录 log.Debug、结构化日志记录 log.Error、结构化日
志记录 log.Info、结构化日志记录 log.NewLogLogger、结构化日
志记录 log.Warn、结构化日志记录

- looping (see for statement)
循环（参见语句）

M

M

- Mac Homebrew installation of Go, [Installing the Go Tools](#)
 - updating Go tools, [Staying Up-to-Date](#)

使用 Homebrew 在 Mac 上安装 Go，安装 Go 工具，更新 Go 工
具，保持最新

- main package, [go build](#)
 - main function, [go build](#), [Declaring and Calling Functions](#)
 - no input parameters, [Declaring and Calling Functions](#)
 - module organization, [Organizing Your Module](#)

主包，go build 主函数，go build，声明和调用无输入参数的函
数，声明和调用模块组织，组织你的模块

- make function

使功能

- channels created, [Channels](#)
 - buffer capacity, [Reading, Writing, and Buffering](#)

已创建的通道、通道缓冲区容量、读取、写入和缓冲

- map declaration, [Maps](#)
地图声明, 地图
- slice declaration, [make](#)
 - append function after, [make](#), [Declaring Your Slice](#)

切片声明, makeappend 函数之后, make, 声明你的切片

- Makefiles, [Makefiles](#)
 - go generate and, [Working with go generate and Makefiles](#)
 - make fmt, [Makefiles](#)
 - make vet, [Makefiles](#)
 - tutorial URL, [Makefiles](#)

Makefiles、Makefilesgo generate 和、使用 go generate 和
Makefilesmake fmt、Makefilesmake vet、Makefiles 教程 URL、
Makefiles

- map literals, [Maps](#)
地图文字, 地图
- maps, [Maps-Using Maps as Sets](#)

- call by value language, [Go Is Call by Value](#)
 - maps for input parameters or return values, [The Difference Between Maps and Slices](#)
 - pointer implementation of maps, [Go Is Call by Value](#), [The Difference Between Maps and Slices](#)
- comma ok idiom for keys not in map, [The comma ok Idiom](#)
- declaration, [Maps](#)
 - make function for sized map that grows, [Maps](#)
- deleting key-value pairs, [Deleting from Maps](#)
- for statement iterating over, [Iterating over maps](#)
- hash map implementation, [Maps](#), [Iterating over maps](#)
- keys as any comparable type, [Maps](#)
- len function for number of key-value pairs, [Maps](#)
 - clear function to set length to zero, [Emptying a Map](#)
- not comparable, [Maps](#)
 - maps.Equal, [Comparing Maps](#)
 - maps.EqualFunc, [Comparing Maps](#)
- reading and writing a map, [Reading and Writing a Map](#)
 - 0 for value for key not in map, [Reading and Writing a Map](#)
 - determining if key not in map, [The comma ok Idiom](#)
- slices versus maps, [Maps](#)
- using as sets, [Using Maps as Sets](#)
 - struct{} for value, [Using Maps as Sets](#)
- zero value as nil, [Maps](#)

- empty versus nil map literals, [Maps](#)
- writing to nil map causing panic, [Maps](#)

映射，映射 - 将映射用作集合，按值调用语言，Go 语言是按值调用，映射用于输入参数或返回值，映射和切片的区别，映射的指针实现，Go 语言是按值调用，映射和切片的区别，对于不在映射中的键，逗号 ok 惯用法，逗号 ok 惯用法声明，用于创建可增长大小映射的 Map 的 make 函数，映射，删除键值对，从映射中删除，for 语句迭代，迭代映射，哈希映射实现，映射，迭代映射，将键作为任何可比较类型，映射，用于获取键值对数量的 len 函数，用于将长度设置为零的 clear 函数，清空映射，不可比较，映射，maps.Equal，比较映射，maps.Equal 函数，比较映射，读取和写入映射，读取和写入映射，对于不在映射中的键，值为 0，读取和写入映射，判断键是否在映射中，逗号 ok 惯用法惯用法切片与映射，映射用作集合，使用映射作为集合，使用 struct{} 表示值，使用映射作为集合，将零值视为 nil，映射空值与 nil 映射字面量，向 nil 映射写入映射导致 panic，映射

- maps package

地图包

- maps.Equal, [Comparing Maps](#)
地图相等，比较地图
- maps.EqualFunc, [Comparing Maps](#)
maps.EqualFunc，比较地图

- marshaling

集结

- about, [encoding/json](#)

关于, 编码/json

- dash for name of ignored field, [Using Struct Tags to Add Metadata](#)

使用 Struct 标签添加元数据, 以破折号表示忽略字段的名称。

- reflection for data marshaler, [Use Reflection to Write a Data Marshaler-Use Reflection to Write a Data Marshaler](#)

使用反射编写数据序列化器

- Marti, Vicent, [Idiomatic Go and Generics](#)

Marti、Vicent、惯用 Go 和泛型

- mathematical operators (see arithmetic operators)

数学运算符 (参见算术运算符)

- mechanical sympathy, [Reducing the Garbage Collector's Workload](#)

机械同情心, 减轻垃圾收集员的工作量

- memory

记忆

- B, KiB, MiB, GiB, TiB suffixes, [Tuning the Garbage Collector](#)

B、KiB、MiB、GiB、TiB 后缀, 调整垃圾回收器

- garbage collector tuning, [Tuning the Garbage Collector-Tuning the Garbage Collector](#)

垃圾回收器调优, 垃圾回收器调优-垃圾回收器调优

- goroutine leak, [Always Clean Up Your Goroutines](#)
goroutine 泄漏, 请务必清理您的 goroutine。
- heap, [Reducing the Garbage Collector's Workload](#)
堆, 减少垃圾收集器的工作量
- heap versus stack information URL, [Reducing the Garbage Collector's Workload](#)
堆与栈信息 URL, 减少垃圾回收器的工作负载
- memory allocation via Go runtime, [Capacity](#)
通过 Go 运行时进行内存分配, 容量
- memory allocation via stack pointer value, [Reducing the Garbage Collector's Workload](#)
通过栈指针值进行内存分配, 降低垃圾回收器的工作负载
- memory limit environment variable, [Tuning the Garbage Collector](#)
内存限制环境变量, 调整垃圾回收器
- reducing garbage collector's workload, [Reducing the Garbage Collector's Workload-Reducing the Garbage Collector's Workload](#)
 - interface trade-off, [Accept Interfaces, Return Structs](#)

减少垃圾回收器的工作负载, 减少垃圾回收器的工作负载
——减少垃圾回收器的工作负载接口权衡, 接受接口, 返回结构体

- slices growing, [Capacity](#)
切片增长, 容量
- stack, [Reducing the Garbage Collector's Workload](#)
 - stack pointer, [Reducing the Garbage Collector's Workload](#)

堆栈, 减少垃圾回收器的工作负载; 堆栈指针, 减少垃圾回收器的工作负载

- thrashing, [Tuning the Garbage Collector](#)
猛烈抨击, 调整垃圾收集器
- variable storage, [A Quick Pointer Primer](#)
可变存储, 快速入门指南
- Mercurial for version control, [Publishing Your Module](#)
Mercurial 用于版本控制, 发布您的模块
- metadata
元数据
 - context
语境
 - about the problem, [The Context](#)
关于问题, 背景
 - cancellation, [Cancellation-Cancellation](#)
取消, 取消-取消
 - cancellation in your own code, [Context Cancellation in Your Own Code](#)
在您自己的代码中取消上下文取消

- contexts with deadlines, [Contexts with Deadlines-Contexts with Deadlines](#)

有截止日期的情境，有截止日期的情境-有截止日期的情境

- description, [What Is the Context?-What Is the Context?](#)

描述，背景是什么？-背景是什么？

- values, [Values-Values](#)

价值观，价值观-价值观

- struct tags for, [Using Struct Tags to Add Metadata](#)

使用结构体标签添加元数据

- method expression, [Methods Are Functions Too](#)

方法表达式，方法也是函数。

- method set, [Pointer Receivers and Value Receivers](#)

- interface implicit implementation, [Interfaces Are Type-Safe Duck Typing](#)

- methods on embedded field and containing struct, [Embedding Is Not Inheritance](#)

方法集、指针接收器和值接收器接口的隐式实现、接口是类型安全的鸭子类型、嵌入字段和包含结构上的方法、嵌入不是继承

- method value, [Methods Are Functions Too](#)

方法值，方法也是函数

- methods

方法

- about, [Methods](#)
关于, 方法
- adding to types, [io and Friends](#)
添加到类型、io 和朋友
- declaring, [Methods](#)
 - functions versus, [Methods](#)
 - receiver specification, [Methods](#)

声明、方法函数与方法接收器规范、方法

- functions replaced by methods, [Methods Are Functions Too](#)
 - when to use functions versus methods, [Functions Versus Methods](#)

函数被方法取代, 方法也是函数, 何时使用函数, 何时使用方法, 函数与方法的区别

- interface declaration, [A Quick Lesson on Interfaces](#)
接口声明, 接口速成课
- invoking, [Methods](#)
调用方法
- nil instance handling, [Code Your Methods for nil Instances](#)
空实例处理, 编写处理空实例的方法
- no inheritance from type based on type, [Type Declarations Aren't Inheritance](#)

- executable documentation, [Types Are Executable Documentation](#)

类型本身不具备继承性，类型声明并非可执行文档，类型本身就是可执行文档。

- no overloading, [Methods](#)
无过载，方法
- as parameters or variables, [Methods Are Functions Too](#)
作为参数或变量，方法也是函数。
- pointer receivers, [Pointer Receivers and Value Receivers-Pointer Receivers and Value Receivers](#)
 - nil instance handling, [Code Your Methods for nil Instances](#)

指针接收器、指针接收器和值接收器 - 指针接收器和值接收器 - 空实例处理、为空实例编写方法

- reflection cannot add to a type, [Reflection Can't Make Methods](#)
反射不能向类型添加内容，反射不能创建方法。
- struct field direct access encouraged, [Pointer Receivers and Value Receivers](#)
鼓励直接访问结构体字段，指针接收器和值接收器
- value receivers, [Pointer Receivers and Value Receivers-Pointer Receivers and Value Receivers](#)
 - nil instance handling, [Code Your Methods for nil Instances](#)

值接收器、指针接收器和值接收器 - 指针接收器和值接收器 -
空实例处理、为空实例编写方法

- Miara, Richard, [defer](#)
米娅拉, 理查德, 推迟
- MiB suffix, [Tuning the Garbage Collector](#)
MiB 后缀, 调整垃圾回收器
- middleware pattern of HTTP server, [Middleware-Middleware](#)
 - context
 - cancellation, [Cancellation-Cancellation](#)
 - cancellation in your own code, [Context Cancellation in Your Own Code](#)
 - contexts with deadlines, [Contexts with Deadlines-Contexts with Deadlines](#)
 - description, [What Is the Context?-What Is the Context?](#)
 - values, [Values-Values](#)

HTTP 服务器中间件模式、中间件-中间件上下文取消、取消-取消、自定义代码中的取消、自定义代码中的上下文取消、带截止时间的上下文、带截止时间的上下文-带截止时间的上下文描述、什么是上下文? -什么是上下文? 值、值-值

- module cache, [Importing Third-Party Code](#)
 - deleting, [Importing Third-Party Code](#)

模块缓存, 导入第三方代码删除, 导入第三方代码

- module path, [Repositories, Modules, and Packages](#)
 - tool to automate versioning, [Versioning Your Module](#)

模块路径、存储库、模块和包工具, 用于自动化版本控制, 模块版本控制

- modules

模块

- about, [Making a Go Module, Repositories, Modules, and Packages](#)

关于、创建 Go 模块、仓库、模块和包

- API renamed and reorganized, [Gracefully Renaming and Reorganizing Your API](#)

API 已重命名和重组, 优雅地重命名和重组您的 API

- code documented with Go Doc comments, [Documenting Your Code with Go Doc Comments-Documenting Your Code with Go Doc Comments](#)

使用 Go Doc 注释编写代码文档, 使用 Go Doc 注释编写代码文档 - 使用 Go Doc 注释编写代码文档

- creating a Go module, [Making a Go Module](#)

创建 Go 模块, 制作 Go 模块

- documentation URL, [Additional Details](#)

文档网址, 更多详情

- go.mod file, [Making a Go Module](#), [Using go.mod](#)
go.mod 文件, 创建 Go 模块, 使用 go.mod
- module path, [Repositories, Modules, and Packages](#)
 - tool to automate versioning, [Versioning Your Module](#)

模块路径、存储库、模块和包工具, 用于自动化版本控制,
模块版本控制

- open source Go module site, [Using pkg.go.dev](#)
开源 Go 模块网站, 使用 pkg.go.dev
- organization of, [Organizing Your Module](#)-[Organizing Your Module](#)
组织, 组织你的模块-组织你的模块
- proxy server, [Module Proxy Servers](#)
 - checksum database, [Module Proxy Servers](#)

代理服务器, 模块代理服务器校验和数据库, 模块代理服务
器

- publishing, [Publishing Your Module](#)
 - retracting versions, [Retracting a Version of Your Module](#)

发布、发布您的模块、撤回版本、撤回您的模块版本

- third-party code imported, [Importing Third-Party Code](#)
导入第三方代码, 导入第三方代码
- versions, [Working with Versions](#)

- dependencies replaced with specified fork, [Overriding Dependencies](#)
- dependency copies inside module, [Vendoring](#)
- excluding module versions, [Overriding Dependencies](#)
- go list to see versions, [Working with Versions](#)
- import compatibility rule, [Minimal Version Selection](#)
- minimal version selection, [Minimal Version Selection](#)
- resolving different version dependencies, [Minimal Version Selection](#)
- semantic import versioning rule, [Updating to Incompatible Versions](#)
- semantic versioning in Go, [Working with Versions](#)
- semantic versioning specifications URL, [Working with Versions](#)
- updating to compatible versions, [Updating to Compatible Versions](#)
- updating to incompatible versions, [Updating to Incompatible Versions](#), [Versioning Your Module](#)
- vendoring, [Vendoring](#)
- version control software, [Publishing Your Module](#)
- version control system documentation URL, [Publishing Your Module](#)
- versioning your module, [Versioning Your Module](#)

- workspaces for simultaneous modifications, [Using Workspaces to Modify Modules Simultaneously](#).-[Using Workspaces to Modify Modules Simultaneously](#).

版本、使用版本、将依赖项替换为指定的分支、覆盖依赖项、模块内部的依赖项副本、Vendoring（排除模块版本）、覆盖依赖项（查看版本列表）、使用版本、导入兼容性规则、最小版本选择、最小版本选择、解析不同的版本依赖项、最小版本选择、语义化导入版本控制规则、更新到不兼容版本、Go 中的语义化版本控制、使用版本、语义化版本控制规范 URL、使用版本、更新到兼容版本、更新到兼容版本、更新到不兼容版本、更新到不兼容版本、模块版本控制、Vendoring、版本控制软件、发布模块、版本控制系统文档 URL、发布模块、模块版本控制、模块版本控制、用于同时修改的工作区、使用工作区同时修改模块

- modulus (%) for integer types, [Integer operators](#)
 - not floating-point types, [Floating-point types](#)

整数类型的模数 (%), 整数运算符 (不适用于浮点类型), 浮点类型

- monotonic time, [Monotonic Time](#)
单调时间, 单调时间
- Musseman, Joyce, [defer](#)
穆斯曼, 乔伊斯, 推迟

- mutexes instead of channels in concurrency, [When to Use Mutexes Instead of Channels](#)
 - critical section, [When to Use Mutexes Instead of Channels](#)
 - passed via pointer, [When to Use Mutexes Instead of Channels](#)

并发中何时使用互斥锁代替通道？何时使用互斥锁代替通道？临界区何时使用互斥锁代替通道？通过指针传递时何时使用互斥锁代替通道？

N

N

- naming variables and constants, [Naming Variables and Constants](#)
 - camelCase instead of snake_case, [Naming Variables and Constants](#)
 - case of first letter in package-level declaration, [Naming Variables and Constants](#)
 - smaller the scope, shorter the name, [Naming Variables and Constants](#)

变量和常量命名，变量和常量命名：驼峰式命名法而非蛇形命名法，变量和常量命名：包级声明中首字母大小写一致，变量和常量命名：作用域越小，名称越短，变量和常量命名

- NaN (Not a Number) when dividing by zero, [Floating-point types](#)
除以零时返回 NaN（非数字），浮点类型
- Navarro, Juan, [defer](#)
纳瓦罗，胡安，推迟
- nested code harder to read, [defer](#)
嵌套代码更难阅读，延迟执行
- net/http package, [net/http-ResponseController](#)
 - Client type, [The Client-ResponseController](#)
 - context
 - about metadata problem, [The Context](#)
 - http.DefaultServeMux and functions avoided, [The Server](#)
 - http.Flusher interface, [ResponseController](#)
 - http.Handler interface, [The Server](#)
 - http.Hijacker interface, [ResponseController](#)
 - http.NewRequestWithContext, [The Client](#)
 - http.NewServeMux, [The Server](#)
 - http.Request context-related methods, [What Is the Context?](#)
 - http.ResponseController, [ResponseController](#)
 - *http.ServeMux, [The Server](#)
 - http.Server, [The Server](#)
 - middleware pattern, [Middleware-Middleware](#)
 - third-party modules to enhance, [Use third-party modules to enhance the server](#)
 - testing via httptest, [Using httptest-Using httptest](#)

net/http 包, net/http-ResponseControllerClient 类型, 客户端-响应
控制器上下文元数据问题, 避免使用 Contexthttp.DefaultServeMux
及其函数, Serverhttp.Flusher 接口,
ResponseControllerhttp.Handler 接口, Serverhttp.Hijacker 接口,
ResponseControllerhttp.NewRequestWithContext,
Clienthttp.NewServeMux, Serverhttp.Request 上下文相关方法, 什
么是 Context? http.ResponseController,
ResponseController*http.ServeMux, Serverhttp.Server, 服务器中间
件模式, 中间件-中间件第三方模块增强, 使用第三方模块通过
httptest 增强服务器测试, 使用 httptest-使用 httptest

- new function for pointer variable creation rarely used, [A Quick Pointer Primer](#)

指针变量创建的新函数很少用到, 《指针快速入门》

- newline rune literal ('\n'), [Literals](#)
换行符字面量 ('\n') , 字面量

- nil

零

- about, [Slices](#), [A Quick Pointer Primer](#)
关于切片, 快速入门指南
- defined in universe block, [A Quick Pointer Primer](#)
 - can be shadowed, [A Quick Pointer Primer](#)

在宇宙块中定义, 《快速指针入门》可以被遮蔽, 《快速指

针入门》

- error return value when no error, [Multiple Return Values](#), [How to Handle Errors: The Basics](#)

当没有错误时返回错误值，多个返回值，如何处理错误：基础知识

- function variable zero value, [Functions Are Values](#)
函数变量值为零，函数即值
- interfaces and, [Interfaces and nil](#)
 - reflection checking for nil value, [Use Reflection to Check If an Interface's Value Is nil](#)

接口和 nil 反射检查 nil 值，使用反射检查接口的值是否为 nil

- methods coded for nil instances, [Code Your Methods for nil Instances](#)

为空实例编写的方法，请为空实例编写方法

- zero value

零值

- channels, [Channels](#)
频道，频道
- maps, [Maps](#)
地图，地图
- pointers, [A Quick Pointer Primer](#)
指针，指针快速入门

- slices, [Slices](#)
切片, 切片
- numeric types, [Numeric Types-Complex types \(you're probably not going to use these\)](#)
 - complex types, [Complex types \(you're probably not going to use these\)-Complex types \(you're probably not going to use these\)](#)
 - explicit type conversion, [Explicit Type Conversion](#)
 - floating-point types, [Floating-point types](#)
 - integer types, [Integer types-Integer operators](#)
 - numerical computing in Go, [Complex types \(you're probably not going to use these\)](#)
 - decimal module by ShopSpring, [Importing Third-Party Code](#)
 - Gonum third-party package, [Complex types \(you're probably not going to use these\)](#)

数值类型、数值类型 - 复杂类型（您可能不会用到这些）复杂类型、复杂类型（您可能不会用到这些） - 复杂类型（您可能不会用到这些）显式类型转换、显式类型转换浮点类型、浮点类型整数类型、整数类型 - 整数运算符 Go 中的数值计算、复杂类型（您可能不会用到这些）ShopSpring 的 decimal 模块、导入第三方代码 Gonum 第三方包、复杂类型（您可能不会用到这些）

O

哦

- octal prefix (0o), [Literals](#)
 - 0 alone should not be used, [Literals](#)
 - uses for octal, [Literals](#)

八进制前缀 (0o), Literals0 不能单独使用, Literalsuses 用于八进制, Literals

- online resources (see resources online)
在线资源 (参见在线资源)
- open source Go module site, [Using pkg.go.dev](#)
开源 Go 模块网站, 使用 pkg.go.dev
- operator generic functions via type terms, [Use Type Terms to Specify Operators-Use Type Terms to Specify Operators](#)
通过类型项实现运算符泛型函数, 使用类型项指定运算符 - 使用类型项指定运算符
- operator overloading not part of Go, [Preface](#), [Things That Are Left Out](#)
运算符重载并非 Go 语言的一部分, 前言, 遗漏的内容
- os package
操作系统软件包
 - os.Args, [defer](#)
os.Args, 延迟

- os.Create, [io and Friends](#)
os.Create、io 和朋友们
- *os.File instance returned, [io and Friends](#)
返回 *os.File 实例, io 和 Friends
- os.File with io.Reader interface, [Interfaces Are Type-Safe Duck Typing](#)
os.File 与 io.Reader 接口, 接口是类型安全的鸭子类型
- os.NewFile, [io and Friends](#)
os.NewFile、io 和 Friends
- os.Open, [defer](#), [Interfaces Are Type-Safe Duck Typing](#), [io and Friends](#)
os.Open、defer、接口是类型安全的鸭子类型、io 和朋友们
- os.OpenFile, [io and Friends](#)
os.OpenFile、io 和相关函数
- os.ReadFile, [io and Friends](#)
os.ReadFile、io 和相关函数
- os.WriteFile, [io and Friends](#)
os.WriteFile、io 和相关函数

P

P

- p for exponent in floating-point literals, [Literals](#)
p 代表浮点字面量中的指数, 字面量

- package blocks, [var Versus :=](#), [Blocks](#)
 - anonymous functions, [Anonymous Functions](#)
 - descriptive variable and constant names, [Naming Variables and Constants](#)
 - method definitions, [Methods](#)
 - sentinel errors declaration, [Sentinel Errors](#)

包块、变量与 :=、块、匿名函数、匿名函数、描述性变量和常量名称、命名变量和常量、方法定义、方法、哨兵错误声明、哨兵错误

- package clause, [Creating and Accessing a Package](#)
 - identical for every Go file in a directory, [Creating and Accessing a Package](#)

package 子句，创建和访问包（目录中每个 Go 文件都相同），创建和访问包

- package keyword, [Creating and Accessing a Package](#)
package 关键字，创建和访问包
- packages, [Repositories, Modules, and Packages](#)
 - building
 - circular dependencies, [Avoiding Circular Dependencies](#)
 - comments in Go Doc format, [Documenting Your Code with Go Doc Comments-Documenting Your Code with Go Doc Comments](#)

- creating and accessing, [Creating and Accessing a Package](#)
- import path, [Creating and Accessing a Package](#)
- imports and exports, [Importing and Exporting](#)
- init function avoided, [Avoiding the init Function if Possible](#)
- internal package, [Using the internal Package](#)
- naming packages, [Naming Packages](#)
- overriding a package's name, [Overriding a Package's Name](#)
- package clause, [Creating and Accessing a Package](#)
- package main, [Creating and Accessing a Package](#), [Creating and Accessing a Package](#)
- package name matching directory containing it, [Creating and Accessing a Package](#)
- renaming and reorganizing API, [Gracefully Renaming and Reorganizing Your API](#)
- case of first letter in package-level declaration, [Naming Variables and Constants](#)
- declaration, [go build](#)
- import statement, [go build](#)
 - shadowing variables, [Shadowing Variables](#)
- main package, [go build](#)
 - module organization, [Organizing Your Module](#)
- module organization, [Organizing Your Module-Organizing Your Module](#)

包、仓库、模块和包构建循环依赖、避免循环依赖、Go Doc 格式的注释、使用 Go Doc 注释记录代码、创建和访问包、导入路径、导入和导出、避免使用 init 函数、尽可能避免使用 init 函数、内部包、使用内部包、命名包、覆盖包的名称、包子句、创建和访问包、主包、创建和访问包、包名与包含它的目录匹配、重命名和重组 API、优雅地重命名和重组 API、包级声明中首字母大小写、命名变量和常量、声明、go build、导入语句、go build、变量遮蔽、主包、go build、模块组织、组织模块、组织模块模块 - 组织您的模块

- panics

恐慌

- about panics, [panic and recover-panic and recover](#)
 - recover, [panic and recover](#)
 - stack trace, [panic and recover](#)

关于恐慌、恐慌和恢复-恐慌和恢复恢复、恐慌和恢复堆栈跟踪、恐慌和恢复

- array access out of bounds, [Arrays—Too Rigid to Use Directly](#)
数组越界访问，数组——过于死板而无法直接使用
- channel behavior chart, [Understanding How Channels Behave](#)
渠道行为图，了解渠道行为
- comparing uncomparable types, [More on comparable](#)
比较不可比较的类型，更多关于可比较的内容

- dereferencing a nil pointer, [A Quick Pointer Primer](#)
如何解引用空指针：指针快速入门
- dividing by zero, [Integer operators](#)
除以零，整数运算符
- function variable of nil value running, [Functions Are Values](#)
函数变量值为 nil 时正在运行，函数即值
- map of nil value written to, [Maps](#)
写入空值的映射，映射
- slice capacity less than length, [make](#)
切片容量小于长度，制作
- type assertion wrong, [Type Assertions and Type Switches](#)
类型断言错误，类型断言和类型切换
- value receiver method called with nil value pointer instance,
[Pointer Receivers and Value Receivers](#)
值接收器方法被调用，传入的值为 nil 的指针实例，指针接收器和值接收器
- parameters
参数
 - call by value language, [Go Is Call by Value-Go Is Call by Value](#),
[Pointers Indicate Mutable Parameters](#)
 - pointer passed to a function, [Pointers Indicate Mutable Parameters-Pointers Indicate Mutable Parameters](#)

按值调用语言，Go 语言是按值调用——Go 语言是按值调

用，指针指示可变参数，传递给函数的指针，指针指示可变参数——指针指示可变参数

- call-by-value language, [append](#)
按值调用语言，追加
- context as first, [What Is the Context?](#)
首先是背景，什么是背景？
- functions declared and called, [Declaring and Calling Functions](#)
 - emulating named and optional parameters, [Simulating Named and Optional Parameters](#)
 - input parameters must be supplied, [Simulating Named and Optional Parameters](#)
 - no input parameters, [Declaring and Calling Functions](#)
 - variadic input parameters, [Variadic Input Parameters and Slices](#)

函数的声明和调用，声明和调用函数，模拟命名参数和可选参数，模拟命名参数和可选参数（必须提供输入参数），模拟命名参数和可选参数（无输入参数），声明和调用函数，可变参数输入，可变参数输入和切片

- goroutine invocation, [Goroutines](#)
goroutine 调用，Goroutines
- input parameters not modified, [append](#), [Go Is Call by Value-Go Is Call by Value](#), [Pointers Indicate Mutable Parameters](#)

- maps for input parameters, [The Difference Between Maps and Slices](#)
- pointer passed to a function, [Pointers Indicate Mutable Parameters-Pointers Indicate Mutable Parameters](#)
- slices for input parameters, [The Difference Between Maps and Slices](#)

输入参数未修改，追加，Go 是按值调用 - Go 是按值调用，指针指示可变参数，用于输入参数的映射，映射和切片的区别，传递给函数的指针，指针指示可变参数 - 指针指示可变参数，用于输入参数的切片，映射和切片的区别

- method passed to, [Methods Are Functions Too](#)
传递给方法，方法也是函数
- passing functions as parameters, [Passing Functions as Parameters](#)
 - when to use interface instead, [Function Types Are a Bridge to Interfaces](#)

将函数作为参数传递，何时应该使用接口，函数类型是连接接口的桥梁

- pointers indicating mutable parameters, [Pointers Indicate Mutable Parameters-Pointers Indicate Mutable Parameters](#), [Pointer Receivers and Value Receivers](#)
 - pointers improving performance, [Pointer Passing Performance](#)

- pointers used as last resort, [Pointers Are a Last Resort](#)

指针指示可变参数，指针指示可变参数——指针指示可变参数，指针接收器和值接收器，指针提升性能，指针传递性能，指针作为最后的手段，指针是最后的手段

- personally identifiable information on Go Playground, [The Go Playground](#)

Go Playground 上的个人身份信息，Go Playground

- Pike, Rob, [Strings and Runes and Bytes](#)

派克、罗布、弦、符文和字节

- pkg.go.dev open source Go module site, [Using pkg.go.dev](#)
 - pkgsite powering, [Documenting Your Code with Go Doc Comments](#)

pkg.go.dev 开源 Go 模块网站，使用 pkg.go.dev (pkgsite) 提供支持，使用 Go Doc 注释记录您的代码

- pkgsite tool

pkgsite 工具

- documentation formatting viewer, [Documenting Your Code with Go Doc Comments](#)

文档格式查看器，使用 Go Doc 注释记录您的代码

- pkg.go.dev powered by, [Documenting Your Code with Go Doc Comments](#)

pkg.go.dev 由 Go Doc 注释提供支持，用于记录您的代码。

- pointer receivers, [Pointer Receivers and Value Receivers-Pointer Receivers and Value Receivers](#)
 - nil instance handling, [Code Your Methods for nil Instances](#)

指针接收器、指针接收器和值接收器 - 指针接收器和值接收器 - 空实例处理、为空实例编写方法

- pointer type, [A Quick Pointer Primer](#)
 - pointer receivers, [Pointer Receivers and Value Receivers-Pointer Receivers and Value Receivers](#)
 - methods with value receiver methods in method set, [Pointer Receivers and Value Receivers](#)

指针类型、指针快速入门、指针接收器、指针接收器和值接收器、方法集中带有值接收器方法的指针接收器和值接收器

- pointers, [A Quick Pointer Primer](#)
 - address pointing to, [A Quick Pointer Primer](#)
 - call by value behavior of maps and slices, [Go Is Call by Value](#)
 - channels passed to functions as, [Channels](#)
 - creating
 - & before struct literal, [A Quick Pointer Primer](#)
 - & before struct literal exception, [A Quick Pointer Primer](#)
 - new function rarely used, [A Quick Pointer Primer](#)
 - numbers, booleans, strings, constants need variable, [A Quick Pointer Primer](#)

- dereferencing, [A Quick Pointer Primer](#)
 - non-nil pointers only, [A Quick Pointer Primer](#)
- indicating mutable parameters, [Pointers Indicate Mutable Parameters-Pointers Indicate Mutable Parameters](#), [Pointer Receivers and Value Receivers](#)
 - pointers improving performance, [Pointer Passing Performance](#)
 - pointers used as last resort, [Pointers Are a Last Resort](#), [Reducing the Garbage Collector's Workload](#)
- pointer passed to a function, [Pointers Indicate Mutable Parameters-Pointers Indicate Mutable Parameters](#)
- pointer type, [A Quick Pointer Primer](#)
 - pointer receivers, [Pointer Receivers and Value Receivers-Pointer Receivers and Value Receivers](#)
- syntax
 - & as address operator, [A Quick Pointer Primer](#)
 - * as indirection operator, [A Quick Pointer Primer](#)
- used as last resort, [Pointers Are a Last Resort](#)
 - but performance improvements, [Pointer Passing Performance](#)
- using values versus pointers, [Don't Fear the Pointers](#)
 - lack of immutable declarations and, [Pointers Indicate Mutable Parameters](#)
- zero value as nil, [A Quick Pointer Primer](#)

- zero value versus no value indication, [The Zero Value Versus No Value](#)

指针, 指针快速入门; 指向的地址, 指针快速入门; 映射和切片的按值调用行为, Go 语言的按值调用; 传递给函数的通道, 通道; 在结构体字面量之前创建 & 符号, 指针快速入门; 在结构体字面量之前创建 & 符号的异常, 指针快速入门; 很少使用 new 函数, 指针快速入门; 数字、布尔值、字符串、常量需要变量, 指针快速入门; 解引用, 指针快速入门; 仅非空指针, 指针快速入门; 指示可变参数, 指针指示可变参数 - 指针指示可变参数; 指针接收器和值接收器; 指针提高性能, 指针传递性能; 指针作为最后的手段, 指针是最后的手段; 减少垃圾回收器的工作负载; 传递给函数的指针, 指针指示可变参数 - 指针指示可变参数; 指针类型, 指针快速入门; 指针接收器, 指针接收器值接收器 - 指针接收器和值接收器语法 & 作为地址运算符, 指针快速入门* 作为间接运算符, 指针快速入门* 作为最后的手段, 指针是最后的手段, 但性能改进, 指针传递性能 使用值与指针, 不要害怕指针 缺少不可变声明, 指针指示可变参数 零值表示为 nil, 指针快速入门* 零值与无值指示, 零值与无值

- pprof code profiler, [Using Benchmarks](#)
pprof 代码分析器, 使用基准测试
- pre-releases in semantic versioning, [Versioning Your Module](#)
语义化版本控制中的预发布版本, 模块版本控制

- predeclared types

预先声明的类型

- about, [The Predeclared Types](#)
关于预先声明的类型
- booleans, [Booleans](#)
布尔值
- complex types, [Complex types \(you're probably not going to use these\)](#)-[Complex types \(you're probably not going to use these\)](#)
复杂类型, 复杂类型 (您可能用不到这些) -复杂类型 (您可能用不到这些)
- explicit type conversion, [Explicit Type Conversion](#)
显式类型转换, 显式类型转换
- floating-point types, [Floating-point types](#)
浮点类型, 浮点类型
- keywords versus, [Shadowing Variables](#)
关键词与阴影变量
- numeric, [Numeric Types-Complex types \(you're probably not going to use these\)](#)
数值型、数值类型 - 复杂类型 (您可能不会用到这些)
- “Preemptive Interface Anti-Pattern in Go” (Lindamood), [Accept Interfaces, Return Structs](#)
“Go 语言中的抢占式接口反模式” (Lindamood) , 接受接口, 返回结构体
- Printf (fmt package), [go vet](#)

- stack trace via verbose output, [Getting a Stack Trace from an Error](#)

Printf (fmt 包) , 通过详细输出获取堆栈跟踪, 从错误中获取堆栈跟踪

- Println (fmt package), [go build](#)
 - passing error to, [Use Strings for Simple Errors](#)

Printf (fmt 包) , go build 传递错误, 使用字符串表示简单错误

- private repositories, [Using Private Repositories](#)
私有仓库, 使用私有仓库
- promotion and composition, [Use Embedding for Composition](#)
推广和组合, 使用嵌入进行组合
- Protocol Buffers (protobufs), [Using go generate](#)
 - schema, [Using go generate](#)

Protocol Buffers (protobufs), 使用 go generated schema, 使用 go generate

- proxy server for modules, [Module Proxy Servers](#)
 - checksum database, [Module Proxy Servers](#)
 - disabling via GOPROXY, [Specifying a Proxy Server](#)
 - running own proxy server, [Specifying a Proxy Server](#)

模块代理服务器、模块代理服务器校验和数据库、通过

GOPROXY 禁用模块代理服务器、指定代理服务器、运行自己的代理服务器、指定代理服务器

- publishing your module, [Publishing Your Module](#)
 - LICENSE file, [Publishing Your Module](#)
 - retracting versions, [Retracting a Version of Your Module](#)

发布模块、发布模块许可证文件、发布模块、撤回版本、撤回模块的某个版本

R

拉

- race checker, [Finding Concurrency Problems with the Data Race Detector](#)
竞争检测器，使用数据竞争检测器查找并发问题
- range keyword, [The for-range Statement-The for-range value is a copy.](#)
range 关键字，for-range 语句——for-range 的值是副本。
- Read method of file, [defer](#)
文件读取方法，延迟
- real function, [Complex types \(you're probably not going to use these\).](#)
实际函数，复杂类型（你可能不会用到这些）
- receiver specification of methods, [Methods](#)
方法的接收者规范，方法

- reflect package, [Here Be Dragons: Reflect, Unsafe, and Cgo](#),
[Reflection Lets You Work with Types at Runtime](#)
 - reflect.DeepEqual, [Slices](#), [Reflection Lets You Work with Types at Runtime](#)
 - reflect.New, [Make New Values](#)
 - reflect.Type, [Make New Values](#)
 - reflect.TypeOf, [Types and kinds](#)-[Types and kinds](#)
 - reflect.Value, [Make New Values](#)
 - reflect.ValueOf, [Values](#)
 - reflection
 - about, [Reflection Lets You Work with Types at Runtime](#)
 - automating repetitive tasks, [Build Functions with Reflection to Automate Repetitive Tasks](#)
 - avoid building structs with, [You Can Build Structs with Reflection, but Don't](#)
 - cannot add methods to a type, [Reflection Can't Make Methods](#)
 - checking for nil interface value, [Use Reflection to Check If an Interface's Value Is nil](#)
 - data marshaler, [Use Reflection to Write a Data Marshaler](#)-
[Use Reflection to Write a Data Marshaler](#)
 - making new values, [Make New Values](#)
 - types and kinds, [Types and kinds](#)-[Types and kinds](#)

- use only if worthwhile, [Use Reflection Only if It's Worthwhile](#)
- values, [Values](#)

reflect 包, 《这里有龙: 反射、Unsafe 和 Cgo》, 反射允许您在运行时处理类型; reflect.DeepEqual, 切片; 反射允许您在运行时处理类型; reflect.New, 创建新值; reflect.Type, 创建新值; reflect.TypeOf, 类型和种类; reflect.Value, 创建新值; reflect.ValueOf, 值; 关于反射, 反射允许您在运行时处理类型; 自动化重复性任务, 使用反射构造函数以自动化重复性任务; 避免使用反射构建结构体, 您可以使用反射构建结构体, 但不要这样做; 无法向类型添加方法, 反射无法创建方法; 检查接口值是否为 nil, 使用反射检查接口的值是否为 nil; 数据封送器, 使用反射编写数据封送器; 创建新值, 创建新值; 类型和种类, 类型和种类; 仅使用如果值得, 请仅在值得时使用反射。

- relative path import paths not working with modules, [Creating and Accessing a Package](#)
相对路径导入路径在模块中不起作用, 创建和访问包
- repetitive code reduced via generic functions, [Generics Reduce Repetitive Code and Increase Type Safety](#)-[Generics Reduce Repetitive Code and Increase Type Safety](#)
通过泛型函数减少重复代码, 泛型减少重复代码并提高类型安全性
- replace directive, [Overriding Dependencies](#)

- workspaces for simultaneous module modifications, [Using Workspaces to Modify Modules Simultaneously](#)

替换指令，覆盖依赖项工作区以实现模块的同步修改，使用工作区同步修改模块

- repositories, [Repositories, Modules, and Packages](#)
 - GitHub
 - Git needed for downloading, [Adding Third-Party Tools with go install](#)
 - pushing workspace files to, [Using Workspaces to Modify Modules Simultaneously](#)
 - module path, [Repositories, Modules, and Packages](#)
 - nonunique module names, [Repositories, Modules, and Packages](#)
 - private repositories, [Using Private Repositories](#)
 - proxy server for modules, [Module Proxy Servers](#)
 - publishing your module, [Publishing Your Module](#)
 - version control software, [Publishing Your Module](#)
 - Git needed for GitHub downloads, [Adding Third-Party Tools with go install](#)

仓库、仓库、模块和包；GitHub 需要 Git 进行下载；使用 go install 添加第三方工具；推送工作区文件；使用工作区同时修改模块；模块路径、仓库、模块和包；非唯一模块名称、仓库、模块和包；私有仓库；使用私有仓库；模块代理服务器；发布模

块；发布模块；版本控制软件；发布模块；GitHub 下载需要 Git；使用 go install 添加第三方工具

- require directives, [The require Directive](#)
 - dependencies, [The require Directive](#), [Importing Third-Party Code](#)
 - imports marked as indirect, [Importing Third-Party Code](#)

require 指令、require 指令依赖项、require 指令、导入第三方代码（标记为间接导入）、导入第三方代码

- resources online

在线资源

- arithmetic operators, [Integer operators](#)
算术运算符，整数运算符
- book supplemental material, [Using Code Examples](#)
 - Chapter 1, [Makefiles-Exercises](#)
 - Chapter 2, [Complex types \(you're probably not going to use these\)](#), [Explicit Type Conversion](#), [Using const](#), [Unused Variables-Exercises](#)
 - Chapter 3, [Capacity](#), [Slicing Slices-Converting Arrays to Slices](#), [Converting Slices to Arrays-Strings and Runes and Bytes](#), [Reading and Writing a Map](#), [Using Maps as Sets](#), [Exercises](#)
 - Chapter 4, [Shadowing Variables-if](#), [The Infinite for Statement](#), [The for-range Statement-Iterating over maps](#),

[Iterating over strings](#), [switch](#), [switch-switch](#), [Choosing Between if and switch-goto—Yes, goto](#), [Exercises](#)

- Chapter 5, [Declaring and Calling Functions-Simulating Named and Optional Parameters](#), [Multiple Return Values](#), [Named Return Values](#), [Functions Are Values](#), [Anonymous Functions-Anonymous Functions](#), [Passing Functions as Parameters-defer, defer, defer-Exercises](#)
- Chapter 6, [Don't Fear the Pointers](#), [Pointer Passing Performance](#), [Exercises](#)
- Chapter 7, [Pointer Receivers and Value Receivers](#), [Code Your Methods for nil Instances](#), [Embedding Is Not Inheritance](#), [A Quick Lesson on Interfaces](#), [Interfaces and nil](#), [Interfaces Are Comparable](#), [Type Assertions and Type Switches](#), [Use Type Assertions and Type Switches Sparingly](#), [Implicit Interfaces Make Dependency Injection Easier-Exercises](#)
- Chapter 8, [Generics Reduce Repetitive Code and Increase Type Safety](#), [Introducing Generics in Go](#), [Generic Functions Abstract Algorithms](#), [Generics and Interfaces-Type Inference and Generics](#), [Combining Generic Functions with Generic Data Structures-More on comparable](#), [Idiomatic Go and Generics-Exercises](#)
- Chapter 9, [How to Handle Errors: The Basics-Sentinel Errors](#), [Errors Are Values-Wrapping Errors](#), [Wrapping](#)

[Multiple Errors-Is and As](#), [Is and As](#), [panic and recover-panic and recover](#), [Exercises](#)

- Chapter 10, [Use the go Directive to Manage Go Build Versions](#), [Overriding a Package's Name](#), [Avoiding Circular Dependencies](#)
- Chapter 11, [Using go run to Try Out Small Programs](#), [staticcheck-golangci-lint](#), [Embedding Content into Your Program](#), [Embedding Content into Your Program-Embedding Hidden Files](#), [Using go generate-Working with go generate and Makefiles](#), [Exercises](#)
- Chapter 12, [Goroutines](#), [select-select](#), [Use the Context to Terminate Goroutines](#)-[Put Your Concurrent Tools Together](#), [When to Use Mutexes Instead of Channels](#), [Exercises](#)
- Chapter 13, [io and Friends](#), [JSON](#), [Readers](#), [and Writers](#)-[Custom JSON Parsing](#), [Custom JSON Parsing](#), [The Client-Middleware](#), [ResponseController-Exercises](#)
- Chapter 14, [What Is the Context?](#), [Values](#), [Values](#), [Cancellation](#), [Cancellation](#), [Contexts with Deadlines](#)-[Contexts with Deadlines](#), [Context Cancellation in Your Own Code](#)
- Chapter 15, [Understanding the Basics of Testing](#), [Storing Sample Test Data](#)-[Using go-cmp to Compare Test Results](#), [Running Table Tests](#), [Running Tests Concurrently](#)-[Checking Your Code Coverage](#), [Using Benchmarks](#), [Using Stubs in](#)

Go, Using Stubs in Go, Using httptest-Finding Concurrency Problems with the Data Race Detector

- Chapter 16, Types and kinds, Make New Values-Use Reflection to Write a Data Marshaler, Build Functions with Reflection to Automate Repetitive Tasks, Use Reflection Only if It's Worthwhile, Using Sizeof and Offsetof-Using unsafe to Convert External Binary Data, Accessing Unexported Fields-Cgo Is for Integration, Not Performance, Exercises
- goroutine_for_loop repository, Goroutines, for Loops, and Varying Variables
- money repository, Importing Third-Party Code
- package examples, Creating and Accessing a Package

书籍补充材料，使用代码示例第 1 章，Makefiles 练习第 2 章，复杂类型（您可能用不到这些），显式类型转换，使用 const，未使用的变量练习第 3 章，容量，切片 - 将数组转换为切片，将切片转换为数组 - 字符串、符文和字节，读取和写入映射，将映射用作集合，练习第 4 章，变量遮蔽 - if，无限循环 for 语句，for-range 语句 - 遍历映射，遍历字符串，switch，switch-switch，if 和 switch 之间的选择 - goto - 是的，goto，练习第 5 章，声明和调用函数 - 模拟命名参数和可选参数，多个返回值，命名返回值，函数即值，匿名函数 - 匿名函数，将函数作为参数传递 - defer defer，defer-练习 第

6章, 不要害怕指针, 指针传递性能, 练习 第7章, 指针接收器和值接收器, 为nil实例编写方法, 嵌入不是继承, 接口速览, 接口和nil, 接口是可比较的, 类型断言和类型切换, 谨慎使用类型断言和类型切换, 隐式接口使依赖注入更容易-练习 第8章, 泛型减少重复代码并提高类型安全性, Go语言中的泛型介绍, 泛型函数抽象算法, 泛型和接口-类型推断和泛型, 将泛型函数与泛型数据结构结合使用-更多关于可比较的内容, 惯用的Go语言和泛型-练习 第9章, 如何处理错误: 基础知识-哨兵错误, 错误是值 - 错误包装、多个错误包装 - Is 和 As、Is 和 As、panic 和 recover、练习 第 10 章、使用 go 指令管理 Go 构建版本、覆盖包名称、避免循环依赖 第 11 章、使用 go run 试运行小程序、staticcheck-golangci-lint、将内容嵌入程序、将内容嵌入程序 - 嵌入隐藏文件、使用 go generate - 使用 go generate 和 Makefile、练习 第 12 章、Goroutine、select-select、使用 Context 终止 Goroutine - 整合并发工具、何时使用互斥锁而不是通道、练习 第 13 章、io 及其他、JSON、读取器和写入器 - 自定义 JSON 解析、自定义 JSON 解析、客户端中间件、ResponseController - 练习 第 14 章、什么是上下文? , 值, 值, 取消, 取消, 带截止时间的上下文, 在您自己的代码中取消上下文第15章, 理解测试基础, 存储示例测试数据-使用go-cmp比较测试结果, 运行表测试, 并发运行测试-检查代码覆盖率, 使用基准测试, 在Go中使用桩, 在Go中使用桩, 使用httptest-使用数据竞争检测器查找并发问题第16章, 类型和种类, 创建新值-使用反射

编写数据编组器，使用反射构造函数以自动化重复性任务，
仅在值得时使用反射，使用sizeof和offsetof-使用unsafe转换外部二进制数据，访问未导出的字段-Cgo用于集成，而不是性能，练习goroutine_for_loop存储库，Goroutine，for循环和Varying Variablesmoney 存储库、导入第三方代码包示例、创建和访问包

- containers with Go app image, [Installing the Go Tools](#)
使用 Go 应用镜像的容器，安装 Go 工具
- data race detector documentation, [Finding Concurrency Problems with the Data Race Detector](#)
数据竞争检测器文档，使用数据竞争检测器查找并发问题
- Effective Go
有效
 - idiomatic Go resource, [go vet](#)
惯用语：去资源，去兽医
 - semicolon insertion rule, [go fmt](#)
分号插入规则， go fmt
- errors package documentation, [Getting a Stack Trace from an Error](#)
errors 包文档，从错误中获取堆栈跟踪
- The Floating Point Guide, [Floating-point types](#)
 - comparing floats, [Floating-point types](#)

浮点数指南、浮点类型、浮点数比较、浮点类型

- garbage collector algorithm implementation details, [Reducing the Garbage Collector's Workload](#)

垃圾回收算法实现细节，降低垃圾回收器的工作负载

- generic functions

通用函数

- Type Parameters Proposal, [Introducing Generics in Go](#)
类型参数提案，在 Go 语言中引入泛型
- why not initially included, [Introducing Generics in Go](#)
为什么最初没有包含？在 Go 语言中引入泛型。
- Go Compatibility Promise, [The Go Compatibility Promise](#)
 - Russ Cox GopherConn 2022 keynote talk, [The Go Compatibility Promise](#)

Go 兼容性承诺，Go 兼容性承诺，Russ Cox 在 GopherConn 2022 上的主题演讲，Go 兼容性承诺

- Go development tools, [Choose Your Tools](#)

Go 开发工具，选择你的工具

- Go Doc comments documentation, [Documenting Your Code with Go Doc Comments](#)

Go Doc 注释文档，使用 Go Doc 注释编写代码文档

- Go Programming Language Specification, [The Zero Value](#)

Go 编程语言规范，零值

- Go runtime

Go 运行时环境

- performance and generics, [Idiomatic Go and Generics](#)
性能和泛型，惯用的 Go 语言和泛型
- scheduler information, [Goroutines](#)
调度器信息，Goroutines
- Go tools on Go website, [Installing the Go Tools](#)
Go 网站上的 Go 工具，安装 Go 工具
- Go wiki Code Review Comments page, [go vet](#)
前往 wiki 代码审查评论页面，前往 vet
- Google security for Playground content, [The Go Playground](#)
Google Playground 内容安全保障，Go Playground
- “A Guide to the Go Garbage Collector”, [Tuning the Garbage Collector](#)
《Go垃圾收集器使用指南》，垃圾收集器调优
- heap versus stack and escape analysis, [Reducing the Garbage Collector’s Workload](#)
堆与栈的比较及逃逸分析，降低垃圾回收器的工作负载
- indentation of code and readability, [defer](#)
代码缩进和可读性，延迟
- “Inside the Map Implementation” video, [Maps](#)
“地图实现内部原理”视频，地图
- language server protocols, [Visual Studio Code](#)
语言服务器协议，Visual Studio Code
- log/slog package documentation, [Structured Logging](#)
log/slog 包文档，结构化日志记录

- Makefile tutorial, [Makefiles](#)
Makefile教程, Makefile文件
- memory efficiency blog post, [Reducing the Garbage Collector's Workload](#)
内存效率博客文章: 减少垃圾回收器的工作负载
- mocks covered in blog post, [Using Stubs in Go](#)
博客文章“在 Go 中使用 Stubs”介绍了模拟对象。
- module documentation, [Additional Details](#)
模块文档, 更多详情
- module organization, [Organizing Your Module](#)
模块组织, 组织你的模块
- module path versioning tool, [Versioning Your Module](#)
模块路径版本控制工具, 模块版本控制
- monotonic time blog post, [Monotonic Time](#)
单调时间博客文章, 单调时间
- open source Go module site pkg.go.dev, [Using pkg.go.dev](#)
开源 Go 模块网站 pkg.go.dev, 使用 pkg.go.dev
- open source licenses, [Publishing Your Module](#)
开源许可, 发布您的模块
- pprof code profiler, [Using Benchmarks](#)
pprof 代码分析器, 使用基准测试
- “Preemptive Interface Anti-Pattern in Go” (Lindamood), [Accept Interfaces, Return Structs](#)

“Go 语言中的抢占式接口反模式” (Lindamood) , 接受接口, 返回结构体

- semantic versioning specifications, [Working with Versions](#)
语义化版本控制规范, 使用版本控制
- standard library documentation, [The Standard Library](#).
标准库文档, 标准库
- unsafe blog post, [unsafe Is Unsafe](#)
不安全的博客文章, “不安全”就是“不安全”。
- unsafe documentation, [Using unsafe Tools](#)
不安全的文档, 使用不安全的工具
- updating code to incompatible versions, [Versioning Your Module](#)
将代码更新到不兼容的版本, 模块版本控制
- retract directive, [Retracting a Version of Your Module](#)
 - exclude directive versus, [Retracting a Version of Your Module](#)

retract 指令, 撤回模块的某个版本; exclude 指令与撤回模块的某个版本之间的区别

- return keyword, [Declaring and Calling Functions](#)
 - blank returns never should be used, [Blank Returns—Never Use These!](#)
 - concrete types returned, [Accept Interfaces, Return Structs](#)
 - defer functions running after, [defer](#)
 - goroutine return values ignored, [Goroutines](#)

- ignoring returned values, [The for-range Statement](#), [Ignoring Returned Values](#)
- maps for return values, [The Difference Between Maps and Slices](#)
- multiple return values, [Multiple Return Values](#)
 - error handling, [How to Handle Errors: The Basics](#)-[How to Handle Errors: The Basics](#)
 - errors as last return value, [How to Handle Errors: The Basics](#)
 - errors returned, [Multiple Return Values](#)
 - ignoring returned values, [The for-range Statement](#), [Ignoring Returned Values](#)
 - multiple variables for, [Multiple Return Values Are Multiple Values](#)
- named return values, [Named Return Values](#)
 - defer using, [defer](#)
- no return value, [Declaring and Calling Functions](#)
- parentheses around returned values, [Multiple Return Values](#)
- returning functions from functions, [Returning Functions from Functions](#)
- stack for returning values, [Reducing the Garbage Collector's Workload](#)
- value types favored over pointer return values, [Pointers Are a Last Resort](#)

return 关键字，声明和调用函数，永远不应该使用空返回值，空

返回值——永远不要使用！返回具体类型，接受接口，返回结构体，延迟运行的函数，延迟，Goroutine 返回值被忽略，Goroutine，对返回值进行赋值，for-range 语句，忽略返回值，使用 map 作为返回值，Map 和 Slice 的区别，多个返回值，多个返回值，错误处理，如何处理错误：基础知识，错误作为最后一个返回值，如何处理错误：基础知识，返回错误，多个返回值，对返回值进行赋值，for-range 语句，忽略返回值，多个变量，多个返回值就是多个值，命名返回值，命名返回值，使用 defer，defer 无返回值，声明和调用函数，返回值周围加括号，多个返回值，从函数返回函数，从函数返回函数，返回值的栈，减少垃圾回收器的工作负载值类型优先于指针返回值，指针是最后的选择。

- revive linter, [revive](#)
复兴棉絮，复兴
- Roussel, Achille, [Reducing the Garbage Collector's Workload](#)
鲁塞尔，阿希尔，《减少垃圾收集员的工作量》
- rune integer type, [The special integer types](#), [A Taste of Strings and Runes](#), [Strings and Runes and Bytes](#)-[Strings and Runes and Bytes](#)
 - characters via, [A Taste of Strings and Runes](#)
 - for statements iterating over strings, [Iterating over strings](#)
 - type conversions to strings and bytes, [Strings and Runes and Bytes](#)

Rune 整数类型，特殊整数类型，字符串和 Rune 简介，字符串、Rune 和字节 - 通过字符访问字符串、字符串和 Rune 简介 - for 语

句遍历字符串、遍历字符串 - 字符串和字节的类型转换、字符串、Rune 和字节

- rune literals, [Literals](#)
 - backslash-escaped rune literals, [Literals](#)
 - default type as rune, [A Taste of Strings and Runes](#)
 - string literals, [Literals](#)

符文字面量、反斜杠转义的符文字面量、默认类型为符文的字面量、字符串和符文入门、字符串字面量、字面量

S

S

- Sandberg, Anna, [defer](#)
桑德伯格, 安娜, 推迟
- The Scheduler Saga (talk by Joshi), [Goroutines](#)
调度器传奇 (Joshi 的演讲) , Goroutines
- schema for Protocol Buffer, [Using go generate](#)
使用 go generate 生成 Protocol Buffer 的 schema
- “scripting” via go run, [Using go run to Try Out Small Programs](#)
通过 go run 进行“脚本编写”, 使用 go run 来尝试运行小型程序
- security vulnerabilities tool govulncheck, [Using govulncheck to Scan for Vulnerable Dependencies](#)-[Using govulncheck to Scan for Vulnerable Dependencies](#)
安全漏洞检测工具 govulncheck, 使用 govulncheck 扫描易受攻击的依赖项 - 使用 govulncheck 扫描易受攻击的依赖项
- select keyword, [select-select](#)
 - default clause, [select](#)
 - for loop for multiple channels, [select](#)
 - selecting a closed channel, [Turn Off a case in a select](#)

选择关键字、SELECT-SELECTDefault 子句、SELECT-FORD 循

环处理多个通道、SELECT 语句中选择一个已关闭的通道、在 SELECT 语句中关闭某个 case

- semantic import versioning rule, [Updating to Incompatible Versions](#)
语义化导入版本控制规则，更新到不兼容版本
- semantic versioning in Go (SemVer), [Working with Versions](#)
 - pre-releases, [Versioning Your Module](#)
 - semantic import versioning rule, [Updating to Incompatible Versions](#)
 - specification URL, [Working with Versions](#)
 - versioning your module, [Versioning Your Module](#)

Go 语言中的语义化版本控制 (SemVer)、版本控制、预发布版本、模块版本控制、语义化导入版本控制规则、更新到不兼容版本、规范 URL、版本控制、模块版本控制、模块版本控制

- semicolon insertion rule, [go fmt](#)
分号插入规则，go fmt
- sentinel errors, [Sentinel Errors-Sentinel Errors](#)
 - constants as, [Sentinel Errors](#)
 - declared in package block, [Sentinel Errors](#)
 - origin of term, [Sentinel Errors](#)
 - testing for, [Sentinel Errors](#)
 - wrapped not checkable by ==, [Is and As](#)
 - errors.Is, [Is and As](#)

哨兵错误、哨兵错误常量、在包块中声明的哨兵错误、术语来源的哨兵错误、测试哨兵错误、包装后无法通过 == 检查的哨兵错误、Is 和 As 错误。Is、Is 和 As

- sets, [Using Maps as Sets](#)
 - maps as sets, [Using Maps as Sets](#)
 - struct{} for value, [Using Maps as Sets](#)

集合, 将 Map 用作集合, 将 Map 用作集合, 将 Map 用作集合, 使用 struct{} 作为值, 将 Map 用作集合

- shadowing variables, [Shadowing Variables-Shadowing Variables](#)
 - type switch making shadowing good, [Type Assertions and Type Switches](#)

变量遮蔽、变量遮蔽 - 变量遮蔽类型切换使遮蔽更有效、类型断言和类型切换

- Shneiderman, Ben, [defer](#)
本·施奈德曼, 推迟
- ShopSpring decimal module, [Importing Third-Party Code](#)
ShopSpring十进制模块, 导入第三方代码
- signature of function, [Functions Are Values](#)
函数签名, 函数即值
- single quote (')
单引号 (')

- not interchangeable with double quote, [Literals](#)
不可与双引号、字面量互换
- rune literals, [Literals](#)
符文字面值，字面值
- single quote rune literal ("\"), [Literals](#)
单引号符文字面量 ("\"), 字面量
- slice expressions, [Slicing Slices-Slicing Slices](#)
 - full slice expressions, [Slicing Slices](#)
 - sharing memory, [Slicing Slices](#)

切片表达式、切片切片 - 切片切片完整表达式、切片切片共享内存、切片切片

- slice literals, [Slices](#)
切片字面量，切片
- slices, [Slices-copy](#)
 - about, [Slices](#)
 - all elements to zero value, [Emptying a Slice](#)
 - append function to grow, [append](#)
 - full slice expression protecting subslices, [Slicing Slices](#)
 - arrays converted to, [Converting Arrays to Slices](#)
 - buffers for reading data, [Slices as Buffers](#)
 - call by value language, [Go Is Call by Value](#)
 - pointer implementation of slices, [Go Is Call by Value](#)
 - capacity, [Capacity](#)

- cap function for capacity, [Capacity](#)
- example code, [Capacity](#)
- never less than length, [make](#)
- converted to arrays, [Converting Slices to Arrays](#)
- copy function, [copy](#)
- declaration, [Slices](#)
 - make function for specifics, [make](#)
 - make function with append function after, [make](#), [Declaring Your Slice](#)
 - which to use when, [Declaring Your Slice](#)
 - without using literal, [Slices](#)
- full slice expressions, [Slicing Slices](#)
- function input parameters, [Variadic Input Parameters and Slices](#), [The Difference Between Maps and Slices](#)
 - how slices passed to functions, [The Difference Between Maps and Slices](#)
 - slice copy appended to, [The Difference Between Maps and Slices](#)
- len function for length, [len](#)
 - example code, [Capacity](#)
- maps versus slices, [Maps](#)
- multidimensional slices simulated, [Slices](#)
- not comparable, [Slices](#)

- DeepEqual in reflect package, [Slices](#), [Reflection Lets You Work with Types at Runtime](#)
- slices.Equal, [Slices](#)
- slices.EqualFunc, [Slices](#)
- reading and writing, [Slices](#)
- slices from slices, [Slicing Slices-Slicing Slices](#)
 - full slice expression protecting from append, [Slicing Slices](#)
 - sharing memory, [Slicing Slices](#)
- slices package in standard library, [Slices](#)
- sort.Slice, [Passing Functions as Parameters](#)
- strings converted to, [Strings and Runes and Bytes](#)
- zero value as nil, [Slices](#)

切片、切片复制、将所有元素切片为零值、清空切片、追加函数以扩展切片、追加完整切片表达式以保护子切片、切片数组转换为切片、将数组转换为切片、用于读取数据的缓冲区、切片作为缓冲区、按值调用语言、Go 语言是否支持按值调用、切片指针实现、Go 语言是否支持按值调用、容量、用于容量的 Capacitycap 函数、容量示例代码、容量永远不会小于长度、转换为数组、将切片转换为数组的 copy 函数、copy 声明、用于特定用途的切片 make 函数、带有 append 函数的 make 函数、make、声明切片时应使用哪个、不使用字面量声明切片、切片完整切片表达式、切片函数输入参数、可变参数输入参数和切片、Map 和 Slice 的区别、传递给函数的切片、Map 和 Slice 的区别复制追加到, Map

和 Slices 的区别, len 函数用于长度, len 示例代码, Capacitymap 与 Slices, Map 模拟多维切片, Slices 不可比较, reflect 包中的 SlicesDeepEqual, Slices, 反射允许您在运行时处理类型, slices.Equal, Slices.EqualFunc, Slices 的读写, Slices 从切片中获取切片, 切片 Slices - 切片 Slices, 完整切片表达式防止追加, 切片 Slices 共享内存, 标准库中的 Slices 包, Slicessort.Slice, 将函数作为参数传递, 字符串转换为, 字符串、符文和字节, 零值作为 nil, 切片

- slices package in standard library, [Slices](#)
标准库中的 slices 包, Slices
- Smith, Forrest, [Reducing the Garbage Collector's Workload](#)
史密斯, 福雷斯特, 《减少垃圾收集员的工作量》
- Sonatype proxy server support, [Specifying a Proxy Server](#)
Sonatype代理服务器支持, 指定代理服务器
- sort.Slice, [Passing Functions as Parameters](#)
sort.Slice, 将函数作为参数传递
- sparse arrays, [Arrays—Too Rigid to Use Directly](#)
稀疏数组, 数组——过于僵化而无法直接使用
- SQLite embedded in Go application, [Cgo Is for Integration, Not Performance](#)
在 Go 应用中嵌入 SQLite, Cgo 用于集成, 而非性能
- stack, [Reducing the Garbage Collector's Workload](#), [Introducing Generics in Go](#)

- garbage collector workload reduced, [Reducing the Garbage Collector's Workload](#)-[Reducing the Garbage Collector's Workload](#)
- generic functions to create, [Introducing Generics in Go](#)
- goroutine leak, [Always Clean Up Your Goroutines](#)
- more information on heap versus stack, [Reducing the Garbage Collector's Workload](#)
- pointer type stored on stack, [Reducing the Garbage Collector's Workload](#)
 - data pointer pointing to, [Reducing the Garbage Collector's Workload](#)
- size increase during execution, [Reducing the Garbage Collector's Workload](#)
- stack frame, [Reducing the Garbage Collector's Workload](#)
- stack pointer, [Reducing the Garbage Collector's Workload](#)
- stack traces
 - errors providing, [Getting a Stack Trace from an Error](#)
 - panics providing, [panic and recover](#)

堆栈，减少垃圾回收器的工作负载，在 Go 中引入泛型，垃圾回收器工作负载减少，减少垃圾回收器的工作负载 - 减少垃圾回收器的工作负载，要创建的泛型函数，在 Go 中引入泛型，goroutine 泄漏，始终清理你的 goroutine，关于堆与栈的更多信息，减少垃圾回收器的工作负载，存储在栈上的指针类型，减少

垃圾回收器的工作负载，数据指针指向，减少垃圾回收器的工作负载，执行期间大小增加，减少垃圾回收器的工作负载，堆栈帧，减少垃圾回收器的工作负载，堆栈指针，减少垃圾回收器的工作负载，堆栈跟踪，提供错误，从错误中获取堆栈跟踪，提供 panic, panic 和 recover

- standard library

标准库

- C pseudopackage, [Cgo Is for Integration, Not Performance-Cgo Is for Integration, Not Performance](#)

C 伪包, Cgo 用于集成, 而非性能 - Cgo 用于集成, 而非性能

- context package, [What Is the Context?](#)
 - (see also context package)

context 包, 什么是上下文? (另请参阅 context 包)

- database/sql/driver package, [defer, Accept Interfaces, Return Structs](#)

数据库/SQL/驱动程序包, 延迟, 接受接口, 返回结构体

- documentation URL, [The Standard Library](#)

文档网址, 标准库

- encoding/csv package, [Use Reflection to Write a Data Marshaler](#)
encoding/csv 包, 使用反射编写数据编组器

- encoding/json package, [encoding/json-Custom JSON Parsing](#)
encoding/json 包, encoding/json-自定义 JSON 解析

- errors package

错误包

- documentation URL, [Getting a Stack Trace from an Error](#)
文档 URL, 如何从错误中获取堆栈跟踪
- errors.As, [Use Type Assertions and Type Switches Sparingly, Is and As](#)

错误。请谨慎使用类型断言和类型切换, Is 和 As

- errors.Is, [Use Type Assertions and Type Switches Sparingly, Is and As](#)

错误。是, 谨慎使用类型断言和类型切换, 是和作为

- errors.New, [How to Handle Errors: The Basics, Use Strings for Simple Errors](#)

错误处理方法: 基础知识, 使用字符串表示简单错误

- fmt package, [go build](#)

fmt 包, go 构建

- generic functions added to, [Adding Generics to the Standard Library](#)
 - implementations pre-generics, [Generics Reduce Repetitive Code and Increase Type Safety](#)

泛型函数已添加到标准库实现中 (泛型出现之前), 泛型可减少重复代码并提高类型安全性

- Go Compatibility Promise, [The Go Compatibility Promise, Accept Interfaces, Return Structs](#)

Go兼容性承诺、Go兼容性承诺、接受接口、返回结构体

- io package, [Interfaces Are Type-Safe Duck Typing](#), [Use Type Assertions and Type Switches Sparingly](#), [io and Friends](#)-[io and Friends](#)

io 包, 接口是类型安全的鸭子类型, 谨慎使用类型断言和类型切换, io 和朋友们-io 和朋友们

- log/slog package, [Structured Logging](#)-[Structured Logging](#)

log/slog 包, 结构化日志记录-结构化日志记录

- maps package, [Comparing Maps](#)

地图包, 地图比较

- net/http package, [net/http-ResponseController](#)

net/http 包, net/http-ResponseController

- os package, [io and Friends](#)

os 包、io 和朋友们

- reflect package, [Here Be Dragons: Reflect, Unsafe, and Cgo](#), [Reflection Lets You Work with Types at Runtime](#)

reflect 包, 《此处有龙: Reflect、Unsafe 和 Cgo》, 反射允许你在运行时处理类型

- slices package, [Slices](#)

切片包装, 切片

- sort package, [Passing Functions as Parameters](#)

sort 包, 将函数作为参数传递

- strconv package, [Choosing which integer to use](#), [goto—Yes, goto](#), [Functions Are Values](#)

strconv 包, 选择要使用的整数, goto—是, goto, 函数即值

- strings package, [Strings and Runes and Bytes](#)
strings 包, 字符串、符文和字节
- sync package, [Use WaitGroups](#), [When to Use Mutexes Instead of Channels](#)

同步包、使用等待组、何时使用互斥锁而不是通道

- sys package, [unsafe Is Unsafe](#)
sys 包, 不安全。
- syscall package, [unsafe Is Unsafe](#)
syscall 包, unsafe 是不安全的
- testing package, [Writing Tests](#)
 - (see also testing)

测试包, 编写测试 (另见测试)

- time package, [time-Timers and Timeouts](#)
时间包、定时器和超时
- unicode/utf8 package, [Strings and Runes and Bytes](#)
unicode/utf8 包, 字符串、符文和字节
- unsafe package, [A Quick Pointer Primer](#), [unsafe Is Unsafe](#)
不安全包装, 快速入门指南, 不安全就是不安全

- Staron, Mirosław, [defer](#)
斯塔隆, 米罗斯拉夫, 推迟
- staticcheck linter, [staticcheck-staticcheck](#)
staticcheck 代码检查工具, staticcheck-staticcheck

- strconv package

strconv 包

- strconv.Atoi, [Functions Are Values](#)
strconv.Atoi, 函数即值
- strconv.FormatInt, [Choosing which integer to use](#)
strconv.FormatInt, 选择要使用的整数

- string literals

字符串字面量

- default type as string, [A Taste of Strings and Runes](#)
默认类型为字符串, 《字符串与符文入门》
- raw string literals, [Literals](#)
原始字符串字面量, 字面量
- string literals, [Literals](#)
字符串字面量, 字面量
- UTF-8, [Strings and Runes and Bytes](#)
 - string bytes versus UTF-8 multi-byte code points, [Strings and Runes and Bytes](#)

UTF-8、字符串、符文和字节字符串字节与 UTF-8 多字节代码点、字符串、符文和字节

- string type, [A Taste of Strings and Runes](#), [Strings and Runes and Bytes](#), [Strings and Runes and Bytes](#)
 - bytes representing string, [Strings and Runes and Bytes](#)
 - converted to slices, [Strings and Runes and Bytes](#)

- error handling, [Use Strings for Simple Errors](#)
- extracting single value from, [Strings and Runes and Bytes](#)
- for statements iterating over, [Iterating over strings](#)
- immutable, [A Taste of Strings and Runes](#)
- len function for length in bytes, [Strings and Runes and Bytes](#)
- slice expression notation working with, [Strings and Runes and Bytes](#)
- strings.NewReader function creating io.Reader, [io and Friends](#)
- time converted to and from, [time](#)
- time duration strings, [time](#)
- type conversions to runes and bytes, [Strings and Runes and Bytes](#)
- zero value as empty string, [A Taste of Strings and Runes](#)

字符串类型，字符串和符文入门，字符串、符文和字节 - 字符串、符文和字节 - 字节表示字符串，字符串、符文和字节转换为切片，字符串、符文和字节错误处理，使用字符串处理简单错误，从字符串中提取单个值，字符串、符文和字节的 for 语句迭代，迭代字符串不可变，字符串和符文入门 - len 函数表示字节长度，字符串、符文和字节 - 切片表达式表示法，字符串、符文和字节 - strings.NewReader 函数创建 io.Reader，io 和 Friends - time 的转换，time 持续时间字符串，time 转换为符文和字节，字符串、符文和字节 - 零值作为空字符串，字符串和符文入门

- stringer tool for printable enumerations, [Using go generate](#)
用于生成可打印枚举的字符串工具，使用 go generate

- strings package in standard library, [Strings and Runes and Bytes](#)
标准库中的 strings 包, Strings、Runes 和 Bytes
- struct keyword, [Structs](#), [Types in Go](#)
Go 语言中的 struct 关键字、结构体和类型
- struct literals, [Structs](#)
结构体字面量, 结构体
- structs, [Structs](#)
 - accept interfaces, return structs, [Accept Interfaces, Return Structs](#)-
[Accept Interfaces, Return Structs](#)
 - anonymous structs, [Anonymous Structs](#)
 - comparing structs, [Comparing and Converting Structs](#)
 - unmarshaling and marshaling data, [Anonymous Structs](#)
 - comparing structs, [Comparing and Converting Structs](#)
 - anonymous structs, [Comparing and Converting Structs](#)
 - declaration, [Structs](#)
 - as user-defined type with underlying struct, [Types in Go](#)
 - direct field access encouraged, [Pointer Receivers and Value Receivers](#)
 - embedded field methods promoted to containing struct, [Use Embedding for Composition](#)
 - embedding is not inheritance, [Embedding Is Not Inheritance](#)
 - fields or methods with same name, [Use Embedding for Composition](#)

- emulating named and optional parameters, [Simulating Named and Optional Parameters](#)
- input parameters, [The Difference Between Maps and Slices](#)
- maps used as sets, [Using Maps as Sets](#)
- pointer input parameter or return value for, [Pointer Passing Performance](#)
- reading and writing, [Structs](#)
- reflection avoided for building, [You Can Build Structs with Reflection, but Don't](#)
- return values, [The Difference Between Maps and Slices](#)
- struct tags for metadata, [Using Struct Tags to Add Metadata](#)
- type conversions, [Comparing and Converting Structs](#)
- zero value as field's zero value, [Structs](#)

结构体、接受接口的结构体、返回结构体、接受接口、返回结构体 - 接受接口、返回结构体、匿名结构体、比较结构体、比较和转换结构体、数据反序列化和序列化、匿名结构体、比较和转换结构体、比较和转换结构体、匿名结构体、比较和转换结构体、声明、结构体作为具有底层结构体的用户定义类型、鼓励使用 Go 中的类型直接访问字段、指针接收器和值接收器、嵌入式字段方法提升为包含结构体、使用嵌入进行组合、嵌入不是继承、嵌入不是继承、同名字段或方法、使用嵌入进行组合、模拟命名参数和可选参数、模拟命名参数和可选参数、输入参数、映射和切片的区别、用作集合的映射、将映射用作集合、指针输入参数或返

返回值，指针传递性能读写，避免使用反射构建结构体，可以使用反射构建结构体，但不要返回值，映射和切片的区别，用于元数据的结构体标签，使用结构体标签添加元数据，数据类型转换，比较和转换结构体，零值作为字段的零值，结构体

- structured logging, [Structured Logging-Structured Logging](#)
结构化日志记录，结构化日志记录-结构化日志记录
- Subversion for version control, [Publishing Your Module](#)
用于版本控制的 Subversion，发布您的模块
- Sulaiman, Marwan, [Versioning Your Module](#)
Sulaiman, Marwan, 《模块版本控制》
- switch keyword, [switch-switch](#)
 - blank switches, [Blank Switches](#)
 - break keyword, [switch](#)
 - case clause, [switch-switch](#)
 - choosing between if and switch, [Choosing Between if and switch](#)
 - default, [switch-switch](#)
 - fallthrough keyword avoided, [switch](#)

switch 关键字、switch-switch 空白开关、空白开关中断关键字、switchcase 子句、switch-switch 在 if 和 switch 之间选择、在 if 和 switchdefault 之间选择、避免使用 switch-switchfallthrough 关键字、switch

- sync package
同步包

- sync.Map type, [When to Use Mutexes Instead of Channels](#)
同步映射类型, 何时使用互斥锁而不是通道
- sync.Mutex, [When to Use Mutexes Instead of Channels](#)
同步互斥锁, 何时使用互斥锁而不是通道
- sync.Once type for running code once, [Run Code Exactly Once](#)
 - helper functions, [Run Code Exactly Once](#)

sync.Once 类型用于运行代码一次, 运行代码仅一次辅助函数, 运行代码仅一次

- sync.RWMutex, [When to Use Mutexes Instead of Channels](#)
sync.RWMutex, 何时使用互斥锁而不是通道
- sync.WaitGroup, [Use WaitGroups-Use WaitGroups](#)
同步.等待组, 使用等待组-使用等待组
- sys package, [unsafe Is Unsafe](#)
sys 包, 不安全。
- syscall package, [unsafe Is Unsafe](#)
syscall 包, unsafe 是不安全的

T

T

- tab rune literal ('\t'), [Literals](#)
tab 符文字面量 ('\t') , 字面量

- table tests, [Running Table Tests-Running Table Tests](#)
表格测试，运行表格测试-运行表格测试
- The Tail at Scale (Dean), [Reducing the Garbage Collector's Workload](#)
规模化尾部（迪恩），减少垃圾收集员的工作量
- testing
测试
 - about, [Writing Tests](#)
关于，编写测试题
 - basics of, [Understanding the Basics of Testing-Understanding the Basics of Testing](#)
 - caching test results, [Caching Test Results](#)
 - environment variables in testing, [Testing with Environment Variables](#)
 - go-cmp to compare test results, [Using go-cmp to Compare Test Results-Using go-cmp to Compare Test Results](#)
 - reporting test failures, [Reporting Test Failures](#)
 - storing sample test data, [Storing Sample Test Data](#)
 - test setup and teardown, [Setting Up and Tearing Down-Setting Up and Tearing Down](#)
 - testing your public API, [Testing Your Public API](#)

测试基础知识 - 理解测试基础知识；缓存测试结果；测试中的环境变量；使用环境变量进行测试；使用 go-cmp 比较测

测试结果；报告测试失败；存储示例测试数据；测试设置和拆卸；测试公共 API。

- benchmarks, [Using Benchmarks-Using Benchmarks](#)
 - profiling your code, [Using Benchmarks](#)

基准测试，使用基准测试——使用基准测试分析代码，使用基准测试

- code coverage, [Checking Your Code Coverage-Checking Your Code Coverage](#)

代码覆盖率，检查您的代码覆盖率-检查您的代码覆盖率

- concurrency problems, [Finding Concurrency Problems with the Data Race Detector](#)

并发问题，使用数据竞争检测器查找并发问题

- data race detector, [Finding Concurrency Problems with the Data Race Detector](#)

数据竞争检测器，利用数据竞争检测器查找并发问题

- fuzzing, [Fuzzing-Fuzzing](#)

模糊处理，模糊处理

- go test, [Pointer Passing Performance](#), [Understanding the Basics of Testing-Understanding the Basics of Testing](#), [Setting Up and Tearing Down](#)

- benchmarks, [Using Benchmarks](#)
- caching test results, [Caching Test Results](#)

- code coverage, [Checking Your Code Coverage-Checking Your Code Coverage](#)
- data race detector, [Finding Concurrency Problems with the Data Race Detector](#)
- fuzzing, [Fuzzing-Fuzzing](#)
- go-cmp to compare results, [Using go-cmp to Compare Test Results-Using go-cmp to Compare Test Results](#)
- integration tests, [Using Integration Tests and Build Tags](#)
- -short flag, [Using Integration Tests and Build Tags](#)
- storing sample test data, [Storing Sample Test Data](#)

go 测试、指针传递性能、理解测试基础、设置和清理基准测试、使用基准测试、缓存测试结果、代码覆盖率、检查代码覆盖率、数据竞争检测器、使用数据竞争检测器查找并发问题、模糊测试、使用 go-cmp 比较测试结果、集成测试、使用集成测试和构建标签、存储示例测试数据

- go vet, [go vet](#)
 - shadowing not reported, [Shadowing Variables](#)
 - struct tag validation, [Using Struct Tags to Add Metadata](#)
 - third-party tools, [Using Code-Quality Scanners-golangci-lint](#)

go vet, go vetshadowing 未报告, Shadowing Variablesstruct 标签验证, 使用 Struct 标签添加元数据第三方工具, 使用代码质量扫描器-golangci-lint

- httpstest, [Using httpstest-Using httpstest](#)
httpstest, 使用 httpstest-使用 httpstest
- integration tests and build tags, [Using Integration Tests and Build Tags](#)
集成测试和构建标签, 使用集成测试和构建标签
- race checker, [Finding Concurrency Problems with the Data Race Detector](#)
竞争检测器, 使用数据竞争检测器查找并发问题
- running tests concurrently, [Running Tests Concurrently-Running Tests Concurrently](#)
并发运行测试, 并发运行测试-并发运行测试
- sentinel errors, [Sentinel Errors](#)
哨兵错误, 哨兵错误
- stubs, [Using Stubs in Go-Using Stubs in Go](#)
 - mocks versus, [Using Stubs in Go](#)

存根, 在 Go 中使用存根 - 在 GoMocks 中使用存根对比, 在 Go 中使用存根

- table tests, [Running Table Tests-Running Table Tests](#)
表格测试, 运行表格测试-运行表格测试
- text package and using test data, [Storing Sample Test Data](#)
文本包和使用测试数据, 存储示例测试数据
- versions of Go, [Testing Versions of Go](#)
Go 版本, 测试 Go 版本

- text package and using test data, [Storing Sample Test Data](#)
文本包和使用测试数据，存储示例测试数据
- third-party code imported, [Importing Third-Party Code](#)
导入第三方代码，导入第三方代码
- third-party modules for http.Server, [Use third-party modules to enhance the server](#)
用于 http.Server 的第三方模块，使用第三方模块增强服务器功能
- Thompson, Ken, [Complex types \(you're probably not going to use these\)](#), [Strings and Runes and Bytes](#)
汤普森、肯、复杂类型（你可能不会用到这些）、字符串、符文和字节
- Thompson, Martin, [Reducing the Garbage Collector's Workload](#)
汤普森，马丁，《减少垃圾收集员的工作量》
- thrashing, [Tuning the Garbage Collector](#)
猛烈抨击，调整垃圾收集器
- TiB suffix, [Tuning the Garbage Collector](#)
TiB 后缀，调整垃圾回收器
- tilde (~) for types with underlying type, [Use Type Terms to Specify Operators](#)
对于具有底层类型的类型，请使用波浪号 (~) 和类型术语来指定运算符。
- time package, [time-Timers and Timeouts](#)
 - Day, Month, Year, Hour, Minute, Second, Weekday methods, [time](#)
 - formatting of date and time, [time](#)

- methods not modifying time.Time instance, [time](#)
- monotonic time, [Monotonic Time](#)
 - importance of handling correctly, [Monotonic Time](#)
- time.Add, [time](#)
- time.AddDate, [time](#)
- time.After, [Timers and Timeouts](#)
- time.AfterFunc, [Timers and Timeouts](#)
- time.Clock, [time](#)
- time.Format, [time](#)
- time.NewTicker, [Timers and Timeouts](#)
- time.Now, [time](#)
- time.Parse, [time](#)
- time.ParseDuration, [time](#)
- time.Round, [time](#), [time](#)
- time.Sub, [time](#)
- time.Tick, [Timers and Timeouts](#)
 - time.NewTicker instead, [Timers and Timeouts](#)
- time.Truncate, [time](#), [time](#)
- timeouts, [Timers and Timeouts](#)
 - timing out code, [Time Out Code](#)
- timers, [Timers and Timeouts](#)
- types
 - time.Duration, [time](#)
 - time.Time, [time](#)

- time.Time comparisons, [time](#), [time](#)

时间包, 时间-计时器和超时, 日期、月份、年份、小时、分钟、秒、星期几方法, 日期和时间的格式化, 不修改 time.Time 实例的时间方法, 单调时间, 正确处理单调时间的重要性, 单调时间, time.Add、time.AddDate、time.After, 计时器和超时, time.AfterFunc, 计时器和超时, time.Clock、time.Format、time.NewTicker, 计时器和超时, time.Now、time.Parse、time.ParseDuration、time.Round, time、time.Sub、time.Tick, 计时器和超时, time.NewTicker 代替, 计时器和超时, time.Truncate, 时间, 超时, 计时器和超时, 超时代码, 超时代码, 计时器, 计时器和超时类型, time.Duration、time.Time、time.Time 比较, 时间, 时间

- time zone in time.Time type, [time](#)
 - Equal method for comparison, [time](#)

时间中的时区。时间类型, 时间。比较的相等方法, 时间。

- timing out code in concurrency, [Time Out Code](#)
并发代码超时, 超时代码
- toolchain directive for version control, [Use the go Directive to Manage Go Build Versions](#)
用于版本控制的工具链指令, 使用 go 指令管理 Go 构建版本

- tools

工具

- about, [Go Tooling](#)
 - (see also go commands)

关于 Go 工具（另请参阅 go 命令）

- building binaries for other platforms, [Building Go Binaries for Other Platforms](#)
 - build tags, [Using Build Tags](#)

为其他平台构建二进制文件，为其他平台构建 Go 二进制文件，构建标签，使用构建标签

- code style check and bug scan, [Using Code-Quality Scanners-golangci-lint](#)
 - golangci-lint, [golangci-lint-golangci-lint](#)
 - revive, [revive](#)
 - staticcheck, [staticcheck-staticcheck](#)

代码风格检查和错误扫描，使用代码质量扫描器-golangci-lintgolangci-lint、golangci-lint-golangci-lintrevive、revivestaticcheck、staticcheck-staticcheck

- concurrency tools working together, [Put Your Concurrent Tools Together-Put Your Concurrent Tools Together](#)

并发工具协同工作，将您的并发工具整合在一起 - 将您的并发工具整合在一起

- embedding content into program, [Embedding Content into Your Program-Embedding Content into Your Program](#)
 - hidden files, [Embedding Hidden Files](#)

将内容嵌入程序、将内容嵌入程序 - 将内容嵌入程序 - 隐藏文件、嵌入隐藏文件

- Go development tools URL, [Choose Your Tools](#)
Go 开发工具 URL, 选择您的工具
- go generate, [Using go generate-Using go generate](#)
 - Makefiles and, [Working with go generate and Makefiles](#)

go generate, 使用 go generate - 使用 go generateMakefiles, 以及使用 go generate 和 Makefiles

- go help, [Using go help to Learn More About Go Tooling](#)
 - go help environment, [Adding Third-Party Tools with go install](#)
 - go help importpath, [Using go help to Learn More About Go Tooling](#)

使用 go help 了解更多关于 Go 工具的信息；使用 go help environment 了解更多关于使用 go install 添加第三方工具的信息；使用 go help importpath 了解更多关于 Go 工具的信息。

- go install, [Adding Third-Party Tools with go install](#)-[Adding Third-Party Tools with go install](#)
 - GOBIN to set install path, [Adding Third-Party Tools with go install](#)
 - goimports tool install command, [Improving Import Formatting with goimports](#)
 - GOPROXY for proxy or repository, [Adding Third-Party Tools with go install](#)
 - installing hey, [Adding Third-Party Tools with go install](#)
 - pkgsite installation, [Documenting Your Code with Go Doc Comments](#)
 - secondary Go environment, [Testing Versions of Go](#)
 - updating a tool to newer version, [Adding Third-Party Tools with go install](#)

使用 go install 添加第三方工具 - 使用 go installGOBIN 设置安装路径, 使用 go installgoimports 工具安装命令添加第三方工具, 使用 goimportsGOPROXY 改进代理或存储库的导入格式, 使用 go install 安装 hey, 使用 go install 安装 pkgsite, 使用 Go Doc 注释为代码添加文档, 辅助 Go 环境, 测试 Go 版本, 将工具更新到更新版本, 使用 go install 添加第三方工具

- The Go Playground, [The Go Playground](#)-[The Go Playground](#)
Go Playground, Go Playground-Go Playground

- go run to build and execute, [Using go run to Try Out Small Programs](#)

使用 `go run` 命令构建并执行程序, 使用 `go run` 尝试运行小型程序。

- goimports improving formatting, [Improving Import Formatting with goimports](#)

goimports 改进导入格式, 使用 goimports 改进导入格式

- govulncheck scan for security vulnerabilities, [Using govulncheck to Scan for Vulnerable Dependencies](#)

govulncheck 扫描安全漏洞, 使用 govulncheck 扫描易受攻击的依赖项

- hey to load test HTTP servers, [Adding Third-Party Tools with go install](#)

嘿, 要对 HTTP 服务器进行负载测试, 请使用 go install 添加第三方工具

- IDEs, [Choose Your Tools](#)
 - GoLand, [GoLand](#)

IDE, 选择你的工具, GoLand, GoLand

- stringer for printable enumerations, [Using go generate](#)

用于可打印枚举的字符串生成器, 使用 go generate

- testing versions of Go, [Testing Versions of Go](#)

测试 Go 版本, 测试 Go 版本

- updating to newer version, [Adding Third-Party Tools with go install](#)

更新到新版本, 使用 go install 添加第三方工具

- troubleshooting

故障排除

- code error detection via go vet, [go vet](#)
 - (see also testing)

通过 go vet 进行代码错误检测, go vet (另请参阅测试)

- installation of Go tools, [Troubleshooting Your Go Installation](#)

Go 工具的安装, Go 安装故障排除

- make function slice with append after, [make](#), [Declaring Your Slice](#)

创建带有追加功能的切片函数, 创建, 声明你的切片

- universe block predeclared identifiers, [Shadowing Variables](#)
宇宙块预声明标识符, 变量遮蔽

- variables

变量

- declaring new with var, [var Versus :=](#)
使用 var 声明新变量, var 与 :=
- package-level avoided, [var Versus :=](#)
避免了包级别的冲突, var Versus :=
- shadowing variables, [Shadowing Variables](#)-[Shadowing Variables](#)

阴影变量, 阴影变量-阴影变量

- true, [Booleans](#)
 - booleans only, [Explicit Type Conversion](#)

真, 仅限布尔值, 显式类型转换

- type assertions, [Type Assertions and Type Switches](#)-[Type Assertions and Type Switches](#)
 - type conversions versus, [Type Assertions and Type Switches](#)
 - uses for, [Use Type Assertions and Type Switches Sparingly](#)-[Use Type Assertions and Type Switches Sparingly](#)

类型断言、类型断言和类型开关 - 类型断言和类型开关类型转换
对比、类型断言和类型开关的用途、谨慎使用类型断言和类型开关 - 谨慎使用类型断言和类型开关

- type conversion, [Explicit Type Conversion](#)
 - arrays of different sizes, [Arrays—Too Rigid to Use Directly](#)
 - strings, runes, and bytes, [Strings and Runes and Bytes](#)
 - structs, [Comparing and Converting Structs](#)
 - type assertion versus, [Type Assertions and Type Switches](#)
 - untyped constants or literals, [var Versus :=](#)

类型转换、显式类型转换、不同大小的数组、数组——过于死板而无法直接使用、字符串、符文和字节、字符串、符文和字节、

结构体、结构体的比较和转换、类型断言与类型转换、无类型常量或字面量、var 与 :=

- type elements, [Use Type Terms to Specify Operators](#)-[Use Type Terms to Specify Operators](#)
 - constants that can be assigned to variables, [Type Elements Limit Constants](#)
 - type terms, [Use Type Terms to Specify Operators](#)-[Use Type Terms to Specify Operators](#)
 - ~ for types with underlying type, [Use Type Terms to Specify Operators](#)

类型元素，使用类型术语指定运算符 - 使用类型术语指定运算符 - 可赋值给变量的常量，类型元素限制常量 - 类型术语，使用类型术语指定运算符 - 使用类型术语指定运算符 - 对于具有底层类型的类型，使用类型术语指定运算符

- type inference and generic functions, [Type Inference and Generics](#)
类型推断和泛型函数，类型推断和泛型
- type keyword, [Types in Go](#)
 - alias declaration, [Gracefully Renaming and Reorganizing Your API](#)
 - functions, [Function Type Declarations](#)
 - interface, [A Quick Lesson on Interfaces](#)
 - structs, [Structs](#), [Types in Go](#)

type 关键字、Go 中的类型、别名声明、优雅地重命名和重组 API、函数、函数类型声明、接口、接口速成课、结构体、Go 中的结构体和类型

- type switches, [Type Assertions and Type Switches](#)
 - uses for, [Use Type Assertions and Type Switches Sparingly](#)-[Use Type Assertions and Type Switches Sparingly](#)

类型开关、类型断言和类型开关的用途、谨慎使用类型断言和类型开关 - 谨慎使用类型断言和类型开关

- types
类型

- about Go, [Types, Methods, and Interfaces](#)
 - types in Go, [Types in Go](#)

关于 Go 语言、类型、方法和接口

- abstract types, [Types in Go](#)
 - interfaces, [A Quick Lesson on Interfaces](#)

抽象类型、Go 接口中的类型、接口速成课

- adding a method to, [io and Friends](#)
向 io 和 Friends 添加方法
- aliases, [Gracefully Renaming and Reorganizing Your API](#)
别名, 优雅地重命名和重组您的 API

- all types as value types, [Go Is Call by Value](#)
所有类型均为值类型，Go 是按值调用
- array size affecting type, [Arrays—Too Rigid to Use Directly](#)
 - converting array to slice to array, [Converting Slices to Arrays](#)
 - slices contrasted, [Slices](#)
 - stack and, [Reducing the Garbage Collector's Workload](#)

数组大小影响类型，数组——过于僵化而无法直接使用，数组到切片再到数组的转换，切片到数组的转换，切片对比，切片堆栈，以及减少垃圾回收器的工作负载

- byte for uint8, [The special integer types](#), [A Taste of Strings and Runes](#)
 - for statements iterating over strings, [Iterating over strings](#)
 - string bytes versus UTF-8 multi-byte code points, [Strings and Runes and Bytes](#)
 - strings as sequences of, [Strings and Runes and Bytes-Strings and Runes and Bytes](#)
 - type conversions to strings and runes, [Strings and Runes and Bytes](#)

uint8 的字节，特殊整数类型，字符串和符文入门，for 语句遍历字符串，遍历字符串，字符串字节与 UTF-8 多字节代码点，字符串、符文和字节，字符串作为序列，字符串、符文

和字节 - 字符串、符文和字节的类型转换，字符串、符文和字节

- channels, [Channels](#)

频道，频道

- composite types

复合类型

- arrays, [Arrays—Too Rigid to Use Directly-Arrays—Too Rigid to Use Directly](#)

数组，数组——过于僵化而无法直接使用——数组——过于僵化而无法直接使用

- maps, [Maps-Using Maps as Sets](#)

地图，地图——将地图用作集合

- slices, [Slices-copy](#)

切片，切片复制

- strings, runes, bytes, [Strings and Runes and Bytes-Strings and Runes and Bytes](#)

字符串、符文、字节、字符串、符文和字节-字符串、符文和字节

- structs, [Structs](#)

结构体，结构体

- concrete types, [Types in Go](#)

- interface implicit implementation, [Interfaces Are Type-Safe Duck Typing](#)

具体类型、Go 接口中的类型、隐式实现、接口是类型安全的鸭子类型

- constants typed and untyped, [Typed and Untyped Constants](#)
类型化常量和非类型化常量
- context.Context, [What Is the Context?](#)
context.Context, 什么是上下文?
- default type, [Literals](#)
默认类型, 字面量
- duck typing, [Interfaces Are Type-Safe Duck Typing](#)
鸭子类型, 接口是类型安全的鸭子类型
- empty interface for variable of any type, [The Empty Interface Says Nothing](#)
 - any as type alias for, [The Empty Interface Says Nothing](#)

空接口, 适用于任何类型的变量, “空接口什么也不说”; 作为类型别名, “空接口什么也不说”

- error interface type, [Accept Interfaces, Return Structs, How to Handle Errors: The Basics](#)
 - wrapped errors, [Use Type Assertions and Type Switches Sparingly](#)

错误接口类型、接受接口、返回结构体、如何处理错误: 基础知识、包装错误、谨慎使用类型断言和类型切换

- explicit type conversion, [Explicit Type Conversion](#)
 - automatic type promotion versus, [Explicit Type Conversion](#)
 - boolean treatment of variable and, [Explicit Type Conversion](#)
 - untyped constants or literals, [var Versus :=](#)

显式类型转换、显式类型转换自动类型提升与显式类型转换、显式类型转换布尔变量处理与、显式类型转换无类型常量或字面量、var 与 :=

- generic function abstract algorithms, [Generic Functions Abstract Algorithms](#)
[Generic Functions Abstract Algorithms](#)
通用函数抽象算法，通用函数抽象算法-通用函数抽象算法
- generic functions and type inference, [Type Inference and Generics](#)
泛型函数和类型推断，类型推断和泛型
- int for int32 or int64, [The special integer types](#)
 - uint as unsigned int, [The special integer types](#)

int 代表 int32 或 int64，特殊的整数类型 uint 也代表 unsigned int，特殊的整数类型

- int32 via rune type, [A Taste of Strings and Runes](#)
int32 通过符文类型，弦乐与符文的滋味
- iota enumeration type, [iota Is for Enumerations—Sometimes](#)
iota 枚举类型，iota 代表枚举——有时
- literals, [Literals](#)

- untyped, [Literals](#), [Literals Are Untyped](#)

字面量，未类型化的字面量，字面量，未类型化的字面量

- names of variables, [Naming Variables and Constants](#)
变量名称、变量和常量的命名
- net/http package Client type, [The Client-ResponseController](#)
net/http 包客户端类型，客户端响应控制器
- nil untyped, [Slices](#), [A Quick Pointer Primer](#)
nil untyped，切片，快速入门指南
- pointer type, [A Quick Pointer Primer](#)
 - pointer receivers, [Pointer Receivers and Value Receivers](#)-
[Pointer Receivers and Value Receivers](#)

指针类型，快速指针入门，指针接收器，指针接收器和值接收器-指针接收器和值接收器

- predeclared types
预先声明的类型
 - about, [The Predeclared Types](#)
关于预先声明的类型
 - booleans, [Booleans](#)
布尔值
 - complex types, [Complex types \(you're probably not going to use these\)](#)-[Complex types \(you're probably not going to use these\)](#)

复杂类型，复杂类型（您可能用不到这些）-复杂类型
（您可能用不到这些）

- floating-point types, [Floating-point types](#)
浮点类型，浮点类型
- numeric, [Numeric Types-Complex types \(you're probably not going to use these\)](#)
数值型、数值类型 - 复杂类型（您可能不会用到这些）
- strings, [A Taste of Strings and Runes](#)
弦乐，弦乐与符文的品味

- reflection

反射

- about, [Reflection Lets You Work with Types at Runtime](#)
关于反射，它允许你在运行时处理类型
- automating repetitive tasks, [Build Functions with Reflection to Automate Repetitive Tasks](#)
自动化重复性任务，使用反射构造函数以实现重复性任务的自动化
- avoid building structs with, [You Can Build Structs with Reflection, but Don't](#)
避免使用反射来构建结构体，虽然你可以使用反射来构建结构体，但不要这样做。
- cannot add methods to a type, [Reflection Can't Make Methods](#)
无法向类型添加方法，反射无法创建方法

- checking for nil interface value, [Use Reflection to Check If an Interface's Value Is nil](#)
检查接口值是否为 nil, 使用反射检查接口值是否为 nil
- data marshaler, [Use Reflection to Write a Data Marshaler-Use Reflection to Write a Data Marshaler](#)
数据序列化器, 使用反射编写数据序列化器 - 使用反射编写数据序列化器
- making new values, [Make New Values](#)
创造新价值, 创造新价值
- types and kinds, [Types and kinds-Types and kinds](#)
类型和种类, 类型和种类——类型和种类
- use only if worthwhile, [Use Reflection Only if It's Worthwhile](#)
仅在值得时使用, 仅在值得时使用反思
- values, [Values](#)
价值观, 价值观
- rune integer type, [The special integer types, A Taste of Strings and Runes, Strings and Runes and Bytes-Strings and Runes and Bytes](#)
 - characters via, [A Taste of Strings and Runes](#)
 - for statements iterating over strings, [Iterating over strings](#)
 - type conversions to strings and bytes, [Strings and Runes and Bytes](#)

Rune 整数类型, 特殊整数类型, 字符串和 Rune 简介, 字符

串、Rune 和字节 - 通过字符访问字符串、字符串和 Rune 简介 - for 语句遍历字符串、遍历字符串 - 字符串和字节的类型转换、字符串、Rune 和字节

- Scanner type for *os.File instance, [io and Friends](#)
*os.File 实例、io 和 Friends 的扫描器类型
- storage in memory, [A Quick Pointer Primer](#)
内存存储, 快速指南
- sync.Map, [When to Use Mutexes Instead of Channels](#)
sync.Map, 何时使用互斥锁而不是通道
- sync.Once type for running code once, [Run Code Exactly Once](#)
sync.Once 类型用于运行代码一次, 即“仅运行一次代码”。
- time package
时间包裹
 - time.Duration, [time](#)
时间.持续时间, 时间
 - time.Time, [time](#)
时间。时间, 时间
- type assertions, [Type Assertions and Type Switches-Type Assertions and Type Switches](#)
 - uses for, [Use Type Assertions and Type Switches Sparingly.- Use Type Assertions and Type Switches Sparingly.](#)

类型断言、类型断言和类型开关 - 类型断言和类型开关的用

途、谨慎使用类型断言和类型开关 - 谨慎使用类型断言和类型开关

- type constraint via generic functions, [Generics and Interfaces](#)
通过泛型函数、泛型和接口实现类型约束
- type elements, [Use Type Terms to Specify Operators](#)-[Use Type Terms to Specify Operators](#)
 - constants that can be assigned to variables, [Type Elements Limit Constants](#)
 - type terms, [Use Type Terms to Specify Operators](#)-[Use Type Terms to Specify Operators](#)
 - ~ for types with underlying type, [Use Type Terms to Specify Operators](#)

类型元素，使用类型术语指定运算符 - 使用类型术语指定运算符 - 可赋值给变量的常量，类型元素限制常量 - 类型术语，使用类型术语指定运算符 - 使用类型术语指定运算符 - 对于具有底层类型的类型，使用类型术语指定运算符

- type switches, [Type Assertions and Type Switches](#)
 - uses for, [Use Type Assertions and Type Switches Sparingly](#)-[Use Type Assertions and Type Switches Sparingly](#)

类型开关、类型断言和类型开关的用途、谨慎使用类型断言和类型开关 - 谨慎使用类型断言和类型开关

- uint as unsigned int, [The special integer types](#)
uint 等同于 unsigned int, 特殊的整数类型
- uint8 as byte, [The special integer types](#), [A Taste of Strings and Runes](#)
uint8 作为字节, 特殊的整数类型, 字符串和符文入门
- uintptr integer type, [The special integer types](#)
uintptr 整数类型, 特殊的整数类型
- unsafe.Pointer type, [unsafe Is Unsafe](#)
unsafe.指针类型, unsafe 是不安全的
- user-defined types, [Types in Go](#)
 - based on another user-defined type, [Type Declarations Aren't Inheritance](#)
 - executable documentation, [Types Are Executable Documentation](#)
 - methods, [Methods-Types Are Executable Documentation](#)
 - underlying type of int or string, [Function Types Are a Bridge to Interfaces](#)

用户自定义类型、Go 语言中基于其他用户自定义类型的类型、类型声明并非继承、可执行文档、类型是可执行文档、方法、方法类型是可执行文档、底层类型为 int 或 string、函数类型是通往接口的桥梁

- value type of value receivers, [Pointer Receivers and Value Receivers-Pointer Receivers and Value Receivers](#)

值接收器的值类型，指针接收器和值接收器 - 指针接收器和值接收器

- zero value of unassigned variables, [The Zero Value](#)
未赋值变量的零值，零值

U

U

- uint as unsigned int, [The special integer types](#)
uint 等同于 unsigned int，特殊的整数类型
- uint8 as byte, [The special integer types](#), [A Taste of Strings and Runes](#)
uint8 作为字节，特殊的整数类型，字符串和符文入门
- uintptr integer type, [The special integer types](#)
uintptr 整数类型，特殊的整数类型
- underscore ()

下划线 ()

- blank imports, [Avoiding the init Function if Possible](#)
空白导入，尽可能避免使用初始化函数
- first iota value when ignored, [iota Is for Enumerations—Sometimes](#)
第一个 iota 值被忽略时，iota 用于枚举——有时
- floating-point literals, [Literals](#)
浮点字面量，字面量

- hidden file name prefix, [Embedding Hidden Files](#)
隐藏文件名前缀，嵌入隐藏文件
- integer literals, [Literals](#)
整数字面量，字面量
- return value ignored, [The for-range Statement](#), [Ignoring Returned Values](#)
返回值被忽略，for-range 语句，忽略返回值
- return value nameless among named, [Named Return Values](#)
返回值无名，但有名返回值
- targeted code build tags, [Using Build Tags](#)
使用构建标签进行定向代码构建标签
- unused variable and constant names, [The for-range Statement](#)
未使用的变量和常量名称，for-range 语句
- variable and constant names not using, [Naming Variables and Constants](#)
未使用的变量和常量名称，变量和常量的命名
- Unicode support
Unicode 支持
 - naming variables and constants, [Naming Variables and Constants](#)
变量和常量的命名，变量和常量的命名
 - rune literals, [Literals](#)
符文字面值，字面值
 - strings, [A Taste of Strings and Runes](#)
弦乐，弦乐与符文的品味

- UTF-8 encoding, [Strings and Runes and Bytes](#)
 - UTF-16 for one or two 2-byte sequences, [Strings and Runes and Bytes](#)
 - UTF-32 for four bytes per code point, [Strings and Runes and Bytes](#)

UTF-8 编码，字符串、符文和字节；UTF-16 用于一个或两个 2 字节序列；字符串、符文和字节；UTF-32 用于每个代码点四个字节；字符串、符文和字节

- unicode/utf8 package in standard library, [Strings and Runes and Bytes](#)
标准库中的 unicode/utf8 包，字符串、符文和字节
- universe block, [Shadowing Variables](#)
 - nil defined, [A Quick Pointer Primer](#)
 - predeclared identifiers defined, [Shadowing Variables](#)

Universe 块，已定义变量阴影，快速指针入门，已定义预声明标识符，已定义变量阴影

- Unmarshal function

反序列化函数

- about unmarshaling, [encoding/json](#)
关于反序列化、编码/JSON
- dash for name of ignored field, [Using Struct Tags to Add Metadata](#)

使用 Struct 标签添加元数据，以破折号表示忽略字段的名称。

- populating variable from JSON, [Pointers Are a Last Resort](#)
 - pointer input parameter, [Pointers Are a Last Resort](#)

从 JSON 填充变量，“指针是最后的手段”指针输入参数，“指针是最后的手段”

- unsafe package, [A Quick Pointer Primer](#), [unsafe Is Unsafe](#)
 - converting external binary data, [Using unsafe to Convert External Binary Data-Using unsafe to Convert External Binary Data](#)
 - documentation URL, [Using unsafe Tools](#)
 - -gcflags=-d=checkptr, [Using unsafe Tools](#)
 - unexported field access, [Accessing Unexported Fields](#)
 - unsafe.Offsetof, [Using Sizeof and Offsetof-Using Sizeof and Offsetof](#)
 - unsafe.Pointer type, [unsafe Is Unsafe](#)
 - unsafe.Sizeof, [Using Sizeof and Offsetof-Using Sizeof and Offsetof](#)

unsafe 包，指针快速入门，unsafe 不安全，转换外部二进制数据，使用 unsafe 转换外部二进制数据 - 使用 unsafe 转换外部二进制数据的文档 URL，使用 unsafe 工具 - gcflags=-d=checkptr，使用 unsafe 工具访问未导出字段，访问未导出字段 unsafe.Offsetof，使用 Sizeof 和 Offsetof - 使用 Sizeof 和 Offsetof unsafe.Pointer 类型，

unsafe 不安全 unsafe.Sizeof, 使用 Sizeof 和 Offsetof - 使用 Sizeof 和 Offsetof

- updating Go managed by go directives, [Use the go Directive to Manage Go Build Versions](#)

使用 go 指令更新 Go 管理的 Go 版本, 使用 go 指令管理 Go 构建版本

- updating modules
更新模块

- updating to compatible versions, [Updating to Compatible Versions](#)

更新至兼容版本

- updating to incompatible versions, [Updating to Incompatible Versions](#)

更新到不兼容的版本, 更新到不兼容的版本

- workspaces for, [Using Workspaces to Modify Modules Simultaneously](#)-[Using Workspaces to Modify Modules Simultaneously](#)

使用工作区同时修改模块

- updating third-party tools, [Adding Third-Party Tools with go install](#)
更新第三方工具, 使用 go install 添加第三方工具

- user-defined types, [Types in Go](#)

- based on another user-defined type, [Type Declarations Aren't Inheritance](#)

- not inheritance, [Type Declarations Aren't Inheritance](#)

- executable documentation, [Types Are Executable Documentation](#)
- methods, [Methods](#)
 - declaring, [Methods](#)
 - invoking, [Methods](#)
- underlying type of int or string, [Function Types Are a Bridge to Interfaces](#)

用户自定义类型、Go 语言中基于其他用户自定义类型的类型、类型声明并非继承、类型声明并非继承、可执行文档、类型是可执行文档、方法、方法声明、方法调用、方法底层类型为 int 或 string、函数类型是通往接口的桥梁

- UTF-8 for string literals, [Strings and Runes and Bytes](#)
 - encoding for Unicode, [Strings and Runes and Bytes](#)
 - UTF-16 for one or two 2-byte sequences, [Strings and Runes and Bytes](#)
 - UTF-32 for four bytes per code point, [Strings and Runes and Bytes](#)
 - invented by Thompson and Pike, [Strings and Runes and Bytes](#)
 - string bytes versus UTF-8 multi-byte code points, [Strings and Runes and Bytes](#)

UTF-8 用于字符串字面量，字符串、符文和字节编码用于 Unicode，字符串、符文和字节 UTF-16 用于一个或两个 2 字节序列，字符串、符文和字节 UTF-32 每个代码点四个字节，字符

串、符文和字节 由 Thompson 和 Pike 发明，字符串、符文和字节
字符串字节与 UTF-8 多字节代码点，字符串、符文和字节

V

V

- value receivers, [Pointer Receivers and Value Receivers-Pointer Receivers and Value Receivers](#)
 - methods in method set, [Pointer Receivers and Value Receivers](#)
 - nil instance handling, [Code Your Methods for nil Instances](#)
 - time package methods, [time](#)

值接收器、指针接收器和值接收器 - 方法集中的指针接收器和值接收器方法、指针接收器和值接收器、nil 实例处理、为 nil 实例编写方法、time 包方法、时间

- value type of value receivers, [Pointer Receivers and Value Receivers-Pointer Receivers and Value Receivers](#)

值接收器的值类型，指针接收器和值接收器 - 指针接收器和值接收器
- van Heumen, Danny, [iota Is for Enumerations—Sometimes](#)

范·休门，丹尼，iota 代表枚举——有时
- var versus :=, [var Versus :=-var Versus :=](#)
 - shadowing variables, [Shadowing Variables-Shadowing Variables](#)

var versus :=, var Versus :=-var Versus :=shadowing variables,
Shadowing Variables-Shadowing Variables

- variables

变量

- atomic variables, [Atomics—You Probably Don't Need These](#)
原子变量, 原子操作——你可能不需要这些
- bool type, [Booleans](#)
布尔类型, 布尔值
- call by value language, [Go Is Call by Value-Go Is Call by Value](#),
[Pointers Indicate Mutable Parameters](#)
 - pointer passed to a function, [Pointers Indicate Mutable Parameters-Pointers Indicate Mutable Parameters](#)

按值调用语言, Go 语言是按值调用——Go 语言是按值调用, 指针指示可变参数, 传递给函数的指针, 指针指示可变参数——指针指示可变参数

- call-by-value language, [append](#)
按值调用语言, 追加
- declaration

宣言

- about, [Blocks](#)
关于, 积木

- arrays, [Arrays—Too Rigid to Use Directly](#)
数组，数组——过于僵化而无法直接使用
- blocks, [Blocks](#)
积木，积木
- declaration list, [var Versus :=](#)
声明列表，var Versus :=
- if and else block scope, [if](#)
如果和否则阻塞作用域，如果
- package-level variables avoided, [var Versus :=](#), [Unused Variables](#)
避免使用包级变量，var 与 := 的区别，未使用的变量
- pointers, [A Quick Pointer Primer](#)
指针，指针快速入门
- sentinel errors in package block, [Sentinel Errors](#)
包块中的哨兵错误，哨兵错误
- shadowing variables, [Shadowing Variables-Shadowing Variables](#)
阴影变量，阴影变量-阴影变量
- shadowing variables good with type switch, [Type Assertions and Type Switches](#)
类型切换、类型断言和类型开关的影子变量
- var versus :=, [var Versus :=-var Versus :=](#)
var 与 :=, var 与 := -var 与 :=

- empty interface to indicate value of any type, [The Empty Interface Says Nothing](#)
 - any as type alias for, [The Empty Interface Says Nothing](#)

空接口表示任何类型的值，“空接口什么也不说”是“空接口什么也不说”的类型别名。

- environment variables (see environment variables)
环境变量（参见环境变量）
- err variable for errors, [How to Handle Errors: The Basics](#)
用于存储错误的 err 变量，如何处理错误：基础知识
- explicit type conversion, [Explicit Type Conversion](#)
 - automatic type promotion versus, [Explicit Type Conversion](#)

显式类型转换，显式类型转换与自动类型提升的比较，显式类型转换

- method assigned to, [Methods Are Functions Too](#)
方法已分配给，方法也是函数
- naming, [Naming Variables and Constants](#)
 - camelCase instead of snake_case, [Naming Variables and Constants](#)
 - smaller the scope, shorter the name, [Naming Variables and Constants](#)

命名，变量和常量的命名：驼峰式命名法（camelcase）而非

蛇形命名法 (snake_case) ; 变量和常量的命名: 作用域越小, 名称越短; 变量和常量的命名

- pointers, [A Quick Pointer Primer](#)
 - (see also pointers)

指针, 指针快速入门 (另见指针)

- slices from slices sharing memory, [Slicing Slices](#)
 - (see also slices)

切片之间共享内存, 切片 (另见切片)

- storage in memory, [A Quick Pointer Primer](#)
- 内存存储, 快速指南
- strings immutable, [A Taste of Strings and Runes](#)
 - (see also string type)

字符串不可变性, 字符串和符文入门 (另见字符串类型)

- unassigned variable zero value, [The Zero Value](#)
 - (see also zero value of unassigned variables)

未赋值变量的零值, 零值 (另见未赋值变量的零值)

- unused, [Unused Variables](#)
 - underscore () for name, [The for-range Statement](#)

未使用的变量名称下划线 (), for 范围语句

- variadic input parameter (. . .), [Variadic Input Parameters and Slices](#)
 - slice as input, [Variadic Input Parameters and Slices](#)

可变输入参数 (...), 可变输入参数和切片作为输入, 可变输入参数和切片

- variadic type parameters not part of Go, [Things That Are Left Out](#)
Go 语言中未包含的可变参数类型参数
- vendoring, [Vendoring](#)
供应商, 供应商
- version control software, [Publishing Your Module](#)
 - version control system documentation URL, [Publishing Your Module](#)
 - versioning your module, [Versioning Your Module](#)

版本控制软件、发布您的模块版本控制系统文档 URL、发布您的模块、版本控制您的模块、模块版本控制

- virtual filesystem in embedded content, [Embedding Content into Your Program](#)
 - example code for treating like real, [Embedding Content into Your Program](#)
 - hidden files, [Embedding Hidden Files](#)

嵌入式内容中的虚拟文件系统, 将内容嵌入到程序中, 示例代码模拟真实文件, 将内容嵌入到程序中, 隐藏文件, 嵌入隐藏文件

- Visual Studio Code, [Visual Studio Code](#)
 - Go extension, [Visual Studio Code](#)

Visual Studio Code、 Visual Studio CodeGo 扩展、 Visual Studio Code

- Vlissides, John, [Use Embedding for Composition](#), [Interfaces Are Type-Safe Duck Typing](#)

Vlissides, John, 《使用嵌入进行组合，接口是类型安全的鸭子类型》

- vulnerabilities scanned for by govulncheck tool, [Using govulncheck to Scan for Vulnerable Dependencies](#)-[Using govulncheck to Scan for Vulnerable Dependencies](#)

govulncheck 工具扫描的漏洞，使用 govulncheck 扫描易受攻击的依赖项 - 使用 govulncheck 扫描易受攻击的依赖项

W

W

- WaitGroups in concurrency, [Use WaitGroups](#)-[Use WaitGroups](#)
 - errgroup.Group for WaitGroups errors, [Use WaitGroups](#)

并发中的 WaitGroups，使用 WaitGroups - 使用

WaitGrouperrgroup.Group 处理 WaitGroups 错误，使用 WaitGroups

- whence constants for io.Seeker, [io and Friends](#)
io.Seeker、io 和 Friends 的常量来源
- while via condition-only for loop, [The Condition-Only for Statement](#)
while 通过仅条件 for 循环，仅条件 for 语句
- whitespace formatting fixed via go fmt, [go fmt](#)
通过 go fmt 修复了空格格式问题，go fmt
- wiki Code Review Comments page, [go vet](#)
wiki 代码审查评论页面，去兽医
- Windows
视窗
 - Chocolatey
巧克力味
 - Go installation, [Installing the Go Tools](#)
Go 安装，安装 Go 工具
 - updating Go tools, [Staying Up-to-Date](#)
更新 Go 工具，保持最新状态
 - make installation, [Makefiles](#)
进行安装，Makefiles
- Wire helper (Google), [Wire](#)
Wire helper (Google) , Wire
- workspaces for simultaneous module modifications, [Using Workspaces to Modify Modules Simultaneously](#)-[Using Workspaces to Modify Modules Simultaneously](#)

用于同时修改模块的工作区，使用工作区同时修改模块-使用工作区同时修改模块

- wrapping errors, [Use Type Assertions and Type Switches Sparingly](#),
[Wrapping Errors](#)-[Wrapping Errors](#)
 - sentinel errors not checkable by ==, [Is and As](#)
 - wrapping multiple errors, [Wrapping Multiple Errors](#)

错误包装，谨慎使用类型断言和类型切换，错误包装——包装错误，哨兵错误无法通过 == 检查，is 和 as 包装多个错误，包装多个错误

Z

Z

- zero value of unassigned variables, [The Zero Value](#)
 - bool types as false, [Booleans](#)
 - channels as nil, [Channels](#)
 - complex numbers as 0, [Complex types \(you're probably not going to use these\)](#)
 - floating-point types as 0, [Floating-point types](#)
 - function variables as nil, [Functions Are Values](#)
 - integer types as 0, [Integer types](#)
 - interface types as nil, [Interfaces and nil](#)
 - maps as nil, [Maps](#)

- pointers as nil, [A Quick Pointer Primer](#)
- slices as nil, [Slices](#)
- strings as empty string, [A Taste of Strings and Runes](#)
- struct fields as field's zero value, [Structs](#)
- using var for initializing, [var Versus :=](#)

未赋值变量的零值，布尔类型为 false，布尔值通道为 nil，通道复数为 0，复数类型（您可能不会用到这些）浮点类型为 0，浮点类型函数变量为 nil，函数即值整型为 0，整型接口类型为 nil，接口和 nil 映射为 nil，映射指针为 nil，指针快速入门切片为 nil，切片字符串为空字符串，字符串和符文入门结构体字段为字段的零值，结构体使用 var 进行初始化，var 与 := 的区别

About the Author

作者简介

Jon Bodner has been a software engineer, lead developer, and architect for over 25 years. In that time, he has worked on software across many fields, including education, finance, commerce, healthcare, law, government, and internet infrastructure.

Jon Bodner 拥有超过 25 年的软件工程师、首席开发人员和架构师经验。在此期间，他曾参与过许多领域的软件开发工作，包括教育、金融、商业、医疗保健、法律、政府和互联网基础设施。

Jon is currently a staff engineer at Datadog, working on improving the onboarding experience for customers.

Jon 目前是 Datadog 的一名工程师，致力于改善客户的注册体验。

Previously, Jon was a senior distinguished engineer at Capital One, where he worked on commercialization of technology, contributed to the company's development and testing workflow, developed patented techniques for web payment page detection and population, and coauthored tools for finding and managing software development issues.

此前，Jon 是 Capital One 的一名高级杰出工程师，他从事技术商业化工作，为公司的开发和测试工作流程做出了贡献，开发了用于网络支付页面检测和填充的专利技术，并参与编写了用于查找和管理软件开发问题的工具。

Jon is a frequent speaker at Go conferences, and his blog posts on Go and software engineering have been viewed more than 300,000 times. He is the creator of the [Proteus data access library](#) and codeveloper of [checks-out](#), a fork of the LGTM project.

Jon 经常在 Go 语言大会上发表演讲，他关于 Go 语言和软件工程的博客文章浏览量超过 30 万次。他是 Proteus 数据访问库的创建者，也是 checks-out (LGTM 项目的一个分支) 的共同开发者。

Colophon

版权页

The animal on the cover of *Learning Go* is a plains pocket gopher (*Geomys bursarius*), a burrowing mammal found across the Great Plains of North America. These rodents are highly adapted to digging and live primarily underground.

《学习Go》封面上的动物是平原囊鼠（*Geomys bursarius*），一种穴居哺乳动物，分布于北美大平原。这些啮齿动物高度适应挖掘，主要生活在地下。

Plains pocket gophers are covered in brown fur and have a nearly naked tail. Physical adaptations to digging include small eyes, short ears, and large, clawed front feet. They also have a tolerance for low oxygen and high carbon dioxide levels. The gophers are named for their external cheek pouches, which can be used for carrying food.

平原囊鼠全身覆盖着棕色皮毛，尾巴几乎无毛。为了适应挖掘环境，它们进化出了小眼睛、短耳朵和大而带爪的前脚。它们还能耐受低氧和高二氧化碳环境。囊鼠因其颊囊而得名，颊囊可以用来携带食物。

The gophers are territorial and aggressive, and will rarely enter another's burrow. They spend nearly three-quarters of their time in their own burrows, which contain their nests and food caches of roots and grasses. They go above ground to search for food and mates.

地鼠具有很强的领地意识和攻击性，很少会擅自闯入其他地鼠的洞穴。它们近四分之三的时间都待在自己的洞穴里，洞穴里有它们的巢穴和用根茎和草料储存的食物。它们会到地面上觅食和寻找配偶。

The plains pocket gopher's conservation status is of least concern. Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

平原囊鼠的保护级别为无危。奥莱利地图上描绘的许多动物都濒临灭绝；它们对世界都至关重要。

The cover illustration is by Susan Thompson, based on an antique black-and-white engraving from an unknown source. The series design is by Edie Freedman, Ellie Volckhausen, and Karen Montgomery. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.

封面插图由苏珊·汤普森绘制，取材于一幅来源不明的黑白古董版画。系列设计由艾迪·弗里德曼、艾莉·沃尔克豪森和凯伦·蒙哥马利共同完成。封面字体为Gilroy Semibold和Guardian Sans。正文字体为Adobe

Minion Pro; 标题字体为Adobe Myriad Condensed; 代码字体为Dalton Maag设计的Ubuntu Mono。